Finite Difference Computing with PDEs - A Modern Software Approach

Hans Petter Langtangen[1,2]

Svein Linge[3,1]

[1]Center for Biomedical Computing, Simula Research Laboratory

[2]Department of Informatics, University of Oslo

[3]Department of Process, Energy and Environmental Technology, University College of Southeast Norway

This easy-to-read book introduces the basics of solving partial differential equations by finite difference methods. The emphasis is on constructing finite difference schemes, formulating algorithms, implementing algorithms, verifying implementations, analyzing the physical behavior of the numerical solutions, and applying the methods and software to solve problems from physics and biology.

Aug 15, 2017

Preface

There are so many excellent books on finite difference methods for ordinary and partial differential equations that writing yet another one requires a different view on the topic. The present book is not so concerned with the traditional academic presentation of the topic, but is focused at teaching the practitioner how to obtain reliable computations involving finite difference methods. This focus is based on a set of learning outcomes:

1. understanding of the ideas behind finite difference methods,
2. understanding how to transform an algorithm to a well-designed computer code,
3. understanding how to test (verify) the code,
4. understanding potential artifacts in simulation results.

Compared to other textbooks, the present one has a particularly strong emphasis on computer implementation and verification. It also has a strong emphasis on an intuitive understanding of constructing finite difference methods. To learn about the potential non-physical artifacts of various methods, we study exact solutions of finite difference schemes as these give deeper insight into the physical behavior of the numerical methods than the traditional (and more general) asymptotic error analysis. However, asymptotic results regarding convergence rates, typically truncation errors, are crucial for testing implementations, so an extensive appendix is devoted to the computation of truncation errors.

Why finite differences? One may ask why we do finite differences when finite element and finite volume methods have been developed to greater

© 2017, Hans Petter Langtangen, Svein Linge. Released under CC Attribution 4.0 license

generality and sophistication than finite differences and can cover more problems. The finite element and finite volume methods are also the industry standard nowadays. Why not just those methods? The reason for finite differences is the method's simplicity, both from a mathematical and coding perspective. Especially in academia, where simple model problems are used a lot for teaching and in research (e.g., for verification of advanced implementations), there is a constant need to solve the model problems from scratch with easy-to-verify computer codes. Here, finite differences are ideal. A simple 1D heat equation can of course be solved by a finite element package, but a 20-line code with a difference scheme is just right to the point and provides an understanding of all details involved in the model and the solution method. Everybody nowadays has a laptop and the natural method to attack a 1D heat equation is a simple Python or Matlab program with a difference scheme. The conclusion goes for other fundamental PDEs like the wave equation and Poisson equation as long as the geometry of the domain is a hypercube. The present book contains all the practical information needed to use the finite difference tool in a safe way.

Various pedagogical elements are utilized to reach the learning outcomes, and these are commented upon next.

Simplify, understand, generalize. The book's overall pedagogical philosophy is the three-step process of first *simplifying* the problem to something we can *understand* in detail, and when that understanding is in place, we can *generalize* and hopefully address real-world applications with a sound scientific problem-solving approach. For example, in the chapter on a particular family of equations we first simplify the problem in question to a 1D, constant-coefficient equation with simple boundary conditions. We learn how to construct a finite difference method, how to implement it, and how to understand the behavior of the numerical solution. Then we can generalize to higher dimensions, variable coefficients, a source term, and more complicated boundary conditions. The solution of a compound problem is in this way an assembly of elements that are well understood in simpler settings.

Constructive mathematics. This text favors a constructive approach to mathematics. Instead of a set of definitions followed by popping up a method, we emphasize how to think about the construction of a method. The aim is to obtain a good intuitive understanding of the mathematical methods.

The text is written in an easy-to-read style much inspired by the following quote.

Some people think that stiff challenges are the best device to induce learning, but I am not one of them. The natural way to learn something is by spending vast amounts of easy, enjoyable time at it. This goes whether you want to speak German, sight-read at the piano, type, or do mathematics. Give me the German storybook for fifth graders that I feel like reading in bed, not Goethe and a dictionary. The latter will bring rapid progress at first, then exhaustion and failure to resolve.

The main thing to be said for stiff challenges is that inevitably we will encounter them, so we had better learn to face them boldly. Putting them in the curriculum can help teach us to do so. But for teaching the skill or subject matter itself, they are overrated. [18, p. 86] Lloyd N. Trefethen, Applied Mathematician, 1955-.

This book assumes some basic knowledge of finite difference approximations, differential equations, and scientific Python or MATLAB programming, as often met in an introductory numerical methods course. Readers without this background may start with the light companion book "Finite Difference Computing with Exponential Decay Models" [9]. That book will in particular be a useful resource for the programming parts of the present book. Since the present book deals with partial differential equations, the reader is assumed to master multi-variable calculus and linear algebra.

Fundamental ideas and their associated scientific details are first introduced in the simplest possible differential equation setting, often an ordinary differential equation, but in a way that easily allows reuse in more complex settings with partial differential equations. With this approach, new concepts are introduced with a minimum of mathematical details. The text should therefore have a potential for use early in undergraduate student programs.

All nuts and bolts. Many have experienced that "vast amounts of easy, enjoyable time", as stated in the quote above, arises when mathematics is implemented on a computer. The implementation process triggers understanding, creativity, and curiosity, but many students find the transition from a mathematical algorithm to a working code difficult and spend a lot of time on "programming issues".

Most books on numerical methods concentrate on the mathematics of the subject while details on going from the mathematics to a computer implementation are less in focus. A major purpose of this text is therefore to help the practitioner by providing *all nuts and bolts* necessary for safely going from the mathematics to a well-designed and well-tested computer code. A significant portion of the text is consequently devoted to programming details.

Python as programming language. While MATLAB enjoys widespread popularity in books on numerical methods, we have chosen to use the

Python programming language. Python is very similar to MATLAB, but contains a lot of modern software engineering tools that have become standard in the software industry and that should be adopted also for numerical computing projects. Python is at present also experiencing an exponential growth in popularity within the scientific computing community. One of the book's goals is to present an up-to-date Python eco system for implementing finite difference methods.

Program verification. Program testing, called *verification*, is a key topic of the book. Good verification techniques are indispensable when debugging computer code, but also fundamental for achieving reliable simulations. Two verification techniques saturate the book: exact solution of discrete equations (where the approximation error vanishes) and empirical estimation of convergence rates in problems with exact (analytical or manufactured) solutions of the differential equation(s).

Vectorized code. Finite difference methods lead to code with loops over large arrays. Such code in plain Python is known to run slowly. We demonstrate, especially in Appendix C, how to port loops to fast, compiled code in C or Fortran. However, an alternative is to vectorize the code to get rid of explicit Python loops, and this technique is met throughout the book. Vectorization becomes closely connected to the underlying array library, here `numpy`, and is often thought of as a difficult subject by students. Through numerous examples in different contexts, we hope that the present book provides a substantial contribution to explaining how algorithms can be vectorized. Not only will this speed up serial code, but with a library that can produce parallel code from `numpy` commands (such as Numba[1]), vectorized code can be automatically turned into parallel code and utilize multi-core processors and GPUs. Also when creating tailored parallel code for today's supercomputers, vectorization is useful as it emphasizes splitting up an algorithm into plain and simple array operations, where each operation is trivial to parallelize efficiently, rather than trying to develop a "smart" overall parallelization strategy.

Analysis via exact solutions of discrete equations. Traditional asymptotic analysis of errors is important for verification of code using convergence rates, but gives a limited understanding of how and why a correctly implemented numerical method may give non-physical results. By developing exact solutions, usually based on Fourier methods, of

[1] `http://numba.pydata.org`

the discrete equations, one can obtain a physical understanding of the behavior of a numerical method. This approach is favored for analysis of methods in this book.

Code-inspired mathematical notation. Our primary aim is to have a clean and easy-to-read computer code, and we want a close one-to-one relationship between the computer code and mathematical description of the algorithm. This principle calls for a mathematical notation that is governed by the natural notation in the computer code. The unknown is mostly called u, but the meaning of the symbol u in the mathematical description changes as we go from the exact solution fulfilling the differential equation to the symbol `u` that is naturally used for the associated data structure in the code.

Limited scope. The aim of this book is not to give an overview of a lot of methods for a wide range of mathematical models. Such information can be found in numerous existing, more advanced books. The aim is rather to introduce basic concepts and a thorough understanding of how to think about computing with finite difference methods. We therefore go in depth with only the most fundamental methods and equations. However, we have a multi-disciplinary scope and address the interplay of mathematics, numerics, computer science, and physics.

Focus on wave phenomena. Most books on finite difference methods, or books on theory with computer examples, have their emphasis on diffusion phenomena. Half of this book (Chapters 1, 2, and Appendix C) is devoted to wave phenomena. Extended material on this topic is not so easy find in the literature, so the book should be a valuable contribution in this respect. Wave phenomena is also a good topic in general for choosing the finite difference method over other discretization methods since one quickly needs fine resolution over the entire mesh and uniform meshes are most natural.

Instead of introducing the finite difference method for diffusion problems, where one soon ends up with matrix systems, we do the introduction in a wave phenomena setting where explicit schemes are most relevant. This slows down the learning curve since we can introduce a lot of theory for differences and for software aspects in a context with simple, explicit stencils for updating the solution.

Independent chapters. Most book authors are careful with avoiding repetitions of material. The chapters in this book, however, contain some overlap, because we want the chapters to appear meaningful on their own. Modern publishing technology makes it easy to take selected chapters

from different books to make a new book tailored to a specific course. The more a chapter builds on details in other chapters, the more difficult it is to reuse chapters in new contexts. Also, most readers find it convenient that important information is explicitly stated, even if it was already met in another chapter.

Supplementary materials. All program and data files referred to in this book are available from the book's primary web site: URL: `https://github.com/hplgit/fdm-book/`.

Acknowledgments. Professor Kent-Andre Mardal at the University of Oslo has kindly contributed to enlightening discussions on several issues. Many students have provided lots of useful feedback on the exposition and found many errors in the text. Special efforts in this regard were made by Imran Ali, Shirin Fallahi, Anders Hafreager, Daniel Alexander Mo Søreide Houshmand, Kristian Gregorius Hustad, Mathilde Nygaard Kamperud, and Fatemeh Miri. The collaboration with the Springer team, with Dr. Martin Peters, Thanh-Ha Le Thi, and their production staff has always been a great pleasure and a very efficient process.

Finally, want really appreciate the strong push from the COE of Simula Research Laboratory, Aslak Tveito, for publishing and financing books in open access format, including this one. We are grateful for the laboratory's financial contribution as well as to the financial contribution from the Department of Process, Energy and Environmental Technology at the University College of Southeast Norway.

Oslo, July 2016 *Hans Petter Langtangen, Svein Linge*

Contents

Preface v

1 Vibration ODEs 1

1.1 Finite difference discretization 1
1.1.1 A basic model for vibrations 2
1.1.2 A centered finite difference scheme 2

1.2 Implementation 5
1.2.1 Making a solver function 5
1.2.2 Verification 7
1.2.3 Scaled model 11

1.3 Visualization of long time simulations 12
1.3.1 Using a moving plot window 13
1.3.2 Making animations 14
1.3.3 Using Bokeh to compare graphs 17
1.3.4 Using a line-by-line ascii plotter 20
1.3.5 Empirical analysis of the solution 21

1.4 Analysis of the numerical scheme 23
1.4.1 Deriving a solution of the numerical scheme 23
1.4.2 The error in the numerical frequency 25
1.4.3 Empirical convergence rates and adjusted ω 26
1.4.4 Exact discrete solution 27
1.4.5 Convergence 27

© 2017, Hans Petter Langtangen, Svein Linge. Released under CC Attribution 4.0 license

1.4.6 The global error . . . 28
1.4.7 Stability . . . 29
1.4.8 About the accuracy at the stability limit . . . 30

1.5 Alternative schemes based on 1st-order equations . . . 32
1.5.1 The Forward Euler scheme . . . 33
1.5.2 The Backward Euler scheme . . . 34
1.5.3 The Crank-Nicolson scheme . . . 34
1.5.4 Comparison of schemes . . . 36
1.5.5 Runge-Kutta methods . . . 37
1.5.6 Analysis of the Forward Euler scheme . . . 39

1.6 Energy considerations . . . 41
1.6.1 Derivation of the energy expression . . . 42
1.6.2 An error measure based on energy . . . 44

1.7 The Euler-Cromer method . . . 46
1.7.1 Forward-backward discretization . . . 46
1.7.2 Equivalence with the scheme for the second-order ODE 48
1.7.3 Implementation . . . 49
1.7.4 The Störmer-Verlet algorithm . . . 51

1.8 Staggered mesh . . . 53
1.8.1 The Euler-Cromer scheme on a staggered mesh . . . 53
1.8.2 Implementation of the scheme on a staggered mesh . . 55

1.9 Exercises and Problems . . . 57

1.10 Generalization: damping, nonlinearities, and excitation . . . 67
1.10.1 A centered scheme for linear damping . . . 67
1.10.2 A centered scheme for quadratic damping . . . 68
1.10.3 A forward-backward discretization of the quadratic damping term . . . 69
1.10.4 Implementation . . . 70
1.10.5 Verification . . . 71
1.10.6 Visualization . . . 72
1.10.7 User interface . . . 73
1.10.8 The Euler-Cromer scheme for the generalized model . . 75
1.10.9 The Störmer-Verlet algorithm for the generalized model 76
1.10.10 A staggered Euler-Cromer scheme for a generalized model . . . 76
1.10.11 The PEFRL 4th-order accurate algorithm . . . 77

1.11 Exercises and Problems . . . 78

1.12 Applications of vibration models 80
1.12.1 Oscillating mass attached to a spring 80
1.12.2 General mechanical vibrating system 82
1.12.3 A sliding mass attached to a spring 84
1.12.4 A jumping washing machine 85
1.12.5 Motion of a pendulum 85
1.12.6 Dynamic free body diagram during pendulum motion 88
1.12.7 Motion of an elastic pendulum 93
1.12.8 Vehicle on a bumpy road 99
1.12.9 Bouncing ball 101
1.12.10 Two-body gravitational problem 102
1.12.11 Electric circuits 104

1.13 Exercises 104

2 Wave equations 111

2.1 Simulation of waves on a string 111
2.1.1 Discretizing the domain 112
2.1.2 The discrete solution 113
2.1.3 Fulfilling the equation at the mesh points 113
2.1.4 Replacing derivatives by finite differences 113
2.1.5 Formulating a recursive algorithm 115
2.1.6 Sketch of an implementation 117

2.2 Verification 118
2.2.1 A slightly generalized model problem 118
2.2.2 Using an analytical solution of physical significance . . 119
2.2.3 Manufactured solution and estimation of convergence rates 120
2.2.4 Constructing an exact solution of the discrete equations 122

2.3 Implementation 124
2.3.1 Callback function for user-specific actions 125
2.3.2 The solver function 126
2.3.3 Verification: exact quadratic solution 127
2.3.4 Verification: convergence rates 128
2.3.5 Visualization: animating the solution 130
2.3.6 Running a case 134
2.3.7 Working with a scaled PDE model 135

2.4 Vectorization 137
2.4.1 Operations on slices of arrays 137

2.4.2 Finite difference schemes expressed as slices 140
2.4.3 Verification 141
2.4.4 Efficiency measurements 142
2.4.5 Remark on the updating of arrays 144

2.5 Exercises 145

2.6 Generalization: reflecting boundaries 149
2.6.1 Neumann boundary condition 150
2.6.2 Discretization of derivatives at the boundary 150
2.6.3 Implementation of Neumann conditions 152
2.6.4 Index set notation 153
2.6.5 Verifying the implementation of Neumann conditions . 155
2.6.6 Alternative implementation via ghost cells 157

2.7 Generalization: variable wave velocity 160
2.7.1 The model PDE with a variable coefficient 161
2.7.2 Discretizing the variable coefficient 161
2.7.3 Computing the coefficient between mesh points 163
2.7.4 How a variable coefficient affects the stability 164
2.7.5 Neumann condition and a variable coefficient 164
2.7.6 Implementation of variable coefficients 165
2.7.7 A more general PDE model with variable coefficients . 166
2.7.8 Generalization: damping 167

2.8 Building a general 1D wave equation solver 168
2.8.1 User action function as a class 168
2.8.2 Pulse propagation in two media 171

2.9 Exercises 175

2.10 Analysis of the difference equations 185
2.10.1 Properties of the solution of the wave equation 185
2.10.2 More precise definition of Fourier representations 187
2.10.3 Stability 189
2.10.4 Numerical dispersion relation 191
2.10.5 Extending the analysis to 2D and 3D 194

2.11 Finite difference methods for 2D and 3D wave equations . . . 198
2.11.1 Multi-dimensional wave equations 198
2.11.2 Mesh 200
2.11.3 Discretization 201

2.12 Implementation 203

2.12.1 Scalar computations . . . 205
2.12.2 Vectorized computations . . . 207
2.12.3 Verification . . . 209
2.12.4 Visualization . . . 210

2.13 Exercises . . . 214

2.14 Applications of wave equations . . . 216
2.14.1 Waves on a string . . . 217
2.14.2 Elastic waves in a rod . . . 220
2.14.3 Waves on a membrane . . . 221
2.14.4 The acoustic model for seismic waves . . . 221
2.14.5 Sound waves in liquids and gases . . . 223
2.14.6 Spherical waves . . . 225
2.14.7 The linear shallow water equations . . . 226
2.14.8 Waves in blood vessels . . . 229
2.14.9 Electromagnetic waves . . . 231

2.15 Exercises . . . 232

3 Diffusion equations . . . 247

3.1 An explicit method for the 1D diffusion equation . . . 248
3.1.1 The initial-boundary value problem for 1D diffusion . . . 248
3.1.2 Forward Euler scheme . . . 249
3.1.3 Implementation . . . 251
3.1.4 Verification . . . 253
3.1.5 Numerical experiments . . . 257

3.2 Implicit methods for the 1D diffusion equation . . . 263
3.2.1 Backward Euler scheme . . . 263
3.2.2 Sparse matrix implementation . . . 267
3.2.3 Crank-Nicolson scheme . . . 268
3.2.4 The unifying θ rule . . . 270
3.2.5 Experiments . . . 271
3.2.6 The Laplace and Poisson equation . . . 273

3.3 Analysis of schemes for the diffusion equation . . . 274
3.3.1 Properties of the solution . . . 275
3.3.2 Analysis of discrete equations . . . 277
3.3.3 Analysis of the finite difference schemes . . . 279
3.3.4 Analysis of the Forward Euler scheme . . . 280
3.3.5 Analysis of the Backward Euler scheme . . . 282

3.3.6 Analysis of the Crank-Nicolson scheme 283
3.3.7 Analysis of the Leapfrog scheme 283
3.3.8 Summary of accuracy of amplification factors 284
3.3.9 Analysis of the 2D diffusion equation 286
3.3.10 Explanation of numerical artifacts 288

3.4 Exercises 289

3.5 Diffusion in heterogeneous media 293
3.5.1 Discretization 293
3.5.2 Implementation 294
3.5.3 Stationary solution 295
3.5.4 Piecewise constant medium 295
3.5.5 Implementation of diffusion in a piecewise constant medium 296
3.5.6 Axi-symmetric diffusion 299
3.5.7 Spherically-symmetric diffusion 302

3.6 Diffusion in 2D 303
3.6.1 Discretization 303
3.6.2 Numbering of mesh points versus equations and unknowns 304
3.6.3 Algorithm for setting up the coefficient matrix 309
3.6.4 Implementation with a dense coefficient matrix 311
3.6.5 Verification: exact numerical solution 314
3.6.6 Verification: convergence rates 316
3.6.7 Implementation with a sparse coefficient matrix 317
3.6.8 The Jacobi iterative method 322
3.6.9 Implementation of the Jacobi method 325
3.6.10 Test problem: diffusion of a sine hill 327
3.6.11 The relaxed Jacobi method and its relation to the Forward Euler method 328
3.6.12 The Gauss-Seidel and SOR methods 329
3.6.13 Scalar implementation of the SOR method 331
3.6.14 Vectorized implementation of the SOR method 331
3.6.15 Direct versus iterative methods 335
3.6.16 The Conjugate gradient method 338
3.6.17 What is the recommended method for solving linear systems? 341

3.7 Random walk 342
3.7.1 Random walk in 1D 342

3.7.2 Statistical considerations 343
3.7.3 Playing around with some code 345
3.7.4 Equivalence with diffusion 348
3.7.5 Implementation of multiple walks 349
3.7.6 Demonstration of multiple walks 356
3.7.7 Ascii visualization of 1D random walk 360
3.7.8 Random walk as a stochastic equation 361
3.7.9 Random walk in 2D 362
3.7.10 Random walk in any number of space dimensions 363
3.7.11 Multiple random walks in any number of space dimensions 364

3.8 Applications 366
3.8.1 Diffusion of a substance 366
3.8.2 Heat conduction 368
3.8.3 Porous media flow 371
3.8.4 Potential fluid flow 372
3.8.5 Streamlines for 2D fluid flow 372
3.8.6 The potential of an electric field 373
3.8.7 Development of flow between two flat plates 373
3.8.8 Flow in a straight tube 374
3.8.9 Tribology: thin film fluid flow 375
3.8.10 Propagation of electrical signals in the brain 376

3.9 Exercises 376

4 Advection-dominated equations 385

4.1 One-dimensional time-dependent advection equations 386
4.1.1 Simplest scheme: forward in time, centered in space . . 387
4.1.2 Analysis of the scheme 390
4.1.3 Leapfrog in time, centered differences in space 391
4.1.4 Upwind differences in space 394
4.1.5 Periodic boundary conditions 397
4.1.6 Implementation 397
4.1.7 A Crank-Nicolson discretization in time and centered differences in space 401
4.1.8 The Lax-Wendroff method 403
4.1.9 Analysis of dispersion relations 405

4.2 One-dimensional stationary advection-diffusion equation 409
4.2.1 A simple model problem 409

4.2.2 A centered finite difference scheme 410
4.2.3 Remedy: upwind finite difference scheme 413

4.3 Time-dependent convection-diffusion equations 415
4.3.1 Forward in time, centered in space scheme 415
4.3.2 Forward in time, upwind in space scheme 416

4.4 Applications of advection equations 416
4.4.1 Transport of a substance 416
4.4.2 Transport of heat in fluids 417

4.5 Exercises 417

5 Nonlinear problems 419

5.1 Introduction of basic concepts 419
5.1.1 Linear versus nonlinear equations 419
5.1.2 A simple model problem 421
5.1.3 Linearization by explicit time discretization 422
5.1.4 Exact solution of nonlinear algebraic equations 423
5.1.5 Linearization 424
5.1.6 Picard iteration 425
5.1.7 Linearization by a geometric mean 427
5.1.8 Newton's method 429
5.1.9 Relaxation 430
5.1.10 Implementation and experiments 431
5.1.11 Generalization to a general nonlinear ODE 433
5.1.12 Systems of ODEs 436

5.2 Systems of nonlinear algebraic equations 439
5.2.1 Picard iteration 439
5.2.2 Newton's method 440
5.2.3 Stopping criteria 442
5.2.4 Example: A nonlinear ODE model from epidemiology 443

5.3 Linearization at the differential equation level 445
5.3.1 Explicit time integration 446
5.3.2 Backward Euler scheme and Picard iteration 446
5.3.3 Backward Euler scheme and Newton's method 447
5.3.4 Crank-Nicolson discretization 450

5.4 1D stationary nonlinear differential equations 451
5.4.1 Finite difference discretization 452
5.4.2 Solution of algebraic equations 453

5.5 Multi-dimensional nonlinear PDE problems 458
5.5.1 Finite difference discretization 458
5.5.2 Continuation methods 461

5.6 Operator splitting methods 462
5.6.1 Ordinary operator splitting for ODEs 462
5.6.2 Strang splitting for ODEs 463
5.6.3 Example: Logistic growth 464
5.6.4 Reaction-diffusion equation 467
5.6.5 Example: Reaction-Diffusion with linear reaction term 469
5.6.6 Analysis of the splitting method 477

5.7 Exercises 478

A Useful formulas 487

A.1 Finite difference operator notation 487

A.2 Truncation errors of finite difference approximations 488

A.3 Finite differences of exponential functions 489

A.4 Finite differences of t^n 490
A.4.1 Software 491

B Truncation error analysis 493

B.1 Overview of truncation error analysis 494
B.1.1 Abstract problem setting 494
B.1.2 Error measures 494

B.2 Truncation errors in finite difference formulas 496
B.2.1 Example: The backward difference for $u'(t)$ 496
B.2.2 Example: The forward difference for $u'(t)$ 497
B.2.3 Example: The central difference for $u'(t)$ 498
B.2.4 Overview of leading-order error terms in finite difference formulas 499
B.2.5 Software for computing truncation errors 500

B.3 Exponential decay ODEs 502
B.3.1 Forward Euler scheme 502
B.3.2 Crank-Nicolson scheme 503
B.3.3 The θ-rule 503
B.3.4 Using symbolic software 504
B.3.5 Empirical verification of the truncation error 505

B.3.6 Increasing the accuracy by adding correction terms .. 509
B.3.7 Extension to variable coefficients 513
B.3.8 Exact solutions of the finite difference equations 514
B.3.9 Computing truncation errors in nonlinear problems .. 514

B.4 Vibration ODEs 515
B.4.1 Linear model without damping 515
B.4.2 Model with damping and nonlinearity 519
B.4.3 Extension to quadratic damping 520
B.4.4 The general model formulated as first-order ODEs ... 521

B.5 Wave equations 522
B.5.1 Linear wave equation in 1D 522
B.5.2 Finding correction terms 524
B.5.3 Extension to variable coefficients 525
B.5.4 Linear wave equation in 2D/3D 527

B.6 Diffusion equations 528
B.6.1 Linear diffusion equation in 1D 528
B.6.2 Nonlinear diffusion equation in 1D 530

B.7 Exercises 531

C Software engineering; wave equation model 535

C.1 A 1D wave equation simulator 535
C.1.1 Mathematical model 535
C.1.2 Numerical discretization 535
C.1.3 A solver function 536

C.2 Saving large arrays in files 539
C.2.1 Using `savez` to store arrays in files 540
C.2.2 Using `joblib` to store arrays in files 541
C.2.3 Using a hash to create a file or directory name 542

C.3 Software for the 1D wave equation 544
C.3.1 Making hash strings from input data 545
C.3.2 Avoiding rerunning previously run cases 546
C.3.3 Verification 546

C.4 Programming the solver with classes 548
C.4.1 Class Parameters 548
C.4.2 Class Problem 550
C.4.3 Class Mesh 551

C.4.4 Class Function 554
C.4.5 Class Solver 557

C.5 Migrating loops to Cython 562
C.5.1 Declaring variables and annotating the code 562
C.5.2 Visual inspection of the C translation 565
C.5.3 Building the extension module 566
C.5.4 Calling the Cython function from Python 567

C.6 Migrating loops to Fortran 568
C.6.1 The Fortran subroutine 568
C.6.2 Building the Fortran module with f2py 569
C.6.3 How to avoid array copying 571

C.7 Migrating loops to C via Cython 573
C.7.1 Translating index pairs to single indices 573
C.7.2 The complete C code 574
C.7.3 The Cython interface file 575
C.7.4 Building the extension module 576

C.8 Migrating loops to C via f2py 577
C.8.1 Migrating loops to C++ via f2py 578

C.9 Exercises 579

References 581

Index 583

List of Exercises, Problems, and Projects

Problem 1.1: Use linear/quadratic functions for verification 57
Exercise 1.2: Show linear growth of the phase with time 59
Exercise 1.3: Improve the accuracy by adjusting the frequency . . . 59
Exercise 1.4: See if adaptive methods improve the phase error . . . 60
Exercise 1.5: Use a Taylor polynomial to compute u^1 60
Problem 1.6: Derive and investigate the velocity Verlet method . . 60
Problem 1.7: Find the minimal resolution of an oscillatory function 61
Exercise 1.8: Visualize the accuracy of finite differences for a cosine function 61
Exercise 1.9: Verify convergence rates of the error in energy 61
Exercise 1.10: Use linear/quadratic functions for verification 62
Exercise 1.11: Use an exact discrete solution for verification 62
Exercise 1.12: Use analytical solution for convergence rate tests . . 62
Exercise 1.13: Investigate the amplitude errors of many solvers . . 63
Problem 1.14: Minimize memory usage of a simple vibration solver 64
Problem 1.15: Minimize memory usage of a general vibration solver 65
Exercise 1.16: Implement the Euler-Cromer scheme for the generalized model 65
Problem 1.17: Interpret $[D_tD_tu]^n$ as a forward-backward difference 66
Exercise 1.18: Analysis of the Euler-Cromer scheme 66
Exercise 1.19: Implement the solver via classes 78
Problem 1.20: Use a backward difference for the damping term . . 79
Exercise 1.21: Use the forward-backward scheme with quadratic damping 79
Exercise 1.22: Simulate resonance 104

© 2017, Hans Petter Langtangen, Svein Linge. Released under CC Attribution 4.0 license

Exercise 1.23: Simulate oscillations of a sliding box 105
Exercise 1.24: Simulate a bouncing ball 105
Exercise 1.25: Simulate a simple pendulum 105
Exercise 1.26: Simulate an elastic pendulum 106
Exercise 1.27: Simulate an elastic pendulum with air resistance . . 107
Exercise 1.28: Implement the PEFRL algorithm 108
Exercise 2.1: Simulate a standing wave 145
Exercise 2.2: Add storage of solution in a user action function . . . 146
Exercise 2.3: Use a class for the user action function 147
Exercise 2.4: Compare several Courant numbers in one movie . . . 147
Exercise 2.5: Implementing the solver function as a generator . . . 147
Project 2.6: Calculus with 1D mesh functions 148
Exercise 2.7: Find the analytical solution to a damped wave equation 175
Problem 2.8: Explore symmetry boundary conditions 176
Exercise 2.9: Send pulse waves through a layered medium 176
Exercise 2.10: Explain why numerical noise occurs 177
Exercise 2.11: Investigate harmonic averaging in a 1D model 177
Problem 2.12: Implement open boundary conditions 177
Exercise 2.13: Implement periodic boundary conditions 179
Exercise 2.14: Compare discretizations of a Neumann condition . . 180
Exercise 2.15: Verification by a cubic polynomial in space 181
Exercise 2.16: Check that a solution fulfills the discrete model . . . 214
Project 2.17: Calculus with 2D mesh functions 215
Exercise 2.18: Implement Neumann conditions in 2D 216
Exercise 2.19: Test the efficiency of compiled loops in 3D 216
Exercise 2.20: Simulate waves on a non-homogeneous string 232
Exercise 2.21: Simulate damped waves on a string 232
Exercise 2.22: Simulate elastic waves in a rod 232
Exercise 2.23: Simulate spherical waves 233
Problem 2.24: Earthquake-generated tsunami over a subsea hill . . 233
Problem 2.25: Earthquake-generated tsunami over a 3D hill 236
Problem 2.26: Investigate Mayavi for visualization 237
Problem 2.27: Investigate visualization packages 237
Problem 2.28: Implement loops in compiled languages 238
Exercise 2.29: Simulate seismic waves in 2D 238
Project 2.30: Model 3D acoustic waves in a room 238
Project 2.31: Solve a 1D transport equation 240
Problem 2.32: General analytical solution of a 1D damped wave equation 243

Problem 2.33: General analytical solution of a 2D damped wave equation . . . 245
Exercise 3.1: Explore symmetry in a 1D problem . . . 289
Exercise 3.2: Investigate approximation errors from a $u_x = 0$ boundary condition . . . 290
Exercise 3.3: Experiment with open boundary conditions in 1D . . . 290
Exercise 3.4: Simulate a diffused Gaussian peak in 2D/3D . . . 292
Exercise 3.5: Examine stability of a diffusion model with a source term . . . 292
Exercise 3.6: Stabilizing the Crank-Nicolson method by Rannacher time stepping . . . 376
Project 3.7: Energy estimates for diffusion problems . . . 377
Exercise 3.8: Splitting methods and preconditioning . . . 380
Problem 3.9: Oscillating surface temperature of the earth . . . 380
Problem 3.10: Oscillating and pulsating flow in tubes . . . 381
Problem 3.11: Scaling a welding problem . . . 382
Exercise 3.12: Implement a Forward Euler scheme for axi-symmetric diffusion . . . 384
Exercise 4.1: Analyze 1D stationary convection-diffusion problem 417
Exercise 4.2: Interpret upwind difference as artificial diffusion . . . 418
Problem 5.1: Determine if equations are nonlinear or not . . . 478
Problem 5.2: Derive and investigate a generalized logistic model . 478
Problem 5.3: Experience the behavior of Newton's method . . . 479
Exercise 5.4: Compute the Jacobian of a 2×2 system . . . 480
Problem 5.5: Solve nonlinear equations arising from a vibration ODE . . . 480
Exercise 5.6: Find the truncation error of arithmetic mean of products . . . 481
Problem 5.7: Newton's method for linear problems . . . 482
Problem 5.8: Discretize a 1D problem with a nonlinear coefficient 482
Problem 5.9: Linearize a 1D problem with a nonlinear coefficient 482
Problem 5.10: Finite differences for the 1D Bratu problem . . . 483
Problem 5.11: Discretize a nonlinear 1D heat conduction PDE by finite differences . . . 484
Problem 5.12: Differentiate a highly nonlinear term . . . 484
Exercise 5.13: Crank-Nicolson for a nonlinear 3D diffusion equation 485
Problem 5.14: Find the sparsity of the Jacobian . . . 485
Problem 5.15: Investigate a 1D problem with a continuation method 485
Exercise B.1: Truncation error of a weighted mean . . . 531
Exercise B.2: Simulate the error of a weighted mean . . . 531

Exercise B.3: Verify a truncation error formula 531
Problem B.4: Truncation error of the Backward Euler scheme . . . 531
Exercise B.5: Empirical estimation of truncation errors 531
Exercise B.6: Correction term for a Backward Euler scheme 532
Problem B.7: Verify the effect of correction terms 532
Problem B.8: Truncation error of the Crank-Nicolson scheme 532
Problem B.9: Truncation error of $u' = f(u,t)$ 533
Exercise B.10: Truncation error of $[D_tD_tu]^n$ 533
Exercise B.11: Investigate the impact of approximating $u'(0)$ 533
Problem B.12: Investigate the accuracy of a simplified scheme . . . 534
Exercise C.1: Explore computational efficiency of numpy.sum versus built-in sum 579
Exercise C.2: Make an improved `numpy.savez` function 579
Exercise C.3: Visualize the impact of the Courant number 580
Exercise C.4: Visualize the impact of the resolution 580

Vibration ODEs

1

Vibration problems lead to differential equations with solutions that oscillate in time, typically in a damped or undamped sinusoidal fashion. Such solutions put certain demands on the numerical methods compared to other phenomena whose solutions are monotone or very smooth. Both the frequency and amplitude of the oscillations need to be accurately handled by the numerical schemes. The forthcoming text presents a range of different methods, from classical ones (Runge-Kutta and midpoint/Crank-Nicolson methods), to more modern and popular symplectic (geometric) integration schemes (Leapfrog, Euler-Cromer, and Störmer-Verlet methods), but with a clear emphasis on the latter. Vibration problems occur throughout mechanics and physics, but the methods discussed in this text are also fundamental for constructing successful algorithms for partial differential equations of wave nature in multiple spatial dimensions.

1.1 Finite difference discretization

Many of the numerical challenges faced when computing oscillatory solutions to ODEs and PDEs can be captured by the very simple ODE $u'' + u = 0$. This ODE is thus chosen as our starting point for method development, implementation, and analysis.

© 2017, Hans Petter Langtangen, Svein Linge. Released under CC Attribution 4.0 license

1.1.1 A basic model for vibrations

The simplest model of a vibrating mechanical system has the following form:

$$u'' + \omega^2 u = 0, \quad u(0) = I, \ u'(0) = 0, \ t \in (0, T]\,. \tag{1.1}$$

Here, ω and I are given constants. Section 1.12.1 derives (1.1) from physical principles and explains what the constants mean.

The exact solution of (1.1) is

$$u(t) = I\cos(\omega t)\,. \tag{1.2}$$

That is, u oscillates with constant amplitude I and angular frequency ω. The corresponding period of oscillations (i.e., the time between two neighboring peaks in the cosine function) is $P = 2\pi/\omega$. The number of periods per second is $f = \omega/(2\pi)$ and measured in the unit Hz. Both f and ω are referred to as frequency, but ω is more precisely named *angular frequency*, measured in rad/s.

In vibrating mechanical systems modeled by (1.1), $u(t)$ very often represents a position or a displacement of a particular point in the system. The derivative $u'(t)$ then has the interpretation of velocity, and $u''(t)$ is the associated acceleration. The model (1.1) is not only applicable to vibrating mechanical systems, but also to oscillations in electrical circuits.

1.1.2 A centered finite difference scheme

To formulate a finite difference method for the model problem (1.1), we follow the four steps explained in Section 1.1.2 in [9].

Step 1: Discretizing the domain. The domain is discretized by introducing a uniformly partitioned time mesh. The points in the mesh are $t_n = n\Delta t$, $n = 0, 1, \ldots, N_t$, where $\Delta t = T/N_t$ is the constant length of the time steps. We introduce a mesh function u^n for $n = 0, 1, \ldots, N_t$, which approximates the exact solution at the mesh points. (Note that $n = 0$ is the known initial condition, so u^n is identical to the mathematical u at this point.) The mesh function u^n will be computed from algebraic equations derived from the differential equation problem.

Step 2: Fulfilling the equation at discrete time points. The ODE is to be satisfied at each mesh point where the solution must be found:

$$u''(t_n) + \omega^2 u(t_n) = 0, \quad n = 1, \ldots, N_t\,. \tag{1.3}$$

Step 3: Replacing derivatives by finite differences. The derivative $u''(t_n)$ is to be replaced by a finite difference approximation. A common second-order accurate approximation to the second-order derivative is

$$u''(t_n) \approx \frac{u^{n+1} - 2u^n + u^{n-1}}{\Delta t^2}\,. \tag{1.4}$$

Inserting (1.4) in (1.3) yields

$$\frac{u^{n+1} - 2u^n + u^{n-1}}{\Delta t^2} = -\omega^2 u^n\,. \tag{1.5}$$

We also need to replace the derivative in the initial condition by a finite difference. Here we choose a centered difference, whose accuracy is similar to the centered difference we used for u'':

$$\frac{u^1 - u^{-1}}{2\Delta t} = 0\,. \tag{1.6}$$

Step 4: Formulating a recursive algorithm. To formulate the computational algorithm, we assume that we have already computed u^{n-1} and u^n, such that u^{n+1} is the unknown value to be solved for:

$$u^{n+1} = 2u^n - u^{n-1} - \Delta t^2\omega^2 u^n\,. \tag{1.7}$$

The computational algorithm is simply to apply (1.7) successively for $n = 1, 2, \ldots, N_t - 1$. This numerical scheme sometimes goes under the name Störmer's method, Verlet integration[1], or the Leapfrog method (one should note that Leapfrog is used for many quite different methods for quite different differential equations!).

Computing the first step. We observe that (1.7) cannot be used for $n = 0$ since the computation of u^1 then involves the undefined value u^{-1} at $t = -\Delta t$. The discretization of the initial condition then comes to our rescue: (1.6) implies $u^{-1} = u^1$ and this relation can be combined with (1.7) for $n = 0$ to yield a value for u^1:

$$u^1 = 2u^0 - u^1 - \Delta t^2\omega^2 u^0,$$

which reduces to

[1] `http://en.wikipedia.org/wiki/Verlet_integration`

$$u^1 = u^0 - \frac{1}{2}\Delta t^2\omega^2 u^0 \,. \tag{1.8}$$

Exercise 1.5 asks you to perform an alternative derivation and also to generalize the initial condition to $u'(0) = V \neq 0$.

The computational algorithm. The steps for solving (1.1) become

1. $u^0 = I$
2. compute u^1 from (1.8)
3. for $n = 1, 2, \ldots, N_t - 1$: compute u^{n+1} from (1.7)

The algorithm is more precisely expressed directly in Python:

```
t = linspace(0, T, Nt+1)  # mesh points in time
dt = t[1] - t[0]          # constant time step
u = zeros(Nt+1)           # solution

u[0] = I
u[1] = u[0] - 0.5*dt**2*w**2*u[0]
for n in range(1, Nt):
    u[n+1] = 2*u[n] - u[n-1] - dt**2*w**2*u[n]
```

Remark on using `w` for ω in computer code

In the code, we use `w` as the symbol for ω. The reason is that the authors prefer `w` for readability and comparison with the mathematical ω instead of the full word `omega` as variable name.

Operator notation. We may write the scheme using a compact difference notation listed in Appendix A.1 (see also Section 1.1.8 in [9]). The difference (1.4) has the operator notation $[D_tD_tu]^n$ such that we can write:

$$[D_tD_tu + \omega^2 u = 0]^n \,. \tag{1.9}$$

Note that $[D_tD_tu]^n$ means applying a central difference with step $\Delta t/2$ twice:

$$[D_t(D_tu)]^n = \frac{[D_tu]^{n+\frac{1}{2}} - [D_tu]^{n-\frac{1}{2}}}{\Delta t}$$

which is written out as

$$\frac{1}{\Delta t}\left(\frac{u^{n+1} - u^n}{\Delta t} - \frac{u^n - u^{n-1}}{\Delta t}\right) = \frac{u^{n+1} - 2u^n + u^{n-1}}{\Delta t^2} \,.$$

The discretization of initial conditions can in the operator notation be expressed as

$$[u = I]^0, \quad [D_{2t}u = 0]^0, \tag{1.10}$$

where the operator $[D_{2t}u]^n$ is defined as

$$[D_{2t}u]^n = \frac{u^{n+1} - u^{n-1}}{2\Delta t}. \tag{1.11}$$

1.2 Implementation

1.2.1 Making a solver function

The algorithm from the previous section is readily translated to a complete Python function for computing and returning $u^0, u^1, \ldots, u^{N_t}$ and $t_0, t_1, \ldots, t_{N_t}$, given the input I, ω, Δt, and T:

```
import numpy as np
import matplotlib.pyplot as plt

def solver(I, w, dt, T):
    """
    Solve u'' + w**2*u = 0 for t in (0,T], u(0)=I and u'(0)=0,
    by a central finite difference method with time step dt.
    """
    dt = float(dt)
    Nt = int(round(T/dt))
    u = np.zeros(Nt+1)
    t = np.linspace(0, Nt*dt, Nt+1)

    u[0] = I
    u[1] = u[0] - 0.5*dt**2*w**2*u[0]
    for n in range(1, Nt):
        u[n+1] = 2*u[n] - u[n-1] - dt**2*w**2*u[n]
    return u, t
```

We have imported `numpy` and `matplotlib` under the names `np` and `plt`, respectively, as this is very common in the Python scientific computing community and a good programming habit (since we explicitly see where the different functions come from). An alternative is to do `from numpy import *` and a similar "import all" for Matplotlib to avoid the `np` and `plt` prefixes and make the code as close as possible to MATLAB. (See Section 5.1.4 in [9] for a discussion of the two types of import in Python.)

A function for plotting the numerical and the exact solution is also convenient to have:

```
def u_exact(t, I, w):
    return I*np.cos(w*t)

def visualize(u, t, I, w):
    plt.plot(t, u, 'r--o')
    t_fine = np.linspace(0, t[-1], 1001)  # very fine mesh for u_e
    u_e = u_exact(t_fine, I, w)
    plt.hold('on')
    plt.plot(t_fine, u_e, 'b-')
    plt.legend(['numerical', 'exact'], loc='upper left')
    plt.xlabel('t')
    plt.ylabel('u')
    dt = t[1] - t[0]
    plt.title('dt=%g' % dt)
    umin = 1.2*u.min();  umax = -umin
    plt.axis([t[0], t[-1], umin, umax])
    plt.savefig('tmp1.png');  plt.savefig('tmp1.pdf')
```

A corresponding main program calling these functions to simulate a given number of periods (`num_periods`) may take the form

```
I = 1
w = 2*pi
dt = 0.05
num_periods = 5
P = 2*pi/w    #  one period
T = P*num_periods
u, t = solver(I, w, dt, T)
visualize(u, t, I, w, dt)
```

Adjusting some of the input parameters via the command line can be handy. Here is a code segment using the `ArgumentParser` tool in the `argparse` module to define option value (`-option value`) pairs on the command line:

```
import argparse
parser = argparse.ArgumentParser()
parser.add_argument('--I', type=float, default=1.0)
parser.add_argument('--w', type=float, default=2*pi)
parser.add_argument('--dt', type=float, default=0.05)
parser.add_argument('--num_periods', type=int, default=5)
a = parser.parse_args()
I, w, dt, num_periods = a.I, a.w, a.dt, a.num_periods
```

Such parsing of the command line is explained in more detail in Section 5.2.3 in [9].

A typical execution goes like

Terminal

```
Terminal> python vib_undamped.py --num_periods 20 --dt 0.1
```

Computing u'. In mechanical vibration applications one is often interested in computing the velocity $v(t) = u'(t)$ after $u(t)$ has been computed. This can be done by a central difference,

$$v(t_n) = u'(t_n) \approx v^n = \frac{u^{n+1} - u^{n-1}}{2\Delta t} = [D_{2t}u]^n . \tag{1.12}$$

This formula applies for all inner mesh points, $n = 1, \ldots, N_t - 1$. For $n = 0$, $v(0)$ is given by the initial condition on $u'(0)$, and for $n = N_t$ we can use a one-sided, backward difference:

$$v^n = [D_t^- u]^n = \frac{u^n - u^{n-1}}{\Delta t} .$$

Typical (scalar) code is

```
v = np.zeros_like(u)  # or v = np.zeros(len(u))
# Use central difference for internal points
for i in range(1, len(u)-1):
    v[i] = (u[i+1] - u[i-1])/(2*dt)
# Use initial condition for u'(0) when i=0
v[0] = 0
# Use backward difference at the final mesh point
v[-1] = (u[-1] - u[-2])/dt
```

Since the loop is slow for large N_t, we can get rid of the loop by vectorizing the central difference. The above code segment goes as follows in its vectorized version (see Problem 1.2 in [9] for explanation of details):

```
v = np.zeros_like(u)
v[1:-1] = (u[2:] - u[:-2])/(2*dt)  # central difference
v[0] = 0                           # boundary condition u'(0)
v[-1] = (u[-1] - u[-2])/dt         # backward difference
```

1.2.2 Verification

Manual calculation. The simplest type of verification, which is also instructive for understanding the algorithm, is to compute u^1, u^2, and u^3 with the aid of a calculator and make a function for comparing these results with those from the `solver` function. The `test_three_steps` function in the file `vib_undamped.py` shows the details of how we use the hand calculations to test the code:

```
def test_three_steps():
    from math import pi
    I = 1;  w = 2*pi;  dt = 0.1;  T = 1
    u_by_hand = np.array([1.000000000000000,
```

```
                          0.802607911978213,
                          0.288358920740053])
    u, t = solver(I, w, dt, T)
    diff = np.abs(u_by_hand - u[:3]).max()
    tol = 1E-14
    assert diff < tol
```

This function is a proper *test function*, compliant with the pytest and nose testing framework for Python code, because

- the function name begins with `test_`
- the function takes no arguments
- the test is formulated as a boolean condition and executed by `assert`

We shall in this book implement all software verification via such proper test functions, also known as unit testing. See Section 5.3.2 in [9] for more details on how to construct test functions and utilize nose or pytest for automatic execution of tests. Our recommendation is to use pytest. With this choice, you can run all test functions in `vib_undamped.py` by

Terminal

```
Terminal> py.test -s -v vib_undamped.py
============================= test session starts ======...
platform linux2 -- Python 2.7.9 -- ...
collected 2 items

vib_undamped.py::test_three_steps PASSED
vib_undamped.py::test_convergence_rates PASSED

========================== 2 passed in 0.19 seconds ===...
```

Testing very simple polynomial solutions. Constructing test problems where the exact solution is constant or linear helps initial debugging and verification as one expects any reasonable numerical method to reproduce such solutions to machine precision. Second-order accurate methods will often also reproduce a quadratic solution. Here $[D_tD_t t^2]^n = 2$, which is the exact result. A solution $u = t^2$ leads to $u'' + \omega^2 u = 2 + (\omega t)^2 \neq 0$. We must therefore add a source in the equation: $u'' + \omega^2 u = f$ to allow a solution $u = t^2$ for $f = 2 + (\omega t)^2$. By simple insertion we can show that the mesh function $u^n = t_n^2$ is also a solution of the discrete equations. Problem 1.1 asks you to carry out all details to show that linear and quadratic solutions are solutions of the discrete equations. Such results are very useful for debugging and verification. You are strongly encouraged to do this problem now!

Checking convergence rates. Empirical computation of convergence rates yields a good method for verification. The method and its compu-

tational details are explained in detail in Section 3.1.6 in [9]. Readers not familiar with the concept should look up this reference before proceeding.

In the present problem, computing convergence rates means that we must

- perform m simulations, halving the time steps as: $\Delta t_i = 2^{-i}\Delta t_0$, $i = 1, \ldots, m-1$, and Δt_i is the time step used in simulation i;
- compute the L^2 norm of the error, $E_i = \sqrt{\Delta t_i \sum_{n=0}^{N_t-1}(u^n - u_{\mathrm{e}}(t_n))^2}$ in each case;
- estimate the convergence rates r_i based on two consecutive experiments $(\Delta t_{i-1}, E_{i-1})$ and $(\Delta t_i, E_i)$, assuming $E_i = C(\Delta t_i)^r$ and $E_{i-1} = C(\Delta t_{i-1})^r$, where C is a constant. From these equations it follows that $r = \ln(E_{i-1}/E_i)/\ln(\Delta t_{i-1}/\Delta t_i)$. Since this r will vary with i, we equip it with an index and call it r_{i-1}, where i runs from 1 to $m-1$.

The computed rates $r_0, r_1, \ldots, r_{m-2}$ hopefully converge to the number 2 in the present problem, because theory (from Section 1.4) shows that the error of the numerical method we use behaves like Δt^2. The convergence of the sequence $r_0, r_1, \ldots, r_{m-2}$ demands that the time steps Δt_i are sufficiently small for the error model $E_i = C(\Delta t_i)^r$ to be valid.

All the implementational details of computing the sequence $r_0, r_1, \ldots, r_{m-2}$ appear below.

```
def convergence_rates(m, solver_function, num_periods=8):
    """
    Return m-1 empirical estimates of the convergence rate
    based on m simulations, where the time step is halved
    for each simulation.
    solver_function(I, w, dt, T) solves each problem, where T
    is based on simulation for num_periods periods.
    """
    from math import pi
    w = 0.35; I = 0.3       # just chosen values
    P = 2*pi/w              # period
    dt = P/30               # 30 time step per period 2*pi/w
    T = P*num_periods

    dt_values = []
    E_values = []
    for i in range(m):
        u, t = solver_function(I, w, dt, T)
        u_e = u_exact(t, I, w)
        E = np.sqrt(dt*np.sum((u_e-u)**2))
        dt_values.append(dt)
        E_values.append(E)
        dt = dt/2
```

```
    r = [np.log(E_values[i-1]/E_values[i])/
         np.log(dt_values[i-1]/dt_values[i])
         for i in range(1, m, 1)]
    return r, E_values, dt_values
```

The error analysis in Section 1.4 is quite detailed and suggests that $r = 2$. It is also a intuitively reasonable result, since we used a second-order accurate finite difference approximation $[D_t D_t u]^n$ to the ODE and a second-order accurate finite difference formula for the initial condition for u'.

In the present problem, when Δt_0 corresponds to 30 time steps per period, the returned `r` list has all its values equal to 2.00 (if rounded to two decimals). This amazingly accurate result means that all Δt_i values are well into the asymptotic regime where the error model $E_i = C(\Delta t_i)^r$ is valid.

We can now construct a proper test function that computes convergence rates and checks that the final (and usually the best) estimate is sufficiently close to 2. Here, a rough tolerance of 0.1 is enough. Later, we will argue for an improvement by adjusting omega and include also that case in our test function here. The unit test goes like

```
def test_convergence_rates():
    r, E, dt = convergence_rates(
        m=5, solver_function=solver, num_periods=8)
    # Accept rate to 1 decimal place
    tol = 0.1
    assert abs(r[-1] - 2.0) < tol
    # Test that adjusted w obtains 4th order convergence
    r, E, dt = convergence_rates(
        m=5, solver_function=solver_adjust_w, num_periods=8)
    print 'adjust w rates:', r
    assert abs(r[-1] - 4.0) < tol
```

The complete code appears in the file `vib_undamped.py`.

Visualizing convergence rates with slope markers. Tony S. Yu has written a script `plotslopes.py`[2] that is very useful to indicate the slope of a graph, especially a graph like $\ln E = r \ln \Delta t + \ln C$ arising from the model $E = C\Delta t^r$. A copy of the script resides in the `src/vib`[3] directory. Let us use it to compare the original method for $u'' + \omega^2 u = 0$ with the same method applied to the equation with a modified ω. We make log-log plots of the error versus Δt. For each curve we attach a slope marker using the `slope_marker((x,y), r)` function from

[2] `http://goo.gl/A4Utm7`
[3] `http://tinyurl.com/nu656p2/vib`

`plotslopes.py`, where `(x,y)` is the position of the marker and `r` and the slope ($(r, 1)$), here (2,1) and (4,1).

```
def plot_convergence_rates():
    r2, E2, dt2 = convergence_rates(
        m=5, solver_function=solver, num_periods=8)
    plt.loglog(dt2, E2)
    r4, E4, dt4 = convergence_rates(
        m=5, solver_function=solver_adjust_w, num_periods=8)
    plt.loglog(dt4, E4)
    plt.legend(['original scheme', r'adjusted $\omega$'],
               loc='upper left')
    plt.title('Convergence of finite difference methods')
    from plotslopes import slope_marker
    slope_marker((dt2[1], E2[1]), (2,1))
    slope_marker((dt4[1], E4[1]), (4,1))
```

Figure 1.1 displays the two curves with the markers. The match of the curve slope and the marker slope is excellent.

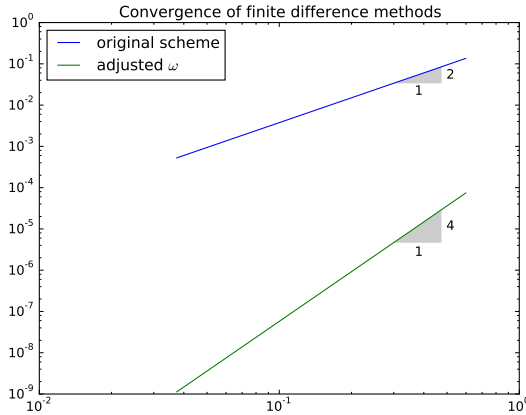


Fig. 1.1 Empirical convergence rate curves with special slope marker.

1.2.3 Scaled model

It is advantageous to use dimensionless variables in simulations, because fewer parameters need to be set. The present problem is made dimensionless by introducing dimensionless variables $\bar{t} = t/t_c$ and $\bar{u} = u/u_c$, where t_c and u_c are characteristic scales for t and u, respectively. We refer to Section 2.2.1 in [11] for all details about this scaling.

The scaled ODE problem reads

$$\frac{u_c}{t_c^2}\frac{d^2\bar{u}}{d\bar{t}^2} + u_c\bar{u} = 0, \quad u_c\bar{u}(0) = I, \ \frac{u_c}{t_c}\frac{d\bar{u}}{d\bar{t}}(0) = 0\,.$$

A common choice is to take t_c as one period of the oscillations, $t_c = 2\pi/w$, and $u_c = I$. This gives the dimensionless model

$$\frac{d^2\bar{u}}{d\bar{t}^2} + 4\pi^2\bar{u} = 0, \quad \bar{u}(0) = 1, \ \bar{u}'(0) = 0\,. \tag{1.13}$$

Observe that there are no physical parameters in (1.13)! We can therefore perform a single numerical simulation $\bar{u}(\bar{t})$ and afterwards recover any $u(t;\omega,I)$ by

$$u(t;\omega,I) = u_c\bar{u}(t/t_c) = I\bar{u}(\omega t/(2\pi))\,.$$

We can easily check this assertion: the solution of the scaled problem is $\bar{u}(\bar{t}) = \cos(2\pi\bar{t})$. The formula for u in terms of $\bar{u}$ gives $u = I\cos(\omega t)$, which is nothing but the solution of the original problem with dimensions.

The scaled model can be run by calling `solver(I=1, w=2*pi, dt, T)`. Each period is now 1 and `T` simply counts the number of periods. Choosing `dt` as `1./M` gives `M` time steps per period.

1.3 Visualization of long time simulations

Figure 1.2 shows a comparison of the exact and numerical solution for the scaled model (1.13) with $\Delta t = 0.1, 0.05$. From the plot we make the following observations:

- The numerical solution seems to have correct amplitude.
- There is an angular frequency error which is reduced by decreasing the time step.
- The total angular frequency error grows with time.

By angular frequency error we mean that the numerical angular frequency differs from the exact ω. This is evident by looking at the peaks of the numerical solution: these have incorrect positions compared with the peaks of the exact cosine solution. The effect can be mathematically expressed by writing the numerical solution as $I\cos\tilde{\omega}t$, where $\tilde{\omega}$ is not exactly equal to ω. Later, we shall mathematically quantify this numerical angular frequency $\tilde{\omega}$.

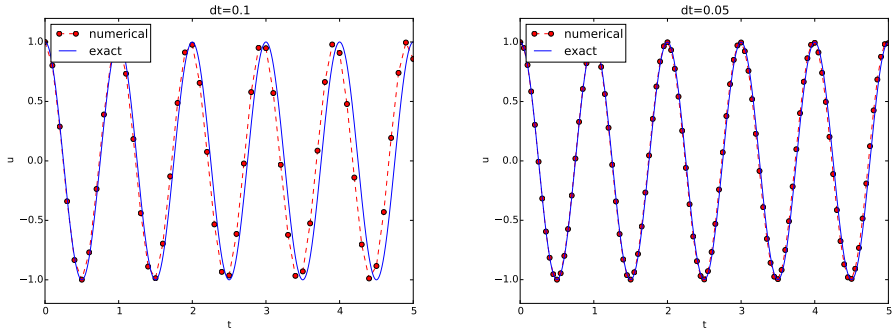


Fig. 1.2 Effect of halving the time step.

1.3.1 Using a moving plot window

In vibration problems it is often of interest to investigate the system's behavior over long time intervals. Errors in the angular frequency accumulate and become more visible as time grows. We can investigate long time series by introducing a moving plot window that can move along with the p most recently computed periods of the solution. The SciTools[4] package contains a convenient tool for this: `MovingPlotWindow`. Typing `pydoc scitools.MovingPlotWindow` shows a demo and a description of its use. The function below utilizes the moving plot window and is in fact called by the `main` function in the `vib_undamped` module if the number of periods in the simulation exceeds 10.

```
def visualize_front(u, t, I, w, savefig=False, skip_frames=1):
    """
    Visualize u and the exact solution vs t, using a
    moving plot window and continuous drawing of the
    curves as they evolve in time.
    Makes it easy to plot very long time series.
    Plots are saved to files if savefig is True.
    Only each skip_frames-th plot is saved (e.g., if
    skip_frame=10, only each 10th plot is saved to file;
    this is convenient if plot files corresponding to
    different time steps are to be compared).
    """
    import scitools.std as st
    from scitools.MovingPlotWindow import MovingPlotWindow
    from math import pi

    # Remove all old plot files tmp_*.png
    import glob, os
    for filename in glob.glob('tmp_*.png'):
        os.remove(filename)
```

[4] `https://github.com/hplgit/scitools`

```
    P = 2*pi/w  # one period
    umin = 1.2*u.min();  umax = -umin
    dt = t[1] - t[0]
    plot_manager = MovingPlotWindow(
        window_width=8*P,
        dt=dt,
        yaxis=[umin, umax],
        mode='continuous drawing')
    frame_counter = 0
    for n in range(1,len(u)):
        if plot_manager.plot(n):
            s = plot_manager.first_index_in_plot
            st.plot(t[s:n+1], u[s:n+1], 'r-1',
                    t[s:n+1], I*cos(w*t)[s:n+1], 'b-1',
                    title='t=%6.3f' % t[n],
                    axis=plot_manager.axis(),
                    show=not savefig) # drop window if savefig
            if savefig and n % skip_frames == 0:
                filename = 'tmp_%04d.png' % frame_counter
                st.savefig(filename)
                print 'making plot file', filename, 'at t=%g' % t[n]
                frame_counter += 1
        plot_manager.update(n)
```

We run the scaled problem (the default values for the command-line arguments `-I` and `-w` correspond to the scaled problem) for 40 periods with 20 time steps per period:

Terminal

```
Terminal> python vib_undamped.py --dt 0.05 --num_periods 40
```

The moving plot window is invoked, and we can follow the numerical and exact solutions as time progresses. From this demo we see that the angular frequency error is small in the beginning, and that it becomes more prominent with time. A new run with $\Delta t = 0.1$ (i.e., only 10 time steps per period) clearly shows that the phase errors become significant even earlier in the time series, deteriorating the solution further.

1.3.2 Making animations

Producing standard video formats. The `visualize_front` function stores all the plots in files whose names are numbered: `tmp_0000.png`, `tmp_0001.png`, `tmp_0002.png`, and so on. From these files we may make a movie. The Flash format is popular,

Terminal

```
Terminal> ffmpeg -r 25 -i tmp_%04d.png -c:v flv movie.flv
```

The `ffmpeg` program can be replaced by the `avconv` program in the above command if desired (but at the time of this writing it seems to be more momentum in the `ffmpeg` project). The `-r` option should come first and describes the number of frames per second in the movie (even if we would like to have slow movies, keep this number as large as 25, otherwise files are skipped from the movie). The `-i` option describes the name of the plot files. Other formats can be generated by changing the video codec and equipping the video file with the right extension:

Format	Codec and filename
Flash	`-c:v flv movie.flv`
MP4	`-c:v libx264 movie.mp4`
WebM	`-c:v libvpx movie.webm`
Ogg	`-c:v libtheora movie.ogg`

The video file can be played by some video player like `vlc`, `mplayer`, `gxine`, or `totem`, e.g.,

Terminal

```
Terminal> vlc movie.webm
```

A web page can also be used to play the movie. Today's standard is to use the HTML5 `video` tag:

```
<video autoplay loop controls
       width='640' height='365' preload='none'>
<source src='movie.webm'  type='video/webm; codecs="vp8, vorbis"'>
</video>
```

Modern browsers do not support all of the video formats. MP4 is needed to successfully play the videos on Apple devices that use the Safari browser. WebM is the preferred format for Chrome, Opera, Firefox, and Internet Explorer v9+. Flash was a popular format, but older browsers that required Flash can play MP4. All browsers that work with Ogg can also work with WebM. This means that to have a video work in all browsers, the video should be available in the MP4 and WebM formats. The proper HTML code reads

```
<video autoplay loop controls
       width='640' height='365' preload='none'>
<source src='movie.mp4'   type='video/mp4;
 codecs="avc1.42E01E, mp4a.40.2"'>
<source src='movie.webm'  type='video/webm;
```

```
  codecs="vp8, vorbis"'>
</video>
```

The MP4 format should appear first to ensure that Apple devices will load the video correctly.

Caution: number the plot files correctly

To ensure that the individual plot frames are shown in correct order, it is important to number the files with zero-padded numbers (0000, 0001, 0002, etc.). The printf format `%04d` specifies an integer in a field of width 4, padded with zeros from the left. A simple Unix wildcard file specification like `tmp_*.png` will then list the frames in the right order. If the numbers in the filenames were not zero-padded, the frame `tmp_11.png` would appear before `tmp_2.png` in the movie.

Playing PNG files in a web browser. The `scitools movie` command can create a movie player for a set of PNG files such that a web browser can be used to watch the movie. This interface has the advantage that the speed of the movie can easily be controlled, a feature that scientists often appreciate. The command for creating an HTML with a player for a set of PNG files `tmp_*.png` goes like

Terminal

```
Terminal> scitools movie output_file=vib.html fps=4 tmp_*.png
```

The `fps` argument controls the speed of the movie ("frames per second").

To watch the movie, load the video file `vib.html` into some browser, e.g.,

Terminal

```
Terminal> google-chrome vib.html  # invoke web page
```

Click on `Start movie` to see the result. Moving this movie to some other place requires moving `vib.html` *and all the PNG files* `tmp_*.png`:

Terminal

```
Terminal> mkdir vib_dt0.1
Terminal> mv tmp_*.png vib_dt0.1
Terminal> mv vib.html vib_dt0.1/index.html
```

Making animated GIF files. The `convert` program from the ImageMagick software suite can be used to produce animated GIF files from a set of PNG files:

Terminal

```
Terminal> convert -delay 25 tmp_vib*.png tmp_vib.gif
```

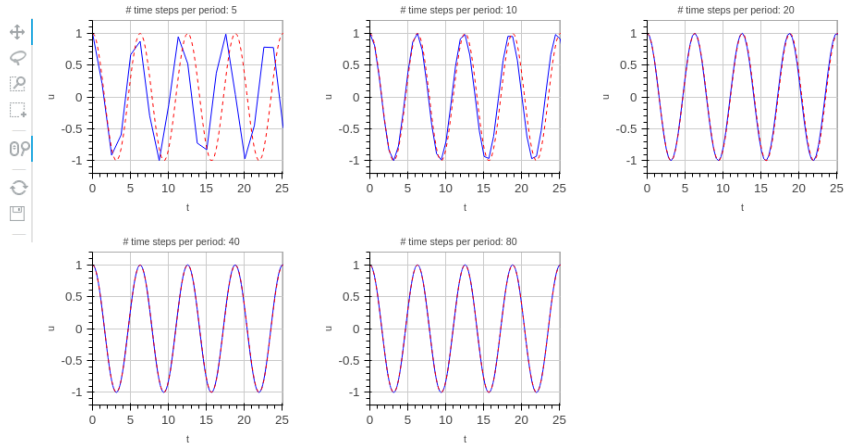
The `-delay` option needs an argument of the delay between each frame, measured in 1/100 s, so 4 frames/s here gives 25/100 s delay. Note, however, that in this particular example with $\Delta t = 0.05$ and 40 periods, making an animated GIF file out of the large number of PNG files is a very heavy process and not considered feasible. Animated GIFs are best suited for animations with not so many frames and where you want to see each frame and play them slowly.

1.3.3 Using Bokeh to compare graphs

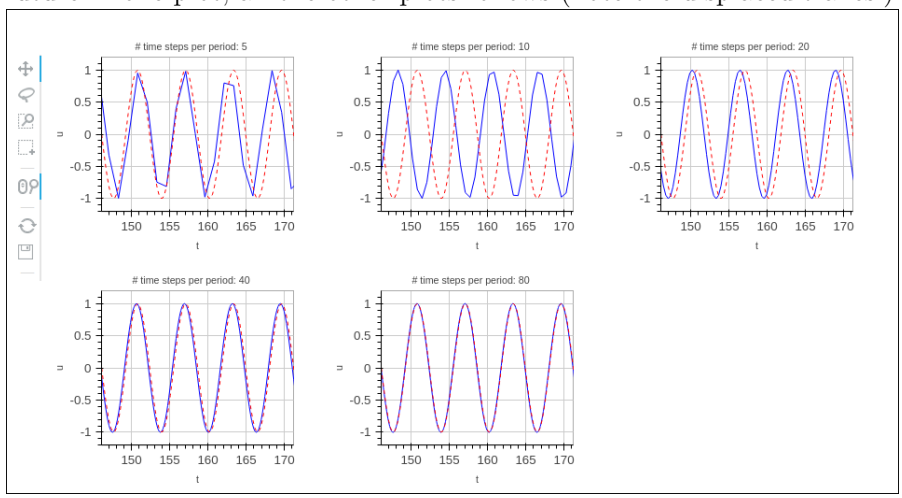
Instead of a moving plot frame, one can use tools that allow panning by the mouse. For example, we can show four periods of several signals in several plots and then scroll with the mouse through the rest of the simulation *simultaneously* in all the plot windows. The Bokeh[5] plotting library offers such tools, but the plots must be displayed in a web browser. The documentation of Bokeh is excellent, so here we just show how the library can be used to compare a set of u curves corresponding to long time simulations. (By the way, the guidance to correct pronunciation of Bokeh in the documentation[6] and on Wikipedia[7] is not directly compatible with a YouTube video[8]...).

Imagine we have performed experiments for a set of Δt values. We want each curve, together with the exact solution, to appear in a plot, and then arrange all plots in a grid-like fashion:

[5] `http://bokeh.pydata.org/en/latest`
[6] `http://bokeh.pydata.org/en/0.10.0/docs/faq.html#how-do-you-pronounce-bokeh`
[7] `https://en.wikipedia.org/wiki/Bokeh`
[8] `https://www.youtube.com/watch?v=OR8HSHevQTM`



Furthermore, we want the axes to couple such that if we move into the future in one plot, all the other plots follows (note the displaced t axes!):



A function for creating a Bokeh plot, given a list of `u` arrays and corresponding `t` arrays, is implemented below. The code combines data from different simulations, described compactly in a list of strings `legends`.

```
def bokeh_plot(u, t, legends, I, w, t_range, filename):
    """
    Make plots for u vs t using the Bokeh library.
    u and t are lists (several experiments can be compared).
    legens contain legend strings for the various u,t pairs.
    """
    if not isinstance(u, (list,tuple)):
        u = [u]  # wrap in list
    if not isinstance(t, (list,tuple)):
```

```
        t = [t]  # wrap in list
    if not isinstance(legends, (list,tuple)):
        legends = [legends]  # wrap in list

    import bokeh.plotting as plt
    plt.output_file(filename, mode='cdn', title='Comparison')
    # Assume that all t arrays have the same range
    t_fine = np.linspace(0, t[0][-1], 1001)  # fine mesh for u_e
    tools = 'pan,wheel_zoom,box_zoom,reset,'\
            'save,box_select,lasso_select'
    u_range = [-1.2*I, 1.2*I]
    font_size = '8pt'
    p = []  # list of plot objects
    # Make the first figure
    p_ = plt.figure(
        width=300, plot_height=250, title=legends[0],
        x_axis_label='t', y_axis_label='u',
        x_range=t_range, y_range=u_range, tools=tools,
        title_text_font_size=font_size)
    p_.xaxis.axis_label_text_font_size=font_size
    p_.yaxis.axis_label_text_font_size=font_size
    p_.line(t[0], u[0], line_color='blue')
    # Add exact solution
    u_e = u_exact(t_fine, I, w)
    p_.line(t_fine, u_e, line_color='red', line_dash='4 4')
    p.append(p_)
    # Make the rest of the figures and attach their axes to
    # the first figure's axes
    for i in range(1, len(t)):
        p_ = plt.figure(
            width=300, plot_height=250, title=legends[i],
            x_axis_label='t', y_axis_label='u',
            x_range=p[0].x_range, y_range=p[0].y_range, tools=tools,
            title_text_font_size=font_size)
        p_.xaxis.axis_label_text_font_size = font_size
        p_.yaxis.axis_label_text_font_size = font_size
        p_.line(t[i], u[i], line_color='blue')
        p_.line(t_fine, u_e, line_color='red', line_dash='4 4')
        p.append(p_)

    # Arrange all plots in a grid with 3 plots per row
    grid = [[]]
    for i, p_ in enumerate(p):
        grid[-1].append(p_)
        if (i+1) % 3 == 0:
            # New row
            grid.append([])
    plot = plt.gridplot(grid, toolbar_location='left')
    plt.save(plot)
    plt.show(plot)
```

A particular example using the `bokeh_plot` function appears below.

```
def demo_bokeh():
```

```
    """Solve a scaled ODE u'' + u = 0."""
    from math import pi
    w = 1.0         # Scaled problem (frequency)
    P = 2*np.pi/w  # Period
    num_steps_per_period = [5, 10, 20, 40, 80]
    T = 40*P        # Simulation time: 40 periods
    u = []          # List of numerical solutions
    t = []          # List of corresponding meshes
    legends = []
    for n in num_steps_per_period:
        dt = P/n
        u_, t_ = solver(I=1, w=w, dt=dt, T=T)
        u.append(u_)
        t.append(t_)
        legends.append('# time steps per period: %d' % n)
    bokeh_plot(u, t, legends, I=1, w=w, t_range=[0, 4*P],
               filename='tmp.html')
```

1.3.4 Using a line-by-line ascii plotter

Plotting functions vertically, line by line, in the terminal window using ascii characters only is a simple, fast, and convenient visualization technique for long time series. Note that the time axis then is positive downwards on the screen, so we can let the solution be visualized "forever". The tool `scitools.avplotter.Plotter` makes it easy to create such plots:

```
def visualize_front_ascii(u, t, I, w, fps=10):
    """
    Plot u and the exact solution vs t line by line in a
    terminal window (only using ascii characters).
    Makes it easy to plot very long time series.
    """
    from scitools.avplotter import Plotter
    import time
    from math import pi
    P = 2*pi/w
    umin = 1.2*u.min();  umax = -umin

    p = Plotter(ymin=umin, ymax=umax, width=60, symbols='+o')
    for n in range(len(u)):
        print p.plot(t[n], u[n], I*cos(w*t[n])), \
              '%.1f' % (t[n]/P)
        time.sleep(1/float(fps))
```

The call `p.plot` returns a line of text, with the t axis marked and a symbol `+` for the first function (`u`) and `o` for the second function (the exact solution). Here we append to this text a time counter reflecting how

many periods the current time point corresponds to. A typical output ($\omega = 2\pi$, $\Delta t = 0.05$) looks like this:

```
                          |                      o+      14.0
                          |                    + o       14.0
                          |                 +    o       14.1
                          |              +     o         14.1
                          |     +         o              14.2
                         +|       o                     14.2
                    +     |                             14.2
              +       o   |                             14.3
         +     o          |                             14.4
       +   o              |                             14.4
      +o                  |                             14.5
      o +                 |                             14.5
       o    +             |                             14.6
           o      +        |                             14.6
                o        + |                             14.7
                    o     | +                           14.7
                          |        +                    14.8
                          |       o       +             14.8
                          |               o     +       14.9
                          |                   o   +     14.9
                          |                       o+    15.0
```

1.3.5 Empirical analysis of the solution

For oscillating functions like those in Figure 1.2 we may compute the amplitude and frequency (or period) empirically. That is, we run through the discrete solution points (t_n, u_n) and find all maxima and minima points. The distance between two consecutive maxima (or minima) points can be used as estimate of the local period, while half the difference between the u value at a maximum and a nearby minimum gives an estimate of the local amplitude.

The local maxima are the points where

$$u^{n-1} < u^n > u^{n+1}, \quad n = 1, \ldots, N_t - 1, \tag{1.14}$$

and the local minima are recognized by

$$u^{n-1} > u^n < u^{n+1}, \quad n = 1, \ldots, N_t - 1\,. \tag{1.15}$$

In computer code this becomes

```
def minmax(t, u):
    minima = []; maxima = []
    for n in range(1, len(u)-1, 1):
```

```
        if u[n-1] > u[n] < u[n+1]:
            minima.append((t[n], u[n]))
        if u[n-1] < u[n] > u[n+1]:
            maxima.append((t[n], u[n]))
    return minima, maxima
```

Note that the two returned objects are lists of tuples.

Let (t_i, e_i), $i = 0, \ldots, M-1$, be the sequence of all the M maxima points, where t_i is the time value and e_i the corresponding u value. The local period can be defined as $p_i = t_{i+1} - t_i$. With Python syntax this reads

```
def periods(maxima):
    p = [extrema[n][0] - maxima[n-1][0]
         for n in range(1, len(maxima))]
    return np.array(p)
```

The list `p` created by a list comprehension is converted to an array since we probably want to compute with it, e.g., find the corresponding frequencies `2*pi/p`.

Having the minima and the maxima, the local amplitude can be calculated as the difference between two neighboring minimum and maximum points:

```
def amplitudes(minima, maxima):
    a = [(abs(maxima[n][1] - minima[n][1]))/2.0
         for n in range(min(len(minima),len(maxima)))]
    return np.array(a)
```

The code segments are found in the file `vib_empirical_analysis.py`.

Since `a[i]` and `p[i]` correspond to the i-th amplitude estimate and the i-th period estimate, respectively, it is most convenient to visualize the `a` and `p` values with the index `i` on the horizontal axis. (There is no unique time point associated with either of these estimate since values at two different time points were used in the computations.)

In the analysis of very long time series, it is advantageous to compute and plot `p` and `a` instead of u to get an impression of the development of the oscillations. Let us do this for the scaled problem and $\Delta t = 0.1, 0.05, 0.01$. A ready-made function

```
plot_empirical_freq_and_amplitude(u, t, I, w)
```

computes the empirical amplitudes and periods, and creates a plot where the amplitudes and angular frequencies are visualized together with the exact amplitude `I` and the exact angular frequency `w`. We can make a little program for creating the plot:

```
from vib_undamped import solver, plot_empirical_freq_and_amplitude
from math import pi
dt_values = [0.1, 0.05, 0.01]
u_cases = []
t_cases = []
for dt in dt_values:
    # Simulate scaled problem for 40 periods
    u, t = solver(I=1, w=2*pi, dt=dt, T=40)
    u_cases.append(u)
    t_cases.append(t)
plot_empirical_freq_and_amplitude(u_cases, t_cases, I=1, w=2*pi)
```

Figure 1.3 shows the result: we clearly see that lowering Δt improves the angular frequency significantly, while the amplitude seems to be more accurate. The lines with $\Delta t = 0.01$, corresponding to 100 steps per period, can hardly be distinguished from the exact values. The next section shows how we can get mathematical insight into why amplitudes are good while frequencies are more inaccurate.

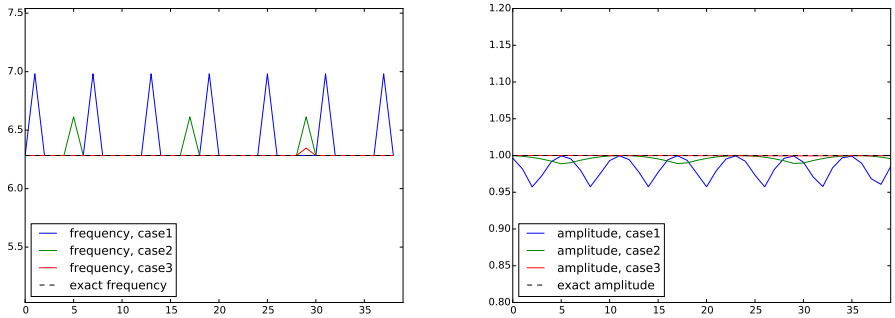


Fig. 1.3 Empirical angular frequency (left) and amplitude (right) for three different time steps.

1.4 Analysis of the numerical scheme

1.4.1 Deriving a solution of the numerical scheme

After having seen the phase error grow with time in the previous section, we shall now quantify this error through mathematical analysis. The key tool in the analysis will be to establish an exact solution of the discrete equations. The difference equation (1.7) has constant coefficients and is homogeneous. Such equations are known to have solutions on the form $u^n = CA^n$, where A is some number to be determined from the difference

equation and C is found as the initial condition ($C = I$). Recall that n in u^n is a superscript labeling the time level, while n in A^n is an exponent.

With oscillating functions as solutions, the algebra will be considerably simplified if we seek an A on the form

$$A = e^{i\tilde{\omega}\Delta t},$$

and solve for the numerical frequency $\tilde{\omega}$ rather than A. Note that $i = \sqrt{-1}$ is the imaginary unit. (Using a complex exponential function gives simpler arithmetics than working with a sine or cosine function.) We have

$$A^n = e^{i\tilde{\omega}\Delta t\, n} = e^{i\tilde{\omega}t_n} = \cos(\tilde{\omega}t_n) + i\sin(\tilde{\omega}t_n)\,.$$

The physically relevant numerical solution can be taken as the real part of this complex expression.

The calculations go as

$$\begin{aligned}
[D_tD_tu]^n &= \frac{u^{n+1} - 2u^n + u^{n-1}}{\Delta t^2} \\
&= I\frac{A^{n+1} - 2A^n + A^{n-1}}{\Delta t^2} \\
&= \frac{I}{\Delta t^2}(e^{i\tilde{\omega}(t_n+\Delta t)} - 2e^{i\tilde{\omega}t_n} + e^{i\tilde{\omega}(t_n-\Delta t)}) \\
&= Ie^{i\tilde{\omega}t_n}\frac{1}{\Delta t^2}\left(e^{i\tilde{\omega}\Delta t} + e^{i\tilde{\omega}(-\Delta t)} - 2\right) \\
&= Ie^{i\tilde{\omega}t_n}\frac{2}{\Delta t^2}\left(\cosh(i\tilde{\omega}\Delta t) - 1\right) \\
&= Ie^{i\tilde{\omega}t_n}\frac{2}{\Delta t^2}\left(\cos(\tilde{\omega}\Delta t) - 1\right) \\
&= -Ie^{i\tilde{\omega}t_n}\frac{4}{\Delta t^2}\sin^2(\frac{\tilde{\omega}\Delta t}{2})
\end{aligned}$$

The last line follows from the relation $\cos x - 1 = -2\sin^2(x/2)$ (try `cos(x)-1` in wolframalpha.com[9] to see the formula).

The scheme (1.7) with $u^n = Ie^{i\tilde{\omega}\Delta t\, n}$ inserted now gives

$$-Ie^{i\tilde{\omega}t_n}\frac{4}{\Delta t^2}\sin^2(\frac{\tilde{\omega}\Delta t}{2}) + \omega^2 Ie^{i\tilde{\omega}t_n} = 0, \tag{1.16}$$

which after dividing by $Ie^{i\tilde{\omega}t_n}$ results in

[9] `http://www.wolframalpha.com`

$$\frac{4}{\Delta t^2}\sin^2(\frac{\tilde{\omega}\Delta t}{2}) = \omega^2\,. \tag{1.17}$$

The first step in solving for the unknown $\tilde{\omega}$ is

$$\sin^2(\frac{\tilde{\omega}\Delta t}{2}) = \left(\frac{\omega\Delta t}{2}\right)^2\,.$$

Then, taking the square root, applying the inverse sine function, and multiplying by $2/\Delta t$, results in

$$\tilde{\omega} = \pm\frac{2}{\Delta t}\sin^{-1}\left(\frac{\omega\Delta t}{2}\right)\,. \tag{1.18}$$

1.4.2 The error in the numerical frequency

The first observation following (1.18) tells that there is a phase error since the numerical frequency $\tilde{\omega}$ never equals the exact frequency ω. But how good is the approximation (1.18)? That is, what is the error $\omega - \tilde{\omega}$ or $\tilde{\omega}/\omega$? Taylor series expansion for small Δt may give an expression that is easier to understand than the complicated function in (1.18):

```
>>> from sympy import *
>>> dt, w = symbols('dt w')
>>> w_tilde_e = 2/dt*asin(w*dt/2)
>>> w_tilde_series = w_tilde_e.series(dt, 0, 4)
>>> print w_tilde_series
w + dt**2*w**3/24 + O(dt**4)
```

This means that

$$\tilde{\omega} = \omega\left(1 + \frac{1}{24}\omega^2\Delta t^2\right) + \mathcal{O}(\Delta t^4)\,. \tag{1.19}$$

The error in the numerical frequency is of second-order in Δt, and the error vanishes as $\Delta t \to 0$. We see that $\tilde{\omega} > \omega$ since the term $\omega^3\Delta t^2/24 > 0$ and this is by far the biggest term in the series expansion for small $\omega\Delta t$. A numerical frequency that is too large gives an oscillating curve that oscillates too fast and therefore "lags behind" the exact oscillations, a feature that can be seen in the left plot in Figure 1.2.

Figure 1.4 plots the discrete frequency (1.18) and its approximation (1.19) for $\omega = 1$ (based on the program `vib_plot_freq.py`). Although $\tilde{\omega}$ is a function of Δt in (1.19), it is misleading to think of Δt as the important discretization parameter. It is the product $\omega\Delta t$ that is the key discretization parameter. This quantity reflects the *number of time steps*

per period of the oscillations. To see this, we set $P = N_P \Delta t$, where P is the length of a period, and N_P is the number of time steps during a period. Since P and ω are related by $P = 2\pi/\omega$, we get that $\omega\Delta t = 2\pi/N_P$, which shows that $\omega\Delta t$ is directly related to N_P.

The plot shows that at least $N_P \sim 25 - 30$ points per period are necessary for reasonable accuracy, but this depends on the length of the simulation (T) as the total phase error due to the frequency error grows linearly with time (see Exercise 1.2).

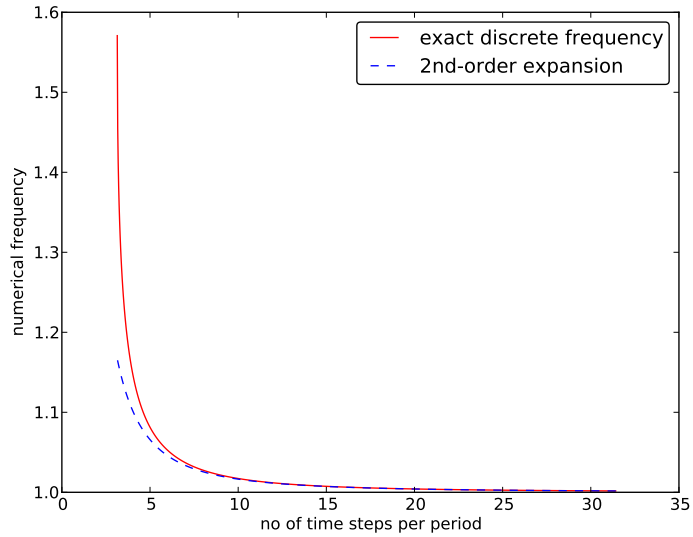


Fig. 1.4 Exact discrete frequency and its second-order series expansion.

1.4.3 Empirical convergence rates and adjusted ω

The expression (1.19) suggests that adjusting omega to

$$\omega\left(1 - \frac{1}{24}\omega^2\Delta t^2\right),$$

could have effect on the *convergence rate* of the global error in u (cf. Section 1.2.2). With the `convergence_rates` function in `vib_undamped.py` we can easily check this. A special solver, with adjusted w, is available as the function `solver_adjust_w`. A call to `convergence_rates` with

this solver reveals that the rate is 4.0! With the original, physical ω the rate is 2.0 - as expected from using second-order finite difference approximations, as expected from the forthcoming derivation of the global error, and as expected from truncation error analysis analysis as explained in Appendix B.4.1.

Adjusting ω is an ideal trick for this simple problem, but when adding damping and nonlinear terms, we have no simple formula for the impact on ω, and therefore we cannot use the trick.

1.4.4 Exact discrete solution

Perhaps more important than the $\tilde{\omega} = \omega + \mathcal{O}(\Delta t^2)$ result found above is the fact that we have an exact discrete solution of the problem:

$$u^n = I\cos\left(\tilde{\omega} n\Delta t\right), \quad \tilde{\omega} = \frac{2}{\Delta t}\sin^{-1}\left(\frac{\omega\Delta t}{2}\right). \tag{1.20}$$

We can then compute the error mesh function

$$e^n = u_{\mathrm{e}}(t_n) - u^n = I\cos\left(\omega n\Delta t\right) - I\cos\left(\tilde{\omega} n\Delta t\right). \tag{1.21}$$

From the formula $\cos 2x - \cos 2y = -2\sin(x-y)\sin(x+y)$ we can rewrite e^n so the expression is easier to interpret:

$$e^n = -2I\sin\left(t\frac{1}{2}\left(\omega - \tilde{\omega}\right)\right)\sin\left(t\frac{1}{2}\left(\omega + \tilde{\omega}\right)\right). \tag{1.22}$$

The error mesh function is ideal for verification purposes and you are strongly encouraged to make a test based on (1.20) by doing Exercise 1.11.

1.4.5 Convergence

We can use (1.19) and (1.21), or (1.22), to show *convergence* of the numerical scheme, i.e., $e^n \to 0$ as $\Delta t \to 0$, which implies that the numerical solution approaches the exact solution as Δt approaches to zero. We have that

$$\lim_{\Delta t\to 0}\tilde{\omega} = \lim_{\Delta t\to 0}\frac{2}{\Delta t}\sin^{-1}\left(\frac{\omega\Delta t}{2}\right) = \omega,$$

by L'Hopital's rule. This result could also been computed WolframAlpha[10], or we could use the limit functionality in `sympy`:

```
>>> import sympy as sym
>>> dt, w = sym.symbols('x w')
>>> sym.limit((2/dt)*sym.asin(w*dt/2), dt, 0, dir='+')
w
```

Also (1.19) can be used to establish that $\tilde{\omega} \to \omega$ when $\Delta t \to 0$. It then follows from the expression(s) for e^n that $e^n \to 0$.

1.4.6 The global error

To achieve more analytical insight into the nature of the global error, we can Taylor expand the error mesh function (1.21). Since $\tilde{\omega}$ in (1.18) contains Δt in the denominator we use the series expansion for $\tilde{\omega}$ inside the cosine function. A relevant `sympy` session is

```
>>> from sympy import *
>>> dt, w, t = symbols('dt w t')
>>> w_tilde_e = 2/dt*asin(w*dt/2)
>>> w_tilde_series = w_tilde_e.series(dt, 0, 4)
>>> w_tilde_series
w + dt**2*w**3/24 + O(dt**4)
```

Series expansions in `sympy` have the inconvenient `O()` term that prevents further calculations with the series. We can use the `removeO()` command to get rid of the `O()` term:

```
>>> w_tilde_series = w_tilde_series.removeO()
>>> w_tilde_series
dt**2*w**3/24 + w
```

Using this `w_tilde_series` expression for $\tilde{w}$ in (1.21), dropping I (which is a common factor), and performing a series expansion of the error yields

```
>>> error = cos(w*t) - cos(w_tilde_series*t)
>>> error.series(dt, 0, 6)
dt**2*t*w**3*sin(t*w)/24 + dt**4*t**2*w**6*cos(t*w)/1152 + O(dt**6)
```

Since we are mainly interested in the leading-order term in such expansions (the term with lowest power in Δt, which goes most slowly to zero), we use the `.as_leading_term(dt)` construction to pick out this term:

```
>>> error.series(dt, 0, 6).as_leading_term(dt)
dt**2*t*w**3*sin(t*w)/24
```

[10] `http://www.wolframalpha.com/input/?i=%282%2Fx%29*asin%28w*x%2F2%29+as+x-%3E0`

The last result means that the leading order global (true) error at a point t is proportional to $\omega^3 t \Delta t^2$. Considering only the discrete t_n values for t, t_n is related to Δt through $t_n = n\Delta t$. The factor $\sin(\omega t)$ can at most be 1, so we use this value to bound the leading-order expression to its maximum value

$$e^n = \frac{1}{24} n \omega^3 \Delta t^3 .$$

This is the dominating term of the error *at a point*.

We are interested in the accumulated global error, which can be taken as the ℓ^2 norm of e^n. The norm is simply computed by summing contributions from all mesh points:

$$||e^n||_{\ell^2}^2 = \Delta t \sum_{n=0}^{N_t} \frac{1}{24^2} n^2 \omega^6 \Delta t^6 = \frac{1}{24^2} \omega^6 \Delta t^7 \sum_{n=0}^{N_t} n^2 .$$

The sum $\sum_{n=0}^{N_t} n^2$ is approximately equal to $\frac{1}{3} N_t^3$. Replacing N_t by $T/\Delta t$ and taking the square root gives the expression

$$||e^n||_{\ell^2} = \frac{1}{24} \sqrt{\frac{T^3}{3}} \omega^3 \Delta t^2 .$$

This is our expression for the global (or integrated) error. A primary result from this expression is that the global error is proportional to Δt^2.

1.4.7 Stability

Looking at (1.20), it appears that the numerical solution has constant and correct amplitude, but an error in the angular frequency. A constant amplitude is not necessarily the case, however! To see this, note that if only Δt is large enough, the magnitude of the argument to $\sin^{-1}$ in (1.18) may be larger than 1, i.e., $\omega \Delta t/2 > 1$. In this case, $\sin^{-1}(\omega \Delta t/2)$ has a complex value and therefore $\tilde{\omega}$ becomes complex. Type, for example, `asin(x)` in wolframalpha.com[11] to see basic properties of $\sin^{-1}(x)$).

A complex $\tilde{\omega}$ can be written $\tilde{\omega} = \tilde{\omega}_r + i\tilde{\omega}_i$. Since $\sin^{-1}(x)$ has a *negative* imaginary part for $x > 1$, $\tilde{\omega}_i < 0$, which means that $e^{i\tilde{\omega}t} = e^{-\tilde{\omega}_i t} e^{i\tilde{\omega}_r t}$ will lead to exponential growth in time because $e^{-\tilde{\omega}_i t}$ with $\tilde{\omega}_i < 0$ has a positive exponent.

[11] `http://www.wolframalpha.com`

Stability criterion

We do not tolerate growth in the amplitude since such growth is not present in the exact solution. Therefore, we must impose a *stability criterion* so that the argument in the inverse sine function leads to real and not complex values of $\tilde{\omega}$. The stability criterion reads

$$\frac{\omega \Delta t}{2} \leq 1 \quad \Rightarrow \quad \Delta t \leq \frac{2}{\omega}. \tag{1.23}$$

With $\omega = 2\pi$, $\Delta t > \pi^{-1} = 0.3183098861837907$ will give growing solutions. Figure 1.5 displays what happens when $\Delta t = 0.3184$, which is slightly above the critical value: $\Delta t = \pi^{-1} + 9.01 \cdot 10^{-5}$.

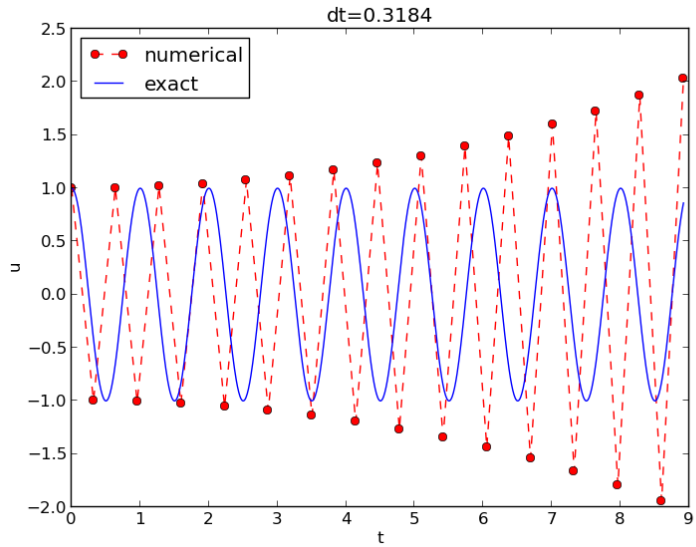


Fig. 1.5 Growing, unstable solution because of a time step slightly beyond the stability limit.

1.4.8 About the accuracy at the stability limit

An interesting question is whether the stability condition $\Delta t < 2/\omega$ is unfortunate, or more precisely: would it be meaningful to take larger time steps to speed up computations? The answer is a clear no. At

the stability limit, we have that $\sin^{-1}\omega\Delta t/2 = \sin^{-1} 1 = \pi/2$, and therefore $\tilde{\omega} = \pi/\Delta t$. (Note that the approximate formula (1.19) is very inaccurate for this value of Δt as it predicts $\tilde{\omega} = 2.34/pi$, which is a 25 percent reduction.) The corresponding period of the numerical solution is $\tilde{P} = 2\pi/\tilde{\omega} = 2\Delta t$, which means that there is just one time step Δt between a peak (maximum) and a through[12] (minimum) in the numerical solution. This is the shortest possible wave that can be represented in the mesh! In other words, it is not meaningful to use a larger time step than the stability limit.

Also, the error in angular frequency when $\Delta t = 2/\omega$ is severe: Figure 1.6 shows a comparison of the numerical and analytical solution with $\omega = 2\pi$ and $\Delta t = 2/\omega = \pi^{-1}$. Already after one period, the numerical solution has a through while the exact solution has a peak (!). The error in frequency when Δt is at the stability limit becomes $\omega - \tilde{\omega} = \omega(1 - \pi/2) \approx -0.57\omega$. The corresponding error in the period is $P - \tilde{P} \approx 0.36P$. The error after m periods is then $0.36mP$. This error has reached half a period when $m = 1/(2 \cdot 0.36) \approx 1.38$, which theoretically confirms the observations in Figure 1.6 that the numerical solution is a through ahead of a peak already after one and a half period. Consequently, Δt should be chosen much less than the stability limit to achieve meaningful numerical computations.

Summary

From the accuracy and stability analysis we can draw three important conclusions:

1. The key parameter in the formulas is $p = \omega\Delta t$. The period of oscillations is $P = 2\pi/\omega$, and the number of time steps per period is $N_P = P/\Delta t$. Therefore, $p = \omega\Delta t = 2\pi/N_P$, showing that the critical parameter is the number of time steps per period. The smallest possible N_P is 2, showing that $p \in (0, \pi]$.
2. Provided $p \leq 2$, the amplitude of the numerical solution is constant.
3. The ratio of the numerical angular frequency and the exact one is $\tilde{\omega}/\omega \approx 1 + \frac{1}{24}p^2$. The error $\frac{1}{24}p^2$ leads to wrongly displaced peaks of the numerical solution, and the error in peak location grows linearly with time (see Exercise 1.2).

[12] https://simple.wikipedia.org/wiki/Wave_(physics)

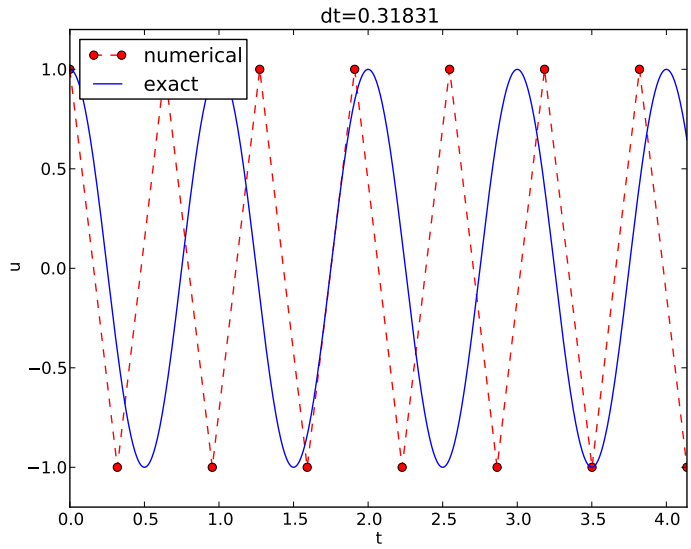


Fig. 1.6 Numerical solution with Δt exactly at the stability limit.

1.5 Alternative schemes based on 1st-order equations

A standard technique for solving second-order ODEs is to rewrite them as a system of first-order ODEs and then choose a solution strategy from the vast collection of methods for first-order ODE systems. Given the second-order ODE problem

$$u'' + \omega^2 u = 0, \quad u(0) = I,\ u'(0) = 0,$$

we introduce the auxiliary variable $v = u'$ and express the ODE problem in terms of first-order derivatives of u and v:

$$u' = v, \tag{1.24}$$

$$v' = -\omega^2 u\,. \tag{1.25}$$

The initial conditions become $u(0) = I$ and $v(0) = 0$.

1.5.1 The Forward Euler scheme

A Forward Euler approximation to our 2×2 system of ODEs (1.24)-(1.25) becomes

$$[D_t^+ u = v]^n, \tag{1.26}$$
$$[D_t^+ v = -\omega^2 u]^n, \tag{1.27}$$

or written out,

$$u^{n+1} = u^n + \Delta t v^n, \tag{1.28}$$
$$v^{n+1} = v^n - \Delta t \omega^2 u^n \,. \tag{1.29}$$

Let us briefly compare this Forward Euler method with the centered difference scheme for the second-order differential equation. We have from (1.28) and (1.29) applied at levels n and $n-1$ that

$$u^{n+1} = u^n + \Delta t v^n = u^n + \Delta t (v^{n-1} - \Delta t \omega^2 u^{n-1}) \,.$$

Since from (1.28)

$$v^{n-1} = \frac{1}{\Delta t}(u^n - u^{n-1}),$$

it follows that

$$u^{n+1} = 2u^n - u^{n-1} - \Delta t^2 \omega^2 u^{n-1},$$

which is very close to the centered difference scheme, but the last term is evaluated at t_{n-1} instead of t_n. Rewriting, so that $\Delta t^2 \omega^2 u^{n-1}$ appears alone on the right-hand side, and then dividing by Δt^2, the new left-hand side is an approximation to u'' at t_n, while the right-hand side is sampled at t_{n-1}. All terms should be sampled at the same mesh point, so using $\omega^2 u^{n-1}$ instead of $\omega^2 u^n$ points to a kind of mathematical error in the derivation of the scheme. This error turns out to be rather crucial for the accuracy of the Forward Euler method applied to vibration problems (Section 1.5.4 has examples).

The reasoning above does not imply that the Forward Euler scheme is not correct, but more that it is almost equivalent to a second-order accurate scheme for the second-order ODE formulation, and that the error committed has to do with a wrong sampling point.

1.5.2 The Backward Euler scheme

A Backward Euler approximation to the ODE system is equally easy to write up in the operator notation:

$$[D_t^- u = v]^{n+1}, \tag{1.30}$$
$$[D_t^- v = -\omega u]^{n+1}\,. \tag{1.31}$$

This becomes a coupled system for u^{n+1} and v^{n+1}:

$$u^{n+1} - \Delta t v^{n+1} = u^n, \tag{1.32}$$
$$v^{n+1} + \Delta t \omega^2 u^{n+1} = v^n\,. \tag{1.33}$$

We can compare (1.32)-(1.33) with the centered scheme (1.7) for the second-order differential equation. To this end, we eliminate v^{n+1} in (1.32) using (1.33) solved with respect to v^{n+1}. Thereafter, we eliminate v^n using (1.32) solved with respect to v^{n+1} and also replacing $n+1$ by n and n by $n-1$. The resulting equation involving only u^{n+1}, u^n, and u^{n-1} can be ordered as

$$\frac{u^{n+1} - 2u^n + u^{n-1}}{\Delta t^2} = -\omega^2 u^{n+1},$$

which has almost the same form as the centered scheme for the second-order differential equation, but the right-hand side is evaluated at u^{n+1} and not u^n. This inconsistent sampling of terms has a dramatic effect on the numerical solution, as we demonstrate in Section 1.5.4.

1.5.3 The Crank-Nicolson scheme

The Crank-Nicolson scheme takes this form in the operator notation:

$$[D_t u = \overline{v}^t]^{n+\frac{1}{2}}, \tag{1.34}$$
$$[D_t v = -\omega^2 \overline{u}^t]^{n+\frac{1}{2}}\,. \tag{1.35}$$

Writing the equations out and rearranging terms, shows that this is also a coupled system of two linear equations at each time level:

$$u^{n+1} - \frac{1}{2}\Delta t v^{n+1} = u^n + \frac{1}{2}\Delta t v^n, \tag{1.36}$$

$$v^{n+1} + \frac{1}{2}\Delta t \omega^2 u^{n+1} = v^n - \frac{1}{2}\Delta t \omega^2 u^n \,. \tag{1.37}$$

We may compare also this scheme to the centered discretization of the second-order ODE. It turns out that the Crank-Nicolson scheme is equivalent to the discretization

$$\frac{u^{n+1} - 2u^n + u^{n-1}}{\Delta t^2} = -\omega^2 \frac{1}{4}(u^{n+1} + 2u^n + u^{n-1}) = -\omega^2 u^n + \mathcal{O}(\Delta t^2)\,. \tag{1.38}$$

That is, the Crank-Nicolson is equivalent to (1.7) for the second-order ODE, apart from an extra term of size Δt^2, but this is an error of the same order as in the finite difference approximation on the left-hand side of the equation anyway. The fact that the Crank-Nicolson scheme is so close to (1.7) makes it a much better method than the Forward or Backward Euler methods for vibration problems, as will be illustrated in Section 1.5.4.

Deriving (1.38) is a bit tricky. We start with rewriting the Crank-Nicolson equations as follows

$$u^{n+1} - u^n = \frac{1}{2}\Delta t(v^{n+1} + v^n), \tag{1.39}$$

$$v^{n+1} = v^n - \frac{1}{2}\Delta t \omega^2 (u^{n+1} + u^n), \tag{1.40}$$

and add the latter at the previous time level as well:

$$v^n = v^{n-1} - \frac{1}{2}\Delta t \omega^2 (u^n + u^{n-1}) \tag{1.41}$$

We can also rewrite (1.39) at the previous time level as

$$v^n + v^{n-1} = \frac{2}{\Delta t}(u^n - u^{n-1})\,. \tag{1.42}$$

Inserting (1.40) for v^{n+1} in (1.39) and (1.41) for v^n in (1.39) yields after some reordering:

$$u^{n+1} - u^n = \frac{1}{2}(-\frac{1}{2}\Delta t \omega^2 (u^{n+1} + 2u^n + u^{n-1}) + v^n + v^{n-1})\,.$$

Now, $v^n + v^{n-1}$ can be eliminated by means of (1.42). The result becomes

$$u^{n+1} - 2u^n + u^{n-1} = -\Delta t^2\omega^2\frac{1}{4}(u^{n+1} + 2u^n + u^{n-1})\,. \tag{1.43}$$

It can be shown that

$$\frac{1}{4}(u^{n+1} + 2u^n + u^{n-1}) \approx u^n + \mathcal{O}(\Delta t^2),$$

meaning that (1.43) is an approximation to the centered scheme (1.7) for the second-order ODE where the sampling error in the term $\Delta t^2\omega^2 u^n$ is of the same order as the approximation errors in the finite differences, i.e., $\mathcal{O}(\Delta t^2)$. The Crank-Nicolson scheme written as (1.43) therefore has consistent sampling of all terms at the same time point t_n.

1.5.4 Comparison of schemes

We can easily compare methods like the ones above (and many more!) with the aid of the Odespy[13] package. Below is a sketch of the code.

```
import odespy
import numpy as np

def f(u, t, w=1):
    # v, u numbering for EulerCromer to work well
    v, u = u  # u is array of length 2 holding our [v, u]
    return [-w**2*u, v]

def run_solvers_and_plot(solvers, timesteps_per_period=20,
                         num_periods=1, I=1, w=2*np.pi):
    P = 2*np.pi/w  # duration of one period
    dt = P/timesteps_per_period
    Nt = num_periods*timesteps_per_period
    T = Nt*dt
    t_mesh = np.linspace(0, T, Nt+1)

    legends = []
    for solver in solvers:
        solver.set(f_kwargs={'w': w})
        solver.set_initial_condition([0, I])
        u, t = solver.solve(t_mesh)
```

There is quite some more code dealing with plots also, and we refer to the source file `vib_undamped_odespy.py` for details. Observe that keyword

[13] `https://github.com/hplgit/odespy`

arguments in `f(u,t,w=1)` can be supplied through a solver parameter `f_kwargs` (dictionary of additional keyword arguments to `f`).

Specification of the Forward Euler, Backward Euler, and Crank-Nicolson schemes is done like this:

```
solvers = [
    odespy.ForwardEuler(f),
    # Implicit methods must use Newton solver to converge
    odespy.BackwardEuler(f, nonlinear_solver='Newton'),
    odespy.CrankNicolson(f, nonlinear_solver='Newton'),
    ]
```

The `vib_undamped_odespy.py` program makes two plots of the computed solutions with the various methods in the `solvers` list: one plot with $u(t)$ versus t, and one *phase plane plot* where v is plotted against u. That is, the phase plane plot is the curve $(u(t), v(t))$ parameterized by t. Analytically, $u = I\cos(\omega t)$ and $v = u' = -\omega I\sin(\omega t)$. The exact curve $(u(t), v(t))$ is therefore an ellipse, which often looks like a circle in a plot if the axes are automatically scaled. The important feature, however, is that the exact curve $(u(t), v(t))$ is closed and repeats itself for every period. Not all numerical schemes are capable of doing that, meaning that the amplitude instead shrinks or grows with time.

Figure 1.7 show the results. Note that Odespy applies the label MidpointImplicit for what we have specified as `CrankNicolson` in the code (`CrankNicolson` is just a synonym for class `MidpointImplicit` in the Odespy code). The Forward Euler scheme in Figure 1.7 has a pronounced spiral curve, pointing to the fact that the amplitude steadily grows, which is also evident in Figure 1.8. The Backward Euler scheme has a similar feature, except that the spriral goes inward and the amplitude is significantly damped. The changing amplitude and the spiral form decreases with decreasing time step. The Crank-Nicolson scheme looks much more accurate. In fact, these plots tell that the Forward and Backward Euler schemes are not suitable for solving our ODEs with oscillating solutions.

1.5.5 Runge-Kutta methods

We may run two other popular standard methods for first-order ODEs, the 2nd- and 4th-order Runge-Kutta methods, to see how they perform. Figures 1.9 and 1.10 show the solutions with larger Δt values than what was used in the previous two plots.

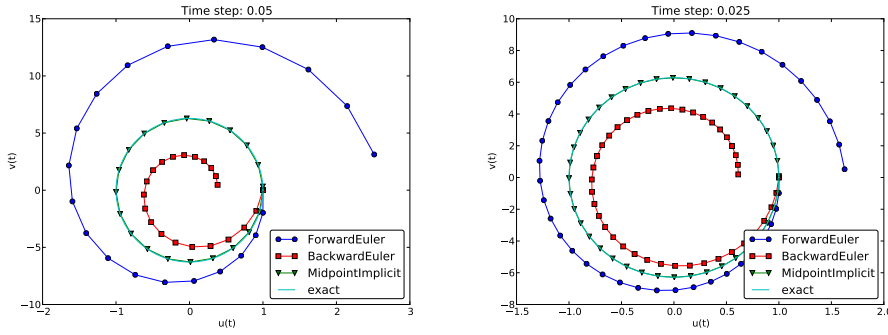


Fig. 1.7 Comparison of classical schemes in the phase plane for two time step values.

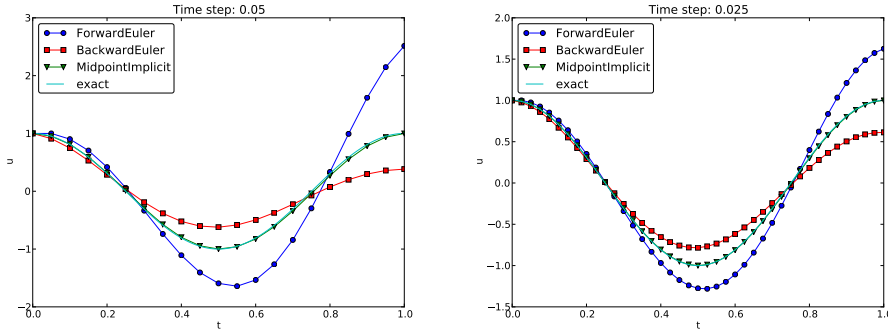


Fig. 1.8 Comparison of solution curves for classical schemes.

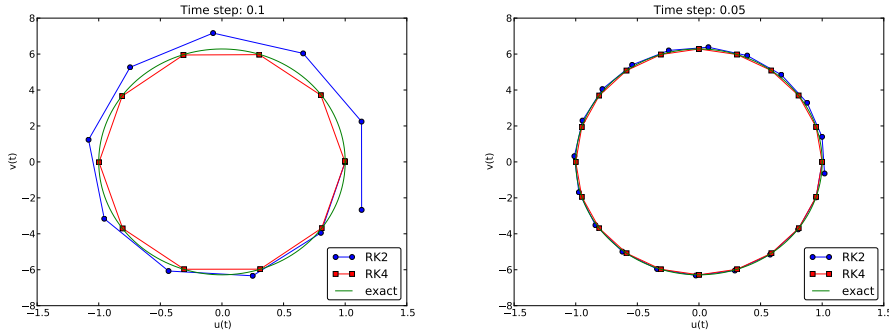


Fig. 1.9 Comparison of Runge-Kutta schemes in the phase plane.

The visual impression is that the 4th-order Runge-Kutta method is very accurate, under all circumstances in these tests, while the 2nd-order scheme suffers from amplitude errors unless the time step is very small.

The corresponding results for the Crank-Nicolson scheme are shown in Figure 1.11. It is clear that the Crank-Nicolson scheme outperforms the 2nd-order Runge-Kutta method. Both schemes have the same order

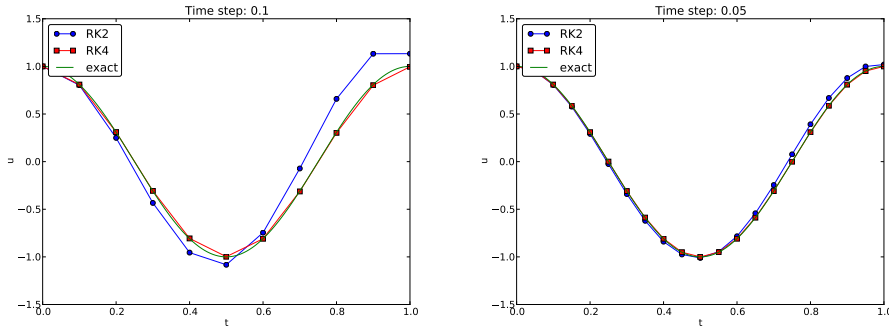


Fig. 1.10 Comparison of Runge-Kutta schemes.

of accuracy $\mathcal{O}(\Delta t^2)$, but their differences in the accuracy that matters in a real physical application is very clearly pronounced in this example. Exercise 1.13 invites you to investigate how the amplitude is computed by a series of famous methods for first-order ODEs.

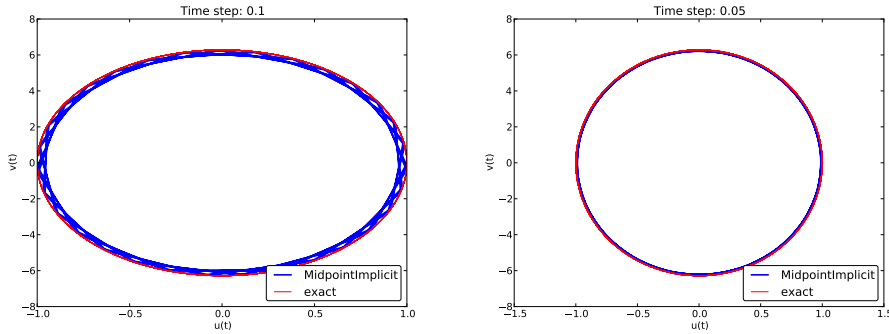


Fig. 1.11 Long-time behavior of the Crank-Nicolson scheme in the phase plane.

1.5.6 Analysis of the Forward Euler scheme

We may try to find exact solutions of the discrete equations (1.28)-(1.29) in the Forward Euler method to better understand why this otherwise useful method has so bad performance for vibration ODEs. An "ansatz" for the solution of the discrete equations is

$$u^n = IA^n,$$
$$v^n = qIA^n,$$

where q and A are scalars to be determined. We could have used a complex exponential form $e^{i\tilde{\omega}n\Delta t}$ since we get oscillatory solutions, but the oscillations grow in the Forward Euler method, so the numerical frequency $\tilde{\omega}$ will be complex anyway (producing an exponentially growing amplitude). Therefore, it is easier to just work with potentially complex A and q as introduced above.

The Forward Euler scheme leads to

$$A = 1 + \Delta t q,$$
$$A = 1 - \Delta t \omega^2 q^{-1}.$$

We can easily eliminate A, get $q^2 + \omega^2 = 0$, and solve for

$$q = \pm i\omega,$$

which gives

$$A = 1 \pm \Delta t i\omega.$$

We shall take the real part of A^n as the solution. The two values of A are complex conjugates, and the real part of A^n will be the same for both roots. This is easy to realize if we rewrite the complex numbers in polar form, which is also convenient for further analysis and understanding. The polar form $re^{i\theta}$ of a complex number $x + iy$ has $r = \sqrt{x^2 + y^2}$ and $\theta = \tan^{-1}(y/x)$. Hence, the polar form of the two values for A becomes

$$1 \pm \Delta t i\omega = \sqrt{1 + \omega^2 \Delta t^2} e^{\pm i \tan^{-1}(\omega \Delta t)}.$$

Now it is very easy to compute A^n:

$$(1 \pm \Delta t i\omega)^n = (1 + \omega^2 \Delta t^2)^{n/2} e^{\pm ni \tan^{-1}(\omega \Delta t)}.$$

Since $\cos(\theta n) = \cos(-\theta n)$, the real parts of the two numbers become the same. We therefore continue with the solution that has the plus sign.

The general solution is $u^n = CA^n$, where C is a constant determined from the initial condition: $u^0 = C = I$. We have $u^n = IA^n$ and $v^n = qIA^n$. The final solutions are just the real part of the expressions in polar form:

$$u^n = I(1 + \omega^2 \Delta t^2)^{n/2} \cos(n \tan^{-1}(\omega \Delta t)), \tag{1.44}$$
$$v^n = -\omega I(1 + \omega^2 \Delta t^2)^{n/2} \sin(n \tan^{-1}(\omega \Delta t)). \tag{1.45}$$

The expression $(1+\omega^2\Delta t^2)^{n/2}$ causes growth of the amplitude, since a number greater than one is raised to a positive exponent $n/2$. We can develop a series expression to better understand the formula for the amplitude. Introducing $p = \omega\Delta t$ as the key variable and using `sympy` gives

```
>>> from sympy import *
>>> p = symbols('p', real=True)
>>> n = symbols('n', integer=True, positive=True)
>>> amplitude = (1 + p**2)**(n/2)
>>> amplitude.series(p, 0, 4)
1 + n*p**2/2 + O(p**4)
```

The amplitude goes like $1 + \frac{1}{2}n\omega^2\Delta t^2$, clearly growing linearly in time (with n).

We can also investigate the error in the angular frequency by a series expansion:

```
>>> n*atan(p).series(p, 0, 4)
n*(p - p**3/3 + O(p**4))
```

This means that the solution for u^n can be written as

$$u^n = (1 + \frac{1}{2}n\omega^2\Delta t^2 + \mathcal{O}(\Delta t^4))\cos\left(\omega t - \frac{1}{3}\omega t\Delta t^2 + \mathcal{O}(\Delta t^4)\right).$$

The error in the angular frequency is of the same order as in the scheme (1.7) for the second-order ODE, but the error in the amplitude is severe.

1.6 Energy considerations

The observations of various methods in the previous section can be better interpreted if we compute a quantity reflecting the total *energy of the system*. It turns out that this quantity,

$$E(t) = \frac{1}{2}(u')^2 + \frac{1}{2}\omega^2 u^2,$$

is *constant* for all t. Checking that $E(t)$ really remains constant brings evidence that the numerical computations are sound. It turns out that E is proportional to the mechanical energy in the system. Conservation of energy is much used to check numerical simulations, so it is well invested time to dive into this subject.

1.6.1 Derivation of the energy expression

We start out with multiplying

$$u'' + \omega^2 u = 0,$$

by u' and integrating from 0 to T:

$$\int_0^T u''u'dt + \int_0^T \omega^2 uu'dt = 0\,.$$

Observing that

$$u''u' = \frac{d}{dt}\frac{1}{2}(u')^2, \quad uu' = \frac{d}{dt}\frac{1}{2}u^2,$$

we get

$$\int_0^T (\frac{d}{dt}\frac{1}{2}(u')^2 + \frac{d}{dt}\frac{1}{2}\omega^2 u^2)dt = E(T) - E(0) = 0,$$

where we have introduced

$$E(t) = \frac{1}{2}(u')^2 + \frac{1}{2}\omega^2 u^2\,. \tag{1.46}$$

The important result from this derivation is that the total energy is constant:

$$E(t) = E(0)\,.$$

$E(t)$ is closely related to the system's energy

The quantity $E(t)$ derived above is physically not the mechanical energy of a vibrating mechanical system, but the energy per unit mass. To see this, we start with Newton's second law $F = ma$ (F is the sum of forces, m is the mass of the system, and a is the acceleration). The displacement u is related to a through $a = u''$. With a spring force as the only force we have $F = -ku$, where k is a spring constant measuring the stiffness of the spring. Newton's second law then implies the differential equation

$$-ku = mu'' \quad \Rightarrow mu'' + ku = 0\,.$$

This equation of motion can be turned into an energy balance equation by finding the work done by each term during a time interval $[0, T]$. To this end, we multiply the equation by $du = u'dt$ and integrate:

$$\int_0^T muu'dt + \int_0^T kuu'dt = 0\,.$$

The result is

$$\tilde{E}(t) = E_k(t) + E_p(t) = 0,$$

where

$$E_k(t) = \frac{1}{2}mv^2, \quad v = u', \tag{1.47}$$

is the *kinetic energy* of the system, and

$$E_p(t) = \frac{1}{2}ku^2 \tag{1.48}$$

is the *potential energy*. The sum $\tilde{E}(t)$ is the total mechanical energy. The derivation demonstrates the famous energy principle that, under the right physical circumstances, any change in the kinetic energy is due to a change in potential energy and vice versa. (This principle breaks down when we introduce damping in the system, as we do in Section 1.10.)

The equation $mu'' + ku = 0$ can be divided by m and written as $u'' + \omega^2 u = 0$ for $\omega = \sqrt{k/m}$. The energy expression $E(t) = \frac{1}{2}(u')^2 + \frac{1}{2}\omega^2 u^2$ derived earlier is then $\tilde{E}(t)/m$, i.e., mechanical energy per unit mass.

Energy of the exact solution. Analytically, we have $u(t) = I\cos\omega t$, if $u(0) = I$ and $u'(0) = 0$, so we can easily check the energy evolution and confirm that $E(t)$ is constant:

$$E(t) = \frac{1}{2}I^2(-\omega\sin\omega t)^2 + \frac{1}{2}\omega^2 I^2\cos^2\omega t = \frac{1}{2}\omega^2(\sin^2\omega t + \cos^2\omega t) = \frac{1}{2}\omega^2\,.$$

Growth of energy in the Forward Euler scheme. It is easy to show that the energy in the Forward Euler scheme increases when stepping from time level n to $n+1$.

$$
\begin{aligned}
E^{n+1} &= \frac{1}{2}(v^{n+1})^2 + \frac{1}{2}\omega^2(u^{n+1})^2 \\
&= \frac{1}{2}(v^n - \omega^2 \Delta t u^n)^2 + \frac{1}{2}\omega^2(u^n + \Delta t v^n)^2 \\
&= (1 + \Delta t^2\omega^2)E^n \,.
\end{aligned}
$$

1.6.2 An error measure based on energy

The constant energy is well expressed by its initial value $E(0)$, so that the error in mechanical energy can be computed as a mesh function by

$$
e_E^n = \frac{1}{2}\left(\frac{u^{n+1} - u^{n-1}}{2\Delta t}\right)^2 + \frac{1}{2}\omega^2(u^n)^2 - E(0), \quad n = 1, \ldots, N_t - 1, \tag{1.49}
$$

where

$$
E(0) = \frac{1}{2}V^2 + \frac{1}{2}\omega^2 I^2,
$$

if $u(0) = I$ and $u'(0) = V$. Note that we have used a centered approximation to u': $u'(t_n) \approx [D_{2t}u]^n$.

A useful norm of the mesh function e_E^n for the discrete mechanical energy can be the maximum absolute value of e_E^n:

$$
||e_E^n||_{\ell^\infty} = \max_{1 \le n < N_t} |e_E^n| \,.
$$

Alternatively, we can compute other norms involving integration over all mesh points, but we are often interested in worst case deviation of the energy, and then the maximum value is of particular relevance.

A vectorized Python implementation of e_E^n takes the form

```
# import numpy as np and compute u, t
dt = t[1]-t[0]
E = 0.5*((u[2:] - u[:-2])/(2*dt))**2 + 0.5*w**2*u[1:-1]**2
E0 = 0.5*V**2 + 0.5**w**2*I**2
e_E = E - E0
e_E_norm = np.abs(e_E).max()
```

The convergence rates of the quantity `e_E_norm` can be used for verification. The value of `e_E_norm` is also useful for comparing schemes through their ability to preserve energy. Below is a table demonstrating

the relative error in total energy for various schemes (computed by the `vib_undamped_odespy.py` program). The test problem is $u'' + 4\pi^2 u = 0$ with $u(0) = 1$ and $u'(0) = 0$, so the period is 1 and $E(t) \approx 4.93$. We clearly see that the Crank-Nicolson and the Runge-Kutta schemes are superior to the Forward and Backward Euler schemes already after one period.

Method	T	Δt	$\max \lvert e_E^n \rvert / e_E^0$
Forward Euler	1	0.025	$1.678 \cdot 10^{0}$
Backward Euler	1	0.025	$6.235 \cdot 10^{-1}$
Crank-Nicolson	1	0.025	$1.221 \cdot 10^{-2}$
Runge-Kutta 2nd-order	1	0.025	$6.076 \cdot 10^{-3}$
Runge-Kutta 4th-order	1	0.025	$8.214 \cdot 10^{-3}$

However, after 10 periods, the picture is much more dramatic:

Method	T	Δt	$\max \lvert e_E^n \rvert / e_E^0$
Forward Euler	10	0.025	$1.788 \cdot 10^{4}$
Backward Euler	10	0.025	$1.000 \cdot 10^{0}$
Crank-Nicolson	10	0.025	$1.221 \cdot 10^{-2}$
Runge-Kutta 2nd-order	10	0.025	$6.250 \cdot 10^{-2}$
Runge-Kutta 4th-order	10	0.025	$8.288 \cdot 10^{-3}$

The Runge-Kutta and Crank-Nicolson methods hardly change their energy error with T, while the error in the Forward Euler method grows to huge levels and a relative error of 1 in the Backward Euler method points to $E(t) \to 0$ as t grows large.

Running multiple values of Δt, we can get some insight into the convergence of the energy error:

Method	T	Δt	$\max \lvert e_E^n \rvert / e_E^0$
Forward Euler	10	0.05	$1.120 \cdot 10^{8}$
Forward Euler	10	0.025	$1.788 \cdot 10^{4}$
Forward Euler	10	0.0125	$1.374 \cdot 10^{2}$
Backward Euler	10	0.05	$1.000 \cdot 10^{0}$
Backward Euler	10	0.025	$1.000 \cdot 10^{0}$
Backward Euler	10	0.0125	$9.928 \cdot 10^{-1}$
Crank-Nicolson	10	0.05	$4.756 \cdot 10^{-2}$
Crank-Nicolson	10	0.025	$1.221 \cdot 10^{-2}$
Crank-Nicolson	10	0.0125	$3.125 \cdot 10^{-3}$
Runge-Kutta 2nd-order	10	0.05	$6.152 \cdot 10^{-1}$
Runge-Kutta 2nd-order	10	0.025	$6.250 \cdot 10^{-2}$
Runge-Kutta 2nd-order	10	0.0125	$7.631 \cdot 10^{-3}$
Runge-Kutta 4th-order	10	0.05	$3.510 \cdot 10^{-2}$
Runge-Kutta 4th-order	10	0.025	$8.288 \cdot 10^{-3}$
Runge-Kutta 4th-order	10	0.0125	$2.058 \cdot 10^{-3}$

A striking fact from this table is that the error of the Forward Euler method is reduced by the same factor as Δt is reduced by, while the error in the Crank-Nicolson method has a reduction proportional to Δt^2 (we cannot say anything for the Backward Euler method). However, for the RK2 method, halving Δt reduces the error by almost a factor of 10 (!), and for the RK4 method the reduction seems proportional to Δt^2 only (and the trend is confirmed by running smaller time steps, so for $\Delta t = 3.9 \cdot 10^{-4}$ the relative error of RK2 is a factor 10 smaller than that of RK4!).

1.7 The Euler-Cromer method

While the Runge-Kutta methods and the Crank-Nicolson scheme work well for the vibration equation modeled as a first-order ODE system, both were inferior to the straightforward centered difference scheme for the second-order equation $u'' + \omega^2 u = 0$. However, there is a similarly successful scheme available for the first-order system $u' = v$, $v' = -\omega^2 u$, to be presented below. The ideas of the scheme and their further developments have become very popular in particle and rigid body dynamics and hence are widely used by physicists.

1.7.1 Forward-backward discretization

The idea is to apply a Forward Euler discretization to the first equation and a Backward Euler discretization to the second. In operator notation this is stated as

$$[D_t^+ u = v]^n, \tag{1.50}$$
$$[D_t^- v = -\omega^2 u]^{n+1}\,. \tag{1.51}$$

We can write out the formulas and collect the unknowns on the left-hand side:

$$u^{n+1} = u^n + \Delta t v^n, \tag{1.52}$$
$$v^{n+1} = v^n - \Delta t \omega^2 u^{n+1}\,. \tag{1.53}$$

We realize that after u^{n+1} has been computed from (1.52), it may be used directly in (1.53) to compute v^{n+1}.

In physics, it is more common to update the v equation first, with a forward difference, and thereafter the u equation, with a backward difference that applies the most recently computed v value:

$$v^{n+1} = v^n - \Delta t \omega^2 u^n, \tag{1.54}$$

$$u^{n+1} = u^n + \Delta t v^{n+1}\,. \tag{1.55}$$

The advantage of ordering the ODEs as in (1.54)-(1.55) becomes evident when considering complicated models. Such models are included if we write our vibration ODE more generally as

$$u'' + g(u, u', t) = 0\,.$$

We can rewrite this second-order ODE as two first-order ODEs,

$$v' = -g(u, v, t),$$
$$u' = v\,.$$

This rewrite allows the following scheme to be used:

$$v^{n+1} = v^n - \Delta t\, g(u^n, v^n, t),$$
$$u^{n+1} = u^n + \Delta t\, v^{n+1}\,.$$

We realize that the first update works well with any g since old values u^n and v^n are used. Switching the equations would demand u^{n+1} and v^{n+1} values in g and result in nonlinear algebraic equations to be solved at each time level.

The scheme (1.54)-(1.55) goes under several names: forward-backward scheme, semi-implicit Euler method[14], semi-explicit Euler, symplectic Euler, Newton-Störmer-Verlet, and Euler-Cromer. We shall stick to the latter name.

How does the Euler-Cromer method preserve the total energy? We may run the example from Section 1.6.2:

[14] `http://en.wikipedia.org/wiki/Semi-implicit_Euler_method`

Method	T	Δt	$\max \lvert e_E^n \rvert / e_E^0$
Euler-Cromer	10	0.05	$2.530 \cdot 10^{-2}$
Euler-Cromer	10	0.025	$6.206 \cdot 10^{-3}$
Euler-Cromer	10	0.0125	$1.544 \cdot 10^{-3}$

The relative error in the total energy decreases as Δt^2, and the error level is slightly lower than for the Crank-Nicolson and Runge-Kutta methods.

1.7.2 Equivalence with the scheme for the second-order ODE

We shall now show that the Euler-Cromer scheme for the system of first-order equations is equivalent to the centered finite difference method for the second-order vibration ODE (!).

We may eliminate the v^n variable from (1.52)-(1.53) or (1.54)-(1.55). The v^{n+1} term in (1.54) can be eliminated from (1.55):

$$u^{n+1} = u^n + \Delta t(v^n - \omega^2 \Delta t u^n)\,. \tag{1.56}$$

The v^n quantity can be expressed by u^n and u^{n-1} using (1.55):

$$v^n = \frac{u^n - u^{n-1}}{\Delta t},$$

and when this is inserted in (1.56) we get

$$u^{n+1} = 2u^n - u^{n-1} - \Delta t^2 \omega^2 u^n, \tag{1.57}$$

which is nothing but the centered scheme (1.7)! The two seemingly different numerical methods are mathematically equivalent. Consequently, the previous analysis of (1.7) also applies to the Euler-Cromer method. In particular, the amplitude is constant, given that the stability criterion is fulfilled, but there is always an angular frequency error (1.19). Exercise 1.18 gives guidance on how to derive the exact discrete solution of the two equations in the Euler-Cromer method.

Although the Euler-Cromer scheme and the method (1.7) are equivalent, there could be differences in the way they handle the initial conditions. Let us look into this topic. The initial condition $u' = 0$ means $u' = v = 0$. From (1.54) we get

$$v^1 = v^0 - \Delta t \omega^2 u^0 = \Delta t \omega^2 u^0,$$

and from (1.55) it follows that

$$u^1 = u^0 + \Delta t v^1 = u^0 - \omega^2 \Delta t^2 u^0 .$$

When we previously used a centered approximation of $u'(0) = 0$ combined with the discretization (1.7) of the second-order ODE, we got a slightly different result: $u^1 = u^0 - \frac{1}{2}\omega^2 \Delta t^2 u^0$. The difference is $\frac{1}{2}\omega^2 \Delta t^2 u^0$, which is of second order in Δt, seemingly consistent with the overall error in the scheme for the differential equation model.

A different view can also be taken. If we approximate $u'(0) = 0$ by a backward difference, $(u^0 - u^{-1})/\Delta t = 0$, we get $u^{-1} = u^0$, and when combined with (1.7), it results in $u^1 = u^0 - \omega^2 \Delta t^2 u^0$. This means that the Euler-Cromer method based on (1.55)-(1.54) corresponds to using only a first-order approximation to the initial condition in the method from Section 1.1.2.

Correspondingly, using the formulation (1.52)-(1.53) with $v^n = 0$ leads to $u^1 = u^0$, which can be interpreted as using a forward difference approximation for the initial condition $u'(0) = 0$. Both Euler-Cromer formulations lead to slightly different values for u^1 compared to the method in Section 1.1.2. The error is $\frac{1}{2}\omega^2 \Delta t^2 u^0$.

1.7.3 Implementation

Solver function. The function below, found in `vib_undamped_EulerCromer.py`, implements the Euler-Cromer scheme (1.54)-(1.55):

```
import numpy as np

def solver(I, w, dt, T):
    """
    Solve v' = - w**2*u, u'=v for t in (0,T], u(0)=I and v(0)=0,
    by an Euler-Cromer method.
    """
    dt = float(dt)
    Nt = int(round(T/dt))
    u = np.zeros(Nt+1)
    v = np.zeros(Nt+1)
    t = np.linspace(0, Nt*dt, Nt+1)

    v[0] = 0
    u[0] = I
    for n in range(0, Nt):
        v[n+1] = v[n] - dt*w**2*u[n]
        u[n+1] = u[n] + dt*v[n+1]
    return u, v, t
```

Verification. Since the Euler-Cromer scheme is equivalent to the finite difference method for the second-order ODE $u'' + \omega^2 u = 0$ (see Sec-

tion 1.7.2), the performance of the above `solver` function is the same as for the `solver` function in Section 1.2. The only difference is the formula for the first time step, as discussed above. This deviation in the Euler-Cromer scheme means that the discrete solution listed in Section 1.4.4 is not a solution of the Euler-Cromer scheme!

To verify the implementation of the Euler-Cromer method we can adjust `v[1]` so that the computer-generated values can be compared with the formula (1.20) from in Section 1.4.4. This adjustment is done in an alternative solver function, `solver_ic_fix` in `vib_EulerCromer.py`. Since we now have an exact solution of the discrete equations available, we can write a test function `test_solver` for checking the equality of computed values with the formula (1.20):

```
def test_solver():
    """
    Test solver with fixed initial condition against
    equivalent scheme for the 2nd-order ODE u'' + u = 0.
    """
    I = 1.2; w = 2.0; T = 5
    dt = 2/w  # longest possible time step
    u, v, t = solver_ic_fix(I, w, dt, T)
    from vib_undamped import solver as solver2  # 2nd-order ODE
    u2, t2 = solver2(I, w, dt, T)
    error = np.abs(u - u2).max()
    tol = 1E-14
    assert error < tol
```

Another function, `demo`, visualizes the difference between the Euler-Cromer scheme and the scheme (1.7) for the second-oder ODE, arising from the mismatch in the first time level.

Using Odespy. The Euler-Cromer method is also available in the Odespy package. The important thing to remember, when using this implementation, is that we must order the unknowns as v and u, so the `u` vector at each time level consists of the velocity v as first component and the displacement u as second component:

```
# Define ODE
def f(u, t, w=1):
    v, u = u
    return [-w**2*u, v]

# Initialize solver
I = 1
w = 2*np.pi
import odespy
solver = odespy.EulerCromer(f, f_kwargs={'w': w})
solver.set_initial_condition([0, I])
```

```
# Compute time mesh
P = 2*np.pi/w  # duration of one period
dt = P/timesteps_per_period
Nt = num_periods*timesteps_per_period
T = Nt*dt
import numpy as np
t_mesh = np.linspace(0, T, Nt+1)

# Solve ODE
u, t = solver.solve(t_mesh)
u = u[:,1]  # Extract displacement
```

Convergence rates. We may use the `convergence_rates` function in the file `vib_undamped.py` to investigate the convergence rate of the Euler-Cromer method, see the `convergence_rate` function in the file `vib_undamped_EulerCromer.py`. Since we could eliminate v to get a scheme for u that is equivalent to the finite difference method for the second-order equation in u, we would expect the convergence rates to be the same, i.e., $r = 2$. However, measuring the convergence rate of u in the Euler-Cromer scheme shows that $r = 1$ only! Adjusting the initial condition does not change the rate. Adjusting ω, as outlined in Section 1.4.2, gives a 4th-order method there, while there is no increase in the measured rate in the Euler-Cromer scheme. It is obvious that the Euler-Cromer scheme is dramatically much better than the two other first-order methods, Forward Euler and Backward Euler, but this is not reflected in the convergence rate of u.

1.7.4 The Störmer-Verlet algorithm

Another very popular algorithm for vibration problems, especially for long time simulations, is the Störmer-Verlet algorithm. It has become *the* method among physicists for molecular simulations as well as particle and rigid body dynamics.

The method can be derived by applying the Euler-Cromer idea twice, in a symmetric fashion, during the interval $[t_n, t_{n+1}]$:

1. solve $v' = -\omega u$ by a Forward Euler step in $[t_n, t_{n+\frac{1}{2}}]$
2. solve $u' = v$ by a Backward Euler step in $[t_n, t_{n+\frac{1}{2}}]$
3. solve $u' = v$ by a Forward Euler step in $[t_{n+\frac{1}{2}}, t_{n+1}]$
4. solve $v' = -\omega u$ by a Backward Euler step in $[t_{n+\frac{1}{2}}, t_{n+1}]$

With mathematics,

$$\frac{v^{n+\frac{1}{2}} - v^n}{\frac{1}{2}\Delta t} = -\omega^2 u^n,$$
$$\frac{u^{n+\frac{1}{2}} - u^n}{\frac{1}{2}\Delta t} = v^{n+\frac{1}{2}},$$
$$\frac{u^{n+1} - u^{n+\frac{1}{2}}}{\frac{1}{2}\Delta t} = v^{n+\frac{1}{2}},$$
$$\frac{v^{n+1} - v^{n+\frac{1}{2}}}{\frac{1}{2}\Delta t} = -\omega^2 u^{n+1}\,.$$

The two steps in the middle can be combined to

$$\frac{u^{n+1} - u^n}{\Delta t} = v^{n+\frac{1}{2}},$$

and consequently

$$v^{n+\frac{1}{2}} = v^n - \frac{1}{2}\Delta t\omega^2 u^n, \tag{1.58}$$
$$u^{n+1} = u^n + \Delta t v^{n+\frac{1}{2}}, \tag{1.59}$$
$$v^{n+1} = v^{n+\frac{1}{2}} - \frac{1}{2}\Delta t\omega^2 u^{n+1}\,. \tag{1.60}$$

Writing the last equation as $v^n = v^{n-\frac{1}{2}} - \frac{1}{2}\Delta t\omega^2 u^n$ and using this v^n in the first equation gives $v^{n+\frac{1}{2}} = v^{n-\frac{1}{2}} - \Delta t\omega^2 u^n$, and the scheme can be written as two steps:

$$v^{n+\frac{1}{2}} = v^{n-\frac{1}{2}} - \Delta t\omega^2 u^n, \tag{1.61}$$
$$u^{n+1} = u^n + \Delta t v^{n+\frac{1}{2}}, \tag{1.62}$$

which is nothing but straightforward centered differences for the 2×2 ODE system on a *staggered mesh*, see Section 1.8.1. We have thus seen that four different reasonings (discretizing $u'' + \omega^2 u$ directly, using Euler-Cromer, using Stömer-Verlet, and using centered differences for the 2×2 system on a staggered mesh) all end up with the same equations! The main difference is that the traditional Euler-Cromer displays first-order

convergence in Δt (due to less symmetry in the way u and v are treated) while the others are $\mathcal{O}(\Delta t^2)$ schemes.

The most numerically stable scheme, with respect to accumulation of rounding errors, is (1.61)-(1.62). It has, according to [6], better properties in this regard than the direct scheme for the second-order ODE.

1.8 Staggered mesh

A more intuitive discretization than the Euler-Cromer method, yet equivalent, employs solely centered differences in a natural way for the 2×2 first-order ODE system. The scheme is in fact fully equivalent to the second-order scheme for $u'' + \omega u = 0$, also for the first time step. Such a scheme needs to operate on a *staggered mesh* in time. Staggered meshes are very popular in many physical application, maybe foremost fluid dynamics and electromagnetics, so the topic is important to learn.

1.8.1 The Euler-Cromer scheme on a staggered mesh

In a staggered mesh, the unknowns are sought at different points in the mesh. Specifically, u is sought at integer time points t_n and v is sought at $t_{n+1/2}$ *between* two u points. The unknowns are then $u^1, v^{3/2}, u^2, v^{5/2}$, and so on. We typically use the notation u^n and $v^{n+\frac{1}{2}}$ for the two unknown mesh functions. Figure 1.12 presents a graphical sketch of two mesh functions u and v on a staggered mesh.

On a staggered mesh it is natural to use centered difference approximations, expressed in operator notation as

$$[D_t u = v]^{n+\frac{1}{2}}, \tag{1.63}$$

$$[D_t v = -\omega^2 u]^{n+1}\,. \tag{1.64}$$

or if we switch the sequence of the equations:

$$[D_t v = -\omega^2 u]^{n}, \tag{1.65}$$

$$[D_t u = v]^{n+\frac{1}{2}}\,. \tag{1.66}$$

Writing out the formulas gives

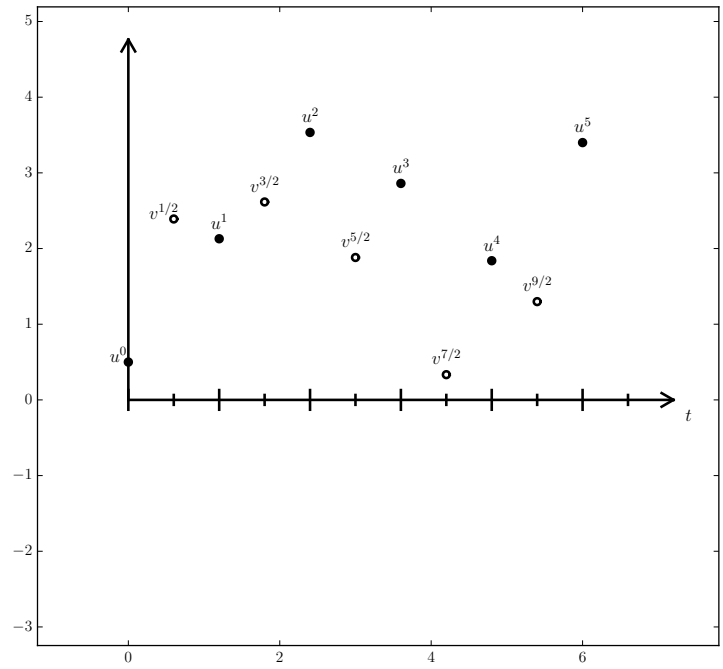


Fig. 1.12 Examples on mesh functions on a staggered mesh in time.

$$v^{n+\frac{1}{2}} = v^{n-\frac{1}{2}} - \Delta t\omega^2 u^n, \tag{1.67}$$

$$u^{n+1} = u^n + \Delta t v^{n+\frac{1}{2}}\,. \tag{1.68}$$

We can eliminate the v values and get back the centered scheme based on the second-order differential equation $u'' + \omega^2 u = 0$, so all these three schemes are equivalent. However, they differ somewhat in the treatment of the initial conditions.

Suppose we have $u(0) = I$ and $u'(0) = v(0) = 0$ as mathematical initial conditions. This means $u^0 = I$ and

$$v(0) \approx \frac{1}{2}(v^{-\frac{1}{2}} + v^{\frac{1}{2}}) = 0, \quad \Rightarrow \quad v^{-\frac{1}{2}} = -v^{\frac{1}{2}}\,.$$

Using the discretized equation (1.67) for $n = 0$ yields

$$v^{\frac{1}{2}} = v^{-\frac{1}{2}} - \Delta t\omega^2 I,$$

and eliminating $v^{-\frac{1}{2}} = -v^{\frac{1}{2}}$ results in

$$v^{\frac{1}{2}} = -\frac{1}{2}\Delta t\omega^2 I,$$

and

$$u^1 = u^0 - \frac{1}{2}\Delta t^2\omega^2 I,$$

which is exactly the same equation for u^1 as we had in the centered scheme based on the second-order differential equation (and hence corresponds to a centered difference approximation of the initial condition for $u'(0)$). The conclusion is that a staggered mesh is fully equivalent with that scheme, while the forward-backward version gives a slight deviation in the computation of u^1.

We can redo the derivation of the initial conditions when $u'(0) = V$:

$$v(0) \approx \frac{1}{2}(v^{-\frac{1}{2}} + v^{\frac{1}{2}}) = V, \quad \Rightarrow \quad v^{-\frac{1}{2}} = 2V - v^{\frac{1}{2}}\,.$$

Using this $v^{-\frac{1}{2}}$ in

$$v^{\frac{1}{2}} = v^{-\frac{1}{2}} - \Delta t\omega^2 I,$$

then gives $v^{\frac{1}{2}} = V - \frac{1}{2}\Delta t\omega^2 I$. The general initial conditions are therefore

$$u^0 = I, \tag{1.69}$$

$$v^{\frac{1}{2}} = V - \frac{1}{2}\Delta t\omega^2 I\,. \tag{1.70}$$

1.8.2 Implementation of the scheme on a staggered mesh

The algorithm goes like this:

1. Set the initial values (1.69) and (1.70).
2. For $n = 1, 2, \ldots$:
   a. Compute u^n from (1.68).
   b. Compute $v^{n+\frac{1}{2}}$ from (1.67).

Implementation with integer indices. Translating the schemes (1.68) and (1.67) to computer code faces the problem of how to store and access $v^{n+\frac{1}{2}}$, since arrays only allow integer indices with base 0. We must then

introduce a convention: $v^{1+\frac{1}{2}}$ is stored in `v[n]` while $v^{1-\frac{1}{2}}$ is stored in `v[n-1]`. We can then write the algorithm in Python as

```
def solver(I, w, dt, T):
    dt = float(dt)
    Nt = int(round(T/dt))
    u = zeros(Nt+1)
    v = zeros(Nt+1)
    t = linspace(0, Nt*dt, Nt+1)  # mesh for u
    t_v = t + dt/2                # mesh for v

    u[0] = I
    v[0] = 0 - 0.5*dt*w**2*u[0]
    for n in range(1, Nt+1):
        u[n] = u[n-1] + dt*v[n-1]
        v[n] = v[n-1] - dt*w**2*u[n]
    return u, t, v, t_v
```

Note that u and v are returned together with the mesh points such that the complete mesh function for u is described by `u` and `t`, while `v` and `t_v` represent the mesh function for v.

Implementation with half-integer indices. Some prefer to see a closer relationship between the code and the mathematics for the quantities with half-integer indices. For example, we would like to replace the updating equation for `v[n]` by

```
v[n+half] = v[n-half] - dt*w**2*u[n]
```

This is easy to do if we could be sure that `n+half` means `n` and `n-half` means `n-1`. A possible solution is to define `half` as a special object such that an integer plus `half` results in the integer, while an integer minus `half` equals the integer minus 1. A simple Python class may realize the `half` object:

```
class HalfInt:
    def __radd__(self, other):
        return other

    def __rsub__(self, other):
        return other - 1

half = HalfInt()
```

The `__radd__` function is invoked for all expressions `n+half` ("right add" with `self` as `half` and `other` as `n`). Similarly, the `__rsub__` function is invoked for `n-half` and results in `n-1`.

Using the `half` object, we can implement the algorithms in an even more readable way:

```
def solver(I, w, dt, T):
    """
    Solve u'=v, v' = - w**2*u for t in (0,T], u(0)=I and v(0)=0,
    by a central finite difference method with time step dt on
    a staggered mesh with v as unknown at (i+1/2)*dt time points.
    """
    dt = float(dt)
    Nt = int(round(T/dt))
    u = zeros(Nt+1)
    v = zeros(Nt+1)
    t = linspace(0, Nt*dt, Nt+1)  # mesh for u
    t_v = t + dt/2                # mesh for v

    u[0] = I
    v[0+half] = 0 - 0.5*dt*w**2*u[0]
    for n in range(1, Nt+1):
        u[n] = u[n-1] + dt*v[n-half]
        v[n+half] = v[n-half] - dt*w**2*u[n]
    return u, t, v[:-1], t_v[:-1]
```

Verification of this code is easy as we can just compare the computed `u` with the `u` produced by the `solver` function in `vib_undamped.py` (which solves $u'' + \omega^2 u = 0$ directly). The values should coincide to machine precision since the two numerical methods are mathematically equivalent. We refer to the file `vib_undamped_staggered.py` for the details of a unit test (`test_staggered`) that checks this property.

1.9 Exercises and Problems

Problem 1.1: Use linear/quadratic functions for verification

Consider the ODE problem

$$u'' + \omega^2 u = f(t), \quad u(0) = I,\ u'(0) = V,\ t \in (0, T]\,.$$

a) Discretize this equation according to $[D_t D_t u + \omega^2 u = f]^n$ and derive the equation for the first time step (u^1).

b) For verification purposes, we use the method of manufactured solutions (MMS) with the choice of $u_{\mathrm{e}}(t) = ct + d$. Find restrictions on c and d from the initial conditions. Compute the corresponding source term f. Show that $[D_t D_t t]^n = 0$ and use the fact that the $D_t D_t$ operator is linear, $[D_t D_t (ct + d)]^n = c[D_t D_t t]^n + [D_t D_t d]^n = 0$, to show that u_{e} is also a perfect solution of the discrete equations.

c) Use `sympy` to do the symbolic calculations above. Here is a sketch of the program `vib_undamped_verify_mms.py`:

```
import sympy as sym
V, t, I, w, dt = sym.symbols('V t I w dt')  # global symbols
f = None  # global variable for the source term in the ODE

def ode_source_term(u):
    """Return the terms in the ODE that the source term
    must balance, here u'' + w**2*u.
    u is symbolic Python function of t."""
    return sym.diff(u(t), t, t) + w**2*u(t)

def residual_discrete_eq(u):
    """Return the residual of the discrete eq. with u inserted."""
    R = ...
    return sym.simplify(R)

def residual_discrete_eq_step1(u):
    """Return the residual of the discrete eq. at the first
    step with u inserted."""
    R = ...
    return sym.simplify(R)

def DtDt(u, dt):
    """Return 2nd-order finite difference for u_tt.
    u is a symbolic Python function of t.
    """
    return ...

def main(u):
    """
    Given some chosen solution u (as a function of t, implemented
    as a Python function), use the method of manufactured solutions
    to compute the source term f, and check if u also solves
    the discrete equations.
    """
    print '=== Testing exact solution: %s ===' % u
    print "Initial conditions u(0)=%s, u'(0)=%s:" % \
          (u(t).subs(t, 0), sym.diff(u(t), t).subs(t, 0))

    # Method of manufactured solution requires fitting f
    global f  # source term in the ODE
    f = sym.simplify(ode_lhs(u))

    # Residual in discrete equations (should be 0)
    print 'residual step1:', residual_discrete_eq_step1(u)
    print 'residual:', residual_discrete_eq(u)

def linear():
    main(lambda t: V*t + I)

if __name__ == '__main__':
```

```
    linear()
```

Fill in the various functions such that the calls in the `main` function works.

d) The purpose now is to choose a quadratic function $u_{\mathrm{e}} = bt^2 + ct + d$ as exact solution. Extend the `sympy` code above with a function `quadratic` for fitting `f` and checking if the discrete equations are fulfilled. (The function is very similar to `linear`.)

e) Will a polynomial of degree three fulfill the discrete equations?

f) Implement a `solver` function for computing the numerical solution of this problem.

g) Write a test function for checking that the quadratic solution is computed correctly (to machine precision, but the round-off errors accumulate and increase with T) by the `solver` function.
Filename: `vib_undamped_verify_mms`.

Exercise 1.2: Show linear growth of the phase with time

Consider an exact solution $I\cos(\omega t)$ and an approximation $I\cos(\tilde{\omega}t)$. Define the phase error as the time lag between the peak I in the exact solution and the corresponding peak in the approximation after m periods of oscillations. Show that this phase error is linear in m.
Filename: `vib_phase_error_growth`.

Exercise 1.3: Improve the accuracy by adjusting the frequency

According to (1.19), the numerical frequency deviates from the exact frequency by a (dominating) amount $\omega^3\Delta t^2/24 > 0$. Replace the `w` parameter in the algorithm in the `solver` function in `vib_undamped.py` by `w*(1 - (1./24)*w**2*dt**2` and test how this adjustment in the numerical algorithm improves the accuracy (use $\Delta t = 0.1$ and simulate for 80 periods, with and without adjustment of ω).
Filename: `vib_adjust_w`.

Exercise 1.4: See if adaptive methods improve the phase error

Adaptive methods for solving ODEs aim at adjusting Δt such that the error is within a user-prescribed tolerance. Implement the equation $u'' + u = 0$ in the Odespy[15] software. Use the example from Section 3.2.11 in [9]. Run the scheme with a very low tolerance (say 10^{-14}) and for a long time, check the number of time points in the solver's mesh (`len(solver.t_all)`), and compare the phase error with that produced by the simple finite difference method from Section 1.1.2 with the same number of (equally spaced) mesh points. The question is whether it pays off to use an adaptive solver or if equally many points with a simple method gives about the same accuracy.
Filename: `vib_undamped_adaptive`.

Exercise 1.5: Use a Taylor polynomial to compute u^1

As an alternative to computing u^1 by (1.8), one can use a Taylor polynomial with three terms:

$$u(t_1) \approx u(0) + u'(0)\Delta t + \frac{1}{2}u''(0)\Delta t^2$$

With $u'' = -\omega^2 u$ and $u'(0) = 0$, show that this method also leads to (1.8). Generalize the condition on $u'(0)$ to be $u'(0) = V$ and compute u^1 in this case with both methods.
Filename: `vib_first_step`.

Problem 1.6: Derive and investigate the velocity Verlet method

The velocity Verlet method for $u'' + \omega^2 u = 0$ is based on the following ideas:

1. step u forward from t_n to t_{n+1} using a three-term Taylor series,
2. replace u'' by $-\omega^2 u$
3. discretize $v' = -\omega^2 u$ by a Crank-Nicolson method.

Derive the scheme, implement it, and determine empirically the convergence rate.

[15] `https://github.com/hplgit/odespy`

Problem 1.7: Find the minimal resolution of an oscillatory function

Sketch the function on a given mesh which has the highest possible frequency. That is, this oscillatory "cos-like" function has its maxima and minima at every two grid points. Find an expression for the frequency of this function, and use the result to find the largest relevant value of $\omega\Delta t$ when ω is the frequency of an oscillating function and Δt is the mesh spacing.
Filename: `vib_largest_wdt`.

Exercise 1.8: Visualize the accuracy of finite differences for a cosine function

We introduce the error fraction

$$E = \frac{[D_t D_t u]^n}{u''(t_n)}$$

to measure the error in the finite difference approximation $D_t D_t u$ to u''. Compute E for the specific choice of a cosine/sine function of the form $u = \exp(i\omega t)$ and show that

$$E = \left(\frac{2}{\omega\Delta t}\right)^2 \sin^2(\frac{\omega\Delta t}{2}) .$$

Plot E as a function of $p = \omega\Delta t$. The relevant values of p are $[0, \pi]$ (see Exercise 1.7 for why $p > \pi$ does not make sense). The deviation of the curve from unity visualizes the error in the approximation. Also expand E as a Taylor polynomial in p up to fourth degree (use, e.g., `sympy`).
Filename: `vib_plot_fd_exp_error`.

Exercise 1.9: Verify convergence rates of the error in energy

We consider the ODE problem $u'' + \omega^2 u = 0$, $u(0) = I$, $u'(0) = V$, for $t \in (0, T]$. The total energy of the solution $E(t) = \frac{1}{2}(u')^2 + \frac{1}{2}\omega^2 u^2$ should stay constant. The error in energy can be computed as explained in Section 1.6.

Make a test function in a separate file, where code from `vib_undamped.py` is imported, but the `convergence_rates` and `test_convergence_rates` functions are copied and modified to also

incorporate computations of the error in energy and the convergence rate of this error. The expected rate is 2, just as for the solution itself.
Filename: `test_error_conv`.

Exercise 1.10: Use linear/quadratic functions for verification

This exercise is a generalization of Problem 1.1 to the extended model problem (1.71) where the damping term is either linear or quadratic. Solve the various subproblems and see how the results and problem settings change with the generalized ODE in case of linear or quadratic damping. By modifying the code from Problem 1.1, `sympy` will do most of the work required to analyze the generalized problem.
Filename: `vib_verify_mms`.

Exercise 1.11: Use an exact discrete solution for verification

Write a test function in a separate file that employs the exact discrete solution (1.20) to verify the implementation of the `solver` function in the file `vib_undamped.py`.
Filename: `test_vib_undamped_exact_discrete_sol`.

Exercise 1.12: Use analytical solution for convergence rate tests

The purpose of this exercise is to perform convergence tests of the problem (1.71) when $s(u) = cu$, $F(t) = A\sin\phi t$ and there is no damping. Find the complete analytical solution to the problem in this case (most textbooks on mechanics or ordinary differential equations list the various elements you need to write down the exact solution, or you can use symbolic tools like `sympy` or `wolframalpha.com`). Modify the `convergence_rate` function from the `vib_undamped.py` program to perform experiments with the extended model. Verify that the error is of order Δt^2.
Filename: `vib_conv_rate`.

Exercise 1.13: Investigate the amplitude errors of many solvers

Use the program `vib_undamped_odespy.py` from Section 1.5.4 (utilize the function `amplitudes`) to investigate how well famous methods for 1st-order ODEs can preserve the amplitude of u in undamped oscillations. Test, for example, the 3rd- and 4th-order Runge-Kutta methods (`RK3`, `RK4`), the Crank-Nicolson method (`CrankNicolson`), the 2nd- and 3rd-order Adams-Bashforth methods (`AdamsBashforth2`, `AdamsBashforth3`), and a 2nd-order Backwards scheme (`Backward2Step`). The relevant governing equations are listed in the beginning of Section 1.5.

Running the code, we get the plots seen in Figure 1.13, 1.14, and 1.15. They show that `RK4` is superior to the others, but that also `CrankNicolson` performs well. In fact, with `RK4` the amplitude changes by less than 0.1 per cent over the interval.

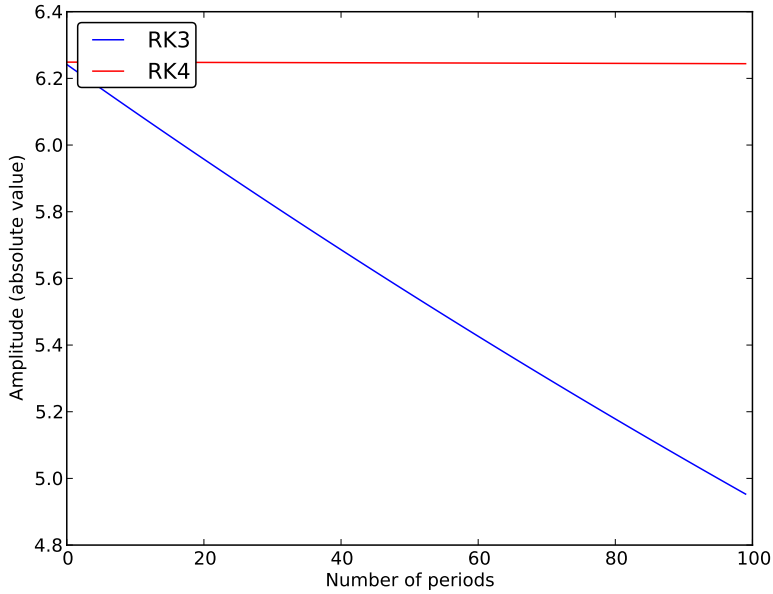


Fig. 1.13 The amplitude as it changes over 100 periods for RK3 and RK4.

Filename: `vib_amplitude_errors`.

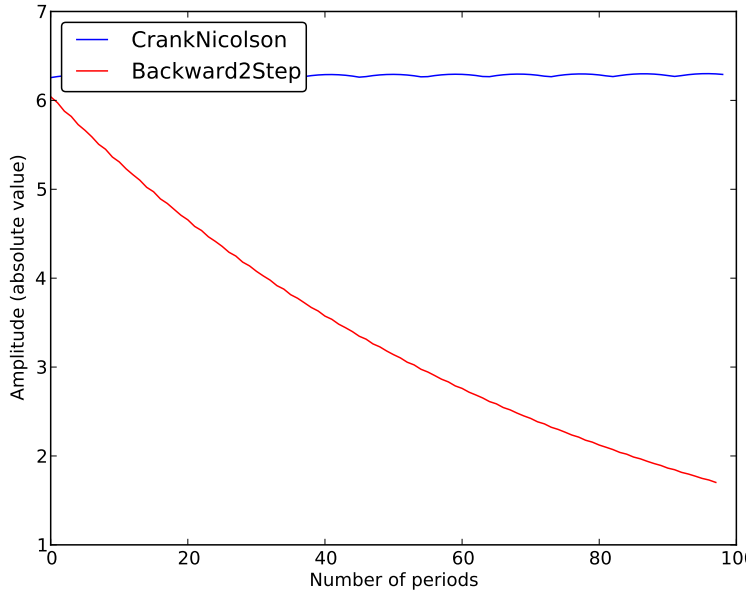


Fig. 1.14 The amplitude as it changes over 100 periods for Crank-Nicolson and Backward 2 step.

Problem 1.14: Minimize memory usage of a simple vibration solver

We consider the model problem $u'' + \omega^2 u = 0$, $u(0) = I$, $u'(0) = V$, solved by a second-order finite difference scheme. A standard implementation typically employs an array `u` for storing all the u^n values. However, at some time level `n+1` where we want to compute `u[n+1]`, all we need of previous `u` values are from level `n` and `n-1`. We can therefore avoid storing the entire array `u`, and instead work with `u[n+1]`, `u[n]`, and `u[n-1]`, named as `u`, `u_n`, `u_nmp1`, for instance. Another possible naming convention is `u`, `u_n[0]`, `u_n[-1]`. Store the solution in a file for later visualization. Make a test function that verifies the implementation by comparing with the another code for the same problem.
Filename: `vib_memsave0`.

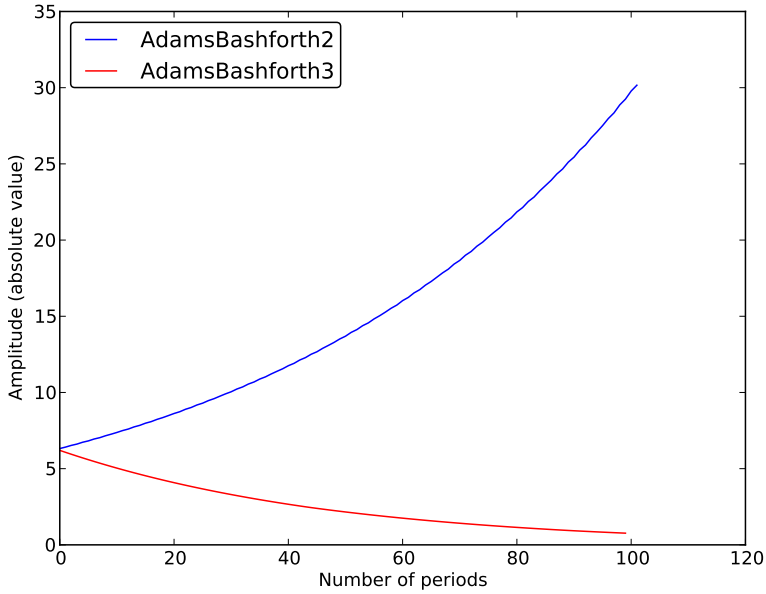


Fig. 1.15 The amplitude as it changes over 100 periods for Adams-Bashforth 2 and 3.

Problem 1.15: Minimize memory usage of a general vibration solver

The program `vib.py` stores the complete solution $u^0, u^1, \ldots, u^{N_t}$ in memory, which is convenient for later plotting. Make a memory minimizing version of this program where only the last three u^{n+1}, u^n, and u^{n-1} values are stored in memory under the names `u`, `u_n`, and `u_nm1` (this is the naming convention used in this book). Write each computed (t_{n+1}, u^{n+1}) pair to file. Visualize the data in the file (a cool solution is to read one line at a time and plot the u value using the line-by-line plotter in the `visualize_front_ascii` function - this technique makes it trivial to visualize very long time simulations).
Filename: `vib_memsave`.

Exercise 1.16: Implement the Euler-Cromer scheme for the generalized model

We consider the generalized model problem

$$mu'' + f(u') + s(u) = F(t), \quad u(0) = I,\ u'(0) = V\,.$$

a) Implement the Euler-Cromer method from Section 1.10.8.

b) We expect the Euler-Cromer method to have first-order convergence rate. Make a unit test based on this expectation.

c) Consider a system with $m = 4$, $f(v) = b|v|v$, $b = 0.2$, $s = 2u$, $F = 0$. Compute the solution using the centered difference scheme from Section 1.10.1 and the Euler-Cromer scheme for the longest possible time step Δt. We can use the result from the case without damping, i.e., the largest $\Delta t = 2/\omega$, $\omega \approx \sqrt{0.5}$ in this case, but since b will modify the frequency, we take the longest possible time step as a safety factor 0.9 times $2/\omega$. Refine Δt three times by a factor of two and compare the two curves.
Filename: `vib_EulerCromer`.

Problem 1.17: Interpret $[D_tD_tu]^n$ as a forward-backward difference

Show that the difference $[D_tD_tu]^n$ is equal to $[D_t^+D_t^-u]^n$ and $D_t^-D_t^+u]^n$. That is, instead of applying a centered difference twice one can alternatively apply a mixture of forward and backward differences.
Filename: `vib_DtDt_fw_bw`.

Exercise 1.18: Analysis of the Euler-Cromer scheme

The Euler-Cromer scheme for the model problem $u'' + \omega^2 u = 0$, $u(0) = I$, $u'(0) = 0$, is given in (1.55)-(1.54). Find the exact discrete solutions of this scheme and show that the solution for u^n coincides with that found in Section 1.4.

Hint. Use an "ansatz" $u^n = I\exp(i\tilde{\omega}\Delta t\, n)$ and $v^n = qu^n$, where $\tilde{\omega}$ and q are unknown parameters. The following formula is handy:

$$e^{i\tilde{\omega}\Delta t} + e^{i\tilde{\omega}(-\Delta t)} - 2 = 2\left(\cosh(i\tilde{\omega}\Delta t) - 1\right) = -4\sin^2(\frac{\tilde{\omega}\Delta t}{2})\,.$$

1.10 Generalization: damping, nonlinearities, and excitation

We shall now generalize the simple model problem from Section 1.1 to include a possibly nonlinear damping term $f(u')$, a possibly nonlinear spring (or restoring) force $s(u)$, and some external excitation $F(t)$:

$$mu'' + f(u') + s(u) = F(t), \quad u(0) = I,\ u'(0) = V,\ t \in (0, T]\,. \quad (1.71)$$

We have also included a possibly nonzero initial value for $u'(0)$. The parameters m, $f(u')$, $s(u)$, $F(t)$, I, V, and T are input data.

There are two main types of damping (friction) forces: linear $f(u') = bu$, or quadratic $f(u') = bu'|u'|$. Spring systems often feature linear damping, while air resistance usually gives rise to quadratic damping. Spring forces are often linear: $s(u) = cu$, but nonlinear versions are also common, the most famous is the gravity force on a pendulum that acts as a spring with $s(u) \sim \sin(u)$.

1.10.1 A centered scheme for linear damping

Sampling (1.71) at a mesh point t_n, replacing $u''(t_n)$ by $[D_tD_tu]^n$, and $u'(t_n)$ by $[D_{2t}u]^n$ results in the discretization

$$[mD_tD_tu + f(D_{2t}u) + s(u) = F]^n, \quad (1.72)$$

which written out means

$$m\frac{u^{n+1} - 2u^n + u^{n-1}}{\Delta t^2} + f(\frac{u^{n+1} - u^{n-1}}{2\Delta t}) + s(u^n) = F^n, \quad (1.73)$$

where F^n as usual means $F(t)$ evaluated at $t = t_n$. Solving (1.73) with respect to the unknown u^{n+1} gives a problem: the u^{n+1} inside the f function makes the equation *nonlinear* unless $f(u')$ is a linear function, $f(u') = bu'$. For now we shall assume that f is linear in u'. Then

$$m\frac{u^{n+1} - 2u^n + u^{n-1}}{\Delta t^2} + b\frac{u^{n+1} - u^{n-1}}{2\Delta t} + s(u^n) = F^n, \quad (1.74)$$

which gives an explicit formula for u at each new time level:

$$u^{n+1} = (2mu^n + (\frac{b}{2}\Delta t - m)u^{n-1} + \Delta t^2(F^n - s(u^n)))(m + \frac{b}{2}\Delta t)^{-1}\,. \quad (1.75)$$

For the first time step we need to discretize $u'(0) = V$ as $[D_{2t}u = V]^0$ and combine with (1.75) for $n = 0$. The discretized initial condition leads to

$$u^{-1} = u^1 - 2\Delta t V, \quad (1.76)$$

which inserted in (1.75) for $n = 0$ gives an equation that can be solved for u^1:

$$u^1 = u^0 + \Delta t\, V + \frac{\Delta t^2}{2m}(-bV - s(u^0) + F^0)\,. \quad (1.77)$$

1.10.2 A centered scheme for quadratic damping

When $f(u') = bu'|u'|$, we get a quadratic equation for u^{n+1} in (1.73). This equation can be straightforwardly solved by the well-known formula for the roots of a quadratic equation. However, we can also avoid the nonlinearity by introducing an approximation with an error of order no higher than what we already have from replacing derivatives with finite differences.

We start with (1.71) and only replace u'' by D_tD_tu, resulting in

$$[mD_tD_tu + bu'|u'| + s(u) = F]^n\,. \quad (1.78)$$

Here, $u'|u'|$ is to be computed at time t_n. The idea is now to introduce a *geometric mean*, defined by

$$(w^2)^n \approx w^{n-\frac{1}{2}}w^{n+\frac{1}{2}},$$

for some quantity w depending on time. The error in the geometric mean approximation is $\mathcal{O}(\Delta t^2)$, the same as in the approximation $u'' \approx D_tD_tu$. With $w = u'$ it follows that

$$[u'|u'|]^n \approx u'(t_{n+\frac{1}{2}})|u'(t_{n-\frac{1}{2}})|\,.$$

The next step is to approximate u' at $t_{n\pm 1/2}$, and fortunately a centered difference fits perfectly into the formulas since it involves u values at the mesh points only. With the approximations

$$u'(t_{n+1/2}) \approx [D_t u]^{n+\frac{1}{2}}, \quad u'(t_{n-1/2}) \approx [D_t u]^{n-\frac{1}{2}}, \tag{1.79}$$

we get

$$[u'|u'|]^n \approx [D_t u]^{n+\frac{1}{2}} |[D_t u]^{n-\frac{1}{2}}| = \frac{u^{n+1} - u^n}{\Delta t} \frac{|u^n - u^{n-1}|}{\Delta t}. \tag{1.80}$$

The counterpart to (1.73) is then

$$m \frac{u^{n+1} - 2u^n + u^{n-1}}{\Delta t^2} + b \frac{u^{n+1} - u^n}{\Delta t} \frac{|u^n - u^{n-1}|}{\Delta t} + s(u^n) = F^n, \tag{1.81}$$

which is linear in the unknown u^{n+1}. Therefore, we can easily solve (1.81) with respect to u^{n+1} and achieve the explicit updating formula

$$u^{n+1} = \Big(m + b|u^n - u^{n-1}|\Big)^{-1} \times \Big(2mu^n - mu^{n-1} + bu^n|u^n - u^{n-1}| + \Delta t^2 (F^n - s(u^n))\Big). \tag{1.82}$$

In the derivation of a special equation for the first time step we run into some trouble: inserting (1.76) in (1.82) for $n = 0$ results in a complicated nonlinear equation for u^1. By thinking differently about the problem we can easily get away with the nonlinearity again. We have for $n = 0$ that $b[u'|u'|]^0 = bV|V|$. Using this value in (1.78) gives

$$[mD_t D_t u + bV|V| + s(u) = F]^0. \tag{1.83}$$

Writing this equation out and using (1.76) results in the special equation for the first time step:

$$u^1 = u^0 + \Delta t V + \frac{\Delta t^2}{2m} \Big(-bV|V| - s(u^0) + F^0\Big). \tag{1.84}$$

1.10.3 A forward-backward discretization of the quadratic damping term

The previous section first proposed to discretize the quadratic damping term $|u'|u'$ using centered differences: $[|D_{2t}|D_{2t}u]^n$. As this gives rise to a nonlinearity in u^{n+1}, it was instead proposed to use a geometric mean

combined with centered differences. But there are other alternatives. To get rid of the nonlinearity in $[|D_{2t}|D_{2t}u]^n$, one can think differently: apply a backward difference to $|u'|$, such that the term involves known values, and apply a forward difference to u' to make the term linear in the unknown u^{n+1}. With mathematics,

$$[\beta|u'|u']^n \approx \beta|[D_t^- u]^n|[D_t^+ u]^n = \beta\left|\frac{u^n - u^{n-1}}{\Delta t}\right|\frac{u^{n+1} - u^n}{\Delta t}\,. \tag{1.85}$$

The forward and backward differences both have an error proportional to Δt so one may think the discretization above leads to a first-order scheme. However, by looking at the formulas, we realize that the forward-backward differences in (1.85) result in exactly the same scheme as in (1.81) where we used a geometric mean and centered differences and committed errors of size $\mathcal{O}(\Delta t^2)$. Therefore, the forward-backward differences in (1.85) act in a symmetric way and actually produce a second-order accurate discretization of the quadratic damping term.

1.10.4 Implementation

The algorithm arising from the methods in Sections 1.10.1 and 1.10.2 is very similar to the undamped case in Section 1.1.2. The difference is basically a question of different formulas for u^1 and u^{n+1}. This is actually quite remarkable. The equation (1.71) is normally impossible to solve by pen and paper, but possible for some special choices of F, s, and f. On the contrary, the complexity of the nonlinear generalized model (1.71) versus the simple undamped model is not a big deal when we solve the problem numerically!

The computational algorithm takes the form

1. $u^0 = I$
2. compute u^1 from (1.77) if linear damping or (1.84) if quadratic damping
3. for $n = 1, 2, \ldots, N_t - 1$:
   a. compute u^{n+1} from (1.75) if linear damping or (1.82) if quadratic damping

Modifying the `solver` function for the undamped case is fairly easy, the big difference being many more terms and if tests on the type of damping:

```
def solver(I, V, m, b, s, F, dt, T, damping='linear'):
    """
    Solve m*u'' + f(u') + s(u) = F(t) for t in (0,T],
    u(0)=I and u'(0)=V,
    by a central finite difference method with time step dt.
    If damping is 'linear', f(u')=b*u, while if damping is
    'quadratic', f(u')=b*u'*abs(u').
    F(t) and s(u) are Python functions.
    """
    dt = float(dt); b = float(b); m = float(m) # avoid integer div.
    Nt = int(round(T/dt))
    u = np.zeros(Nt+1)
    t = np.linspace(0, Nt*dt, Nt+1)

    u[0] = I
    if damping == 'linear':
        u[1] = u[0] + dt*V + dt**2/(2*m)*(-b*V - s(u[0]) + F(t[0]))
    elif damping == 'quadratic':
        u[1] = u[0] + dt*V + \
               dt**2/(2*m)*(-b*V*abs(V) - s(u[0]) + F(t[0]))

    for n in range(1, Nt):
        if damping == 'linear':
            u[n+1] = (2*m*u[n] + (b*dt/2 - m)*u[n-1] +
                      dt**2*(F(t[n]) - s(u[n])))/(m + b*dt/2)
        elif damping == 'quadratic':
            u[n+1] = (2*m*u[n] - m*u[n-1] + b*u[n]*abs(u[n] - u[n-1])
                      + dt**2*(F(t[n]) - s(u[n])))/\
                      (m + b*abs(u[n] - u[n-1]))
    return u, t
```

The complete code resides in the file `vib.py`.

1.10.5 Verification

Constant solution. For debugging and initial verification, a constant solution is often very useful. We choose $u_{\mathrm{e}}(t) = I$, which implies $V = 0$. Inserted in the ODE, we get $F(t) = s(I)$ for any choice of f. Since the discrete derivative of a constant vanishes (in particular, $[D_{2t}I]^n = 0$, $[D_tI]^n = 0$, and $[D_tD_tI]^n = 0$), the constant solution also fulfills the discrete equations. The constant should therefore be reproduced to machine precision. The function `test_constant` in `vib.py` implements this test.

Linear solution. Now we choose a linear solution: $u_{\mathrm{e}} = ct + d$. The initial condition $u(0) = I$ implies $d = I$, and $u'(0) = V$ forces c to be V. Inserting $u_{\mathrm{e}} = Vt + I$ in the ODE with linear damping results in

$$0 + bV + s(Vt + I) = F(t),$$

while quadratic damping requires the source term

$$0 + b|V|V + s(Vt + I) = F(t)\,.$$

Since the finite difference approximations used to compute u' all are exact for a linear function, it turns out that the linear u_{e} is also a solution of the discrete equations. Exercise 1.10 asks you to carry out all the details.

Quadratic solution. Choosing $u_{\mathrm{e}} = bt^2 + Vt + I$, with b arbitrary, fulfills the initial conditions and fits the ODE if F is adjusted properly. The solution also solves the discrete equations with linear damping. However, this quadratic polynomial in t does not fulfill the discrete equations in case of quadratic damping, because the geometric mean used in the approximation of this term introduces an error. Doing Exercise 1.10 will reveal the details. One can fit F^n in the discrete equations such that the quadratic polynomial is reproduced by the numerical method (to machine precision).

Catching bugs. How good are the constant and quadratic solutions at catching bugs in the implementation? Let us check that by introducing some bugs.

- Use `m` instead of `2*m` in the denominator of `u[1]`: code works for constant solution, but fails (as it should) for a quadratic one.
- Use `b*dt` instead of `b*dt/2` in the updating formula for `u[n+1]` in case of linear damping: constant and quadratic both fail.
- Use `F[n+1]` instead of `F[n]` in case of linear or quadratic damping: constant solution works, quadratic fails.

We realize that the constant solution is very useful for catching certain bugs because of its simplicity (easy to predict what the different terms in the formula should evaluate to), while the quadratic solution seems capable of detecting all (?) other kinds of typos in the scheme. These results demonstrate why we focus so much on exact, simple polynomial solutions of the numerical schemes in these writings.

1.10.6 Visualization

The functions for visualizations differ significantly from those in the undamped case in the `vib_undamped.py` program because, in the present

general case, we do not have an exact solution to include in the plots. Moreover, we have no good estimate of the periods of the oscillations as there will be one period determined by the system parameters, essentially the approximate frequency $\sqrt{s'(0)/m}$ for linear s and small damping, and one period dictated by $F(t)$ in case the excitation is periodic. This is, however, nothing that the program can depend on or make use of. Therefore, the user has to specify T and the window width to get a plot that moves with the graph and shows the most recent parts of it in long time simulations.

The `vib.py` code contains several functions for analyzing the time series signal and for visualizing the solutions.

1.10.7 User interface

The `main` function is changed substantially from the `vib_undamped.py` code, since we need to specify the new data c, $s(u)$, and $F(t)$. In addition, we must set T and the plot window width (instead of the number of periods we want to simulate as in `vib_undamped.py`). To figure out whether we can use one plot for the whole time series or if we should follow the most recent part of u, we can use the `plot_empricial_freq_and_amplitude` function's estimate of the number of local maxima. This number is now returned from the function and used in `main` to decide on the visualization technique.

```
def main():
    import argparse
    parser = argparse.ArgumentParser()
    parser.add_argument('--I', type=float, default=1.0)
    parser.add_argument('--V', type=float, default=0.0)
    parser.add_argument('--m', type=float, default=1.0)
    parser.add_argument('--c', type=float, default=0.0)
    parser.add_argument('--s', type=str, default='u')
    parser.add_argument('--F', type=str, default='0')
    parser.add_argument('--dt', type=float, default=0.05)
    parser.add_argument('--T', type=float, default=140)
    parser.add_argument('--damping', type=str, default='linear')
    parser.add_argument('--window_width', type=float, default=30)
    parser.add_argument('--savefig', action='store_true')
    a = parser.parse_args()
    from scitools.std import StringFunction
    s = StringFunction(a.s, independent_variable='u')
    F = StringFunction(a.F, independent_variable='t')
    I, V, m, c, dt, T, window_width, savefig, damping = \
       a.I, a.V, a.m, a.c, a.dt, a.T, a.window_width, a.savefig, \
       a.damping
```

```
    u, t = solver(I, V, m, c, s, F, dt, T)
    num_periods = empirical_freq_and_amplitude(u, t)
    if num_periods <= 15:
        figure()
        visualize(u, t)
    else:
        visualize_front(u, t, window_width, savefig)
    show()
```

The program `vib.py` contains the above code snippets and can solve the model problem (1.71). As a demo of `vib.py`, we consider the case $I = 1$, $V = 0$, $m = 1$, $c = 0.03$, $s(u) = \sin(u)$, $F(t) = 3\cos(4t)$, $\Delta t = 0.05$, and $T = 140$. The relevant command to run is

Terminal

```
Terminal> python vib.py --s 'sin(u)' --F '3*cos(4*t)' --c 0.03
```

This results in a moving window following the function[16] on the screen. Figure 1.16 shows a part of the time series.

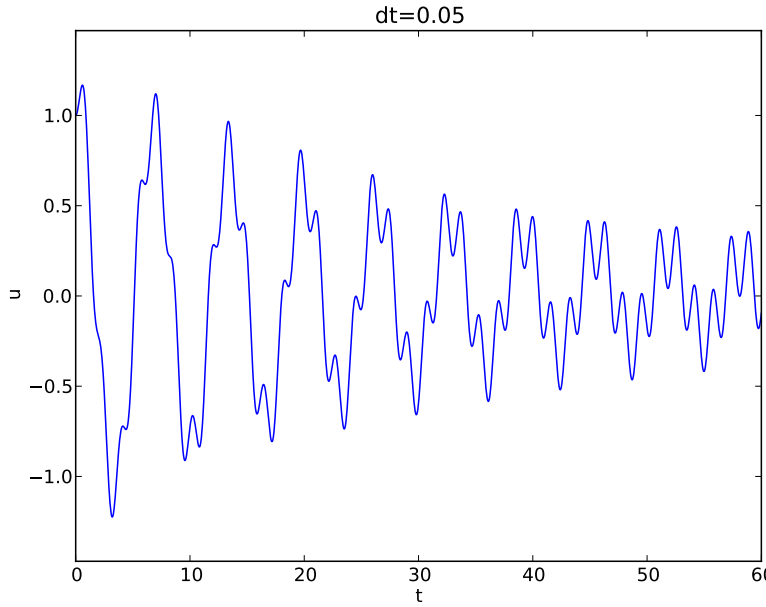


Fig. 1.16 Damped oscillator excited by a sinusoidal function.

[16] `http://tinyurl.com/hbcasmj/vib/html//mov-vib/vib_generalized_dt0.05/index.html`

1.10.8 The Euler-Cromer scheme for the generalized model

The ideas of the Euler-Cromer method from Section 1.7 carry over to the generalized model. We write (1.71) as two equations for u and $v = u'$. The first equation is taken as the one with v' on the left-hand side:

$$v' = \frac{1}{m}(F(t) - s(u) - f(v)), \tag{1.86}$$

$$u' = v\,. \tag{1.87}$$

Again, the idea is to step (1.86) forward using a standard Forward Euler method, while we update u from (1.87) with a Backward Euler method, utilizing the recent, computed v^{n+1} value. In detail,

$$\frac{v^{n+1} - v^n}{\Delta t} = \frac{1}{m}(F(t_n) - s(u^n) - f(v^n)), \tag{1.88}$$

$$\frac{u^{n+1} - u^n}{\Delta t} = v^{n+1}, \tag{1.89}$$

resulting in the explicit scheme

$$v^{n+1} = v^n + \Delta t \frac{1}{m}(F(t_n) - s(u^n) - f(v^n)), \tag{1.90}$$

$$u^{n+1} = u^n + \Delta t\, v^{n+1}\,. \tag{1.91}$$

We immediately note one very favorable feature of this scheme: all the nonlinearities in $s(u)$ and $f(v)$ are evaluated at a previous time level. This makes the Euler-Cromer method easier to apply and hence much more convenient than the centered scheme for the second-order ODE (1.71).

The initial conditions are trivially set as

$$v^0 = V, \tag{1.92}$$

$$u^0 = I\,. \tag{1.93}$$

1.10.9 The Störmer-Verlet algorithm for the generalized model

We can easily apply the ideas from Section 1.7.4 to extend that method to the generalized model

$$v' = \frac{1}{m}(F(t) - s(u) - f(v)),$$
$$u' = v\,.$$

However, since the scheme is essentially centered differences for the ODE system on a staggered mesh, we do not go into detail here, but refer to Section 1.10.10.

1.10.10 A staggered Euler-Cromer scheme for a generalized model

The more general model for vibration problems,

$$mu'' + f(u') + s(u) = F(t), \quad u(0) = I,\ u'(0) = V,\ t \in (0, T], \quad (1.94)$$

can be rewritten as a first-order ODE system

$$v' = m^{-1}\left(F(t) - f(v) - s(u)\right), \quad (1.95)$$
$$u' = v\,. \quad (1.96)$$

It is natural to introduce a staggered mesh (see Section 1.8.1) and seek u at mesh points t_n (the numerical value is denoted by u^n) and v between mesh points at $t_{n+1/2}$ (the numerical value is denoted by $v^{n+\frac{1}{2}}$). A centered difference approximation to (1.96)-(1.95) can then be written in operator notation as

$$[D_t v = m^{-1}\left(F(t) - f(v) - s(u)\right)]^n, \quad (1.97)$$
$$[D_t u = v]^{n+\frac{1}{2}}\,. \quad (1.98)$$

Written out,

$$\frac{v^{n+\frac{1}{2}} - v^{n-\frac{1}{2}}}{\Delta t} = m^{-1}\left(F^n - f(v^n) - s(u^n)\right), \tag{1.99}$$

$$\frac{u^n - u^{n-1}}{\Delta t} = v^{n+\frac{1}{2}}. \tag{1.100}$$

With linear damping, $f(v) = bv$, we can use an arithmetic mean for $f(v^n)$: $f(v^n) \approx= \frac{1}{2}(f(v^{n-\frac{1}{2}}) + f(v^{n+\frac{1}{2}}))$. The system (1.99)-(1.100) can then be solved with respect to the unknowns u^n and $v^{n+\frac{1}{2}}$:

$$v^{n+\frac{1}{2}} = \left(1 + \frac{b}{2m}\Delta t\right)^{-1}\left(v^{n-\frac{1}{2}} + \Delta t m^{-1}\left(F^n - \frac{1}{2}f(v^{n-\frac{1}{2}}) - s(u^n)\right)\right), \tag{1.101}$$

$$u^n = u^{n-1} + \Delta t v^{n-\frac{1}{2}}. \tag{1.102}$$

In case of quadratic damping, $f(v) = b|v|v$, we can use a geometric mean: $f(v^n) \approx b|v^{n-\frac{1}{2}}|v^{n+\frac{1}{2}}$. Inserting this approximation in (1.99)-(1.100) and solving for the unknowns u^n and $v^{n+\frac{1}{2}}$ results in

$$v^{n+\frac{1}{2}} = (1 + \frac{b}{m}|v^{n-\frac{1}{2}}|\Delta t)^{-1}\left(v^{n-\frac{1}{2}} + \Delta t m^{-1}\left(F^n - s(u^n)\right)\right), \tag{1.103}$$

$$u^n = u^{n-1} + \Delta t v^{n-\frac{1}{2}}. \tag{1.104}$$

The initial conditions are derived at the end of Section 1.8.1:

$$u^0 = I, \tag{1.105}$$

$$v^{\frac{1}{2}} = V - \frac{1}{2}\Delta t\omega^2 I. \tag{1.106}$$

1.10.11 The PEFRL 4th-order accurate algorithm

A variant of the Euler-Cromer type of algorithm, which provides an error $\mathcal{O}(\Delta t^4)$ if $f(v) = 0$, is called PEFRL [14]. This algorithm is very well suited for integrating dynamic systems (especially those without damping) over very long time periods. Define

$$g(u, v) = \frac{1}{m}(F(t) - s(u) - f(v)).$$

The algorithm is explicit and features these steps:

$$u^{n+1,1} = u^n + \xi \Delta t v^n, \tag{1.107}$$
$$v^{n+1,1} = v^n + \frac{1}{2}(1-2\lambda)\Delta t g(u^{n+1,1}, v^n), \tag{1.108}$$
$$u^{n+1,2} = u^{n+1,1} + \chi \Delta t v^{n+1,1}, \tag{1.109}$$
$$v^{n+1,2} = v^{n+1,1} + \lambda \Delta t g(u^{n+1,2}, v^{n+1,1}), \tag{1.110}$$
$$u^{n+1,3} = u^{n+1,2} + (1-2(\chi+\xi))\Delta t v^{n+1,2}, \tag{1.111}$$
$$v^{n+1,3} = v^{n+1,2} + \lambda \Delta t g(u^{n+1,3}, v^{n+1,2}), \tag{1.112}$$
$$u^{n+1,4} = u^{n+1,3} + \chi \Delta t v^{n+1,3}, \tag{1.113}$$
$$v^{n+1} = v^{n+1,3} + \frac{1}{2}(1-2\lambda)\Delta t g(u^{n+1,4}, v^{n+1,3}), \tag{1.114}$$
$$u^{n+1} = u^{n+1,4} + \xi \Delta t v^{n+1} \tag{1.115}$$

The parameters ξ, λ, and ξ have the values

$$\xi = 0.1786178958448091, \tag{1.116}$$
$$\lambda = -0.2123418310626054, \tag{1.117}$$
$$\chi = -0.06626458266981849 \tag{1.118}$$

1.11 Exercises and Problems

Exercise 1.19: Implement the solver via classes

Reimplement the `vib.py` program using a class `Problem` to hold all the physical parameters of the problem, a class `Solver` to hold the numerical parameters and compute the solution, and a class `Visualizer` to display the solution.

Hint. Use the ideas and examples from Sections 5.5.1 and 5.5.2 in [9]. More specifically, make a superclass `Problem` for holding the scalar physical parameters of a problem and let subclasses implement the $s(u)$ and $F(t)$ functions as methods. Try to call up as much existing functionality in `vib.py` as possible.
Filename: `vib_class`.

Problem 1.20: Use a backward difference for the damping term

As an alternative to discretizing the damping terms $\beta u'$ and $\beta|u'|u'$ by centered differences, we may apply backward differences:

$$
\begin{aligned}
[u']^n &\approx [D_t^- u]^n, \\
[|u'|u']^n &\approx [|D_t^- u| D_t^- u]^n \\
&= |[D_t^- u]^n| [D_t^- u]^n \, .
\end{aligned}
$$

The advantage of the backward difference is that the damping term is evaluated using known values u^n and u^{n-1} only. Extend the `vib.py` code with a scheme based on using backward differences in the damping terms. Add statements to compare the original approach with centered difference and the new idea launched in this exercise. Perform numerical experiments to investigate how much accuracy that is lost by using the backward differences.
Filename: `vib_gen_bwdamping`.

Exercise 1.21: Use the forward-backward scheme with quadratic damping

We consider the generalized model with quadratic damping, expressed as a system of two first-order equations as in Section 1.10.10:

$$
\begin{aligned}
u' &= v, \\
v' &= \frac{1}{m}\left(F(t) - \beta|v|v - s(u)\right) \, .
\end{aligned}
$$

However, contrary to what is done in Section 1.10.10, we want to apply the idea of a forward-backward discretization: u is marched forward by a one-sided Forward Euler scheme applied to the first equation, and thereafter v can be marched forward by a Backward Euler scheme in the second equation, see in Section 1.7. Express the idea in operator notation and write out the scheme. Unfortunately, the backward difference for the v equation creates a nonlinearity $|v^{n+1}|v^{n+1}$. To linearize this nonlinearity, use the known value v^n inside the absolute value factor, i.e., $|v^{n+1}|v^{n+1} \approx |v^n|v^{n+1}$. Show that the resulting scheme is equivalent to the one in Section 1.10.10 for some time level $n \geq 1$.

What we learn from this exercise is that the first-order differences and the linearization trick play together in "the right way" such that the scheme is as good as when we (in Section 1.10.10) carefully apply centered differences and a geometric mean on a staggered mesh to achieve second-order accuracy. There is a difference in the handling of the initial conditions, though, as explained at the end of Section 1.7. Filename: `vib_gen_bwdamping`.

1.12 Applications of vibration models

The following text derives some of the most well-known physical problems that lead to second-order ODE models of the type addressed in this book. We consider a simple spring-mass system; thereafter extended with nonlinear spring, damping, and external excitation; a spring-mass system with sliding friction; a simple and a physical (classical) pendulum; and an elastic pendulum.

1.12.1 Oscillating mass attached to a spring

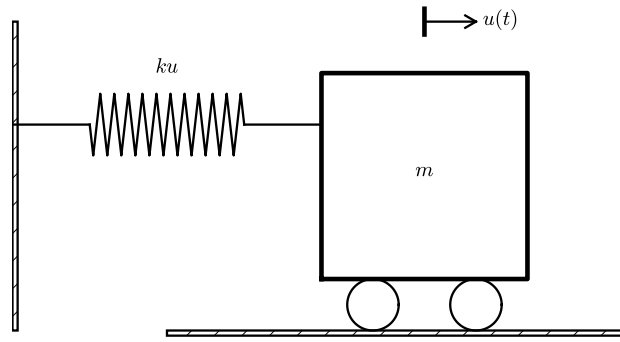


Fig. 1.17 Simple oscillating mass.

The most fundamental mechanical vibration system is depicted in Figure 1.17. A body with mass m is attached to a spring and can move horizontally without friction (in the wheels). The position of the body is given by the vector $\boldsymbol{r}(t) = u(t)\boldsymbol{i}$, where $\boldsymbol{i}$ is a unit vector in x direction. There is only one force acting on the body: a spring force $\boldsymbol{F}_s = -ku\boldsymbol{i}$, where k is a constant. The point $x = 0$, where $u = 0$, must therefore

correspond to the body's position where the spring is neither extended nor compressed, so the force vanishes.

The basic physical principle that governs the motion of the body is Newton's second law of motion: $\boldsymbol{F} = m\boldsymbol{a}$, where $\boldsymbol{F}$ is the sum of forces on the body, m is its mass, and $\boldsymbol{a} = \ddot{\boldsymbol{r}}$ is the acceleration. We use the dot for differentiation with respect to time, which is usual in mechanics. Newton's second law simplifies here to $-\boldsymbol{F}_s = m\ddot{u}\boldsymbol{i}$, which translates to

$$-ku = m\ddot{u}\,.$$

Two initial conditions are needed: $u(0) = I$, $\dot{u}(0) = V$. The ODE problem is normally written as

$$m\ddot{u} + ku = 0, \quad u(0) = I,\ \dot{u}(0) = V\,. \tag{1.119}$$

It is not uncommon to divide by m and introduce the frequency $\omega = \sqrt{k/m}$:

$$\ddot{u} + \omega^2 u = 0, \quad u(0) = I,\ \dot{u}(0) = V\,. \tag{1.120}$$

This is the model problem in the first part of this chapter, with the small difference that we write the time derivative of u with a dot above, while we used u' and u'' in previous parts of the book.

Since only one scalar mathematical quantity, $u(t)$, describes the complete motion, we say that the mechanical system has one degree of freedom (DOF).

Scaling. For numerical simulations it is very convenient to scale (1.120) and thereby get rid of the problem of finding relevant values for all the parameters m, k, I, and V. Since the amplitude of the oscillations are dictated by I and V (or more precisely, V/ω), we scale u by I (or V/ω if $I = 0$):

$$\bar{u} = \frac{u}{I}, \quad \bar{t} = \frac{t}{t_c}\,.$$

The time scale t_c is normally chosen as the inverse period $2\pi/\omega$ or angular frequency $1/\omega$, most often as $t_c = 1/\omega$. Inserting the dimensionless quantities $\bar{u}$ and $\bar{t}$ in (1.120) results in the scaled problem

$$\frac{d^2\bar{u}}{d\bar{t}^2} + \bar{u} = 0, \quad \bar{u}(0) = 1,\ \frac{\bar{u}}{\bar{t}}(0) = \beta = \frac{V}{I\omega},$$

where β is a dimensionless number. Any motion that starts from rest ($V = 0$) is free of parameters in the scaled model!

The physics. The typical physics of the system in Figure 1.17 can be described as follows. Initially, we displace the body to some position I, say at rest ($V = 0$). After releasing the body, the spring, which is extended, will act with a force $-kI\boldsymbol{i}$ and pull the body to the left. This force causes an acceleration and therefore increases velocity. The body passes the point $x = 0$, where $u = 0$, and the spring will then be compressed and act with a force $kx\boldsymbol{i}$ against the motion and cause retardation. At some point, the motion stops and the velocity is zero, before the spring force $kx\boldsymbol{i}$ has worked long enough to push the body in positive direction. The result is that the body accelerates back and forth. As long as there is no friction forces to damp the motion, the oscillations will continue forever.

1.12.2 General mechanical vibrating system

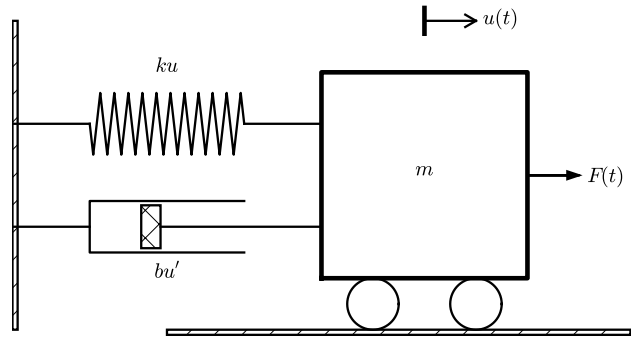


Fig. 1.18 General oscillating system.

The mechanical system in Figure 1.17 can easily be extended to the more general system in Figure 1.18, where the body is attached to a spring and a dashpot, and also subject to an environmental force $F(t)\boldsymbol{i}$. The system has still only one degree of freedom since the body can only move back and forth parallel to the x axis. The spring force was linear, $\boldsymbol{F}_s = -ku\boldsymbol{i}$, in Section 1.12.1, but in more general cases it can depend nonlinearly on the position. We therefore set $\boldsymbol{F}_s = s(u)\boldsymbol{i}$. The dashpot, which acts as a damper, results in a force $\boldsymbol{F}_d$ that depends on the body's velocity $\dot{u}$ and that always acts against the motion. The mathematical model of the force is written $\boldsymbol{F}_d = f(\dot{u})\boldsymbol{i}$. A positive $\dot{u}$ must result in

a force acting in the positive x direction. Finally, we have the external environmental force $\boldsymbol{F}_e = F(t)\boldsymbol{i}$.

Newton's second law of motion now involves three forces:

$$F(t)\boldsymbol{i} - f(\dot{u})\boldsymbol{i} - s(u)\boldsymbol{i} = m\ddot{u}\boldsymbol{i}\,.$$

The common mathematical form of the ODE problem is

$$m\ddot{u} + f(\dot{u}) + s(u) = F(t), \quad u(0) = I,\ \dot{u}(0) = V\,. \tag{1.121}$$

This is the generalized problem treated in the last part of the present chapter, but with prime denoting the derivative instead of the dot.

The most common models for the spring and dashpot are linear: $f(\dot{u}) = b\dot{u}$ with a constant $b \geq 0$, and $s(u) = ku$ for a constant k.

Scaling. A specific scaling requires specific choices of f, s, and F. Suppose we have

$$f(\dot{u}) = b|\dot{u}|\dot{u}, \quad s(u) = ku, \quad F(t) = A\sin(\phi t)\,.$$

We introduce dimensionless variables as usual, $\bar{u} = u/u_c$ and $\bar{t} = t/t_c$. The scale u_c depends both on the initial conditions and F, but as time grows, the effect of the initial conditions die out and F will drive the motion. Inserting $\bar{u}$ and $\bar{t}$ in the ODE gives

$$m\frac{u_c}{t_c^2}\frac{d^2\bar{u}}{d\bar{t}^2} + b\frac{u_c^2}{t_c^2}\left|\frac{d\bar{u}}{d\bar{t}}\right|\frac{d\bar{u}}{d\bar{t}} + ku_c\bar{u} = A\sin(\phi t_c\bar{t})\,.$$

We divide by u_c/t_c^2 and demand the coefficients of the $\bar{u}$ and the forcing term from $F(t)$ to have unit coefficients. This leads to the scales

$$t_c = \sqrt{\frac{m}{k}}, \quad u_c = \frac{A}{k}\,.$$

The scaled ODE becomes

$$\frac{d^2\bar{u}}{d\bar{t}^2} + 2\beta\left|\frac{d\bar{u}}{d\bar{t}}\right|\frac{d\bar{u}}{d\bar{t}} + \bar{u} = \sin(\gamma\bar{t}), \tag{1.122}$$

where there are two dimensionless numbers:

$$\beta = \frac{Ab}{2mk}, \quad \gamma = \phi\sqrt{\frac{m}{k}}\,.$$

The β number measures the size of the damping term (relative to unity) and is assumed to be small, basically because b is small. The ϕ number

is the ratio of the time scale of free vibrations and the time scale of the forcing. The scaled initial conditions have two other dimensionless numbers as values:

$$\bar{u}(0) = \frac{Ik}{A}, \quad \frac{d\bar{u}}{d\bar{t}} = \frac{t_c}{u_c}V = \frac{V}{A}\sqrt{mk}\,.$$

1.12.3 A sliding mass attached to a spring

Consider a variant of the oscillating body in Section 1.12.1 and Figure 1.17: the body rests on a flat surface, and there is sliding friction between the body and the surface. Figure 1.19 depicts the problem.

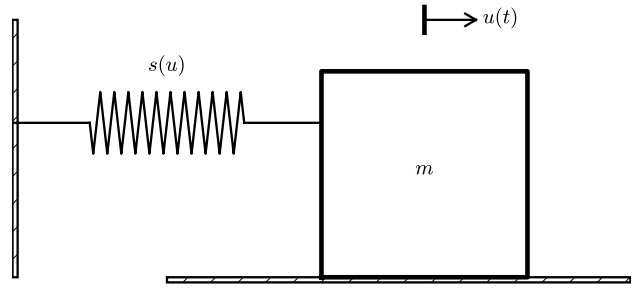


Fig. 1.19 Sketch of a body sliding on a surface.

The body is attached to a spring with spring force $-s(u)\boldsymbol{i}$. The friction force is proportional to the normal force on the surface, $-mg\boldsymbol{j}$, and given by $-f(\dot{u})\boldsymbol{i}$, where

$$f(\dot{u}) = \begin{cases} -\mu mg, & \dot{u} < 0, \\ \mu mg, & \dot{u} > 0, \\ 0, & \dot{u} = 0 \end{cases}$$

Here, μ is a friction coefficient. With the signum function

$$\text{sign(x)} = \begin{cases} -1, & x < 0, \\ 1, & x > 0, \\ 0, & x = 0 \end{cases}$$

we can simply write $f(\dot{u}) = \mu mg\,\text{sign}(\dot{u})$ (the sign function is implemented by `numpy.sign`).

The equation of motion becomes

$$m\ddot{u} + \mu mg\text{sign}(\dot{u}) + s(u) = 0, \quad u(0) = I,\ \dot{u}(0) = V\,. \tag{1.123}$$

1.12.4 A jumping washing machine

A washing machine is placed on four springs with efficient dampers. If the machine contains just a few clothes, the circular motion of the machine induces a sinusoidal external force from the floor and the machine will jump up and down if the frequency of the external force is close to the natural frequency of the machine and its spring-damper system.

1.12.5 Motion of a pendulum

Simple pendulum. A classical problem in mechanics is the motion of a pendulum. We first consider a simplified pendulum[17] (sometimes also called a mathematical pendulum): a small body of mass m is attached to a massless wire and can oscillate back and forth in the gravity field. Figure 1.20 shows a sketch of the problem.

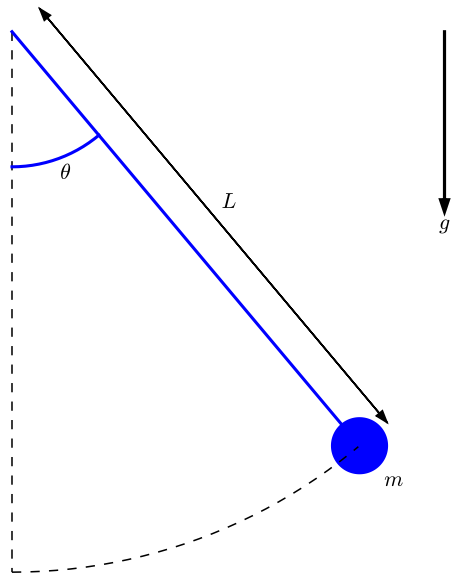


Fig. 1.20 Sketch of a simple pendulum.

[17] https://en.wikipedia.org/wiki/Pendulum

The motion is governed by Newton's 2nd law, so we need to find expressions for the forces and the acceleration. Three forces on the body are considered: an unknown force S from the wire, the gravity force mg, and an air resistance force, $\frac{1}{2}C_D\varrho A|v|v$, hereafter called the drag force, directed against the velocity of the body. Here, C_D is a drag coefficient, ϱ is the density of air, A is the cross section area of the body, and v is the magnitude of the velocity.

We introduce a coordinate system with polar coordinates and unit vectors $\boldsymbol{i}_r$ and $\boldsymbol{i}_\theta$ as shown in Figure 1.21. The position of the center of mass of the body is

$$\boldsymbol{r}(t) = x_0\boldsymbol{i} + y_0\boldsymbol{j} + L\boldsymbol{i}_r,$$

where $\boldsymbol{i}$ and $\boldsymbol{j}$ are unit vectors in the corresponding Cartesian coordinate system in the x and y directions, respectively. We have that $\boldsymbol{i}_r = \cos\theta\boldsymbol{i} + \sin\theta\boldsymbol{j}$.

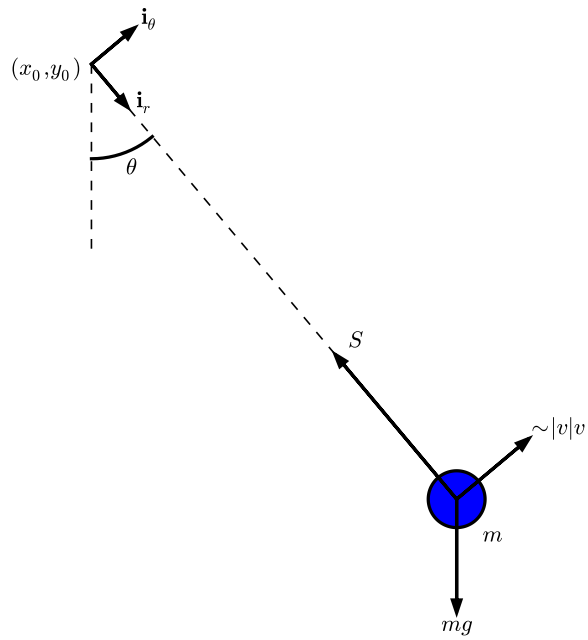


Fig. 1.21 Forces acting on a simple pendulum.

The forces are now expressed as follows.

- Wire force: $-S\boldsymbol{i}_r$
- Gravity force: $-mg\boldsymbol{j} = mg(-\sin\theta\,\boldsymbol{i}_\theta + \cos\theta\,\boldsymbol{i}_r)$

- Drag force: $-\frac{1}{2}C_D\varrho A|v|v\,\boldsymbol{i}_\theta$

Since a positive velocity means movement in the direction of $\boldsymbol{i}_\theta$, the drag force must be directed along $-\boldsymbol{i}_\theta$ so it works against the motion. We assume motion in air so that the added mass effect can be neglected (for a spherical body, the added mass is $\frac{1}{2}\varrho V$, where V is the volume of the body). Also the buoyancy effect can be neglected for motion in the air when the density difference between the fluid and the body is so significant.

The velocity of the body is found from $\boldsymbol{r}$:

$$\boldsymbol{v}(t) = \dot{\boldsymbol{r}}(t) = \frac{d}{d\theta}(x_0\boldsymbol{i} + y_0\boldsymbol{j} + L\boldsymbol{i}_r)\frac{d\theta}{dt} = L\dot{\theta}\boldsymbol{i}_\theta,$$

since $\frac{d}{d\theta}\boldsymbol{i}_r = \boldsymbol{i}_\theta$. It follows that $v = |\boldsymbol{v}| = L\dot{\theta}$. The acceleration is

$$\boldsymbol{a}(t) = \dot{\boldsymbol{v}}(r) = \frac{d}{dt}(L\dot{\theta}\boldsymbol{i}_\theta) = L\ddot{\theta}\boldsymbol{i}_\theta + L\dot{\theta}\frac{d\boldsymbol{i}_\theta}{d\theta}\dot{\theta} == L\ddot{\theta}\boldsymbol{i}_\theta - L\dot{\theta}^2\boldsymbol{i}_r,$$

since $\frac{d}{d\theta}\boldsymbol{i}_\theta = -\boldsymbol{i}_r$.

Newton's 2nd law of motion becomes

$$-S\boldsymbol{i}_r + mg(-\sin\theta\,\boldsymbol{i}_\theta + \cos\theta\,\boldsymbol{i}_r) - \frac{1}{2}C_D\varrho AL^2|\dot{\theta}|\dot{\theta}\,\boldsymbol{i}_\theta = mL\ddot{\theta}\dot{\theta}\,\boldsymbol{i}_\theta - L\dot{\theta}^2\boldsymbol{i}_r,$$

leading to two component equations

$$-S + mg\cos\theta = -L\dot{\theta}^2, \tag{1.124}$$

$$-mg\sin\theta - \frac{1}{2}C_D\varrho AL^2|\dot{\theta}|\dot{\theta} = mL\ddot{\theta}\,. \tag{1.125}$$

From (1.124) we get an expression for $S = mg\cos\theta + L\dot{\theta}^2$, and from (1.125) we get a differential equation for the angle $\theta(t)$. This latter equation is ordered as

$$m\ddot{\theta} + \frac{1}{2}C_D\varrho AL|\dot{\theta}|\dot{\theta} + \frac{mg}{L}\sin\theta = 0\,. \tag{1.126}$$

Two initial conditions are needed: $\theta = \Theta$ and $\dot{\theta} = \Omega$. Normally, the pendulum motion is started from rest, which means $\Omega = 0$.

Equation (1.126) fits the general model used in (1.71) in Section 1.10 if we define $u = \theta$, $f(u') = \frac{1}{2}C_D\varrho AL|\dot{u}|\dot{u}$, $s(u) = L^{-1}mg\sin u$, and

$F = 0$. If the body is a sphere with radius R, we can take $C_D = 0.4$ and $A = \pi R^2$. Exercise 1.25 asks you to scale the equations and carry out specific simulations with this model.

Physical pendulum. The motion of a compound or physical pendulum where the wire is a rod with mass, can be modeled very similarly. The governing equation is $I\boldsymbol{a} = \boldsymbol{T}$ where I is the moment of inertia of the entire body about the point (x_0, y_0), and $\boldsymbol{T}$ is the sum of moments of the forces with respect to (x_0, y_0). The vector equation reads

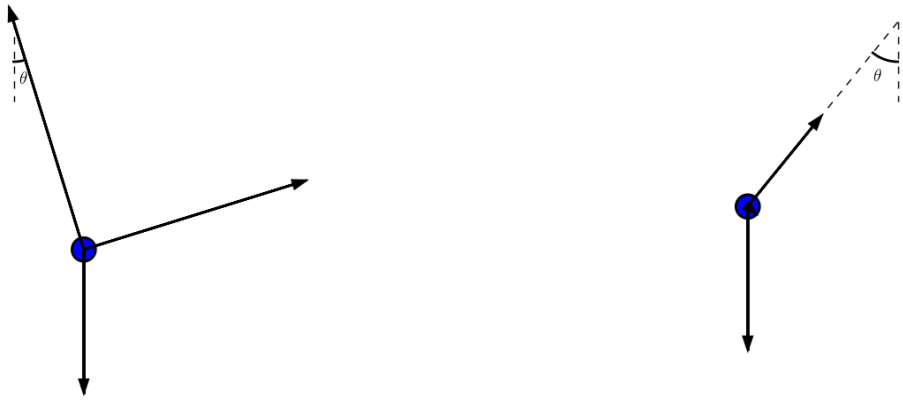
$$\boldsymbol{r}\times(-S\boldsymbol{i}_r+mg(-\sin\theta\boldsymbol{i}_\theta+\cos\theta\boldsymbol{i}_r)-\frac{1}{2}C_D\varrho AL^2|\dot\theta|\dot\theta\boldsymbol{i}_\theta) = I(L\ddot\theta\dot\theta\boldsymbol{i}_\theta-L\dot\theta^2\boldsymbol{i}_r)\,.$$

The component equation in $\boldsymbol{i}_\theta$ direction gives the equation of motion for $\theta(t)$:

$$I\ddot\theta + \frac{1}{2}C_D\varrho AL^3|\dot\theta|\dot\theta + mgL\sin\theta = 0\,. \tag{1.127}$$

1.12.6 Dynamic free body diagram during pendulum motion

Usually one plots the mathematical quantities as functions of time to visualize the solution of ODE models. Exercise 1.25 asks you to do this for the motion of a pendulum in the previous section. However, sometimes it is more instructive to look at other types of visualizations. For example, we have the pendulum and the free body diagram in Figures 1.20 and 1.21. We may think of these figures as animations in time instead. Especially the free body diagram will show both the motion of the pendulum *and* the size of the forces during the motion. The present section exemplifies how to make such a dynamic body diagram. Two typical snapshots of free body diagrams are displayed below (the drag force is magnified 5 times to become more visual!).



Dynamic physical sketches, coupled to the numerical solution of differential equations, requires a program to produce a sketch for the situation at each time level. Pysketcher[18] is such a tool. In fact (and not surprising!) Figures 1.20 and 1.21 were drawn using Pysketcher. The details of the drawings are explained in the Pysketcher tutorial[19]. Here, we outline how this type of sketch can be used to create an animated free body diagram during the motion of a pendulum.

Pysketcher is actually a layer of useful abstractions on top of standard plotting packages. This means that we in fact apply Matplotlib to make the animated free body diagram, but instead of dealing with a wealth of detailed Matplotlib commands, we can express the drawing in terms of more high-level objects, e.g., objects for the wire, angle θ, body with mass m, arrows for forces, etc. When the position of these objects are given through variables, we can just couple those variables to the dynamic solution of our ODE and thereby make a unique drawing for each θ value in a simulation.

Writing the solver. Let us start with the most familiar part of the current problem: writing the solver function. We use Odespy for this purpose. We also work with dimensionless equations. Since θ can be viewed as dimensionless, we only need to introduce a dimensionless time, here taken as $\bar{t} = t/\sqrt{L/g}$. The resulting dimensionless mathematical model for θ, the dimensionless angular velocity ω, the dimensionless wire force $\bar{S}$, and the dimensionless drag force $\bar{D}$ is then

[18] `https://github.com/hplgit/pysketcher`
[19] `http://hplgit.github.io/pysketcher/doc/web/index.html`

$$\frac{d\omega}{d\bar{t}} = -\alpha|\omega|\omega - \sin\theta, \tag{1.128}$$

$$\frac{d\theta}{d\bar{t}} = \omega, \tag{1.129}$$

$$\bar{S} = \omega^2 + \cos\theta, \tag{1.130}$$

$$\bar{D} = -\alpha|\omega|\omega, \tag{1.131}$$

with

$$\alpha = \frac{C_D \varrho \pi R^2 L}{2m}.$$

as a dimensionless parameter expressing the ratio of the drag force and the gravity force. The dimensionless ω is made non-dimensional by the time, so $\omega\sqrt{L/g}$ is the corresponding angular frequency with dimensions.

A suitable function for computing (1.128)-(1.131) is listed below.

```
def simulate(alpha, Theta, dt, T):
    import odespy

    def f(u, t, alpha):
        omega, theta = u
        return [-alpha*omega*abs(omega) - sin(theta),
                omega]

    import numpy as np
    Nt = int(round(T/float(dt)))
    t = np.linspace(0, Nt*dt, Nt+1)
    solver = odespy.RK4(f, f_args=[alpha])
    solver.set_initial_condition([0, Theta])
    u, t = solver.solve(
        t, terminate=lambda u, t, n: abs(u[n,1]) < 1E-3)
    omega = u[:,0]
    theta = u[:,1]
    S = omega**2 + np.cos(theta)
    drag = -alpha*np.abs(omega)*omega
    return t, theta, omega, S, drag
```

Drawing the free body diagram. The `sketch` function below applies Pysketcher objects to build a diagram like that in Figure 1.21, except that we have removed the rotation point (x_0, y_0) and the unit vectors in polar coordinates as these objects are not important for an animated free body diagram.

```
import sys
try:
    from pysketcher import *
except ImportError:
```

```
    print 'Pysketcher must be installed from'
    print 'https://github.com/hplgit/pysketcher'
    sys.exit(1)

# Overall dimensions of sketch
H = 15.
W = 17.

drawing_tool.set_coordinate_system(
    xmin=0, xmax=W, ymin=0, ymax=H,
    axis=False)

def sketch(theta, S, mg, drag, t, time_level):
    """
    Draw pendulum sketch with body forces at a time level
    corresponding to time t. The drag force is in
    drag[time_level], the force in the wire is S[time_level],
    the angle is theta[time_level].
    """
    import math
    a = math.degrees(theta[time_level])  # angle in degrees
    L = 0.4*H         # Length of pendulum
    P = (W/2, 0.8*H)  # Fixed rotation point

    mass_pt = path.geometric_features()['end']
    rod = Line(P, mass_pt)

    mass = Circle(center=mass_pt, radius=L/20.)
    mass.set_filled_curves(color='blue')
    rod_vec = rod.geometric_features()['end'] - \
              rod.geometric_features()['start']
    unit_rod_vec = unit_vec(rod_vec)
    mass_symbol = Text('$m$', mass_pt + L/10*unit_rod_vec)

    rod_start = rod.geometric_features()['start']  # Point P
    vertical = Line(rod_start, rod_start + point(0,-L/3))

    def set_dashed_thin_blackline(*objects):
        """Set linestyle of objects to dashed, black, width=1."""
        for obj in objects:
            obj.set_linestyle('dashed')
            obj.set_linecolor('black')
            obj.set_linewidth(1)

    set_dashed_thin_blackline(vertical)
    set_dashed_thin_blackline(rod)
    angle = Arc_wText(r'$\theta$', rod_start, L/6, -90, a,
                      text_spacing=1/30.)

    magnitude = 1.2*L/2   # length of a unit force in figure
    force = mg[time_level]  # constant (scaled eq: about 1)
    force *= magnitude
    mg_force  = Force(mass_pt, mass_pt + force*point(0,-1),
                      '', text_pos='end')
```

```
    force = S[time_level]
    force *= magnitude
    rod_force = Force(mass_pt, mass_pt - force*unit_vec(rod_vec),
                      '', text_pos='end',
                      text_spacing=(0.03, 0.01))
    force = drag[time_level]
    force *= magnitude
    air_force = Force(mass_pt, mass_pt -
                      force*unit_vec((rod_vec[1], -rod_vec[0])),
                      '', text_pos='end',
                      text_spacing=(0.04,0.005))

    body_diagram = Composition(
        {'mg': mg_force, 'S': rod_force, 'air': air_force,
         'rod': rod, 'body': mass
         'vertical': vertical, 'theta': angle,})

    body_diagram.draw(verbose=0)
    drawing_tool.savefig('tmp_%04d.png' % time_level, crop=False)
    # (No cropping: otherwise movies will be very strange!)
```

Making the animated free body diagram. It now remains to couple the `simulate` and `sketch` functions. We first run `simulate`:

```
from math import pi, radians, degrees
import numpy as np
alpha = 0.4
period = 2*pi   # Use small theta approximation
T = 12*period   # Simulate for 12 periods
dt = period/40  # 40 time steps per period
a = 70          # Initial amplitude in degrees
Theta = radians(a)

t, theta, omega, S, drag = simulate(alpha, Theta, dt, T)
```

The next step is to run through the time levels in the simulation and make a sketch at each level:

```
for time_level, t_ in enumerate(t):
    sketch(theta, S, mg, drag, t_, time_level)
```

The individual sketches are (by the `sketch` function) saved in files with names `tmp_%04d.png`. These can be combined to videos using (e.g.) `ffmpeg`. A complete function `animate` for running the simulation and creating video files is listed below.

```
def animate():
    # Clean up old plot files
    import os, glob
    for filename in glob.glob('tmp_*.png') + glob.glob('movie.*'):
        os.remove(filename)
    # Solve problem
```

```
from math import pi, radians, degrees
import numpy as np
alpha = 0.4
period = 2*pi   # Use small theta approximation
T = 12*period   # Simulate for 12 periods
dt = period/40  # 40 time steps per period
a = 70          # Initial amplitude in degrees
Theta = radians(a)

t, theta, omega, S, drag = simulate(alpha, Theta, dt, T)

# Visualize drag force 5 times as large
drag *= 5
mg = np.ones(S.size)  # Gravity force (needed in sketch)

# Draw animation
import time
for time_level, t_ in enumerate(t):
    sketch(theta, S, mg, drag, t_, time_level)
    time.sleep(0.2)  # Pause between each frame on the screen

# Make videos
prog = 'ffmpeg'
filename = 'tmp_%04d.png'
fps = 6
codecs = {'flv': 'flv', 'mp4': 'libx264',
          'webm': 'libvpx', 'ogg': 'libtheora'}
for ext in codecs:
    lib = codecs[ext]
    cmd = '%(prog)s -i %(filename)s -r %(fps)s ' % vars()
    cmd += '-vcodec %(lib)s movie.%(ext)s' % vars()
    print(cmd)
    os.system(cmd)
```

1.12.7 Motion of an elastic pendulum

Consider a pendulum as in Figure 1.20, but this time the wire is elastic. The length of the wire when it is not stretched is L_0, while $L(t)$ is the stretched length at time t during the motion.

Stretching the elastic wire a distance ΔL gives rise to a spring force $k\Delta L$ in the opposite direction of the stretching. Let $\boldsymbol{n}$ be a unit normal vector along the wire from the point $\boldsymbol{r}_0 = (x_0, y_0)$ and in the direction of $\boldsymbol{i}_\theta$, see Figure 1.21 for definition of (x_0, y_0) and $\boldsymbol{i}_\theta$. Obviously, we have $\boldsymbol{n} = \boldsymbol{i}_\theta$, but in this modeling of an elastic pendulum we do not need polar coordinates. Instead, it is more straightforward to develop the equation in Cartesian coordinates.

A mathematical expression for $\boldsymbol{n}$ is

$$\boldsymbol{n} = \frac{\boldsymbol{r} - \boldsymbol{r}_0}{L(t)},$$

where $L(t) = ||\boldsymbol{r} - \boldsymbol{r}_0||$ is the current length of the elastic wire. The position vector $\boldsymbol{r}$ in Cartesian coordinates reads $\boldsymbol{r}(t) = x(t)\boldsymbol{i} + y(t)\boldsymbol{j}$, where $\boldsymbol{i}$ and $\boldsymbol{j}$ are unit vectors in the x and y directions, respectively. It is convenient to introduce the Cartesian components n_x and n_y of the normal vector:

$$\boldsymbol{n} = \frac{\boldsymbol{r} - \boldsymbol{r}_0}{L(t)} = \frac{x(t) - x_0}{L(t)}\boldsymbol{i} + \frac{y(t) - y_0}{L(t)}\boldsymbol{j} = n_x\boldsymbol{i} + n_y\boldsymbol{j}\,.$$

The stretch ΔL in the wire is

$$\Delta t = L(t) - L_0\,.$$

The force in the wire is then $-S\boldsymbol{n} = -k\Delta L\boldsymbol{n}$.

The other forces are the gravity and the air resistance, just as in Figure 1.21. For motion in air we can neglect the added mass and buoyancy effects. The main difference is that we have a *model* for S in terms of the motion (as soon as we have expressed ΔL by $\boldsymbol{r}$). For simplicity, we drop the air resistance term (but Exercise 1.27 asks you to include it).

Newton's second law of motion applied to the body now results in

$$m\ddot{\boldsymbol{r}} = -k(L - L_0)\boldsymbol{n} - mg\boldsymbol{j} \tag{1.132}$$

The two components of (1.132) are

$$\ddot{x} = -\frac{k}{m}(L - L_0)n_x, \tag{1.133}$$

$$\tag{1.134}$$

$$\ddot{y} = -\frac{k}{m}(L - L_0)n_y - g\,. \tag{1.135}$$

Remarks about an elastic vs a non-elastic pendulum. Note that the derivation of the ODEs for an elastic pendulum is more straightforward than for a classical, non-elastic pendulum, since we avoid the details with polar coordinates, but instead work with Newton's second law directly in Cartesian coordinates. The reason why we can do this is that the elastic pendulum undergoes a general two-dimensional motion where all the forces are known or expressed as functions of $x(t)$ and $y(t)$, such that

we get two ordinary differential equations. The motion of the non-elastic pendulum, on the other hand, is constrained: the body has to move along a circular path, and the force S in the wire is unknown.

The non-elastic pendulum therefore leads to a *differential-algebraic* equation, i.e., ODEs for $x(t)$ and $y(t)$ combined with an extra constraint $(x-x_0)^2+(y-y_0)^2=L^2$ ensuring that the motion takes place along a circular path. The extra constraint (equation) is compensated by an extra unknown force $-S\boldsymbol{n}$. Differential-algebraic equations are normally hard to solve, especially with pen and paper. Fortunately, for the non-elastic pendulum we can do a trick: in polar coordinates the unknown force S appears only in the radial component of Newton's second law, while the unknown degree of freedom for describing the motion, the angle $\theta(t)$, is completely governed by the asimuthal component. This allows us to decouple the unknowns S and θ. But this is a kind of trick and not a widely applicable method. With an elastic pendulum we use straightforward reasoning with Newton's 2nd law and arrive at a standard ODE problem that (after scaling) is easy to solve on a computer.

Initial conditions. What is the initial position of the body? We imagine that first the pendulum hangs in equilibrium in its vertical position, and then it is displaced an angle Θ. The equilibrium position is governed by the ODEs with the accelerations set to zero. The x component leads to $x(t)=x_0$, while the y component gives

$$0=-\frac{k}{m}(L-L_0)n_y-g=\frac{k}{m}(L(0)-L_0)-g \quad \Rightarrow \quad L(0)=L_0+mg/k,$$

since $n_y=-11$ in this position. The corresponding y value is then from $n_y=-1$:

$$y(t)=y_0-L(0)=y_0-(L_0+mg/k)\,.$$

Let us now choose (x_0,y_0) such that the body is at the origin in the equilibrium position:

$$x_0=0, \quad y_0=L_0+mg/k\,.$$

Displacing the body an angle Θ to the right leads to the initial position

$$x(0)=(L_0+mg/k)\sin\Theta, \quad y(0)=(L_0+mg/k)(1-\cos\Theta)\,.$$

The initial velocities can be set to zero: $x'(0) = y'(0) = 0$.

The complete ODE problem. We can summarize all the equations as follows:

$$
\begin{aligned}
\ddot{x} &= -\frac{k}{m}(L - L_0)n_x, \\
\ddot{y} &= -\frac{k}{m}(L - L_0)n_y - g, \\
L &= \sqrt{(x - x_0)^2 + (y - y_0)^2}, \\
n_x &= \frac{x - x_0}{L}, \\
n_y &= \frac{y - y_0}{L}, \\
x(0) &= (L_0 + mg/k)\sin\Theta, \\
x'(0) &= 0, \\
y(0) &= (L_0 + mg/k)(1 - \cos\Theta), \\
y'(0) &= 0\,.
\end{aligned}
$$

We insert n_x and n_y in the ODEs:

$$
\ddot{x} = -\frac{k}{m}\left(1 - \frac{L_0}{L}\right)(x - x_0), \tag{1.136}
$$

$$
\ddot{y} = -\frac{k}{m}\left(1 - \frac{L_0}{L}\right)(y - y_0) - g, \tag{1.137}
$$

$$
L = \sqrt{(x - x_0)^2 + (y - y_0)^2}, \tag{1.138}
$$

$$
x(0) = (L_0 + mg/k)\sin\Theta, \tag{1.139}
$$

$$
x'(0) = 0, \tag{1.140}
$$

$$
y(0) = (L_0 + mg/k)(1 - \cos\Theta), \tag{1.141}
$$

$$
y'(0) = 0\,. \tag{1.142}
$$

Scaling. The elastic pendulum model can be used to study both an elastic pendulum and a classic, non-elastic pendulum. The latter problem is obtained by letting $k \to \infty$. Unfortunately, a serious problem with the ODEs (1.136)-(1.137) is that for large k, we have a very large factor k/m multiplied by a very small number $1 - L_0/L$, since for large k, $L \approx L_0$ (very small deformations of the wire). The product is subject to significant round-off errors for many relevant physical values of the

parameters. To circumvent the problem, we introduce a scaling. This will also remove physical parameters from the problem such that we end up with only one dimensionless parameter, closely related to the elasticity of the wire. Simulations can then be done by setting just this dimensionless parameter.

The characteristic length can be taken such that in equilibrium, the scaled length is unity, i.e., the characteristic length is $L_0 + mg/k$:

$$\bar{x} = \frac{x}{L_0 + mg/k}, \quad \bar{y} = \frac{y}{L_0 + mg/k}\,.$$

We must then also work with the scaled length $\bar{L} = L/(L_0 + mg/k)$.

Introducing $\bar{t} = t/t_c$, where t_c is a characteristic time we have to decide upon later, one gets

$$\begin{aligned}
\frac{d^2\bar{x}}{d\bar{t}^2} &= -t_c^2\frac{k}{m}\left(1 - \frac{L_0}{L_0 + mg/k}\frac{1}{\bar{L}}\right)\bar{x},\\
\frac{d^2\bar{y}}{d\bar{t}^2} &= -t_c^2\frac{k}{m}\left(1 - \frac{L_0}{L_0 + mg/k}\frac{1}{\bar{L}}\right)(\bar{y} - 1) - t_c^2\frac{g}{L_0 + mg/k},\\
\bar{L} &= \sqrt{\bar{x}^2 + (\bar{y} - 1)^2},\\
\bar{x}(0) &= \sin\Theta,\\
\bar{x}'(0) &= 0,\\
\bar{y}(0) &= 1 - \cos\Theta,\\
\bar{y}'(0) &= 0\,.
\end{aligned}$$

For a non-elastic pendulum with small angles, we know that the frequency of the oscillations are $\omega = \sqrt{L/g}$. It is therefore natural to choose a similar expression here, either the length in the equilibrium position,

$$t_c^2 = \frac{L_0 + mg/k}{g}\,.$$

or simply the unstretched length,

$$t_c^2 = \frac{L_0}{g}\,.$$

These quantities are not very different (since the elastic model is valid only for quite small elongations), so we take the latter as it is the simplest one.

The ODEs become

$$\frac{d^2\bar{x}}{d\bar{t}^2} = -\frac{L_0 k}{mg}\left(1 - \frac{L_0}{L_0 + mg/k}\frac{1}{\bar{L}}\right)\bar{x},$$
$$\frac{d^2\bar{y}}{d\bar{t}^2} = -\frac{L_0 k}{mg}\left(1 - \frac{L_0}{L_0 + mg/k}\frac{1}{\bar{L}}\right)(\bar{y} - 1) - \frac{L_0}{L_0 + mg/k},$$
$$\bar{L} = \sqrt{\bar{x}^2 + (\bar{y} - 1)^2}\,.$$

We can now identify a dimensionless number

$$\beta = \frac{L_0}{L_0 + mg/k} = \frac{1}{1 + \frac{mg}{L_0 k}},$$

which is the ratio of the unstretched length and the stretched length in equilibrium. The non-elastic pendulum will have $\beta = 1$ ($k \to \infty$). With β the ODEs read

$$\frac{d^2\bar{x}}{d\bar{t}^2} = -\frac{\beta}{1-\beta}\left(1 - \frac{\beta}{\bar{L}}\right)\bar{x}, \tag{1.143}$$
$$\frac{d^2\bar{y}}{d\bar{t}^2} = -\frac{\beta}{1-\beta}\left(1 - \frac{\beta}{\bar{L}}\right)(\bar{y} - 1) - \beta, \tag{1.144}$$
$$\bar{L} = \sqrt{\bar{x}^2 + (\bar{y} - 1)^2}, \tag{1.145}$$
$$\bar{x}(0) = (1 + \epsilon)\sin\Theta, \tag{1.146}$$
$$\frac{d\bar{x}}{d\bar{t}}(0) = 0, \tag{1.147}$$
$$\bar{y}(0) = 1 - (1 + \epsilon)\cos\Theta, \tag{1.148}$$
$$\frac{d\bar{y}}{d\bar{t}}(0) = 0, \tag{1.149}$$

We have here added a parameter ϵ, which is an additional downward stretch of the wire at $t = 0$. This parameter makes it possible to do a desired test: vertical oscillations of the pendulum. Without ϵ, starting the motion from $(0, 0)$ with zero velocity will result in $x = y = 0$ for all times (also a good test!), but with an initial stretch so the body's position is $(0, \epsilon)$, we will have oscillatory vertical motion with amplitude ϵ (see Exercise 1.26).

Remark on the non-elastic limit. We immediately see that as $k \to \infty$ (i.e., we obtain a non-elastic pendulum), $\beta \to 1$, $\bar{L} \to 1$, and we have very small values $1 - \beta\bar{L}^{-1}$ divided by very small values $1 - \beta$ in the ODEs.

However, it turns out that we can set β very close to one and obtain a path of the body that within the visual accuracy of a plot does not show any elastic oscillations. (Should the division of very small values become a problem, one can study the limit by L'Hospital's rule:

$$\lim_{\beta\to 1}\frac{1-\beta\bar{L}^{-1}}{1-\beta}=\frac{1}{\bar{L}},$$

and use the limit $\bar{L}^{-1}$ in the ODEs for β values very close to 1.)

1.12.8 Vehicle on a bumpy road

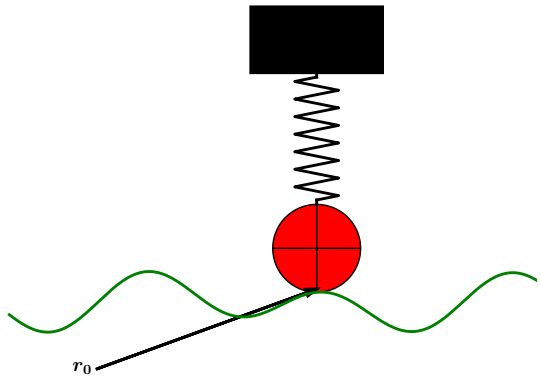


Fig. 1.22 Sketch of one-wheel vehicle on a bumpy road.

We consider a very simplistic vehicle, on one wheel, rolling along a bumpy road. The oscillatory nature of the road will induce an external forcing on the spring system in the vehicle and cause vibrations. Figure 1.22 outlines the situation.

To derive the equation that governs the motion, we must first establish the position vector of the black mass at the top of the spring. Suppose the spring has length L without any elongation or compression, suppose the radius of the wheel is R, and suppose the height of the black mass at the top is H. With the aid of the $\boldsymbol{r}_0$ vector in Figure 1.22, the position $\boldsymbol{r}$ of the center point of the mass is

$$\boldsymbol{r}=\boldsymbol{r}_0+2R\boldsymbol{j}+L\boldsymbol{j}+u\boldsymbol{j}+\frac{1}{2}H\boldsymbol{j}, \tag{1.150}$$

where u is the elongation or compression in the spring according to the (unknown and to be computed) vertical displacement u relative to the road. If the vehicle travels with constant horizontal velocity v and $h(x)$ is the shape of the road, then the vector $\boldsymbol{r}_0$ is

$$\boldsymbol{r}_0 = vt\boldsymbol{i} + h(vt)\boldsymbol{j},$$

if the motion starts from $x = 0$ at time $t = 0$.

The forces on the mass is the gravity, the spring force, and an optional damping force that is proportional to the vertical velocity $\dot{u}$. Newton's second law of motion then tells that

$$m\ddot{\boldsymbol{r}} = -mg\boldsymbol{j} - s(u) - b\dot{u}\boldsymbol{j}\,.$$

This leads to

$$m\ddot{u} = -s(u) - b\dot{u} - mg - mh''(vt)v^2$$

To simplify a little bit, we omit the gravity force mg in comparison with the other terms. Introducing u' for $\dot{u}$ then gives a standard damped, vibration equation with external forcing:

$$mu'' + bu' + s(u) = -mh''(vt)v^2\,. \tag{1.151}$$

Since the road is normally known just as a set of array values, h'' must be computed by finite differences. Let Δx be the spacing between measured values $h_i = h(i\Delta x)$ on the road. The discrete second-order derivative h'' reads

$$q_i = \frac{h_{i-1} - 2h_i + h_{i+1}}{\Delta x^2}, \quad i = 1, \ldots, N_x - 1\,.$$

We may for maximum simplicity set the end points as $q_0 = q_1$ and $q_{N_x} = q_{N_x-1}$. The term $-mh''(vt)v^2$ corresponds to a force with discrete time values

$$F^n = -mq_n v^2, \quad \Delta t = v^{-1}\Delta x\,.$$

This force can be directly used in a numerical model

$$[mD_tD_tu + bD_{2t}u + s(u) = F]^n\,.$$

Software for computing u and also making an animated sketch of the motion like we did in Section 1.12.6 is found in a separate project on

the web: `https://github.com/hplgit/bumpy`. You may start looking at the tutorial[20].

1.12.9 Bouncing ball

A bouncing ball is a ball in free vertically fall until it impacts the ground, but during the impact, some kinetic energy is lost, and a new motion upwards with reduced velocity starts. After the motion is retarded, a new free fall starts, and the process is repeated. At some point the velocity close to the ground is so small that the ball is considered to be finally at rest.

The motion of the ball falling in air is governed by Newton's second law $F = ma$, where a is the acceleration of the body, m is the mass, and F is the sum of all forces. Here, we neglect the air resistance so that gravity $-mg$ is the only force. The height of the ball is denoted by h and v is the velocity. The relations between h, v, and a,

$$h'(t) = v(t), \quad v'(t) = a(t),$$

combined with Newton's second law gives the ODE model

$$h''(t) = -g, \tag{1.152}$$

or expressed alternatively as a system of first-order equations:

$$v'(t) = -g, \tag{1.153}$$

$$h'(t) = v(t)\,. \tag{1.154}$$

These equations govern the motion as long as the ball is away from the ground by a small distance $\epsilon_h > 0$. When $h < \epsilon_h$, we have two cases.

1. The ball impacts the ground, recognized by a sufficiently large negative velocity ($v < -\epsilon_v$). The velocity then changes sign and is reduced by a factor C_R, known as the coefficient of restitution[21]. For plotting purposes, one may set $h = 0$.
2. The motion stops, recognized by a sufficiently small velocity ($|v| < \epsilon_v$) close to the ground.

[20] `http://hplgit.github.io/bumpy/doc/pub/bumpy.pdf`
[21] `http://en.wikipedia.org/wiki/Coefficient_of_restitution`

1.12.10 Two-body gravitational problem

Consider two astronomical objects A and B that attract each other by gravitational forces. A and B could be two stars in a binary system, a planet orbiting a star, or a moon orbiting a planet. Each object is acted upon by the gravitational force due to the other object. Consider motion in a plane (for simplicity) and let (x_A, y_A) and (x_B, y_B) be the positions of object A and B, respectively.

The governing equations. Newton's second law of motion applied to each object is all we need to set up a mathematical model for this physical problem:

$$m_A \ddot{\boldsymbol{x}}_A = \boldsymbol{F}, \tag{1.155}$$

$$m_B \ddot{\boldsymbol{x}}_B = -\boldsymbol{F}, \tag{1.156}$$

where F is the gravitational force

$$\boldsymbol{F} = \frac{G m_A m_B}{||\boldsymbol{r}||^3}\boldsymbol{r},$$

where

$$\boldsymbol{r}(t) = \boldsymbol{x}_B(t) - \boldsymbol{x}_A(t),$$

and G is the gravitational constant: $G = 6.674 \cdot 10^{-11}$ $\mathrm{Nm}^2/\mathrm{kg}^2$.

Scaling. A problem with these equations is that the parameters are very large (m_A, m_B, $||\boldsymbol{r}||$) or very small (G). The rotation time for binary stars can be very small and large as well. It is therefore advantageous to scale the equations. A natural length scale could be the initial distance between the objects: $L = \boldsymbol{r}(0)$. We write the dimensionless quantities as

$$\bar{\boldsymbol{x}}_A = \frac{\boldsymbol{x}_A}{L}, \quad \bar{\boldsymbol{x}}_B = \frac{\boldsymbol{x}_B}{L}, \quad \bar{t} = \frac{t}{t_c}\,.$$

The gravity force is transformed to

$$\boldsymbol{F} = \frac{G m_A m_B}{L^2 ||\bar{\boldsymbol{r}}||^3}\bar{\boldsymbol{r}}, \quad \bar{\boldsymbol{r}} = \bar{\boldsymbol{x}}_B - \bar{\boldsymbol{x}}_A,$$

so the first ODE for $\boldsymbol{x}_A$ becomes

$$\frac{d^2\bar{\boldsymbol{x}}_A}{d\bar{t}^2} = \frac{G m_B t_c^2}{L^3}\frac{\bar{\boldsymbol{r}}}{||\bar{\boldsymbol{r}}||^3}\,.$$

Assuming that quantities with a bar and their derivatives are around unity in size, it is natural to choose t_c such that the fraction $Gm_B t_c/L^2 = 1$:

$$t_c = \sqrt{\frac{L^3}{Gm_B}}\,.$$

From the other equation for $\boldsymbol{x}_B$ we get another candidate for t_c with m_A instead of m_B. Which mass we choose play a role if $m_A \ll m_B$ or $m_B \ll m_A$. One solution is to use the sum of the masses:

$$t_c = \sqrt{\frac{L^3}{G(m_A + m_B)}}\,.$$

Taking a look at Kepler's laws[22] of planetary motion, the orbital period for a planet around the star is given by the t_c above, except for a missing factor of 2π, but that means that t_c^{-1} is just the angular frequency of the motion. Our characteristic time t_c is therefore highly relevant. Introducing the dimensionless number

$$\alpha = \frac{m_A}{m_B},$$

we can write the dimensionless ODE as

$$\frac{d^2\bar{\boldsymbol{x}}_A}{d\bar{t}^2} = \frac{1}{1+\alpha}\frac{\bar{\boldsymbol{r}}}{||\bar{\boldsymbol{r}}||^3}, \tag{1.157}$$

$$\frac{d^2\bar{\boldsymbol{x}}_B}{d\bar{t}^2} = \frac{1}{1+\alpha^{-1}}\frac{\bar{\boldsymbol{r}}}{||\bar{\boldsymbol{r}}||^3}\,. \tag{1.158}$$

In the limit $m_A \ll m_B$, i.e., $\alpha \ll 1$, object B stands still, say $\bar{\boldsymbol{x}}_B = 0$, and object A orbits according to

$$\frac{d^2\bar{\boldsymbol{x}}_A}{d\bar{t}^2} = -\frac{\bar{\boldsymbol{x}}_A}{||\bar{\boldsymbol{x}}_A||^3}\,.$$

Solution in a special case: planet orbiting a star. To better see the motion, and that our scaling is reasonable, we introduce polar coordinates r and θ:

$$\bar{\boldsymbol{x}}_A = r\cos\theta\boldsymbol{i} + r\sin\theta\boldsymbol{j},$$

which means $\bar{\boldsymbol{x}}_A$ can be written as $\bar{\boldsymbol{x}}_A = r\boldsymbol{i}_r$. Since

[22] `https://en.wikipedia.org/wiki/Kepler%27s_laws_of_planetary_motion`

$$\frac{d}{dt}\boldsymbol{i}_r = \dot{\theta}\boldsymbol{i}_\theta, \quad \frac{d}{dt}\boldsymbol{i}_\theta = -\dot{\theta}\boldsymbol{i}_r,$$

we have

$$\frac{d^2\bar{\boldsymbol{x}}_A}{d\bar{t}^2} = (\ddot{r} - r\dot{\theta}^2)\boldsymbol{i}_r + (r\ddot{\theta} + 2\dot{r}\dot{\theta})\boldsymbol{i}_\theta\,.$$

The equation of motion for mass A is then

$$\begin{aligned}\ddot{r} - r\dot{\theta}^2 &= -\frac{1}{r^2},\\ r\ddot{\theta} + 2\dot{r}\dot{\theta} &= 0\,.\end{aligned}$$

The special case of circular motion, $r = 1$, fulfills the equations, since the latter equation then gives $\dot{\theta} = \text{const}$ and the former then gives $\dot{\theta} = 1$, i.e., the motion is $r(t) = 1$, $\theta(t) = t$, with unit angular frequency as expected and period 2π as expected.

1.12.11 Electric circuits

Although the term "mechanical vibrations" is used in the present book, we must mention that the same type of equations arise when modeling electric circuits. The current $I(t)$ in a circuit with an inductor with inductance L, a capacitor with capacitance C, and overall resistance R, is governed by

$$\ddot{I} + \frac{R}{L}\dot{I} + \frac{1}{LC}I = \dot{V}(t), \tag{1.159}$$

where $V(t)$ is the voltage source powering the circuit. This equation has the same form as the general model considered in Section 1.10 if we set $u = I$, $f(u') = bu'$ and define $b = R/L$, $s(u) = L^{-1}C^{-1}u$, and $F(t) = \dot{V}(t)$.

1.13 Exercises

Exercise 1.22: Simulate resonance

We consider the scaled ODE model (1.122) from Section 1.12.2. After scaling, the amplitude of u will have a size about unity as time grows and

the effect of the initial conditions die out due to damping. However, as $\gamma \to 1$, the amplitude of u increases, especially if β is small. This effect is called *resonance*. The purpose of this exercise is to explore resonance.

a) Figure out how the `solver` function in `vib.py` can be called for the scaled ODE (1.122).

b) Run $\gamma = 5, 1.5, 1.1, 1$ for $\beta = 0.005, 0.05, 0.2$. For each β value, present an image with plots of $u(t)$ for the four γ values.
Filename: `resonance`.

Exercise 1.23: Simulate oscillations of a sliding box

Consider a sliding box on a flat surface as modeled in Section 1.12.3. As spring force we choose the nonlinear formula

$$s(u) = \frac{k}{\alpha}\tanh(\alpha u) = ku + \frac{1}{3}\alpha^2 ku^3 + \frac{2}{15}\alpha^4 ku^5 + \mathcal{O}(u^6)\,.$$

a) Plot $g(u) = \alpha^{-1}\tanh(\alpha u)$ for various values of α. Assume $u \in [-1, 1]$.

b) Scale the equations using I as scale for u and $\sqrt{m/k}$ as time scale.

c) Implement the scaled model in b). Run it for some values of the dimensionless parameters.
Filename: `sliding_box`.

Exercise 1.24: Simulate a bouncing ball

Section 1.12.9 presents a model for a bouncing ball. Choose one of the two ODE formulation, (1.152) or (1.153)-(1.154), and simulate the motion of a bouncing ball. Plot $h(t)$. Think about how to plot $v(t)$.

Hint. A naive implementation may get stuck in repeated impacts for large time step sizes. To avoid this situation, one can introduce a state variable that holds the mode of the motion: free fall, impact, or rest. Two consecutive impacts imply that the motion has stopped.
Filename: `bouncing_ball`.

Exercise 1.25: Simulate a simple pendulum

Simulation of simple pendulum can be carried out by using the mathematical model derived in Section 1.12.5 and calling up functionality

in the `vib.py` file (i.e., solve the second-order ODE by centered finite differences).

a) Scale the model. Set up the dimensionless governing equation for θ and expressions for dimensionless drag and wire forces.

b) Write a function for computing θ and the dimensionless drag force and the force in the wire, using the `solver` function in the `vib.py` file. Plot these three quantities below each other (in subplots) so the graphs can be compared. Run two cases, first one in the limit of Θ small and no drag, and then a second one with $\Theta = 40$ degrees and $\alpha = 0.8$.
Filename: `simple_pendulum`.

Exercise 1.26: Simulate an elastic pendulum

Section 1.12.7 describes a model for an elastic pendulum, resulting in a system of two ODEs. The purpose of this exercise is to implement the scaled model, test the software, and generalize the model.

a) Write a function `simulate` that can simulate an elastic pendulum using the scaled model. The function should have the following arguments:

```
def simulate(
    beta=0.9,                 # dimensionless parameter
    Theta=30,                 # initial angle in degrees
    epsilon=0,                # initial stretch of wire
    num_periods=6,            # simulate for num_periods
    time_steps_per_period=60, # time step resolution
    plot=True,                # make plots or not
    ):
```

To set the total simulation time and the time step, we use our knowledge of the scaled, classical, non-elastic pendulum: $u'' + u = 0$, with solution $u = \Theta \cos \bar{t}$. The period of these oscillations is $P = 2\pi$ and the frequency is unity. The time for simulation is taken as `num_periods` times P. The time step is set as P divided by `time_steps_per_period`.

The `simulate` function should return the arrays of x, y, θ, and t, where $\theta = \tan^{-1}(x/(1-y))$ is the angular displacement of the elastic pendulum corresponding to the position (x, y).

If `plot` is `True`, make a plot of $\bar{y}(\bar{t})$ versus $\bar{x}(\bar{t})$, i.e., the physical motion of the mass at $(\bar{x}, \bar{y})$. Use the equal aspect ratio on the axis such that we get a physically correct picture of the motion. Also make a plot of $\theta(\bar{t})$, where θ is measured in degrees. If $\Theta < 10$ degrees, add a plot that compares the solutions of the scaled, classical, non-elastic pendulum and the elastic pendulum ($\theta(t)$).

Although the mathematics here employs a bar over scaled quantities, the code should feature plain names `x` for $\bar{x}$, `y` for $\bar{y}$, and `t` for $\bar{t}$ (rather than `x_bar`, etc.). These variable names make the code easier to read and compare with the mathematics.

Hint 1. Equal aspect ratio is set by `plt.gca().set_aspect('equal')` in Matplotlib (`import matplotlib.pyplot as plt`) and in SciTools by the command `plt.plot(..., daspect=[1,1,1], daspectmode='equal')` (provided you have done `import scitools.std as plt`).

Hint 2. If you want to use Odespy to solve the equations, order the ODEs like $\dot{\bar{x}}, \bar{x}, \dot{\bar{y}}, \bar{y}$ such that `odespy.EulerCromer` can be applied.

b) Write a test function for testing that $\Theta = 0$ and $\epsilon = 0$ gives $x = y = 0$ for all times.

c) Write another test function for checking that the pure vertical motion of the elastic pendulum is correct. Start with simplifying the ODEs for pure vertical motion and show that $\bar{y}(\bar{t})$ fulfills a vibration equation with frequency $\sqrt{\beta/(1-\beta)}$. Set up the exact solution.

Write a test function that uses this special case to verify the `simulate` function. There will be numerical approximation errors present in the results from `simulate` so you have to believe in correct results and set a (low) tolerance that corresponds to the computed maximum error. Use a small Δt to obtain a small numerical approximation error.

d) Make a function `demo(beta, Theta)` for simulating an elastic pendulum with a given β parameter and initial angle Θ. Use 600 time steps per period to get every accurate results, and simulate for 3 periods.
Filename: `elastic_pendulum`.

Exercise 1.27: Simulate an elastic pendulum with air resistance

This is a continuation Exercise 1.26. Air resistance on the body with mass m can be modeled by the force $-\frac{1}{2}\varrho C_D A|\boldsymbol{v}|\boldsymbol{v}$, where C_D is a drag coefficient (0.2 for a sphere), ϱ is the density of air (1.2 kg m^{-3}), A is the cross section area ($A = \pi R^2$ for a sphere, where R is the radius), and $\boldsymbol{v}$ is the velocity of the body. Include air resistance in the original model, scale the model, write a function `simulate_drag` that is a copy of the `simulate` function from Exercise 1.26, but with the new ODEs included, and show plots of how air resistance influences the motion.
Filename: `elastic_pendulum_drag`.

Remarks. Test functions are challenging to construct for the problem with air resistance. You can reuse the tests from Exercise 1.27 for `simulate_drag`, but these tests does not verify the new terms arising from air resistance.

Exercise 1.28: Implement the PEFRL algorithm

We consider the motion of a planet around a star (Section 1.12.10). The simplified case where one mass is very much bigger than the other and one object is at rest, results in the scaled ODE model

$$\ddot{x} + (x^2 + y^2)^{-3/2}x = 0,$$
$$\ddot{y} + (x^2 + y^2)^{-3/2}y = 0\,.$$

a) It is easy to show that $x(t)$ and $y(t)$ go like sine and cosine functions. Use this idea to derive the exact solution.

b) One believes that a planet may orbit a star for billions of years. We are now interested in how accurate methods we actually need for such calculations. A first task is to determine what the time interval of interest is in scaled units. Take the earth and sun as typical objects and find the characteristic time used in the scaling of the equations ($t_c = \sqrt{L^3/(mG)}$), where m is the mass of the sun, L is the distance between the sun and the earth, and G is the gravitational constant. Find the scaled time interval corresponding to one billion years.

c) Solve the equations using 4th-order Runge-Kutta and the Euler-Cromer methods. You may benefit from applying Odespy for this purpose. With each solver, simulate 10000 orbits and print the maximum position error and CPU time as a function of time step. Note that the maximum position error does not necessarily occur at the end of the simulation. The position error achieved with each solver will depend heavily on the size of the time step. Let the time step correspond to 200, 400, 800 and 1600 steps per orbit, respectively. Are the results as expected? Explain briefly. When you develop your program, have in mind that it will be extended with an implementation of the other algorithms (as requested in d) and e) later) and experiments with this algorithm as well.

d) Implement a solver based on the PEFRL method from Section 1.10.11. Verify its 4th-order convergence using an equation $u'' + u = 0$.

e) The simulations done previously with the 4th-order Runge-Kutta and Euler-Cromer are now to be repeated with the PEFRL solver, so the code must be extended accordingly. Then run the simulations and comment on the performance of PEFRL compared to the other two.

f) Use the PEFRL solver to simulate 100000 orbits with a fixed time step corresponding to 1600 steps per period. Record the maximum error within each subsequent group of 1000 orbits. Plot these errors and fit (least squares) a mathematical function to the data. Print also the total CPU time spent for all 100000 orbits.

Now, predict the error and required CPU time for a simulation of 1 billion years (orbits). Is it feasible on today's computers to simulate the planetary motion for one billion years?
Filename: `vib_PEFRL`.

Remarks. This exercise investigates whether it is feasible to predict planetary motion for the life time of a solar system.

Wave equations

2

A very wide range of physical processes lead to wave motion, where signals are propagated through a medium in space and time, normally with little or no permanent movement of the medium itself. The shape of the signals may undergo changes as they travel through matter, but usually not so much that the signals cannot be recognized at some later point in space and time. Many types of wave motion can be described by the equation $u_{tt} = \nabla \cdot (c^2 \nabla u) + f$, which we will solve in the forthcoming text by finite difference methods.

2.1 Simulation of waves on a string

We begin our study of wave equations by simulating one-dimensional waves on a string, say on a guitar or violin. Let the string in the undeformed state coincide with the interval $[0, L]$ on the x axis, and let $u(x, t)$ be the displacement at time t in the y direction of a point initially at x. The displacement function u is governed by the mathematical model

© 2017, Hans Petter Langtangen, Svein Linge. Released under CC Attribution 4.0 license

$$\frac{\partial^2 u}{\partial t^2} = c^2 \frac{\partial^2 u}{\partial x^2}, \qquad x \in (0, L),\ t \in (0, T] \tag{2.1}$$

$$u(x, 0) = I(x), \qquad x \in [0, L] \tag{2.2}$$

$$\frac{\partial}{\partial t} u(x, 0) = 0, \qquad x \in [0, L] \tag{2.3}$$

$$u(0, t) = 0, \qquad t \in (0, T] \tag{2.4}$$

$$u(L, t) = 0, \qquad t \in (0, T] \tag{2.5}$$

The constant c and the function $I(x)$ must be prescribed.

Equation (2.1) is known as the one-dimensional *wave equation*. Since this PDE contains a second-order derivative in time, we need *two initial conditions*. The condition (2.2) specifies the initial shape of the string, $I(x)$, and (2.3) expresses that the initial velocity of the string is zero. In addition, PDEs need *boundary conditions*, given here as (2.4) and (2.5). These two conditions specify that the string is fixed at the ends, i.e., that the displacement u is zero.

The solution $u(x, t)$ varies in space and time and describes waves that move with velocity c to the left and right.

Sometimes we will use a more compact notation for the partial derivatives to save space:

$$u_t = \frac{\partial u}{\partial t}, \quad u_{tt} = \frac{\partial^2 u}{\partial t^2}, \tag{2.6}$$

and similar expressions for derivatives with respect to other variables. Then the wave equation can be written compactly as $u_{tt} = c^2 u_{xx}$.

The PDE problem (2.1)-(2.5) will now be discretized in space and time by a finite difference method.

2.1.1 Discretizing the domain

The temporal domain $[0, T]$ is represented by a finite number of mesh points

$$0 = t_0 < t_1 < t_2 < \cdots < t_{N_t - 1} < t_{N_t} = T\,. \tag{2.7}$$

Similarly, the spatial domain $[0, L]$ is replaced by a set of mesh points

$$0 = x_0 < x_1 < x_2 < \cdots < x_{N_x - 1} < x_{N_x} = L\,. \tag{2.8}$$

One may view the mesh as two-dimensional in the x, t plane, consisting of points (x_i, t_n), with $i = 0, \ldots, N_x$ and $n = 0, \ldots, N_t$.

Uniform meshes. For uniformly distributed mesh points we can introduce the constant mesh spacings Δt and Δx. We have that

$$x_i = i\Delta x,\ i = 0, \ldots, N_x, \quad t_n = n\Delta t,\ n = 0, \ldots, N_t\,. \tag{2.9}$$

We also have that $\Delta x = x_i - x_{i-1}$, $i = 1, \ldots, N_x$, and $\Delta t = t_n - t_{n-1}$, $n = 1, \ldots, N_t$. Figure 2.1 displays a mesh in the x, t plane with $N_t = 5$, $N_x = 5$, and constant mesh spacings.

2.1.2 The discrete solution

The solution $u(x, t)$ is sought at the mesh points. We introduce the mesh function u_i^n, which approximates the exact solution at the mesh point (x_i, t_n) for $i = 0, \ldots, N_x$ and $n = 0, \ldots, N_t$. Using the finite difference method, we shall develop algebraic equations for computing the mesh function.

2.1.3 Fulfilling the equation at the mesh points

In the finite difference method, we relax the condition that (2.1) holds at all points in the space-time domain $(0, L) \times (0, T]$ to the requirement that the PDE is fulfilled at the *interior* mesh points only:

$$\frac{\partial^2}{\partial t^2} u(x_i, t_n) = c^2 \frac{\partial^2}{\partial x^2} u(x_i, t_n), \tag{2.10}$$

for $i = 1, \ldots, N_x - 1$ and $n = 1, \ldots, N_t - 1$. For $n = 0$ we have the initial conditions $u = I(x)$ and $u_t = 0$, and at the boundaries $i = 0, N_x$ we have the boundary condition $u = 0$.

2.1.4 Replacing derivatives by finite differences

The second-order derivatives can be replaced by central differences. The most widely used difference approximation of the second-order derivative is

$$\frac{\partial^2}{\partial t^2} u(x_i, t_n) \approx \frac{u_i^{n+1} - 2u_i^n + u_i^{n-1}}{\Delta t^2}\,.$$

It is convenient to introduce the finite difference operator notation

$$[D_t D_t u]_i^n = \frac{u_i^{n+1} - 2u_i^n + u_i^{n-1}}{\Delta t^2}\,.$$

A similar approximation of the second-order derivative in the x direction reads

$$\frac{\partial^2}{\partial x^2}u(x_i, t_n) \approx \frac{u_{i+1}^n - 2u_i^n + u_{i-1}^n}{\Delta x^2} = [D_x D_x u]_i^n\,.$$

Algebraic version of the PDE. We can now replace the derivatives in (2.10) and get

$$\frac{u_i^{n+1} - 2u_i^n + u_i^{n-1}}{\Delta t^2} = c^2 \frac{u_{i+1}^n - 2u_i^n + u_{i-1}^n}{\Delta x^2}, \tag{2.11}$$

or written more compactly using the operator notation:

$$[D_t D_t u = c^2 D_x D_x]_i^n\,. \tag{2.12}$$

Interpretation of the equation as a stencil. A characteristic feature of (2.11) is that it involves u values from neighboring points only: u_i^{n+1}, $u_{i\pm1}^n$, u_i^n, and u_i^{n-1}. The circles in Figure 2.1 illustrate such neighboring mesh points that contribute to an algebraic equation. In this particular case, we have sampled the PDE at the point $(2, 2)$ and constructed (2.11), which then involves a coupling of u_1^2, u_2^3, u_2^2, u_2^1, and u_3^2. The term *stencil* is often used about the algebraic equation at a mesh point, and the geometry of a typical stencil is illustrated in Figure 2.1. One also often refers to the algebraic equations as *discrete equations*, *(finite) difference equations* or a *finite difference scheme*.

Algebraic version of the initial conditions. We also need to replace the derivative in the initial condition (2.3) by a finite difference approximation. A centered difference of the type

$$\frac{\partial}{\partial t}u(x_i, t_0) \approx \frac{u_i^1 - u_i^{-1}}{2\Delta t} = [D_{2t} u]_i^0,$$

seems appropriate. Writing out this equation and ordering the terms give

$$u_i^{-1} = u_i^1, \quad i = 0, \ldots, N_x\,. \tag{2.13}$$

The other initial condition can be computed by

$$u_i^0 = I(x_i), \quad i = 0, \ldots, N_x\,.$$

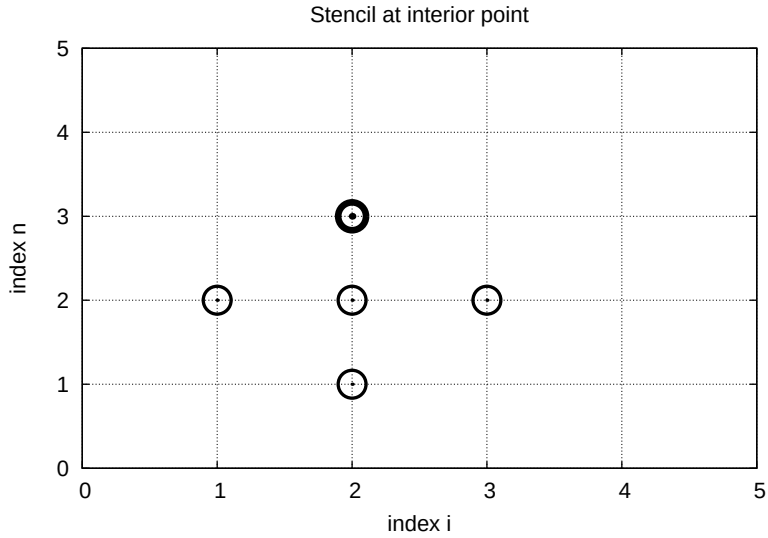


Fig. 2.1 Mesh in space and time. The circles show points connected in a finite difference equation.

2.1.5 Formulating a recursive algorithm

We assume that u_i^n and u_i^{n-1} are available for $i = 0, \ldots, N_x$. The only unknown quantity in (2.11) is therefore u_i^{n+1}, which we now can solve for:

$$u_i^{n+1} = -u_i^{n-1} + 2u_i^n + C^2 \left(u_{i+1}^n - 2u_i^n + u_{i-1}^n\right) . \tag{2.14}$$

We have here introduced the parameter

$$C = c\frac{\Delta t}{\Delta x}, \tag{2.15}$$

known as the *Courant number*.

C is the key parameter in the discrete wave equation

We see that the discrete version of the PDE features only one parameter, C, which is therefore the key parameter, together with N_x, that governs the quality of the numerical solution (see Section 2.10 for details). Both the primary physical parameter c and the numerical parameters Δx and Δt are lumped together in C. Note that C is a dimensionless parameter.

Given that u_i^{n-1} and u_i^n are known for $i = 0, \ldots, N_x$, we find new values at the next time level by applying the formula (2.14) for $i = 1, \ldots, N_x - 1$. Figure 2.1 illustrates the points that are used to compute u_2^3. For the boundary points, $i = 0$ and $i = N_x$, we apply the boundary conditions $u_i^{n+1} = 0$.

Even though sound reasoning leads up to (2.14), there is still a minor challenge with it that needs to be resolved. Think of the very first computational step to be made. The scheme (2.14) is supposed to start at $n = 1$, which means that we compute u^2 from u^1 and u^0. Unfortunately, we do not know the value of u^1, so how to proceed? A standard procedure in such cases is to apply (2.14) also for $n = 0$. This immediately seems strange, since it involves u_i^{-1}, which is an undefined quantity outside the time mesh (and the time domain). However, we can use the initial condition (2.13) in combination with (2.14) when $n = 0$ to eliminate u_i^{-1} and arrive at a special formula for u_i^1:

$$u_i^1 = u_i^0 - \frac{1}{2}C^2\left(u_{i+1}^0 - 2u_i^0 + u_{i-1}^0\right) . \tag{2.16}$$

Figure 2.2 illustrates how (2.16) connects four instead of five points: u_2^1, u_1^0, u_2^0, and u_3^0.

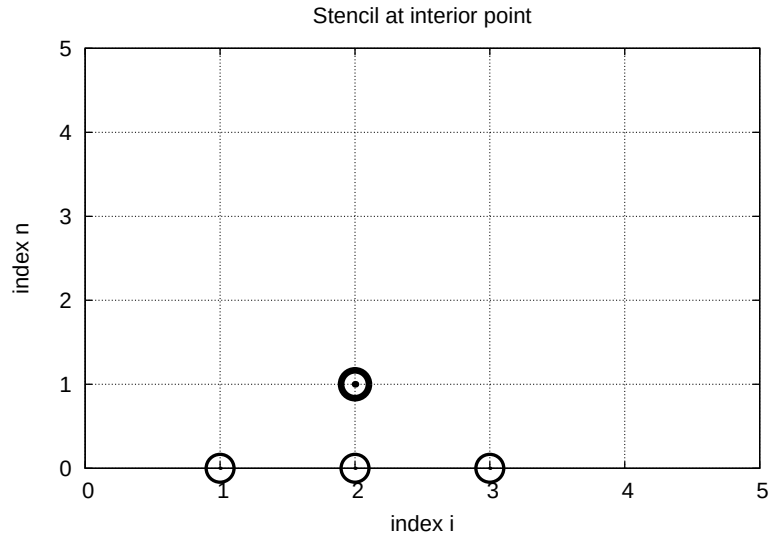


Fig. 2.2 Modified stencil for the first time step.

We can now summarize the computational algorithm:

1. Compute $u_i^0 = I(x_i)$ for $i = 0, \ldots, N_x$
2. Compute u_i^1 by (2.16) for $i = 1, 2, \ldots, N_x - 1$ and set $u_i^1 = 0$ for the boundary points given by $i = 0$ and $i = N_x$,
3. For each time level $n = 1, 2, \ldots, N_t - 1$
   a. apply (2.14) to find u_i^{n+1} for $i = 1, \ldots, N_x - 1$
   b. set $u_i^{n+1} = 0$ for the boundary points having $i = 0$, $i = N_x$.

The algorithm essentially consists of moving a finite difference stencil through all the mesh points, which can be seen as an animation in a web page[1] or a movie file[2].

2.1.6 Sketch of an implementation

The algorithm only involves the three most recent time levels, so we need only three arrays for u_i^{n+1}, u_i^n, and u_i^{n-1}, $i = 0, \ldots, N_x$. Storing all the solutions in a two-dimensional array of size $(N_x + 1) \times (N_t + 1)$ would be possible in this simple one-dimensional PDE problem, but is normally out of the question in three-dimensional (3D) and large two-dimensional (2D) problems. We shall therefore, in all our PDE solving programs, have the unknown in memory at as few time levels as possible.

In a Python implementation of this algorithm, we use the array elements `u[i]` to store u_i^{n+1}, `u_n[i]` to store u_i^n, and `u_nm1[i]` to store u_i^{n-1}.

The following Python snippet realizes the steps in the computational algorithm.

```
# Given mesh points as arrays x and t (x[i], t[n])
dx = x[1] - x[0]
dt = t[1] - t[0]
C = c*dt/dx             # Courant number
Nt = len(t)-1
C2 = C**2               # Help variable in the scheme

# Set initial condition u(x,0) = I(x)
for i in range(0, Nx+1):
    u_n[i] = I(x[i])

# Apply special formula for first step, incorporating du/dt=0
for i in range(1, Nx):
    u[i] = u_n[i] - \
           0.5*C**2(u_n[i+1] - 2*u_n[i] + u_n[i-1])
```

[1] http://tinyurl.com/hbcasmj/book/html/mov-wave/D_stencil_gpl/index.html
[2] http://tinyurl.com/gokgkov/mov-wave/D_stencil_gpl/movie.ogg

```
u[0] = 0;  u[Nx] = 0   # Enforce boundary conditions

# Switch variables before next step
u_nm1[:], u_n[:] = u_n, u

for n in range(1, Nt):
    # Update all inner mesh points at time t[n+1]
    for i in range(1, Nx):
        u[i] = 2u_n[i] - u_nm1[i] - \
               C**2(u_n[i+1] - 2*u_n[i] + u_n[i-1])

    # Insert boundary conditions
    u[0] = 0;  u[Nx] = 0

    # Switch variables before next step
    u_nm1[:], u_n[:] = u_n, u
```

2.2 Verification

Before implementing the algorithm, it is convenient to add a source term to the PDE (2.1), since that gives us more freedom in finding test problems for verification. Physically, a source term acts as a generator for waves in the interior of the domain.

2.2.1 A slightly generalized model problem

We now address the following extended initial-boundary value problem for one-dimensional wave phenomena:

$$u_{tt} = c^2 u_{xx} + f(x,t), \qquad x \in (0,L),\ t \in (0,T] \tag{2.17}$$

$$u(x,0) = I(x), \qquad x \in [0,L] \tag{2.18}$$

$$u_t(x,0) = V(x), \qquad x \in [0,L] \tag{2.19}$$

$$u(0,t) = 0, \qquad t > 0 \tag{2.20}$$

$$u(L,t) = 0, \qquad t > 0 \tag{2.21}$$

Sampling the PDE at (x_i, t_n) and using the same finite difference approximations as above, yields

$$[D_t D_t u = c^2 D_x D_x u + f]_i^n \,. \tag{2.22}$$

Writing this out and solving for the unknown u_i^{n+1} results in

$$u_i^{n+1} = -u_i^{n-1} + 2u_i^n + C^2(u_{i+1}^n - 2u_i^n + u_{i-1}^n) + \Delta t^2 f_i^n \,. \tag{2.23}$$

The equation for the first time step must be rederived. The discretization of the initial condition $u_t = V(x)$ at $t = 0$ becomes

$$[D_{2t}u = V]_i^0 \quad \Rightarrow \quad u_i^{-1} = u_i^1 - 2\Delta t V_i,$$

which, when inserted in (2.23) for $n = 0$, gives the special formula

$$u_i^1 = u_i^0 - \Delta t V_i + \frac{1}{2}C^2\left(u_{i+1}^0 - 2u_i^0 + u_{i-1}^0\right) + \frac{1}{2}\Delta t^2 f_i^0 \,. \tag{2.24}$$

2.2.2 Using an analytical solution of physical significance

Many wave problems feature sinusoidal oscillations in time and space. For example, the original PDE problem (2.1)-(2.5) allows an exact solution

$$u_{\mathrm{e}}(x,t) = A\sin\left(\frac{\pi}{L}x\right)\cos\left(\frac{\pi}{L}ct\right) \,. \tag{2.25}$$

This u_{e} fulfills the PDE with $f = 0$, boundary conditions $u_{\mathrm{e}}(0,t) = u_{\mathrm{e}}(L,t) = 0$, as well as initial conditions $I(x) = A\sin\left(\frac{\pi}{L}x\right)$ and $V = 0$.

How to use exact solutions for verification

It is common to use such exact solutions of physical interest to verify implementations. However, the numerical solution u_i^n will only be an approximation to $u_{\mathrm{e}}(x_i, t_n)$. We have no knowledge of the precise size of the error in this approximation, and therefore we can never know if discrepancies between u_i^n and $u_{\mathrm{e}}(x_i, t_n)$ are caused by mathematical approximations or programming errors. In particular, if plots of the computed solution u_i^n and the exact one (2.25) look similar, many are tempted to claim that the implementation works. However, even if color plots look nice and the accuracy is "deemed good", there can still be serious programming errors present!

The only way to use exact physical solutions like (2.25) for serious and thorough verification is to run a series of simulations on finer and finer meshes, measure the integrated error in each mesh, and

from this information estimate the empirical convergence rate of the method.

An introduction to the computing of convergence rates is given in Section 3.1.6 in [9]. There is also a detailed example on computing convergence rates in Section 1.2.2.

In the present problem, one expects the method to have a convergence rate of 2 (see Section 2.10), so if the computed rates are close to 2 on a sufficiently fine mesh, we have good evidence that the implementation is free of programming mistakes.

2.2.3 Manufactured solution and estimation of convergence rates

Specifying the solution and computing corresponding data. One problem with the exact solution (2.25) is that it requires a simplification ($V = 0, f = 0$) of the implemented problem (2.17)-(2.21). An advantage of using a *manufactured solution* is that we can test all terms in the PDE problem. The idea of this approach is to set up some chosen solution and fit the source term, boundary conditions, and initial conditions to be compatible with the chosen solution. Given that our boundary conditions in the implementation are $u(0,t) = u(L,t) = 0$, we must choose a solution that fulfills these conditions. One example is

$$u_{\mathrm{e}}(x,t) = x(L-x)\sin t\,.$$

Inserted in the PDE $u_{tt} = c^2 u_{xx} + f$ we get

$$-x(L-x)\sin t = -c^2 2\sin t + f \quad \Rightarrow f = (2c^2 - x(L-x))\sin t\,.$$

The initial conditions become

$$\begin{aligned} u(x,0) =& I(x) = 0, \\ u_t(x,0) =& V(x) = x(L-x)\,. \end{aligned}$$

Defining a single discretization parameter. To verify the code, we compute the convergence rates in a series of simulations, letting each simulation use a finer mesh than the previous one. Such empirical estima-

tion of convergence rates relies on an assumption that some measure E of the numerical error is related to the discretization parameters through

$$E = C_t \Delta t^r + C_x \Delta x^p,$$

where C_t, C_x, r, and p are constants. The constants r and p are known as the *convergence rates* in time and space, respectively. From the accuracy in the finite difference approximations, we expect $r = p = 2$, since the error terms are of order Δt^2 and Δx^2. This is confirmed by truncation error analysis and other types of analysis.

By using an exact solution of the PDE problem, we will next compute the error measure E on a sequence of refined meshes and see if the rates $r = p = 2$ are obtained. We will not be concerned with estimating the constants C_t and C_x, simply because we are not interested in their values.

It is advantageous to introduce a single discretization parameter $h = \Delta t = \hat{c}\Delta x$ for some constant $\hat{c}$. Since Δt and Δx are related through the Courant number, $\Delta t = C\Delta x/c$, we set $h = \Delta t$, and then $\Delta x = hc/C$. Now the expression for the error measure is greatly simplified:

$$E = C_t \Delta t^r + C_x \Delta x^r = C_t h^r + C_x \left(\frac{c}{C}\right)^r h^r = Dh^r, \quad D = C_t + C_x \left(\frac{c}{C}\right)^r.$$

Computing errors. We choose an initial discretization parameter h_0 and run experiments with decreasing h: $h_i = 2^{-i}h_0$, $i = 1, 2, \ldots, m$. Halving h in each experiment is not necessary, but it is a common choice. For each experiment we must record E and h. Standard choices of error measure are the ℓ^2 and ℓ^∞ norms of the error mesh function e_i^n:

$$E = ||e_i^n||_{\ell^2} = \left(\Delta t \Delta x \sum_{n=0}^{N_t} \sum_{i=0}^{N_x} (e_i^n)^2\right)^{\frac{1}{2}}, \quad e_i^n = u_{\mathrm{e}}(x_i, t_n) - u_i^n, \quad (2.26)$$

$$E = ||e_i^n||_{\ell^\infty} = \max_{i,n} |e_i^n|. \quad (2.27)$$

In Python, one can compute $\sum_i (e_i^n)^2$ at each time step and accumulate the value in some sum variable, say `e2_sum`. At the final time step one can do `sqrt(dt*dx*e2_sum)`. For the ℓ^∞ norm one must compare the maximum error at a time level (`e.max()`) with the global maximum over the time domain: `e_max = max(e_max, e.max())`.

An alternative error measure is to use a spatial norm at one time step only, e.g., the end time T ($n = N_t$):

$$E = ||e_i^n||_{\ell^2} = \left(\Delta x \sum_{i=0}^{N_x} (e_i^n)^2 \right)^{\frac{1}{2}}, \quad e_i^n = u_{\mathrm{e}}(x_i, t_n) - u_i^n, \tag{2.28}$$

$$E = ||e_i^n||_{\ell^\infty} = \max_{0 \leq i \leq N_x} |e_i^n| \,. \tag{2.29}$$

The important point is that the error measure (E) for the simulation is represented by a single number.

Computing rates. Let E_i be the error measure in experiment (mesh) number i (not to be confused with the spatial index i) and let h_i be the corresponding discretization parameter (h). With the error model $E_i = Dh_i^r$, we can estimate r by comparing two consecutive experiments:

$$E_{i+1} = Dh_{i+1}^r,$$
$$E_i = Dh_i^r \,.$$

Dividing the two equations eliminates the (uninteresting) constant D. Thereafter, solving for r yields

$$r = \frac{\ln E_{i+1}/E_i}{\ln h_{i+1}/h_i} \,.$$

Since r depends on i, i.e., which simulations we compare, we add an index to r: r_i, where $i = 0, \ldots, m-2$, if we have m experiments: $(h_0, E_0), \ldots, (h_{m-1}, E_{m-1})$.

In our present discretization of the wave equation we expect $r = 2$, and hence the r_i values should converge to 2 as i increases.

2.2.4 Constructing an exact solution of the discrete equations

With a manufactured or known analytical solution, as outlined above, we can estimate convergence rates and see if they have the correct asymptotic behavior. Experience shows that this is a quite good verification technique in that many common bugs will destroy the convergence rates. A significantly better test though, would be to check that the numerical solution is exactly what it should be. This will in general require exact knowledge of the numerical error, which we do not normally have (although we in Section 2.10 establish such knowledge in simple cases).

However, it is possible to look for solutions where we can show that the numerical error vanishes, i.e., the solution of the original continuous PDE problem is also a solution of the discrete equations. This property often arises if the exact solution of the PDE is a lower-order polynomial. (Truncation error analysis leads to error measures that involve derivatives of the exact solution. In the present problem, the truncation error involves 4th-order derivatives of u in space and time. Choosing u as a polynomial of degree three or less will therefore lead to vanishing error.)

We shall now illustrate the construction of an exact solution to both the PDE itself and the discrete equations. Our chosen manufactured solution is quadratic in space and linear in time. More specifically, we set

$$u_{\mathrm{e}}(x,t) = x(L-x)(1+\frac{1}{2}t), \tag{2.30}$$

which by insertion in the PDE leads to $f(x,t) = 2(1+t)c^2$. This u_{e} fulfills the boundary conditions $u = 0$ and demands $I(x) = x(L-x)$ and $V(x) = \frac{1}{2}x(L-x)$.

To realize that the chosen u_{e} is also an exact solution of the discrete equations, we first remind ourselves that $t_n = n\Delta t$ so that

$$[D_tD_tt^2]^n = \frac{t_{n+1}^2 - 2t_n^2 + t_{n-1}^2}{\Delta t^2} = (n+1)^2 - 2n^2 + (n-1)^2 = 2, \tag{2.31}$$

$$[D_tD_tt]^n = \frac{t_{n+1} - 2t_n + t_{n-1}}{\Delta t^2} = \frac{((n+1) - 2n + (n-1))\Delta t}{\Delta t^2} = 0\,. \tag{2.32}$$

Hence,

$$[D_tD_tu_{\mathrm{e}}]_i^n = x_i(L-x_i)[D_tD_t(1+\frac{1}{2}t)]^n = x_i(L-x_i)\frac{1}{2}[D_tD_tt]^n = 0\,.$$

Similarly, we get that

$$\begin{aligned}
[D_xD_xu_{\mathrm{e}}]_i^n &= (1+\frac{1}{2}t_n)[D_xD_x(xL-x^2)]_i \\
&= (1+\frac{1}{2}t_n)[LD_xD_xx - D_xD_xx^2]_i \\
&= -2(1+\frac{1}{2}t_n)\,.
\end{aligned}$$

Now, $f_i^n = 2(1 + \frac{1}{2}t_n)c^2$, which results in

$$[D_tD_tu_\mathrm{e} - c^2D_xD_xu_\mathrm{e} - f]_i^n = 0 + c^2 2(1 + \frac{1}{2}t_n) + 2(1 + \frac{1}{2}t_n)c^2 = 0\,.$$

Moreover, $u_\mathrm{e}(x_i, 0) = I(x_i)$, $\partial u_\mathrm{e}/\partial t = V(x_i)$ at $t = 0$, and $u_\mathrm{e}(x_0, t) = u_\mathrm{e}(x_{N_x}, 0) = 0$. Also the modified scheme for the first time step is fulfilled by $u_\mathrm{e}(x_i, t_n)$.

Therefore, the exact solution $u_\mathrm{e}(x, t) = x(L - x)(1 + t/2)$ of the PDE problem is also an exact solution of the discrete problem. This means that we know beforehand what numbers the numerical algorithm should produce. We can use this fact to check that the computed u_i^n values from an implementation equals $u_\mathrm{e}(x_i, t_n)$, within machine precision. This result is valid *regardless of the mesh spacings* Δx and Δt! Nevertheless, there might be stability restrictions on Δx and Δt, so the test can only be run for a mesh that is compatible with the stability criterion (which in the present case is $C \leq 1$, to be derived later).

Notice

A product of quadratic or linear expressions in the various independent variables, as shown above, will often fulfill both the PDE problem and the discrete equations, and can therefore be very useful solutions for verifying implementations.

However, for 1D wave equations of the type $u_{tt} = c^2u_{xx}$ we shall see that there is always another much more powerful way of generating exact solutions (which consists in just setting $C = 1$ (!), as shown in Section 2.10).

2.3 Implementation

This section presents the complete computational algorithm, its implementation in Python code, animation of the solution, and verification of the implementation.

A real implementation of the basic computational algorithm from Sections 2.1.5 and 2.1.6 can be encapsulated in a function, taking all the input data for the problem as arguments. The physical input data

consists of c, $I(x)$, $V(x)$, $f(x,t)$, L, and T. The numerical input is the mesh parameters Δt and Δx.

Instead of specifying Δt *and* Δx, we can specify one of them and the Courant number C instead, since having explicit control of the Courant number is convenient when investigating the numerical method. Many find it natural to prescribe the resolution of the spatial grid and set N_x. The solver function can then compute $\Delta t = CL/(cN_x)$. However, for comparing $u(x,t)$ curves (as functions of x) for various Courant numbers it is more convenient to keep Δt fixed for all C and let Δx vary according to $\Delta x = c\Delta t/C$. With Δt fixed, all frames correspond to the same time t, and this simplifies animations that compare simulations with different mesh resolutions. Plotting functions of x with different spatial resolution is trivial, so it is easier to let Δx vary in the simulations than Δt.

2.3.1 Callback function for user-specific actions

The solution at all spatial points at a new time level is stored in an array `u` of length $N_x + 1$. We need to decide what to do with this solution, e.g., visualize the curve, analyze the values, or write the array to file for later use. The decision about what to do is left to the user in the form of a user-supplied function

```
user_action(u, x, t, n)
```

where `u` is the solution at the spatial points `x` at time `t[n]`. The `user_action` function is called from the solver at each time level `n`.

If the user wants to plot the solution or store the solution at a time point, she needs to write such a function and take appropriate actions inside it. We will show examples on many such `user_action` functions.

Since the solver function makes calls back to the user's code via such a function, this type of function is called a *callback function*. When writing general software, like our solver function, which also needs to carry out special problem- or solution-dependent actions (like visualization), it is a common technique to leave those actions to user-supplied callback functions.

The callback function can be used to terminate the solution process if the user returns `True`. For example,

```
def my_user_action_function(u, x, t, n):
    return np.abs(u).max() > 10
```

is a callback function that will terminate the solver function (given below) of the amplitude of the waves exceed 10, which is here considered as a numerical instability.

2.3.2 The solver function

A first attempt at a solver function is listed below.

```
import numpy as np

def solver(I, V, f, c, L, dt, C, T, user_action=None):
    """Solve u_tt=c^2*u_xx + f on (0,L)x(0,T]."""
    Nt = int(round(T/dt))
    t = np.linspace(0, Nt*dt, Nt+1)   # Mesh points in time
    dx = dt*c/float(C)
    Nx = int(round(L/dx))
    x = np.linspace(0, L, Nx+1)       # Mesh points in space
    C2 = C**2                         # Help variable in the scheme
    # Make sure dx and dt are compatible with x and t
    dx = x[1] - x[0]
    dt = t[1] - t[0]

    if f is None or f == 0 :
        f = lambda x, t: 0
    if V is None or V == 0:
        V = lambda x: 0

    u     = np.zeros(Nx+1)   # Solution array at new time level
    u_n   = np.zeros(Nx+1)   # Solution at 1 time level back
    u_nm1 = np.zeros(Nx+1)   # Solution at 2 time levels back

    import time;  t0 = time.clock()  # Measure CPU time

    # Load initial condition into u_n
    for i in range(0,Nx+1):
        u_n[i] = I(x[i])

    if user_action is not None:
        user_action(u_n, x, t, 0)

    # Special formula for first time step
    n = 0
    for i in range(1, Nx):
        u[i] = u_n[i] + dt*V(x[i]) + \
               0.5*C2*(u_n[i-1] - 2*u_n[i] + u_n[i+1]) + \
               0.5*dt**2*f(x[i], t[n])
    u[0] = 0;  u[Nx] = 0

    if user_action is not None:
        user_action(u, x, t, 1)
```

```
    # Switch variables before next step
    u_nm1[:] = u_n;  u_n[:] = u

    for n in range(1, Nt):
        # Update all inner points at time t[n+1]
        for i in range(1, Nx):
            u[i] = - u_nm1[i] + 2*u_n[i] + \
                     C2*(u_n[i-1] - 2*u_n[i] + u_n[i+1]) + \
                     dt**2*f(x[i], t[n])

        # Insert boundary conditions
        u[0] = 0;  u[Nx] = 0
        if user_action is not None:
            if user_action(u, x, t, n+1):
                break

        # Switch variables before next step
        u_nm1[:] = u_n;  u_n[:] = u

    cpu_time = time.clock() - t0
    return u, x, t, cpu_time
```

A couple of remarks about the above code is perhaps necessary:

- Although we give `dt` and compute `dx` via `C` and `c`, the resulting `t` and `x` meshes do not necessarily correspond exactly to these values because of rounding errors. To explicitly ensure that `dx` and `dt` correspond to the cell sizes in `x` and `t`, we recompute the values.
- According to the particular choice made in Section 2.3.1, a true value returned from `user_action` should terminate the simulation. This is here implemented by a `break` statement inside the for loop in the solver.

2.3.3 Verification: exact quadratic solution

We use the test problem derived in Section 2.2.1 for verification. Below is a unit test based on this test problem and realized as a proper *test function* compatible with the unit test frameworks nose or pytest.

```
def test_quadratic():
    """Check that u(x,t)=x(L-x)(1+t/2) is exactly reproduced."""

    def u_exact(x, t):
        return x*(L-x)*(1 + 0.5*t)

    def I(x):
        return u_exact(x, 0)
```

```
    def V(x):
        return 0.5*u_exact(x, 0)

    def f(x, t):
        return 2*(1 + 0.5*t)*c**2

    L = 2.5
    c = 1.5
    C = 0.75
    Nx = 6  # Very coarse mesh for this exact test
    dt = C*(L/Nx)/c
    T = 18

    def assert_no_error(u, x, t, n):
        u_e = u_exact(x, t[n])
        diff = np.abs(u - u_e).max()
        tol = 1E-13
        assert diff < tol

    solver(I, V, f, c, L, dt, C, T,
           user_action=assert_no_error)
```

When this function resides in the file `wave1D_u0.py`, one can run pytest to check that all test functions with names `test_*()` in this file work:

Terminal

```
Terminal> py.test -s -v wave1D_u0.py
```

2.3.4 Verification: convergence rates

A more general method, but not so reliable as a verification method, is to compute the convergence rates and see if they coincide with theoretical estimates. Here we expect a rate of 2 according to the various results in Section 2.10. A general function for computing convergence rates can be written like this:

```
def convergence_rates(
    u_exact,                # Python function for exact solution
    I, V, f, c, L,          # physical parameters
    dt0, num_meshes, C, T):  # numerical parameters
    """
    Half the time step and estimate convergence rates for
    for num_meshes simulations.
    """
    # First define an appropriate user action function
    global error
    error = 0  # error computed in the user action function

    def compute_error(u, x, t, n):
```

```
        global error  # must be global to be altered here
        # (otherwise error is a local variable, different
        # from error defined in the parent function)
        if n == 0:
            error = 0
        else:
            error = max(error, np.abs(u - u_exact(x, t[n])).max())

    # Run finer and finer resolutions and compute true errors
    E = []
    h = []  # dt, solver adjusts dx such that C=dt*c/dx
    dt = dt0
    for i in range(num_meshes):
        solver(I, V, f, c, L, dt, C, T,
               user_action=compute_error)
        # error is computed in the final call to compute_error
        E.append(error)
        h.append(dt)
        dt /= 2  # halve the time step for next simulation
    print 'E:', E
    print 'h:', h
    # Convergence rates for two consecutive experiments
    r = [np.log(E[i]/E[i-1])/np.log(h[i]/h[i-1])
         for i in range(1,num_meshes)]
    return r
```

Using the analytical solution from Section 2.2.2, we can call `convergence_rates` to see if we get a convergence rate that approaches 2 and use the final estimate of the rate in an `assert` statement such that this function becomes a proper test function:

```
def test_convrate_sincos():
    n = m = 2
    L = 1.0
    u_exact = lambda x, t: np.cos(m*np.pi/L*t)*np.sin(m*np.pi/L*x)

    r = convergence_rates(
        u_exact=u_exact,
        I=lambda x: u_exact(x, 0),
        V=lambda x: 0,
        f=0,
        c=1,
        L=L,
        dt0=0.1,
        num_meshes=6,
        C=0.9,
        T=1)
    print 'rates sin(x)*cos(t) solution:', \
          [round(r_,2) for r_ in r]
    assert abs(r[-1] - 2) < 0.002
```

Doing `py.test -s -v wave1D_u0.py` will run also this test function and show the rates 2.05, 1.98, 2.00, 2.00, and 2.00 (to two decimals).

2.3.5 Visualization: animating the solution

Now that we have verified the implementation it is time to do a real computation where we also display evolution of the waves on the screen. Since the `solver` function knows nothing about what type of visualizations we may want, it calls the callback function `user_action(u, x, t, n)`. We must therefore write this function and find the proper statements for plotting the solution.

Function for administering the simulation. The following `viz` function

1. defines a `user_action` callback function for plotting the solution at each time level,
2. calls the `solver` function, and
3. combines all the plots (in files) to video in different formats.

```
def viz(
    I, V, f, c, L, dt, C, T,  # PDE parameters
    umin, umax,               # Interval for u in plots
    animate=True,             # Simulation with animation?
    tool='matplotlib',        # 'matplotlib' or 'scitools'
    solver_function=solver,   # Function with numerical algorithm
    ):
    """Run solver and visualize u at each time level."""

    def plot_u_st(u, x, t, n):
        """user_action function for solver."""
        plt.plot(x, u, 'r-',
                 xlabel='x', ylabel='u',
                 axis=[0, L, umin, umax],
                 title='t=%f' % t[n], show=True)
        # Let the initial condition stay on the screen for 2
        # seconds, else insert a pause of 0.2 s between each plot
        time.sleep(2) if t[n] == 0 else time.sleep(0.2)
        plt.savefig('frame_%04d.png' % n)  # for movie making

    class PlotMatplotlib:
        def __call__(self, u, x, t, n):
            """user_action function for solver."""
            if n == 0:
                plt.ion()
                self.lines = plt.plot(x, u, 'r-')
                plt.xlabel('x');  plt.ylabel('u')
                plt.axis([0, L, umin, umax])
                plt.legend(['t=%f' % t[n]], loc='lower left')
            else:
                self.lines[0].set_ydata(u)
                plt.legend(['t=%f' % t[n]], loc='lower left')
                plt.draw()
            time.sleep(2) if t[n] == 0 else time.sleep(0.2)
```

```
            plt.savefig('tmp_%04d.png' % n)  # for movie making

    if tool == 'matplotlib':
        import matplotlib.pyplot as plt
        plot_u = PlotMatplotlib()
    elif tool == 'scitools':
        import scitools.std as plt  # scitools.easyviz interface
        plot_u = plot_u_st
    import time, glob, os

    # Clean up old movie frames
    for filename in glob.glob('tmp_*.png'):
        os.remove(filename)

    # Call solver and do the simulaton
    user_action = plot_u if animate else None
    u, x, t, cpu = solver_function(
        I, V, f, c, L, dt, C, T, user_action)

    # Make video files
    fps = 4  # frames per second
    codec2ext = dict(flv='flv', libx264='mp4', libvpx='webm',
                     libtheora='ogg')  # video formats
    filespec = 'tmp_%04d.png'
    movie_program = 'ffmpeg'  # or 'avconv'
    for codec in codec2ext:
        ext = codec2ext[codec]
        cmd = '%(movie_program)s -r %(fps)d -i %(filespec)s '\
              '-vcodec %(codec)s movie.%(ext)s' % vars()
        os.system(cmd)

    if tool == 'scitools':
        # Make an HTML play for showing the animation in a browser
        plt.movie('tmp_*.png', encoder='html', fps=fps,
                  output_file='movie.html')
    return cpu
```

Dissection of the code. The `viz` function can either use SciTools or Matplotlib for visualizing the solution. The `user_action` function based on SciTools is called `plot_u_st`, while the `user_action` function based on Matplotlib is a bit more complicated as it is realized as a class and needs statements that differ from those for making static plots. SciTools can utilize both Matplotlib and Gnuplot (and many other plotting programs) for doing the graphics, but Gnuplot is a relevant choice for large N_x or in two-dimensional problems as Gnuplot is significantly faster than Matplotlib for screen animations.

A function inside another function, like `plot_u_st` in the above code segment, has access to *and remembers* all the local variables in the surrounding code inside the `viz` function (!). This is known in computer

science as a *closure* and is very convenient to program with. For example, the `plt` and `time` modules defined outside `plot_u` are accessible for `plot_u_st` when the function is called (as `user_action`) in the `solver` function. Some may think, however, that a class instead of a closure is a cleaner and easier-to-understand implementation of the user action function, see Section 2.8.

The `plot_u_st` function just makes a standard SciTools `plot` command for plotting `u` as a function of `x` at time `t[n]`. To achieve a smooth animation, the `plot` command should take keyword arguments instead of being broken into separate calls to `xlabel`, `ylabel`, `axis`, `time`, and `show`. Several `plot` calls will automatically cause an animation on the screen. In addition, we want to save each frame in the animation to file. We then need a filename where the frame number is padded with zeros, here `tmp_0000.png`, `tmp_0001.png`, and so on. The proper printf construction is then `tmp_%04d.png`. Section 1.3.2 contains more basic information on making animations.

The solver is called with an argument `plot_u` as `user_function`. If the user chooses to use SciTools, `plot_u` is the `plot_u_st` callback function, but for Matplotlib it is an instance of the class `PlotMatplotlib`. Also this class makes use of variables defined in the `viz` function: `plt` and `time`. With Matplotlib, one has to make the first plot the standard way, and then update the y data in the plot at every time level. The update requires active use of the returned value from `plt.plot` in the first plot. This value would need to be stored in a local variable if we were to use a closure for the `user_action` function when doing the animation with Matplotlib. It is much easier to store the variable as a class attribute `self.lines`. Since the class is essentially a function, we implement the function as the special method `__call__` such that the instance `plot_u(u, x, t, n)` can be called as a standard callback function from `solver`.

Making movie files. From the `frame_*.png` files containing the frames in the animation we can make video files. Section 1.3.2 presents basic information on how to use the `ffmpeg` (or `avconv`) program for producing video files in different modern formats: Flash, MP4, Webm, and Ogg.

The `viz` function creates an `ffmpeg` or `avconv` command with the proper arguments for each of the formats Flash, MP4, WebM, and Ogg. The task is greatly simplified by having a `codec2ext` dictionary for mapping video codec names to filename extensions. As mentioned in Section 1.3.2, only two formats are actually needed to ensure that all browsers can successfully play the video: MP4 and WebM.

Some animations having a large number of plot files may not be properly combined into a video using `ffmpeg` or `avconv`. A method that always works is to play the PNG files as an animation in a browser using JavaScript code in an HTML file. The SciTools package has a function `movie` (or a stand-alone command `scitools movie`) for creating such an HTML player. The `plt.movie` call in the `viz` function shows how the function is used. The file `movie.html` can be loaded into a browser and features a user interface where the speed of the animation can be controlled. Note that the movie in this case consists of the `movie.html` file and all the frame files `tmp_*.png`.

Skipping frames for animation speed. Sometimes the time step is small and T is large, leading to an inconveniently large number of plot files and a slow animation on the screen. The solution to such a problem is to decide on a total number of frames in the animation, `num_frames`, and plot the solution only for every `skip_frame` frames. For example, setting `skip_frame=5` leads to plots of every 5 frames. The default value `skip_frame=1` plots every frame. The total number of time levels (i.e., maximum possible number of frames) is the length of `t`, `t.size` (or `len(t)`), so if we want `num_frames` frames in the animation, we need to plot every `t.size/num_frames` frames:

```
skip_frame = int(t.size/float(num_frames))
if n % skip_frame == 0 or n == t.size-1:
    st.plot(x, u, 'r-', ...)
```

The initial condition (`n=0`) is included by `n % skip_frame == 0`, as well as every `skip_frame`-th frame. As `n % skip_frame == 0` will very seldom be true for the very final frame, we must also check if `n == t.size-1` to get the final frame included.

A simple choice of numbers may illustrate the formulas: say we have 801 frames in total (`t.size`) and we allow only 60 frames to be plotted. As `n` then runs from 801 to 0, we need to plot every 801/60 frame, which with integer division yields 13 as `skip_frame`. Using the mod function, `n % skip_frame`, this operation is zero every time `n` can be divided by 13 without a remainder. That is, the `if` test is true when `n` equals $0, 13, 26, 39, ..., 780, 801$. The associated code is included in the `plot_u` function, inside the `viz` function, in the file `wave1D_u0.py`.

2.3.6 Running a case

The first demo of our 1D wave equation solver concerns vibrations of a string that is initially deformed to a triangular shape, like when picking a guitar string:

$$I(x) = \begin{cases} ax/x_0, & x < x_0, \\ a(L-x)/(L-x_0), & \text{otherwise} \end{cases} \tag{2.33}$$

We choose $L = 75$ cm, $x_0 = 0.8L$, $a = 5$ mm, and a time frequency $\nu = 440$ Hz. The relation between the wave speed c and ν is $c = \nu\lambda$, where λ is the wavelength, taken as $2L$ because the longest wave on the string forms half a wavelength. There is no external force, so $f = 0$ (meaning we can neglect gravity), and the string is at rest initially, implying $V = 0$.

Regarding numerical parameters, we need to specify a Δt. Sometimes it is more natural to think of a spatial resolution instead of a time step. A natural semi-coarse spatial resolution in the present problem is $N_x = 50$. We can then choose the associated Δt (as required by the `viz` and `solver` functions) as the stability limit: $\Delta t = L/(N_x c)$. This is the Δt to be specified, but notice that if $C < 1$, the actual Δx computed in `solver` gets larger than L/N_x: $\Delta x = c\Delta t/C = L/(N_x C)$. (The reason is that we fix Δt and adjust Δx, so if C gets smaller, the code implements this effect in terms of a larger Δx.)

A function for setting the physical and numerical parameters and calling `viz` in this application goes as follows:

```
def guitar(C):
    """Triangular wave (pulled guitar string)."""
    L = 0.75
    x0 = 0.8*L
    a = 0.005
    freq = 440
    wavelength = 2*L
    c = freq*wavelength
    omega = 2*pi*freq
    num_periods = 1
    T = 2*pi/omega*num_periods
    # Choose dt the same as the stability limit for Nx=50
    dt = L/50./c

    def I(x):
        return a*x/x0 if x < x0 else a/(L-x0)*(L-x)

    umin = -1.2*a;  umax = -umin
    cpu = viz(I, 0, 0, c, L, dt, C, T, umin, umax,
              animate=True, tool='scitools')
```

The associated program has the name `wave1D_u0.py`. Run the program and watch the movie of the vibrating string[3]. The string should ideally consist of straight segments, but these are somewhat wavy due to numerical approximation. Run the case with the `wave1D_u0.py` code and $C = 1$ to see the exact solution.

2.3.7 Working with a scaled PDE model

Depending on the model, it may be a substantial job to establish consistent and relevant physical parameter values for a case. The guitar string example illustrates the point. However, by *scaling* the mathematical problem we can often reduce the need to estimate physical parameters dramatically. The scaling technique consists of introducing new independent and dependent variables, with the aim that the absolute values of these lie in $[0, 1]$. We introduce the dimensionless variables (details are found in Section 3.1.1 in [11])

$$\bar{x} = \frac{x}{L}, \quad \bar{t} = \frac{c}{L}t, \quad \bar{u} = \frac{u}{a}.$$

Here, L is a typical length scale, e.g., the length of the domain, and a is a typical size of u, e.g., determined from the initial condition: $a = \max_x |I(x)|$.

We get by the chain rule that

$$\frac{\partial u}{\partial t} = \frac{\partial}{\partial \bar{t}}(a\bar{u})\frac{d\bar{t}}{dt} = \frac{ac}{L}\frac{\partial \bar{u}}{\partial \bar{t}}.$$

Similarly,

$$\frac{\partial u}{\partial x} = \frac{a}{L}\frac{\partial \bar{u}}{\partial \bar{x}}.$$

Inserting the dimensionless variables in the PDE gives, in case $f = 0$,

$$\frac{a^2c^2}{L^2}\frac{\partial^2 \bar{u}}{\partial \bar{t}^2} = \frac{a^2c^2}{L^2}\frac{\partial^2 \bar{u}}{\partial \bar{x}^2}.$$

Dropping the bars, we arrive at the scaled PDE

$$\frac{\partial^2 u}{\partial t^2} = \frac{\partial^2 u}{\partial x^2}, \quad x \in (0,1),\ t \in (0, cT/L), \tag{2.34}$$

which has no parameter c^2 anymore. The initial conditions are scaled as

[3] `http://tinyurl.com/hbcasmj/wave/html/mov-wave/guitar_C0.8/movie.html`

$$a\bar{u}(\bar{x},0) = I(L\bar{x})$$

and

$$\frac{a}{L/c}\frac{\partial \bar{u}}{\partial \bar{t}}(\bar{x},0) = V(L\bar{x}),$$

resulting in

$$\bar{u}(\bar{x},0) = \frac{I(L\bar{x})}{\max_x |I(x)|}, \quad \frac{\partial \bar{u}}{\partial \bar{t}}(\bar{x},0) = \frac{L}{ac}V(L\bar{x})\,.$$

In the common case $V = 0$ we see that there are no physical parameters to be estimated in the PDE model!

If we have a program implemented for the physical wave equation with dimensions, we can obtain the dimensionless, scaled version by setting $c = 1$. The initial condition of a guitar string, given in (2.33), gets its scaled form by choosing $a = 1$, $L = 1$, and $x_0 \in [0, 1]$. This means that we only need to decide on the x_0 value as a fraction of unity, because the scaled problem corresponds to setting all other parameters to unity. In the code we can just set `a=c=L=1`, `x0=0.8`, and there is no need to calculate with wavelengths and frequencies to estimate c!

The only non-trivial parameter to estimate in the scaled problem is the final end time of the simulation, or more precisely, how it relates to periods in periodic solutions in time, since we often want to express the end time as a certain number of periods. The period in the dimensionless problem is 2, so the end time can be set to the desired number of periods times 2.

Why the dimensionless period is 2 can be explained by the following reasoning. Suppose that u behaves as $\cos(\omega t)$ in time in the original problem with dimensions. The corresponding period is then $P = 2\pi/\omega$, but we need to estimate ω. A typical solution of the wave equation is $u(x,t) = A\cos(kx)\cos(\omega t)$, where A is an amplitude and k is related to the wave length λ in space: $\lambda = 2\pi/k$. Both λ and A will be given by the initial condition $I(x)$. Inserting this $u(x,t)$ in the PDE yields $-\omega^2 = -c^2k^2$, i.e., $\omega = kc$. The period is therefore $P = 2\pi/(kc)$. If the boundary conditions are $u(0,t) = u(L,t)$, we need to have $kL = n\pi$ for integer n. The period becomes $P = 2L/nc$. The longest period is $P = 2L/c$. The dimensionless period $\tilde{P}$ is obtained by dividing P by the time scale L/c, which results in $\tilde{P} = 2$. Shorter waves in the initial condition will have a dimensionless shorter period $\tilde{P} = 2/n$ ($n > 1$).

2.4 Vectorization

The computational algorithm for solving the wave equation visits one mesh point at a time and evaluates a formula for the new value u_i^{n+1} at that point. Technically, this is implemented by a loop over array elements in a program. Such loops may run slowly in Python (and similar interpreted languages such as R and MATLAB). One technique for speeding up loops is to perform operations on entire arrays instead of working with one element at a time. This is referred to as *vectorization*, *vector computing*, or *array computing*. Operations on whole arrays are possible if the computations involving each element is independent of each other and therefore can, at least in principle, be performed simultaneously. That is, vectorization not only speeds up the code on serial computers, but also makes it easy to exploit parallel computing. Actually, there are Python tools like Numba[4] that can automatically turn vectorized code into parallel code.

2.4.1 Operations on slices of arrays

Efficient computing with `numpy` arrays demands that we avoid loops and compute with entire arrays at once (or at least large portions of them). Consider this calculation of differences $d_i = u_{i+1} - u_i$:

```
n = u.size
for i in range(0, n-1):
    d[i] = u[i+1] - u[i]
```

All the differences here are independent of each other. The computation of `d` can therefore alternatively be done by subtracting the array $(u_0, u_1, \ldots, u_{n-1})$ from the array where the elements are shifted one index upwards: $(u_1, u_2, \ldots, u_n)$, see Figure 2.3. The former subset of the array can be expressed by `u[0:n-1]`, `u[0:-1]`, or just `u[:-1]`, meaning from index 0 up to, but not including, the last element (`-1`). The latter subset is obtained by `u[1:n]` or `u[1:]`, meaning from index 1 and the rest of the array. The computation of `d` can now be done without an explicit Python loop:

```
d = u[1:] - u[:-1]
```

or with explicit limits if desired:

[4] `http://numba.pydata.org`

```
d = u[1:n] - u[0:n-1]
```

Indices with a colon, going from an index to (but not including) another index are called *slices*. With `numpy` arrays, the computations are still done by loops, but in efficient, compiled, highly optimized C or Fortran code. Such loops are sometimes referred to as *vectorized loops*. Such loops can also easily be distributed among many processors on parallel computers. We say that the *scalar code* above, working on an element (a scalar) at a time, has been replaced by an equivalent *vectorized code*. The process of vectorizing code is called *vectorization*.

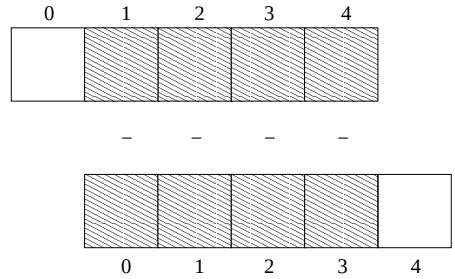


Fig. 2.3 Illustration of subtracting two slices of two arrays.

Test your understanding

Newcomers to vectorization are encouraged to choose a small array `u`, say with five elements, and simulate with pen and paper both the loop version and the vectorized version above.

Finite difference schemes basically contain differences between array elements with shifted indices. As an example, consider the updating formula

```
for i in range(1, n-1):
    u2[i] = u[i-1] - 2*u[i] + u[i+1]
```

The vectorization consists of replacing the loop by arithmetics on slices of arrays of length `n-2`:

```
u2 = u[:-2] - 2*u[1:-1] + u[2:]
u2 = u[0:n-2] - 2*u[1:n-1] + u[2:n]   # alternative
```

Note that the length of `u2` becomes `n-2`. If `u2` is already an array of length `n` and we want to use the formula to update all the "inner" elements of `u2`, as we will when solving a 1D wave equation, we can write

```
u2[1:-1]  = u[:-2] - 2*u[1:-1] + u[2:]
u2[1:n-1] = u[0:n-2] - 2*u[1:n-1] + u[2:n]   # alternative
```

The first expression's right-hand side is realized by the following steps, involving temporary arrays with intermediate results, since each array operation can only involve one or two arrays. The `numpy` package performs (behind the scenes) the first line above in four steps:

```
temp1 = 2*u[1:-1]
temp2 = u[:-2] - temp1
temp3 = temp2 + u[2:]
u2[1:-1] = temp3
```

We need three temporary arrays, but a user does not need to worry about such temporary arrays.

Common mistakes with array slices

Array expressions with slices demand that the slices have the same shape. It easy to make a mistake in, e.g.,

```
u2[1:n-1] = u[0:n-2] - 2*u[1:n-1] + u[2:n]
```

and write

```
u2[1:n-1] = u[0:n-2] - 2*u[1:n-1] + u[1:n]
```

Now `u[1:n]` has wrong length (`n-1`) compared to the other array slices, causing a `ValueError` and the message `could not broadcast input array from shape 103 into shape 104` (if `n` is 105). When such errors occur one must closely examine all the slices. Usually, it is easier to get upper limits of slices right when they use `-1` or `-2` or empty limit rather than expressions involving the length.

Another common mistake, when `u2` has length `n`, is to forget the slice in the array on the left-hand side,

```
u2 = u[0:n-2] - 2*u[1:n-1] + u[1:n]
```

This is really crucial: now `u2` becomes a *new* array of length `n-2`, which is the wrong length as we have no entries for the boundary values. We meant to insert the right-hand side array *into* the original

u2 array for the entries that correspond to the internal points in the mesh (1:n-1 or 1:-1).

Vectorization may also work nicely with functions. To illustrate, we may extend the previous example as follows:

```
def f(x):
    return x**2 + 1

for i in range(1, n-1):
    u2[i] = u[i-1] - 2*u[i] + u[i+1] + f(x[i])
```

Assuming u2, u, and x all have length n, the vectorized version becomes

```
u2[1:-1] = u[:-2] - 2*u[1:-1] + u[2:] + f(x[1:-1])
```

Obviously, f must be able to take an array as argument for f(x[1:-1]) to make sense.

2.4.2 Finite difference schemes expressed as slices

We now have the necessary tools to vectorize the wave equation algorithm as described mathematically in Section 2.1.5 and through code in Section 2.3.2. There are three loops: one for the initial condition, one for the first time step, and finally the loop that is repeated for all subsequent time levels. Since only the latter is repeated a potentially large number of times, we limit our vectorization efforts to this loop. Within the time loop, the space loop reads:

```
for i in range(1, Nx):
    u[i] = 2*u_n[i] - u_nm1[i] + \
           C2*(u_n[i-1] - 2*u_n[i] + u_n[i+1])
```

The vectorized version becomes

```
u[1:-1] = - u_nm1[1:-1] + 2*u_n[1:-1] + \
          C2*(u_n[:-2] - 2*u_n[1:-1] + u_n[2:])
```

or

```
u[1:Nx] = 2*u_n[1:Nx]- u_nm1[1:Nx] + \
          C2*(u_n[0:Nx-1] - 2*u_n[1:Nx] + u_n[2:Nx+1])
```

The program wave1D_u0v.py contains a new version of the function solver where both the scalar and the vectorized loops are included (the argument version is set to scalar or vectorized, respectively).

2.4.3 Verification

We may reuse the quadratic solution $u_e(x,t) = x(L-x)(1+\frac{1}{2}t)$ for verifying also the vectorized code. A test function can now verify both the scalar and the vectorized version. Moreover, we may use a `user_action` function that compares the computed and exact solution at each time level and performs a test:

```
def test_quadratic():
    """
    Check the scalar and vectorized versions for
    a quadratic u(x,t)=x(L-x)(1+t/2) that is exactly reproduced.
    """
    # The following function must work for x as array or scalar
    u_exact = lambda x, t: x*(L - x)*(1 + 0.5*t)
    I = lambda x: u_exact(x, 0)
    V = lambda x: 0.5*u_exact(x, 0)
    # f is a scalar (zeros_like(x) works for scalar x too)
    f = lambda x, t: np.zeros_like(x) + 2*c**2*(1 + 0.5*t)

    L = 2.5
    c = 1.5
    C = 0.75
    Nx = 3  # Very coarse mesh for this exact test
    dt = C*(L/Nx)/c
    T = 18

    def assert_no_error(u, x, t, n):
        u_e = u_exact(x, t[n])
        tol = 1E-13
        diff = np.abs(u - u_e).max()
        assert diff < tol

    solver(I, V, f, c, L, dt, C, T,
           user_action=assert_no_error, version='scalar')
    solver(I, V, f, c, L, dt, C, T,
           user_action=assert_no_error, version='vectorized')
```

Lambda functions

The code segment above demonstrates how to achieve very compact code, without degraded readability, by use of lambda functions for the various input parameters that require a Python function. In essence,

```
f = lambda x, t: L*(x-t)**2
```

is equivalent to

```
def f(x, t):
    return L(x-t)**2
```

Note that lambda functions can just contain a single expression and no statements.

One advantage with lambda functions is that they can be used directly in calls:

```
solver(I=lambda x: sin(pi*x/L), V=0, f=0, ...)
```

2.4.4 Efficiency measurements

The `wave1D_u0v.py` contains our new `solver` function with both scalar and vectorized code. For comparing the efficiency of scalar versus vectorized code, we need a `viz` function as discussed in Section 2.3.5. All of this `viz` function can be reused, except the call to `solver_function`. This call lacks the parameter `version`, which we want to set to `vectorized` and `scalar` for our efficiency measurements.

One solution is to copy the `viz` code from `wave1D_u0` into `wave1D_u0v.py` and add a `version` argument to the `solver_function` call. Taking into account how much animation code we then duplicate, this is not a good idea. Alternatively, introducing the `version` argument in `wave1D_u0.viz`, so that this function can be imported into `wave1D_u0v.py`, is not a good solution either, since `version` has no meaning in that file. We need better ideas!

Solution 1. Calling `viz` in `wave1D_u0` with `solver_function` as our new solver in `wave1D_u0v` works fine, since this solver has `version='vectorized'` as default value. The problem arises when we want to test `version='scalar'`. The simplest solution is then to use `wave1D_u0.solver` instead. We make a new `viz` function in `wave1D_u0v.py` that has a `version` argument and that just calls `wave1D_u0.viz`:

```
def viz(
    I, V, f, c, L, dt, C, T,  # PDE parameters
    umin, umax,               # Interval for u in plots
    animate=True,             # Simulation with animation?
    tool='matplotlib',        # 'matplotlib' or 'scitools'
    solver_function=solver,   # Function with numerical algorithm
    version='vectorized',     # 'scalar' or 'vectorized'
    ):
```

```
    import wave1D_u0
    if version == 'vectorized':
        # Reuse viz from wave1D_u0, but with the present
        # modules' new vectorized solver (which has
        # version='vectorized' as default argument;
        # wave1D_u0.viz does not feature this argument)
        cpu = wave1D_u0.viz(
            I, V, f, c, L, dt, C, T, umin, umax,
            animate, tool, solver_function=solver)
    elif version == 'scalar':
        # Call wave1D_u0.viz with a solver with
        # scalar code and use wave1D_u0.solver.
        cpu = wave1D_u0.viz(
            I, V, f, c, L, dt, C, T, umin, umax,
            animate, tool,
            solver_function=wave1D_u0.solver)
```

Solution 2. There is a more advanced and fancier solution featuring a very useful trick: we can make a new function that will always call `wave1D_u0v.solver` with `version='scalar'`. The `functools.partial` function from standard Python takes a function `func` as argument and a series of positional and keyword arguments and returns a new function that will call `func` with the supplied arguments, while the user can control all the other arguments in `func`. Consider a trivial example,

```
def f(a, b, c=2):
    return a + b + c
```

We want to ensure that `f` is always called with `c=3`, i.e., `f` has only two "free" arguments `a` and `b`. This functionality is obtained by

```
import functools
f2 = functools.partial(f, c=3)

print f2(1, 2)  # results in 1+2+3=6
```

Now `f2` calls `f` with whatever the user supplies as `a` and `b`, but `c` is always 3.

Back to our `viz` code, we can do

```
import functools
# Call wave1D_u0.solver with version fixed to scalar
scalar_solver = functools.partial(wave1D_u0.solver, version='scalar')
cpu = wave1D_u0.viz(
        I, V, f, c, L, dt, C, T, umin, umax,
        animate, tool, solver_function=scalar_solver)
```

The new `scalar_solver` takes the same arguments as `wave1D_u0.scalar` and calls `wave1D_u0v.scalar`, but always supplies the extra argument `version='scalar'`. When sending this `solver_function` to

`wave1D_u0.viz`, the latter will call `wave1D_u0v.solver` with all the `I`, `V`, `f`, etc., arguments we supply, plus `version='scalar'`.

Efficiency experiments. We now have a `viz` function that can call our solver function both in scalar and vectorized mode. The function `run_efficiency_experiments` in `wave1D_u0v.py` performs a set of experiments and reports the CPU time spent in the scalar and vectorized solver for the previous string vibration example with spatial mesh resolutions $N_x = 50, 100, 200, 400, 800$. Running this function reveals that the vectorized code runs substantially faster: the vectorized code runs approximately $N_x/10$ times as fast as the scalar code!

2.4.5 Remark on the updating of arrays

At the end of each time step we need to update the `u_nm1` and `u_n` arrays such that they have the right content for the next time step:

```
u_nm1[:] = u_n
u_n[:] = u
```

The order here is important: updating `u_n` first, makes `u_nm1` equal to `u`, which is wrong!

The assignment `u_n[:] = u` copies the content of the `u` array into the elements of the `u_n` array. Such copying takes time, but that time is negligible compared to the time needed for computing `u` from the finite difference formula, even when the formula has a vectorized implementation. However, efficiency of program code is a key topic when solving PDEs numerically (particularly when there are two or three space dimensions), so it must be mentioned that there exists a much more efficient way of making the arrays `u_nm1` and `u_n` ready for the next time step. The idea is based on *switching references* and explained as follows.

A Python variable is actually a reference to some object (C programmers may think of pointers). Instead of copying data, we can let `u_nm1` refer to the `u_n` object and `u_n` refer to the `u` object. This is a very efficient operation (like switching pointers in C). A naive implementation like

```
u_nm1 = u_n
u_n = u
```

will fail, however, because now `u_nm1` refers to the `u_n` object, but then the name `u_n` refers to `u`, so that this `u` object has two references, `u_n` and `u`, while our third array, originally referred to by `u_nm1`, has no more

references and is lost. This means that the variables `u`, `u_n`, and `u_nm1` refer to two arrays and not three. Consequently, the computations at the next time level will be messed up, since updating the elements in `u` will imply updating the elements in `u_n` too, thereby destroying the solution at the previous time step.

While `u_nm1 = u_n` is fine, `u_n = u` is problematic, so the solution to this problem is to ensure that `u` points to the `u_nm1` array. This is mathematically wrong, but new correct values will be filled into `u` at the next time step and make it right.

The correct switch of references is

```
tmp = u_nm1
u_nm1 = u_n
u_n = u
u = tmp
```

We can get rid of the temporary reference `tmp` by writing

```
u_nm1, u_n, u = u_n, u, u_nm1
```

This switching of references for updating our arrays will be used in later implementations.

Caution:

The update `u_nm1, u_n, u = u_n, u, u_nm1` leaves wrong content in `u` at the final time step. This means that if we return `u`, as we do in the example codes here, we actually return `u_nm1`, which is obviously wrong. It is therefore important to adjust the content of `u` to `u = u_n` before returning `u`. (Note that the `user_action` function reduces the need to return the solution from the solver.)

2.5 Exercises

Exercise 2.1: Simulate a standing wave

The purpose of this exercise is to simulate standing waves on $[0, L]$ and illustrate the error in the simulation. Standing waves arise from an initial condition

$$u(x, 0) = A \sin\left(\frac{\pi}{L} m x\right),$$

where m is an integer and A is a freely chosen amplitude. The corresponding exact solution can be computed and reads

$$u_{\mathrm{e}}(x,t) = A\sin\left(\frac{\pi}{L}mx\right)\cos\left(\frac{\pi}{L}mct\right) .$$

a) Explain that for a function $\sin kx\cos\omega t$ the wave length in space is $\lambda = 2\pi/k$ and the period in time is $P = 2\pi/\omega$. Use these expressions to find the wave length in space and period in time of u_{e} above.

b) Import the `solver` function from `wave1D_u0.py` into a new file where the `viz` function is reimplemented such that it plots either the numerical *and* the exact solution, *or* the error.

c) Make animations where you illustrate how the error $e_i^n = u_{\mathrm{e}}(x_i,t_n) - u_i^n$ develops and increases in time. Also make animations of u and u_{e} simultaneously.

Hint 1. Quite long time simulations are needed in order to display significant discrepancies between the numerical and exact solution.

Hint 2. A possible set of parameters is $L = 12$, $m = 9$, $c = 2$, $A = 1$, $N_x = 80$, $C = 0.8$. The error mesh function e^n can be simulated for 10 periods, while 20-30 periods are needed to show significant differences between the curves for the numerical and exact solution.
Filename: `wave_standing`.

Remarks. The important parameters for numerical quality are C and $k\Delta x$, where $C = c\Delta t/\Delta x$ is the Courant number and k is defined above ($k\Delta x$ is proportional to how many mesh points we have per wave length in space, see Section 2.10.4 for explanation).

Exercise 2.2: Add storage of solution in a user action function

Extend the `plot_u` function in the file `wave1D_u0.py` to also store the solutions `u` in a list. To this end, declare `all_u` as an empty list in the `viz` function, outside `plot_u`, and perform an append operation inside the `plot_u` function. Note that a function, like `plot_u`, inside another function, like `viz`, remembers all local variables in `viz` function, including `all_u`, even when `plot_u` is called (as `user_action`) in the `solver` function. Test both `all_u.append(u)` and `all_u.append(u.copy())`. Why does one of these constructions fail to store the solution correctly? Let the `viz` function return the `all_u` list converted to a two-dimensional `numpy` array.
Filename: `wave1D_u0_s_store`.

Exercise 2.3: Use a class for the user action function

Redo Exercise 2.2 using a class for the user action function. Let the `all_u` list be an attribute in this class and implement the user action function as a method (the special method `__call__` is a natural choice). The class versions avoid that the user action function depends on parameters defined outside the function (such as `all_u` in Exercise 2.2).
Filename: `wave1D_u0_s2c`.

Exercise 2.4: Compare several Courant numbers in one movie

The goal of this exercise is to make movies where several curves, corresponding to different Courant numbers, are visualized. Write a program that resembles `wave1D_u0_s2c.py` in Exercise 2.3, but with a `viz` function that can take a list of `C` values as argument and create a movie with solutions corresponding to the given `C` values. The `plot_u` function must be changed to store the solution in an array (see Exercise 2.2 or 2.3 for details), `solver` must be computed for each value of the Courant number, and finally one must run through each time step and plot all the spatial solution curves in one figure and store it in a file.

The challenge in such a visualization is to ensure that the curves in one plot correspond to the same time point. The easiest remedy is to keep the time resolution constant and change the space resolution to change the Courant number. Note that each spatial grid is needed for the final plotting, so it is an option to store those grids too.
Filename: `wave_numerics_comparison`.

Exercise 2.5: Implementing the solver function as a generator

The callback function `user_action(u, x, t, n)` is called from the `solver` function (in, e.g., `wave1D_u0.py`) at every time level and lets the user work perform desired actions with the solution, like plotting it on the screen. We have implemented the callback function in the typical way it would have been done in C and Fortran. Specifically, the code looks like

```
if user_action is not None:
    if user_action(u, x, t, n):
        break
```

Many Python programmers, however, may claim that `solver` is an iterative process, and that iterative processes with callbacks to the user code is more elegantly implemented as *generators*. The rest of the text has little meaning unless you are familiar with Python generators and the `yield` statement.

Instead of calling `user_action`, the `solver` function issues a `yield` statement, which is a kind of `return` statement:

```
yield u, x, t, n
```

The program control is directed back to the calling code:

```
for u, x, t, n in solver(...):
    # Do something with u at t[n]
```

When the block is done, `solver` continues with the statement after `yield`. Note that the functionality of terminating the solution process if `user_action` returns a `True` value is not possible to implement in the generator case.

Implement the `solver` function as a generator, and plot the solution at each time step. Filename: `wave1D_u0_generator`.

Project 2.6: Calculus with 1D mesh functions

This project explores integration and differentiation of mesh functions, both with scalar and vectorized implementations. We are given a mesh function f_i on a spatial one-dimensional mesh $x_i = i\Delta x$, $i = 0, \ldots, N_x$, over the interval $[a, b]$.

a) Define the discrete derivative of f_i by using centered differences at internal mesh points and one-sided differences at the end points. Implement a scalar version of the computation in a Python function and write an associated unit test for the linear case $f(x) = 4x - 2.5$ where the discrete derivative should be exact.

b) Vectorize the implementation of the discrete derivative. Extend the unit test to check the validity of the implementation.

c) To compute the discrete integral F_i of f_i, we assume that the mesh function f_i varies linearly between the mesh points. Let $f(x)$ be such a linear interpolant of f_i. We then have

$$F_i = \int_{x_0}^{x_i} f(x)dx\,.$$

The exact integral of a piecewise linear function $f(x)$ is given by the Trapezoidal rule. Show that if F_i is already computed, we can find F_{i+1} from

$$F_{i+1} = F_i + \frac{1}{2}(f_i + f_{i+1})\Delta x\,.$$

Make a function for the scalar implementation of the discrete integral as a mesh function. That is, the function should return F_i for $i = 0, \ldots, N_x$. For a unit test one can use the fact that the above defined discrete integral of a linear function (say $f(x) = 4x - 2.5$) is exact.

d) Vectorize the implementation of the discrete integral. Extend the unit test to check the validity of the implementation.

Hint. Interpret the recursive formula for F_{i+1} as a sum. Make an array with each element of the sum and use the "cumsum" (`numpy.cumsum`) operation to compute the accumulative sum: `numpy.cumsum([1,3,5])` is `[1,4,9]`.

e) Create a class `MeshCalculus` that can integrate and differentiate mesh functions. The class can just define some methods that call the previously implemented Python functions. Here is an example on the usage:

```
import numpy as np
calc = MeshCalculus(vectorized=True)
x = np.linspace(0, 1, 11)        # mesh
f = np.exp(x)                    # mesh function
df = calc.differentiate(f, x)    # discrete derivative
F = calc.integrate(f, x)         # discrete anti-derivative
```

Filename: `mesh_calculus_1D`.

2.6 Generalization: reflecting boundaries

The boundary condition $u = 0$ in a wave equation reflects the wave, but u changes sign at the boundary, while the condition $u_x = 0$ reflects the wave as a mirror and preserves the sign, see a web page[5] or a movie file[6] for demonstration.

Our next task is to explain how to implement the boundary condition $u_x = 0$, which is more complicated to express numerically and also to

[5] `http://tinyurl.com/hbcasmj/book/html/mov-wave/demo_BC_gaussian/index.html`
[6] `http://tinyurl.com/gokgkov/mov-wave/demo_BC_gaussian/movie.flv`

implement than a given value of u. We shall present two methods for implementing $u_x = 0$ in a finite difference scheme, one based on deriving a modified stencil at the boundary, and another one based on extending the mesh with ghost cells and ghost points.

2.6.1 Neumann boundary condition

When a wave hits a boundary and is to be reflected back, one applies the condition

$$\frac{\partial u}{\partial n} \equiv \boldsymbol{n} \cdot \nabla u = 0\,. \tag{2.35}$$

The derivative $\partial/\partial n$ is in the outward normal direction from a general boundary. For a 1D domain $[0, L]$, we have that

$$\left.\frac{\partial}{\partial n}\right|_{x=L} = \left.\frac{\partial}{\partial x}\right|_{x=L}, \quad \left.\frac{\partial}{\partial n}\right|_{x=0} = -\left.\frac{\partial}{\partial x}\right|_{x=0}.$$

Boundary condition terminology

Boundary conditions that specify the value of $\partial u/\partial n$ (or shorter u_n) are known as Neumann[a] conditions, while Dirichlet conditions[b] refer to specifications of u. When the values are zero ($\partial u/\partial n = 0$ or $u = 0$) we speak about *homogeneous* Neumann or Dirichlet conditions.

[a] http://en.wikipedia.org/wiki/Neumann_boundary_condition
[b] http://en.wikipedia.org/wiki/Dirichlet_conditions

2.6.2 Discretization of derivatives at the boundary

How can we incorporate the condition (2.35) in the finite difference scheme? Since we have used central differences in all the other approximations to derivatives in the scheme, it is tempting to implement (2.35) at $x = 0$ and $t = t_n$ by the difference

$$[D_{2x}u]_0^n = \frac{u_{-1}^n - u_1^n}{2\Delta x} = 0\,. \tag{2.36}$$

The problem is that u^n_{-1} is not a u value that is being computed since the point is outside the mesh. However, if we combine (2.36) with the scheme

$$u^{n+1}_i = -u^{n-1}_i + 2u^n_i + C^2\left(u^n_{i+1} - 2u^n_i + u^n_{i-1}\right), \tag{2.37}$$

for $i = 0$, we can eliminate the fictitious value u^n_{-1}. We see that $u^n_{-1} = u^n_1$ from (2.36), which can be used in (2.37) to arrive at a modified scheme for the boundary point u^{n+1}_0:

$$u^{n+1}_i = -u^{n-1}_i + 2u^n_i + 2C^2\left(u^n_{i+1} - u^n_i\right), \quad i = 0\,. \tag{2.38}$$

Figure 2.4 visualizes this equation for computing u^3_0 in terms of u^2_0, u^1_0, and u^2_1.

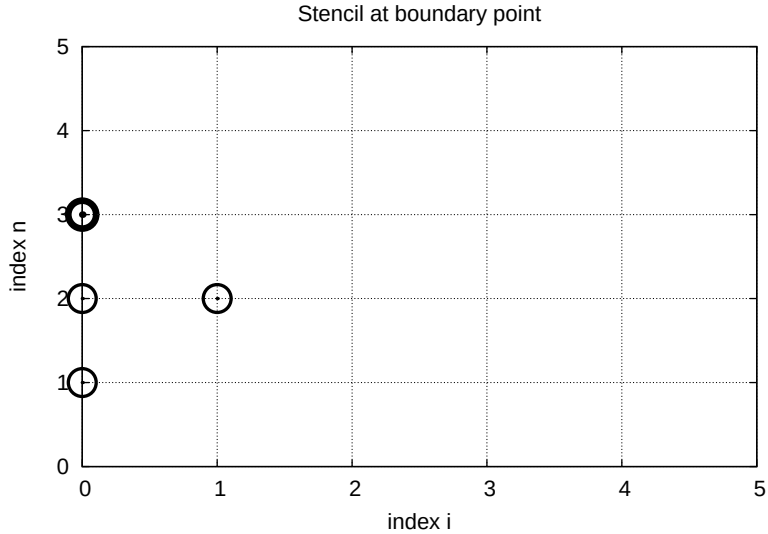


Fig. 2.4 Modified stencil at a boundary with a Neumann condition.

Similarly, (2.35) applied at $x = L$ is discretized by a central difference

$$\frac{u^n_{N_x+1} - u^n_{N_x-1}}{2\Delta x} = 0\,. \tag{2.39}$$

Combined with the scheme for $i = N_x$ we get a modified scheme for the boundary value $u^{n+1}_{N_x}$:

$$u^{n+1}_i = -u^{n-1}_i + 2u^n_i + 2C^2\left(u^n_{i-1} - u^n_i\right), \quad i = N_x\,. \tag{2.40}$$

The modification of the scheme at the boundary is also required for the special formula for the first time step. How the stencil moves through the mesh and is modified at the boundary can be illustrated by an animation in a web page[7] or a movie file[8].

2.6.3 Implementation of Neumann conditions

We have seen in the preceding section that the special formulas for the boundary points arise from replacing u^n_{i-1} by u^n_{i+1} when computing u^{n+1}_i from the stencil formula for $i = 0$. Similarly, we replace u^n_{i+1} by u^n_{i-1} in the stencil formula for $i = N_x$. This observation can conveniently be used in the coding: we just work with the general stencil formula, but write the code such that it is easy to replace `u[i-1]` by `u[i+1]` and vice versa. This is achieved by having the indices `i+1` and `i-1` as variables `ip1` (`i` plus 1) and `im1` (`i` minus 1), respectively. At the boundary we can easily define `im1=i+1` while we use `im1=i-1` in the internal parts of the mesh. Here are the details of the implementation (note that the updating formula for `u[i]` is the general stencil formula):

```
i = 0
ip1 = i+1
im1 = ip1  # i-1 -> i+1
u[i] = u_n[i] + C2*(u_n[im1] - 2*u_n[i] + u_n[ip1])

i = Nx
im1 = i-1
ip1 = im1  # i+1 -> i-1
u[i] = u_n[i] + C2*(u_n[im1] - 2*u_n[i] + u_n[ip1])
```

We can in fact create one loop over both the internal and boundary points and use only one updating formula:

```
for i in range(0, Nx+1):
    ip1 = i+1 if i < Nx else i-1
    im1 = i-1 if i > 0  else i+1
    u[i] = u_n[i] + C2*(u_n[im1] - 2*u_n[i] + u_n[ip1])
```

The program `wave1D_n0.py` contains a complete implementation of the 1D wave equation with boundary conditions $u_x = 0$ at $x = 0$ and $x = L$.

It would be nice to modify the `test_quadratic` test case from the `wave1D_u0.py` with Dirichlet conditions, described in Section 2.4.3. However, the Neumann conditions require the polynomial variation in the

[7] `http://tinyurl.com/hbcasmj/book/html/mov-wave/N_stencil_gpl/index.html`
[8] `http://tinyurl.com/gokgkov/mov-wave/N_stencil_gpl/movie.ogg`

x direction to be of third degree, which causes challenging problems when designing a test where the numerical solution is known exactly. Exercise 2.15 outlines ideas and code for this purpose. The only test in `wave1D_n0.py` is to start with a plug wave at rest and see that the initial condition is reached again perfectly after one period of motion, but such a test requires $C = 1$ (so the numerical solution coincides with the exact solution of the PDE, see Section 2.10.4).

2.6.4 Index set notation

To improve our mathematical writing and our implementations, it is wise to introduce a special notation for index sets. This means that we write x_i, followed by $i \in \mathcal{I}_x$, instead of $i = 0, \ldots, N_x$. Obviously, $\mathcal{I}_x$ must be the index set $\mathcal{I}_x = \{0, \ldots, N_x\}$, but it is often advantageous to have a symbol for this set rather than specifying all its elements (all the time, as we have done up to now). This new notation saves writing and makes specifications of algorithms and their implementation as computer code simpler.

The first index in the set will be denoted $\mathcal{I}_x^0$ and the last $\mathcal{I}_x^{-1}$. When we need to skip the first element of the set, we use $\mathcal{I}_x^+$ for the remaining subset $\mathcal{I}_x^+ = \{1, \ldots, N_x\}$. Similarly, if the last element is to be dropped, we write $\mathcal{I}_x^- = \{0, \ldots, N_x - 1\}$ for the remaining indices. All the indices corresponding to inner grid points are specified by $\mathcal{I}_x^i = \{1, \ldots, N_x - 1\}$. For the time domain we find it natural to explicitly use 0 as the first index, so we will usually write $n = 0$ and t_0 rather than $n = \mathcal{I}_t^0$. We also avoid notation like $x_{\mathcal{I}_x^{-1}}$ and will instead use x_i, $i = \mathcal{I}_x^{-1}$.

The Python code associated with index sets applies the following conventions:

Notation	Python
$\mathcal{I}_x$	`Ix`
$\mathcal{I}_x^0$	`Ix[0]`
$\mathcal{I}_x^{-1}$	`Ix[-1]`
$\mathcal{I}_x^-$	`Ix[:-1]`
$\mathcal{I}_x^+$	`Ix[1:]`
$\mathcal{I}_x^i$	`Ix[1:-1]`

Why index sets are useful

An important feature of the index set notation is that it keeps our formulas and code independent of how we count mesh points. For example, the notation $i \in \mathcal{I}_x$ or $i = \mathcal{I}_x^0$ remains the same whether $\mathcal{I}_x$ is defined as above or as starting at 1, i.e., $\mathcal{I}_x = \{1, \ldots, Q\}$. Similarly, we can in the code define `Ix=range(Nx+1)` or `Ix=range(1,Q)`, and expressions like `Ix[0]` and `Ix[1:-1]` remain correct. One application where the index set notation is convenient is conversion of code from a language where arrays has base index 0 (e.g., Python and C) to languages where the base index is 1 (e.g., MATLAB and Fortran). Another important application is implementation of Neumann conditions via ghost points (see next section).

For the current problem setting in the x, t plane, we work with the index sets

$$\mathcal{I}_x = \{0, \ldots, N_x\}, \quad \mathcal{I}_t = \{0, \ldots, N_t\}, \tag{2.41}$$

defined in Python as

```
Ix = range(0, Nx+1)
It = range(0, Nt+1)
```

A finite difference scheme can with the index set notation be specified as

$$\begin{aligned}
u_i^{n+1} &= u_i^n - \frac{1}{2}C^2\left(u_{i+1}^n - 2u_i^n + u_{i-1}^n\right), \quad , i \in \mathcal{I}_x^i,\ n = 0,\\
u_i^{n+1} &= -u_i^{n-1} + 2u_i^n + C^2\left(u_{i+1}^n - 2u_i^n + u_{i-1}^n\right), \quad i \in \mathcal{I}_x^i,\ n \in \mathcal{I}_t^i,\\
u_i^{n+1} &= 0, \quad i = \mathcal{I}_x^0,\ n \in \mathcal{I}_t^-,\\
u_i^{n+1} &= 0, \quad i = \mathcal{I}_x^{-1},\ n \in \mathcal{I}_t^- .
\end{aligned}$$

The corresponding implementation becomes

```
# Initial condition
for i in Ix[1:-1]:
    u[i] = u_n[i] - 0.5*C2*(u_n[i-1] - 2*u_n[i] + u_n[i+1])

# Time loop
for n in It[1:-1]:
    # Compute internal points
    for i in Ix[1:-1]:
        u[i] = - u_nm1[i] + 2*u_n[i] + \
               C2*(u_n[i-1] - 2*u_n[i] + u_n[i+1])
```

```
    # Compute boundary conditions
    i = Ix[0];  u[i] = 0
    i = Ix[-1]; u[i] = 0
```

Notice

The program `wave1D_dn.py` applies the index set notation and solves the 1D wave equation $u_{tt} = c^2 u_{xx} + f(x,t)$ with quite general boundary and initial conditions:

- $x = 0$: $u = U_0(t)$ or $u_x = 0$
- $x = L$: $u = U_L(t)$ or $u_x = 0$
- $t = 0$: $u = I(x)$
- $t = 0$: $u_t = V(x)$

The program combines Dirichlet and Neumann conditions, scalar and vectorized implementation of schemes, and the index set notation into one piece of code. A lot of test examples are also included in the program:

- A rectangular plug-shaped initial condition. (For $C = 1$ the solution will be a rectangle that jumps one cell per time step, making the case well suited for verification.)
- A Gaussian function as initial condition.
- A triangular profile as initial condition, which resembles the typical initial shape of a guitar string.
- A sinusoidal variation of u at $x = 0$ and either $u = 0$ or $u_x = 0$ at $x = L$.
- An analytical solution $u(x,t) = \cos(m\pi t/L)\sin(\frac{1}{2}m\pi x/L)$, which can be used for convergence rate tests.

2.6.5 Verifying the implementation of Neumann conditions

How can we test that the Neumann conditions are correctly implemented? The `solver` function in the `wave1D_dn.py` program described in the box above accepts Dirichlet or Neumann conditions at $x = 0$ and $x = L$. It is tempting to apply a quadratic solution as described in Sections 2.2.1 and 2.3.3, but it turns out that this solution is no longer an exact solution

of the discrete equations if a Neumann condition is implemented on the boundary. A linear solution does not help since we only have homogeneous Neumann conditions in `wave1D_dn.py`, and we are consequently left with testing just a constant solution: $u = \text{const}$.

```
def test_constant():
    """
    Check the scalar and vectorized versions for
    a constant u(x,t). We simulate in [0, L] and apply
    Neumann and Dirichlet conditions at both ends.
    """
    u_const = 0.45
    u_exact = lambda x, t: u_const
    I = lambda x: u_exact(x, 0)
    V = lambda x: 0
    f = lambda x, t: 0

    def assert_no_error(u, x, t, n):
        u_e = u_exact(x, t[n])
        diff = np.abs(u - u_e).max()
        msg = 'diff=%E, t_%d=%g' % (diff, n, t[n])
        tol = 1E-13
        assert diff < tol, msg

    for U_0 in (None, lambda t: u_const):
        for U_L in (None, lambda t: u_const):
            L = 2.5
            c = 1.5
            C = 0.75
            Nx = 3  # Very coarse mesh for this exact test
            dt = C*(L/Nx)/c
            T = 18  # long time integration

            solver(I, V, f, c, U_0, U_L, L, dt, C, T,
                   user_action=assert_no_error,
                   version='scalar')
            solver(I, V, f, c, U_0, U_L, L, dt, C, T,
                   user_action=assert_no_error,
                   version='vectorized')
            print U_0, U_L
```

The quadratic solution is very useful for testing, but it requires Dirichlet conditions at both ends.

Another test may utilize the fact that the approximation error vanishes when the Courant number is unity. We can, for example, start with a plug profile as initial condition, let this wave split into two plug waves, one in each direction, and check that the two plug waves come back and form the initial condition again after "one period" of the solution process. Neumann conditions can be applied at both ends. A proper test function reads

```
def test_plug():
    """Check that an initial plug is correct back after one period."""
    L = 1.0
    c = 0.5
    dt = (L/10)/c  # Nx=10
    I = lambda x: 0 if abs(x-L/2.0) > 0.1 else 1

    u_s, x, t, cpu = solver(
        I=I,
        V=None, f=None, c=0.5, U_0=None, U_L=None, L=L,
        dt=dt, C=1, T=4, user_action=None, version='scalar')
    u_v, x, t, cpu = solver(
        I=I,
        V=None, f=None, c=0.5, U_0=None, U_L=None, L=L,
        dt=dt, C=1, T=4, user_action=None, version='vectorized')
    tol = 1E-13
    diff = abs(u_s - u_v).max()
    assert diff < tol
    u_0 = np.array([I(x_) for x_ in x])
    diff = np.abs(u_s - u_0).max()
    assert diff < tol
```

Other tests must rely on an unknown approximation error, so effectively we are left with tests on the convergence rate.

2.6.6 Alternative implementation via ghost cells

Idea. Instead of modifying the scheme at the boundary, we can introduce extra points outside the domain such that the fictitious values u^n_{-1} and $u^n_{N_x+1}$ are defined in the mesh. Adding the intervals $[-\Delta x, 0]$ and $[L, L+\Delta x]$, known as *ghost cells*, to the mesh gives us all the needed mesh points, corresponding to $i = -1, 0, \ldots, N_x, N_x+1$. The extra points with $i = -1$ and $i = N_x + 1$ are known as *ghost points*, and values at these points, u^n_{-1} and $u^n_{N_x+1}$, are called *ghost values*.

The important idea is to ensure that we always have

$$u^n_{-1} = u^n_1 \text{ and } u^n_{N_x+1} = u^n_{N_x-1},$$

because then the application of the standard scheme at a boundary point $i = 0$ or $i = N_x$ will be correct and guarantee that the solution is compatible with the boundary condition $u_x = 0$.

Some readers may find it strange to just extend the domain with ghost cells as a general technique, because in some problems there is a completely different medium with different physics and equations right outside of a boundary. Nevertheless, one should view the ghost cell

technique as a purely mathematical technique, which is valid in the limit $\Delta x \to 0$ and helps us to implement derivatives.

Implementation. The `u` array now needs extra elements corresponding to the ghost points. Two new point values are needed:

```
u   = zeros(Nx+3)
```

The arrays `u_n` and `u_nm1` must be defined accordingly.

Unfortunately, a major indexing problem arises with ghost cells. The reason is that Python indices *must* start at 0 and `u[-1]` will always mean the last element in `u`. This fact gives, apparently, a mismatch between the mathematical indices $i = -1, 0, \ldots, N_x + 1$ and the Python indices running over `u`: `0,..,Nx+2`. One remedy is to change the mathematical indexing of i in the scheme and write

$$u_i^{n+1} = \cdots, \quad i = 1, \ldots, N_x + 1,$$

instead of $i = 0, \ldots, N_x$ as we have previously used. The ghost points now correspond to $i = 0$ and $i = N_x + 1$. A better solution is to use the ideas of Section 2.6.4: we hide the specific index value in an index set and operate with inner and boundary points using the index set notation.

To this end, we define `u` with proper length and `Ix` to be the corresponding indices for the real physical mesh points $(1, 2, \ldots, N_x + 1)$:

```
u = zeros(Nx+3)
Ix = range(1, u.shape[0]-1)
```

That is, the boundary points have indices `Ix[0]` and `Ix[-1]` (as before). We first update the solution at all physical mesh points (i.e., interior points in the mesh):

```
for i in Ix:
    u[i] = - u_nm1[i] + 2*u_n[i] + \
           C2*(u_n[i-1] - 2*u_n[i] + u_n[i+1])
```

The indexing becomes a bit more complicated when we call functions like `V(x)` and `f(x, t)`, as we must remember that the appropriate x coordinate is given as `x[i-Ix[0]]`:

```
for i in Ix:
    u[i] = u_n[i] + dt*V(x[i-Ix[0]]) + \
           0.5*C2*(u_n[i-1] - 2*u_n[i] + u_n[i+1]) + \
           0.5*dt2*f(x[i-Ix[0]], t[0])
```

It remains to update the solution at ghost points, i.e., `u[0]` and `u[-1]` (or `u[Nx+2]`). For a boundary condition $u_x = 0$, the ghost value must

equal the value at the associated inner mesh point. Computer code makes this statement precise:

```
i = Ix[0]          # x=0 boundary
u[i-1] = u[i+1]
i = Ix[-1]         # x=L boundary
u[i+1] = u[i-1]
```

The physical solution to be plotted is now in `u[1:-1]`, or equivalently `u[Ix[0]:Ix[-1]+1]`, so this slice is the quantity to be returned from a solver function. A complete implementation appears in the program `wave1D_n0_ghost.py`.

Warning

We have to be careful with how the spatial and temporal mesh points are stored. Say we let `x` be the physical mesh points,

```
x = linspace(0, L, Nx+1)
```

"Standard coding" of the initial condition,

```
for i in Ix:
    u_n[i] = I(x[i])
```

becomes wrong, since `u_n` and `x` have different lengths and the index `i` corresponds to two different mesh points. In fact, `x[i]` corresponds to `u[1+i]`. A correct implementation is

```
for i in Ix:
    u_n[i] = I(x[i-Ix[0]])
```

Similarly, a source term usually coded as `f(x[i], t[n])` is incorrect if `x` is defined to be the physical points, so `x[i]` must be replaced by `x[i-Ix[0]]`.

An alternative remedy is to let `x` also cover the ghost points such that `u[i]` is the value at `x[i]`.

The ghost cell is only added to the boundary where we have a Neumann condition. Suppose we have a Dirichlet condition at $x = L$ and a homogeneous Neumann condition at $x = 0$. One ghost cell $[-\Delta x, 0]$ is added to the mesh, so the index set for the physical points becomes $\{1, \ldots, N_x + 1\}$. A relevant implementation is

```
u = zeros(Nx+2)
Ix = range(1, u.shape[0])
...
```

```
for i in Ix[:-1]:
    u[i] = - u_nm1[i] + 2*u_n[i] + \
           C2*(u_n[i-1] - 2*u_n[i] + u_n[i+1]) + \
           dt2*f(x[i-Ix[0]], t[n])
i = Ix[-1]
u[i] = U_0       # set Dirichlet value
i = Ix[0]
u[i-1] = u[i+1]  # update ghost value
```

The physical solution to be plotted is now in `u[1:]` or (as always) `u[Ix[0]:Ix[-1]+1]`.

2.7 Generalization: variable wave velocity

Our next generalization of the 1D wave equation (2.1) or (2.17) is to allow for a variable wave velocity c: $c = c(x)$, usually motivated by wave motion in a domain composed of different physical media. When the media differ in physical properties like density or porosity, the wave velocity c is affected and will depend on the position in space. Figure 2.5 shows a wave propagating in one medium $[0, 0.7] \cup [0.9, 1]$ with wave velocity c_1 (left) before it enters a second medium $(0.7, 0.9)$ with wave velocity c_2 (right). When the wave meets the boundary where c jumps from c_1 to c_2, a part of the wave is reflected back into the first medium (the *reflected* wave), while one part is transmitted through the second medium (the *transmitted* wave).

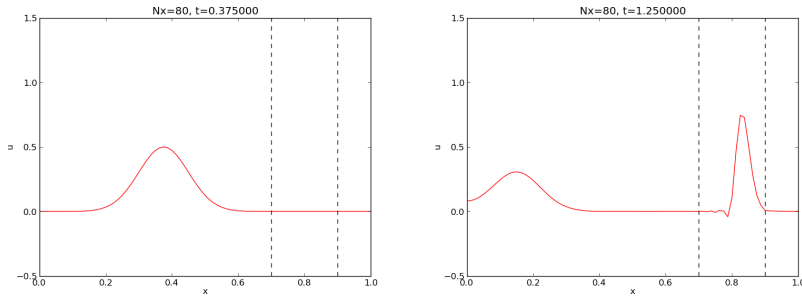


Fig. 2.5 Left: wave entering another medium; right: transmitted and reflected wave.

2.7.1 The model PDE with a variable coefficient

Instead of working with the squared quantity $c^2(x)$, we shall for notational convenience introduce $q(x) = c^2(x)$. A 1D wave equation with variable wave velocity often takes the form

$$\frac{\partial^2 u}{\partial t^2} = \frac{\partial}{\partial x}\left(q(x)\frac{\partial u}{\partial x}\right) + f(x,t)\,. \tag{2.42}$$

This is the most frequent form of a wave equation with variable wave velocity, but other forms also appear, see Section 2.14.1 and equation (2.125).

As usual, we sample (2.42) at a mesh point,

$$\frac{\partial^2}{\partial t^2}u(x_i,t_n) = \frac{\partial}{\partial x}\left(q(x_i)\frac{\partial}{\partial x}u(x_i,t_n)\right) + f(x_i,t_n),$$

where the only new term to discretize is

$$\frac{\partial}{\partial x}\left(q(x_i)\frac{\partial}{\partial x}u(x_i,t_n)\right) = \left[\frac{\partial}{\partial x}\left(q(x)\frac{\partial u}{\partial x}\right)\right]_i^n\,.$$

2.7.2 Discretizing the variable coefficient

The principal idea is to first discretize the outer derivative. Define

$$\phi = q(x)\frac{\partial u}{\partial x},$$

and use a centered derivative around $x = x_i$ for the derivative of ϕ:

$$\left[\frac{\partial \phi}{\partial x}\right]_i^n \approx \frac{\phi_{i+\frac{1}{2}} - \phi_{i-\frac{1}{2}}}{\Delta x} = [D_x\phi]_i^n\,.$$

Then discretize

$$\phi_{i+\frac{1}{2}} = q_{i+\frac{1}{2}}\left[\frac{\partial u}{\partial x}\right]_{i+\frac{1}{2}}^n \approx q_{i+\frac{1}{2}}\frac{u_{i+1}^n - u_i^n}{\Delta x} = [qD_x u]_{i+\frac{1}{2}}^n\,.$$

Similarly,

$$\phi_{i-\frac{1}{2}} = q_{i-\frac{1}{2}}\left[\frac{\partial u}{\partial x}\right]_{i-\frac{1}{2}}^n \approx q_{i-\frac{1}{2}}\frac{u_i^n - u_{i-1}^n}{\Delta x} = [qD_x u]_{i-\frac{1}{2}}^n\,.$$

These intermediate results are now combined to

$$\left[\frac{\partial}{\partial x}\left(q(x)\frac{\partial u}{\partial x}\right)\right]_i^n \approx \frac{1}{\Delta x^2}\left(q_{i+\frac{1}{2}}\left(u_{i+1}^n - u_i^n\right) - q_{i-\frac{1}{2}}\left(u_i^n - u_{i-1}^n\right)\right). \tag{2.43}$$

With operator notation we can write the discretization as

$$\left[\frac{\partial}{\partial x}\left(q(x)\frac{\partial u}{\partial x}\right)\right]_i^n \approx [D_x(\overline{q}^x D_x u)]_i^n. \tag{2.44}$$

Do not use the chain rule on the spatial derivative term!

Many are tempted to use the chain rule on the term $\frac{\partial}{\partial x}\left(q(x)\frac{\partial u}{\partial x}\right)$, but this is not a good idea when discretizing such a term.

The term with a variable coefficient expresses the net flux qu_x into a small volume (i.e., interval in 1D):

$$\frac{\partial}{\partial x}\left(q(x)\frac{\partial u}{\partial x}\right) \approx \frac{1}{\Delta x}(q(x+\Delta x)u_x(x+\Delta x) - q(x)u_x(x)).$$

Our discretization reflects this principle directly: qu_x at the right end of the cell minus qu_x at the left end, because this follows from the formula (2.43) or $[D_x(qD_x u)]_i^n$.

When using the chain rule, we get two terms $qu_{xx} + q_x u_x$. The typical discretization is

$$[D_x q D_x u + D_{2x} q D_{2x} u]_i^n, \tag{2.45}$$

Writing this out shows that it is different from $[D_x(qD_x u)]_i^n$ and lacks the physical interpretation of net flux into a cell. With a smooth and slowly varying $q(x)$ the differences between the two discretizations are not substantial. However, when q exhibits (potentially large) jumps, $[D_x(qD_x u)]_i^n$ with harmonic averaging of q yields a better solution than arithmetic averaging or (2.45). In the literature, the discretization $[D_x(qD_x u)]_i^n$ totally dominates and very few mention the alternative in (2.45).

2.7.3 Computing the coefficient between mesh points

If q is a known function of x, we can easily evaluate $q_{i+\frac{1}{2}}$ simply as $q(x_{i+\frac{1}{2}})$ with $x_{i+\frac{1}{2}} = x_i + \frac{1}{2}\Delta x$. However, in many cases c, and hence q, is only known as a discrete function, often at the mesh points x_i. Evaluating q between two mesh points x_i and x_{i+1} must then be done by *interpolation* techniques, of which three are of particular interest in this context:

$$q_{i+\frac{1}{2}} \approx \frac{1}{2}\left(q_i + q_{i+1}\right) = [\overline{q}^x]_i \quad \text{(arithmetic mean)} \tag{2.46}$$

$$q_{i+\frac{1}{2}} \approx 2\left(\frac{1}{q_i} + \frac{1}{q_{i+1}}\right)^{-1} \quad \text{(harmonic mean)} \tag{2.47}$$

$$q_{i+\frac{1}{2}} \approx \left(q_i q_{i+1}\right)^{1/2} \quad \text{(geometric mean)} \tag{2.48}$$

The arithmetic mean in (2.46) is by far the most commonly used averaging technique and is well suited for smooth $q(x)$ functions. The harmonic mean is often preferred when $q(x)$ exhibits large jumps (which is typical for geological media). The geometric mean is less used, but popular in discretizations to linearize quadratic nonlinearities (see Section 1.10.2 for an example).

With the operator notation from (2.46) we can specify the discretization of the complete variable-coefficient wave equation in a compact way:

$$[D_t D_t u = D_x \overline{q}^x D_x u + f]_i^n \,. \tag{2.49}$$

Strictly speaking, $[D_x \overline{q}^x D_x u]_i^n = [D_x(\overline{q}^x D_x u)]_i^n$.

From the compact difference notation we immediately see what kind of differences that each term is approximated with. The notation $\overline{q}^x$ also specifies that the variable coefficient is approximated by an arithmetic mean, the definition being $[\overline{q}^x]_{i+\frac{1}{2}} = (q_i + q_{i+1})/2$.

Before implementing, it remains to solve (2.49) with respect to u_i^{n+1}:

$$\begin{aligned} u_i^{n+1} = &-u_i^{n-1} + 2u_i^n + \\ &\left(\frac{\Delta t}{\Delta x}\right)^2 \left(\frac{1}{2}(q_i + q_{i+1})(u_{i+1}^n - u_i^n) - \frac{1}{2}(q_i + q_{i-1})(u_i^n - u_{i-1}^n)\right) + \\ &\Delta t^2 f_i^n \,. \end{aligned} \tag{2.50}$$

2.7.4 How a variable coefficient affects the stability

The stability criterion derived later (Section 2.10.3) reads $\Delta t \leq \Delta x/c$. If $c = c(x)$, the criterion will depend on the spatial location. We must therefore choose a Δt that is small enough such that no mesh cell has $\Delta t > \Delta x/c(x)$. That is, we must use the largest c value in the criterion:

$$\Delta t \leq \beta \frac{\Delta x}{\max_{x\in[0,L]} c(x)}\,. \tag{2.51}$$

The parameter β is included as a safety factor: in some problems with a significantly varying c it turns out that one must choose $\beta < 1$ to have stable solutions ($\beta = 0.9$ may act as an all-round value).

A different strategy to handle the stability criterion with variable wave velocity is to use a spatially varying Δt. While the idea is mathematically attractive at first sight, the implementation quickly becomes very complicated, so we stick to a constant Δt and a worst case value of $c(x)$ (with a safety factor β).

2.7.5 Neumann condition and a variable coefficient

Consider a Neumann condition $\partial u/\partial x = 0$ at $x = L = N_x\Delta x$, discretized as

$$[D_{2x}u]_i^n = \frac{u_{i+1}^n - u_{i-1}^n}{2\Delta x} = 0 \quad \Rightarrow \quad u_{i+1}^n = u_{i-1}^n,$$

for $i = N_x$. Using the scheme (2.50) at the end point $i = N_x$ with $u_{i+1}^n = u_{i-1}^n$ results in

$$\begin{aligned} u_i^{n+1} &= -u_i^{n-1} + 2u_i^n + \\ &\quad \left(\frac{\Delta t}{\Delta x}\right)^2 \left(q_{i+\frac{1}{2}}(u_{i-1}^n - u_i^n) - q_{i-\frac{1}{2}}(u_i^n - u_{i-1}^n)\right) + \Delta t^2 f_i^n \qquad (2.52)\\ &= -u_i^{n-1} + 2u_i^n + \left(\frac{\Delta t}{\Delta x}\right)^2 (q_{i+\frac{1}{2}} + q_{i-\frac{1}{2}})(u_{i-1}^n - u_i^n) + \Delta t^2 f_i^n \qquad (2.53)\\ &\approx -u_i^{n-1} + 2u_i^n + \left(\frac{\Delta t}{\Delta x}\right)^2 2q_i(u_{i-1}^n - u_i^n) + \Delta t^2 f_i^n\,. \qquad (2.54) \end{aligned}$$

Here we used the approximation

$$
\begin{aligned}
q_{i+\frac{1}{2}} + q_{i-\frac{1}{2}} &= q_i + \left(\frac{dq}{dx}\right)_i \Delta x + \left(\frac{d^2q}{dx^2}\right)_i \Delta x^2 + \cdots + \\
&\quad q_i - \left(\frac{dq}{dx}\right)_i \Delta x + \left(\frac{d^2q}{dx^2}\right)_i \Delta x^2 + \cdots \\
&= 2q_i + 2\left(\frac{d^2q}{dx^2}\right)_i \Delta x^2 + \mathcal{O}(\Delta x^4) \\
&\approx 2q_i\,. \qquad (2.55)
\end{aligned}
$$

An alternative derivation may apply the arithmetic mean of $q_{n-\frac{1}{2}}$ and $q_{n+\frac{1}{2}}$ in (2.53), leading to the term

$$
(q_i + \frac{1}{2}(q_{i+1} + q_{i-1}))(u^n_{i-1} - u^n_i)\,.
$$

Since $\frac{1}{2}(q_{i+1}+q_{i-1}) = q_i + \mathcal{O}(\Delta x^2)$, we can approximate with $2q_i(u^n_{i-1} - u^n_i)$ for $i = N_x$ and get the same term as we did above.

A common technique when implementing $\partial u/\partial x = 0$ boundary conditions, is to assume $dq/dx = 0$ as well. This implies $q_{i+1} = q_{i-1}$ and $q_{i+1/2} = q_{i-1/2}$ for $i = N_x$. The implications for the scheme are

$$
\begin{aligned}
u^{n+1}_i &= -u^{n-1}_i + 2u^n_i + \\
&\quad \left(\frac{\Delta t}{\Delta x}\right)^2 \left(q_{i+\frac{1}{2}}(u^n_{i-1} - u^n_i) - q_{i-\frac{1}{2}}(u^n_i - u^n_{i-1})\right) + \\
&\quad \Delta t^2 f^n_i \qquad (2.56) \\
&= -u^{n-1}_i + 2u^n_i + \left(\frac{\Delta t}{\Delta x}\right)^2 2q_{i-\frac{1}{2}}(u^n_{i-1} - u^n_i) + \Delta t^2 f^n_i\,. \qquad (2.57)
\end{aligned}
$$

2.7.6 Implementation of variable coefficients

The implementation of the scheme with a variable wave velocity $q(x) = c^2(x)$ may assume that q is available as an array `q[i]` at the spatial mesh points. The following loop is a straightforward implementation of the scheme (2.50):

```
for i in range(1, Nx):
    u[i] = - u_nm1[i] + 2*u_n[i] + \
           C2*(0.5*(q[i] + q[i+1])*(u_n[i+1] - u_n[i])  - \
               0.5*(q[i] + q[i-1])*(u_n[i] - u_n[i-1])) + \
           dt2*f(x[i], t[n])
```

The coefficient `C2` is now defined as `(dt/dx)**2`, i.e., *not* as the squared Courant number, since the wave velocity is variable and appears inside the parenthesis.

With Neumann conditions $u_x = 0$ at the boundary, we need to combine this scheme with the discrete version of the boundary condition, as shown in Section 2.7.5. Nevertheless, it would be convenient to reuse the formula for the interior points and just modify the indices `ip1=i+1` and `im1=i-1` as we did in Section 2.6.3. Assuming $dq/dx = 0$ at the boundaries, we can implement the scheme at the boundary with the following code.

```
i = 0
ip1 = i+1
im1 = ip1
u[i] = - u_nm1[i] + 2*u_n[i] + \
       C2*(0.5*(q[i] + q[ip1])*(u_n[ip1] - u_n[i])  - \
           0.5*(q[i] + q[im1])*(u_n[i] - u_n[im1])) + \
       dt2*f(x[i], t[n])
```

With ghost cells we can just reuse the formula for the interior points also at the boundary, provided that the ghost values of both u and q are correctly updated to ensure $u_x = 0$ and $q_x = 0$.

A vectorized version of the scheme with a variable coefficient at internal mesh points becomes

```
u[1:-1] = - u_nm1[1:-1] + 2*u_n[1:-1] + \
          C2*(0.5*(q[1:-1] + q[2:])*(u_n[2:] - u_n[1:-1]) -
              0.5*(q[1:-1] + q[:-2])*(u_n[1:-1] - u_n[:-2])) + \
          dt2*f(x[1:-1], t[n])
```

2.7.7 A more general PDE model with variable coefficients

Sometimes a wave PDE has a variable coefficient in front of the time-derivative term:

$$\varrho(x)\frac{\partial^2 u}{\partial t^2} = \frac{\partial}{\partial x}\left(q(x)\frac{\partial u}{\partial x}\right) + f(x,t)\,. \tag{2.58}$$

One example appears when modeling elastic waves in a rod with varying density, cf. (2.14.1) with $\varrho(x)$.

A natural scheme for (2.58) is

$$[\varrho D_t D_t u = D_x \overline{q}^x D_x u + f]_i^n\,. \tag{2.59}$$

We realize that the ϱ coefficient poses no particular difficulty, since ϱ enters the formula just as a simple factor in front of a derivative. There

is hence no need for any averaging of ϱ. Often, ϱ will be moved to the right-hand side, also without any difficulty:

$$[D_t D_t u = \varrho^{-1} D_x \overline{q}^x D_x u + f]_i^n \,. \tag{2.60}$$

2.7.8 Generalization: damping

Waves die out by two mechanisms. In 2D and 3D the energy of the wave spreads out in space, and energy conservation then requires the amplitude to decrease. This effect is not present in 1D. Damping is another cause of amplitude reduction. For example, the vibrations of a string die out because of damping due to air resistance and non-elastic effects in the string.

The simplest way of including damping is to add a first-order derivative to the equation (in the same way as friction forces enter a vibrating mechanical system):

$$\frac{\partial^2 u}{\partial t^2} + b\frac{\partial u}{\partial t} = c^2 \frac{\partial^2 u}{\partial x^2} + f(x,t), \tag{2.61}$$

where $b \geq 0$ is a prescribed damping coefficient.

A typical discretization of (2.61) in terms of centered differences reads

$$[D_t D_t u + b D_{2t} u = c^2 D_x D_x u + f]_i^n \,. \tag{2.62}$$

Writing out the equation and solving for the unknown u_i^{n+1} gives the scheme

$$u_i^{n+1} = (1+\frac{1}{2}b\Delta t)^{-1}((\frac{1}{2}b\Delta t - 1)u_i^{n-1} + 2u_i^n + C^2\left(u_{i+1}^n - 2u_i^n + u_{i-1}^n\right) + \Delta t^2 f_i^n), \tag{2.63}$$

for $i \in \mathcal{I}_x^i$ and $n \geq 1$. New equations must be derived for u_i^1, and for boundary points in case of Neumann conditions.

The damping is very small in many wave phenomena and thus only evident for very long time simulations. This makes the standard wave equation without damping relevant for a lot of applications.

2.8 Building a general 1D wave equation solver

The program `wave1D_dn_vc.py` is a fairly general code for 1D wave propagation problems that targets the following initial-boundary value problem

$$u_{tt} = (c^2(x)u_x)_x + f(x,t), \qquad x \in (0,L),\ t \in (0,T] \tag{2.64}$$
$$u(x,0) = I(x), \qquad x \in [0,L] \tag{2.65}$$
$$u_t(x,0) = V(t), \qquad x \in [0,L] \tag{2.66}$$
$$u(0,t) = U_0(t) \text{ or } u_x(0,t) = 0, \qquad t \in (0,T] \tag{2.67}$$
$$u(L,t) = U_L(t) \text{ or } u_x(L,t) = 0, \qquad t \in (0,T] \tag{2.68}$$

The only new feature here is the time-dependent Dirichlet conditions, but they are trivial to implement:

```
i = Ix[0]  # x=0
u[i] = U_0(t[n+1])

i = Ix[-1] # x=L
u[i] = U_L(t[n+1])
```

The `solver` function is a natural extension of the simplest `solver` function in the initial `wave1D_u0.py` program, extended with Neumann boundary conditions ($u_x = 0$), time-varying Dirichlet conditions, as well as a variable wave velocity. The different code segments needed to make these extensions have been shown and commented upon in the preceding text. We refer to the `solver` function in the `wave1D_dn_vc.py` file for all the details. Note in that `solver` function, however, that the technique of "hashing" is used to check whether a certain simulation has been run before, or not. This technique is further explained in Section C.2.3.

The vectorization is only applied inside the time loop, not for the initial condition or the first time steps, since this initial work is negligible for long time simulations in 1D problems.

The following sections explain various more advanced programming techniques applied in the general 1D wave equation solver.

2.8.1 User action function as a class

A useful feature in the `wave1D_dn_vc.py` program is the specification of the `user_action` function as a class. This part of the program may need

some motivation and explanation. Although the `plot_u_st` function (and the `PlotMatplotlib` class) in the `wave1D_u0.viz` function remembers the local variables in the `viz` function, it is a cleaner solution to store the needed variables together with the function, which is exactly what a class offers.

The code. A class for flexible plotting, cleaning up files, making movie files, like the function `wave1D_u0.viz` did, can be coded as follows:

```
class PlotAndStoreSolution:
    """
    Class for the user_action function in solver.
    Visualizes the solution only.
    """
    def __init__(
        self,
        casename='tmp',    # Prefix in filenames
        umin=-1, umax=1,   # Fixed range of y axis
        pause_between_frames=None,  # Movie speed
        backend='matplotlib',       # or 'gnuplot' or None
        screen_movie=True, # Show movie on screen?
        title='',          # Extra message in title
        skip_frame=1,      # Skip every skip_frame frame
        filename=None):    # Name of file with solutions
        self.casename = casename
        self.yaxis = [umin, umax]
        self.pause = pause_between_frames
        self.backend = backend
        if backend is None:
            # Use native matplotlib
            import matplotlib.pyplot as plt
        elif backend in ('matplotlib', 'gnuplot'):
            module = 'scitools.easyviz.' + backend + '_'
            exec('import %s as plt' % module)
        self.plt = plt
        self.screen_movie = screen_movie
        self.title = title
        self.skip_frame = skip_frame
        self.filename = filename
        if filename is not None:
            # Store time points when u is written to file
            self.t = []
            filenames = glob.glob('.' + self.filename + '*.dat.npz')
            for filename in filenames:
                os.remove(filename)

        # Clean up old movie frames
        for filename in glob.glob('frame_*.png'):
            os.remove(filename)

    def __call__(self, u, x, t, n):
        """
```

```
        Callback function user_action, call by solver:
        Store solution, plot on screen and save to file.
        """
        # Save solution u to a file using numpy.savez
        if self.filename is not None:
            name = 'u%04d' % n  # array name
            kwargs = {name: u}
            fname = '.' + self.filename + '_' + name + '.dat'
            np.savez(fname, **kwargs)
            self.t.append(t[n])  # store corresponding time value
            if n == 0:           # save x once
                np.savez('.' + self.filename + '_x.dat', x=x)

        # Animate
        if n % self.skip_frame != 0:
            return
        title = 't=%.3f' % t[n]
        if self.title:
            title = self.title + ' ' + title
        if self.backend is None:
            # native matplotlib animation
            if n == 0:
                self.plt.ion()
                self.lines = self.plt.plot(x, u, 'r-')
                self.plt.axis([x[0], x[-1],
                               self.yaxis[0], self.yaxis[1]])
                self.plt.xlabel('x')
                self.plt.ylabel('u')
                self.plt.title(title)
                self.plt.legend(['t=%.3f' % t[n]])
            else:
                # Update new solution
                self.lines[0].set_ydata(u)
                self.plt.legend(['t=%.3f' % t[n]])
                self.plt.draw()
        else:
            # scitools.easyviz animation
            self.plt.plot(x, u, 'r-',
                          xlabel='x', ylabel='u',
                          axis=[x[0], x[-1],
                                self.yaxis[0], self.yaxis[1]],
                          title=title,
                          show=self.screen_movie)
        # pause
        if t[n] == 0:
            time.sleep(2)  # let initial condition stay 2 s
        else:
            if self.pause is None:
                pause = 0.2 if u.size < 100 else 0
            time.sleep(pause)

        self.plt.savefig('frame_%04d.png' % (n))
```

Dissection. Understanding this class requires quite some familiarity with Python in general and class programming in particular. The class supports plotting with Matplotlib (`backend=None`) or SciTools (`backend=matplotlib` or `backend=gnuplot`) for maximum flexibility.

The constructor shows how we can flexibly import the plotting engine as (typically) `scitools.easyviz.gnuplot_` or `scitools.easyviz.matplotlib_` (note the trailing underscore - it is required). With the `screen_movie` parameter we can suppress displaying each movie frame on the screen. Alternatively, for slow movies associated with fine meshes, one can set `skip_frame=10`, causing every 10 frames to be shown.

The `__call__` method makes `PlotAndStoreSolution` instances behave like functions, so we can just pass an instance, say `p`, as the `user_action` argument in the `solver` function, and any call to `user_action` will be a call to `p.__call__`. The `__call__` method plots the solution on the screen, saves the plot to file, and stores the solution in a file for later retrieval.

More details on storing the solution in files appear in Section C.2.

2.8.2 Pulse propagation in two media

The function `pulse` in `wave1D_dn_vc.py` demonstrates wave motion in heterogeneous media where c varies. One can specify an interval where the wave velocity is decreased by a factor `slowness_factor` (or increased by making this factor less than one). Figure 2.5 shows a typical simulation scenario.

Four types of initial conditions are available:

1. a rectangular pulse (`plug`),
2. a Gaussian function (`gaussian`),
3. a "cosine hat" consisting of one period of the cosine function (`cosinehat`),
4. half a period of a "cosine hat" (`half-cosinehat`)

These peak-shaped initial conditions can be placed in the middle (`loc='center'`) or at the left end (`loc='left'`) of the domain. With the pulse in the middle, it splits in two parts, each with half the initial amplitude, traveling in opposite directions. With the pulse at the left end, centered at $x = 0$, and using the symmetry condition $\partial u/\partial x = 0$, only a right-going pulse is generated. There is also a left-going pulse, but

it travels from $x = 0$ in negative x direction and is not visible in the domain $[0, L]$.

The `pulse` function is a flexible tool for playing around with various wave shapes and jumps in the wave velocity (i.e., discontinuous media). The code is shown to demonstrate how easy it is to reach this flexibility with the building blocks we have already developed:

```
def pulse(
    C=1,            # Maximum Courant number
    Nx=200,         # spatial resolution
    animate=True,
    version='vectorized',
    T=2,            # end time
    loc='left',     # location of initial condition
    pulse_tp='gaussian',  # pulse/init.cond. type
    slowness_factor=2, # inverse of wave vel. in right medium
    medium=[0.7, 0.9], # interval for right medium
    skip_frame=1,      # skip frames in animations
    sigma=0.05         # width measure of the pulse
    ):
    """
    Various peaked-shaped initial conditions on [0,1].
    Wave velocity is decreased by the slowness_factor inside
    medium. The loc parameter can be 'center' or 'left',
    depending on where the initial pulse is to be located.
    The sigma parameter governs the width of the pulse.
    """
    # Use scaled parameters: L=1 for domain length, c_0=1
    # for wave velocity outside the domain.
    L = 1.0
    c_0 = 1.0
    if loc == 'center':
        xc = L/2
    elif loc == 'left':
        xc = 0

    if pulse_tp in ('gaussian','Gaussian'):
        def I(x):
            return np.exp(-0.5*((x-xc)/sigma)**2)
    elif pulse_tp == 'plug':
        def I(x):
            return 0 if abs(x-xc) > sigma else 1
    elif pulse_tp == 'cosinehat':
        def I(x):
            # One period of a cosine
            w = 2
            a = w*sigma
            return 0.5*(1 + np.cos(np.pi*(x-xc)/a)) \
                   if xc - a <= x <= xc + a else 0

    elif pulse_tp == 'half-cosinehat':
        def I(x):
```

```
            # Half a period of a cosine
            w = 4
            a = w*sigma
            return np.cos(np.pi*(x-xc)/a) \
                   if xc - 0.5*a <= x <= xc + 0.5*a else 0
    else:
        raise ValueError('Wrong pulse_tp="%s"' % pulse_tp)

    def c(x):
        return c_0/slowness_factor \
               if medium[0] <= x <= medium[1] else c_0

    umin=-0.5; umax=1.5*I(xc)
    casename = '%s_Nx%s_sf%s' % \
               (pulse_tp, Nx, slowness_factor)
    action = PlotMediumAndSolution(
        medium, casename=casename, umin=umin, umax=umax,
        skip_frame=skip_frame, screen_movie=animate,
        backend=None, filename='tmpdata')

    # Choose the stability limit with given Nx, worst case c
    # (lower C will then use this dt, but smaller Nx)
    dt = (L/Nx)/c_0
    cpu, hashed_input = solver(
        I=I, V=None, f=None, c=c,
        U_0=None, U_L=None,
        L=L, dt=dt, C=C, T=T,
        user_action=action,
        version=version,
        stability_safety_factor=1)

    if cpu > 0:  # did we generate new data?
        action.close_file(hashed_input)
        action.make_movie_file()
    print 'cpu (-1 means no new data generated):', cpu

def convergence_rates(
    u_exact,
    I, V, f, c, U_0, U_L, L,
    dt0, num_meshes,
    C, T, version='scalar',
    stability_safety_factor=1.0):
    """
    Half the time step and estimate convergence rates for
    for num_meshes simulations.
    """
    class ComputeError:
        def __init__(self, norm_type):
            self.error = 0

        def __call__(self, u, x, t, n):
            """Store norm of the error in self.E."""
            error = np.abs(u - u_exact(x, t[n])).max()
            self.error = max(self.error, error)
```

```
    E = []
    h = []  # dt, solver adjusts dx such that C=dt*c/dx
    dt = dt0
    for i in range(num_meshes):
        error_calculator = ComputeError('Linf')
        solver(I, V, f, c, U_0, U_L, L, dt, C, T,
               user_action=error_calculator,
               version='scalar',
               stability_safety_factor=1.0)
        E.append(error_calculator.error)
        h.append(dt)
        dt /= 2  # halve the time step for next simulation
    print 'E:', E
    print 'h:', h
    r = [np.log(E[i]/E[i-1])/np.log(h[i]/h[i-1])
         for i in range(1,num_meshes)]
    return r

def test_convrate_sincos():
    n = m = 2
    L = 1.0
    u_exact = lambda x, t: np.cos(m*np.pi/L*t)*np.sin(m*np.pi/L*x)

    r = convergence_rates(
        u_exact=u_exact,
        I=lambda x: u_exact(x, 0),
        V=lambda x: 0,
        f=0,
        c=1,
        U_0=0,
        U_L=0,
        L=L,
        dt0=0.1,
        num_meshes=6,
        C=0.9,
        T=1,
        version='scalar',
        stability_safety_factor=1.0)
    print 'rates sin(x)*cos(t) solution:', \
          [round(r_,2) for r_ in r]
    assert abs(r[-1] - 2) < 0.002
```

The `PlotMediumAndSolution` class used here is a subclass of `PlotAndStoreSolution` where the medium with reduced c value, as specified by the `medium` interval, is visualized in the plots.

Comment on the choices of discretization parameters

The argument N_x in the `pulse` function does not correspond to the actual spatial resolution of $C < 1$, since the `solver` function takes a fixed Δt and C, and adjusts Δx accordingly. As seen in the `pulse` function, the specified Δt is chosen according to the limit $C = 1$, so if $C < 1$, Δt remains the same, but the `solver` function operates with a larger Δx and smaller N_x than was specified in the call to `pulse`. The practical reason is that we always want to keep Δt fixed such that plot frames and movies are synchronized in time regardless of the value of C (i.e., Δx is varied when the Courant number varies).

The reader is encouraged to play around with the `pulse` function:

```
>>> import wave1D_dn_vc as w
>>> w.pulse(Nx=50, loc='left', pulse_tp='cosinehat', slowness_factor=2)
```

To easily kill the graphics by Ctrl-C and restart a new simulation it might be easier to run the above two statements from the command line with

Terminal

```
Terminal> python -c 'import wave1D_dn_vc as w; w.pulse(...)'
```

2.9 Exercises

Exercise 2.7: Find the analytical solution to a damped wave equation

Consider the wave equation with damping (2.61). The goal is to find an exact solution to a wave problem with damping and zero source term. A starting point is the standing wave solution from Exercise 2.1. It becomes necessary to include a damping term $e^{-\beta t}$ and also have both a sine and cosine component in time:

$$u_{\mathrm{e}}(x,t) = e^{-\beta t}\sin kx\left(A\cos\omega t + B\sin\omega t\right) .$$

Find k from the boundary conditions $u(0,t) = u(L,t) = 0$. Then use the PDE to find constraints on β, ω, A, and B. Set up a complete initial-boundary value problem and its solution.
Filename: `damped_waves`.

Problem 2.8: Explore symmetry boundary conditions

Consider the simple "plug" wave where $\Omega = [-L, L]$ and

$$I(x) = \begin{cases} 1, x \in [-\delta, \delta], \\ 0, \text{otherwise} \end{cases}$$

for some number $0 < \delta < L$. The other initial condition is $u_t(x, 0) = 0$ and there is no source term f. The boundary conditions can be set to $u = 0$. The solution to this problem is symmetric around $x = 0$. This means that we can simulate the wave process in only half of the domain $[0, L]$.

a) Argue why the symmetry boundary condition is $u_x = 0$ at $x = 0$.

Hint. Symmetry of a function about $x = x_0$ means that $f(x_0 + h) = f(x_0 - h)$.

b) Perform simulations of the complete wave problem on $[-L, L]$. Thereafter, utilize the symmetry of the solution and run a simulation in half of the domain $[0, L]$, using a boundary condition at $x = 0$. Compare plots from the two solutions and confirm that they are the same.

c) Prove the symmetry property of the solution by setting up the complete initial-boundary value problem and showing that if $u(x, t)$ is a solution, then also $u(-x, t)$ is a solution.

d) If the code works correctly, the solution $u(x, t) = x(L - x)(1 + \frac{t}{2})$ should be reproduced exactly. Write a test function `test_quadratic` that checks whether this is the case. Simulate for x in $[0, \frac{L}{2}]$ with a symmetry condition at the end $x = \frac{L}{2}$.
Filename: `wave1D_symmetric`.

Exercise 2.9: Send pulse waves through a layered medium

Use the `pulse` function in `wave1D_dn_vc.py` to investigate sending a pulse, located with its peak at $x = 0$, through two media with different wave velocities. The (scaled) velocity in the left medium is 1 while it is $\frac{1}{s_f}$ in the right medium. Report what happens with a Gaussian pulse, a "cosine hat" pulse, half a "cosine hat" pulse, and a plug pulse for resolutions $N_x = 40, 80, 160$, and $s_f = 2, 4$. Simulate until $T = 2$.
Filename: `pulse1D`.

Exercise 2.10: Explain why numerical noise occurs

The experiments performed in Exercise 2.9 shows considerable numerical noise in the form of non-physical waves, especially for $s_f = 4$ and the plug pulse or the half a "cosinehat" pulse. The noise is much less visible for a Gaussian pulse. Run the case with the plug and half a "cosinehat" pulse for $s_f = 1$, $C = 0.9, 0.25$, and $N_x = 40, 80, 160$. Use the numerical dispersion relation to explain the observations. Filename: `pulse1D_analysis`.

Exercise 2.11: Investigate harmonic averaging in a 1D model

Harmonic means are often used if the wave velocity is non-smooth or discontinuous. Will harmonic averaging of the wave velocity give less numerical noise for the case $s_f = 4$ in Exercise 2.9? Filename: `pulse1D_harmonic`.

Problem 2.12: Implement open boundary conditions

To enable a wave to leave the computational domain and travel undisturbed through the boundary $x = L$, one can in a one-dimensional problem impose the following condition, called a *radiation condition* or *open boundary condition*:

$$\frac{\partial u}{\partial t} + c\frac{\partial u}{\partial x} = 0\,. \tag{2.69}$$

The parameter c is the wave velocity.

Show that (2.69) accepts a solution $u = g_R(x - ct)$ (right-going wave), but not $u = g_L(x + ct)$ (left-going wave). This means that (2.69) will allow any right-going wave $g_R(x - ct)$ to pass through the boundary undisturbed.

A corresponding open boundary condition for a left-going wave through $x = 0$ is

$$\frac{\partial u}{\partial t} - c\frac{\partial u}{\partial x} = 0\,. \tag{2.70}$$

a) A natural idea for discretizing the condition (2.69) at the spatial end point $i = N_x$ is to apply centered differences in time and space:

$$[D_{2t}u + cD_{2x}u = 0]_i^n, \quad i = N_x\,. \tag{2.71}$$

Eliminate the fictitious value $u^n_{N_x+1}$ by using the discrete equation at the same point.

The equation for the first step, u_i^1, is in principle also affected, but we can then use the condition $u_{N_x} = 0$ since the wave has not yet reached the right boundary.

b) A much more convenient implementation of the open boundary condition at $x = L$ can be based on an explicit discretization

$$[D_t^+u + cD_x^-u = 0]_i^n, \quad i = N_x\,. \tag{2.72}$$

From this equation, one can solve for $u^{n+1}_{N_x}$ and apply the formula as a Dirichlet condition at the boundary point. However, the finite difference approximations involved are of first order.

Implement this scheme for a wave equation $u_{tt} = c^2u_{xx}$ in a domain $[0, L]$, where you have $u_x = 0$ at $x = 0$, the condition (2.69) at $x = L$, and an initial disturbance in the middle of the domain, e.g., a plug profile like

$$u(x,0) = \begin{cases} 1, & L/2 - \ell \leq x \leq L/2 + \ell, \\ 0, \text{otherwise} \end{cases}$$

Observe that the initial wave is split in two, the left-going wave is reflected at $x = 0$, and both waves travel out of $x = L$, leaving the solution as $u = 0$ in $[0, L]$. Use a unit Courant number such that the numerical solution is exact. Make a movie to illustrate what happens.

Because this simplified implementation of the open boundary condition works, there is no need to pursue the more complicated discretization in a).

Hint. Modify the solver function in `wave1D_dn.py`.

c) Add the possibility to have either $u_x = 0$ or an open boundary condition at the left boundary. The latter condition is discretized as

$$[D_t^+u - cD_x^+u = 0]_i^n, \quad i = 0, \tag{2.73}$$

leading to an explicit update of the boundary value u_0^{n+1}.

The implementation can be tested with a Gaussian function as initial condition:

$$g(x; m, s) = \frac{1}{\sqrt{2\pi}s} e^{-\frac{(x-m)^2}{2s^2}}\,.$$

Run two tests:

1. Disturbance in the middle of the domain, $I(x) = g(x; L/2, s)$, and open boundary condition at the left end.
2. Disturbance at the left end, $I(x) = g(x; 0, s)$, and $u_x = 0$ as symmetry boundary condition at this end.

Make test functions for both cases, testing that the solution is zero after the waves have left the domain.

d) In 2D and 3D it is difficult to compute the correct wave velocity normal to the boundary, which is needed in generalizations of the open boundary conditions in higher dimensions. Test the effect of having a slightly wrong wave velocity in (2.72). Make movies to illustrate what happens.
Filename: `wave1D_open_BC`.

Remarks. The condition (2.69) works perfectly in 1D when c is known. In 2D and 3D, however, the condition reads $u_t + c_x u_x + c_y u_y = 0$, where c_x and c_y are the wave speeds in the x and y directions. Estimating these components (i.e., the direction of the wave) is often challenging. Other methods are normally used in 2D and 3D to let waves move out of a computational domain.

Exercise 2.13: Implement periodic boundary conditions

It is frequently of interest to follow wave motion over large distances and long times. A straightforward approach is to work with a very large domain, but that might lead to a lot of computations in areas of the domain where the waves cannot be noticed. A more efficient approach is to let a right-going wave out of the domain and at the same time let it enter the domain on the left. This is called a *periodic boundary condition*.

The boundary condition at the right end $x = L$ is an open boundary condition (see Exercise 2.12) to let a right-going wave out of the domain. At the left end, $x = 0$, we apply, in the beginning of the simulation, either a symmetry boundary condition (see Exercise 2.8) $u_x = 0$, or an open boundary condition.

This initial wave will split in two and either be reflected or transported out of the domain at $x = 0$. The purpose of the exercise is to follow the right-going wave. We can do that with a *periodic boundary condition*. This means that when the right-going wave hits the boundary $x = L$, the

open boundary condition lets the wave out of the domain, but at the same time we use a boundary condition on the left end $x = 0$ that feeds the outgoing wave into the domain again. This periodic condition is simply $u(0) = u(L)$. The switch from $u_x = 0$ or an open boundary condition at the left end to a periodic condition can happen when $u(L, t) > \epsilon$, where $\epsilon = 10^{-4}$ might be an appropriate value for determining when the right-going wave hits the boundary $x = L$.

The open boundary conditions can conveniently be discretized as explained in Exercise 2.12. Implement the described type of boundary conditions and test them on two different initial shapes: a plug $u(x, 0) = 1$ for $x \leq 0.1$, $u(x, 0) = 0$ for $x > 0.1$, and a Gaussian function in the middle of the domain: $u(x, 0) = \exp(-\frac{1}{2}(x - 0.5)^2/0.05)$. The domain is the unit interval $[0, 1]$. Run these two shapes for Courant numbers 1 and 0.5. Assume constant wave velocity. Make movies of the four cases. Reason why the solutions are correct. Filename: `periodic`.

Exercise 2.14: Compare discretizations of a Neumann condition

We have a 1D wave equation with variable wave velocity: $u_{tt} = (qu_x)_x$. A Neumann condition u_x at $x = 0, L$ can be discretized as shown in (2.54) and (2.57).

The aim of this exercise is to examine the rate of the numerical error when using different ways of discretizing the Neumann condition.

a) As a test problem, $q = 1+(x-L/2)^4$ can be used, with $f(x, t)$ adapted such that the solution has a simple form, say $u(x, t) = \cos(\pi x/L)\cos(\omega t)$ for, e.g., $\omega = 1$. Perform numerical experiments and find the convergence rate of the error using the approximation (2.54).

b) Switch to $q(x) = 1 + \cos(\pi x/L)$, which is symmetric at $x = 0, L$, and check the convergence rate of the scheme (2.57). Now, $q_{i-1/2}$ is a 2nd-order approximation to q_i, $q_{i-1/2} = q_i + 0.25q_i''\Delta x^2 + \cdots$, because $q_i' = 0$ for $i = N_x$ (a similar argument can be applied to the case $i = 0$).

c) A third discretization can be based on a simple and convenient, but less accurate, one-sided difference: $u_i - u_{i-1} = 0$ at $i = N_x$ and $u_{i+1} - u_i = 0$ at $i = 0$. Derive the resulting scheme in detail and implement it. Run experiments with q from a) or b) to establish the rate of convergence of the scheme.

d) A fourth technique is to view the scheme as

$$[D_tD_tu]_i^n = \frac{1}{\Delta x}\left([qD_xu]_{i+\frac{1}{2}}^n - [qD_xu]_{i-\frac{1}{2}}^n\right) + [f]_i^n,$$

and place the boundary at $x_{i+\frac{1}{2}}$, $i = N_x$, instead of exactly at the physical boundary. With this idea of approximating (moving) the boundary, we can just set $[qD_xu]_{i+\frac{1}{2}}^n = 0$. Derive the complete scheme using this technique. The implementation of the boundary condition at $L - \Delta x/2$ is $\mathcal{O}(\Delta x^2)$ accurate, but the interesting question is what impact the movement of the boundary has on the convergence rate. Compute the errors as usual over the entire mesh and use q from a) or b).
Filename: `Neumann_discr`.

Exercise 2.15: Verification by a cubic polynomial in space

The purpose of this exercise is to verify the implementation of the `solver` function in the program `wave1D_n0.py` by using an exact numerical solution for the wave equation $u_{tt} = c^2u_{xx} + f$ with Neumann boundary conditions $u_x(0,t) = u_x(L,t) = 0$.

A similar verification is used in the file `wave1D_u0.py`, which solves the same PDE, but with Dirichlet boundary conditions $u(0,t) = u(L,t) = 0$. The idea of the verification test in function `test_quadratic` in `wave1D_u0.py` is to produce a solution that is a lower-order polynomial such that both the PDE problem, the boundary conditions, and all the discrete equations are exactly fulfilled. Then the `solver` function should reproduce this exact solution to machine precision. More precisely, we seek $u = X(x)T(t)$, with $T(t)$ as a linear function and $X(x)$ as a parabola that fulfills the boundary conditions. Inserting this u in the PDE determines f. It turns out that u also fulfills the discrete equations, because the truncation error of the discretized PDE has derivatives in x and t of order four and higher. These derivatives all vanish for a quadratic $X(x)$ and linear $T(t)$.

It would be attractive to use a similar approach in the case of Neumann conditions. We set $u = X(x)T(t)$ and seek lower-order polynomials X and T. To force u_x to vanish at the boundary, we let X_x be a parabola. Then X is a cubic polynomial. The fourth-order derivative of a cubic polynomial vanishes, so $u = X(x)T(t)$ will fulfill the discretized PDE also in this case, if f is adjusted such that u fulfills the PDE.

However, the discrete boundary condition is not exactly fulfilled by this choice of u. The reason is that

$$[D_{2x}u]_i^n = u_x(x_i, t_n) + \frac{1}{6}u_{xxx}(x_i, t_n)\Delta x^2 + \mathcal{O}(\Delta x^4)\,. \tag{2.74}$$

At the two boundary points, we must demand that the derivative $X_x(x) = 0$ such that $u_x = 0$. However, u_{xxx} is a constant and not zero when $X(x)$ is a cubic polynomial. Therefore, our $u = X(x)T(t)$ fulfills

$$[D_{2x}u]_i^n = \frac{1}{6}u_{xxx}(x_i, t_n)\Delta x^2,$$

and not

$$[D_{2x}u]_i^n = 0, \quad i = 0, N_x,$$

as it should. (Note that all the higher-order terms $\mathcal{O}(\Delta x^4)$ also have higher-order derivatives that vanish for a cubic polynomial.) So to summarize, the fundamental problem is that u as a product of a cubic polynomial and a linear or quadratic polynomial in time is not an exact solution of the discrete boundary conditions.

To make progress, we assume that $u = X(x)T(t)$, where T for simplicity is taken as a prescribed linear function $1 + \frac{1}{2}t$, and $X(x)$ is taken as an *unknown* cubic polynomial $\sum_{j=0}^{3} a_j x^j$. There are two different ways of determining the coefficients $a_0, \ldots, a_3$ such that both the discretized PDE and the discretized boundary conditions are fulfilled, under the constraint that we can specify a function $f(x, t)$ for the PDE to feed to the `solver` function in `wave1D_n0.py`. Both approaches are explained in the subexercises.

a) One can insert u in the discretized PDE and find the corresponding f. Then one can insert u in the discretized boundary conditions. This yields two equations for the four coefficients $a_0, \ldots, a_3$. To find the coefficients, one can set $a_0 = 0$ and $a_1 = 1$ for simplicity and then determine a_2 and a_3. This approach will make a_2 and a_3 depend on Δx and f will depend on both Δx and Δt.

Use `sympy` to perform analytical computations. A starting point is to define u as follows:

```
def test_cubic1():
    import sympy as sm
    x, t, c, L, dx, dt = sm.symbols('x t c L dx dt')
    i, n = sm.symbols('i n', integer=True)
```

```
# Assume discrete solution is a polynomial of degree 3 in x
T = lambda t: 1 + sm.Rational(1,2)*t  # Temporal term
a = sm.symbols('a_0 a_1 a_2 a_3')
X = lambda x: sum(a[q]*x**q for q in range(4))  # Spatial term
u = lambda x, t: X(x)*T(t)
```

The symbolic expression for u is reached by calling `u(x,t)` with `x` and `t` as `sympy` symbols.

Define `DxDx(u, i, n)`, `DtDt(u, i, n)`, and `D2x(u, i, n)` as Python functions for returning the difference approximations $[D_xD_xu]_i^n$, $[D_tD_tu]_i^n$, and $[D_{2x}u]_i^n$. The next step is to set up the residuals for the equations $[D_{2x}u]_0^n = 0$ and $[D_{2x}u]_{N_x}^n = 0$, where $N_x = L/\Delta x$. Call the residuals `R_0` and `R_L`. Substitute a_0 and a_1 by 0 and 1, respectively, in `R_0`, `R_L`, and `a`:

```
R_0 = R_0.subs(a[0], 0).subs(a[1], 1)
R_L = R_L.subs(a[0], 0).subs(a[1], 1)
a = list(a)  # enable in-place assignment
a[0:2] = 0, 1
```

Determining a_2 and a_3 from the discretized boundary conditions is then about solving two equations with respect to a_2 and a_3, i.e., `a[2:]`:

```
s = sm.solve([R_0, R_L], a[2:])
# s is dictionary with the unknowns a[2] and a[3] as keys
a[2:] = s[a[2]], s[a[3]]
```

Now, `a` contains computed values and `u` will automatically use these new values since `X` accesses `a`.

Compute the source term f from the discretized PDE: $f_i^n = [D_tD_tu - c^2D_xD_xu]_i^n$. Turn u, the time derivative u_t (needed for the initial condition $V(x)$), and f into Python functions. Set numerical values for L, N_x, C, and c. Prescribe the time interval as $\Delta t = CL/(N_xc)$, which imply $\Delta x = c\Delta t/C = L/N_x$. Define new functions `I(x)`, `V(x)`, and `f(x,t)` as wrappers of the ones made above, where fixed values of L, c, Δx, and Δt are inserted, such that `I`, `V`, and `f` can be passed on to the `solver` function. Finally, call `solver` with a `user_action` function that compares the numerical solution to this exact solution u of the discrete PDE problem.

Hint. To turn a `sympy` expression `e`, depending on a series of symbols, say `x`, `t`, `dx`, `dt`, `L`, and `c`, into a plain Python function `e_exact(x,t,L,dx,dt,c)`, one can write

```
e_exact = sm.lambdify([x,t,L,dx,dt,c], e, 'numpy')
```

The `'numpy'` argument is a good habit as the `e_exact` function will then work with array arguments if it contains mathematical functions (but here we only do plain arithmetics, which automatically work with arrays).

b) An alternative way of determining $a_0, \ldots, a_3$ is to reason as follows. We first construct $X(x)$ such that the boundary conditions are fulfilled: $X = x(L-x)$. However, to compensate for the fact that this choice of X does not fulfill the discrete boundary condition, we seek u such that

$$u_x = \frac{\partial}{\partial x} x(L-x)T(t) - \frac{1}{6}u_{xxx}\Delta x^2,$$

since this u will fit the discrete boundary condition. Assuming $u = T(t)\sum_{j=0}^{3} a_j x^j$, we can use the above equation to determine the coefficients a_1, a_2, a_3. A value, e.g., 1 can be used for a_0. The following `sympy` code computes this u:

```
def test_cubic2():
    import sympy as sm
    x, t, c, L, dx = sm.symbols('x t c L dx')
    T = lambda t: 1 + sm.Rational(1,2)*t  # Temporal term
    # Set u as a 3rd-degree polynomial in space
    X = lambda x: sum(a[i]*x**i for i in range(4))
    a = sm.symbols('a_0 a_1 a_2 a_3')
    u = lambda x, t: X(x)*T(t)
    # Force discrete boundary condition to be zero by adding
    # a correction term the analytical suggestion x*(L-x)*T
    # u_x = x*(L-x)*T(t) - 1/6*u_xxx*dx**2
    R = sm.diff(u(x,t), x) - (
        x*(L-x) - sm.Rational(1,6)*sm.diff(u(x,t), x, x, x)*dx**2)
    # R is a polynomial: force all coefficients to vanish.
    # Turn R to Poly to extract coefficients:
    R = sm.poly(R, x)
    coeff = R.all_coeffs()
    s = sm.solve(coeff, a[1:])  # a[0] is not present in R
    # s is dictionary with a[i] as keys
    # Fix a[0] as 1
    s[a[0]] = 1
    X = lambda x: sm.simplify(sum(s[a[i]]*x**i for i in range(4)))
    u = lambda x, t: X(x)*T(t)
    print 'u:', u(x,t)
```

The next step is to find the source term `f_e` by inserting `u_e` in the PDE. Thereafter, turn `u`, `f`, and the time derivative of `u` into plain Python functions as in a), and then wrap these functions in new functions `I`, `V`, and `f`, with the right signature as required by the `solver` function. Set parameters as in a) and check that the solution is exact to machine precision at each time level using an appropriate `user_action` function.

Filename: `wave1D_n0_test_cubic`.

2.10 Analysis of the difference equations

2.10.1 Properties of the solution of the wave equation

The wave equation

$$\frac{\partial^2 u}{\partial t^2} = c^2 \frac{\partial^2 u}{\partial x^2}$$

has solutions of the form

$$u(x,t) = g_R(x - ct) + g_L(x + ct), \tag{2.75}$$

for any functions g_R and g_L sufficiently smooth to be differentiated twice. The result follows from inserting (2.75) in the wave equation. A function of the form $g_R(x - ct)$ represents a signal moving to the right in time with constant velocity c. This feature can be explained as follows. At time $t = 0$ the signal looks like $g_R(x)$. Introducing a moving horizontal coordinate $\xi = x - ct$, we see the function $g_R(\xi)$ is "at rest" in the ξ coordinate system, and the shape is always the same. Say the $g_R(\xi)$ function has a peak at $\xi = 0$. This peak is located at $x = ct$, which means that it moves with the velocity $dx/dt = c$ in the x coordinate system. Similarly, $g_L(x + ct)$ is a function, initially with shape $g_L(x)$, that moves in the negative x direction with constant velocity c (introduce $\xi = x + ct$, look at the point $\xi = 0$, $x = -ct$, which has velocity $dx/dt = -c$).

With the particular initial conditions

$$u(x,0) = I(x), \quad \frac{\partial}{\partial t} u(x,0) = 0,$$

we get, with u as in (2.75),

$$g_R(x) + g_L(x) = I(x), \quad -cg_R'(x) + cg_L'(x) = 0\,.$$

The former suggests $g_R = g_L$, and the former then leads to $g_R = g_L = I/2$. Consequently,

$$u(x,t) = \frac{1}{2} I(x - ct) + \frac{1}{2} I(x + ct)\,. \tag{2.76}$$

The interpretation of (2.76) is that the initial shape of u is split into two parts, each with the same shape as I but half of the initial amplitude. One part is traveling to the left and the other one to the right.

The solution has two important physical features: constant amplitude of the left and right wave, and constant velocity of these two waves. It turns out that the numerical solution will also preserve the constant amplitude, but the velocity depends on the mesh parameters Δt and Δx.

The solution (2.76) will be influenced by boundary conditions when the parts $\frac{1}{2}I(x-ct)$ and $\frac{1}{2}I(x+ct)$ hit the boundaries and get, e.g., reflected back into the domain. However, when $I(x)$ is nonzero only in a small part in the middle of the spatial domain $[0, L]$, which means that the boundaries are placed far away from the initial disturbance of u, the solution (2.76) is very clearly observed in a simulation.

A useful representation of solutions of wave equations is a linear combination of sine and/or cosine waves. Such a sum of waves is a solution if the governing PDE is linear and each sine or cosine wave fulfills the equation. To ease analytical calculations by hand we shall work with complex exponential functions instead of real-valued sine or cosine functions. The real part of complex expressions will typically be taken as the physical relevant quantity (whenever a physical relevant quantity is strictly needed). The idea now is to build $I(x)$ of complex wave components e^{ikx}:

$$I(x) \approx \sum_{k \in K} b_k e^{ikx}\,. \tag{2.77}$$

Here, k is the frequency of a component, K is some set of all the discrete k values needed to approximate $I(x)$ well, and b_k are constants that must be determined. We will very seldom need to compute the b_k coefficients: most of the insight we look for, and the understanding of the numerical methods we want to establish, come from investigating how the PDE and the scheme treat a single component e^{ikx} wave.

Letting the number of k values in K tend to infinity, makes the sum (2.77) converge to $I(x)$. This sum is known as a *Fourier series* representation of $I(x)$. Looking at (2.76), we see that the solution $u(x,t)$, when $I(x)$ is represented as in (2.77), is also built of basic complex exponential wave components of the form $e^{ik(x\pm ct)}$ according to

$$u(x,t) = \frac{1}{2}\sum_{k \in K} b_k e^{ik(x-ct)} + \frac{1}{2}\sum_{k \in K} b_k e^{ik(x+ct)}\,. \tag{2.78}$$

It is common to introduce the frequency in time $\omega = kc$ and assume that $u(x,t)$ is a sum of basic wave components written as $e^{ikx-\omega t}$. (Observe that inserting such a wave component in the governing PDE reveals that $\omega^2 = k^2c^2$, or $\omega = \pm kc$, reflecting the two solutions: one $(+kc)$ traveling to the right and the other $(-kc)$ traveling to the left.)

2.10.2 More precise definition of Fourier representations

The above introduction to function representation by sine and cosine waves was quick and intuitive, but will suffice as background knowledge for the following material of single wave component analysis. However, to understand all details of how different wave components sum up to the analytical and numerical solutions, a more precise mathematical treatment is helpful and therefore summarized below.

It is well known that periodic functions can be represented by Fourier series. A generalization of the Fourier series idea to non-periodic functions defined on the real line is the *Fourier transform*:

$$I(x) = \int_{-\infty}^{\infty} A(k)e^{ikx}dk, \tag{2.79}$$

$$A(k) = \int_{-\infty}^{\infty} I(x)e^{-ikx}dx\,. \tag{2.80}$$

The function $A(k)$ reflects the weight of each wave component e^{ikx} in an infinite sum of such wave components. That is, $A(k)$ reflects the frequency content in the function $I(x)$. Fourier transforms are particularly fundamental for analyzing and understanding time-varying signals.

The solution of the linear 1D wave PDE can be expressed as

$$u(x,t) = \int_{-\infty}^{\infty} A(k)e^{i(kx-\omega(k)t)}dx\,.$$

In a finite difference method, we represent u by a mesh function u_q^n, where n counts temporal mesh points and q counts the spatial ones (the usual counter for spatial points, i, is here already used as imaginary unit). Similarly, $I(x)$ is approximated by the mesh function I_q, $q = 0, \ldots, N_x$. On a mesh, it does not make sense to work with wave components e^{ikx} for very large k, because the shortest possible sine or cosine wave that can be represented uniquely on a mesh with spacing Δx is the wave with wavelength $2\Delta x$. This wave has its peaks and throughs at every two

mesh points. That is, the wave "jumps up and down" between the mesh points.

The corresponding k value for the shortest possible wave in the mesh is $k = 2\pi/(2\Delta x) = \pi/\Delta x$. This maximum frequency is known as the *Nyquist frequency*. Within the range of relevant frequencies $(0, \pi/\Delta x]$ one defines the discrete Fourier transform[9], using $N_x + 1$ discrete frequencies:

$$I_q = \frac{1}{N_x + 1} \sum_{k=0}^{N_x} A_k e^{i2\pi kq/(N_x+1)}, \quad q = 0, \ldots, N_x, \tag{2.81}$$

$$A_k = \sum_{q=0}^{N_x} I_q e^{-i2\pi kq/(N_x+1)}, \quad k = 0, \ldots, N_x\,. \tag{2.82}$$

The A_k values represent the discrete Fourier transform of the I_q values, which themselves are the inverse discrete Fourier transform of the A_k values.

The discrete Fourier transform is efficiently computed by the *Fast Fourier transform* algorithm. For a real function $I(x)$, the relevant Python code for computing and plotting the discrete Fourier transform appears in the example below.

```
import numpy as np
from numpy import sin, pi

def I(x):
    return sin(2*pi*x) + 0.5*sin(4*pi*x) + 0.1*sin(6*pi*x)

# Mesh
L = 10; Nx = 100
x = np.linspace(0, L, Nx+1)
dx = L/float(Nx)

# Discrete Fourier transform
A = np.fft.rfft(I(x))
A_amplitude = np.abs(A)

# Compute the corresponding frequencies
freqs = np.linspace(0, pi/dx, A_amplitude.size)

import matplotlib.pyplot as plt
plt.plot(freqs, A_amplitude)
plt.show()
```

[9] http://en.wikipedia.org/wiki/Discrete_Fourier_transform

2.10.3 Stability

The scheme

$$[D_t D_t u = c^2 D_x D_x u]_q^n \tag{2.83}$$

for the wave equation $u_{tt} = c^2 u_{xx}$ allows basic wave components

$$u_q^n = e^{i(kx_q - \tilde{\omega} t_n)}$$

as solution, but it turns out that the frequency in time, $\tilde{\omega}$, is not equal to the exact frequency $\omega = kc$. The goal now is to find exactly what $\tilde{\omega}$ is. We ask two key questions:

- How accurate is $\tilde{\omega}$ compared to ω?
- Does the amplitude of such a wave component preserve its (unit) amplitude, as it should, or does it get amplified or damped in time (because of a complex $\tilde{\omega}$)?

The following analysis will answer these questions. We shall continue using q as an identifier for a certain mesh point in the x direction.

Preliminary results. A key result needed in the investigations is the finite difference approximation of a second-order derivative acting on a complex wave component:

$$[D_t D_t e^{i\omega t}]^n = -\frac{4}{\Delta t^2} \sin^2\left(\frac{\omega \Delta t}{2}\right) e^{i\omega n \Delta t}.$$

By just changing symbols ($\omega \to k$, $t \to x$, $n \to q$) it follows that

$$[D_x D_x e^{ikx}]_q = -\frac{4}{\Delta x^2} \sin^2\left(\frac{k\Delta x}{2}\right) e^{ikq\Delta x}.$$

Numerical wave propagation. Inserting a basic wave component $u_q^n = e^{i(kx_q - \tilde{\omega} t_n)}$ in (2.83) results in the need to evaluate two expressions:

$$\begin{aligned}
[D_t D_t e^{ikx} e^{-i\tilde{\omega}t}]_q^n &= [D_t D_t e^{-i\tilde{\omega}t}]^n e^{ikq\Delta x} \\
&= -\frac{4}{\Delta t^2} \sin^2\left(\frac{\tilde{\omega}\Delta t}{2}\right) e^{-i\tilde{\omega}n\Delta t} e^{ikq\Delta x} \qquad (2.84) \\
[D_x D_x e^{ikx} e^{-i\tilde{\omega}t}]_q^n &= [D_x D_x e^{ikx}]_q e^{-i\tilde{\omega}n\Delta t} \\
&= -\frac{4}{\Delta x^2} \sin^2\left(\frac{k\Delta x}{2}\right) e^{ikq\Delta x} e^{-i\tilde{\omega}n\Delta t}. \qquad (2.85)
\end{aligned}$$

Then the complete scheme,

$$[D_t D_t e^{ikx} e^{-i\tilde{\omega}t} = c^2 D_x D_x e^{ikx} e^{-i\tilde{\omega}t}]^n_q$$

leads to the following equation for the unknown numerical frequency $\tilde{\omega}$ (after dividing by $-e^{ikx}e^{-i\tilde{\omega}t}$):

$$\frac{4}{\Delta t^2}\sin^2\left(\frac{\tilde{\omega}\Delta t}{2}\right) = c^2\frac{4}{\Delta x^2}\sin^2\left(\frac{k\Delta x}{2}\right),$$

or

$$\sin^2\left(\frac{\tilde{\omega}\Delta t}{2}\right) = C^2\sin^2\left(\frac{k\Delta x}{2}\right), \tag{2.86}$$

where

$$C = \frac{c\Delta t}{\Delta x} \tag{2.87}$$

is the Courant number. Taking the square root of (2.86) yields

$$\sin\left(\frac{\tilde{\omega}\Delta t}{2}\right) = C\sin\left(\frac{k\Delta x}{2}\right), \tag{2.88}$$

Since the exact ω is real it is reasonable to look for a real solution $\tilde{\omega}$ of (2.88). The right-hand side of (2.88) must then be in $[-1,1]$ because the sine function on the left-hand side has values in $[-1,1]$ for real $\tilde{\omega}$. The magnitude of the sine function on the right-hand side attains the value 1 when

$$\frac{k\Delta x}{2} = \frac{\pi}{2} + m\pi, \quad m \in \mathbb{Z}.$$

With $m = 0$ we have $k\Delta x = \pi$, which means that the wavelength $\lambda = 2\pi/k$ becomes $2\Delta x$. This is the absolutely shortest wavelength that can be represented on the mesh: the wave jumps up and down between each mesh point. Larger values of $|m|$ are irrelevant since these correspond to k values whose waves are too short to be represented on a mesh with spacing Δx. For the shortest possible wave in the mesh, $\sin(k\Delta x/2) = 1$, and we must require

$$C \leq 1. \tag{2.89}$$

Consider a right-hand side in (2.88) of magnitude larger than unity. The solution $\tilde{\omega}$ of (2.88) must then be a complex number $\tilde{\omega} = \tilde{\omega}_r + i\tilde{\omega}_i$ because the sine function is larger than unity for a complex argument.

One can show that for any ω_i there will also be a corresponding solution with $-\omega_i$. The component with $\omega_i > 0$ gives an amplification factor $e^{\omega_i t}$ that grows exponentially in time. We cannot allow this and must therefore require $C \leq 1$ as a *stability criterion*.

Remark on the stability requirement

For smoother wave components with longer wave lengths per length Δx, (2.89) can in theory be relaxed. However, small round-off errors are always present in a numerical solution and these vary arbitrarily from mesh point to mesh point and can be viewed as unavoidable noise with wavelength $2\Delta x$. As explained, $C > 1$ will for this very small noise lead to exponential growth of the shortest possible wave component in the mesh. This noise will therefore grow with time and destroy the whole solution.

2.10.4 Numerical dispersion relation

Equation (2.88) can be solved with respect to $\tilde{\omega}$:

$$\tilde{\omega} = \frac{2}{\Delta t}\sin^{-1}\left(C\sin\left(\frac{k\Delta x}{2}\right)\right). \tag{2.90}$$

The relation between the numerical frequency $\tilde{\omega}$ and the other parameters k, c, Δx, and Δt is called a *numerical dispersion relation*. Correspondingly, $\omega = kc$ is the *analytical dispersion relation*. In general, dispersion refers to the phenomenon where the wave velocity depends on the spatial frequency (k, or the wave length $\lambda = 2\pi/k$) of the wave. Since the wave velocity is $\omega/k = c$, we realize that the analytical dispersion relation reflects the fact that there is no dispersion. However, in a numerical scheme we have dispersive waves where the wave velocity depends on k.

The special case $C = 1$ deserves attention since then the right-hand side of (2.90) reduces to

$$\frac{2}{\Delta t}\frac{k\Delta x}{2} = \frac{1}{\Delta t}\frac{\omega\Delta x}{c} = \frac{\omega}{C} = \omega\,.$$

That is, $\tilde{\omega} = \omega$ and the numerical solution is exact at all mesh points regardless of Δx and Δt! This implies that the numerical solution method is also an analytical solution method, at least for computing u at discrete

points (the numerical method says nothing about the variation of u *between* the mesh points, and employing the common linear interpolation for extending the discrete solution gives a curve that in general deviates from the exact one).

For a closer examination of the error in the numerical dispersion relation when $C < 1$, we can study $\tilde{\omega} - \omega$, $\tilde{\omega}/\omega$, or the similar error measures in wave velocity: $\tilde{c} - c$ and $\tilde{c}/c$, where $c = \omega/k$ and $\tilde{c} = \tilde{\omega}/k$. It appears that the most convenient expression to work with is $\tilde{c}/c$, since it can be written as a function of just two parameters:

$$\frac{\tilde{c}}{c} = \frac{1}{Cp}\sin^{-1}\left(C\sin p\right),$$

with $p = k\Delta x/2$ as a non-dimensional measure of the spatial frequency. In essence, p tells how many spatial mesh points we have per wave length in space for the wave component with frequency k (recall that the wave length is $2\pi/k$). That is, p reflects how well the spatial variation of the wave component is resolved in the mesh. Wave components with wave length less than $2\Delta x$ ($2\pi/k < 2\Delta x$) are not visible in the mesh, so it does not make sense to have $p > \pi/2$.

We may introduce the function $r(C, p) = \tilde{c}/c$ for further investigation of numerical errors in the wave velocity:

$$r(C,p) = \frac{1}{Cp}\sin^{-1}\left(C\sin p\right), \quad C \in (0,1],\ p \in (0,\pi/2]\,. \tag{2.91}$$

This function is very well suited for plotting since it combines several parameters in the problem into a dependence on two dimensionless numbers, C and p.

Defining

```
def r(C, p):
    return 2/(C*p)*asin(C*sin(p))
```

we can plot $r(C, p)$ as a function of p for various values of C, see Figure 2.6. Note that the shortest waves have the most erroneous velocity, and that short waves move more slowly than they should.

We can also easily make a Taylor series expansion in the discretization parameter p:

```
>>> import sympy as sym
>>> C, p = sym.symbols('C p')
>>> # Compute the 7 first terms around p=0 with no O() term
>>> rs = r(C, p).series(p, 0, 7).removeO()
```

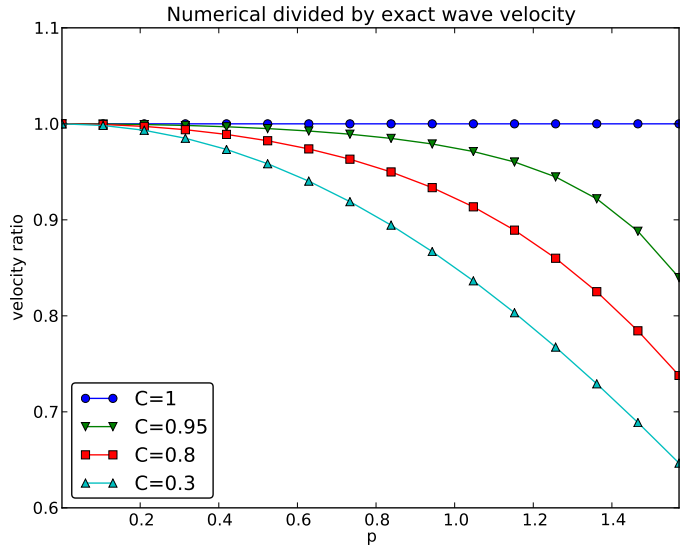


Fig. 2.6 The fractional error in the wave velocity for different Courant numbers.

```
>>> rs
p**6*(5*C**6/112 - C**4/16 + 13*C**2/720 - 1/5040) +
p**4*(3*C**4/40 - C**2/12 + 1/120) +
p**2*(C**2/6 - 1/6) + 1

>>> # Pick out the leading order term, but drop the constant 1
>>> rs_error_leading_order = (rs - 1).extract_leading_order(p)
>>> rs_error_leading_order
p**2*(C**2/6 - 1/6)

>>> # Turn the series expansion into a Python function
>>> rs_pyfunc = lambdify([C, p], rs, modules='numpy')

>>> # Check: rs_pyfunc is exact (=1) for C=1
>>> rs_pyfunc(1, 0.1)
1.0
```

Note that without the `.removeO()` call the series gets an `O(x**7)` term that makes it impossible to convert the series to a Python function (for, e.g., plotting).

From the `rs_error_leading_order` expression above, we see that the leading order term in the error of this series expansion is

$$\frac{1}{6}\left(\frac{k\Delta x}{2}\right)^2 (C^2-1) = \frac{k^2}{24}\left(c^2\Delta t^2 - \Delta x^2\right), \tag{2.92}$$

pointing to an error $\mathcal{O}(\Delta t^2, \Delta x^2)$, which is compatible with the errors in the difference approximations ($D_t D_t u$ and $D_x D_x u$).

We can do more with a series expansion, e.g., factor it to see how the factor $C-1$ plays a significant role. To this end, we make a list of the terms, factor each term, and then sum the terms:

```
>>> rs = r(C, p).series(p, 0, 4).removeO().as_ordered_terms()
>>> rs
[1, C**2*p**2/6 - p**2/6,
 3*C**4*p**4/40 - C**2*p**4/12 + p**4/120,
 5*C**6*p**6/112 - C**4*p**6/16 + 13*C**2*p**6/720 - p**6/5040]
>>> rs = [factor(t) for t in rs]
>>> rs
[1, p**2*(C - 1)*(C + 1)/6,
 p**4*(C - 1)*(C + 1)*(3*C - 1)*(3*C + 1)/120,
 p**6*(C - 1)*(C + 1)*(225*C**4 - 90*C**2 + 1)/5040]
>>> rs = sum(rs)  # Python's sum function sums the list
>>> rs
p**6*(C - 1)*(C + 1)*(225*C**4 - 90*C**2 + 1)/5040 +
p**4*(C - 1)*(C + 1)*(3*C - 1)*(3*C + 1)/120 +
p**2*(C - 1)*(C + 1)/6 + 1
```

We see from the last expression that $C = 1$ makes all the terms in `rs` vanish. Since we already know that the numerical solution is exact for $C = 1$, the remaining terms in the Taylor series expansion will also contain factors of $C - 1$ and cancel for $C = 1$.

2.10.5 Extending the analysis to 2D and 3D

The typical analytical solution of a 2D wave equation

$$u_{tt} = c^2(u_{xx} + u_{yy}),$$

is a wave traveling in the direction of $\boldsymbol{k} = k_x\boldsymbol{i} + k_y\boldsymbol{j}$, where $\boldsymbol{i}$ and $\boldsymbol{j}$ are unit vectors in the x and y directions, respectively ($\boldsymbol{i}$ should not be confused with $i = \sqrt{-1}$ here). Such a wave can be expressed by

$$u(x, y, t) = g(k_x x + k_y y - kct)$$

for some twice differentiable function g, or with $\omega = kc$, $k = |\boldsymbol{k}|$:

$$u(x, y, t) = g(k_x x + k_y y - \omega t)\,.$$

We can, in particular, build a solution by adding complex Fourier components of the form

$$e^{(i(k_x x + k_y y - \omega t))}\,.$$

A discrete 2D wave equation can be written as

$$[D_t D_t u = c^2 (D_x D_x u + D_y D_y u)]^n_{q,r}\,. \tag{2.93}$$

This equation admits a Fourier component

$$u^n_{q,r} = e^{(i(k_x q \Delta x + k_y r \Delta y - \tilde{\omega} n \Delta t))}, \tag{2.94}$$

as solution. Letting the operators $D_t D_t$, $D_x D_x$, and $D_y D_y$ act on $u^n_{q,r}$ from (2.94) transforms (2.93) to

$$\frac{4}{\Delta t^2} \sin^2\left(\frac{\tilde{\omega}\Delta t}{2}\right) = c^2 \frac{4}{\Delta x^2} \sin^2\left(\frac{k_x \Delta x}{2}\right) + c^2 \frac{4}{\Delta y^2} \sin^2\left(\frac{k_y \Delta y}{2}\right)\,. \tag{2.95}$$

or

$$\sin^2\left(\frac{\tilde{\omega}\Delta t}{2}\right) = C_x^2 \sin^2 p_x + C_y^2 \sin^2 p_y, \tag{2.96}$$

where we have eliminated the factor 4 and introduced the symbols

$$C_x = \frac{c\Delta t}{\Delta x}, \quad C_y = \frac{c\Delta t}{\Delta y}, \quad p_x = \frac{k_x \Delta x}{2}, \quad p_y = \frac{k_y \Delta y}{2}\,.$$

For a real-valued $\tilde{\omega}$ the right-hand side must be less than or equal to unity in absolute value, requiring in general that

$$C_x^2 + C_y^2 \leq 1\,. \tag{2.97}$$

This gives the stability criterion, more commonly expressed directly in an inequality for the time step:

$$\Delta t \leq \frac{1}{c} \left(\frac{1}{\Delta x^2} + \frac{1}{\Delta y^2}\right)^{-1/2} \tag{2.98}$$

A similar, straightforward analysis for the 3D case leads to

$$\Delta t \leq \frac{1}{c} \left(\frac{1}{\Delta x^2} + \frac{1}{\Delta y^2} + \frac{1}{\Delta z^2}\right)^{-1/2} \tag{2.99}$$

In the case of a variable coefficient $c^2 = c^2(\boldsymbol{x})$, we must use the worst-case value

$$\bar{c} = \sqrt{\max_{\boldsymbol{x} \in \Omega} c^2(\boldsymbol{x})} \tag{2.100}$$

in the stability criteria. Often, especially in the variable wave velocity case, it is wise to introduce a safety factor $\beta \in (0,1]$ too:

$$\Delta t \leq \beta \frac{1}{\bar{c}} \left(\frac{1}{\Delta x^2} + \frac{1}{\Delta y^2} + \frac{1}{\Delta z^2} \right)^{-1/2} \tag{2.101}$$

The exact numerical dispersion relations in 2D and 3D becomes, for constant c,

$$\tilde{\omega} = \frac{2}{\Delta t} \sin^{-1} \left(\left(C_x^2 \sin^2 p_x + C_y^2 \sin^2 p_y \right)^{\frac{1}{2}} \right), \tag{2.102}$$

$$\tilde{\omega} = \frac{2}{\Delta t} \sin^{-1} \left(\left(C_x^2 \sin^2 p_x + C_y^2 \sin^2 p_y + C_z^2 \sin^2 p_z \right)^{\frac{1}{2}} \right). \tag{2.103}$$

We can visualize the numerical dispersion error in 2D much like we did in 1D. To this end, we need to reduce the number of parameters in $\tilde{\omega}$. The direction of the wave is parameterized by the polar angle θ, which means that

$$k_x = k \sin\theta, \quad k_y = k \cos\theta \,.$$

A simplification is to set $\Delta x = \Delta y = h$. Then $C_x = C_y = c\Delta t/h$, which we call C. Also,

$$p_x = \frac{1}{2} kh \cos\theta, \quad p_y = \frac{1}{2} kh \sin\theta \,.$$

The numerical frequency $\tilde{\omega}$ is now a function of three parameters:

- C, reflecting the number of cells a wave is displaced during a time step,
- $p = \frac{1}{2}kh$, reflecting the number of cells per wave length in space,
- θ, expressing the direction of the wave.

We want to visualize the error in the numerical frequency. To avoid having Δt as a free parameter in $\tilde{\omega}$, we work with $\tilde{c}/c = \tilde{\omega}/(kc)$. The coefficient in front of the $\sin^{-1}$ factor is then

$$\frac{2}{kc\Delta t} = \frac{2}{2kc\Delta t h/h} = \frac{1}{Ckh} = \frac{2}{Cp},$$

and

$$\frac{\tilde{c}}{c} = \frac{2}{Cp} \sin^{-1} \left(C \left(\sin^2(p\cos\theta) + \sin^2(p\sin\theta) \right)^{\frac{1}{2}} \right).$$

We want to visualize this quantity as a function of p and θ for some values of $C \leq 1$. It is instructive to make color contour plots of $1 - \tilde{c}/c$ in *polar coordinates* with θ as the angular coordinate and p as the radial coordinate.

The stability criterion (2.97) becomes $C \leq C_{\max} = 1/\sqrt{2}$ in the present 2D case with the C defined above. Let us plot $1 - \tilde{c}/c$ in polar coordinates for $C_{\max}, 0.9C_{\max}, 0.5C_{\max}, 0.2C_{\max}$. The program below does the somewhat tricky work in Matplotlib, and the result appears in Figure 2.7. From the figure we clearly see that the maximum C value gives the best results, and that waves whose propagation direction makes an angle of 45 degrees with an axis are the most accurate.

```
def dispersion_relation_2D(p, theta, C):
    arg = C*sqrt(sin(p*cos(theta))**2 +
                 sin(p*sin(theta))**2)
    c_frac = 2./(C*p)*arcsin(arg)

    return c_frac

import numpy as np
from numpy import \
     cos, sin, arcsin, sqrt, pi  # for nicer math formulas

r = p = np.linspace(0.001, pi/2, 101)
theta = np.linspace(0, 2*pi, 51)
r, theta = np.meshgrid(r, theta)

# Make 2x2 filled contour plots for 4 values of C
import matplotlib.pyplot as plt
C_max = 1/sqrt(2)
C = [[C_max, 0.9*C_max], [0.5*C_max, 0.2*C_max]]
fix, axes = plt.subplots(2, 2, subplot_kw=dict(polar=True))
for row in range(2):
    for column in range(2):
        error = 1 - dispersion_relation_2D(
            p, theta, C[row][column])
        print error.min(), error.max()
        # use vmin=error.min(), vmax=error.max()
        cax = axes[row][column].contourf(
            theta, r, error, 50, vmin=-1, vmax=-0.28)
        axes[row][column].set_xticks([])
        axes[row][column].set_yticks([])

# Add colorbar to the last plot
cbar = plt.colorbar(cax)
cbar.ax.set_ylabel('error in wave velocity')
plt.savefig('disprel2D.png');  plt.savefig('disprel2D.pdf')
plt.show()
```

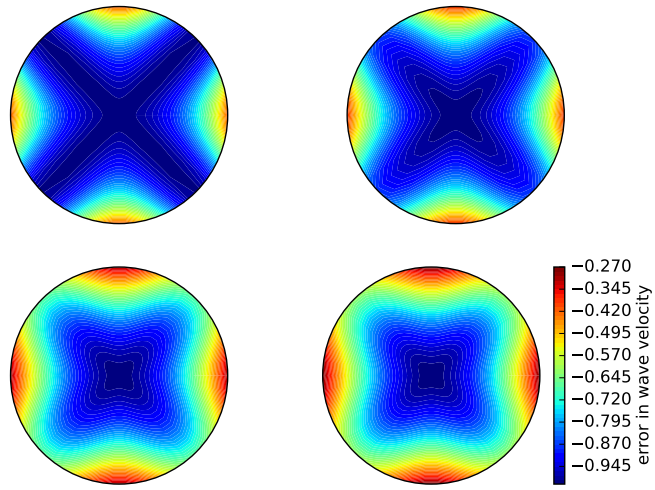


Fig. 2.7 Error in numerical dispersion in 2D.

2.11 Finite difference methods for 2D and 3D wave equations

A natural next step is to consider extensions of the methods for various variants of the one-dimensional wave equation to two-dimensional (2D) and three-dimensional (3D) versions of the wave equation.

2.11.1 Multi-dimensional wave equations

The general wave equation in d space dimensions, with constant wave velocity c, can be written in the compact form

$$\frac{\partial^2 u}{\partial t^2} = c^2 \nabla^2 u \text{ for } \boldsymbol{x} \in \Omega \subset \mathbb{R}^d,\ t \in (0, T], \tag{2.104}$$

where

$$\nabla^2 u = \frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2},$$

in a 2D problem ($d = 2$) and

$$\nabla^2 u = \frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} + \frac{\partial^2 u}{\partial z^2},$$

in three space dimensions ($d = 3$).

Many applications involve variable coefficients, and the general wave equation in d dimensions is in this case written as

$$\varrho \frac{\partial^2 u}{\partial t^2} = \nabla \cdot (q \nabla u) + f \text{ for } \boldsymbol{x} \in \Omega \subset \mathbb{R}^d,\ t \in (0, T], \tag{2.105}$$

which in, e.g., 2D becomes

$$\varrho(x,y) \frac{\partial^2 u}{\partial t^2} = \frac{\partial}{\partial x}\left(q(x,y)\frac{\partial u}{\partial x}\right) + \frac{\partial}{\partial y}\left(q(x,y)\frac{\partial u}{\partial y}\right) + f(x,y,t)\,. \tag{2.106}$$

To save some writing and space we may use the index notation, where subscript t, x, or y means differentiation with respect to that coordinate. For example,

$$\frac{\partial^2 u}{\partial t^2} = u_{tt},$$
$$\frac{\partial}{\partial y}\left(q(x,y)\frac{\partial u}{\partial y}\right) = (q u_y)_y\,.$$

These comments extend straightforwardly to 3D, which means that the 3D versions of the two wave PDEs, with and without variable coefficients, can be stated as

$$u_{tt} = c^2(u_{xx} + u_{yy} + u_{zz}) + f, \tag{2.107}$$
$$\varrho u_{tt} = (q u_x)_x + (q u_y)_y + (q u_z)_z + f\,. \tag{2.108}$$

At *each point* of the boundary $\partial\Omega$ (of Ω) we need *one* boundary condition involving the unknown u. The boundary conditions are of three principal types:

1. u is prescribed ($u = 0$ or a known time variation of u at the boundary points, e.g., modeling an incoming wave),
2. $\partial u/\partial n = \boldsymbol{n} \cdot \nabla u$ is prescribed (zero for reflecting boundaries),

3. an open boundary condition (also called radiation condition) is specified to let waves travel undisturbed out of the domain, see Exercise 2.12 for details.

All the listed wave equations with *second-order* derivatives in time need *two* initial conditions:

1. $u = I$,
2. $u_t = V$.

2.11.2 Mesh

We introduce a mesh in time and in space. The mesh in time consists of time points

$$t_0 = 0 < t_1 < \cdots < t_{N_t},$$

normally, for wave equation problems, with a constant spacing $\Delta t = t_{n+1} - t_n$, $n \in \mathcal{I}_t^-$.

Finite difference methods are easy to implement on simple rectangle- or box-shaped *spatial domains*. More complicated shapes of the spatial domain require substantially more advanced techniques and implementational efforts (and a finite element method is usually a more convenient approach). On a rectangle- or box-shaped domain, mesh points are introduced separately in the various space directions:

$$\begin{aligned} &x_0 < x_1 < \cdots < x_{N_x} \text{ in the } x \text{ direction,} \\ &y_0 < y_1 < \cdots < y_{N_y} \text{ in the } y \text{ direction,} \\ &z_0 < z_1 < \cdots < z_{N_z} \text{ in the } z \text{ direction}. \end{aligned}$$

We can write a general mesh point as (x_i, y_j, z_k, t_n), with $i \in \mathcal{I}_x$, $j \in \mathcal{I}_y$, $k \in \mathcal{I}_z$, and $n \in \mathcal{I}_t$.

It is a very common choice to use constant mesh spacings: $\Delta x = x_{i+1} - x_i$, $i \in \mathcal{I}_x^-$, $\Delta y = y_{j+1} - y_j$, $j \in \mathcal{I}_y^-$, and $\Delta z = z_{k+1} - z_k$, $k \in \mathcal{I}_z^-$. With equal mesh spacings one often introduces $h = \Delta x = \Delta y = \Delta z$.

The unknown u at mesh point (x_i, y_j, z_k, t_n) is denoted by $u^n_{i,j,k}$. In 2D problems we just skip the z coordinate (by assuming no variation in that direction: $\partial/\partial z = 0$) and write $u^n_{i,j}$.

2.11.3 Discretization

Two- and three-dimensional wave equations are easily discretized by assembling building blocks for discretization of 1D wave equations, because the multi-dimensional versions just contain terms of the same type as those in 1D.

Discretizing the PDEs. Equation (2.107) can be discretized as

$$[D_t D_t u = c^2(D_x D_x u + D_y D_y u + D_z D_z u) + f]^n_{i,j,k}\,. \tag{2.109}$$

A 2D version might be instructive to write out in detail:

$$[D_t D_t u = c^2(D_x D_x u + D_y D_y u) + f]^n_{i,j},$$

which becomes

$$\frac{u^{n+1}_{i,j} - 2u^n_{i,j} + u^{n-1}_{i,j}}{\Delta t^2} = c^2\frac{u^n_{i+1,j} - 2u^n_{i,j} + u^n_{i-1,j}}{\Delta x^2} + c^2\frac{u^n_{i,j+1} - 2u^n_{i,j} + u^n_{i,j-1}}{\Delta y^2} + f^n_{i,j},$$

Assuming, as usual, that all values at time levels n and $n-1$ are known, we can solve for the only unknown $u^{n+1}_{i,j}$. The result can be compactly written as

$$u^{n+1}_{i,j} = 2u^n_{i,j} + u^{n-1}_{i,j} + c^2\Delta t^2[D_x D_x u + D_y D_y u]^n_{i,j}\,. \tag{2.110}$$

As in the 1D case, we need to develop a special formula for $u^1_{i,j}$ where we combine the general scheme for $u^{n+1}_{i,j}$, when $n = 0$, with the discretization of the initial condition:

$$[D_{2t}u = V]^0_{i,j} \quad \Rightarrow \quad u^{-1}_{i,j} = u^1_{i,j} - 2\Delta t V_{i,j}\,.$$

The result becomes, in compact form,

$$u^1_{i,j} = u^0_{i,j} - 2\Delta V_{i,j} + \frac{1}{2}c^2\Delta t^2[D_x D_x u + D_y D_y u]^0_{i,j}\,. \tag{2.111}$$

The PDE (2.108) with variable coefficients is discretized term by term using the corresponding elements from the 1D case:

$$[\varrho D_t D_t u = (D_x \overline{q}^x D_x u + D_y \overline{q}^y D_y u + D_z \overline{q}^z D_z u) + f]^n_{i,j,k}\,. \tag{2.112}$$

When written out and solved for the unknown $u^{n+1}_{i,j,k}$, one gets the scheme

$$
\begin{aligned}
u^{n+1}_{i,j,k} = & -u^{n-1}_{i,j,k} + 2u^n_{i,j,k} + \\
& \frac{1}{\varrho_{i,j,k}}\frac{1}{\Delta x^2}(\frac{1}{2}(q_{i,j,k}+q_{i+1,j,k})(u^n_{i+1,j,k}-u^n_{i,j,k}) - \\
& \qquad \frac{1}{2}(q_{i-1,j,k}+q_{i,j,k})(u^n_{i,j,k}-u^n_{i-1,j,k})) + \\
& \frac{1}{\varrho_{i,j,k}}\frac{1}{\Delta y^2}(\frac{1}{2}(q_{i,j,k}+q_{i,j+1,k})(u^n_{i,j+1,k}-u^n_{i,j,k}) - \\
& \qquad \frac{1}{2}(q_{i,j-1,k}+q_{i,j,k})(u^n_{i,j,k}-u^n_{i,j-1,k})) + \\
& \frac{1}{\varrho_{i,j,k}}\frac{1}{\Delta z^2}(\frac{1}{2}(q_{i,j,k}+q_{i,j,k+1})(u^n_{i,j,k+1}-u^n_{i,j,k}) - \\
& \qquad \frac{1}{2}(q_{i,j,k-1}+q_{i,j,k})(u^n_{i,j,k}-u^n_{i,j,k-1})) + \\
& \Delta t^2 f^n_{i,j,k}\,.
\end{aligned}
$$

Also here we need to develop a special formula for $u^1_{i,j,k}$ by combining the scheme for $n = 0$ with the discrete initial condition, which is just a matter of inserting $u^{-1}_{i,j,k} = u^1_{i,j,k} - 2\Delta t V_{i,j,k}$ in the scheme and solving for $u^1_{i,j,k}$.

Handling boundary conditions where u is known. The schemes listed above are valid for the internal points in the mesh. After updating these, we need to visit all the mesh points at the boundaries and set the prescribed u value.

Discretizing the Neumann condition. The condition $\partial u/\partial n = 0$ was implemented in 1D by discretizing it with a $D_{2x}u$ centered difference, followed by eliminating the fictitious u point outside the mesh by using the general scheme at the boundary point. Alternatively, one can introduce ghost cells and update a ghost value for use in the Neumann condition. Exactly the same ideas are reused in multiple dimensions.

Consider the condition $\partial u/\partial n = 0$ at a boundary $y = 0$ of a rectangular domain $[0, L_x]\times[0, L_y]$ in 2D. The normal direction is then in $-y$ direction, so

$$
\frac{\partial u}{\partial n} = -\frac{\partial u}{\partial y},
$$

and we set

$$[-D_{2y}u = 0]^n_{i,0} \quad \Rightarrow \quad \frac{u^n_{i,1} - u^n_{i,-1}}{2\Delta y} = 0\,.$$

From this it follows that $u^n_{i,-1} = u^n_{i,1}$. The discretized PDE at the boundary point $(i, 0)$ reads

$$\frac{u^{n+1}_{i,0} - 2u^n_{i,0} + u^{n-1}_{i,0}}{\Delta t^2} = c^2\frac{u^n_{i+1,0} - 2u^n_{i,0} + u^n_{i-1,0}}{\Delta x^2} + c^2\frac{u^n_{i,1} - 2u^n_{i,0} + u^n_{i,-1}}{\Delta y^2} + f^n_{i,j},$$

We can then just insert $u^n_{i,1}$ for $u^n_{i,-1}$ in this equation and solve for the boundary value $u^{n+1}_{i,0}$, just as was done in 1D.

From these calculations, we see a pattern: the general scheme applies at the boundary $j = 0$ too if we just replace $j-1$ by $j+1$. Such a pattern is particularly useful for implementations. The details follow from the explained 1D case in Section 2.6.3.

The alternative approach to eliminating fictitious values outside the mesh is to have $u^n_{i,-1}$ available as a ghost value. The mesh is extended with one extra line (2D) or plane (3D) of ghost cells at a Neumann boundary. In the present example it means that we need a line with ghost cells below the y axis. The ghost values must be updated according to $u^{n+1}_{i,-1} = u^{n+1}_{i,1}$.

2.12 Implementation

We shall now describe in detail various Python implementations for solving a standard 2D, linear wave equation with constant wave velocity and $u = 0$ on the boundary. The wave equation is to be solved in the space-time domain $\Omega\times(0, T]$, where $\Omega = (0, L_x)\times(0, L_y)$ is a rectangular spatial domain. More precisely, the complete initial-boundary value problem is defined by

$$u_{tt} = c^2(u_{xx} + u_{yy}) + f(x, y, t), \qquad (x, y) \in \Omega,\ t \in (0, T], \tag{2.113}$$
$$u(x, y, 0) = I(x, y), \qquad (x, y) \in \Omega, \tag{2.114}$$
$$u_t(x, y, 0) = V(x, y), \qquad (x, y) \in \Omega, \tag{2.115}$$
$$u = 0, \qquad (x, y) \in \partial\Omega,\ t \in (0, T], \tag{2.116}$$

where $\partial\Omega$ is the boundary of Ω, in this case the four sides of the rectangle $\Omega = [0, L_x] \times [0, L_y]$: $x = 0$, $x = L_x$, $y = 0$, and $y = L_y$.

The PDE is discretized as

$$[D_t D_t u = c^2(D_x D_x u + D_y D_y u) + f]^n_{i,j},$$

which leads to an explicit updating formula to be implemented in a program:

$$u^{n+1}_{i,j} = -u^{n-1}_{i,j} + 2u^n_{i,j} + C_x^2(u^n_{i+1,j} - 2u^n_{i,j} + u^n_{i-1,j}) + C_y^2(u^n_{i,j+1} - 2u^n_{i,j} + u^n_{i,j-1}) + \Delta t^2 f^n_{i,j}, \tag{2.117}$$

for all interior mesh points $i \in \mathcal{I}^i_x$ and $j \in \mathcal{I}^i_y$, for $n \in \mathcal{I}^+_t$. The constants C_x and C_y are defined as

$$C_x = c\frac{\Delta t}{\Delta x}, \quad C_y = c\frac{\Delta t}{\Delta y}\,.$$

At the boundary, we simply set $u^{n+1}_{i,j} = 0$ for $i = 0$, $j = 0, \ldots, N_y$; $i = N_x$, $j = 0, \ldots, N_y$; $j = 0$, $i = 0, \ldots, N_x$; and $j = N_y$, $i = 0, \ldots, N_x$. For the first step, $n = 0$, (2.117) is combined with the discretization of the initial condition $u_t = V$, $[D_{2t}u = V]^0_{i,j}$ to obtain a special formula for $u^1_{i,j}$ at the interior mesh points:

$$u^1_{i,j} = u^0_{i,j} + \Delta t V_{i,j} + \frac{1}{2}C_x^2(u^0_{i+1,j} - 2u^0_{i,j} + u^0_{i-1,j}) + \frac{1}{2}C_y^2(u^0_{i,j+1} - 2u^0_{i,j} + u^0_{i,j-1}) + \frac{1}{2}\Delta t^2 f^n_{i,j}, \tag{2.118}$$

The algorithm is very similar to the one in 1D:

1. Set initial condition $u^0_{i,j} = I(x_i, y_j)$
2. Compute $u^1_{i,j}$ from (2.117)
3. Set $u^1_{i,j} = 0$ for the boundaries $i = 0, N_x$, $j = 0, N_y$
4. For $n = 1, 2, \ldots, N_t$:
   a. Find $u^{n+1}_{i,j}$ from (2.117) for all internal mesh points, $i \in \mathcal{I}^i_x$, $j \in \mathcal{I}^i_y$
   b. Set $u^{n+1}_{i,j} = 0$ for the boundaries $i = 0, N_x$, $j = 0, N_y$

2.12.1 Scalar computations

The `solver` function for a 2D case with constant wave velocity and boundary condition $u = 0$ is analogous to the 1D case with similar parameter values (see `wave1D_u0.py`), apart from a few necessary extensions. The code is found in the program `wave2D_u0.py`.

Domain and mesh. The spatial domain is now $[0, L_x] \times [0, L_y]$, specified by the arguments `Lx` and `Ly`. Similarly, the number of mesh points in the x and y directions, N_x and N_y, become the arguments `Nx` and `Ny`. In multi-dimensional problems it makes less sense to specify a Courant number since the wave velocity is a vector and mesh spacings may differ in the various spatial directions. We therefore give Δt explicitly. The signature of the `solver` function is then

```
def solver(I, V, f, c, Lx, Ly, Nx, Ny, dt, T,
           user_action=None, version='scalar'):
```

Key parameters used in the calculations are created as

```
x = linspace(0, Lx, Nx+1)                  # mesh points in x dir
y = linspace(0, Ly, Ny+1)                  # mesh points in y dir
dx = x[1] - x[0]
dy = y[1] - y[0]
Nt = int(round(T/float(dt)))
t = linspace(0, N*dt, N+1)                 # mesh points in time
Cx2 = (c*dt/dx)**2;  Cy2 = (c*dt/dy)**2    # help variables
dt2 = dt**2
```

Solution arrays. We store $u_{i,j}^{n+1}$, $u_{i,j}^{n}$, and $u_{i,j}^{n-1}$ in three two-dimensional arrays,

```
u   = zeros((Nx+1,Ny+1))   # solution array
u_n = [zeros((Nx+1,Ny+1)), zeros((Nx+1,Ny+1))]  # t-dt, t-2*dt
```

where $u_{i,j}^{n+1}$ corresponds to `u[i,j]`, $u_{i,j}^{n}$ to `u_n[i,j]`, and $u_{i,j}^{n-1}$ to `u_nm1[i,j]`.

Index sets. It is also convenient to introduce the index sets (cf. Section 2.6.4)

```
Ix = range(0, u.shape[0])
Iy = range(0, u.shape[1])
It = range(0, t.shape[0])
```

Computing the solution. Inserting the initial condition `I` in `u_n` and making a callback to the user in terms of the `user_action` function is a straightforward generalization of the 1D code from Section 2.1.6:

```
for i in Ix:
    for j in Iy:
        u_n[i,j] = I(x[i], y[j])

if user_action is not None:
    user_action(u_n, x, xv, y, yv, t, 0)
```

The `user_action` function has additional arguments compared to the 1D case. The arguments `xv` and `yv` will be commented upon in Section 2.12.2.

The key finite difference formula (2.110) for updating the solution at a time level is implemented in a separate function as

```
def advance_scalar(u, u_n, u_nm1, f, x, y, t, n, Cx2, Cy2, dt2,
                   V=None, step1=False):
    Ix = range(0, u.shape[0]);  Iy = range(0, u.shape[1])
    if step1:
        dt = sqrt(dt2)  # save
        Cx2 = 0.5*Cx2;  Cy2 = 0.5*Cy2; dt2 = 0.5*dt2  # redefine
        D1 = 1;  D2 = 0
    else:
        D1 = 2;  D2 = 1
    for i in Ix[1:-1]:
        for j in Iy[1:-1]:
            u_xx = u_n[i-1,j] - 2*u_n[i,j] + u_n[i+1,j]
            u_yy = u_n[i,j-1] - 2*u_n[i,j] + u_n[i,j+1]
            u[i,j] = D1*u_n[i,j] - D2*u_nm1[i,j] + \
                     Cx2*u_xx + Cy2*u_yy + dt2*f(x[i], y[j], t[n])
            if step1:
                u[i,j] += dt*V(x[i], y[j])
    # Boundary condition u=0
    j = Iy[0]
    for i in Ix: u[i,j] = 0
    j = Iy[-1]
    for i in Ix: u[i,j] = 0
    i = Ix[0]
    for j in Iy: u[i,j] = 0
    i = Ix[-1]
    for j in Iy: u[i,j] = 0
    return u
```

The `step1` variable has been introduced to allow the formula to be reused for the first step, computing $u^1_{i,j}$:

```
u = advance_scalar(u, u_n, f, x, y, t,
                   n, Cx2, Cy2, dt, V, step1=True)
```

Below, we will make many alternative implementations of the `advance_scalar` function to speed up the code since most of the CPU time in simulations is spent in this function.

Remark: How to use the solution

The `solver` function in the `wave2D_u0.py` code updates arrays for the next time step by switching references as described in Section 2.4.5. Any use of `u` on the user's side is assumed to take place in the user action function. However, should the code be changed such that `u` is returned and used as solution, have in mind that you must return `u_n` after the time limit, otherwise a `return u` will actually return `u_nm1` (due to the switching of array indices in the loop)!

2.12.2 Vectorized computations

The scalar code above turns out to be extremely slow for large 2D meshes, and probably useless in 3D beyond debugging of small test cases. Vectorization is therefore a must for multi-dimensional finite difference computations in Python. For example, with a mesh consisting of 30×30 cells, vectorization brings down the CPU time by a factor of 70 (!). Equally important, vectorized code can also easily be parallelized to take (usually) optimal advantage of parallel computer platforms.

In the vectorized case, we must be able to evaluate user-given functions like $I(x, y)$ and $f(x, y, t)$ for the entire mesh in one operation (without loops). These user-given functions are provided as Python functions `I(x,y)` and `f(x,y,t)`, respectively. Having the one-dimensional coordinate arrays `x` and `y` is not sufficient when calling `I` and `f` in a vectorized way. We must extend `x` and `y` to their vectorized versions `xv` and `yv`:

```
from numpy import newaxis
xv = x[:,newaxis]
yv = y[newaxis,:]
# or
xv = x.reshape((x.size, 1))
yv = y.reshape((1, y.size))
```

This is a standard required technique when evaluating functions over a 2D mesh, say `sin(xv)*cos(xv)`, which then gives a result with shape `(Nx+1,Ny+1)`. Calling `I(xv, yv)` and `f(xv, yv, t[n])` will now return `I` and `f` values for the entire set of mesh points.

With the `xv` and `yv` arrays for vectorized computing, setting the initial condition is just a matter of

```
u_n[:,:] = I(xv, yv)
```

One could also have written `u_n = I(xv, yv)` and let `u_n` point to a new object, but vectorized operations often make use of direct insertion in the original array through `u_n[:,:]`, because sometimes not all of the array is to be filled by such a function evaluation. This is the case with the computational scheme for $u_{i,j}^{n+1}$:

```
def advance_vectorized(u, u_n, u_nm1, f_a, Cx2, Cy2, dt2,
                       V=None, step1=False):
    if step1:
        dt = np.sqrt(dt2)  # save
        Cx2 = 0.5*Cx2;  Cy2 = 0.5*Cy2; dt2 = 0.5*dt2  # redefine
        D1 = 1;  D2 = 0
    else:
        D1 = 2;  D2 = 1
    u_xx = u_n[:-2,1:-1] - 2*u_n[1:-1,1:-1] + u_n[2:,1:-1]
    u_yy = u_n[1:-1,:-2] - 2*u_n[1:-1,1:-1] + u_n[1:-1,2:]
    u[1:-1,1:-1] = D1*u_n[1:-1,1:-1] - D2*u_nm1[1:-1,1:-1] + \
                   Cx2*u_xx + Cy2*u_yy + dt2*f_a[1:-1,1:-1]
    if step1:
        u[1:-1,1:-1] += dt*V[1:-1, 1:-1]
    # Boundary condition u=0
    j = 0
    u[:,j] = 0
    j = u.shape[1]-1
    u[:,j] = 0
    i = 0
    u[i,:] = 0
    i = u.shape[0]-1
    u[i,:] = 0
    return u
```

Array slices in 2D are more complicated to understand than those in 1D, but the logic from 1D applies to each dimension separately. For example, when doing $u_{i,j}^n - u_{i-1,j}^n$ for $i \in \mathcal{I}_x^+$, we just keep `j` constant and make a slice in the first index: `u_n[1:,j] - u_n[:-1,j]`, exactly as in 1D. The `1:` slice specifies all the indices $i = 1, 2, \ldots, N_x$ (up to the last valid index), while `:-1` specifies the relevant indices for the second term: $0, 1, \ldots, N_x - 1$ (up to, but not including the last index).

In the above code segment, the situation is slightly more complicated, because each displaced slice in one direction is accompanied by a `1:-1` slice in the other direction. The reason is that we only work with the internal points for the index that is kept constant in a difference.

The boundary conditions along the four sides make use of a slice consisting of all indices along a boundary:

```
u[: ,0] = 0
```

```
u[:,Ny] = 0
u[0 ,:] = 0
u[Nx,:] = 0
```

In the vectorized update of `u` (above), the function `f` is first computed as an array over all mesh points:

```
f_a = f(xv, yv, t[n])
```

We could, alternatively, have used the call `f(xv, yv, t[n])[1:-1,1:-1]` in the last term of the update statement, but other implementations in compiled languages benefit from having `f` available in an array rather than calling our Python function `f(x,y,t)` for every point.

Also in the `advance_vectorized` function we have introduced a boolean `step1` to reuse the formula for the first time step in the same way as we did with `advance_scalar`. We refer to the `solver` function in `wave2D_u0.py` for the details on how the overall algorithm is implemented.

The callback function now has the arguments `u, x, xv, y, yv, t, n`. The inclusion of `xv` and `yv` makes it easy to, e.g., compute an exact 2D solution in the callback function and compute errors, through an expression like `u - u_exact(xv, yv, t[n])`.

2.12.3 Verification

Testing a quadratic solution. The 1D solution from Section 2.2.4 can be generalized to multi-dimensions and provides a test case where the exact solution also fulfills the discrete equations, such that we know (to machine precision) what numbers the solver function should produce. In 2D we use the following generalization of (2.30):

$$u_{\mathrm{e}}(x,y,t) = x(L_x-x)y(L_y-y)(1+\frac{1}{2}t)\,. \tag{2.119}$$

This solution fulfills the PDE problem if $I(x,y) = u_{\mathrm{e}}(x,y,0)$, $V = \frac{1}{2}u_{\mathrm{e}}(x,y,0)$, and $f = 2c^2(1+\frac{1}{2}t)(y(L_y-y)+x(L_x-x))$. To show that u_{e} also solves the discrete equations, we start with the general results $[D_tD_t1]^n = 0$, $[D_tD_tt]^n = 0$, and $[D_tD_tt^2] = 2$, and use these to compute

$$\begin{aligned}[D_xD_xu_{\mathrm{e}}]^n_{i,j} &= [y(L_y-y)(1+\frac{1}{2}t)D_xD_xx(L_x-x)]^n_{i,j}\\ &= y_j(L_y-y_j)(1+\frac{1}{2}t_n)(-2)\,.\end{aligned}$$

A similar calculation must be carried out for the $[D_yD_yu_{\mathrm{e}}]^n_{i,j}$ and $[D_tD_tu_{\mathrm{e}}]^n_{i,j}$ terms. One must also show that the quadratic solution fits the special formula for $u^1_{i,j}$. The details are left as Exercise 2.16. The `test_quadratic` function in the `wave2D_u0.py` program implements this verification as a proper test function for the pytest and nose frameworks.

2.12.4 Visualization

Eventually, we are ready for a real application with our code! Look at the `wave2D_u0.py` and the `gaussian` function. It starts with a Gaussian function to see how it propagates in a square with $u = 0$ on the boundaries:

```
def gaussian(plot_method=2, version='vectorized', save_plot=True):
    """
    Initial Gaussian bell in the middle of the domain.
    plot_method=1 applies mesh function,
    =2 means surf, =3 means Matplotlib, =4 means mayavi,
    =0 means no plot.
    """
    # Clean up plot files
    for name in glob('tmp_*.png'):
        os.remove(name)

    Lx = 10
    Ly = 10
    c = 1.0

    from numpy import exp

    def I(x, y):
        """Gaussian peak at (Lx/2, Ly/2)."""
        return exp(-0.5*(x-Lx/2.0)**2 - 0.5*(y-Ly/2.0)**2)

    def plot_u(u, x, xv, y, yv, t, n):
        """User action function for plotting."""
    ...

    Nx = 40; Ny = 40; T = 20
    dt, cpu = solver(I, None, None, c, Lx, Ly, Nx, Ny, -1, T,
                     user_action=plot_u, version=version)
```

Matplotlib. We want to animate a 3D surface in Matplotlib, but this is a really slow process and not recommended, so we consider Matplotlib not an option as long as on-screen animation is desired. One can use the recipes for single shots of u, where it does produce high-quality 3D plots.

Gnuplot. Let us look at different ways for visualization. We import SciTools as `st` and can access `st.mesh` and `st.surf` in Matplotlib or Gnuplot, but this is not supported except for the Gnuplot package, where it works really well (Figure 2.8). Then we choose `plot_method=2` (or less relevant `plot_method=1`) and force the backend for SciTools to be Gnuplot (if you have the C package Gnuplot and the `Gnuplot.py` Python interface module installed):

Terminal

```
Terminal> python wave2D_u0.py --SCITOOLS_easyviz_backend gnuplot
```

It gives a nice visualization with lifted surface and contours beneath. Figure 2.8 shows four plots of u.

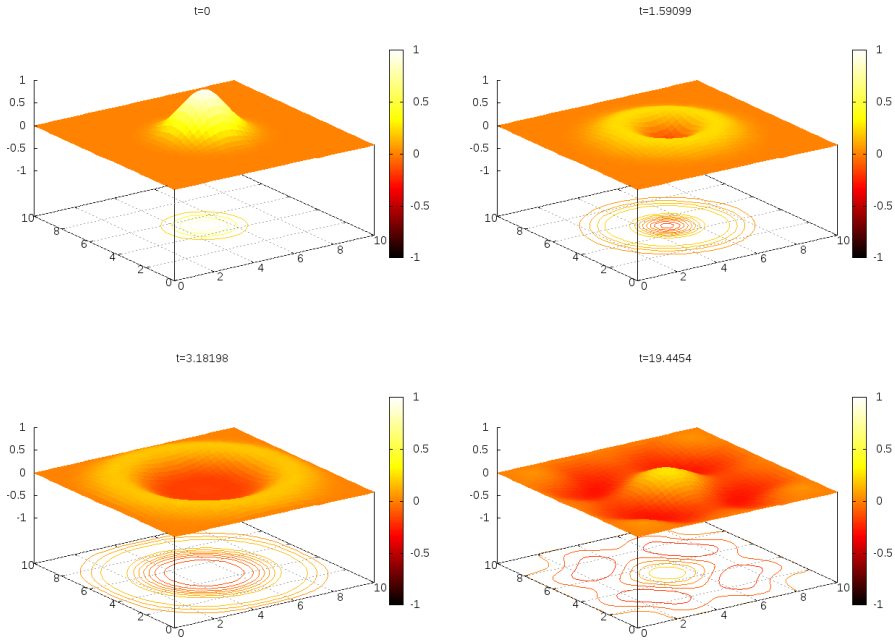


Fig. 2.8 Snapshots of the surface plotted by Gnuplot.

Video files can be made of the PNG frames:

Terminal

```
Terminal> ffmpeg -i tmp_%04d.png -r 25 -vcodec flv movie.flv
Terminal> ffmpeg -i tmp_%04d.png -r 25 -vcodec linx264 movie.mp4
Terminal> ffmpeg -i tmp_%04d.png -r 25 -vcodec libvpx movie.webm
Terminal> ffmpeg -i tmp_%04d.png -r 25 -vcodec libtheora movie.ogg
```

It is wise to use a high frame rate – a low one will just skip many frames. There may also be considerable quality differences between the different formats.

Movie 1: `https://raw.githubusercontent.com/hplgit/fdm-book/master/doc/pub/book/html/mov-wave/gnuplot/wave2D_u0_gaussian/movie25.mp4`

Mayavi. The best option for doing visualization of 2D and 3D scalar and vector fields in Python programs is Mayavi, which is an interface to the high-quality package VTK in C++. There is good online documentation and also an introduction in Chapter 5 of [10].

To obtain Mayavi on Ubuntu platforms you can write

Terminal

```
pip install mayavi --upgrade
```

For Mac OS X and Windows, we recommend using Anaconda. To obtain Mayavi for Anaconda you can write

Terminal

```
conda install mayavi
```

Mayavi has a MATLAB-like interface called `mlab`. We can do

```
import mayavi.mlab as plt
# or
from mayavi import mlab
```

and have `plt` (as usual) or `mlab` as a kind of MATLAB visualization access inside our program (just more powerful and with higher visual quality).

The official documentation of the `mlab` module is provided in two places, one for the basic functionality[10] and one for further functionality[11]. Basic figure handling[12] is very similar to the one we know from Matplotlib. Just as for Matplotlib, all plotting commands you do in `mlab` will go into the same figure, until you manually change to a new figure.

Back to our application, the following code for the user action function with plotting in Mayavi is relevant to add.

```
# Top of the file
try:
    import mayavi.mlab as mlab
```

[10] `http://docs.enthought.com/mayavi/mayavi/auto/mlab_helper_functions.html`
[11] `http://docs.enthought.com/mayavi/mayavi/auto/mlab_other_functions.html`
[12] `http://docs.enthought.com/mayavi/mayavi/auto/mlab_figure.html`

```
except:
    # We don't have mayavi
    pass

def solver(...):
    ...

def gaussian(...):
    ...
    if plot_method == 3:
        from mpl_toolkits.mplot3d import axes3d
        import matplotlib.pyplot as plt
        from matplotlib import cm
        plt.ion()
        fig = plt.figure()
        u_surf = None

    def plot_u(u, x, xv, y, yv, t, n):
        """User action function for plotting."""
        if t[n] == 0:
            time.sleep(2)
        if plot_method == 1:
            # Works well with Gnuplot backend, not with Matplotlib
            st.mesh(x, y, u, title='t=%g' % t[n], zlim=[-1,1],
                    caxis=[-1,1])
        elif plot_method == 2:
            # Works well with Gnuplot backend, not with Matplotlib
            st.surfc(xv, yv, u, title='t=%g' % t[n], zlim=[-1, 1],
                  colorbar=True, colormap=st.hot(), caxis=[-1,1],
                  shading='flat')
        elif plot_method == 3:
            print 'Experimental 3D matplotlib...not recommended'
        elif plot_method == 4:
        # Mayavi visualization
            mlab.clf()
            extent1 = (0, 20, 0, 20,-2, 2)
            s = mlab.surf(x , y, u,
                          colormap='Blues',
                          warp_scale=5,extent=extent1)
            mlab.axes(s, color=(.7, .7, .7), extent=extent1,
                      ranges=(0, 10, 0, 10, -1, 1),
                      xlabel='', ylabel='', zlabel='',
                      x_axis_visibility=False,
                      z_axis_visibility=False)
            mlab.outline(s, color=(0.7, .7, .7), extent=extent1)
            mlab.text(6, -2.5, '', z=-4, width=0.14)
            mlab.colorbar(object=None, title=None,
                          orientation='horizontal',
                          nb_labels=None, nb_colors=None,
                          label_fmt=None)
            mlab.title('Gaussian t=%g' % t[n])
            mlab.view(142, -72, 50)
            f = mlab.gcf()
            camera = f.scene.camera
```

```
            camera.yaw(0)

        if plot_method > 0:
            time.sleep(0) # pause between frames
            if save_plot:
                filename = 'tmp_%04d.png' % n
        if plot_method == 4:
                    mlab.savefig(filename)  # time consuming!
        elif plot_method in (1,2):
                    st.savefig(filename)  # time consuming!
```

This is a point to get started – visualization is as always a very time-consuming and experimental discipline. With the PNG files we can use `ffmpeg` to create videos.

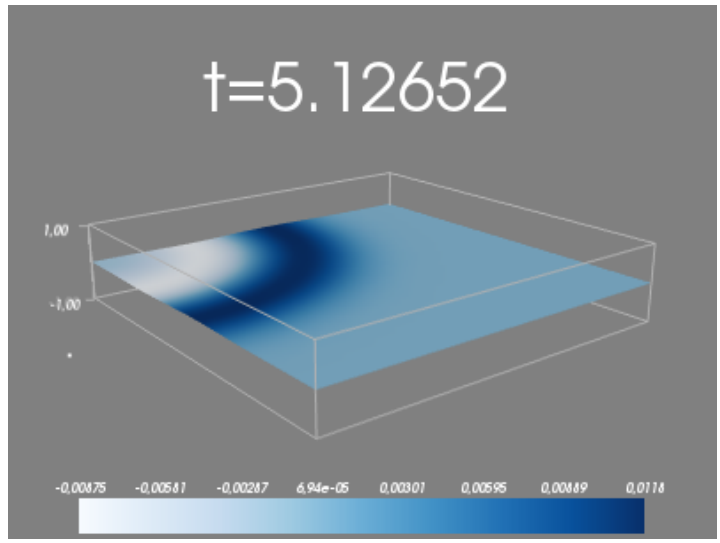


Fig. 2.9 Plot with Mayavi.

Movie 2: `https://github.com/hplgit/fdm-book/blob/master/doc/pub/book/html/mov-wave/mayavi/wave2D_u0_gaussian/movie.mp4`

2.13 Exercises

Exercise 2.16: Check that a solution fulfills the discrete model

Carry out all mathematical details to show that (2.119) is indeed a solution of the discrete model for a 2D wave equation with $u = 0$ on the

boundary. One must check the boundary conditions, the initial conditions, the general discrete equation at a time level and the special version of this equation for the first time level. Filename: `check_quadratic_solution`.

Project 2.17: Calculus with 2D mesh functions

The goal of this project is to redo Project 2.6 with 2D mesh functions $(f_{i,j})$.

Differentiation. The differentiation results in a discrete gradient function, which in the 2D case can be represented by a three-dimensional array `df[d,i,j]` where `d` represents the direction of the derivative, and `i,j` is a mesh point in 2D. Use centered differences for the derivative at inner points and one-sided forward or backward differences at the boundary points. Construct unit tests and write a corresponding test function.

Integration. The integral of a 2D mesh function $f_{i,j}$ is defined as

$$F_{i,j} = \int_{y_0}^{y_j}\int_{x_0}^{x_i} f(x,y)dxdy,$$

where $f(x,y)$ is a function that takes on the values of the discrete mesh function $f_{i,j}$ at the mesh points, but can also be evaluated in between the mesh points. The particular variation between mesh points can be taken as bilinear, but this is not important as we will use a product Trapezoidal rule to approximate the integral over a cell in the mesh and then we only need to evaluate $f(x,y)$ at the mesh points.

Suppose $F_{i,j}$ is computed. The calculation of $F_{i+1,j}$ is then

$$\begin{aligned} F_{i+1,j} &= F_{i,j} + \int_{x_i}^{x_{i+1}}\int_{y_0}^{y_j} f(x,y)dydx \\ &\approx \Delta x\frac{1}{2}\left(\int_{y_0}^{y_j} f(x_i,y)dy + \int_{y_0}^{y_j} f(x_{i+1},y)dy\right) \end{aligned}$$

The integrals in the y direction can be approximated by a Trapezoidal rule. A similar idea can be used to compute $F_{i,j+1}$. Thereafter, $F_{i+1,j+1}$ can be computed by adding the integral over the final corner cell to $F_{i+1,j} + F_{i,j+1} - F_{i,j}$. Carry out the details of these computations and implement a function that can return $F_{i,j}$ for all mesh indices i and j. Use the fact that the Trapezoidal rule is exact for linear functions and write a test function. Filename: `mesh_calculus_2D`.

Exercise 2.18: Implement Neumann conditions in 2D

Modify the `wave2D_u0.py` program, which solves the 2D wave equation $u_{tt} = c^2(u_{xx} + u_{yy})$ with constant wave velocity c and $u = 0$ on the boundary, to have Neumann boundary conditions: $\partial u/\partial n = 0$. Include both scalar code (for debugging and reference) and vectorized code (for speed).

To test the code, use $u = 1.2$ as solution ($I(x, y) = 1.2$, $V = f = 0$, and c arbitrary), which should be exactly reproduced with any mesh as long as the stability criterion is satisfied. Another test is to use the plug-shaped pulse in the `pulse` function from Section 2.8 and the `wave1D_dn_vc.py` program. This pulse is exactly propagated in 1D if $c\Delta t/\Delta x = 1$. Check that also the 2D program can propagate this pulse exactly in x direction ($c\Delta t/\Delta x = 1$, Δy arbitrary) and y direction ($c\Delta t/\Delta y = 1$, Δx arbitrary). Filename: `wave2D_dn`.

Exercise 2.19: Test the efficiency of compiled loops in 3D

Extend the `wave2D_u0.py` code and the Cython, Fortran, and C versions to 3D. Set up an efficiency experiment to determine the relative efficiency of pure scalar Python code, vectorized code, Cython-compiled loops, Fortran-compiled loops, and C-compiled loops. Normalize the CPU time for each mesh by the fastest version. Filename: `wave3D_u0`.

2.14 Applications of wave equations

This section presents a range of wave equation models for different physical phenomena. Although many wave motion problems in physics can be modeled by the standard linear wave equation, or a similar formulation with a system of first-order equations, there are some exceptions. Perhaps the most important is water waves: these are modeled by the Laplace equation with time-dependent boundary conditions at the water surface (long water waves, however, can be approximated by a standard wave equation, see Section 2.14.7). Quantum mechanical waves constitute another example where the waves are governed by the Schrödinger equation, i.e., not by a standard wave equation. Many wave phenomena also need to take nonlinear effects into account when the wave amplitude is significant. Shock waves in the air is a primary example.

The derivations in the following are very brief. Those with a firm background in continuum mechanics will probably have enough knowledge to fill in the details, while other readers will hopefully get some impression of the physics and approximations involved when establishing wave equation models.

2.14.1 Waves on a string

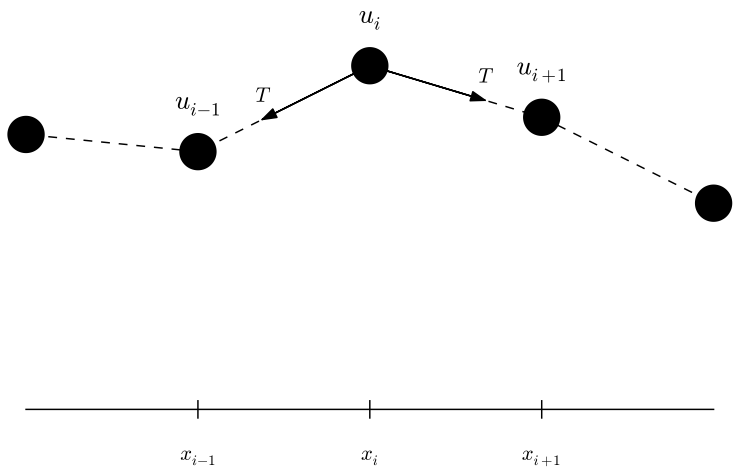


Fig. 2.10 Discrete string model with point masses connected by elastic strings.

Figure 2.10 shows a model we may use to derive the equation for waves on a string. The string is modeled as a set of discrete point

masses (at mesh points) with elastic strings in between. The string has a large constant tension T. We let the mass at mesh point x_i be m_i. The displacement of this mass point in the y direction is denoted by $u_i(t)$.

The motion of mass m_i is governed by Newton's second law of motion. The position of the mass at time t is $x_i\boldsymbol{i} + u_i(t)\boldsymbol{j}$, where $\boldsymbol{i}$ and $\boldsymbol{j}$ are unit vectors in the x and y direction, respectively. The acceleration is then $u_i''(t)\boldsymbol{j}$. Two forces are acting on the mass as indicated in Figure 2.10. The force $\boldsymbol{T}^-$ acting toward the point x_{i-1} can be decomposed as

$$\boldsymbol{T}^- = -T\sin\phi\boldsymbol{i} - T\cos\phi\boldsymbol{j},$$

where ϕ is the angle between the force and the line $x = x_i$. Let $\Delta u_i = u_i - u_{i-1}$ and let $\Delta s_i = \sqrt{\Delta u_i^2 + (x_i - x_{i-1})^2}$ be the distance from mass m_{i-1} to mass m_i. It is seen that $\cos\phi = \Delta u_i/\Delta s_i$ and $\sin\phi = (x_i - x_{i-1})/\Delta s$ or $\Delta x/\Delta s_i$ if we introduce a constant mesh spacing $\Delta x = x_i - x_{i-1}$. The force can then be written

$$\boldsymbol{T}^- = -T\frac{\Delta x}{\Delta s_i}\boldsymbol{i} - T\frac{\Delta u_i}{\Delta s_i}\boldsymbol{j}\,.$$

The force $\boldsymbol{T}^+$ acting toward x_{i+1} can be calculated in a similar way:

$$\boldsymbol{T}^+ = T\frac{\Delta x}{\Delta s_{i+1}}\boldsymbol{i} + T\frac{\Delta u_{i+1}}{\Delta s_{i+1}}\boldsymbol{j}\,.$$

Newton's second law becomes

$$m_i u_i''(t)\boldsymbol{j} = \boldsymbol{T}^+ + \boldsymbol{T}^-,$$

which gives the component equations

$$T\frac{\Delta x}{\Delta s_i} = T\frac{\Delta x}{\Delta s_{i+1}}, \tag{2.120}$$

$$m_i u_i''(t) = T\frac{\Delta u_{i+1}}{\Delta s_{i+1}} - T\frac{\Delta u_i}{\Delta s_i}\,. \tag{2.121}$$

A basic reasonable assumption for a string is small displacements u_i and small displacement gradients $\Delta u_i/\Delta x$. For small $g = \Delta u_i/\Delta x$ we have that

$$\Delta s_i = \sqrt{\Delta u_i^2 + \Delta x^2} = \Delta x\sqrt{1+g^2} + \Delta x(1 + \frac{1}{2}g^2 + \mathcal{O}(g^4)) \approx \Delta x\,.$$

Equation (2.120) is then simply the identity $T = T$, while (2.121) can be written as

$$m_i u_i''(t) = T\frac{\Delta u_{i+1}}{\Delta x} - T\frac{\Delta u_i}{\Delta x},$$

which upon division by Δx and introducing the density $\varrho_i = m_i/\Delta x$ becomes

$$\varrho_i u_i''(t) = T\frac{1}{\Delta x^2}\left(u_{i+1} - 2u_i + u_{i-1}\right). \tag{2.122}$$

We can now choose to approximate u_i'' by a finite difference in time and get the discretized wave equation,

$$\varrho_i \frac{1}{\Delta t^2}\left(u_i^{n+1} - 2u_i^n - u_i^{n-1}\right) = T\frac{1}{\Delta x^2}\left(u_{i+1} - 2u_i + u_{i-1}\right). \tag{2.123}$$

On the other hand, we may go to the continuum limit $\Delta x \to 0$ and replace $u_i(t)$ by $u(x,t)$, ϱ_i by $\varrho(x)$, and recognize that the right-hand side of (2.122) approaches $\partial^2 u/\partial x^2$ as $\Delta x \to 0$. We end up with the continuous model for waves on a string:

$$\varrho\frac{\partial^2 u}{\partial t^2} = T\frac{\partial^2 u}{\partial x^2}. \tag{2.124}$$

Note that the density ϱ may change along the string, while the tension T is a constant. With variable wave velocity $c(x) = \sqrt{T/\varrho(x)}$ we can write the wave equation in the more standard form

$$\frac{\partial^2 u}{\partial t^2} = c^2(x)\frac{\partial^2 u}{\partial x^2}. \tag{2.125}$$

Because of the way ϱ enters the equations, the variable wave velocity does *not* appear inside the derivatives as in many other versions of the wave equation. However, most strings of interest have constant ϱ.

The end points of a string are fixed so that the displacement u is zero. The boundary conditions are therefore $u = 0$.

Damping. Air resistance and non-elastic effects in the string will contribute to reduce the amplitudes of the waves so that the motion dies out after some time. This damping effect can be modeled by a term bu_t on the left-hand side of the equation

$$\varrho\frac{\partial^2 u}{\partial t^2} + b\frac{\partial u}{\partial t} = T\frac{\partial^2 u}{\partial x^2}. \tag{2.126}$$

The parameter $b \geq 0$ is small for most wave phenomena, but the damping effect may become significant in long time simulations.

External forcing. It is easy to include an external force acting on the string. Say we have a vertical force $\tilde{f}_i \boldsymbol{j}$ acting on mass m_i, modeling the effect of gravity on a string. This force affects the vertical component of Newton's law and gives rise to an extra term $\tilde{f}(x,t)$ on the right-hand side of (2.124). In the model (2.125) we would add a term $f(x,t) = \tilde{f}(x,t)/\varrho(x)$.

Modeling the tension via springs. We assumed, in the derivation above, that the tension in the string, T, was constant. It is easy to check this assumption by modeling the string segments between the masses as standard springs, where the force (tension T) is proportional to the elongation of the spring segment. Let k be the spring constant, and set $T_i = k\Delta\ell$ for the tension in the spring segment between x_{i-1} and x_i, where $\Delta\ell$ is the elongation of this segment from the tension-free state. A basic feature of a string is that it has high tension in the equilibrium position $u = 0$. Let the string segment have an elongation $\Delta\ell_0$ in the equilibrium position. After deformation of the string, the elongation is $\Delta\ell = \Delta\ell_0 + \Delta s_i$: $T_i = k(\Delta\ell_0 + \Delta s_i) \approx k(\Delta\ell_0 + \Delta x)$. This shows that T_i is independent of i. Moreover, the extra approximate elongation Δx is very small compared to $\Delta\ell_0$, so we may well set $T_i = T = k\Delta\ell_0$. This means that the tension is completely dominated by the initial tension determined by the tuning of the string. The additional deformations of the spring during the vibrations do not introduce significant changes in the tension.

2.14.2 Elastic waves in a rod

Consider an elastic rod subject to a hammer impact at the end. This experiment will give rise to an elastic deformation pulse that travels through the rod. A mathematical model for longitudinal waves along an elastic rod starts with the general equation for deformations and stresses in an elastic medium,

$$\varrho \boldsymbol{u}_{tt} = \nabla \cdot \boldsymbol{\sigma} + \varrho \boldsymbol{f}, \tag{2.127}$$

where ϱ is the density, $\boldsymbol{u}$ the displacement field, $\boldsymbol{\sigma}$ the stress tensor, and $\boldsymbol{f}$ body forces. The latter has normally no impact on elastic waves.

For stationary deformation of an elastic rod, aligned with the x axis, one has that $\sigma_{xx} = Eu_x$, with all other stress components being zero.

The parameter E is known as Young's modulus. Moreover, we set $\boldsymbol{u} = u(x,t)\boldsymbol{i}$ and neglect the radial contraction and expansion (where Poisson's ratio is the important parameter). Assuming that this simple stress and deformation field is a good approximation, (2.127) simplifies to

$$\varrho \frac{\partial^2 u}{\partial t^2} = \frac{\partial}{\partial x}\left(E\frac{\partial u}{\partial x}\right). \tag{2.128}$$

The associated boundary conditions are u or $\sigma_{xx} = Eu_x$ known, typically $u = 0$ for a fixed end and $\sigma_{xx} = 0$ for a free end.

2.14.3 Waves on a membrane

Think of a thin, elastic membrane with shape as a circle or rectangle. This membrane can be brought into oscillatory motion and will develop elastic waves. We can model this phenomenon somewhat similar to waves in a rod: waves in a membrane are simply the two-dimensional counterpart. We assume that the material is deformed in the z direction only and write the elastic displacement field on the form $\boldsymbol{u}(x,y,t) = w(x,y,t)\boldsymbol{i}$. The z coordinate is omitted since the membrane is thin and all properties are taken as constant throughout the thickness. Inserting this displacement field in Newton's 2nd law of motion (2.127) results in

$$\varrho \frac{\partial^2 w}{\partial t^2} = \frac{\partial}{\partial x}\left(\mu\frac{\partial w}{\partial x}\right) + \frac{\partial}{\partial y}\left(\mu\frac{\partial w}{\partial y}\right). \tag{2.129}$$

This is nothing but a wave equation in $w(x,y,t)$, which needs the usual initial conditions on w and w_t as well as a boundary condition $w = 0$. When computing the stress in the membrane, one needs to split $\boldsymbol{\sigma}$ into a constant high-stress component due to the fact that all membranes are normally pre-stressed, plus a component proportional to the displacement and governed by the wave motion.

2.14.4 The acoustic model for seismic waves

Seismic waves are used to infer properties of subsurface geological structures. The physical model is a heterogeneous elastic medium where sound is propagated by small elastic vibrations. The general mathematical model for deformations in an elastic medium is based on Newton's second law,

$$\varrho \boldsymbol{u}_{tt} = \nabla \cdot \boldsymbol{\sigma} + \varrho \boldsymbol{f}, \tag{2.130}$$

and a constitutive law relating $\boldsymbol{\sigma}$ to $\boldsymbol{u}$, often Hooke's generalized law,

$$\boldsymbol{\sigma} = K\nabla \cdot \boldsymbol{u}\, \boldsymbol{I} + G(\nabla \boldsymbol{u} + (\nabla \boldsymbol{u})^T - \frac{2}{3}\nabla \cdot \boldsymbol{u}\, \boldsymbol{I})\,. \tag{2.131}$$

Here, $\boldsymbol{u}$ is the displacement field, $\boldsymbol{\sigma}$ is the stress tensor, $\boldsymbol{I}$ is the identity tensor, ϱ is the medium's density, $\boldsymbol{f}$ are body forces (such as gravity), K is the medium's bulk modulus and G is the shear modulus. All these quantities may vary in space, while $\boldsymbol{u}$ and $\boldsymbol{\sigma}$ will also show significant variation in time during wave motion.

The acoustic approximation to elastic waves arises from a basic assumption that the second term in Hooke's law, representing the deformations that give rise to shear stresses, can be neglected. This assumption can be interpreted as approximating the geological medium by a fluid. Neglecting also the body forces $\boldsymbol{f}$, (2.130) becomes

$$\varrho \boldsymbol{u}_{tt} = \nabla(K\nabla \cdot \boldsymbol{u}) \tag{2.132}$$

Introducing p as a pressure via

$$p = -K\nabla \cdot \boldsymbol{u}, \tag{2.133}$$

and dividing (2.132) by ϱ, we get

$$\boldsymbol{u}_{tt} = -\frac{1}{\varrho}\nabla p\,. \tag{2.134}$$

Taking the divergence of this equation, using $\nabla \cdot \boldsymbol{u} = -p/K$ from (2.133), gives the *acoustic approximation to elastic waves*:

$$p_{tt} = K\nabla \cdot \left(\frac{1}{\varrho}\nabla p\right)\,. \tag{2.135}$$

This is a standard, linear wave equation with variable coefficients. It is common to add a source term $s(x, y, z, t)$ to model the generation of sound waves:

$$p_{tt} = K\nabla \cdot \left(\frac{1}{\varrho}\nabla p\right) + s\,. \tag{2.136}$$

A common additional approximation of (2.136) is based on using the chain rule on the right-hand side,

$$K\nabla\cdot\left(\frac{1}{\varrho}\nabla p\right) = \frac{K}{\varrho}\nabla^2 p + K\nabla\left(\frac{1}{\varrho}\right)\cdot\nabla p \approx \frac{K}{\varrho}\nabla^2 p,$$

under the assumption that the relative spatial gradient $\nabla\varrho^{-1} = -\varrho^{-2}\nabla\varrho$ is small. This approximation results in the simplified equation

$$p_{tt} = \frac{K}{\varrho}\nabla^2 p + s\,. \tag{2.137}$$

The acoustic approximations to seismic waves are used for sound waves in the ground, and the Earth's surface is then a boundary where p equals the atmospheric pressure p_0 such that the boundary condition becomes $p = p_0$.

Anisotropy. Quite often in geological materials, the effective wave velocity $c = \sqrt{K/\varrho}$ is different in different spatial directions because geological layers are compacted, and often twisted, in such a way that the properties in the horizontal and vertical direction differ. With z as the vertical coordinate, we can introduce a vertical wave velocity c_z and a horizontal wave velocity c_h, and generalize (2.137) to

$$p_{tt} = c_z^2 p_{zz} + c_h^2(p_{xx} + p_{yy}) + s\,. \tag{2.138}$$

2.14.5 Sound waves in liquids and gases

Sound waves arise from pressure and density variations in fluids. The starting point of modeling sound waves is the basic equations for a compressible fluid where we omit viscous (frictional) forces, body forces (gravity, for instance), and temperature effects:

$$\varrho_t + \nabla\cdot(\varrho\boldsymbol{u}) = 0, \tag{2.139}$$

$$\varrho\boldsymbol{u}_t + \varrho\boldsymbol{u}\cdot\nabla\boldsymbol{u} = -\nabla p, \tag{2.140}$$

$$\varrho = \varrho(p)\,. \tag{2.141}$$

These equations are often referred to as the Euler equations for the motion of a fluid. The parameters involved are the density ϱ, the velocity $\boldsymbol{u}$, and the pressure p. Equation (2.139) reflects mass balance, (2.140) is Newton's second law for a fluid, with frictional and body forces omitted, and (2.141) is a constitutive law relating density to pressure by thermodynamic

considerations. A typical model for (2.141) is the so-called isentropic relation[13], valid for adiabatic processes where there is no heat transfer:

$$\varrho = \varrho_0 \left(\frac{p}{p_0}\right)^{1/\gamma} . \tag{2.142}$$

Here, p_0 and ϱ_0 are reference values for p and ϱ when the fluid is at rest, and γ is the ratio of specific heat at constant pressure and constant volume ($\gamma = 5/3$ for air).

The key approximation in a mathematical model for sound waves is to assume that these waves are small perturbations to the density, pressure, and velocity. We therefore write

$$\begin{aligned} p &= p_0 + \hat{p}, \\ \varrho &= \varrho_0 + \hat{\varrho}, \\ \boldsymbol{u} &= \hat{\boldsymbol{u}}, \end{aligned}$$

where we have decomposed the fields in a constant equilibrium value, corresponding to $\boldsymbol{u} = 0$, and a small perturbation marked with a hat symbol. By inserting these decompositions in (2.139) and (2.140), neglecting all product terms of small perturbations and/or their derivatives, and dropping the hat symbols, one gets the following linearized PDE system for the small perturbations in density, pressure, and velocity:

$$\varrho_t + \varrho_0 \nabla \cdot \boldsymbol{u} = 0, \tag{2.143}$$

$$\varrho_0 \boldsymbol{u}_t = -\nabla p\,. \tag{2.144}$$

Now we can eliminate ϱ_t by differentiating the relation $\varrho(p)$,

$$\varrho_t = \varrho_0 \frac{1}{\gamma} \left(\frac{p}{p_0}\right)^{1/\gamma - 1} \frac{1}{p_0} p_t = \frac{\varrho_0}{\gamma p_0} \left(\frac{p}{p_0}\right)^{1/\gamma - 1} p_t\,.$$

The product term $p^{1/\gamma-1} p_t$ can be linearized as $p_0^{1/\gamma-1} p_t$, resulting in

$$\varrho_t \approx \frac{\varrho_0}{\gamma p_0} p_t\,.$$

We then get

[13] http://en.wikipedia.org/wiki/Isentropic_process

$$p_t + \gamma p_0 \nabla \cdot \boldsymbol{u} = 0, \tag{2.145}$$

$$\boldsymbol{u}_t = -\frac{1}{\varrho_0}\nabla p, \ . \tag{2.146}$$

Taking the divergence of (2.146) and differentiating (2.145) with respect to time gives the possibility to easily eliminate $\nabla \cdot \boldsymbol{u}_t$ and arrive at a standard, linear wave equation for p:

$$p_{tt} = c^2 \nabla^2 p, \tag{2.147}$$

where $c = \sqrt{\gamma p_0 / \varrho_0}$ is the speed of sound in the fluid.

2.14.6 Spherical waves

Spherically symmetric three-dimensional waves propagate in the radial direction r only so that $u = u(r,t)$. The fully three-dimensional wave equation

$$\frac{\partial^2 u}{\partial t^2} = \nabla \cdot (c^2 \nabla u) + f$$

then reduces to the spherically symmetric wave equation

$$\frac{\partial^2 u}{\partial t^2} = \frac{1}{r^2}\frac{\partial}{\partial r}\left(c^2(r) r^2 \frac{\partial u}{\partial r}\right) + f(r,t), \quad r \in (0,R),\ t > 0\,. \tag{2.148}$$

One can easily show that the function $v(r,t) = ru(r,t)$ fulfills a standard wave equation in Cartesian coordinates if c is constant. To this end, insert $u = v/r$ in

$$\frac{1}{r^2}\frac{\partial}{\partial r}\left(c^2(r) r^2 \frac{\partial u}{\partial r}\right)$$

to obtain

$$r\left(\frac{dc^2}{dr}\frac{\partial v}{\partial r} + c^2 \frac{\partial^2 v}{\partial r^2}\right) - \frac{dc^2}{dr} v\,.$$

The two terms in the parenthesis can be combined to

$$r\frac{\partial}{\partial r}\left(c^2 \frac{\partial v}{\partial r}\right),$$

which is recognized as the variable-coefficient Laplace operator in one Cartesian coordinate. The spherically symmetric wave equation in terms of $v(r,t)$ now becomes

$$\frac{\partial^2 v}{\partial t^2} = \frac{\partial}{\partial r}\left(c^2(r)\frac{\partial v}{\partial r}\right) - \frac{1}{r}\frac{dc^2}{dr}v + rf(r,t), \quad r \in (0,R),\ t > 0\,. \tag{2.149}$$

In the case of constant wave velocity c, this equation reduces to the wave equation in a single Cartesian coordinate called r:

$$\frac{\partial^2 v}{\partial t^2} = c^2\frac{\partial^2 v}{\partial r^2} + rf(r,t), \quad r \in (0,R),\ t > 0\,. \tag{2.150}$$

That is, any program for solving the one-dimensional wave equation in a Cartesian coordinate system can be used to solve (2.150), provided the source term is multiplied by the coordinate, and that we divide the Cartesian mesh solution by r to get the spherically symmetric solution. Moreover, if $r = 0$ is included in the domain, spherical symmetry demands that $\partial u/\partial r = 0$ at $r = 0$, which means that

$$\frac{\partial u}{\partial r} = \frac{1}{r^2}\left(r\frac{\partial v}{\partial r} - v\right) = 0, \quad r = 0\,.$$

For this to hold in the limit $r \to 0$, we must have $v(0,t) = 0$ at least as a necessary condition. In most practical applications, we exclude $r = 0$ from the domain and assume that some boundary condition is assigned at $r = \epsilon$, for some $\epsilon > 0$.

2.14.7 The linear shallow water equations

The next example considers water waves whose wavelengths are much larger than the depth and whose wave amplitudes are small. This class of waves may be generated by catastrophic geophysical events, such as earthquakes at the sea bottom, landslides moving into water, or underwater slides (or a combination, as earthquakes frequently release avalanches of masses). For example, a subsea earthquake will normally have an extension of many kilometers but lift the water only a few meters. The wave length will have a size dictated by the earthquake area, which is much lager than the water depth, and compared to this wave length, an amplitude of a few meters is very small. The water is essentially a thin film, and mathematically we can average the problem

in the vertical direction and approximate the 3D wave phenomenon by 2D PDEs. Instead of a moving water domain in three space dimensions, we get a horizontal 2D domain with an unknown function for the surface elevation and the water depth as a variable coefficient in the PDEs.

Let $\eta(x,y,t)$ be the elevation of the water surface, $H(x,y)$ the water depth corresponding to a flat surface ($\eta = 0$), $u(x,y,t)$ and $v(x,y,t)$ the depth-averaged horizontal velocities of the water. Mass and momentum balance of the water volume give rise to the PDEs involving these quantities:

$$\eta_t = -(Hu)_x - (Hv)_x \tag{2.151}$$

$$u_t = -g\eta_x, \tag{2.152}$$

$$v_t = -g\eta_y, \tag{2.153}$$

where g is the acceleration of gravity. Equation (2.151) corresponds to mass balance while the other two are derived from momentum balance (Newton's second law).

The initial conditions associated with (2.151)-(2.153) are η, u, and v prescribed at $t = 0$. A common condition is to have some water elevation $\eta = I(x,y)$ and assume that the surface is at rest: $u = v = 0$. A subsea earthquake usually means a sufficiently rapid motion of the bottom and the water volume to say that the bottom deformation is mirrored at the water surface as an initial lift $I(x,y)$ and that $u = v = 0$.

Boundary conditions may be η prescribed for incoming, known waves, or zero normal velocity at reflecting boundaries (steep mountains, for instance): $un_x + vn_y = 0$, where (n_x, n_y) is the outward unit normal to the boundary. More sophisticated boundary conditions are needed when waves run up at the shore, and at open boundaries where we want the waves to leave the computational domain undisturbed.

Equations (2.151), (2.152), and (2.153) can be transformed to a standard, linear wave equation. First, multiply (2.152) and (2.153) by H, differentiate (2.152)) with respect to x and (2.153) with respect to y. Second, differentiate (2.151) with respect to t and use that $(Hu)_{xt} = (Hu_t)_x$ and $(Hv)_{yt} = (Hv_t)_y$ when H is independent of t. Third, eliminate $(Hu_t)_x$ and $(Hv_t)_y$ with the aid of the other two differentiated equations. These manipulations result in a standard, linear wave equation for η:

$$\eta_{tt} = (gH\eta_x)_x + (gH\eta_y)_y = \nabla\cdot(gH\nabla\eta)\,. \tag{2.154}$$

In the case we have an initial non-flat water surface at rest, the initial conditions become $\eta = I(x,y)$ and $\eta_t = 0$. The latter follows from (2.151) if $u = v = 0$, or simply from the fact that the vertical velocity of the surface is η_t, which is zero for a surface at rest.

The system (2.151)-(2.153) can be extended to handle a time-varying bottom topography, which is relevant for modeling long waves generated by underwater slides. In such cases the water depth function H is also a function of t, due to the moving slide, and one must add a time-derivative term H_t to the left-hand side of (2.151). A moving bottom is best described by introducing $z = H_0$ as the still-water level, $z = B(x,y,t)$ as the time- and space-varying bottom topography, so that $H = H_0 - B(x,y,t)$. In the elimination of u and v one may assume that the dependence of H on t can be neglected in the terms $(Hu)_{xt}$ and $(Hv)_{yt}$. We then end up with a source term in (2.154), because of the moving (accelerating) bottom:

$$\eta_{tt} = \nabla\cdot(gH\nabla\eta) + B_{tt}\,. \tag{2.155}$$

The reduction of (2.155) to 1D, for long waves in a straight channel, or for approximately plane waves in the ocean, is trivial by assuming no change in y direction ($\partial/\partial y = 0$):

$$\eta_{tt} = (gH\eta_x)_x + B_{tt}\,. \tag{2.156}$$

Wind drag on the surface. Surface waves are influenced by the drag of the wind, and if the wind velocity some meters above the surface is (U,V), the wind drag gives contributions $C_V\sqrt{U^2+V^2}U$ and $C_V\sqrt{U^2+V^2}V$ to (2.152) and (2.153), respectively, on the right-hand sides.

Bottom drag. The waves will experience a drag from the bottom, often roughly modeled by a term similar to the wind drag: $C_B\sqrt{u^2+v^2}u$ on the right-hand side of (2.152) and $C_B\sqrt{u^2+v^2}v$ on the right-hand side of (2.153). Note that in this case the PDEs (2.152) and (2.153) become nonlinear and the elimination of u and v to arrive at a 2nd-order wave equation for η is not possible anymore.

Effect of the Earth's rotation. Long geophysical waves will often be affected by the rotation of the Earth because of the Coriolis force. This force gives rise to a term fv on the right-hand side of (2.152) and $-fu$ on the right-hand side of (2.153). Also in this case one cannot eliminate u and v to work with a single equation for η. The Coriolis parameter is $f = 2\Omega\sin\phi$, where Ω is the angular velocity of the earth and ϕ is the latitude.

2.14.8 Waves in blood vessels

The flow of blood in our bodies is basically fluid flow in a network of pipes. Unlike rigid pipes, the walls in the blood vessels are elastic and will increase their diameter when the pressure rises. The elastic forces will then push the wall back and accelerate the fluid. This interaction between the flow of blood and the deformation of the vessel wall results in waves traveling along our blood vessels.

A model for one-dimensional waves along blood vessels can be derived from averaging the fluid flow over the cross section of the blood vessels. Let x be a coordinate along the blood vessel and assume that all cross sections are circular, though with different radii $R(x,t)$. The main quantities to compute is the cross section area $A(x,t)$, the averaged pressure $P(x,t)$, and the total volume flux $Q(x,t)$. The area of this cross section is

$$A(x,t) = 2\pi \int_0^{R(x,t)} r dr, \tag{2.157}$$

Let $v_x(x,t)$ be the velocity of blood averaged over the cross section at point x. The volume flux, being the total volume of blood passing a cross section per time unit, becomes

$$Q(x,t) = A(x,t)v_x(x,t) \tag{2.158}$$

Mass balance and Newton's second law lead to the PDEs

$$\frac{\partial A}{\partial t} + \frac{\partial Q}{\partial x} = 0, \tag{2.159}$$

$$\frac{\partial Q}{\partial t} + \frac{\gamma+2}{\gamma+1}\frac{\partial}{\partial x}\left(\frac{Q^2}{A}\right) + \frac{A}{\varrho}\frac{\partial P}{\partial x} = -2\pi(\gamma+2)\frac{\mu}{\varrho}\frac{Q}{A}, \tag{2.160}$$

where γ is a parameter related to the velocity profile, ϱ is the density of blood, and μ is the dynamic viscosity of blood.

We have three unknowns A, Q, and P, and two equations (2.159) and (2.160). A third equation is needed to relate the flow to the deformations of the wall. A common form for this equation is

$$\frac{\partial P}{\partial t} + \frac{1}{C}\frac{\partial Q}{\partial x} = 0, \tag{2.161}$$

where C is the compliance of the wall, given by the constitutive relation

$$C = \frac{\partial A}{\partial P} + \frac{\partial A}{\partial t}, \tag{2.162}$$

which requires a relationship between A and P. One common model is to view the vessel wall, locally, as a thin elastic tube subject to an internal pressure. This gives the relation

$$P = P_0 + \frac{\pi h E}{(1-\nu^2)A_0}(\sqrt{A} - \sqrt{A_0}),$$

where P_0 and A_0 are corresponding reference values when the wall is not deformed, h is the thickness of the wall, and E and ν are Young's modulus and Poisson's ratio of the elastic material in the wall. The derivative becomes

$$C = \frac{\partial A}{\partial P} = \frac{2(1-\nu^2)A_0}{\pi h E}\sqrt{A_0} + 2\left(\frac{(1-\nu^2)A_0}{\pi h E}\right)^2 (P - P_0)\,. \tag{2.163}$$

Another (nonlinear) deformation model of the wall, which has a better fit with experiments, is

$$P = P_0 \exp\left(\beta(A/A_0 - 1)\right),$$

where β is some parameter to be estimated. This law leads to

$$C = \frac{\partial A}{\partial P} = \frac{A_0}{\beta P}\,. \tag{2.164}$$

Reduction to the standard wave equation. It is not uncommon to neglect the viscous term on the right-hand side of (2.160) and also the quadratic term with Q^2 on the left-hand side. The reduced equations (2.160) and (2.161) form a first-order linear wave equation system:

$$C\frac{\partial P}{\partial t} = -\frac{\partial Q}{\partial x}, \tag{2.165}$$

$$\frac{\partial Q}{\partial t} = -\frac{A}{\varrho}\frac{\partial P}{\partial x}\,. \tag{2.166}$$

These can be combined into standard 1D wave PDE by differentiating the first equation with respect to t and the second with respect to x,

$$\frac{\partial}{\partial t}\left(C\frac{\partial P}{\partial t}\right) = \frac{\partial}{\partial x}\left(\frac{A}{\varrho}\frac{\partial P}{\partial x}\right),$$

which can be approximated by

$$\frac{\partial^2 Q}{\partial t^2} = c^2 \frac{\partial^2 Q}{\partial x^2}, \quad c = \sqrt{\frac{A}{\varrho C}}, \tag{2.167}$$

where A and C in the expression for c are taken as constant reference values.

2.14.9 Electromagnetic waves

Light and radio waves are governed by standard wave equations arising from Maxwell's general equations. When there are no charges and no currents, as in a vacuum, Maxwell's equations take the form

$$\begin{aligned}
\nabla \cdot \boldsymbol{E} &= 0, \\
\nabla \cdot \boldsymbol{B} &= 0, \\
\nabla \times \boldsymbol{E} &= -\frac{\partial \boldsymbol{B}}{\partial t}, \\
\nabla \times \boldsymbol{B} &= \mu_0 \epsilon_0 \frac{\partial \boldsymbol{E}}{\partial t},
\end{aligned}$$

where $\epsilon_0 = 8.854187817620 \cdot 10^{-12}$ (F/m) is the permittivity of free space, also known as the electric constant, and $\mu_0 = 1.2566370614 \cdot 10^{-6}$ (H/m) is the permeability of free space, also known as the magnetic constant. Taking the curl of the two last equations and using the mathematical identity

$$\nabla \times (\nabla \times \boldsymbol{E}) = \nabla(\nabla \cdot \boldsymbol{E}) - \nabla^2 \boldsymbol{E} = -\nabla^2 \boldsymbol{E} \text{ when } \nabla \cdot \boldsymbol{E} = 0,$$

gives the wave equation governing the electric and magnetic field:

$$\frac{\partial^2 \boldsymbol{E}}{\partial t^2} = c^2 \nabla^2 \boldsymbol{E}, \tag{2.168}$$

$$\frac{\partial^2 \boldsymbol{B}}{\partial t^2} = c^2 \nabla^2 \boldsymbol{B}, \tag{2.169}$$

with $c = 1/\sqrt{\mu_0 \epsilon_0}$ as the velocity of light. Each component of $\boldsymbol{E}$ and $\boldsymbol{B}$ fulfills a wave equation and can hence be solved independently.

2.15 Exercises

Exercise 2.20: Simulate waves on a non-homogeneous string

Simulate waves on a string that consists of two materials with different density. The tension in the string is constant, but the density has a jump at the middle of the string. Experiment with different sizes of the jump and produce animations that visualize the effect of the jump on the wave motion.

Hint. According to Section 2.14.1, the density enters the mathematical model as ϱ in $\varrho u_{tt} = Tu_{xx}$, where T is the string tension. Modify, e.g., the `wave1D_u0v.py` code to incorporate the tension and two density values. Make a mesh function `rho` with density values at each spatial mesh point. A value for the tension may be 150 N. Corresponding density values can be computed from the wave velocity estimations in the `guitar` function in the `wave1D_u0v.py` file.
Filename: `wave1D_u0_sv_discont`.

Exercise 2.21: Simulate damped waves on a string

Formulate a mathematical model for damped waves on a string. Use data from Section 2.3.6, and tune the damping parameter so that the string is very close to the rest state after 15 s. Make a movie of the wave motion.
Filename: `wave1D_u0_sv_damping`.

Exercise 2.22: Simulate elastic waves in a rod

A hammer hits the end of an elastic rod. The exercise is to simulate the resulting wave motion using the model (2.128) from Section 2.14.2. Let the rod have length L and let the boundary $x = L$ be stress free so that $\sigma_{xx} = 0$, implying that $\partial u/\partial x = 0$. The left end $x = 0$ is subject to a strong stress pulse (the hammer), modeled as

$$\sigma_{xx}(t) = \begin{cases} S, 0 < t \leq t_s, \\ 0, \ t > t_s \end{cases}$$

The corresponding condition on u becomes $u_x = S/E$ for $t \leq t_s$ and zero afterwards (recall that $\sigma_{xx} = Eu_x$). This is a non-homogeneous Neumann condition, and you will need to approximate this condition and

combine it with the scheme (the ideas and manipulations follow closely the handling of a non-zero initial condition $u_t = V$ in wave PDEs or the corresponding second-order ODEs for vibrations). Filename: `wave_rod`.

Exercise 2.23: Simulate spherical waves

Implement a model for spherically symmetric waves using the method described in Section 2.14.6. The boundary condition at $r = 0$ must be $\partial u/\partial r = 0$, while the condition at $r = R$ can either be $u = 0$ or a radiation condition as described in Problem 2.12. The $u = 0$ condition is sufficient if R is so large that the amplitude of the spherical wave has become insignificant. Make movie(s) of the case where the source term is located around $r = 0$ and sends out pulses

$$f(r,t) = \begin{cases} Q\exp\left(-\frac{r^2}{2\Delta r^2}\right)\sin\omega t, & \sin\omega t \geq 0 \\ 0, & \sin\omega t < 0 \end{cases}$$

Here, Q and ω are constants to be chosen.

Hint. Use the program `wave1D_u0v.py` as a starting point. Let `solver` compute the v function and then set $u = v/r$. However, $u = v/r$ for $r = 0$ requires special treatment. One possibility is to compute `u[1:] = v[1:]/r[1:]` and then set `u[0]=u[1]`. The latter makes it evident that $\partial u/\partial r = 0$ in a plot.
Filename: `wave1D_spherical`.

Problem 2.24: Earthquake-generated tsunami over a subsea hill

A subsea earthquake leads to an immediate lift of the water surface, see Figure 2.11. The lifted water surface splits into two tsunamis, one traveling to the right and one to the left, as depicted in Figure 2.12. Since tsunamis are normally very long waves, compared to the depth, with a small amplitude, compared to the wave length, a standard wave equation is relevant:

$$\eta_{tt} = (gH(x)\eta_x)_x,$$

where η is the elevation of the water surface, g is the acceleration of gravity, and $H(x)$ is the still water depth.

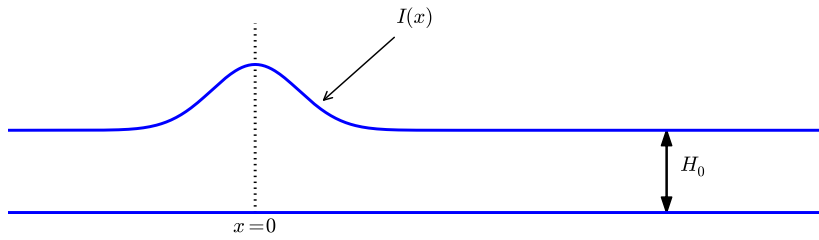


Fig. 2.11 Sketch of initial water surface due to a subsea earthquake.

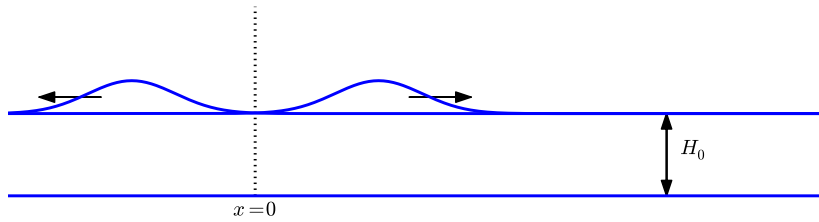


Fig. 2.12 An initial surface elevation is split into two waves.

To simulate the right-going tsunami, we can impose a symmetry boundary at $x = 0$: $\partial\eta/\partial x = 0$. We then simulate the wave motion in $[0, L]$. Unless the ocean ends at $x = L$, the waves should travel undisturbed through the boundary $x = L$. A radiation condition as explained in Problem 2.12 can be used for this purpose. Alternatively, one can just stop the simulations before the wave hits the boundary at $x = L$. In that case it does not matter what kind of boundary condition we use at $x = L$. Imposing $\eta = 0$ and stopping the simulations when $|\eta_i^n| > \epsilon$, $i = N_x - 1$, is a possibility (ϵ is a small parameter).

The shape of the initial surface can be taken as a Gaussian function,

$$I(x; I_0, I_a, I_m, I_s) = I_0 + I_a \exp\left(-\left(\frac{x - I_m}{I_s}\right)^2\right), \tag{2.170}$$

with $I_m = 0$ reflecting the location of the peak of $I(x)$ and I_s being a measure of the width of the function $I(x)$ (I_s is $\sqrt{2}$ times the standard deviation of the familiar normal distribution curve).

Now we extend the problem with a hill at the sea bottom, see Figure 2.13. The wave speed $c = \sqrt{gH(x)} = \sqrt{g(H_0 - B(x))}$ will then be reduced in the shallow water above the hill.

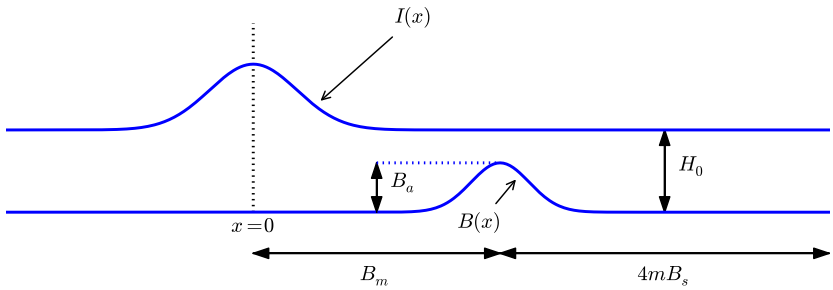


Fig. 2.13 Sketch of an earthquake-generated tsunami passing over a subsea hill.

One possible form of the hill is a Gaussian function,

$$B(x; B_0, B_a, B_m, B_s) = B_0 + B_a \exp\left(-\left(\frac{x - B_m}{B_s}\right)^2\right), \tag{2.171}$$

but many other shapes are also possible, e.g., a "cosine hat" where

$$B(x; B_0, B_a, B_m, B_s) = B_0 + B_a \cos\left(\pi \frac{x - B_m}{2B_s}\right), \tag{2.172}$$

when $x \in [B_m - B_s, B_m + B_s]$ while $B = B_0$ outside this interval.

Also an abrupt construction may be tried:

$$B(x; B_0, B_a, B_m, B_s) = B_0 + B_a, \tag{2.173}$$

for $x \in [B_m - B_s, B_m + B_s]$ while $B = B_0$ outside this interval.

The `wave1D_dn_vc.py` program can be used as starting point for the implementation. Visualize both the bottom topography and the water surface elevation in the same plot. Allow for a flexible choice of bottom shape: (2.171), (2.172), (2.173), or $B(x) = B_0$ (flat).

The purpose of this problem is to explore the quality of the numerical solution η_i^n for different shapes of the bottom obstruction. The "cosine hat" and the box-shaped hills have abrupt changes in the derivative of $H(x)$ and are more likely to generate numerical noise than the smooth Gaussian shape of the hill. Investigate if this is true. Filename: `tsunami1D_hill`.

Problem 2.25: Earthquake-generated tsunami over a 3D hill

This problem extends Problem 2.24 to a three-dimensional wave phenomenon, governed by the 2D PDE

$$\eta_{tt} = (gH\eta_x)_x + (gH\eta_y)_y = \nabla\cdot(gH\nabla\eta)\,. \tag{2.174}$$

We assume that the earthquake arises from a fault along the line $x = 0$ in the xy-plane so that the initial lift of the surface can be taken as $I(x)$ in Problem 2.24. That is, a plane wave is propagating to the right, but will experience bending because of the bottom.

The bottom shape is now a function of x and y. An "elliptic" Gaussian function in two dimensions, with its peak at (B_{mx}, B_{my}), generalizes (2.171):

$$B = B_0 + B_a\exp\left(-\left(\frac{x-B_{mx}}{B_s}\right)^2 - \left(\frac{y-B_{my}}{bB_s}\right)^2\right), \tag{2.175}$$

where b is a scaling parameter: $b = 1$ gives a circular Gaussian function with circular contour lines, while $b \neq 1$ gives an elliptic shape with elliptic contour lines. To indicate the input parameters in the model, we may write

$$B = B(x; B_0, B_a, B_{mx}, B_{my}, B_s, b)\,.$$

The "cosine hat" (2.172) can also be generalized to

$$B = B_0 + B_a\cos\left(\pi\frac{x-B_{mx}}{2B_s}\right)\cos\left(\pi\frac{y-B_{my}}{2B_s}\right), \tag{2.176}$$

when $0 \leq \sqrt{x^2+y^2} \leq B_s$ and $B = B_0$ outside this circle.

A box-shaped obstacle means that

$$B(x; B_0, B_a, B_m, B_s, b) = B_0 + B_a \tag{2.177}$$

for x and y inside a rectangle

$$B_{mx} - B_s \leq x \leq B_{mx} + B_s, \quad B_{my} - bB_s \leq y \leq B_{my} + bB_s,$$

and $B = B_0$ outside this rectangle. The b parameter controls the rectangular shape of the cross section of the box.

Note that the initial condition and the listed bottom shapes are symmetric around the line $y = B_{my}$. We therefore expect the surface elevation also to be symmetric with respect to this line. This means that we can halve the computational domain by working with $[0, L_x] \times [0, B_{my}]$. Along the upper boundary, $y = B_{my}$, we must impose the symmetry condition $\partial\eta/\partial n = 0$. Such a symmetry condition ($-\eta_x = 0$) is also needed at the $x = 0$ boundary because the initial condition has a symmetry here. At the lower boundary $y = 0$ we also set a Neumann condition (which becomes $-\eta_y = 0$). The wave motion is to be simulated until the wave hits the reflecting boundaries where $\partial\eta/\partial n = \eta_x = 0$ (one can also set $\eta = 0$ - the particular condition does not matter as long as the simulation is stopped before the wave is influenced by the boundary condition).

Visualize the surface elevation. Investigate how different hill shapes, different sizes of the water gap above the hill, and different resolutions $\Delta x = \Delta y = h$ and Δt influence the numerical quality of the solution. Filename: `tsunami2D_hill`.

Problem 2.26: Investigate Mayavi for visualization

Play with Mayavi code for visualizing 2D solutions of the wave equation with variable wave velocity. See if there are effective ways to visualize both the solution and the wave velocity scalar field at the same time. Filename: `tsunami2D_hill_mlab`.

Problem 2.27: Investigate visualization packages

Create some fancy 3D visualization of the water waves *and* the subsea hill in Problem 2.25. Try to make the hill transparent. Possible

visualization tools are Mayavi[14], Paraview[15], and OpenDX[16]. Filename: `tsunami2D_hill_viz`.

Problem 2.28: Implement loops in compiled languages

Extend the program from Problem 2.25 such that the loops over mesh points, inside the time loop, are implemented in compiled languages. Consider implementations in Cython, Fortran via `f2py`, C via Cython, C via `f2py`, C/C++ via Instant, and C/C++ via `scipy.weave`. Perform efficiency experiments to investigate the relative performance of the various implementations. It is often advantageous to normalize CPU times by the fastest method on a given mesh. Filename: `tsunami2D_hill_compiled`.

Exercise 2.29: Simulate seismic waves in 2D

The goal of this exercise is to simulate seismic waves using the PDE model (2.138) in a 2D xz domain with geological layers. Introduce m horizontal layers of thickness h_i, $i = 0, \ldots, m-1$. Inside layer number i we have a vertical wave velocity $c_{z,i}$ and a horizontal wave velocity $c_{h,i}$. Make a program for simulating such 2D waves. Test it on a case with 3 layers where

$$c_{z,0} = c_{z,1} = c_{z,2}, \quad c_{h,0} = c_{h,2}, \quad c_{h,1} \ll c_{h,0}\,.$$

Let s be a localized point source at the middle of the Earth's surface (the upper boundary) and investigate how the resulting wave travels through the medium. The source can be a localized Gaussian peak that oscillates in time for some time interval. Place the boundaries far enough from the expanding wave so that the boundary conditions do not disturb the wave. Then the type of boundary condition does not matter, except that we physically need to have $p = p_0$, where p_0 is the atmospheric pressure, at the upper boundary. Filename: `seismic2D`.

Project 2.30: Model 3D acoustic waves in a room

The equation for sound waves in air is derived in Section 2.14.5 and reads

[14] `http://code.enthought.com/projects/mayavi/`
[15] `http://www.paraview.org/`
[16] `http://www.opendx.org/`

$$p_{tt} = c^2 \nabla^2 p,$$

where $p(x, y, z, t)$ is the pressure and c is the speed of sound, taken as 340 m/s. However, sound is absorbed in the air due to relaxation of molecules in the gas. A model for simple relaxation, valid for gases consisting only of one type of molecules, is a term $c^2 \tau_s \nabla^2 p_t$ in the PDE, where τ_s is the relaxation time. If we generate sound from, e.g., a loudspeaker in the room, this sound source must also be added to the governing equation.

The PDE with the mentioned type of damping and source then becomes

$$p_t t = c^2 \nabla^p + c^2 \tau_s \nabla^2 p_t + f, \tag{2.178}$$

where $f(x, y, z, t)$ is the source term.

The walls can absorb some sound. A possible model is to have a "wall layer" (thicker than the physical wall) outside the room where c is changed such that some of the wave energy is reflected and some is absorbed in the wall. The absorption of energy can be taken care of by adding a damping term bp_t in the equation:

$$p_t t + bp_t = c^2 \nabla^p + c^2 \tau_s \nabla^2 p_t + f\,. \tag{2.179}$$

Typically, $b = 0$ in the room and $b > 0$ in the wall. A discontinuity in b or c will give rise to reflections. It can be wise to use a constant c in the wall to control reflections because of the discontinuity between c in the air and in the wall, while b is gradually increased as we go into the wall to avoid reflections because of rapid changes in b. At the outer boundary of the wall the condition $p = 0$ or $\partial p/\partial n = 0$ can be imposed. The waves should anyway be approximately dampened to $p = 0$ this far out in the wall layer.

There are two strategies for discretizing the $\nabla^2 p_t$ term: using a center difference between times $n + 1$ and $n - 1$ (if the equation is sampled at level n), or use a one-sided difference based on levels n and $n - 1$. The latter has the advantage of not leading to any equation system, while the former is second-order accurate as the scheme for the simple wave equation $p_t t = c^2 \nabla^2 p$. To avoid an equation system, go for the one-sided difference such that the overall scheme becomes explicit and only of first order in time.

Develop a 3D solver for the specified PDE and introduce a wall layer. Test the solver with the method of manufactured solutions. Make some demonstrations where the wall reflects and absorbs the waves (reflection

because of discontinuity in b and absorption because of growing b). Experiment with the impact of the τ_s parameter. Filename: `acoustics`.

Project 2.31: Solve a 1D transport equation

We shall study the wave equation

$$u_t + cu_x = 0, \quad x \in (0, L],\ t \in (0, T], \tag{2.180}$$

with initial condition

$$u(x, 0) = I(x), \quad x \in [0, L], \tag{2.181}$$

and *one* periodic boundary condition

$$u(0, t) = u(L, t)\,. \tag{2.182}$$

This boundary condition means that what goes out of the domain at $x = L$ comes in at $x = 0$. Roughly speaking, we need only one boundary condition because the spatial derivative is of first order only.

Physical interpretation. The parameter c can be constant or variable, $c = c(x)$. The equation (2.180) arises in *transport* problems where a quantity u, which could be temperature or concentration of some contaminant, is transported with the velocity c of a fluid. In addition to the transport imposed by "travelling with the fluid", u may also be transported by diffusion (such as heat conduction or Fickian diffusion), but we have in the model $u_t + cu_x$ assumed that diffusion effects are negligible, which they often are.

a) Show that under the assumption of $a = \text{const}$,

$$u(x, t) = I(x - ct) \tag{2.183}$$

fulfills the PDE as well as the initial and boundary condition (provided $I(0) = I(L)$).

A widely used numerical scheme for (2.180) applies a forward difference in time and a backward difference in space when $c > 0$:

$$[D_t^+ u + cD_x^- u = 0]_i^n\,. \tag{2.184}$$

For $c < 0$ we use a forward difference in space: $[cD_x^+ u]_i^n$.

b) Set up a computational algorithm and implement it in a function. Assume a is constant and positive.

c) Test the implementation by using the remarkable property that the numerical solution is exact at the mesh points if $\Delta t = c^{-1}\Delta x$.

d) Make a movie comparing the numerical and exact solution for the following two choices of initial conditions:

$$I(x) = \left[\sin\left(\pi\frac{x}{L}\right)\right]^{2n} \tag{2.185}$$

where n is an integer, typically $n = 5$, and

$$I(x) = \exp\left(-\frac{(x - L/2)^2}{2\sigma 2}\right). \tag{2.186}$$

Choose $\Delta t = c^{-1}\Delta x, 0.9c^{-1}\Delta x, 0.5c^{-1}\Delta x$.

e) The performance of the suggested numerical scheme can be investigated by analyzing the numerical dispersion relation. Analytically, we have that the *Fourier component*

$$u(x,t) = e^{i(kx-\omega t)},$$

is a solution of the PDE if $\omega = kc$. This is the *analytical dispersion relation*. A complete solution of the PDE can be built by adding up such Fourier components with different amplitudes, where the initial condition I determines the amplitudes. The solution u is then represented by a Fourier series.

A similar discrete Fourier component at (x_p, t_n) is

$$u_p^q = e^{i(kp\Delta x - \tilde{\omega} n\Delta t)},$$

where in general $\tilde{\omega}$ is a function of k, Δt, and Δx, and differs from the exact $\omega = kc$.

Insert the discrete Fourier component in the numerical scheme and derive an expression for $\tilde{\omega}$, i.e., the discrete dispersion relation. Show in particular that if $\Delta t/(c\Delta x) = 1$, the discrete solution coincides with the exact solution at the mesh points, regardless of the mesh resolution (!). Show that if the stability condition

$$\frac{\Delta t}{c\Delta x} \leq 1,$$

the discrete Fourier component cannot grow (i.e., $\tilde{\omega}$ is real).

f) Write a test for your implementation where you try to use information from the numerical dispersion relation.

We shall hereafter assume that $c(x) > 0$.

g) Set up a computational algorithm for the variable coefficient case and implement it in a function. Make a test that the function works for constant a.

h) It can be shown that for an observer moving with velocity $c(x)$, u is constant. This can be used to derive an exact solution when a varies with x. Show first that

$$u(x,t) = f(C(x) - t), \tag{2.187}$$

where

$$C'(x) = \frac{1}{c(x)},$$

is a solution of (2.180) for any differentiable function f.

i) Use the initial condition to show that an exact solution is

$$u(x,t) = I(C^{-1}(C(x) - t)),$$

with C^{-1} being the inverse function of $C = \int c^1 dx$. Since $C(x)$ is an integral $\int_0^x (1/c)dx$, $C(x)$ is monotonically increasing and there exists hence an inverse function C^{-1} with values in $[0, L]$.

To compute (2.187) we need to integrate $1/c$ to obtain C and then compute the inverse of C.

The inverse function computation can be easily done if we first think discretely. Say we have some function $y = g(x)$ and seek its inverse. Plotting (x_i, y_i), where $y_i = g(x_i)$ for some mesh points x_i, displays g as a function of x. The inverse function is simply x as a function of g, i.e., the curve with points (y_i, x_i). We can therefore quickly compute points at the curve of the inverse function. One way of extending these points to a continuous function is to assume a linear variation (known as linear interpolation) between the points (which actually means to draw straight lines between the points, exactly as done by a plotting program).

The function `wrap2callable` in `scitools.std` can take a set of points and return a continuous function that corresponds to linear variation between the points. The computation of the inverse of a function g on $[0, L]$ can then be done by

```
def inverse(g, domain, resolution=101):
    x = linspace(domain[0], domain[L], resolution)
    y = g(x)
```

```
    from scitools.std import wrap2callable
    g_inverse = wrap2callable((y, x))
    return g_inverse
```

To compute $C(x)$ we need to integrate $1/c$, which can be done by a Trapezoidal rule. Suppose we have computed $C(x_i)$ and need to compute $C(x_{i+1})$. Using the Trapezoidal rule with m subintervals over the integration domain $[x_i, x_{i+1}]$ gives

$$C(x_{i+1}) = C(x_i) + \int_{x_i}^{x_{i+1}} \frac{dx}{c} \approx h\left(\frac{1}{2}\frac{1}{c(x_i)} + \frac{1}{2}\frac{1}{c(x_{i+1})} + \sum_{j=1}^{m-1}\frac{1}{c(x_i+jh)}\right), \tag{2.188}$$

where $h = (x_{i+1} - x_i)/m$ is the length of the subintervals used for the integral over $[x_i, x_{i+1}]$. We observe that (2.188) is a *difference equation* which we can solve by repeatedly applying (2.188) for $i = 0, 1, \ldots, N_x - 1$ if a mesh $x_0, x_, \ldots, x_{N_x}$ is prescribed. Note that $C(0) = 0$.

j) Implement a function for computing $C(x_i)$ and one for computing $C^{-1}(x)$ for any x. Use these two functions for computing the exact solution $I(C^{-1}(C(x) - t))$. End up with a function `u_exact_variable_c(x, n, c, I)` that returns the value of $I(C^{-1}(C(x) - t_n))$.

k) Make movies showing a comparison of the numerical and exact solutions for the two initial conditions (2.185) and (2.31). Choose $\Delta t = \Delta x / \max_{0,L} c(x)$ and the velocity of the medium as

1. $c(x) = 1 + \epsilon \sin(k\pi x/L)$, $\epsilon < 1$,
2. $c(x) = 1 + I(x)$, where I is given by (2.185) or (2.31).

The PDE $u_t + cu_x = 0$ expresses that the initial condition $I(x)$ is transported with velocity $c(x)$.
Filename: `advec1D`.

Problem 2.32: General analytical solution of a 1D damped wave equation

We consider an initial-boundary value problem for the damped wave equation:

$$
\begin{aligned}
u_{tt} + bu_t &= c^2 u_{xx}, \quad x \in (0, L),\ t \in (0, T] \\
u(0, t) &= 0, \\
u(L, t) &= 0, \\
u(x, 0) &= I(x), \\
u_t(x, 0) &= V(x)\,.
\end{aligned}
$$

Here, $b \geq 0$ and c are given constants. The aim is to derive a general analytical solution of this problem. Familiarity with the method of separation of variables for solving PDEs will be assumed.

a) Seek a solution on the form $u(x, t) = X(x)T(t)$. Insert this solution in the PDE and show that it leads to two differential equations for X and T:

$$
T'' + bT' + \lambda T = 0, \quad c^2 X'' + \lambda X = 0,
$$

with $X(0) = X(L) = 0$ as boundary conditions, and λ as a constant to be determined.

b) Show that $X(x)$ is on the form

$$
X_n(x) = C_n \sin kx, \quad k = \frac{n\pi}{L}, \quad n = 1, 2, \ldots
$$

where C_n is an arbitrary constant.

c) Under the assumption that $(b/2)^2 < k^2$, show that $T(t)$ is on the form

$$
T_n(t) = e^{-\frac{1}{2}bt}(a_n \cos \omega t + b_n \sin \omega t), \quad \omega = \sqrt{k^2 - \frac{1}{4}b^2}, \quad n = 1, 2, \ldots
$$

The complete solution is then

$$
u(x, t) = \sum_{n=1}^{\infty} \sin kx e^{-\frac{1}{2}bt}(A_n \cos \omega t + B_n \sin \omega t),
$$

where the constants A_n and B_n must be computed from the initial conditions.

d) Derive a formula for A_n from $u(x, 0) = I(x)$ and developing $I(x)$ as a sine Fourier series on $[0, L]$.

e) Derive a formula for B_n from $u_t(x, 0) = V(x)$ and developing $V(x)$ as a sine Fourier series on $[0, L]$.

f) Calculate A_n and B_n from vibrations of a string where $V(x) = 0$ and

$$I(x) = \begin{cases} ax/x_0, & x < x_0, \\ a(L-x)/(L-x_0), & \text{otherwise} \end{cases} \tag{2.189}$$

g) Implement a function `u_series(x, t, tol=1E-10)` for the series for $u(x,t)$, where `tol` is a tolerance for truncating the series. Simply sum the terms until $|a_n|$ and $|b_b|$ both are less than `tol`.

h) What will change in the derivation of the analytical solution if we have $u_x(0,t) = u_x(L,t) = 0$ as boundary conditions? And how will you solve the problem with $u(0,t) = 0$ and $u_x(L,t) = 0$?
Filename: `damped_wave1D`.

Problem 2.33: General analytical solution of a 2D damped wave equation

Carry out Problem 2.32 in the 2D case: $u_{tt} + bu_t = c^2(u_{xx} + u_{yy})$, where $(x,y) \in (0,L_x) \times (0,L_y)$. Assume a solution on the form $u(x,y,t) = X(x)Y(y)T(t)$. Filename: `damped_wave2D`.

Diffusion equations

3

The famous *diffusion equation*, also known as the *heat equation*, reads

$$\frac{\partial u}{\partial t} = \alpha \frac{\partial^2 u}{\partial x^2},$$

where $u(x,t)$ is the unknown function to be solved for, x is a coordinate in space, and t is time. The coefficient α is the *diffusion coefficient* and determines how fast u changes in time. A quick short form for the diffusion equation is $u_t = \alpha u_{xx}$.

Compared to the wave equation, $u_{tt} = c^2 u_{xx}$, which looks very similar, the diffusion equation features solutions that are very different from those of the wave equation. Also, the diffusion equation makes quite different demands to the numerical methods.

Typical diffusion problems may experience rapid change in the very beginning, but then the evolution of u becomes slower and slower. The solution is usually very smooth, and after some time, one cannot recognize the initial shape of u. This is in sharp contrast to solutions of the wave equation where the initial shape is preserved in homogeneous media – the solution is then basically a moving initial condition. The standard wave equation $u_{tt} = c^2 u_{xx}$ has solutions that propagate with speed c forever, without changing shape, while the diffusion equation converges to a *stationary solution* $\bar{u}(x)$ as $t \to \infty$. In this limit, $u_t = 0$, and $\bar{u}$ is governed by $\bar{u}''(x) = 0$. This stationary limit of the diffusion equation is called the *Laplace* equation and arises in a very wide range of applications throughout the sciences.

© 2017, Hans Petter Langtangen, Svein Linge. Released under CC Attribution 4.0 license

It is possible to solve for $u(x,t)$ using an explicit scheme, as we do in Section 3.1, but the time step restrictions soon become much less favorable than for an explicit scheme applied to the wave equation. And of more importance, since the solution u of the diffusion equation is very smooth and changes slowly, small time steps are not convenient and not required by accuracy as the diffusion process converges to a stationary state. Therefore, implicit schemes (as described in Section 3.2) are popular, but these require solutions of systems of algebraic equations. We shall use ready-made software for this purpose, but also program some simple iterative methods. The exposition is, as usual in this book, very basic and focuses on the basic ideas and how to implement. More comprehensive mathematical treatments and classical analysis of the methods are found in lots of textbooks. A favorite of ours in this respect is the one by LeVeque [13]. The books by Strikwerda [17] and by Lapidus and Pinder [12] are also highly recommended as additional material on the topic.

3.1 An explicit method for the 1D diffusion equation

Explicit finite difference methods for the wave equation $u_{tt} = c^2 u_{xx}$ can be used, with small modifications, for solving $u_t = \alpha u_{xx}$ as well. The exposition below assumes that the reader is familiar with the basic ideas of discretization and implementation of wave equations from Chapter 2. Readers not familiar with the Forward Euler, Backward Euler, and Crank-Nicolson (or centered or midpoint) discretization methods in time should consult, e.g., Section 1.1 in [9].

3.1.1 The initial-boundary value problem for 1D diffusion

To obtain a unique solution of the diffusion equation, or equivalently, to apply numerical methods, we need initial and boundary conditions. The diffusion equation goes with one initial condition $u(x,0) = I(x)$, where I is a prescribed function. One boundary condition is required at each point on the boundary, which in 1D means that u must be known, u_x must be known, or some combination of them.

We shall start with the simplest boundary condition: $u = 0$. The complete initial-boundary value diffusion problem in one space dimension can then be specified as

$$\frac{\partial u}{\partial t} = \alpha \frac{\partial^2 u}{\partial x^2} + f, \quad x \in (0, L),\ t \in (0, T] \tag{3.1}$$

$$u(x, 0) = I(x), \qquad x \in [0, L] \tag{3.2}$$

$$u(0, t) = 0, \qquad t > 0, \tag{3.3}$$

$$u(L, t) = 0, \qquad t > 0\,. \tag{3.4}$$

With only a first-order derivative in time, only one *initial condition* is needed, while the second-order derivative in space leads to a demand for two *boundary conditions*. We have added a source term $f = f(x, t)$, which is convenient when testing implementations.

Diffusion equations like (3.1) have a wide range of applications throughout physical, biological, and financial sciences. One of the most common applications is propagation of heat, where $u(x, t)$ represents the temperature of some substance at point x and time t. Other applications are listed in Section 3.8.

3.1.2 Forward Euler scheme

The first step in the discretization procedure is to replace the domain $[0, L] \times [0, T]$ by a set of mesh points. Here we apply equally spaced mesh points

$$x_i = i\Delta x, \quad i = 0, \ldots, N_x,$$

and

$$t_n = n\Delta t, \quad n = 0, \ldots, N_t\,.$$

Moreover, u_i^n denotes the mesh function that approximates $u(x_i, t_n)$ for $i = 0, \ldots, N_x$ and $n = 0, \ldots, N_t$. Requiring the PDE (3.1) to be fulfilled at a mesh point (x_i, t_n) leads to the equation

$$\frac{\partial}{\partial t} u(x_i, t_n) = \alpha \frac{\partial^2}{\partial x^2} u(x_i, t_n) + f(x_i, t_n), \tag{3.5}$$

The next step is to replace the derivatives by finite difference approximations. The computationally simplest method arises from using a forward difference in time and a central difference in space:

$$[D_t^+ u = \alpha D_x D_x u + f]_i^n\,. \tag{3.6}$$

Written out,

$$\frac{u_i^{n+1} - u_i^n}{\Delta t} = \alpha \frac{u_{i+1}^n - 2u_i^n + u_{i-1}^n}{\Delta x^2} + f_i^n \,. \tag{3.7}$$

We have turned the PDE into algebraic equations, also often called discrete equations. The key property of the equations is that they are algebraic, which makes them easy to solve. As usual, we anticipate that u_i^n is already computed such that u_i^{n+1} is the only unknown in (3.7). Solving with respect to this unknown is easy:

$$u_i^{n+1} = u_i^n + F\left(u_{i+1}^n - 2u_i^n + u_{i-1}^n\right) + \Delta t f_i^n, \tag{3.8}$$

where we have introduced the *mesh Fourier number*:

$$F = \alpha \frac{\Delta t}{\Delta x^2} \,. \tag{3.9}$$

F is the key parameter in the discrete diffusion equation

Note that F is a *dimensionless* number that lumps the key physical parameter in the problem, α, and the discretization parameters Δx and Δt into a single parameter. Properties of the numerical method are critically dependent upon the value of F (see Section 3.3 for details).

The computational algorithm then becomes

1. compute $u_i^0 = I(x_i)$ for $i = 0, \ldots, N_x$
2. for $n = 0, 1, \ldots, N_t$:
   a. apply (3.8) for all the internal spatial points $i = 1, \ldots, N_x - 1$
   b. set the boundary values $u_i^{n+1} = 0$ for $i = 0$ and $i = N_x$

The algorithm is compactly and fully specified in Python:

```
import numpy as np
x = np.linspace(0, L, Nx+1)     # mesh points in space
dx = x[1] - x[0]
t = np.linspace(0, T, Nt+1)     # mesh points in time
dt = t[1] - t[0]
F = a*dt/dx**2
u   = np.zeros(Nx+1)            # unknown u at new time level
u_n = np.zeros(Nx+1)            # u at the previous time level
```

```
# Set initial condition u(x,0) = I(x)
for i in range(0, Nx+1):
    u_n[i] = I(x[i])

for n in range(0, Nt):
    # Compute u at inner mesh points
    for i in range(1, Nx):
        u[i] = u_n[i] + F*(u_n[i-1] - 2*u_n[i] + u_n[i+1]) + \
           dt*f(x[i], t[n])

    # Insert boundary conditions
    u[0] = 0;  u[Nx] = 0

    # Update u_n before next step
    u_n[:]= u
```

Note that we use `a` for α in the code, motivated by easy visual mapping between the variable name and the mathematical symbol in formulas.

We need to state already now that the shown algorithm does not produce meaningful results unless $F \leq 1/2$. Why is explained in Section 3.3.

3.1.3 Implementation

The file `diffu1D_u0.py` contains a complete function `solver_FE_simple` for solving the 1D diffusion equation with $u = 0$ on the boundary as specified in the algorithm above:

```
import numpy as np

def solver_FE_simple(I, a, f, L, dt, F, T):
    """
    Simplest expression of the computational algorithm
    using the Forward Euler method and explicit Python loops.
    For this method F <= 0.5 for stability.
    """
    import time;  t0 = time.clock()  # For measuring the CPU time

    Nt = int(round(T/float(dt)))
    t = np.linspace(0, Nt*dt, Nt+1)   # Mesh points in time
    dx = np.sqrt(a*dt/F)
    Nx = int(round(L/dx))
    x = np.linspace(0, L, Nx+1)       # Mesh points in space
    # Make sure dx and dt are compatible with x and t
    dx = x[1] - x[0]
    dt = t[1] - t[0]

    u   = np.zeros(Nx+1)
    u_n = np.zeros(Nx+1)

    # Set initial condition u(x,0) = I(x)
```

```
    for i in range(0, Nx+1):
        u_n[i] = I(x[i])

    for n in range(0, Nt):
        # Compute u at inner mesh points
        for i in range(1, Nx):
            u[i] = u_n[i] + F*(u_n[i-1] - 2*u_n[i] + u_n[i+1]) + \
                   dt*f(x[i], t[n])

        # Insert boundary conditions
        u[0] = 0;  u[Nx] = 0

        # Switch variables before next step
        #u_n[:] = u  # safe, but slow
        u_n, u = u, u_n

    t1 = time.clock()
    return u_n, x, t, t1-t0  # u_n holds latest u
```

A faster alternative is available in the function `solver_FE`, which adds the possibility of solving the finite difference scheme by vectorization. The vectorized version replaces the explicit loop

```
for i in range(1, Nx):
    u[i] = u_n[i] + F*(u_n[i-1] - 2*u_n[i] + u_n[i+1]) \
           + dt*f(x[i], t[n])
```

by arithmetics on displaced slices of the `u` array:

```
u[1:Nx] = u_n[1:Nx] + F*(u_n[0:Nx-1] - 2*u_n[1:Nx] + u_n[2:Nx+1]) \
          + dt*f(x[1:Nx], t[n])
# or
u[1:-1] = u_n[1:-1] + F*(u_n[0:-2] - 2*u_n[1:-1] + u_n[2:]) \
          + dt*f(x[1:-1], t[n])
```

For example, the vectorized version runs 70 times faster than the scalar version in a case with 100 time steps and a spatial mesh of 10^5 cells.

The `solver_FE` function also features a callback function such that the user can process the solution at each time level. The callback function looks like `user_action(u, x, t, n)`, where `u` is the array containing the solution at time level `n`, `x` holds all the spatial mesh points, while `t` holds all the temporal mesh points. The `solver_FE` function is very similar to `solver_FE_simple` above:

```
def solver_FE(I, a, f, L, dt, F, T,
              user_action=None, version='scalar'):
    """
    Vectorized implementation of solver_FE_simple.
    """
    import time;  t0 = time.clock()  # for measuring the CPU time
```

```
    Nt = int(round(T/float(dt)))
    t = np.linspace(0, Nt*dt, Nt+1)   # Mesh points in time
    dx = np.sqrt(a*dt/F)
    Nx = int(round(L/dx))
    x = np.linspace(0, L, Nx+1)       # Mesh points in space
    # Make sure dx and dt are compatible with x and t
    dx = x[1] - x[0]
    dt = t[1] - t[0]

    u   = np.zeros(Nx+1)   # solution array
    u_n = np.zeros(Nx+1)   # solution at t-dt

    # Set initial condition
    for i in range(0,Nx+1):
        u_n[i] = I(x[i])

    if user_action is not None:
        user_action(u_n, x, t, 0)

    for n in range(0, Nt):
        # Update all inner points
        if version == 'scalar':
            for i in range(1, Nx):
                u[i] = u_n[i] +\
                       F*(u_n[i-1] - 2*u_n[i] + u_n[i+1]) +\
                       dt*f(x[i], t[n])

        elif version == 'vectorized':
            u[1:Nx] = u_n[1:Nx] +  \
                      F*(u_n[0:Nx-1] - 2*u_n[1:Nx] + u_n[2:Nx+1]) +\
                      dt*f(x[1:Nx], t[n])
        else:
            raise ValueError('version=%s' % version)

        # Insert boundary conditions
        u[0] = 0;  u[Nx] = 0
        if user_action is not None:
            user_action(u, x, t, n+1)

        # Switch variables before next step
        u_n, u = u, u_n

    t1 = time.clock()
    return t1-t0
```

3.1.4 Verification

Exact solution of discrete equations. Before thinking about running the functions in the previous section, we need to construct a suitable test example for verification. It appears that a manufactured solution that is

linear in time and at most quadratic in space fulfills the Forward Euler scheme exactly. With the restriction that $u = 0$ for $x = 0, L$, we can try the solution

$$u(x,t) = 5tx(L-x)\,.$$

Inserted in the PDE, it requires a source term

$$f(x,t) = 10\alpha t + 5x(L-x)\,.$$

With the formulas from Appendix A.4 we can easily check that the manufactured `u` fulfills the scheme:

$$\begin{aligned}[D_t^+ u = \alpha D_x D_x u + f]_i^n &= [5x(L-x)D_t^+ t = 5t\alpha D_x D_x(xL - x^2) + \\ &\qquad 10\alpha t + 5x(L-x)]_i^n \\ &= [5x(L-x) = 5t\alpha(-2) + 10\alpha t + 5x(L-x)]_i^n,\end{aligned}$$

which is a 0=0 expression. The computation of the source term, given any u, is easily automated with `sympy`:

```
import sympy as sym
x, t, a, L = sym.symbols('x t a L')
u = x*(L-x)*5*t

def pde(u):
    return sym.diff(u, t) - a*sym.diff(u, x, x)

f = sym.simplify(pde(u))
```

Now we can choose any expression for `u` and automatically get the suitable source term `f`. However, the manufactured solution `u` will in general not be exactly reproduced by the scheme: only constant and linear functions are differentiated correctly by a forward difference, while only constant, linear, and quadratic functions are differentiated exactly by a $[D_x D_x u]_i^n$ difference.

The numerical code will need to access the `u` and `f` above as Python functions. The exact solution is wanted as a Python function `u_exact(x, t)`, while the source term is wanted as `f(x, t)`. The parameters `a` and `L` in `u` and `f` above are symbols and must be replaced by `float` objects in a Python function. This can be done by redefining `a` and `L` as `float` objects and performing substitutions of symbols by numbers in `u` and `f`. The appropriate code looks like this:

```
a = 0.5
```

```
L = 1.5
u_exact = sym.lambdify(
    [x, t], u.subs('L', L).subs('a', a), modules='numpy')
f = sym.lambdify(
    [x, t], f.subs('L', L).subs('a', a), modules='numpy')
I = lambda x: u_exact(x, 0)
```

Here we also make a function `I` for the initial condition.

The idea now is that our manufactured solution should be exactly reproduced by the code (to machine precision). For this purpose we make a test function for comparing the exact and numerical solutions at the end of the time interval:

```
def test_solver_FE():
    # Define u_exact, f, I as explained above

    dx = L/3  # 3 cells
    F = 0.5
    dt = F*dx**2

    u, x, t, cpu = solver_FE_simple(
        I=I, a=a, f=f, L=L, dt=dt, F=F, T=2)
    u_e = u_exact(x, t[-1])
    diff = abs(u_e - u).max()
    tol = 1E-14
    assert diff < tol, 'max diff solver_FE_simple: %g' % diff

    u, x, t, cpu = solver_FE(
        I=I, a=a, f=f, L=L, dt=dt, F=F, T=2,
        user_action=None, version='scalar')
    u_e = u_exact(x, t[-1])
    diff = abs(u_e - u).max()
    tol = 1E-14
    assert diff < tol, 'max diff solver_FE, scalar: %g' % diff

    u, x, t, cpu = solver_FE(
        I=I, a=a, f=f, L=L, dt=dt, F=F, T=2,
        user_action=None, version='vectorized')
    u_e = u_exact(x, t[-1])
    diff = abs(u_e - u).max()
    tol = 1E-14
    assert diff < tol, 'max diff solver_FE, vectorized: %g' % diff
```

The critical value $F = 0.5$

We emphasize that the value `F=0.5` is critical: the tests above will fail if `F` has a larger value. This is because the Forward Euler scheme is unstable for $F > 1/2$.

The reader may wonder if $F = 1/2$ is safe or if $F < 1/2$ should be required. Experiments show that $F = 1/2$ works fine for $u_t = \alpha u_{xx}$, so there is no accumulation of rounding errors in this case and hence no need to introduce any safety factor to keep F away from the limiting value 0.5.

Checking convergence rates. If our chosen exact solution does not satisfy the discrete equations exactly, we are left with checking the convergence rates, just as we did previously for the wave equation. However, with the Euler scheme here, we have different accuracies in time and space, since we use a second order approximation to the spatial derivative and a first order approximation to the time derivative. Thus, we must expect different convergence rates in time and space. For the numerical error,

$$E = C_t \Delta t^r + C_x \Delta x^p,$$

we should get convergence rates $r = 1$ and $p = 2$ (C_t and C_x are unknown constants). As previously, in Section 2.2.3, we simplify matters by introducing a single discretization parameter h:

$$h = \Delta t, \quad \Delta x = K h^{r/p},$$

where K is any constant. This allows us to factor out only *one* discretization parameter h from the formula:

$$E = C_t h + C_x (K h^{r/p})^p = \tilde{C} h^r, \quad \tilde{C} = C_t + C_s K^r \,.$$

The computed rate r should approach 1 with increasing resolution.

It is tempting, for simplicity, to choose $K = 1$, which gives $\Delta x = h^{r/p}$, expected to be $\sqrt{\Delta t}$. However, we have to control the stability requirement: $F \leq \frac{1}{2}$, which means

$$\frac{\alpha \Delta t}{\Delta x^2} \leq \frac{1}{2} \quad \Rightarrow \quad \Delta x \geq \sqrt{2\alpha} h^{1/2},$$

implying that $K = \sqrt{2\alpha}$ is our choice in experiments where we lie on the stability limit $F = 1/2$.

3.1.5 Numerical experiments

When a test function like the one above runs silently without errors, we have some evidence for a correct implementation of the numerical method. The next step is to do some experiments with more interesting solutions.

We target a scaled diffusion problem where x/L is a new spatial coordinate and $\alpha t/L^2$ is a new time coordinate. The source term f is omitted, and u is scaled by $\max_{x\in[0,L]}|I(x)|$ (see Section 3.2 in [11] for details). The governing PDE is then

$$\frac{\partial u}{\partial t} = \frac{\partial^2 u}{\partial x^2},$$

in the spatial domain $[0, L]$, with boundary conditions $u(0) = u(1) = 0$. Two initial conditions will be tested: a discontinuous plug,

$$I(x) = \begin{cases} 0, & |x - L/2| > 0.1 \\ 1, & \text{otherwise} \end{cases}$$

and a smooth Gaussian function,

$$I(x) = e^{-\frac{1}{2\sigma^2}(x-L/2)^2}.$$

The functions `plug` and `gaussian` in `diffu1D_u0.py` run the two cases, respectively:

```
def plug(scheme='FE', F=0.5, Nx=50):
    L = 1.
    a = 1.
    T = 0.1
    # Compute dt from Nx and F
    dx = L/Nx;  dt = F/a*dx**2

    def I(x):
        """Plug profile as initial condition."""
        if abs(x-L/2.0) > 0.1:
            return 0
        else:
            return 1

    cpu = viz(I, a, L, dt, F, T,
              umin=-0.1, umax=1.1,
              scheme=scheme, animate=True, framefiles=True)
    print 'CPU time:', cpu

def gaussian(scheme='FE', F=0.5, Nx=50, sigma=0.05):
    L = 1.
    a = 1.
```

```
    T = 0.1
    # Compute dt from Nx and F
    dx = L/Nx;  dt = F/a*dx**2

    def I(x):
        """Gaussian profile as initial condition."""
        return exp(-0.5*((x-L/2.0)**2)/sigma**2)

    u, cpu = viz(I, a, L, dt, F, T,
                 umin=-0.1, umax=1.1,
                 scheme=scheme, animate=True, framefiles=True)
    print 'CPU time:', cpu
```

These functions make use of the function `viz` for running the solver and visualizing the solution using a callback function with plotting:

```
def viz(I, a, L, dt, F, T, umin, umax,
        scheme='FE', animate=True, framefiles=True):

    def plot_u(u, x, t, n):
        plt.plot(x, u, 'r-', axis=[0, L, umin, umax],
                 title='t=%f' % t[n])
        if framefiles:
            plt.savefig('tmp_frame%04d.png' % n)
        if t[n] == 0:
            time.sleep(2)
        elif not framefiles:
            # It takes time to write files so pause is needed
            # for screen only animation
            time.sleep(0.2)

    user_action = plot_u if animate else lambda u,x,t,n: None

    cpu = eval('solver_'+scheme)(I, a, L, dt, F, T,
                                 user_action=user_action)
    return cpu
```

Notice that this `viz` function stores all the solutions in a list `solutions` in the callback function. Modern computers have hardly any problem with storing a lot of such solutions for moderate values of N_x in 1D problems, but for 2D and 3D problems, this technique cannot be used and solutions must be stored in files.

Our experiments employ a time step $\Delta t = 0.0002$ and simulate for $t \in [0, 0.1]$. First we try the highest value of F: $F = 0.5$. This resolution corresponds to $N_x = 50$. A possible terminal command is

Terminal

```
Terminal> python -c 'from diffu1D_u0 import gaussian
          gaussian("solver_FE", F=0.5, dt=0.0002)'
```

The $u(x,t)$ curve as a function of x is shown in Figure 3.1 at four time levels.

Movie 3: `https://raw.githubusercontent.com/hplgit/fdm-book/master/doc/pub/book/html/mov-diffu/diffu1D_u0_FE_plug/movie.ogg`

We see that the curves have saw-tooth waves in the beginning of the simulation. This non-physical noise is smoothed out with time, but solutions of the diffusion equations are known to be smooth, and this numerical solution is definitely not smooth. Lowering F helps: $F \leq 0.25$ gives a smooth solution, see Figure 3.2 (and a movie[1]).

Increasing F slightly beyond the limit 0.5, to $F = 0.51$, gives growing, non-physical instabilities, as seen in Figure 3.3.

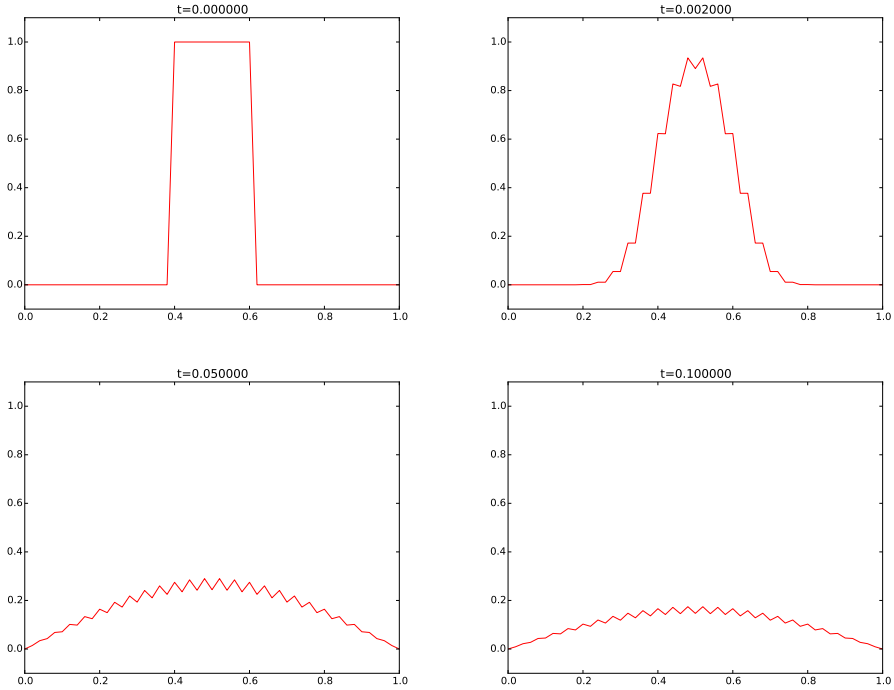


Fig. 3.1 Forward Euler scheme for $F = 0.5$.

Instead of a discontinuous initial condition we now try the smooth Gaussian function for $I(x)$. A simulation for $F = 0.5$ is shown in Figure 3.4. Now the numerical solution is smooth for all times, and this is true for any $F \leq 0.5$.

[1] `http://tinyurl.com/gokgkov/mov-diffu/diffu1D_u0_FE_plug_F025/movie.ogg`

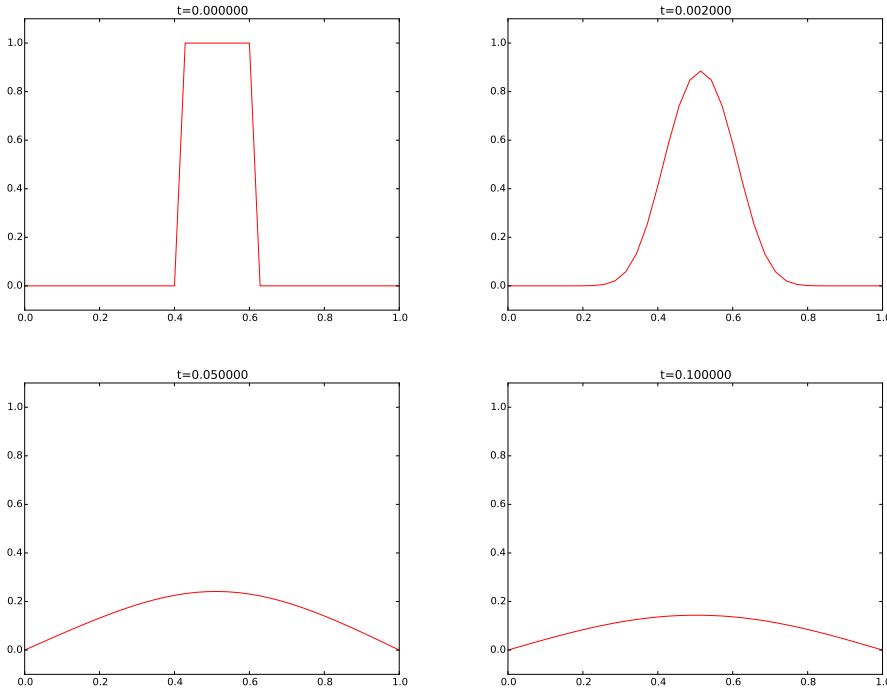


Fig. 3.2 Forward Euler scheme for $F = 0.25$.

Experiments with these two choices of $I(x)$ reveal some important observations:

- The Forward Euler scheme leads to growing solutions if $F > \frac{1}{2}$.
- $I(x)$ as a discontinuous plug leads to a saw tooth-like noise for $F = \frac{1}{2}$, which is absent for $F \leq \frac{1}{4}$.
- The smooth Gaussian initial function leads to a smooth solution for all relevant F values ($F \leq \frac{1}{2}$).

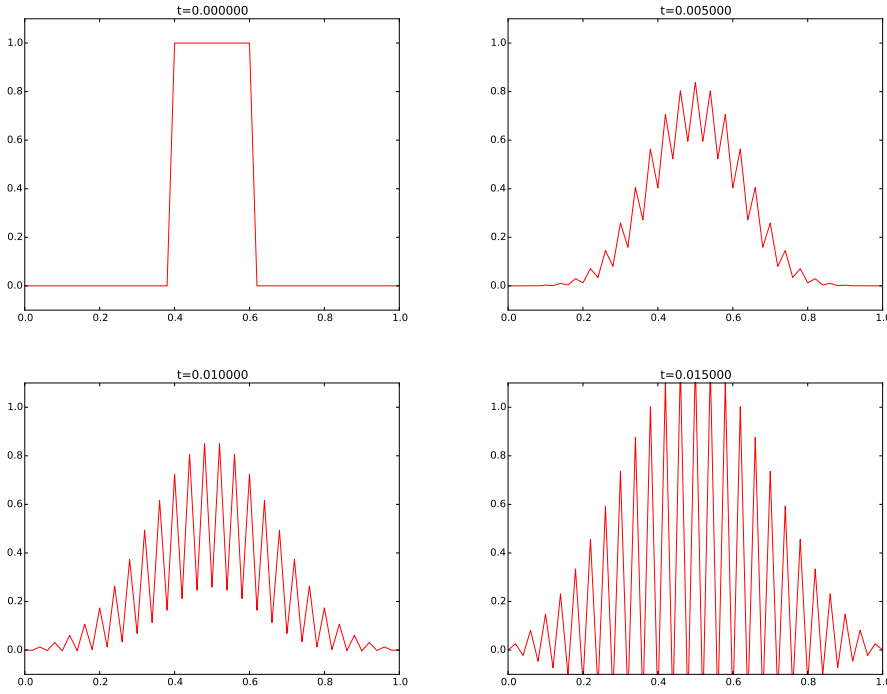


Fig. 3.3 Forward Euler scheme for $F = 0.51$.

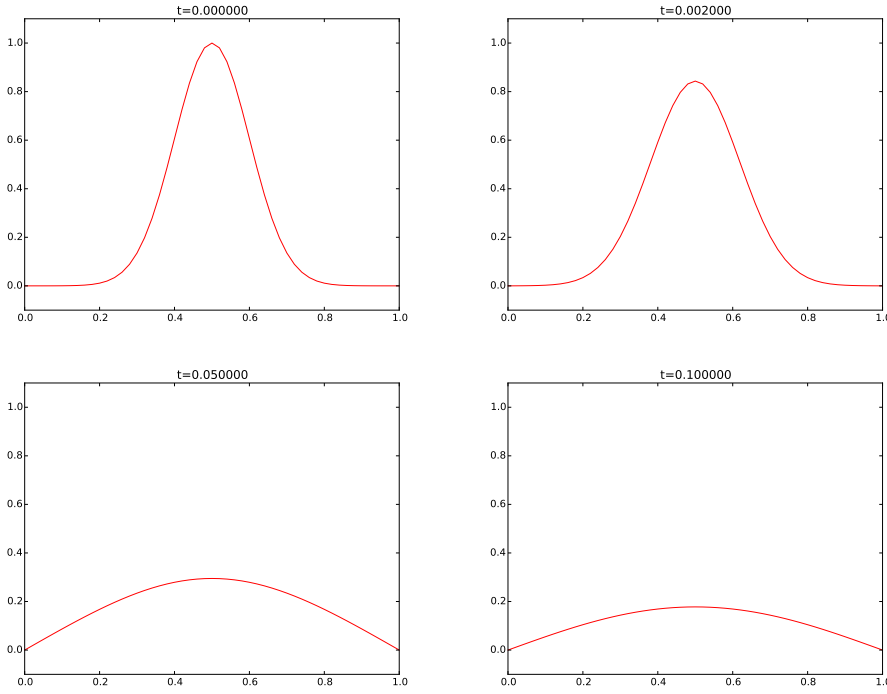


Fig. 3.4 Forward Euler scheme for $F = 0.5$.

3.2 Implicit methods for the 1D diffusion equation

Simulations with the Forward Euler scheme show that the time step restriction, $F \leq \frac{1}{2}$, which means $\Delta t \leq \Delta x^2/(2\alpha)$, may be relevant in the beginning of the diffusion process, when the solution changes quite fast, but as time increases, the process slows down, and a small Δt may be inconvenient. With *implicit schemes*, which lead to coupled systems of linear equations to be solved at each time level, any size of Δt is possible (but the accuracy decreases with increasing Δt). The Backward Euler scheme, derived and implemented below, is the simplest implicit scheme for the diffusion equation.

3.2.1 Backward Euler scheme

In (3.5), we now apply a backward difference in time, but the same central difference in space:

$$[D_t^- u = D_x D_x u + f]_i^n, \tag{3.10}$$

which written out reads

$$\frac{u_i^n - u_i^{n-1}}{\Delta t} = \alpha \frac{u_{i+1}^n - 2u_i^n + u_{i-1}^n}{\Delta x^2} + f_i^n \,. \tag{3.11}$$

Now we assume u_i^{n-1} is already computed, but that all quantities at the "new" time level n are unknown. This time it is not possible to solve with respect to u_i^n because this value couples to its neighbors in space, u_{i-1}^n and u_{i+1}^n, which are also unknown. Let us examine this fact for the case when $N_x = 3$. Equation (3.11) written for $i = 1, \ldots, Nx - 1 = 1, 2$ becomes

$$\frac{u_1^n - u_1^{n-1}}{\Delta t} = \alpha \frac{u_2^n - 2u_1^n + u_0^n}{\Delta x^2} + f_1^n \tag{3.12}$$

$$\frac{u_2^n - u_2^{n-1}}{\Delta t} = \alpha \frac{u_3^n - 2u_2^n + u_1^n}{\Delta x^2} + f_2^n \tag{3.13}$$

The boundary values u_0^n and u_3^n are known as zero. Collecting the unknown new values u_1^n and u_2^n on the left-hand side and multiplying by Δt gives

$$(1+2F)\,u_1^n - Fu_2^n = u_1^{n-1} + \Delta t f_1^n, \tag{3.14}$$
$$-Fu_1^n + (1+2F)\,u_2^n = u_2^{n-1} + \Delta t f_2^n\,. \tag{3.15}$$

This is a coupled 2×2 system of algebraic equations for the unknowns u_1^n and u_2^n. The equivalent matrix form is

$$\begin{pmatrix} 1+2F & -F \\ -F & 1+2F \end{pmatrix} \begin{pmatrix} u_1^n \\ u_2^n \end{pmatrix} = \begin{pmatrix} u_1^{n-1} + \Delta t f_1^n \\ u_2^{n-1} + \Delta t f_2^n \end{pmatrix}$$

Terminology: implicit vs. explicit methods

Discretization methods that lead to a coupled system of equations for the unknown function at a new time level are said to be *implicit methods*. The counterpart, *explicit methods*, refers to discretization methods where there is a simple explicit formula for the values of the unknown function at each of the spatial mesh points at the new time level. From an implementational point of view, implicit methods are more comprehensive to code since they require the solution of coupled equations, i.e., a matrix system, at each time level. With explicit methods we have a closed-form formula for the value of the unknown at each mesh point.

Very often explicit schemes have a restriction on the size of the time step that can be relaxed by using implicit schemes. In fact, implicit schemes are frequently unconditionally stable, so the size of the time step is governed by accuracy and not by stability. This is the great advantage of implicit schemes.

In the general case, (3.11) gives rise to a coupled $(N_x-1) \times (N_x-1)$ system of algebraic equations for all the unknown u_i^n at the interior spatial points $i = 1, \ldots, N_x - 1$. Collecting the unknowns on the left-hand side, (3.11) can be written

$$-Fu_{i-1}^n + (1+2F)\,u_i^n - Fu_{i+1}^n = u_{i-1}^{n-1}, \tag{3.16}$$

for $i = 1, \ldots, N_x - 1$. One can either view these equations as a system where the u_i^n values at the internal mesh points, $i = 1, \ldots, N_x - 1$, are unknown, or we may append the boundary values u_0^n and $u_{N_x}^n$ to the system. In the latter case, all u_i^n for $i = 0, \ldots, N_x$ are considered unknown, and we must add the boundary equations to the $N_x - 1$ equations in (3.16):

$$u_0^n = 0, \tag{3.17}$$
$$u_{N_x}^n = 0\,. \tag{3.18}$$

A coupled system of algebraic equations can be written on matrix form, and this is important if we want to call up ready-made software for solving the system. The equations (3.16) and (3.17)–(3.18) correspond to the matrix equation

$$AU = b$$

where $U = (u_0^n, \ldots, u_{N_x}^n)$, and the matrix A has the following structure:

$$A = \begin{pmatrix} A_{0,0} & A_{0,1} & 0 & \cdots & \cdots & \cdots & \cdots & \cdots & 0 \\ A_{1,0} & A_{1,1} & A_{1,2} & \ddots & & & & & \vdots \\ 0 & A_{2,1} & A_{2,2} & A_{2,3} & \ddots & & & & \vdots \\ \vdots & \ddots & & \ddots & \ddots & 0 & & & \vdots \\ \vdots & & \ddots & \ddots & \ddots & \ddots & \ddots & & \vdots \\ \vdots & & & 0 & A_{i,i-1} & A_{i,i} & A_{i,i+1} & \ddots & \vdots \\ \vdots & & & & \ddots & \ddots & \ddots & \ddots & 0 \\ \vdots & & & & & \ddots & \ddots & \ddots & A_{N_x-1,N_x} \\ 0 & \cdots & \cdots & \cdots & \cdots & \cdots & 0 & A_{N_x,N_x-1} & A_{N_x,N_x} \end{pmatrix} \tag{3.19}$$

The nonzero elements are given by

$$A_{i,i-1} = -F \tag{3.20}$$
$$A_{i,i} = 1 + 2F \tag{3.21}$$
$$A_{i,i+1} = -F \tag{3.22}$$

in the equations for internal points, $i = 1, \ldots, N_x - 1$. The first and last equation correspond to the boundary condition, where we know the solution, and therefore we must have

$$A_{0,0} = 1, \tag{3.23}$$
$$A_{0,1} = 0, \tag{3.24}$$
$$A_{N_x,N_x-1} = 0, \tag{3.25}$$
$$A_{N_x,N_x} = 1\,. \tag{3.26}$$

The right-hand side b is written as

$$b = \begin{pmatrix} b_0 \\ b_1 \\ \vdots \\ b_i \\ \vdots \\ b_{N_x} \end{pmatrix} \tag{3.27}$$

with

$$b_0 = 0, \tag{3.28}$$
$$b_i = u_i^{n-1}, \quad i = 1, \ldots, N_x - 1, \tag{3.29}$$
$$b_{N_x} = 0\,. \tag{3.30}$$

We observe that the matrix A contains quantities that do not change in time. Therefore, A can be formed once and for all before we enter the recursive formulas for the time evolution. The right-hand side b, however, must be updated at each time step. This leads to the following computational algorithm, here sketched with Python code:

```
x = np.linspace(0, L, Nx+1)   # mesh points in space
dx = x[1] - x[0]
t = np.linspace(0, T, N+1)    # mesh points in time
u   = np.zeros(Nx+1)          # unknown u at new time level
u_n = np.zeros(Nx+1)          # u at the previous time level

# Data structures for the linear system
A = np.zeros((Nx+1, Nx+1))
b = np.zeros(Nx+1)

for i in range(1, Nx):
    A[i,i-1] = -F
    A[i,i+1] = -F
    A[i,i] = 1 + 2*F
A[0,0] = A[Nx,Nx] = 1

# Set initial condition u(x,0) = I(x)
```

```
for i in range(0, Nx+1):
    u_n[i] = I(x[i])

import scipy.linalg

for n in range(0, Nt):
    # Compute b and solve linear system
    for i in range(1, Nx):
        b[i] = -u_n[i]
    b[0] = b[Nx] = 0
    u[:] = scipy.linalg.solve(A, b)

    # Update u_n before next step
    u_n[:] = u
```

Regarding verification, the same considerations apply as for the Forward Euler method (Section 3.1.4).

3.2.2 Sparse matrix implementation

We have seen from (3.19) that the matrix A is tridiagonal. The code segment above used a full, dense matrix representation of A, which stores a lot of values we know are zero beforehand, and worse, the solution algorithm computes with all these zeros. With N_x+1 unknowns, the work by the solution algorithm is $\frac{1}{3}(N_x+1)^3$ and the storage requirements $(N_x+1)^2$. By utilizing the fact that A is tridiagonal and employing corresponding software tools that work with the three diagonals, the work and storage demands can be proportional to N_x only. This leads to a dramatic improvement: with $N_x = 200$, which is a realistic resolution, the code runs about 40,000 times faster and reduces the storage to just 1.5%! It is no doubt that we should take advantage of the fact that A is tridiagonal.

The key idea is to apply a data structure for a tridiagonal or sparse matrix. The `scipy.sparse` package has relevant utilities. For example, we can store only the nonzero diagonals of a matrix. The package also has linear system solvers that operate on sparse matrix data structures. The code below illustrates how we can store only the main diagonal and the upper and lower diagonals.

```
# Representation of sparse matrix and right-hand side
main  = np.zeros(Nx+1)
lower = np.zeros(Nx)
upper = np.zeros(Nx)
b     = np.zeros(Nx+1)
```

```
# Precompute sparse matrix
main[:] = 1 + 2*F
lower[:] = -F
upper[:] = -F
# Insert boundary conditions
main[0] = 1
main[Nx] = 1

A = scipy.sparse.diags(
    diagonals=[main, lower, upper],
    offsets=[0, -1, 1], shape=(Nx+1, Nx+1),
    format='csr')
print A.todense()  # Check that A is correct

# Set initial condition
for i in range(0,Nx+1):
    u_n[i] = I(x[i])

for n in range(0, Nt):
    b = u_n
    b[0] = b[-1] = 0.0  # boundary conditions
    u[:] = scipy.sparse.linalg.spsolve(A, b)
    u_n[:] = u
```

The `scipy.sparse.linalg.spsolve` function utilizes the sparse storage structure of `A` and performs, in this case, a very efficient Gaussian elimination solve.

The program `diffu1D_u0.py` contains a function `solver_BE`, which implements the Backward Euler scheme sketched above. As mentioned in Section 3.1.2, the functions `plug` and `gaussian` run the case with $I(x)$ as a discontinuous plug or a smooth Gaussian function. All experiments point to two characteristic features of the Backward Euler scheme: 1) it is always stable, and 2) it always gives a smooth, decaying solution.

3.2.3 Crank-Nicolson scheme

The idea in the Crank-Nicolson scheme is to apply centered differences in space and time, combined with an average in time. We demand the PDE to be fulfilled at the spatial mesh points, but midway between the points in the time mesh:

$$\frac{\partial}{\partial t}u(x_i, t_{n+\frac{1}{2}}) = \alpha\frac{\partial^2}{\partial x^2}u(x_i, t_{n+\frac{1}{2}}) + f(x_i, t_{n+\frac{1}{2}}),$$

for $i = 1, \ldots, N_x - 1$ and $n = 0, \ldots, N_t - 1$.

With centered differences in space and time, we get

$$[D_t u = \alpha D_x D_x u + f]_i^{n+\frac{1}{2}} .$$

On the right-hand side we get an expression

$$\frac{1}{\Delta x^2}\left(u_{i-1}^{n+\frac{1}{2}} - 2u_i^{n+\frac{1}{2}} + u_{i+1}^{n+\frac{1}{2}}\right) + f_i^{n+\frac{1}{2}} .$$

This expression is problematic since $u_i^{n+\frac{1}{2}}$ is not one of the unknowns we compute. A possibility is to replace $u_i^{n+\frac{1}{2}}$ by an arithmetic average:

$$u_i^{n+\frac{1}{2}} \approx \frac{1}{2}\left(u_i^n + u_i^{n+1}\right) .$$

In the compact notation, we can use the arithmetic average notation $\overline{u}^t$:

$$[D_t u = \alpha D_x D_x \overline{u}^t + f]_i^{n+\frac{1}{2}} .$$

We can also use an average for $f_i^{n+\frac{1}{2}}$:

$$[D_t u = \alpha D_x D_x \overline{u}^t + \overline{f}^t]_i^{n+\frac{1}{2}} .$$

After writing out the differences and average, multiplying by Δt, and collecting all unknown terms on the left-hand side, we get

$$u_i^{n+1} - \frac{1}{2}F(u_{i-1}^{n+1} - 2u_i^{n+1} + u_{i+1}^{n+1}) = u_i^n + \frac{1}{2}F(u_{i-1}^n - 2u_i^n + u_{i+1}^n) + \frac{1}{2}f_i^{n+1} + \frac{1}{2}f_i^n . \tag{3.31}$$

Also here, as in the Backward Euler scheme, the new unknowns u_{i-1}^{n+1}, u_i^{n+1}, and u_{i+1}^{n+1} are coupled in a linear system $AU = b$, where A has the same structure as in (3.19), but with slightly different entries:

$$A_{i,i-1} = -\frac{1}{2}F \tag{3.32}$$

$$A_{i,i} = 1 + F \tag{3.33}$$

$$A_{i,i+1} = -\frac{1}{2}F \tag{3.34}$$

in the equations for internal points, $i = 1, \ldots, N_x - 1$. The equations for the boundary points correspond to

$$A_{0,0} = 1, \tag{3.35}$$
$$A_{0,1} = 0, \tag{3.36}$$
$$A_{N_x,N_x-1} = 0, \tag{3.37}$$
$$A_{N_x,N_x} = 1\,. \tag{3.38}$$

The right-hand side b has entries

$$b_0 = 0, \tag{3.39}$$
$$b_i = u_i^{n-1} + \frac{1}{2}(f_i^n + f_i^{n+1}), \quad i = 1, \ldots, N_x - 1, \tag{3.40}$$
$$b_{N_x} = 0\,. \tag{3.41}$$

When verifying some implementation of the Crank-Nicolson scheme by convergence rate testing, one should note that the scheme is second order accurate in both space and time. The numerical error then reads

$$E = C_t \Delta t^r + C_x \Delta x^r,$$

where $r = 2$ (C_t and C_x are unknown constants, as before). When introducing a single discretization parameter, we may now simply choose

$$h = \Delta x = \Delta t,$$

which gives

$$E = C_t h^r + C_x h^r = (C_t + C_x) h^r,$$

where r should approach 2 as resolution is increased in the convergence rate computations.

3.2.4 The unifying θ rule

For the equation

$$\frac{\partial u}{\partial t} = G(u),$$

where $G(u)$ is some spatial differential operator, the θ-rule looks like

$$\frac{u_i^{n+1} - u_i^n}{\Delta t} = \theta G(u_i^{n+1}) + (1-\theta) G(u_i^n)\,.$$

The important feature of this time discretization scheme is that we can implement one formula and then generate a family of well-known and widely used schemes:

- $\theta = 0$ gives the Forward Euler scheme in time
- $\theta = 1$ gives the Backward Euler scheme in time
- $\theta = \frac{1}{2}$ gives the Crank-Nicolson scheme in time

In the compact difference notation, we write the θ rule as

$$[\overline{D}_t u = \alpha D_x D_x u]^{n+\theta} .$$

We have that $t_{n+\theta} = \theta t_{n+1} + (1-\theta)t_n$.

Applied to the 1D diffusion problem, the θ-rule gives

$$\frac{u_i^{n+1} - u_i^n}{\Delta t} = \alpha \left(\theta \frac{u_{i+1}^{n+1} - 2u_i^{n+1} + u_{i-1}^{n+1}}{\Delta x^2} + (1-\theta)\frac{u_{i+1}^n - 2u_i^n + u_{i-1}^n}{\Delta x^2} \right) + \theta f_i^{n+1} + (1-\theta) f_i^n .$$

This scheme also leads to a matrix system with entries

$$A_{i,i-1} = -F\theta, \quad A_{i,i} = 1 + 2F\theta \quad , A_{i,i+1} = -F\theta,$$

while right-hand side entry b_i is

$$b_i = u_i^n + F(1-\theta)\frac{u_{i+1}^n - 2u_i^n + u_{i-1}^n}{\Delta x^2} + \Delta t \theta f_i^{n+1} + \Delta t(1-\theta) f_i^n .$$

The corresponding entries for the boundary points are as in the Backward Euler and Crank-Nicolson schemes listed earlier.

Note that convergence rate testing with implementations of the theta rule must adjust the error expression according to which of the underlying schemes is actually being run. That is, if $\theta = 0$ (i.e., Forward Euler) or $\theta = 1$ (i.e., Backward Euler), there should be first order convergence, whereas with $\theta = 0.5$ (i.e., Crank-Nicolson), one should get second order convergence (as outlined in previous sections).

3.2.5 Experiments

We can repeat the experiments from Section 3.1.5 to see if the Backward Euler or Crank-Nicolson schemes have problems with sawtooth-like noise

when starting with a discontinuous initial condition. We can also verify that we can have $F > \frac{1}{2}$, which allows larger time steps than in the Forward Euler method.

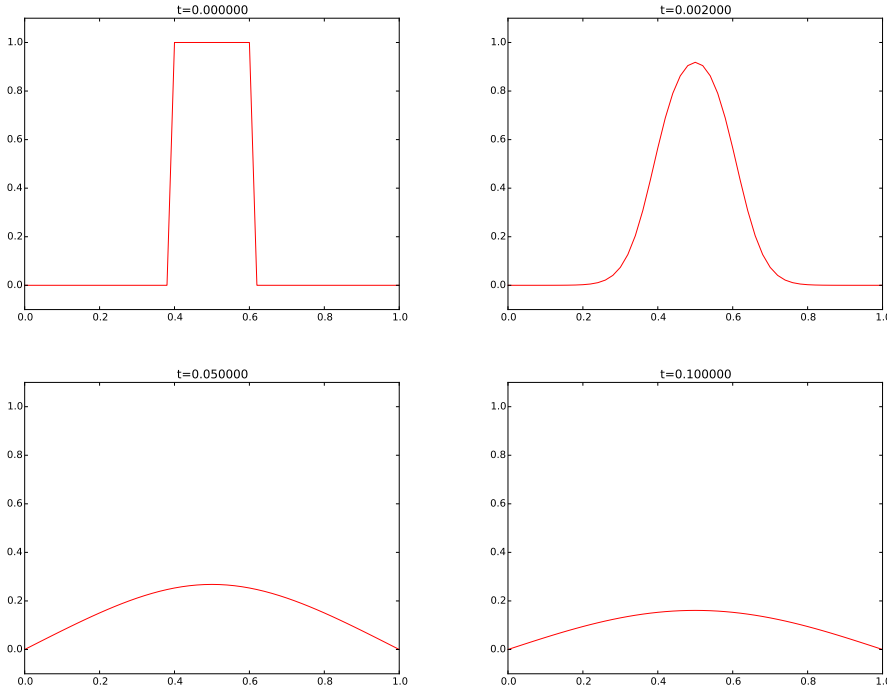


Fig. 3.5 Backward Euler scheme for $F = 0.5$.

The Backward Euler scheme always produces smooth solutions for any F. Figure 3.5 shows one example. Note that the mathematical discontinuity at $t = 0$ leads to a linear variation on a mesh, but the approximation to a jump becomes better as N_x increases. In our simulation, we specify Δt and F, and set N_x to $L/\sqrt{\alpha \Delta t/F}$. Since $N_x \sim \sqrt{F}$, the discontinuity looks sharper in the Crank-Nicolson simulations with larger F.

The Crank-Nicolson method produces smooth solutions for small F, $F \leq \frac{1}{2}$, but small noise gets more and more evident as F increases. Figures 3.6 and 3.7 demonstrate the effect for $F = 3$ and $F = 10$, respectively. Section 3.3 explains why such noise occur.

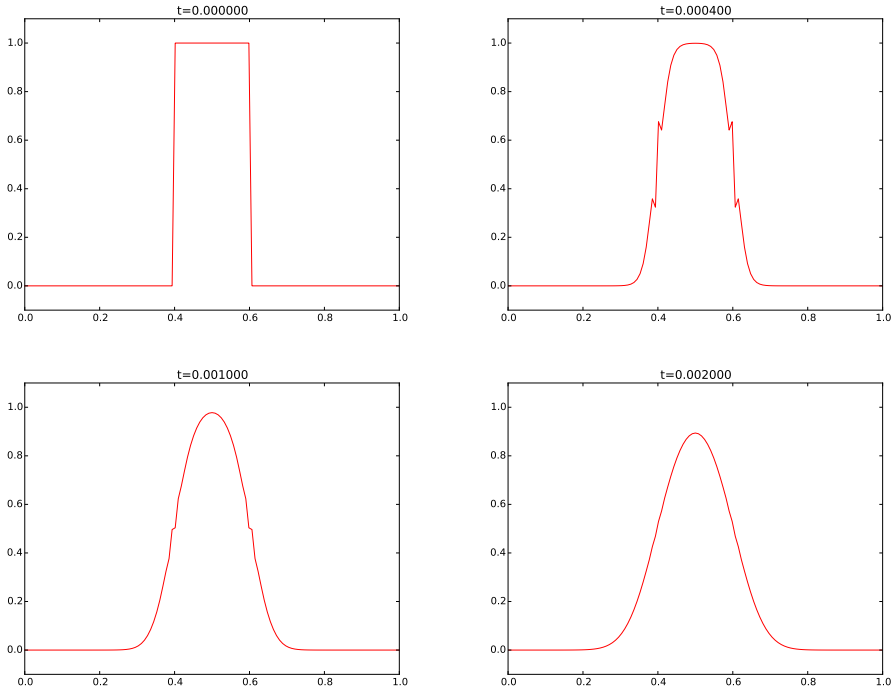


Fig. 3.6 Crank-Nicolson scheme for $F = 3$.

3.2.6 The Laplace and Poisson equation

The Laplace equation, $\nabla^2 u = 0$, and the Poisson equation, $-\nabla^2 u = f$, occur in numerous applications throughout science and engineering. In 1D these equations read $u''(x) = 0$ and $-u''(x) = f(x)$, respectively. We can solve 1D variants of the Laplace equations with the listed software, because we can interpret $u_{xx} = 0$ as the limiting solution of $u_t = \alpha u_{xx}$ when u reaches a steady state limit where $u_t \to 0$. Similarly, Poisson's equation $-u_{xx} = f$ arises from solving $u_t = u_{xx} + f$ and letting $t \to \infty$ so $u_t \to 0$.

Technically in a program, we can simulate $t \to \infty$ by just taking one large time step: $\Delta t \to \infty$. In the limit, the Backward Euler scheme gives

$$-\frac{u_{i+1}^{n+1} - 2u_i^{n+1} + u_{i-1}^{n+1}}{\Delta x^2} = f_i^{n+1},$$

which is nothing but the discretization $[-D_x D_x u = f]_i^{n+1} = 0$ of $-u_{xx} = f$.

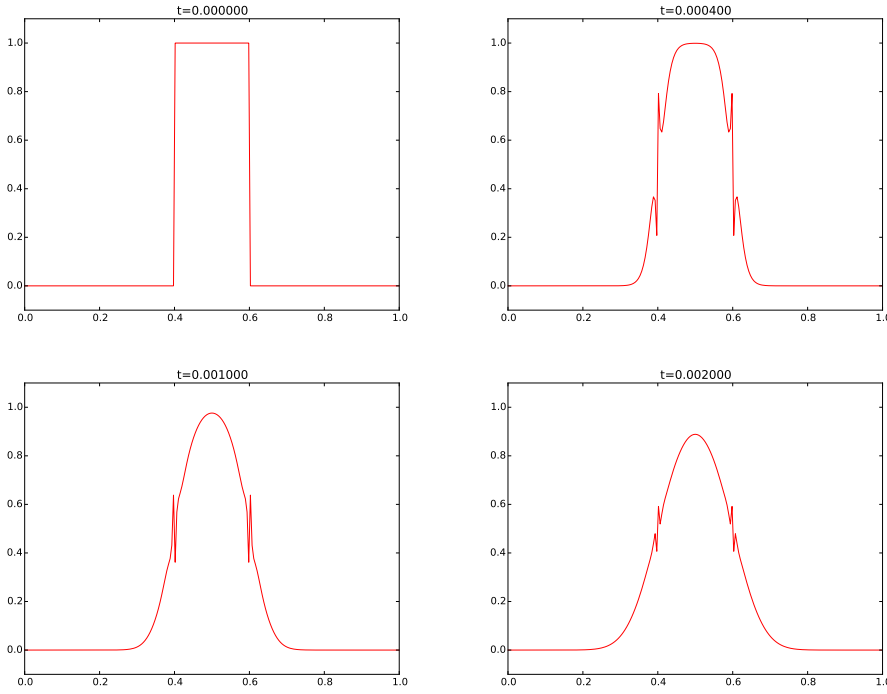


Fig. 3.7 Crank-Nicolson scheme for $F = 10$.

The result above means that the Backward Euler scheme can solve the limit equation directly and hence produce a solution of the 1D Laplace equation. With the Forward Euler scheme we must do the time stepping since $\Delta t > \Delta x^2/\alpha$ is illegal and leads to instability. We may interpret this time stepping as solving the equation system from $-u_{xx} = f$ by iterating on a pseudo time variable.

3.3 Analysis of schemes for the diffusion equation

The numerical experiments in Sections 3.1.5 and 3.2.5 reveal that there are some numerical problems with the Forward Euler and Crank-Nicolson schemes: sawtooth-like noise is sometimes present in solutions that are, from a mathematical point of view, expected to be smooth. This section presents a mathematical analysis that explains the observed behavior and arrives at criteria for obtaining numerical solutions that reproduce the qualitative properties of the exact solutions. In short, we shall explain what is observed in Figures 3.1-3.7.

3.3.1 Properties of the solution

A particular characteristic of diffusive processes, governed by an equation like

$$u_t = \alpha u_{xx}, \tag{3.42}$$

is that the initial shape $u(x,0) = I(x)$ spreads out in space with time, along with a decaying amplitude. Three different examples will illustrate the spreading of u in space and the decay in time.

Similarity solution. The diffusion equation (3.42) admits solutions that depend on $\eta = (x-c)/\sqrt{4\alpha t}$ for a given value of c. One particular solution is

$$u(x,t) = a\,\mathrm{erf}(\eta) + b, \tag{3.43}$$

where

$$\mathrm{erf}(\eta) = \frac{2}{\sqrt{\pi}} \int_0^\eta e^{-\zeta^2} d\zeta, \tag{3.44}$$

is the *error function*, and a and b are arbitrary constants. The error function lies in $(-1,1)$, is odd around $\eta = 0$, and goes relatively quickly to ± 1:

$$\begin{aligned}
\lim_{\eta\to-\infty} \mathrm{erf}(\eta) &= -1,\\
\lim_{\eta\to\infty} \mathrm{erf}(\eta) &= 1,\\
\mathrm{erf}(\eta) &= -\mathrm{erf}(-\eta),\\
\mathrm{erf}(0) &= 0,\\
\mathrm{erf}(2) &= 0.99532227,\\
\mathrm{erf}(3) &= 0.99997791\,.
\end{aligned}$$

As $t \to 0$, the error function approaches a step function centered at $x = c$. For a diffusion problem posed on the unit interval $[0,1]$, we may choose the step at $x = 1/2$ (meaning $c = 1/2$), $a = -1/2$, $b = 1/2$. Then

$$u(x,t) = \frac{1}{2}\left(1 - \mathrm{erf}\left(\frac{x-\frac{1}{2}}{\sqrt{4\alpha t}}\right)\right) = \frac{1}{2}\mathrm{erfc}\left(\frac{x-\frac{1}{2}}{\sqrt{4\alpha t}}\right), \tag{3.45}$$

where we have introduced the *complementary error function* $\mathrm{erfc}(\eta) = 1 - \mathrm{erf}(\eta)$. The solution (3.45) implies the boundary conditions

$$u(0,t) = \frac{1}{2}\left(1 - \mathrm{erf}\left(\frac{-1/2}{\sqrt{4\alpha t}}\right)\right), \tag{3.46}$$

$$u(1,t) = \frac{1}{2}\left(1 - \mathrm{erf}\left(\frac{1/2}{\sqrt{4\alpha t}}\right)\right). \tag{3.47}$$

For small enough t, $u(0,t) \approx 1$ and $u(1,t) \approx 0$, but as $t \to \infty$, $u(x,t) \to 1/2$ on $[0,1]$.

Solution for a Gaussian pulse. The standard diffusion equation $u_t = \alpha u_{xx}$ admits a Gaussian function as solution:

$$u(x,t) = \frac{1}{\sqrt{4\pi\alpha t}} \exp\left(-\frac{(x-c)^2}{4\alpha t}\right). \tag{3.48}$$

At $t = 0$ this is a Dirac delta function, so for computational purposes one must start to view the solution at some time $t = t_\epsilon > 0$. Replacing t by $t_\epsilon + t$ in (3.48) makes it easy to operate with a (new) t that starts at $t = 0$ with an initial condition with a finite width. The important feature of (3.48) is that the standard deviation σ of a sharp initial Gaussian pulse increases in time according to $\sigma = \sqrt{2\alpha t}$, making the pulse diffuse and flatten out.

Solution for a sine component. Also, (3.42) admits a solution of the form

$$u(x,t) = Qe^{-at}\sin(kx). \tag{3.49}$$

The parameters Q and k can be freely chosen, while inserting (3.49) in (3.42) gives the constraint

$$a = -\alpha k^2.$$

A very important feature is that the initial shape $I(x) = Q\sin(kx)$ undergoes a damping $\exp(-\alpha k^2 t)$, meaning that rapid oscillations in space, corresponding to large k, are very much faster dampened than slow oscillations in space, corresponding to small k. This feature leads to a smoothing of the initial condition with time. (In fact, one can use a few steps of the diffusion equation as a method for removing noise in signal processing.) To judge how good a numerical method is, we may

look at its ability to smoothen or dampen the solution in the same way as the PDE does.

The following example illustrates the damping properties of (3.49). We consider the specific problem

$$\begin{aligned} u_t &= u_{xx}, \quad x \in (0,1),\ t \in (0,T], \\ u(0,t) &= u(1,t) = 0, \quad t \in (0,T], \\ u(x,0) &= \sin(\pi x) + 0.1\sin(100\pi x)\,. \end{aligned}$$

The initial condition has been chosen such that adding two solutions like (3.49) constructs an analytical solution to the problem:

$$u(x,t) = e^{-\pi^2 t}\sin(\pi x) + 0.1e^{-\pi^2 10^4 t}\sin(100\pi x)\,. \tag{3.50}$$

Figure 3.8 illustrates the rapid damping of rapid oscillations $\sin(100\pi x)$ and the very much slower damping of the slowly varying $\sin(\pi x)$ term. After about $t = 0.5 \cdot 10^{-4}$ the rapid oscillations do not have a visible amplitude, while we have to wait until $t \sim 0.5$ before the amplitude of the long wave $\sin(\pi x)$ becomes very small.

3.3.2 Analysis of discrete equations

A counterpart to (3.49) is the complex representation of the same function:

$$u(x,t) = Qe^{-at}e^{ikx},$$

where $i = \sqrt{-1}$ is the imaginary unit. We can add such functions, often referred to as wave components, to make a Fourier representation of a general solution of the diffusion equation:

$$u(x,t) \approx \sum_{k \in K} b_k e^{-\alpha k^2 t} e^{ikx}, \tag{3.51}$$

where K is a set of an infinite number of k values needed to construct the solution. In practice, however, the series is truncated and K is a finite set of k values needed to build a good approximate solution. Note that (3.50) is a special case of (3.51) where $K = \{\pi, 100\pi\}$, $b_\pi = 1$, and $b_{100\pi} = 0.1$.

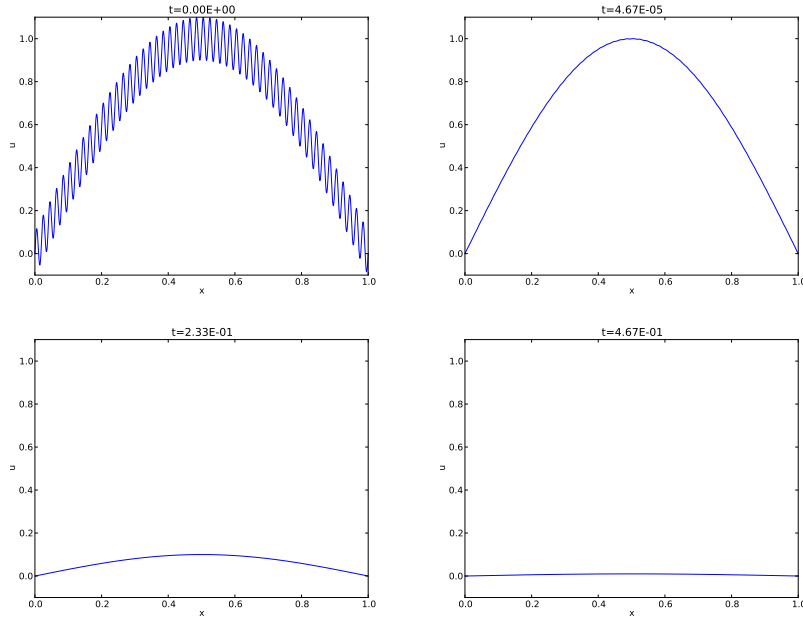


Fig. 3.8 Evolution of the solution of a diffusion problem: initial condition (upper left), 1/100 reduction of the small waves (upper right), 1/10 reduction of the long wave (lower left), and 1/100 reduction of the long wave (lower right).

The amplitudes b_k of the individual Fourier waves must be determined from the initial condition. At $t = 0$ we have $u \approx \sum_k b_k \exp(ikx)$ and find K and b_k such that

$$I(x) \approx \sum_{k \in K} b_k e^{ikx}\,. \tag{3.52}$$

(The relevant formulas for b_k come from Fourier analysis, or equivalently, a least-squares method for approximating $I(x)$ in a function space with basis $\exp(ikx)$.)

Much insight about the behavior of numerical methods can be obtained by investigating how a wave component $\exp(-\alpha k^2 t)\exp(ikx)$ is treated by the numerical scheme. It appears that such wave components are also solutions of the schemes, but the damping factor $\exp(-\alpha k^2 t)$ varies among the schemes. To ease the forthcoming algebra, we write the damping factor as A^n. The exact amplification factor corresponding to A is $A_\mathrm{e} = \exp(-\alpha k^2 \Delta t)$.

3.3.3 Analysis of the finite difference schemes

We have seen that a general solution of the diffusion equation can be built as a linear combination of basic components

$$e^{-\alpha k^2 t} e^{ikx}.$$

A fundamental question is whether such components are also solutions of the finite difference schemes. This is indeed the case, but the amplitude $\exp(-\alpha k^2 t)$ might be modified (which also happens when solving the ODE counterpart $u' = -\alpha u$). We therefore look for numerical solutions of the form

$$u_q^n = A^n e^{ikq\Delta x} = A^n e^{ikx}, \tag{3.53}$$

where the amplification factor A must be determined by inserting the component into an actual scheme. Note that A^n means A raised to the power of n, n being the index in the time mesh, while the superscript n in u_q^n just denotes u at time t_n.

Stability. The exact amplification factor is $A_\mathrm{e} = \exp(-\alpha^2 k^2 \Delta t)$. We should therefore require $|A| < 1$ to have a decaying numerical solution as well. If $-1 \leq A < 0$, A^n will change sign from time level to time level, and we get stable, non-physical oscillations in the numerical solutions that are not present in the exact solution.

Accuracy. To determine how accurately a finite difference scheme treats one wave component (3.53), we see that the basic deviation from the exact solution is reflected in how well A^n approximates A_e^n, or how well A approximates A_e. We can plot A_e and the various expressions for A, and we can make Taylor expansions of A/A_e to see the error more analytically.

Truncation error. As an alternative to examining the accuracy of the damping of a wave component, we can perform a general truncation error analysis as explained in Appendix B. Such results are more general, but less detailed than what we get from the wave component analysis. The truncation error can almost always be computed and represents the error in the numerical model when the exact solution is substituted into the equations. In particular, the truncation error analysis tells the order of the scheme, which is of fundamental importance when verifying codes based on empirical estimation of convergence rates.

3.3.4 Analysis of the Forward Euler scheme

The Forward Euler finite difference scheme for $u_t = \alpha u_{xx}$ can be written as

$$[D_t^+ u = \alpha D_x D_x u]_q^n .$$

Inserting a wave component (3.53) in the scheme demands calculating the terms

$$e^{ikq\Delta x}[D_t^+ A]^n = e^{ikq\Delta x} A^n \frac{A-1}{\Delta t},$$

and

$$A^n D_x D_x [e^{ikx}]_q = A^n \left(-e^{ikq\Delta x} \frac{4}{\Delta x^2} \sin^2 \left(\frac{k\Delta x}{2} \right) \right) .$$

Inserting these terms in the discrete equation and dividing by $A^n e^{ikq\Delta x}$ leads to

$$\frac{A-1}{\Delta t} = -\alpha \frac{4}{\Delta x^2} \sin^2 \left(\frac{k\Delta x}{2} \right),$$

and consequently

$$A = 1 - 4F \sin^2 p \tag{3.54}$$

where

$$F = \frac{\alpha \Delta t}{\Delta x^2} \tag{3.55}$$

is the *numerical Fourier number*, and $p = k\Delta x/2$. The complete numerical solution is then

$$u_q^n = \left(1 - 4F \sin^2 p\right)^n e^{ikq\Delta x} . \tag{3.56}$$

Stability. We easily see that $A \leq 1$. However, the A can be less than -1, which will lead to growth of a numerical wave component. The criterion $A \geq -1$ implies

$$4F \sin^2(p/2) \leq 2 .$$

The worst case is when $\sin^2(p/2) = 1$, so a sufficient criterion for stability is

$$F \leq \frac{1}{2}, \tag{3.57}$$

or expressed as a condition on Δt:

$$\Delta t \leq \frac{\Delta x^2}{2\alpha}\,. \tag{3.58}$$

Note that halving the spatial mesh size, $\Delta x \to \frac{1}{2}\Delta x$, requires Δt to be reduced by a factor of 1/4. The method hence becomes very expensive for fine spatial meshes.

Accuracy. Since A is expressed in terms of F and the parameter we now call $p = k\Delta x/2$, we should also express A_{e} by F and p. The exponent in A_{e} is $-\alpha k^2 \Delta t$, which equals $-Fk^2\Delta x^2 = -F4p^2$. Consequently,

$$A_{\mathrm{e}} = \exp\left(-\alpha k^2 \Delta t\right) = \exp\left(-4Fp^2\right).$$

All our A expressions as well as A_{e} are now functions of the two dimensionless parameters F and p.

Computing the Taylor series expansion of A/A_{e} in terms of F can easily be done with aid of `sympy`:

```
def A_exact(F, p):
    return exp(-4*F*p**2)

def A_FE(F, p):
    return 1 - 4*F*sin(p)**2

from sympy import *
F, p = symbols('F p')
A_err_FE = A_FE(F, p)/A_exact(F, p)
print A_err_FE.series(F, 0, 6)
```

The result is

$$\frac{A}{A_{\mathrm{e}}} = 1 - 4F\sin^2 p + 2Fp^2 - 16F^2p^2\sin^2 p + 8F^2p^4 + \cdots$$

Recalling that $F = \alpha\Delta t/\Delta x^2$, $p = k\Delta x/2$, and that $\sin^2 p \leq 1$, we realize that the dominating terms in A/A_{e} are at most

$$1 - 4\alpha\frac{\Delta t}{\Delta x^2} + \alpha\Delta t - 4\alpha^2\Delta t^2 + \alpha^2\Delta t^2\Delta x^2 + \cdots\,.$$

Truncation error. We follow the theory explained in Appendix B. The recipe is to set up the scheme in operator notation and use formulas from Appendix B.2.4 to derive an expression for the residual. The details are documented in Appendix B.6.1. We end up with a truncation error

$$R_i^n = \mathcal{O}(\Delta t) + \mathcal{O}(\Delta x^2)\,.$$

Although this is not the true error $u_{\mathrm{e}}(x_i, t_n) - u_i^n$, it indicates that the true error is of the form

$$E = C_t \Delta t + C_x \Delta x^2$$

for two unknown constants C_t and C_x.

3.3.5 Analysis of the Backward Euler scheme

Discretizing $u_t = \alpha u_{xx}$ by a Backward Euler scheme,

$$[D_t^- u = \alpha D_x D_x u]_q^n,$$

and inserting a wave component (3.53), leads to calculations similar to those arising from the Forward Euler scheme, but since

$$e^{ikq\Delta x}[D_t^- A]^n = A^n e^{ikq\Delta x}\frac{1 - A^{-1}}{\Delta t},$$

we get

$$\frac{1 - A^{-1}}{\Delta t} = -\alpha\frac{4}{\Delta x^2}\sin^2\left(\frac{k\Delta x}{2}\right),$$

and then

$$A = \left(1 + 4F\sin^2 p\right)^{-1}\,. \tag{3.59}$$

The complete numerical solution can be written

$$u_q^n = \left(1 + 4F\sin^2 p\right)^{-n} e^{ikq\Delta x}\,. \tag{3.60}$$

Stability. We see from (3.59) that $0 < A < 1$, which means that all numerical wave components are stable and non-oscillatory for any $\Delta t > 0$.

Truncation error. The derivation of the truncation error for the Backward Euler scheme is almost identical to that for the Forward Euler scheme. We end up with

$$R_i^n = \mathcal{O}(\Delta t) + \mathcal{O}(\Delta x^2)\,.$$

3.3.6 Analysis of the Crank-Nicolson scheme

The Crank-Nicolson scheme can be written as

$$[D_t u = \alpha D_x D_x \overline{u}^x]_q^{n+\frac{1}{2}},$$

or

$$[D_t u]_q^{n+\frac{1}{2}} = \frac{1}{2}\alpha \left([D_x D_x u]_q^n + [D_x D_x u]_q^{n+1} \right) .$$

Inserting (3.53) in the time derivative approximation leads to

$$[D_t A^n e^{ikq\Delta x}]^{n+\frac{1}{2}} = A^{n+\frac{1}{2}} e^{ikq\Delta x} \frac{A^{\frac{1}{2}} - A^{-\frac{1}{2}}}{\Delta t} = A^n e^{ikq\Delta x} \frac{A-1}{\Delta t} .$$

Inserting (3.53) in the other terms and dividing by $A^n e^{ikq\Delta x}$ gives the relation

$$\frac{A-1}{\Delta t} = -\frac{1}{2}\alpha \frac{4}{\Delta x^2} \sin^2\left(\frac{k\Delta x}{2}\right)(1+A),$$

and after some more algebra,

$$A = \frac{1 - 2F\sin^2 p}{1 + 2F\sin^2 p} . \tag{3.61}$$

The exact numerical solution is hence

$$u_q^n = \left(\frac{1 - 2F\sin^2 p}{1 + 2F\sin^2 p} \right)^n e^{ikq\Delta x} . \tag{3.62}$$

Stability. The criteria $A > -1$ and $A < 1$ are fulfilled for any $\Delta t > 0$. Therefore, the solution cannot grow, but it will oscillate if $1 - 2F\sin^p < 0$. To avoid such non-physical oscillations, we must demand $F \leq \frac{1}{2}$.

Truncation error. The truncation error is derived in Appendix B.6.1:

$$R_i^{n+\frac{1}{2}} = \mathcal{O}(\Delta x^2) + \mathcal{O}(\Delta t^2) .$$

3.3.7 Analysis of the Leapfrog scheme

An attractive feature of the Forward Euler scheme is the explicit time stepping and no need for solving linear systems. However, the accuracy

in time is only $\mathcal{O}(\Delta t)$. We can get an explicit *second-order* scheme in time by using the Leapfrog method:

$$[D_{2t}u = \alpha D_x Dx u + f]_q^n .$$

Written out,

$$u_q^{n+1} = u_q^{n-1} + \frac{2\alpha\Delta t}{\Delta x^2}(u_{q+1}^n - 2u_q^n + u_{q-1}^n) + f(x_q, t_n) .$$

We need some formula for the first step, u_q^1, but for that we can use a Forward Euler step.

Unfortunately, the Leapfrog scheme is always unstable for the diffusion equation. To see this, we insert a wave component $A^n e^{ikx}$ and get

$$\frac{A - A^{-1}}{\Delta t} = -\alpha \frac{4}{\Delta x^2} \sin^2 p,$$

or

$$A^2 + 4F \sin^2 p\, A - 1 = 0,$$

which has roots

$$A = -2F \sin^2 p \pm \sqrt{4F^2 \sin^4 p + 1} .$$

Both roots have $|A| > 1$ so the amplitude always grows, which is not in accordance with the physics of the problem. However, for a PDE with a first-order derivative in space, instead of a second-order one, the Leapfrog scheme performs very well. Details are provided in Section 4.1.3.

3.3.8 Summary of accuracy of amplification factors

We can plot the various amplification factors against $p = k\Delta x/2$ for different choices of the F parameter. Figures 3.9, 3.10, and 3.11 show how long and small waves are damped by the various schemes compared to the exact damping. As long as all schemes are stable, the amplification factor is positive, except for Crank-Nicolson when $F > 0.5$.

The effect of negative amplification factors is that A^n changes sign from one time level to the next, thereby giving rise to oscillations in time in an animation of the solution. We see from Figure 3.9 that for $F = 20$, waves with $p \geq \pi/4$ undergo a damping close to -1, which means that the amplitude does not decay and that the wave component

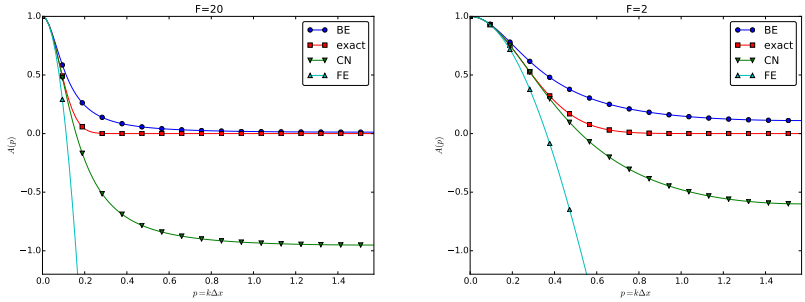


Fig. 3.9 Amplification factors for large time steps.

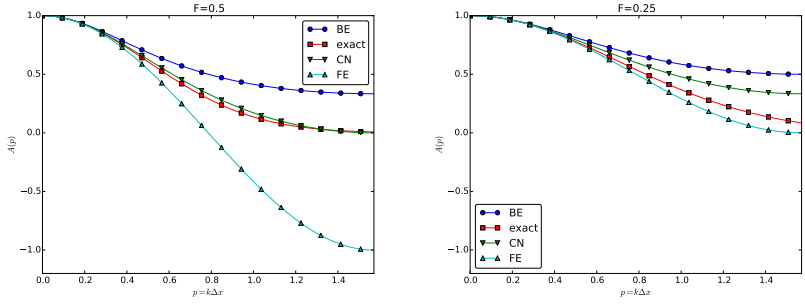


Fig. 3.10 Amplification factors for time steps around the Forward Euler stability limit.

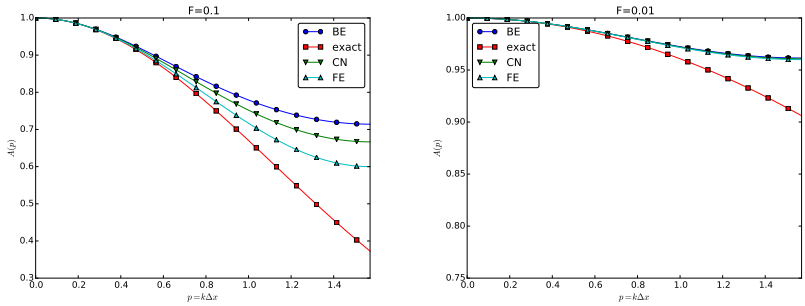


Fig. 3.11 Amplification factors for small time steps.

jumps up and down (flips amplitude) in time. For $F = 2$ we have a damping of a factor of 0.5 from one time level to the next, which is very much smaller than the exact damping. Short waves will therefore fail to be effectively dampened. These waves will manifest themselves as high frequency oscillatory noise in the solution.

A value $p = \pi/4$ corresponds to four mesh points per wave length of e^{ikx}, while $p = \pi/2$ implies only two points per wave length, which is the smallest number of points we can have to represent the wave on the mesh.

To demonstrate the oscillatory behavior of the Crank-Nicolson scheme, we choose an initial condition that leads to short waves with significant amplitude. A discontinuous $I(x)$ will in particular serve this purpose: Figures 3.6 and 3.7 correspond to $F = 3$ and $F = 10$, respectively, and we see how short waves pollute the overall solution.

3.3.9 Analysis of the 2D diffusion equation

Diffusion in several dimensions is treated later, but it is appropriate to include the analysis here. We first consider the 2D diffusion equation

$$u_t = \alpha(u_{xx} + u_{yy}),$$

which has Fourier component solutions of the form

$$u(x, y, t) = Ae^{-\alpha k^2 t} e^{i(k_x x + k_y y)},$$

and the schemes have discrete versions of this Fourier component:

$$u^n_{q,r} = A\xi^n e^{i(k_x q\Delta x + k_y r\Delta y)}\,.$$

The Forward Euler scheme. For the Forward Euler discretization,

$$[D_t^+ u = \alpha(D_x D_x u + D_y D_y u)]^n_{q,r},$$

we get

$$\frac{\xi - 1}{\Delta t} = -\alpha\frac{4}{\Delta x^2}\sin^2\left(\frac{k_x\Delta x}{2}\right) - \alpha\frac{4}{\Delta y^2}\sin^2\left(\frac{k_y\Delta y}{2}\right)\,.$$

Introducing

$$p_x = \frac{k_x\Delta x}{2}, \quad p_y = \frac{k_y\Delta y}{2},$$

we can write the equation for ξ more compactly as

$$\frac{\xi - 1}{\Delta t} = -\alpha\frac{4}{\Delta x^2}\sin^2 p_x - \alpha\frac{4}{\Delta y^2}\sin^2 p_y,$$

and solve for ξ:

$$\xi = 1 - 4F_x\sin^2 p_x - 4F_y\sin^2 p_y\,. \tag{3.63}$$

The complete numerical solution for a wave component is

$$u^n_{q,r} = A(1 - 4F_x \sin^2 p_x - 4F_y \sin^2 p_y)^n e^{i(k_x q\Delta x + k_y r\Delta y)}\,. \tag{3.64}$$

For stability we demand $-1 \leq \xi \leq 1$, and $-1 \leq \xi$ is the critical limit, since clearly $\xi \leq 1$, and the worst case happens when the sines are at their maximum. The stability criterion becomes

$$F_x + F_y \leq \frac{1}{2}\,. \tag{3.65}$$

For the special, yet common, case $\Delta x = \Delta y = h$, the stability criterion can be written as

$$\Delta t \leq \frac{h^2}{2d\alpha},$$

where d is the number of space dimensions: $d = 1, 2, 3$.

The Backward Euler scheme. The Backward Euler method,

$$[D_t^- u = \alpha(D_x D_x u + D_y D_y u)]^n_{q,r},$$

results in

$$1 - \xi^{-1} = -4F_x \sin^2 p_x - 4F_y \sin^2 p_y,$$

and

$$\xi = (1 + 4F_x \sin^2 p_x + 4F_y \sin^2 p_y)^{-1},$$

which is always in $(0, 1]$. The solution for a wave component becomes

$$u^n_{q,r} = A(1 + 4F_x \sin^2 p_x + 4F_y \sin^2 p_y)^{-n} e^{i(k_x q\Delta x + k_y r\Delta y)}\,. \tag{3.66}$$

The Crank-Nicolson scheme. With a Crank-Nicolson discretization,

$$[D_t u]^{n+\frac{1}{2}}_{q,r} = \frac{1}{2}[\alpha(D_x D_x u + D_y D_y u)]^{n+1}_{q,r} + \frac{1}{2}[\alpha(D_x D_x u + D_y D_y u)]^n_{q,r},$$

we have, after some algebra,

$$\xi = \frac{1 - 2(F_x \sin^2 p_x + F_x \sin^2 p_y)}{1 + 2(F_x \sin^2 p_x + F_x \sin^2 p_y)}\,.$$

The fraction on the right-hand side is always less than 1, so stability in the sense of non-growing wave components is guaranteed for all physical and numerical parameters. However, the fraction can become negative and result in non-physical oscillations. This phenomenon happens when

$$F_x \sin^2 p_x + F_x \sin^2 p_y > \frac{1}{2}\,.$$

A criterion against non-physical oscillations is therefore

$$F_x + F_y \leq \frac{1}{2},$$

which is the same limit as the stability criterion for the Forward Euler scheme.

The exact discrete solution is

$$u^n_{q,r} = A\left(\frac{1 - 2(F_x \sin^2 p_x + F_x \sin^2 p_y)}{1 + 2(F_x \sin^2 p_x + F_x \sin^2 p_y)}\right)^n e^{i(k_x q\Delta x + k_y r\Delta y)}\,. \tag{3.67}$$

3.3.10 Explanation of numerical artifacts

The behavior of the solution generated by Forward Euler discretization in time (and centered differences in space) is summarized at the end of Section 3.1.5. Can we, from the analysis above, explain the behavior?

We may start by looking at Figure 3.3 where $F = 0.51$. The figure shows that the solution is unstable and grows in time. The stability limit for such growth is $F = 0.5$ and since the F in this simulation is slightly larger, growth is unavoidable.

Figure 3.1 has unexpected features: we would expect the solution of the diffusion equation to be smooth, but the graphs in Figure 3.1 contain non-smooth noise. Turning to Figure 3.4, which has a quite similar initial condition, we see that the curves are indeed smooth. The problem with the results in Figure 3.1 is that the initial condition is discontinuous. To represent it, we need a significant amplitude on the shortest waves in the mesh. However, for $F = 0.5$, the shortest wave ($p = \pi/2$) gives the amplitude in the numerical solution as $(1-4F)^n$, which oscillates between negative and positive values at subsequent time levels for $F > \frac{1}{4}$. Since the shortest waves have visible amplitudes in the solution profile, the oscillations becomes visible. The smooth initial condition in Figure 3.4, on the other hand, leads to very small amplitudes of the shortest waves.

That these waves then oscillate in a non-physical way for $F = 0.5$ is not a visible effect. The oscillations in time in the amplitude $(1 - 4F)^n$ disappear for $F \leq \frac{1}{4}$, and that is why also the discontinuous initial condition always leads to smooth solutions in Figure 3.2, where $F = \frac{1}{4}$.

Turning the attention to the Backward Euler scheme and the experiments in Figure 3.5, we see that even the discontinuous initial condition gives smooth solutions for $F = 0.5$ (and in fact all other F values). From the exact expression of the numerical amplitude, $(1 + 4F \sin^2 p)^{-1}$, we realize that this factor can never flip between positive and negative values, and no instabilities can occur. The conclusion is that the Backward Euler scheme always produces smooth solutions. Also, the Backward Euler scheme guarantees that the solution cannot grow in time (unless we add a source term to the PDE, but that is meant to represent a physically relevant growth).

Finally, we have some small, strange artifacts when simulating the development of the initial plug profile with the Crank-Nicolson scheme, see Figure 3.7, where $F = 3$. The Crank-Nicolson scheme cannot give growing amplitudes, but it may give oscillating amplitudes in time. The critical factor is $1 - 2F \sin^2 p$, which for the shortest waves ($p = \pi/2$) indicates a stability limit $F = 0.5$. With the discontinuous initial condition, we have enough amplitude on the shortest waves so their wrong behavior is visible, and this is what we see as small instabilities in Figure 3.7. The only remedy is to lower the F value.

3.4 Exercises

Exercise 3.1: Explore symmetry in a 1D problem

This exercise simulates the exact solution (3.48). Suppose for simplicity that $c = 0$.

a) Formulate an initial-boundary value problem that has (3.48) as solution in the domain $[-L, L]$. Use the exact solution (3.48) as Dirichlet condition at the boundaries. Simulate the diffusion of the Gaussian peak. Observe that the solution is symmetric around $x = 0$.

b) Show from (3.48) that $u_x(c, t) = 0$. Since the solution is symmetric around $x = c = 0$, we can solve the numerical problem in half of the domain, using a *symmetry boundary condition* $u_x = 0$ at $x = 0$. Set up the initial-boundary value problem in this case. Simulate the diffusion problem in $[0, L]$ and compare with the solution in a).

Filename: `diffu_symmetric_gaussian`.

Exercise 3.2: Investigate approximation errors from a $u_x = 0$ boundary condition

We consider the problem solved in Exercise 3.1 part b). The boundary condition $u_x(0,t) = 0$ can be implemented in two ways: 1) by a standard symmetric finite difference $[D_{2x}u]_i^n = 0$, or 2) by a one-sided difference $[D^+u = 0]_i^n = 0$. Investigate the effect of these two conditions on the convergence rate in space.

Hint. If you use a Forward Euler scheme, choose a discretization parameter $h = \Delta t = \Delta x^2$ and assume the error goes like $E \sim h^r$. The error in the scheme is $\mathcal{O}(\Delta t, \Delta x^2)$ so one should expect that the estimated r approaches 1. The question is if a one-sided difference approximation to $u_x(0,t) = 0$ destroys this convergence rate.
Filename: `diffu_onesided_fd`.

Exercise 3.3: Experiment with open boundary conditions in 1D

We address diffusion of a Gaussian function as in Exercise 3.1, in the domain $[0, L]$, but now we shall explore different types of boundary conditions on $x = L$. In real-life problems we do not know the exact solution on $x = L$ and must use something simpler.

a) Imagine that we want to solve the problem numerically on $[0, L]$, with a symmetry boundary condition $u_x = 0$ at $x = 0$, but we do not know the exact solution and cannot of that reason assign a correct Dirichlet condition at $x = L$. One idea is to simply set $u(L,t) = 0$ since this will be an accurate approximation before the diffused pulse reaches $x = L$ and even thereafter it might be a satisfactory condition if the exact u has a small value. Let u_e be the exact solution and let u be the solution of $u_t = \alpha u_{xx}$ with an initial Gaussian pulse and the boundary conditions $u_x(0,t) = u(L,t) = 0$. Derive a diffusion problem for the error $e = u_\mathrm{e} - u$. Solve this problem numerically using an exact Dirichlet condition at $x = L$. Animate the evolution of the error and make a curve plot of the error measure

$$E(t) = \sqrt{\frac{\int_0^L e^2 dx}{\int_0^L u dx}}\,.$$

Is this a suitable error measure for the present problem?

b) Instead of using $u(L,t) = 0$ as approximate boundary condition for letting the diffused Gaussian pulse move out of our finite domain, one may try $u_x(L,t) = 0$ since the solution for large t is quite flat. Argue that this condition gives a completely wrong asymptotic solution as $t \to 0$. To do this, integrate the diffusion equation from 0 to L, integrate u_{xx} by parts (or use Gauss' divergence theorem in 1D) to arrive at the important property

$$\frac{d}{dt}\int_0^L u(x,t)dx = 0,$$

implying that $\int_0^L u dx$ must be constant in time, and therefore

$$\int_0^L u(x,t)dx = \int_0^L I(x)dx\,.$$

The integral of the initial pulse is 1.

c) Another idea for an artificial boundary condition at $x = L$ is to use a cooling law

$$-\alpha u_x = q(u - u_S), \tag{3.68}$$

where q is an unknown heat transfer coefficient and u_S is the surrounding temperature in the medium outside of $[0, L]$. (Note that arguing that u_S is approximately $u(L,t)$ gives the $u_x = 0$ condition from the previous subexercise that is qualitatively wrong for large t.) Develop a diffusion problem for the error in the solution using (3.68) as boundary condition. Assume one can take $u_S = 0$ "outside the domain" since $u_\mathrm{e} \to 0$ as $x \to \infty$. Find a function $q = q(t)$ such that the exact solution obeys the condition (3.68). Test some constant values of q and animate how the corresponding error function behaves. Also compute $E(t)$ curves as defined above.
Filename: `diffu_open_BC`.

Exercise 3.4: Simulate a diffused Gaussian peak in 2D/3D

a) Generalize (3.48) to multi dimensions by assuming that one-dimensional solutions can be multiplied to solve $u_t = \alpha\nabla^2 u$. Set $c = 0$ such that the peak of the Gaussian is at the origin.

b) One can from the exact solution show that $u_x = 0$ on $x = 0$, $u_y = 0$ on $y = 0$, and $u_z = 0$ on $z = 0$. The approximately correct condition $u = 0$ can be set on the remaining boundaries (say $x = L$, $y = L$, $z = L$), cf. Exercise 3.3. Simulate a 2D case and make an animation of the diffused Gaussian peak.

c) The formulation in b) makes use of symmetry of the solution such that we can solve the problem in the first quadrant (2D) or octant (3D) only. To check that the symmetry assumption is correct, formulate the problem without symmetry in a domain $[-L, L] \times [L, L]$ in 2D. Use $u = 0$ as approximately correct boundary condition. Simulate the same case as in b), but in a four times as large domain. Make an animation and compare it with the one in b).
Filename: `diffu_symmetric_gaussian_2D`.

Exercise 3.5: Examine stability of a diffusion model with a source term

Consider a diffusion equation with a linear u term:

$$u_t = \alpha u_{xx} + \beta u\,.$$

a) Derive in detail the Forward Euler, Backward Euler, and Crank-Nicolson schemes for this type of diffusion model. Thereafter, formulate a θ-rule to summarize the three schemes.

b) Assume a solution like (3.49) and find the relation between a, k, α, and β.

Hint. Insert (3.49) in the PDE problem.

c) Calculate the stability of the Forward Euler scheme. Design numerical experiments to confirm the results.

Hint. Insert the discrete counterpart to (3.49) in the numerical scheme. Run experiments at the stability limit and slightly above.

d) Repeat c) for the Backward Euler scheme.

e) Repeat c) for the Crank-Nicolson scheme.

f) How does the extra term bu impact the accuracy of the three schemes?

Hint. For analysis of the accuracy, compare the numerical and exact amplification factors, in graphs and/or by Taylor series expansion.
Filename: `diffu_stability_uterm`.

3.5 Diffusion in heterogeneous media

Diffusion in heterogeneous media normally implies a non-constant diffusion coefficient $\alpha = \alpha(x)$. A 1D diffusion model with such a variable diffusion coefficient reads

$$\frac{\partial u}{\partial t} = \frac{\partial}{\partial x}\left(\alpha(x)\frac{\partial u}{\partial x}\right) + f(x,t), \quad x \in (0,L),\ t \in (0,T], \tag{3.69}$$

$$u(x,0) = I(x), \qquad x \in [0,L], \tag{3.70}$$

$$u(0,t) = U_0, \qquad t > 0, \tag{3.71}$$

$$u(L,t) = U_L, \qquad t > 0. \tag{3.72}$$

A short form of the diffusion equation with variable coefficients is $u_t = (\alpha u_x)_x + f$.

3.5.1 Discretization

We can discretize (3.69) by a θ-rule in time and centered differences in space:

$$[D_t u]_i^{n+\frac{1}{2}} = \theta[D_x(\overline{\alpha}^x D_x u) + f]_i^{n+1} + (1-\theta)[D_x(\overline{\alpha}^x D_x u) + f]_i^n\,.$$

Written out, this becomes

$$\begin{aligned}\frac{u_i^{n+1} - u_i^n}{\Delta t} = \theta &\frac{1}{\Delta x^2}(\alpha_{i+\frac{1}{2}}(u_{i+1}^{n+1} - u_i^{n+1}) - \alpha_{i-\frac{1}{2}}(u_i^{n+1} - u_{i-1}^{n+1})) + \\ (1-\theta) &\frac{1}{\Delta x^2}(\alpha_{i+\frac{1}{2}}(u_{i+1}^{n} - u_i^{n}) - \alpha_{i-\frac{1}{2}}(u_i^{n} - u_{i-1}^{n})) + \\ &\theta f_i^{n+1} + (1-\theta) f_i^n,\end{aligned}$$

where, e.g., an arithmetic mean can to be used for $\alpha_{i+\frac{1}{2}}$:

$$\alpha_{i+\frac{1}{2}} = \frac{1}{2}(\alpha_i + \alpha_{i+1}) .$$

3.5.2 Implementation

Suitable code for solving the discrete equations is very similar to what we created for a constant α. Since the Fourier number has no meaning for varying α, we introduce a related parameter $D = \Delta t/\Delta x^2$.

```
def solver_theta(I, a, L, Nx, D, T, theta=0.5, u_L=1, u_R=0,
                 user_action=None):
    x = linspace(0, L, Nx+1)   # mesh points in space
    dx = x[1] - x[0]
    dt = D*dx**2
    Nt = int(round(T/float(dt)))
    t = linspace(0, T, Nt+1)   # mesh points in time

    u   = zeros(Nx+1)   # solution array at t[n+1]
    u_n = zeros(Nx+1)   # solution at t[n]

    Dl = 0.5*D*theta
    Dr = 0.5*D*(1-theta)

    # Representation of sparse matrix and right-hand side
    diagonal = zeros(Nx+1)
    lower    = zeros(Nx)
    upper    = zeros(Nx)
    b        = zeros(Nx+1)

    # Precompute sparse matrix (scipy format)
    diagonal[1:-1] = 1 + Dl*(a[2:] + 2*a[1:-1] + a[:-2])
    lower[:-1] = -Dl*(a[1:-1] + a[:-2])
    upper[1:]  = -Dl*(a[2:] + a[1:-1])
    # Insert boundary conditions
    diagonal[0] = 1
    upper[0] = 0
    diagonal[Nx] = 1
    lower[-1] = 0

    A = scipy.sparse.diags(
        diagonals=[diagonal, lower, upper],
        offsets=[0, -1, 1],
        shape=(Nx+1, Nx+1),
        format='csr')

    # Set initial condition
    for i in range(0,Nx+1):
        u_n[i] = I(x[i])
```

```
    if user_action is not None:
        user_action(u_n, x, t, 0)

    # Time loop
    for n in range(0, Nt):
        b[1:-1] = u_n[1:-1] + Dr*(
            (a[2:] + a[1:-1])*(u_n[2:] - u_n[1:-1]) -
            (a[1:-1] + a[0:-2])*(u_n[1:-1] - u_n[:-2]))
        # Boundary conditions
        b[0]  = u_L(t[n+1])
        b[-1] = u_R(t[n+1])
        # Solve
        u[:] = scipy.sparse.linalg.spsolve(A, b)

        if user_action is not None:
            user_action(u, x, t, n+1)

        # Switch variables before next step
        u_n, u = u, u_n
```

The code is found in the file `diffu1D_vc.py`.

3.5.3 Stationary solution

As $t \to \infty$, the solution of the problem (3.69)-(3.72) will approach a stationary limit where $\partial u/\partial t = 0$. The governing equation is then

$$\frac{d}{dx}\left(\alpha \frac{du}{dx}\right) = 0, \tag{3.73}$$

with boundary conditions $u(0) = U_0$ and $u(L) = U_L$. It is possible to obtain an exact solution of (3.73) for any α. Integrating twice and applying the boundary conditions to determine the integration constants gives

$$u(x) = U_0 + (U_L - U_0)\frac{\int_0^x (\alpha(\xi))^{-1} d\xi}{\int_0^L (\alpha(\xi))^{-1} d\xi}\,. \tag{3.74}$$

3.5.4 Piecewise constant medium

Consider a medium built of M layers. The layer boundaries are denoted $b_0, \ldots, b_M$, where $b_0 = 0$ and $b_M = L$. If the layers potentially have different material properties, but these properties are constant within each layer, we can express α as a *piecewise constant function* according to

$$\alpha(x) = \begin{cases} \alpha_0, & b_0 \leq x < b_1, \\ \vdots & \\ \alpha_i, & b_i \leq x < b_{i+1}, \\ \vdots & \\ \alpha_{M-1}, & b_{M-1} \leq x \leq b_M. \end{cases} \tag{3.75}$$

The exact solution (3.74) in case of such a piecewise constant α function is easy to derive. Assume that x is in the m-th layer: $x \in [b_m, b_{m+1}]$. In the integral $\int_0^x (a(\xi))^{-1} d\xi$ we must integrate through the first $m-1$ layers and then add the contribution from the remaining part $x - b_m$ into the m-th layer:

$$u(x) = U_0 + (U_L - U_0) \frac{\sum_{j=0}^{m-1} (b_{j+1} - b_j)/\alpha(b_j) + (x - b_m)/\alpha(b_m)}{\sum_{j=0}^{M-1} (b_{j+1} - b_j)/\alpha(b_j)} \tag{3.76}$$

Remark. It may sound strange to have a discontinuous α in a differential equation where one is to differentiate, but a discontinuous α is compensated by a discontinuous u_x such that αu_x is continuous and therefore can be differentiated as $(\alpha u_x)_x$.

3.5.5 Implementation of diffusion in a piecewise constant medium

Programming with piecewise function definitions quickly becomes cumbersome as the most naive approach is to test for which interval x lies, and then start evaluating a formula like (3.76). In Python, vectorized expressions may help to speed up the computations. The convenience classes `PiecewiseConstant` and `IntegratedPiecewiseConstant` in the `Heaviside` module were made to simplify programming with functions like (3.5.4) and expressions like (3.76). These utilities not only represent piecewise constant functions, but also *smoothed* versions of them where the discontinuities can be smoothed out in a controlled fashion.

The `PiecewiseConstant` class is created by sending in the domain as a 2-tuple or 2-list and a `data` object describing the boundaries $b_0, \ldots, b_M$ and the corresponding function values $\alpha_0, \ldots, \alpha_{M-1}$. More precisely, `data` is a nested list, where `data[i][0]` holds b_i and `data[i][1]` holds the

corresponding value α_i, for $i = 0, \ldots, M-1$. Given b_i and α_i in arrays `b` and `a`, it is easy to fill out the nested list `data`.

In our application, we want to represent α and $1/\alpha$ as piecewise constant functions, in addition to the $u(x)$ function which involves the integrals of $1/\alpha$. A class creating the functions we need and a method for evaluating u, can take the form

```
class SerialLayers:
    """
    b: coordinates of boundaries of layers, b[0] is left boundary
    and b[-1] is right boundary of the domain [0,L].
    a: values of the functions in each layer (len(a) = len(b)-1).
    U_0: u(x) value at left boundary x=0=b[0].
    U_L: u(x) value at right boundary x=L=b[0].
    """

    def __init__(self, a, b, U_0, U_L, eps=0):
        self.a, self.b = np.asarray(a), np.asarray(b)
        self.eps = eps  # smoothing parameter for smoothed a
        self.U_0, self.U_L = U_0, U_L

        a_data = [[bi, ai] for bi, ai in zip(self.b, self.a)]
        domain = [b[0], b[-1]]
        self.a_func = PiecewiseConstant(domain, a_data, eps)

        # inv_a = 1/a is needed in formulas
        inv_a_data = [[bi, 1./ai] for bi, ai in zip(self.b, self.a)]
        self.inv_a_func = \
             PiecewiseConstant(domain, inv_a_data, eps)
        self.integral_of_inv_a_func = \
             IntegratedPiecewiseConstant(domain, inv_a_data, eps)
        # Denominator in the exact formula is constant
        self.inv_a_0L = self.integral_of_inv_a_func(b[-1])

    def __call__(self, x):
        solution = self.U_0 + (self.U_L-self.U_0)*\
                   self.integral_of_inv_a_func(x)/self.inv_a_0L
        return solution
```

A visualization method is also convenient to have. Below we plot $u(x)$ along with $\alpha(x)$ (which works well as long as $\max \alpha(x)$ is of the same size as $\max u = \max(U_0, U_L)$).

```
class SerialLayers:
    ...

    def plot(self):
        x, y_a = self.a_func.plot()
        x = np.asarray(x); y_a = np.asarray(y_a)
        y_u = self.u_exact(x)
        import matplotlib.pyplot as plt
```

```
        plt.figure()
        plt.plot(x, y_u, 'b')
        plt.hold('on')  # Matlab style
        plt.plot(x, y_a, 'r')
        ymin = -0.1
        ymax = 1.2*max(y_u.max(), y_a.max())
        plt.axis([x[0], x[-1], ymin, ymax])
        plt.legend(['solution $u$', 'coefficient $a$'], loc='upper left')
        if self.eps > 0:
            plt.title('Smoothing eps: %s' % self.eps)
        plt.savefig('tmp.pdf')
        plt.savefig('tmp.png')
        plt.show()
```

Figure 3.12 shows the case where

```
b = [0, 0.25, 0.5, 1]   # material boundaries
a = [0.2, 0.4, 4]       # material values
U_0 = 0.5;  U_L = 5     # boundary conditions
```

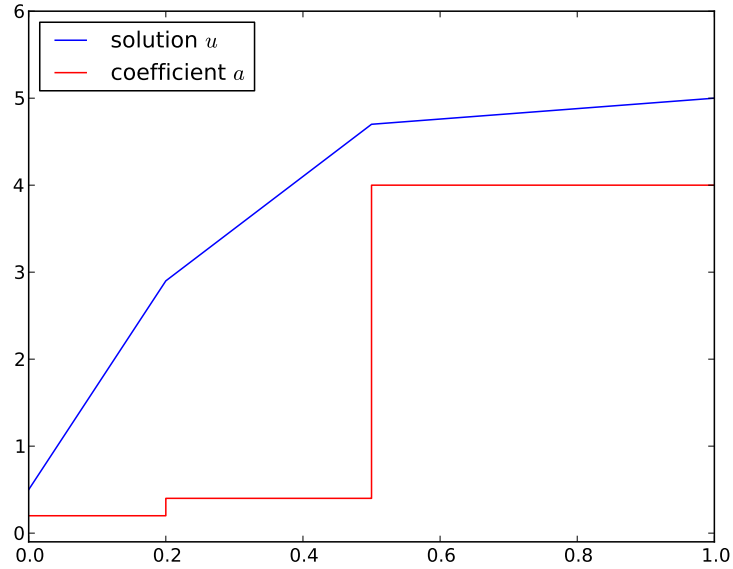


Fig. 3.12 Solution of the stationary diffusion equation corresponding to a piecewise constant diffusion coefficient.

By adding the `eps` parameter to the constructor of the `SerialLayers` class, we can experiment with smoothed versions of α and see the (small) impact on u. Figure 3.13 shows the result.

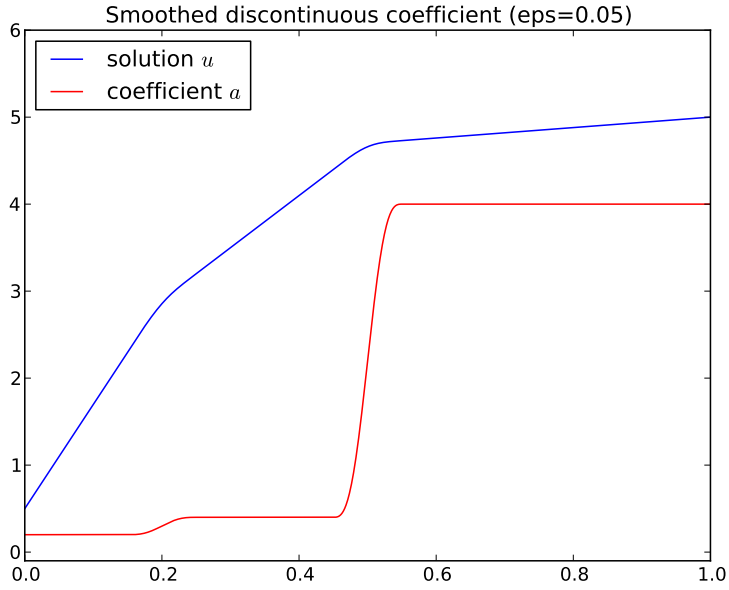


Fig. 3.13 Solution of the stationary diffusion equation corresponding to a *smoothed* piecewise constant diffusion coefficient.

3.5.6 Axi-symmetric diffusion

Suppose we have a diffusion process taking place in a straight tube with radius R. We assume axi-symmetry such that u is just a function of r and t, with r being the radial distance from the center axis of the tube to a point. With such axi-symmetry it is advantageous to introduce *cylindrical coordinates* r, θ, and z, where z is in the direction of the tube and (r, θ) are polar coordinates in a cross section. Axi-symmetry means that all quantities are independent of θ. From the relations $x = \cos\theta$, $y = \sin\theta$, and $z = z$, between Cartesian and cylindrical coordinates, one can (with some effort) derive the diffusion equation in cylindrical coordinates, which with axi-symmetry takes the form

$$\frac{\partial u}{\partial t} = \frac{1}{r}\frac{\partial}{\partial r}\left(r\alpha(r,z)\frac{\partial u}{\partial r}\right) + \frac{\partial}{\partial z}\left(\alpha(r,z)\frac{\partial u}{\partial z}\right) + f(r,z,t)\,.$$

Let us assume that u does not change along the tube axis so it suffices to compute variations in a cross section. Then $\partial u/\partial z = 0$ and we have a

1D diffusion equation in the radial coordinate r and time t. In particular, we shall address the initial-boundary value problem

$$\frac{\partial u}{\partial t} = \frac{1}{r}\frac{\partial}{\partial r}\left(r\alpha(r)\frac{\partial u}{\partial r}\right) + f(t), r \in (0,R),\ t \in (0,T], \tag{3.77}$$

$$\frac{\partial u}{\partial r}(0,t) = 0, \qquad t \in (0,T], \tag{3.78}$$

$$u(R,t) = 0, \qquad t \in (0,T], \tag{3.79}$$

$$u(r,0) = I(r), \qquad r \in [0,R]. \tag{3.80}$$

The condition (3.78) is a necessary symmetry condition at $r = 0$, while (3.79) could be any Dirichlet or Neumann condition (or Robin condition in case of cooling or heating).

The finite difference approximation will need the discretized version of the PDE for $r = 0$ (just as we use the PDE at the boundary when implementing Neumann conditions). However, discretizing the PDE at $r = 0$ poses a problem because of the $1/r$ factor. We therefore need to work out the PDE for discretization at $r = 0$ with care. Let us, for the case of constant α, expand the spatial derivative term to

$$\alpha\frac{\partial^2 u}{\partial r^2} + \alpha\frac{1}{r}\frac{\partial u}{\partial r}.$$

The last term faces a difficulty at $r = 0$, since it becomes a $0/0$ expression caused by the symmetry condition at $r = 0$. However, L'Hosptial's rule can be used:

$$\lim_{r\to 0}\frac{1}{r}\frac{\partial u}{\partial r} = \frac{\partial^2 u}{\partial r^2}.$$

The PDE at $r = 0$ therefore becomes

$$\frac{\partial u}{\partial t} = 2\alpha\frac{\partial^2 u}{\partial r^2} + f(t). \tag{3.81}$$

For a variable coefficient $\alpha(r)$ the expanded spatial derivative term reads

$$\alpha(r)\frac{\partial^2 u}{\partial r^2} + \frac{1}{r}(\alpha(r) + r\alpha'(r))\frac{\partial u}{\partial r}.$$

We are interested in this expression for $r = 0$. A necessary condition for u to be axi-symmetric is that all input data, including α, must also

be axi-symmetric, implying that $\alpha'(0) = 0$ (the second term vanishes anyway because of $r = 0$). The limit of interest is

$$\lim_{r\to 0} \frac{1}{r}\alpha(r)\frac{\partial u}{\partial r} = \alpha(0)\frac{\partial^2 u}{\partial r^2}\,.$$

The PDE at $r = 0$ now looks like

$$\frac{\partial u}{\partial t} = 2\alpha(0)\frac{\partial^2 u}{\partial r^2} + f(t), \tag{3.82}$$

so there is no essential difference between the constant coefficient and variable coefficient cases.

The second-order derivative in (3.81) and (3.82) is discretized in the usual way.

$$2\alpha\frac{\partial^2}{\partial r^2}u(r_0, t_n) \approx [2\alpha D_r D_r u]_0^n = 2\alpha\frac{u_1^n - 2u_0^n + u_{-1}^n}{\Delta r^2}\,.$$

The fictitious value u_{-1}^n can be eliminated using the discrete symmetry condition

$$[D_{2r}u = 0]_0^n \quad \Rightarrow \quad u_{-1}^n = u_1^n,$$

which then gives the modified approximation to the term with the second-order derivative of u in r at $r = 0$:

$$4\alpha\frac{u_1^n - u_0^n}{\Delta r^2}\,. \tag{3.83}$$

The discretization of the term with the second-order derivative in r at any internal mesh point is straightforward:

$$\begin{aligned}\left[\frac{1}{r}\frac{\partial}{\partial r}\left(r\alpha\frac{\partial u}{\partial r}\right)\right]_i^n &\approx [r^{-1}D_r(r\alpha D_r u)]_i^n \\ &= \frac{1}{r_i}\frac{1}{\Delta r^2}\left(r_{i+\frac{1}{2}}\alpha_{i+\frac{1}{2}}(u_{i+1}^n - u_i^n) - r_{i-\frac{1}{2}}\alpha_{i-\frac{1}{2}}(u_i^n - u_{i-1}^n)\right)\,.\end{aligned}$$

To complete the discretization, we need a scheme in time, but that can be done as before and does not interfere with the discretization in space.

3.5.7 Spherically-symmetric diffusion

Discretization in spherical coordinates. Let us now pose the problem from Section 3.5.6 in spherical coordinates, where u only depends on the radial coordinate r and time t. That is, we have spherical symmetry. For simplicity we restrict the diffusion coefficient α to be a constant. The PDE reads

$$\frac{\partial u}{\partial t} = \frac{\alpha}{r^\gamma}\frac{\partial}{\partial r}\left(r^\gamma \frac{\partial u}{\partial r}\right) + f(t), \tag{3.84}$$

for $r \in (0, R)$ and $t \in (0, T]$. The parameter γ is 2 for spherically-symmetric problems and 1 for axi-symmetric problems. The boundary and initial conditions have the same mathematical form as in (3.77)-(3.80).

Since the PDE in spherical coordinates has the same form as the PDE in Section 3.5.6, just with the γ parameter being different, we can use the same discretization approach. At the origin $r = 0$ we get problems with the term

$$\frac{\gamma}{r}\frac{\partial u}{\partial t},$$

but L'Hosptial's rule shows that this term equals $\gamma\partial^2 u/\partial r^2$, and the PDE at $r = 0$ becomes

$$\frac{\partial u}{\partial t} = (\gamma + 1)\alpha\frac{\partial^2 u}{\partial r^2} + f(t)\,. \tag{3.85}$$

The associated discrete form is then

$$[D_t u = \frac{1}{2}(\gamma + 1)\alpha D_r D_r \overline{u}^t + \overline{f}^t]_i^{n+\frac{1}{2}}, \tag{3.86}$$

for a Crank-Nicolson scheme.

Discretization in Cartesian coordinates. The spherically-symmetric spatial derivative can be transformed to the Cartesian counterpart by introducing

$$v(r, t) = ru(r, t)\,.$$

Inserting $u = v/r$ in

$$\frac{1}{r^2}\frac{\partial}{\partial r}\left(\alpha(r)r^2\frac{\partial u}{\partial r}\right),$$

yields

$$r\left(\frac{d\alpha}{dr}\frac{\partial v}{\partial r} + \alpha\frac{\partial^2 v}{\partial r^2}\right) - \frac{d\alpha}{dr}v\,.$$

The two terms in the parenthesis can be combined to

$$r\frac{\partial}{\partial r}\left(\alpha\frac{\partial v}{\partial r}\right)\,.$$

The PDE for v takes the form

$$\frac{\partial v}{\partial t} = \frac{\partial}{\partial r}\left(\alpha\frac{\partial v}{\partial r}\right) - \frac{1}{r}\frac{d\alpha}{dr}v + rf(r,t), \quad r\in(0,R),\ t\in(0,T]\,. \tag{3.87}$$

For α constant we immediately realize that we can reuse a solver in Cartesian coordinates to compute v. With variable α, a "reaction" term v/r needs to be added to the Cartesian solver. The boundary condition $\partial u/\partial r = 0$ at $r = 0$, implied by symmetry, forces $v(0,t) = 0$, because

$$\frac{\partial u}{\partial r} = \frac{1}{r^2}\left(r\frac{\partial v}{\partial r} - v\right) = 0, \quad r = 0\,.$$

3.6 Diffusion in 2D

We now address diffusion in two space dimensions:

$$\frac{\partial u}{\partial t} = \alpha\left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial x^2}\right) + f(x,y), \tag{3.88}$$

in a domain

$$(x,y)\in(0,L_x)\times(0,L_y),\ t\in(0,T],$$

with $u = 0$ on the boundary and $u(x,y,0) = I(x,y)$ as initial condition.

3.6.1 Discretization

For generality, it is natural to use a θ-rule for the time discretization. Standard, second-order accurate finite differences are used for the spatial

derivatives. We sample the PDE at a space-time point $(i, j, n + \frac{1}{2})$ and apply the difference approximations:

$$[D_t u]^{n+\frac{1}{2}} = \theta[\alpha(D_x D_x u + D_y D_y u) + f]^{n+1} + (1-\theta)[\alpha(D_x D_x u + D_y D_y u) + f]^n . \tag{3.89}$$

Written out,

$$\frac{u_{i,j}^{n+1} - u_{i,j}^n}{\Delta t} = \theta(\alpha(\frac{u_{i-1,j}^{n+1} - 2u_{i,j}^{n+1} + u_{i+1,j}^{n+1}}{\Delta x^2} + \frac{u_{i,j-1}^{n+1} - 2u_{i,j}^{n+1} + u_{i,j+1}^{n+1}}{\Delta y^2}) + f_{i,j}^{n+1}) + (1-\theta)(\alpha(\frac{u_{i-1,j}^n - 2u_{i,j}^n + u_{i+1,j}^n}{\Delta x^2} + \frac{u_{i,j-1}^n - 2u_{i,j}^n + u_{i,j+1}^n}{\Delta y^2}) + f_{i,j}^n) \tag{3.90}$$

We collect the unknowns on the left-hand side

$$u_{i,j}^{n+1} - \theta\left(F_x(u_{i-1,j}^{n+1} - 2u_{i,j}^{n+1} + u_{i+1,j}^{n+1}) + F_y(u_{i,j-1}^{n+1} - 2u_{i,j}^{n+1} + u_{i,j+1}^{n+1})\right) = (1-\theta)\left(F_x(u_{i-1,j}^n - 2u_{i,j}^n + u_{i+1,j}^n) + F_y(u_{i,j-1}^n - 2u_{i,j}^n + u_{i,j+1}^n)\right) + \theta\Delta t f_{i,j}^{n+1} + (1-\theta)\Delta t f_{i,j}^n + u_{i,j}^n, \tag{3.91}$$

where

$$F_x = \frac{\alpha\Delta t}{\Delta x^2}, \quad F_y = \frac{\alpha\Delta t}{\Delta y^2},$$

are the Fourier numbers in x and y direction, respectively.

3.6.2 Numbering of mesh points versus equations and unknowns

The equations (3.91) are coupled at the new time level $n+1$. That is, we must solve a system of (linear) algebraic equations, which we will write as $Ac = b$, where A is the coefficient matrix, c is the vector of unknowns, and b is the right-hand side.

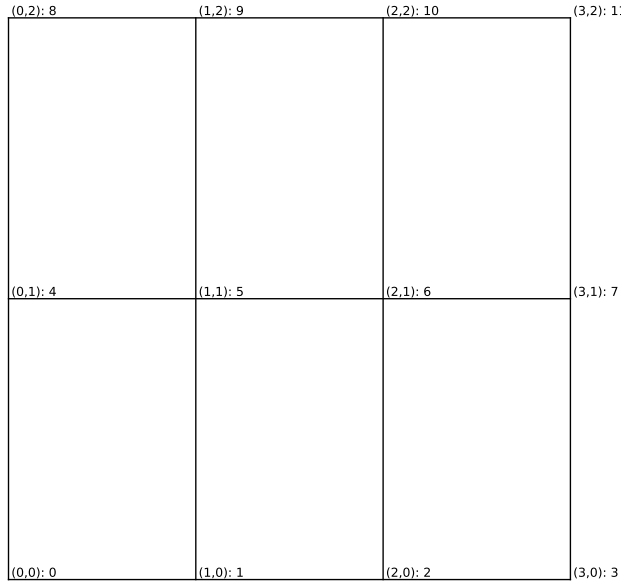


Fig. 3.14 3x2 2D mesh.

Let us examine the equations in $Ac = b$ on a mesh with $N_x = 3$ and $N_y = 2$ cells in the respective spatial directions. The spatial mesh is depicted in Figure 3.14. The equations at the boundary just implement the boundary condition $u = 0$:

$$u_{0,0}^{n+1} = u_{1,0}^{n+1} = u_{2,0}^{n+1} = u_{3,0}^{n+1} = u_{0,1}^{n+1} =$$
$$u_{3,1}^{n+1} = u_{0,2}^{n+1} = u_{1,2}^{n+1} = u_{2,2}^{n+1} = u_{3,2}^{n+1} = 0\,.$$

We are left with two interior points, with $i = 1$, $j = 1$ and $i = 2$, $j = 1$. The corresponding equations are

$$\begin{aligned} u_{i,j}^{n+1} - \theta \left(F_x(u_{i-1,j}^{n+1} - 2u_{i,j}^{n+1} + u_{i+1,j}^{n+1}) + F_y(u_{i,j-1}^{n+1} - 2u_{i,j}^{n+1} + u_{i,j+1}^{n+1}) \right) = \\ (1-\theta) \left(F_x(u_{i-1,j}^{n} - 2u_{i,j}^{n} + u_{i+1,j}^{n}) + F_y(u_{i,j-1}^{n} - 2u_{i,j}^{n} + u_{i,j+1}^{n}) \right) + \\ \theta \Delta t f_{i,j}^{n+1} + (1-\theta)\Delta t f_{i,j}^{n} + u_{i,j}^{n}, \end{aligned}$$

There are in total 12 unknowns $u_{i,j}^{n+1}$ for $i = 0, 1, 2, 3$ and $j = 0, 1, 2$. To solve the equations, we need to form a matrix system $Ac = b$. In that system, the solution vector c can only have one index. Thus, we need a numbering of the unknowns with one index, not two as used in the mesh.

We introduce a mapping $m(i,j)$ from a mesh point with indices (i,j) to the corresponding unknown p in the equation system:

$$p = m(i,j) = j(N_x+1) + i\,.$$

When i and j run through their values, we see the following mapping to p:

$$\begin{aligned}
&(0,0) \to 0,\ (0,1) \to 1,\ (0,2) \to 2,\ (0,3) \to 3,\\
&(1,0) \to 4,\ (1,1) \to 5,\ (1,2) \to 6,\ (1,3) \to 7,\\
&(2,0) \to 8,\ (2,1) \to 9,\ (2,2) \to 10,\ (2,3) \to 11\,.
\end{aligned}$$

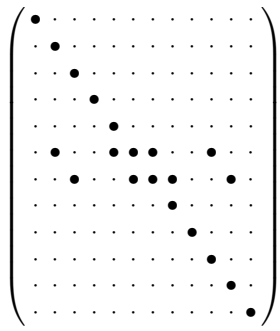
That is, we number the points along the x axis, starting with $y = 0$, and then progress one "horizontal" mesh line at a time. In Figure 3.14 you can see that the (i,j) and the corresponding single index (p) are listed for each mesh point.

We could equally well have numbered the equations in other ways, e.g., let the j index be the fastest varying index: $p = m(i,j) = i(N_y+1) + j$.

Let us form the coefficient matrix A, or more precisely, insert a matrix element (according Python's convention with zero as base index) for each of the nonzero elements in A (the indices run through the values of p, i.e., $p = 0, \ldots, 11$):

$$\begin{pmatrix}
(0,0) & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0\\
0 & (1,1) & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0\\
0 & 0 & (2,2) & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0\\
0 & 0 & 0 & (3,3) & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0\\
0 & 0 & 0 & 0 & (4,4) & 0 & 0 & 0 & 0 & 0 & 0 & 0\\
0 & (5,1) & 0 & 0 & (5,4) & (5,5) & (5,6) & 0 & 0 & (5,9) & 0 & 0\\
0 & 0 & (6,2) & 0 & 0 & (6,5) & (6,6) & (6,7) & 0 & 0 & (6,10) & 0\\
0 & 0 & 0 & 0 & 0 & 0 & 0 & (7,7) & 0 & 0 & 0 & 0\\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & (8,8) & 0 & 0 & 0\\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & (9,9) & 0 & 0\\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & (10,10) & 0\\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & (11,11)
\end{pmatrix}$$

Here is a more compact visualization of the coefficient matrix where we insert dots for zeros and bullets for non-zero elements:



It is clearly seen that most of the elements are zero. This is a general feature of coefficient matrices arising from discretizing PDEs by finite difference methods. We say that the matrix is *sparse*.

Let $A_{p,q}$ be the value of element (p, q) in the coefficient matrix A, where p and q now correspond to the numbering of the unknowns in the equation system. We have $A_{p,q} = 1$ for $p = q = 0, 1, 2, 3, 4, 7, 8, 9, 10, 11$, corresponding to all the known boundary values. Let p be $m(i, j)$, i.e., the single index corresponding to mesh point (i, j). Then we have

$$A_{m(i,j),m(i,j)} = A_{p,p} = 1 + \theta(F_x + F_y), \tag{3.92}$$

$$A_{p,m(i-1,j)} = A_{p,p-1} = -\theta F_x, \tag{3.93}$$

$$A_{p,m(i+1,j)} = A_{p,p+1} = -\theta F_x, \tag{3.94}$$

$$A_{p,m(i,j-1)} = A_{p,p-(N_x+1)} = -\theta F_y, \tag{3.95}$$

$$A_{p,m(i,j+1)} = A_{p,p+(N_x+1)} = -\theta F_y, \tag{3.96}$$

for the equations associated with the two interior mesh points. At these interior points, the single index p takes on the specific values $p = 5, 6$, corresponding to the values $(1, 1)$ and $(1, 2)$ of the pair (i, j).

The above values for $A_{p,q}$ can be inserted in the matrix:

$$\begin{pmatrix}
1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & -\theta F_y & 0 & 0 & -\theta F_x & 1+2\theta F_x & -\theta F_x & 0 & 0 & -\theta F_y & 0 & 0 \\
0 & 0 & -\theta F_y & 0 & 0 & -\theta F_x & 1+2\theta F_x & -\theta F_x & 0 & 0 & -\theta F_y & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1
\end{pmatrix}$$

The corresponding right-hand side vector in the equation system has the entries b_p, where p numbers the equations. We have

$$b_0 = b_1 = b_2 = b_3 = b_4 = b_7 = b_8 = b_9 = b_{10} = b_{11} = 0,$$

for the boundary values. For the equations associated with the interior points, we get for $p = 5, 6$, corresponding to $i = 1, 2$ and $j = 1$:

$$b_p = u^n_{i,j} + (1-\theta)\left(F_x(u^n_{i-1,j} - 2u^n_{i,j} + u^n_{i+1,j}) + F_y(u^n_{i,j-1} - 2u^n_{i,j} + u^n_{i,j+1})\right) + \theta\Delta t f^{n+1}_{i,j} + (1-\theta)\Delta t f^n_{i,j}\,.$$

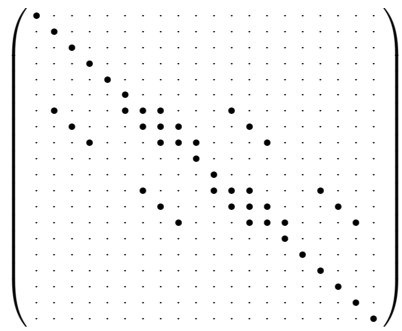
Recall that $p = m(i, j) = j(N_x + 1) + j$ in this expression.

We can, as an alternative, leave the boundary mesh points out of the matrix system. For a mesh with $N_x = 3$ and $N_y = 2$ there are only two internal mesh points whose unknowns will enter the matrix system. We must now number the unknowns at the interior points:

$$p = (j-1)(N_x - 1) + i,$$

for $i = 1, \ldots, N_x - 1$, $j = 1, \ldots, N_y - 1$.

We can continue with illustrating a bit larger mesh, $N_x = 4$ and $N_y = 3$, see Figure 3.15. The corresponding coefficient matrix with dots for zeros and bullets for non-zeroes looks as follows (values at boundary points are included in the equation system):



The coefficient matrix is banded

Besides being sparse, we observe that the coefficient matrix is *banded*: it has five distinct bands. We have the diagonal $A_{i,i}$, the subdiagonal $A_{i-1,j}$, the superdiagonal $A_{i,i+1}$, a lower diagonal $A_{i,i-(Nx+1)}$, and an upper diagonal $A_{i,i+(Nx+1)}$. The other matrix entries are known to be zero. With $N_x + 1 = N_y + 1 = N$, only a fraction $5N^{-2}$ of the matrix entries are nonzero, so the matrix is clearly very sparse

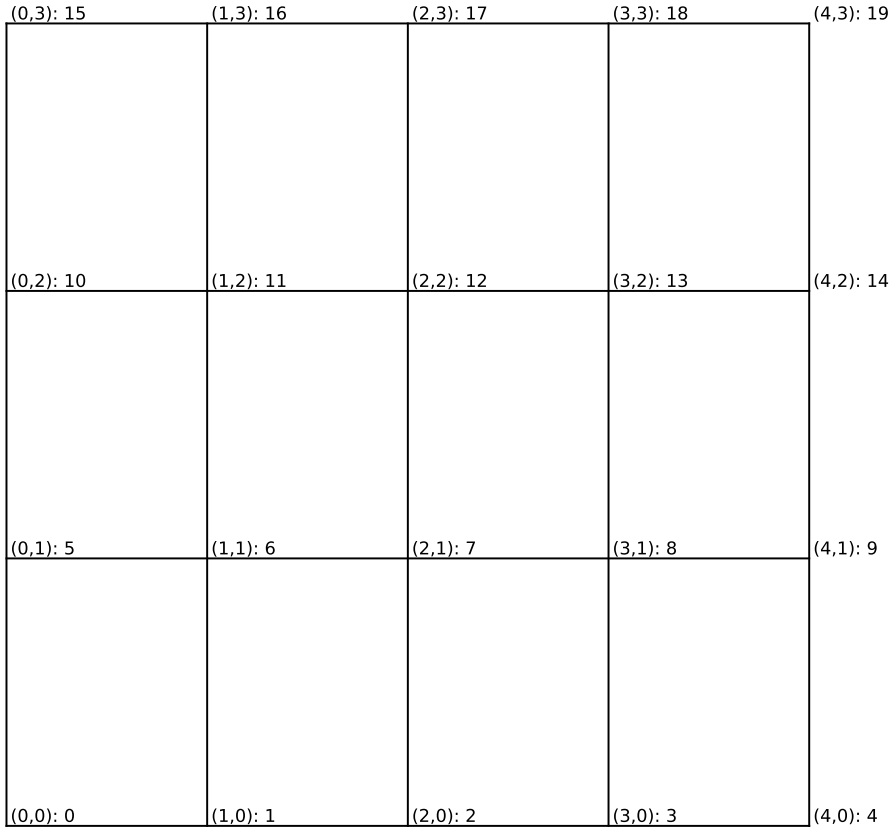


Fig. 3.15 4x3 2D mesh.

for relevant N values. The more we can compute with the nonzeros only, the faster the solution methods will potentially be.

3.6.3 Algorithm for setting up the coefficient matrix

We looked at a specific mesh in the previous section, formulated the equations, and saw what the corresponding coefficient matrix and right-hand side are. Now our aim is to set up a general algorithm, for any choice of N_x and N_y, that produces the coefficient matrix and the right-hand side vector. We start with a zero matrix and vector, run through each mesh point, and fill in the values depending on whether the mesh point is an interior point or on the boundary.

- for $i = 0, \ldots, N_x$
  - for $j = 0, \ldots, N_y$
    - $p = j(N_x + 1) + i$
    - if point (i, j) is on the boundary:
      - $A_{p,p} = 1$, $b_p = 0$
    - else:
      - fill $A_{p,m(i-1,j)}$, $A_{p,m(i+1,j)}$, $A_{p,m(i,j)}$, $A_{p,m(i,j-1)}$, $A_{p,m(i,j+1)}$, and b_p

To ease the test on whether (i, j) is on the boundary or not, we can split the loops a bit, starting with the boundary line $j = 0$, then treat the interior lines $1 \leq j < N_y$, and finally treat the boundary line $j = N_y$:

- for $i = 0, \ldots, N_x$
  - boundary $j = 0$: $p = j(N_x + 1) + i$, $A_{p,p} = 1$
- for $j = 0, \ldots, N_y$
  - boundary $i = 0$: $p = j(N_x + 1) + i$, $A_{p,p} = 1$
  - for $i = 1, \ldots, N_x - 1$
    - interior point $p = j(N_x + 1) + i$
    - fill $A_{p,m(i-1,j)}$, $A_{p,m(i+1,j)}$, $A_{p,m(i,j)}$, $A_{p,m(i,j-1)}$, $A_{p,m(i,j+1)}$, and b_p
  - boundary $i = N_x$: $p = j(N_x + 1) + i$, $A_{p,p} = 1$
- for $i = 0, \ldots, N_x$
  - boundary $j = N_y$: $p = j(N_x + 1) + i$, $A_{p,p} = 1$

The right-hand side is set up as follows.

- for $i = 0, \ldots, N_x$
  - boundary $j = 0$: $p = j(N_x + 1) + i$, $b_p = 0$
- for $j = 0, \ldots, N_y$
  - boundary $i = 0$: $p = j(N_x + 1) + i$, $b_p = 0$
  - for $i = 1, \ldots, N_x - 1$
    - interior point $p = j(N_x + 1) + i$
    - fill b_p
  - boundary $i = N_x$: $p = j(N_x + 1) + i$, $b_p = 0$

- for $i = 0, \ldots, N_x$
  - boundary $j = N_y$: $p = j(N_x + 1) + i$, $b_p = 0$

3.6.4 Implementation with a dense coefficient matrix

The goal now is to map the algorithms in the previous section to Python code. One should, for computational efficiency reasons, take advantage of the fact that the coefficient matrix is sparse and/or banded, i.e., take advantage of all the zeros. However, we first demonstrate how to fill an $N \times N$ dense square matrix, where N is the number of unknowns, here $N = (N_x + 1)(N_y + 1)$. The dense matrix is much easier to understand than the sparse matrix case.

```
import numpy as np

def solver_dense(
    I, a, f, Lx, Ly, Nx, Ny, dt, T, theta=0.5, user_action=None):
    """
    Solve u_t = a*(u_xx + u_yy) + f, u(x,y,0)=I(x,y), with u=0
    on the boundary, on [0,Lx]x[0,Ly]x[0,T], with time step dt,
    using the theta-scheme.
    """
    x = np.linspace(0, Lx, Nx+1)       # mesh points in x dir
    y = np.linspace(0, Ly, Ny+1)       # mesh points in y dir
    dx = x[1] - x[0]
    dy = y[1] - y[0]

    dt = float(dt)                     # avoid integer division
    Nt = int(round(T/float(dt)))
    t = np.linspace(0, Nt*dt, Nt+1)    # mesh points in time

    # Mesh Fourier numbers in each direction
    Fx = a*dt/dx**2
    Fy = a*dt/dy**2
```

The $u_{i,j}^{n+1}$ and $u_{i,j}^{n}$ mesh functions are represented by their spatial values at the mesh points:

```
u   = np.zeros((Nx+1, Ny+1))      # unknown u at new time level
u_n = np.zeros((Nx+1, Ny+1))      # u at the previous time level
```

It is a good habit (for extensions) to introduce index sets for all mesh points:

```
Ix = range(0, Nx+1)
Iy = range(0, Ny+1)
It = range(0, Nt+1)
```

The initial condition is easy to fill in:

```
# Load initial condition into u_n
for i in Ix:
    for j in Iy:
        u_n[i,j] = I(x[i], y[j])
```

The memory for the coefficient matrix and right-hand side vector is allocated by

```
N = (Nx+1)*(Ny+1)  # no of unknowns
A = np.zeros((N, N))
b = np.zeros(N)
```

The filling of `A` goes like this:

```
m = lambda i, j: j*(Nx+1) + i

# Equations corresponding to j=0, i=0,1,... (u known)
j = 0
for i in Ix:
    p = m(i,j);  A[p, p] = 1

# Loop over all internal mesh points in y diretion
# and all mesh points in x direction
for j in Iy[1:-1]:
    i = 0;  p = m(i,j);  A[p, p] = 1   # Boundary
    for i in Ix[1:-1]:                 # Interior points
        p = m(i,j)
        A[p, m(i,j-1)] = - theta*Fy
        A[p, m(i-1,j)] = - theta*Fx
        A[p, p]        = 1 + 2*theta*(Fx+Fy)
        A[p, m(i+1,j)] = - theta*Fx
        A[p, m(i,j+1)] = - theta*Fy
    i = Nx;  p = m(i,j);  A[p, p] = 1  # Boundary
# Equations corresponding to j=Ny, i=0,1,... (u known)
j = Ny
for i in Ix:
    p = m(i,j);  A[p, p] = 1
```

Since `A` is independent of time, it can be filled once and for all before the time loop. The right-hand side vector must be filled at each time level inside the time loop:

```
import scipy.linalg

for n in It[0:-1]:
    # Compute b
    j = 0
    for i in Ix:
        p = m(i,j);  b[p] = 0          # Boundary
    for j in Iy[1:-1]:
        i = 0;  p = m(i,j);  b[p] = 0  # Boundary
```

```
        for i in Ix[1:-1]:              # Interior points
            p = m(i,j)
            b[p] = u_n[i,j] + \
              (1-theta)*(
              Fx*(u_n[i+1,j] - 2*u_n[i,j] + u_n[i-1,j]) +\
              Fy*(u_n[i,j+1] - 2*u_n[i,j] + u_n[i,j-1]))\
                + theta*dt*f(i*dx,j*dy,(n+1)*dt) + \
              (1-theta)*dt*f(i*dx,j*dy,n*dt)
        i = Nx;  p = m(i,j);  b[p] = 0  # Boundary
    j = Ny
    for i in Ix:
        p = m(i,j);  b[p] = 0           # Boundary

    # Solve matrix system A*c = b
    c = scipy.linalg.solve(A, b)

    # Fill u with vector c
    for i in Ix:
        for j in Iy:
            u[i,j] = c[m(i,j)]

    # Update u_n before next step
    u_n, u = u, u_n
```

We use `solve` from `scipy.linalg` and not from `numpy.linalg`. The difference is stated below.

`scipy.linalg` versus `numpy.linalg`

Quote from the SciPy documentation[a]:

`scipy.linalg` contains all the functions in `numpy.linalg` plus some other more advanced ones not contained in `numpy.linalg`.

Another advantage of using `scipy.linalg` over numpy.linalg is that it is always compiled with BLAS/LAPACK support, while for NumPy this is optional. Therefore, the SciPy version might be faster depending on how NumPy was installed.

Therefore, unless you don't want to add SciPy as a dependency to your NumPy program, use `scipy.linalg` instead of `numpy.linalg`.

[a] `http://docs.scipy.org/doc/scipy/reference/tutorial/linalg.html`

The code shown above is available in the `solver_dense` function in the file `diffu2D_u0.py`, differing only in the boundary conditions, which in the code can be an arbitrary function along each side of the domain.

We do not bother to look at vectorized versions of filling `A` since a dense matrix is just used of pedagogical reasons for the very first

implementation. Vectorization will be treated when `A` has a sparse matrix representation, as in Section 3.6.7.

How to debug the computation of A and b

A good starting point for debugging the filling of A and b is to choose a very coarse mesh, say $N_x = N_y = 2$, where there is just one internal mesh point, compute the equations by hand, and print out `A` and `b` for comparison in the code. If wrong elements in `A` or `b` occur, print out each assignment to elements in `A` and `b` inside the loops and compare with what you expect.

To let the user store, analyze, or visualize the solution at each time level, we include a callback function, named `user_action`, to be called before the time loop and in each pass in that loop. The function has the signature

```
user_action(u, x, xv, y, yv, t, n)
```

where `u` is a two-dimensional array holding the solution at time level `n` and time `t[n]`. The x and y coordinates of the mesh points are given by the arrays `x` and `y`, respectively. The arrays `xv` and `yv` are vectorized representations of the mesh points such that vectorized function evaluations can be invoked. The `xv` and `yv` arrays are defined by

```
xv = x[:,np.newaxis]
yv = y[np.newaxis,:]
```

One can then evaluate, e.g., $f(x, y, t)$ at all internal mesh points at time level `n` by first evaluating f at all points,

```
f_a = f(xv, yv, t[n])
```

and then use slices to extract a view of the values at the internal mesh points: `f_a[1:-1,1:-1]`. The next section features an example on writing a `user_action` callback function.

3.6.5 Verification: exact numerical solution

A good test example to start with is one that preserves the solution $u = 0$, i.e., $f = 0$ and $I(x, y) = 0$. This trivial solution can uncover some bugs.

The first real test example is based on having an exact solution of the discrete equations. This solution is linear in time and quadratic in space:

$$u(x, y, t) = 5tx(L_x - x)y(y - L_y)\,.$$

Inserting this manufactured solution in the PDE shows that the source term f must be

$$f(x, y, t) = 5x(L_x - x)y(y - L_y) + 10\alpha t(x(L_x - x) + y(y - L_y))\,.$$

We can use the `user_action` function to compare the numerical solution with the exact solution at each time level. A suitable helper function for checking the solution goes like this:

```
def quadratic(theta, Nx, Ny):

    def u_exact(x, y, t):
        return 5*t*x*(Lx-x)*y*(Ly-y)
    def I(x, y):
        return u_exact(x, y, 0)
    def f(x, y, t):
        return 5*x*(Lx-x)*y*(Ly-y) + 10*a*t*(y*(Ly-y)+x*(Lx-x))

    # Use rectangle to detect errors in switching i and j in scheme
    Lx = 0.75
    Ly = 1.5
    a = 3.5
    dt = 0.5
    T = 2

    def assert_no_error(u, x, xv, y, yv, t, n):
        """Assert zero error at all mesh points."""
        u_e = u_exact(xv, yv, t[n])
        diff = abs(u - u_e).max()
        tol = 1E-12
        msg = 'diff=%g, step %d, time=%g' % (diff, n, t[n])
        print msg
        assert diff < tol, msg

    solver_dense(
        I, a, f, Lx, Ly, Nx, Ny,
        dt, T, theta, user_action=assert_no_error)
```

A true test function for checking the quadratic solution for several different meshes and θ values can take the form

```
def test_quadratic():
    # For each of the three schemes (theta = 1, 0.5, 0), a series of
    # meshes are tested (Nx > Ny and Nx < Ny)
```

```
    for theta in [1, 0.5, 0]:
        for Nx in range(2, 6, 2):
            for Ny in range(2, 6, 2):
                print 'testing for %dx%d mesh' % (Nx, Ny)
                quadratic(theta, Nx, Ny)
```

3.6.6 Verification: convergence rates

For 2D verification with convergence rate computations, the expressions and computations just build naturally on what we saw for 1D diffusion. Truncation error analysis and other forms of error analysis point to a numerical error formula like

$$E = C_t \Delta t^p + C_x \Delta x^2 + C_y \Delta y^2,$$

where p, C_t, C_x, and C_y are constants. Often, the analysis of a Crank-Nicolson method can show that $p = 2$, while the Forward and Backward Euler schemes have $p = 1$.

When checking the error formula empirically, we need to reduce it to a form $E = Ch^r$ with a single discretization parameter h and some rate r to be estimated. For the Backward Euler method, where $p = 1$, we can introduce a single discretization parameter according to

$$h = \Delta x^2 = \Delta y^2, \quad h = K^{-1}\Delta t,$$

where K is a constant. The error formula then becomes

$$E = C_t K h + C_x h + C_y h = \tilde{C} h, \quad \tilde{C} = C_t K + C_x + C_y\,.$$

The simplest choice is obviously $K = 1$. With the Forward Euler method, however, stability requires $\Delta t = hK \leq h/(4\alpha)$, so $K \leq 1/(4\alpha)$.

For the Crank-Nicolson method, $p = 2$, and we can simply choose

$$h = \Delta x = \Delta y = \Delta t,$$

since there is no restriction on Δt in terms of Δx and Δy.

A frequently used error measure is the ℓ^2 norm of the error mesh point values. Section 2.2.3 and the formula (2.26) shows the error measure for a 1D time-dependent problem. The extension to the current 2D problem reads

$$E = \left(\Delta t \Delta x \Delta y \sum_{n=0}^{N_t} \sum_{i=0}^{N_x} \sum_{j=0}^{N_y} (u_{\mathrm{e}}(x_i, y_j, t_n) - u^n_{i,j})^2 \right)^{\frac{1}{2}} .$$

One attractive manufactured solution is

$$u_{\mathrm{e}} = e^{-pt} \sin(k_x x) \sin(k_y y), \quad k_x = \frac{\pi}{L_x}, k_y = \frac{\pi}{L_y},$$

where p can be arbitrary. The required source term is

$$f = (\alpha(k_x^2 + k_y^2) - p) u_{\mathrm{e}} .$$

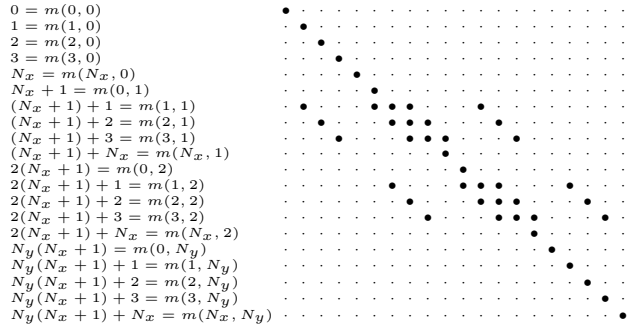
The function `convergence_rates` in `diffu2D_u0.py` implements a convergence rate test. Two potential difficulties are important to be aware of:

1. The error formula is assumed to be correct when $h \to 0$, so for coarse meshes the estimated rate r may be somewhat away from the expected value. Fine meshes may lead to prohibitively long execution times.
2. Choosing $p = \alpha(k_x^2 + k_y^2)$ in the manufactured solution above seems attractive ($f = 0$), but leads to a slower approach to the asymptotic range where the error formula is valid (i.e., r fluctuates and needs finer meshes to stabilize).

3.6.7 Implementation with a sparse coefficient matrix

We used a sparse matrix implementation in Section 3.2.2 for a 1D problem with a tridiagonal matrix. The present matrix, arising from a 2D problem, has five diagonals, but we can use the same sparse matrix data structure `scipy.sparse.diags`.

Understanding the diagonals. Let us look closer at the diagonals in the example with a 4×3 mesh as depicted in Figure 3.15 and its associated matrix visualized by dots for zeros and bullets for nonzeros. From the example mesh, we may generalize to an $N_x \times N_y$ mesh.



The main diagonal has $N = (N_x+1)(N_y+1)$ elements, while the sub- and super-diagonals have $N-1$ elements. By looking at the matrix above, we realize that the lower diagonal starts in row N_x+1 and goes to row N, so its length is $N-(N_x+1)$. Similarly, the upper diagonal starts at row 0 and lasts to row $N-(N_x+1)$, so it has the same length. Based on this information, we declare the diagonals by

```
main   = np.zeros(N)             # diagonal
lower  = np.zeros(N-1)           # subdiagonal
upper  = np.zeros(N-1)           # superdiagonal
lower2 = np.zeros(N-(Nx+1))      # lower diagonal
upper2 = np.zeros(N-(Nx+1))      # upper diagonal
b      = np.zeros(N)             # right-hand side
```

Filling the diagonals. We run through all mesh points and fill in elements on the various diagonals. The line of mesh points corresponding to $j = 0$ are all on the boundary, and only the main diagonal gets a contribution:

```
m = lambda i, j: j*(Nx+1) + i
j = 0; main[m(0,j):m(Nx+1,j)] = 1  # j=0 boundary line
```

Then we run through all interior $j =$ const lines of mesh points. The first and the last point on each line, $i = 0$ and $i = N_x$, correspond to boundary points:

```
for j in Iy[1:-1]:               # Interior mesh lines j=1,...,Ny-1
    i = 0;   main[m(i,j)] = 1
    i = Nx;  main[m(i,j)] = 1  # Boundary
```

For the interior mesh points $i = 1, \ldots, N_x - 1$ on a mesh line $y =$ const we can start with the main diagonal. The entries to be filled go from $i = 1$ to $i = N_x - 1$ so the relevant slice in the `main` vector is `m(1,j):m(Nx,j)`:

```
main[m(1,j):m(Nx,j)] = 1 + 2*theta*(Fx+Fy)
```

The `upper` array for the superdiagonal has its index 0 corresponding to row 0 in the matrix, and the array entries to be set go from $m(1,j)$ to $m(N_x-1,j)$:

```
upper[m(1,j):m(Nx,j)] = - theta*Fx
```

The subdiagonal (`lower` array), however, has its index 0 corresponding to row 1, so there is an offset of 1 in indices compared to the matrix. The first nonzero occurs (interior point) at a mesh line $j=$ const corresponding to matrix row $m(1,j)$, and the corresponding array index in `lower` is then $m(1,j)$. To fill the entries from $m(1,j)$ to $m(N_x-1,j)$ we set the following slice in `lower`:

```
lower_offset = 1
lower[m(1,j)-lower_offset:m(Nx,j)-lower_offset] = - theta*Fx
```

For the upper diagonal, its index 0 corresponds to matrix row 0, so there is no offset and we can set the entries correspondingly to `upper`:

```
upper2[m(1,j):m(Nx,j)] = - theta*Fy
```

The `lower2` diagonal, however, has its first index 0 corresponding to row N_x+1, so here we need to subtract the offset N_x+1:

```
lower2_offset = Nx+1
lower2[m(1,j)-lower2_offset:m(Nx,j)-lower2_offset] = - theta*Fy
```

We can now summarize the above code lines for setting the entries in the sparse matrix representation of the coefficient matrix:

```
lower_offset = 1
lower2_offset = Nx+1
m = lambda i, j: j*(Nx+1) + i

j = 0; main[m(0,j):m(Nx+1,j)] = 1  # j=0 boundary line
for j in Iy[1:-1]:              # Interior mesh lines j=1,...,Ny-1
    i = 0;   main[m(i,j)] = 1  # Boundary
    i = Nx;  main[m(i,j)] = 1  # Boundary
    # Interior i points: i=1,...,N_x-1
    lower2[m(1,j)-lower2_offset:m(Nx,j)-lower2_offset] = - theta*Fy
    lower[m(1,j)-lower_offset:m(Nx,j)-lower_offset] = - theta*Fx
    main[m(1,j):m(Nx,j)] = 1 + 2*theta*(Fx+Fy)
    upper[m(1,j):m(Nx,j)] = - theta*Fx
    upper2[m(1,j):m(Nx,j)] = - theta*Fy
j = Ny; main[m(0,j):m(Nx+1,j)] = 1  # Boundary line
```

The next task is to create the sparse matrix from these diagonals:

```
import scipy.sparse

A = scipy.sparse.diags(
```

```
    diagonals=[main, lower, upper, lower2, upper2],
    offsets=[0, -lower_offset, lower_offset,
             -lower2_offset, lower2_offset],
    shape=(N, N), format='csr')
```

Filling the right-hand side; scalar version. Setting the entries in the right-hand side is easier, since there are no offsets in the array to take into account. The right-hand side is in fact similar to the one previously shown, when we used a dense matrix representation (the right-hand side vector is, of course, independent of what type of representation we use for the coefficient matrix). The complete time loop goes as follows.

```
import scipy.sparse.linalg

for n in It[0:-1]:
    # Compute b
    j = 0
    for i in Ix:
        p = m(i,j);  b[p] = 0                       # Boundary
    for j in Iy[1:-1]:
        i = 0;  p = m(i,j);  b[p] = 0               # Boundary
        for i in Ix[1:-1]:
            p = m(i,j)                              # Interior
            b[p] = u_n[i,j] + \
              (1-theta)*(
              Fx*(u_n[i+1,j] - 2*u_n[i,j] + u_n[i-1,j]) +\
              Fy*(u_n[i,j+1] - 2*u_n[i,j] + u_n[i,j-1]))\
                + theta*dt*f(i*dx,j*dy,(n+1)*dt) + \
              (1-theta)*dt*f(i*dx,j*dy,n*dt)
        i = Nx;  p = m(i,j);  b[p] = 0              # Boundary
    j = Ny
    for i in Ix:
        p = m(i,j);  b[p] = 0                       # Boundary

    # Solve matrix system A*c = b
    c = scipy.sparse.linalg.spsolve(A, b)

    # Fill u with vector c
    for i in Ix:
        for j in Iy:
            u[i,j] = c[m(i,j)]

    # Update u_n before next step
    u_n, u = u, u_n
```

Filling the right-hand side; vectorized version. Since we use a sparse matrix and try to speed up the computations, we should examine the loops and see if some can be easily removed by vectorization. In the filling of A we have already used vectorized expressions at each $j = \text{const}$

line of mesh points. We can very easily do the same in the code above and remove the need for loops over the `i` index:

```
for n in It[0:-1]:
    # Compute b, vectorized version

    # Precompute f in array so we can make slices
    f_a_np1 = f(xv, yv, t[n+1])
    f_a_n   = f(xv, yv, t[n])

    j = 0; b[m(0,j):m(Nx+1,j)] = 0     # Boundary
    for j in Iy[1:-1]:
        i = 0;   p = m(i,j);  b[p] = 0 # Boundary
        i = Nx;  p = m(i,j);  b[p] = 0 # Boundary
        imin = Ix[1]
        imax = Ix[-1]  # for slice, max i index is Ix[-1]-1
        b[m(imin,j):m(imax,j)] = u_n[imin:imax,j] + \
              (1-theta)*(Fx*(
          u_n[imin+1:imax+1,j] -
        2*u_n[imin:imax,j] + \
          u_n[imin-1:imax-1,j]) +
                         Fy*(
          u_n[imin:imax,j+1] -
        2*u_n[imin:imax,j] +
          u_n[imin:imax,j-1])) + \
            theta*dt*f_a_np1[imin:imax,j] + \
          (1-theta)*dt*f_a_n[imin:imax,j]
    j = Ny;  b[m(0,j):m(Nx+1,j)] = 0 # Boundary

    # Solve matrix system A*c = b
    c = scipy.sparse.linalg.spsolve(A, b)

    # Fill u with vector c
    u[:,:] = c.reshape(Ny+1,Nx+1).T

    # Update u_n before next step
    u_n, u = u, u_n
```

The most tricky part of this code snippet is the loading of values from the one-dimensional array `c` into the two-dimensional array `u`. With our numbering of unknowns from left to right along "horizontal" mesh lines, the correct reordering of the one-dimensional array `c` as a two-dimensional array requires first a reshaping to an `(Ny+1,Nx+1)` two-dimensional array and then taking the transpose. The result is an `(Nx+1,Ny+1)` array compatible with `u` both in size and appearance of the function values.

The `spsolve` function in `scipy.sparse.linalg` is an efficient version of Gaussian elimination suited for matrices described by diagonals. The

algorithm is known as *sparse Gaussian elimination*, and `spsolve` calls up a well-tested C code called SuperLU[2].

The complete code utilizing `spsolve` is found in the `solver_sparse` function in the file `diffu2D_u0.py`.

Verification. We can easily extend the function `quadratic` from Section 3.6.5 to include a test of the `solver_sparse` function as well.

```
def quadratic(theta, Nx, Ny):
    ...
    t, cpu = solver_sparse(
        I, a, f, Lx, Ly, Nx, Ny,
        dt, T, theta, user_action=assert_no_error)
```

3.6.8 The Jacobi iterative method

So far we have created a matrix and right-hand side of a linear system $Ac = b$ and solved the system for c by calling an exact algorithm based on Gaussian elimination. A much simpler implementation, which requires no memory for the coefficient matrix A, arises if we solve the system by *iterative* methods. These methods are only approximate, and the core algorithm is repeated many times until the solution is considered to be converged.

Numerical scheme and linear system. To illustrate the idea of the Jacobi method, we simplify the numerical scheme to the Backward Euler case, $\theta = 1$, so there are fewer terms to write:

$$u^{n+1}_{i,j} - \Big(F_x(u^{n+1}_{i-1,j} - 2u^{n+1}_{i,j} + u^{n+1}_{i+1,j}) + F_y(u^{n+1}_{i,j-1} - 2u^{n+1}_{i,j} + u^{n+1}_{i,j+1})\Big) = u^n_{i,j} + \Delta t f^{n+1}_{i,j} \tag{3.97}$$

The idea of the *Jacobi* iterative method is to introduce an iteration, here with index r, where we in each iteration treat $u^{n+1}_{i,j}$ as unknown, but use values from the previous iteration for the other unknowns $u^{n+1}_{i\pm1,j\pm1}$.

Iterations. Let $u^{n+1,r}_{i,j}$ be the approximation to $u^{n+1}_{i,j}$ in iteration r, for all relevant i and j indices. We first solve with respect to $u^{n+1}_{i,j}$ to get the equation to solve:

[2] `http://crd-legacy.lbl.gov/ xiaoye/SuperLU/`

$$u_{i,j}^{n+1} = (1 + 2F_x + 2F_y)^{-1} \left(F_x(u_{i-1,j}^{n+1} + u_{i+1,j}^{n+1}) + F_y(u_{i,j-1}^{n+1} + u_{i,j+1}^{n+1}) \right) + u_{i,j}^{n} + \Delta t f_{i,j}^{n+1} \tag{3.98}$$

The iteration is introduced by using iteration index r, for computed values, on the right-hand side and $r+1$ (unknown in this iteration) on the left-hand side:

$$u_{i,j}^{n+1,r+1} = (1 + 2F_x + 2F_y)^{-1} \left(F_x(u_{i-1,j}^{n+1,r} + u_{i+1,j}^{n+1,r}) + F_y(u_{i,j-1}^{n+1,r} + u_{i,j+1}^{n+1,r}) \right) + u_{i,j}^{n} + \Delta t f_{i,j}^{n+1} \tag{3.99}$$

Initial guess. We start the iteration with the computed values at the previous time level:

$$u_{i,j}^{n+1,0} = u_{i,j}^{n}, \quad i = 0, \ldots, N_x, \ j = 0, \ldots, N_y \,. \tag{3.100}$$

Relaxation. A common technique in iterative methods is to introduce a *relaxation*, which means that the new approximation is a weighted mean of the approximation as suggested by the algorithm and the previous approximation. Naming the quantity on the left-hand side of (3.99) as $u_{i,j}^{n+1,*}$, a new approximation based on relaxation reads

$$u^{n+1,r+1} = \omega u_{i,j}^{n+1,*} + (1 - \omega) u_{i,j}^{n+1,r} \,. \tag{3.101}$$

Under-relaxation means $\omega < 1$, while over-relaxation has $\omega > 1$.

Stopping criteria. The iteration can be stopped when the change from one iteration to the next is sufficiently small ($\leq \epsilon$), using either an infinity norm,

$$\max_{i,j} \left| u_{i,j}^{n+1,r+1} - u_{i,j}^{n+1,r} \right| \leq \epsilon, \tag{3.102}$$

or an L^2 norm,

$$\left(\Delta x \Delta y \sum_{i,j} (u_{i,j}^{n+1,r+1} - u_{i,j}^{n+1,r})^2 \right)^{\frac{1}{2}} \leq \epsilon \,. \tag{3.103}$$

Another widely used criterion measures how well the equations are solved by looking at the residual (essentially $b - Ac^{r+1}$ if c^{r+1} is the approximation to the solution in iteration $r+1$). The residual, defined in terms of the finite difference stencil, is

$$R_{i,j} = u_{i,j}^{n+1,r+1} - (F_x(u_{i-1,j}^{n+1,r+1} - 2u_{i,j}^{n+1,r+1} + u_{i+1,j}^{n+1,r+1}) + F_y(u_{i,j-1}^{n+1,r+1} - 2u_{i,j}^{n+1,r+1} + u_{i,j+1}^{n+1,r+1})) - u_{i,j}^{n} - \Delta t f_{i,j}^{n+1} \tag{3.104}$$

One can then iterate until the norm of the mesh function $R_{i,j}$ is less than some tolerance:

$$\left(\Delta x \Delta y \sum_{i,j} R_{i,j}^2 \right)^{\frac{1}{2}} \leq \epsilon \,. \tag{3.105}$$

Code-friendly notation. To make the mathematics as close as possible to what we will write in a computer program, we may introduce some new notation: $u_{i,j}$ is a short notation for $u_{i,j}^{n+1,r+1}$, $u_{i,j}^-$ is a short notation for $u_{i,j}^{n+1,r}$, and $u_{i,j}^{(s)}$ denotes $u_{i,j}^{n+1-s}$. That is, $u_{i,j}$ is the unknown, $u_{i,j}^-$ is its most recently computed approximation, and s counts time levels backwards in time. The Jacobi method (3.99)) takes the following form with the new notation:

$$u_{i,j}^* = (1 + 2F_x + 2F_y)^{-1}((F_x(u_{i-1,j}^- + u_{i+1,j}^-) + F_y(u_{i,j-1}^- + u_{i,j+1}^-)) + u_{i,j}^{(1)} + \Delta t f_{i,j}^{n+1}) \tag{3.106}$$

Generalization of the scheme. We can also quite easily introduce the θ rule for discretization in time and write up the Jacobi iteration in that case as well:

$$u_{i,j}^* = (1 + 2\theta(F_x + F_y))^{-1}(\theta(F_x(u_{i-1,j}^- + u_{i+1,j}^-) + F_y(u_{i,j-1}^- + u_{i,j+1}^-)) + u_{i,j}^{(1)} + \theta\Delta t f_{i,j}^{n+1} + (1-\theta)\Delta t f_{i,j}^{n} + (1-\theta)(F_x(u_{i-1,j}^{(1)} - 2u_{i,j}^{(1)} + u_{i+1,j}^{(1)}) + F_y(u_{i,j-1}^{(1)} - 2u_{i,j}^{(1)} + u_{i,j+1}^{(1)}))) \,. \tag{3.107}$$

The final update of u applies relaxation:

$$u_{i,j} = \omega u_{i,j}^* + (1-\omega)u_{i,j}^- \,.$$

3.6.9 Implementation of the Jacobi method

The Jacobi method needs no coefficient matrix and right-hand side vector, but it needs an array for u in the previous iteration. We call this array `u_`, using the notation at the end of the previous section (at the same time level). The unknown itself is called `u`, while `u_n` is the computed solution one time level back in time. With a θ rule in time, the time loop can be coded like this:

```
for n in It[0:-1]:
    # Solve linear system by Jacobi iteration at time level n+1
    u_[:,:] = u_n  # Start value
    converged = False
    r = 0
    while not converged:
        if version == 'scalar':
            j = 0
            for i in Ix:
                u[i,j] = U_0y(t[n+1])           # Boundary
            for j in Iy[1:-1]:
                i = 0;   u[i,j] = U_0x(t[n+1])  # Boundary
                i = Nx;  u[i,j] = U_Lx(t[n+1])  # Boundary
        # Interior points
                for i in Ix[1:-1]:
                    u_new = 1.0/(1.0 + 2*theta*(Fx + Fy))*(theta*(
                        Fx*(u_[i+1,j] + u_[i-1,j]) +
                        Fy*(u_[i,j+1] + u_[i,j-1])) + \
                    u_n[i,j] + \
                    (1-theta)*(Fx*(
                    u_n[i+1,j] - 2*u_n[i,j] + u_n[i-1,j]) +
                      Fy*(
                    u_n[i,j+1] - 2*u_n[i,j] + u_n[i,j-1]))\
                      + theta*dt*f(i*dx,j*dy,(n+1)*dt) + \
                    (1-theta)*dt*f(i*dx,j*dy,n*dt))
                    u[i,j] = omega*u_new + (1-omega)*u_[i,j]
            j = Ny
            for i in Ix:
                u[i,j] = U_Ly(t[n+1])      # Boundary

        elif version == 'vectorized':
            j = 0;  u[:,j] = U_0y(t[n+1])  # Boundary
            i = 0;  u[i,:] = U_0x(t[n+1])  # Boundary
            i = Nx; u[i,:] = U_Lx(t[n+1])  # Boundary
            j = Ny; u[:,j] = U_Ly(t[n+1])  # Boundary
        # Internal points
            f_a_np1 = f(xv, yv, t[n+1])
            f_a_n   = f(xv, yv, t[n])
            u_new = 1.0/(1.0 + 2*theta*(Fx + Fy))*(theta*(Fx*(
              u_[2:,1:-1] + u_[:-2,1:-1]) +
                Fy*(
              u_[1:-1,2:] + u_[1:-1,:-2])) +\
            u_n[1:-1,1:-1] + \
```

```
                (1-theta)*(Fx*(
                u_n[2:,1:-1] - 2*u_n[1:-1,1:-1] + u_n[:-2,1:-1]) +\
                  Fy*(
                u_n[1:-1,2:] - 2*u_n[1:-1,1:-1] + u_n[1:-1,:-2]))\
                + theta*dt*f_a_np1[1:-1,1:-1] + \
                (1-theta)*dt*f_a_n[1:-1,1:-1])
            u[1:-1,1:-1] = omega*u_new + (1-omega)*u_[1:-1,1:-1]
        r += 1
        converged = np.abs(u-u_).max() < tol or r >= max_iter
        u_[:,:] = u

    # Update u_n before next step
    u_n, u = u, u_n
```

The vectorized version should be quite straightforward to understand once one has an understanding of how a standard 2D finite stencil is vectorized.

The first natural verification is to use the test problem in the function `quadratic` from Section 3.6.5. This problem is known to have no approximation error, but any iterative method will produce an approximate solution with unknown error. For a tolerance 10^{-k} in the iterative method, we can, e.g., use a slightly larger tolerance $10^{-(k-1)}$ for the difference between the exact and the computed solution.

```
def quadratic(theta, Nx, Ny):
    ...
    def assert_small_error(u, x, xv, y, yv, t, n):
        """Assert small error for iterative methods."""
        u_e = u_exact(xv, yv, t[n])
        diff = abs(u - u_e).max()
        tol = 1E-4
        msg = 'diff=%g, step %d, time=%g' % (diff, n, t[n])
        assert diff < tol, msg

    for version in 'scalar', 'vectorized':
        for theta in 1, 0.5:
            print 'testing Jacobi, %s version, theta=%g' % \
                  (version, theta)
            t, cpu = solver_Jacobi(
                I=I, a=a, f=f, Lx=Lx, Ly=Ly, Nx=Nx, Ny=Ny,
                dt=dt, T=T, theta=theta,
                U_0x=0, U_0y=0, U_Lx=0, U_Ly=0,
                user_action=assert_small_error,
                version=version, iteration='Jacobi',
                omega=1.0, max_iter=100, tol=1E-5)
```

Even for a very coarse 4×4 mesh, the Jacobi method requires 26 iterations to reach a tolerance of 10^{-5}, which is quite many iterations, given that there are only 25 unknowns.

3.6.10 Test problem: diffusion of a sine hill

It can be shown that

$$u_{\mathrm{e}} = Ae^{-\alpha\pi^2(L_x^{-2}+L_y^{-2})t}\sin\left(\frac{\pi}{L_x}x\right)\sin\left(\frac{\pi}{L_y}y\right), \tag{3.108}$$

is a solution of the 2D homogeneous diffusion equation $u_t = \alpha(u_{xx} + u_{yy})$ in a rectangle $[0, L_x] \times [0, L_y]$, for any value of the amplitude A. This solution vanishes at the boundaries, and the initial condition is the product of two sines. We may choose $A = 1$ for simplicity.

It is difficult to know if our solver based on the Jacobi method works properly since we are faced with two sources of errors: one from the discretization, E_Δ, and one from the iterative Jacobi method, E_i. The total error in the computed u can be represented as

$$E_u = E_\Delta + E_i\,.$$

One error measure is to look at the maximum value, which is obtained for the midpoint $x = L_x/2$ and $y = L_x/2$. This midpoint is represented in the discrete `u` if N_x and N_y are even numbers. We can then compute E_u as $E_u = |\max u_{\mathrm{e}} - \max u|$, when we know an exact solution u_{e} of the problem.

What about E_Δ? If we use the maximum value as a measure of the error, we have in fact analytical insight into the approximation error in this particular problem. According to Section 3.3.9, the exact solution (3.108) of the PDE problem is also an exact solution of the discrete equations, except that the damping factor in time is different. More precisely, (3.66) and (3.67) are solutions of the discrete problem for $\theta = 1$ (Backward Euler) and $\theta = \frac{1}{2}$ (Crank-Nicolson), respectively. The factors raised to the power n is the numerical amplitude, and the errors in these factors become

$$E_\Delta = e^{-\alpha k^2 t} - \left(\frac{1 - 2(F_x\sin^2 p_x + F_x\sin^2 p_y)}{1 + 2(F_x\sin^2 p_x + F_x\sin^2 p_y)}\right)^n, \quad \theta = \frac{1}{2},$$

$$E_\Delta = e^{-\alpha k^2 t} - (1 + 4F_x\sin^2 p_x + 4F_y\sin^2 p_y)^{-n}, \quad \theta = 1\,.$$

We are now in a position to compute E_i numerically. That is, we can compute the error due to iterative solution of the linear system and see if it corresponds to the convergence tolerance used in the method. Note that the convergence is based on measuring the difference in two

consecutive approximations, which is not exactly the error due to the iteration, but it is a kind of measure, and it should have about the same size as E_i.

The function `demo_classic_iterative` in `diffu2D_u0.py` implements the idea above (also for the methods in Section 3.6.12). The value of E_i is in particular printed at each time level. By changing the tolerance in the convergence criterion of the Jacobi method, we can see that E_i is of the same order of magnitude as the prescribed tolerance in the Jacobi method. For example: $E_\Delta \sim 10^{-2}$ with $N_x = N_y = 10$ and $\theta = \frac{1}{2}$, as long as $\max u$ has some significant size ($\max u > 0.02$). An appropriate value of the tolerance is then 10^{-3}, such that the error in the Jacobi method does not become bigger than the discretization error. In that case, E_i is around $5 \cdot 10^{-3}$. The corresponding number of Jacobi iterations (with $\omega = 1$) varies from 31 to 12 during the time simulation (for $\max u > 0.02$). Changing the tolerance to 10^{-5} causes many more iterations (61 to 42) without giving any contribution to the overall accuracy, because the total error is dominated by E_Δ.

Also, with an $N_x = N_y = 20$, the spatial accuracy increases and many more iterations are needed (143 to 45), but the dominating error is from the time discretization. However, with such a finer spatial mesh, a higher tolerance in the convergence criterion 10^{-4} is needed to keep $E_i \sim 10^{-3}$. More experiments show the disadvantage of the very simple Jacobi iteration method: the number of iterations increases with the number of unknowns, keeping the tolerance fixed, but the tolerance should also be lowered to avoid the iteration error to dominate the total error. A small adjustment of the Jacobi method, as described in Section 3.6.12, provides a better method.

3.6.11 The relaxed Jacobi method and its relation to the Forward Euler method

We shall now show that solving the Poisson equation $-\alpha\nabla^2 u = f$ by the Jacobi iterative method is in fact equivalent to using a Forward Euler scheme on $u_t = \alpha\nabla^2 u + f$ and letting $t \to \infty$.

A Forward Euler discretization of the 2D diffusion equation,

$$[D_t^+ u = \alpha(D_x D_x u + D_y D_y u) + f]_{i,j}^n,$$

can be written out as

$$u_{i,j}^{n+1} = u_{i,j}^n + \frac{\Delta t}{\alpha h^2}\left(u_{i-1,j}^n + u_{i+1,j}^n + u_{i,j-1}^n + u_{i,j+1}^n - 4u_{i,j}^n + h^2 f_{i,j}\right),$$

where $h = \Delta x = \Delta y$ has been introduced for simplicity. The scheme can be reordered as

$$u_{i,j}^{n+1} = (1-\omega)\, u_{i,j}^n + \frac{1}{4}\omega\left(u_{i-1,j}^n + u_{i+1,j}^n + u_{i,j-1}^n + u_{i,j+1}^n - 4u_{i,j}^n + h^2 f_{i,j}\right),$$

with

$$\omega = 4\frac{\Delta t}{\alpha h^2},$$

but this latter form is nothing but the relaxed Jacobi method applied to

$$[D_x D_x u + D_y D_y u = -f]_{i,j}^n\,.$$

From the equivalence above we know a couple of things about the Jacobi method for solving $-\nabla^2 u = f$:

1. The method is unstable if $\omega > 1$ (since the Forward Euler method is then unstable).
2. The convergence is really slow as the iteration index increases (coming from the fact that the Forward Euler scheme requires many small time steps to reach the stationary solution).

These observations are quite disappointing: if we already have a time-dependent diffusion problem and want to take larger time steps by an implicit time discretization method, we will with the Jacobi method end up with something close to a slow Forward Euler simulation of the original problem at each time level. Nevertheless, the are two reasons for why the Jacobi method remains a fundamental building block for solving linear systems arising from PDEs: 1) a couple of iterations remove large parts of the error and this is effectively used in the very efficient class of multigrid methods; and 2) the idea of the Jacobi method can be developed into more efficient methods, especially the SOR method, which is treated next.

3.6.12 The Gauss-Seidel and SOR methods

If we update the mesh points according to the Jacobi method (3.98) for a Backward Euler discretization with a loop over $i = 1, \ldots, N_x - 1$

and $j = 1, \ldots, N_y - 1$, we realize that when $u_{i,j}^{n+1,r+1}$ is computed, $u_{i-1,j}^{n+1,r+1}$ and $u_{i,j-1}^{n+1,r+1}$ are already computed, so these new values can be used rather than $u_{i-1,j}^{n+1,r}$ and $u_{i,j-1}^{n+1,r}$ (respectively) in the formula for $u_{i,j}^{n+1,r+1}$. This idea gives rise to the *Gauss-Seidel* iteration method, which mathematically is just a small adjustment of (3.98):

$$u_{i,j}^{n+1,r+1} = (1 + 2F_x + 2F_y)^{-1}((\\ F_x(u_{i-1,j}^{n+1,r+1} + u_{i+1,j}^{n+1,r}) + F_y(u_{i,j-1}^{n+1,r+1} + u_{i,j+1}^{n+1,r})) + u_{i,j}^{n} + \Delta t f_{i,j}^{n+1}) \,. \tag{3.109}$$

Observe that the way we access the mesh points in the formula (3.109) is important: points with $i - 1$ must be computed before points with i, and points with $j - 1$ must be computed before points with j. Any sequence of mesh points can be used in the Gauss-Seidel method, but the particular math formula must distinguish between already visited points in the current iteration and the points not yet visited.

The idea of relaxation (3.101) can equally well be applied to the Gauss-Seidel method. Actually, the Gauss-Seidel method with an arbitrary $0 < \omega \leq 2$ has its own name: the *Successive Over-Relaxation* method, abbreviated as SOR.

The SOR method for a θ rule discretization, with the shortened u and u^- notation, can be written

$$u_{i,j}^{*} = (1 + 2\theta(F_x + F_y))^{-1}(\theta(F_x(u_{i-1,j} + u_{i+1,j}^{-}) + F_y(u_{i,j-1} + u_{i,j+1}^{-})) + \\ u_{i,j}^{(1)} + \theta \Delta t f_{i,j}^{n+1} + (1 - \theta)\Delta t f_{i,j}^{n} + \\ (1 - \theta)(F_x(u_{i-1,j}^{(1)} - 2u_{i,j}^{(1)} + u_{i+1,j}^{(1)}) + F_y(u_{i,j-1}^{(1)} - 2u_{i,j}^{(1)} + u_{i,j+1}^{(1)}))), \tag{3.110}$$

$$u_{i,j} = \omega u_{i,j}^{*} + (1 - \omega)u_{i,j}^{-} \tag{3.111}$$

The sequence of mesh points in (3.110) is $i = 1, \ldots, N_x - 1$, $j = 1, \ldots, N_y - 1$ (but whether i runs faster or slower than j does not matter).

3.6.13 Scalar implementation of the SOR method

Since the Jacobi and Gauss-Seidel methods with relaxation are so similar, we can easily make a common code for the two:

```
for n in It[0:-1]:
    # Solve linear system by Jacobi/SOR iteration at time level n+1
    u_[:,:] = u_n  # Start value
    converged = False
    r = 0
    while not converged:
        if version == 'scalar':
            if iteration == 'Jacobi':
                u__ = u_
            elif iteration == 'SOR':
                u__ = u
            j = 0
            for i in Ix:
                u[i,j] = U_0y(t[n+1])  # Boundary
            for j in Iy[1:-1]:
                i = 0;   u[i,j] = U_0x(t[n+1])  # Boundary
                i = Nx;  u[i,j] = U_Lx(t[n+1])  # Boundary
                for i in Ix[1:-1]:
                    u_new = 1.0/(1.0 + 2*theta*(Fx + Fy))*(theta*(
                        Fx*(u_[i+1,j] + u__[i-1,j]) +
                        Fy*(u_[i,j+1] + u__[i,j-1])) + \
                    u_n[i,j] + (1-theta)*(
                      Fx*(
                    u_n[i+1,j] - 2*u_n[i,j] + u_n[i-1,j]) +
                      Fy*(
                    u_n[i,j+1] - 2*u_n[i,j] + u_n[i,j-1]))\
                      + theta*dt*f(i*dx,j*dy,(n+1)*dt) + \
                    (1-theta)*dt*f(i*dx,j*dy,n*dt))
                    u[i,j] = omega*u_new + (1-omega)*u_[i,j]
                j = Ny
                for i in Ix:
                    u[i,j] = U_Ly(t[n+1])  # boundary
        r += 1
        converged = np.abs(u-u_).max() < tol or r >= max_iter
        u_[:,:] = u

    u_n, u = u, u_n  # Get ready for next iteration
```

The idea here is to introduce `u__` to be used for already computed values (`u`) in the Gauss-Seidel/SOR version of the implementation, or just values from the previous iteration (`u_`) in case of the Jacobi method.

3.6.14 Vectorized implementation of the SOR method

Vectorizing the Gauss-Seidel iteration step turns out to be non-trivial. The problem is that vectorized operations typically imply operations on

arrays where the sequence in which we visit the elements does not matter. In particular, this principle makes vectorized code trivial to parallelize. However, in the Gauss-Seidel algorithm, the sequence in which we visit the elements in the arrays does matter, and it is well known that the basic method as explained above cannot be parallelized. Therefore, also vectorization will require new thinking.

The strategy for vectorizing (and parallelizing) the Gauss-Seidel method is to use a special numbering of the mesh points called red-black numbering: every other point is red or black as in a checkerboard pattern. This numbering requires N_x and N_y to be even numbers. Here is an example of a 6×6 mesh:

```
r b r b r b r
b r b r b r b
r b r b r b r
b r b r b r b
r b r b r b r
b r b r b r b
r b r b r b r
```

The idea now is to first update all the red points. Each formula for updating a red point involves only the black neighbors. Thereafter, we update all the black points, and at each black point, only the recently computed red points are involved.

The scalar implementation of the red-black numbered Gauss-Seidel method is really compact, since we can update values directly in `u` (this guarantees that we use the most recently computed values). Here is the relevant code for the Backward Euler scheme in time and without a source term:

```
# Update internal points
for sweep in 'red', 'black':
    for j in range(1, Ny, 1):
        if sweep == 'red':
            start = 1 if j % 2 == 1 else 2
        elif sweep == 'black':
            start = 2 if j % 2 == 1 else 1
        for i in range(start, Nx, 2):
        u[i,j] = 1.0/(1.0 + 2*(Fx + Fy))*(
                     Fx*(u[i+1,j] + u[i-1,j]) +
                     Fy*(u[i,j+1] + u[i,j-1]) + u_n[i,j])
```

The vectorized version must be based on slices. Looking at a typical red-black pattern, e.g.,

```
r b r b r b r
b r b r b r b
r b r b r b r
```

```
b r b r b r b
r b r b r b r
b r b r b r b
r b r b r b r
```

we want to update the internal points (marking boundary points with `x`):

```
x x x x x x x
x r b r b r x
x b r b r b x
x r b r b r x
x b r b r b x
x r b r b r x
x x x x x x x
```

It is impossible to make one slice that picks out all the internal red points. Instead, we need two slices. The first involves points marked with `R`:

```
x x x x x x x
x R b R b R x
x b r b r b x
x R b R b R x
x b r b r b x
x R b R b R x
x x x x x x x
```

This slice is specified as `1::2` for `i` and `1::2` for `j`, or with `slice` objects:

```
i = slice(1, None, 2);  j = slice(1, None, 2)
```

The second slice involves the red points with `R`:

```
x x x x x x x
x r b r b r x
x b R b R b x
x r b r b r x
x b R b R b x
x r b r b r x
x x x x x x x
```

The slices are

```
i = slice(2, None, 2);  j = slice(2, None, 2)
```

For the black points, the first slice involves the `B` points:

```
x x x x x x x
x r B r B r x
x b r b r b x
x r B r B r x
x b r b r b x
x r B r B r x
x x x x x x x
```

with slice objects

```
i = slice(2, None, 2);  j = slice(1, None, 2)
```

The second set of black points is shown here:

```
x x x x x x x
x r b r b r x
x B r B r B x
x r b r b r x
x B r B r B x
x r b r b r x
x x x x x x x
```

with slice objects

```
i = slice(1, None, 2);  j = slice(2, None, 2)
```

That is, we need four sets of slices. The simplest way of implementing the algorithm is to make a function with variables for the slices representing i, $i-1$, $i+1$, j, $j-1$, and $j+1$, here called `ic` ("i center"), `im1` ("i minus 1", `ip1` ("i plus 1"), `jc`, `jm1`, and `jp1`, respectively.

```
def update(u_, u_n, ic, im1, ip1, jc, jm1, jp1):
    return \
       1.0/(1.0 + 2*theta*(Fx + Fy))*(theta*(
           Fx*(u_[ip1,jc] + u_[im1,jc]) +
           Fy*(u_[ic,jp1] + u_[ic,jm1])) +\
       u_n[ic,jc] + (1-theta)*(
         Fx*(u_n[ip1,jc] - 2*u_n[ic,jc] + u_n[im1,jc]) +\
         Fy*(u_n[ic,jp1] - 2*u_n[ic,jc] + u_n[ic,jm1]))+\
         theta*dt*f_a_np1[ic,jc] + \
         (1-theta)*dt*f_a_n[ic,jc])
```

The formula returned from `update` is to be compared with (3.110).

The relaxed Jacobi iteration can be implemented by

```
ic  = jc  = slice(1,-1)
im1 = jm1 = slice(0,-2)
ip1 = jp1 = slice(2,None)
u_new[ic,jc] = update(
    u_, u_n, ic, im1, ip1, jc, jm1, jp1)
u[ic,jc] = omega*u_new[ic,jc] + (1-omega)*u_[ic,jc]
```

The Gauss-Seidel (or SOR) updates need four different steps. The `ic` and `jc` slices are specified above. For each of these, we must specify the corresponding `im1`, `ip1`, `jm1`, and `jp1` slices. The code below contains the details.

```
# Red points
ic  = slice(1,-1,2)
im1 = slice(0,-2,2)
```

```
ip1 = slice(2,None,2)
jc  = slice(1,-1,2)
jm1 = slice(0,-2,2)
jp1 = slice(2,None,2)
u_new[ic,jc] = update(
    u_new, u_n, ic, im1, ip1, jc, jm1, jp1)

ic  = slice(2,-1,2)
im1 = slice(1,-2,2)
ip1 = slice(3,None,2)
jc  = slice(2,-1,2)
jm1 = slice(1,-2,2)
jp1 = slice(3,None,2)
u_new[ic,jc] = update(
    u_new, u_n, ic, im1, ip1, jc, jm1, jp1)

# Black points
ic  = slice(2,-1,2)
im1 = slice(1,-2,2)
ip1 = slice(3,None,2)
jc  = slice(1,-1,2)
jm1 = slice(0,-2,2)
jp1 = slice(2,None,2)
u_new[ic,jc] = update(
    u_new, u_n, ic, im1, ip1, jc, jm1, jp1)

ic  = slice(1,-1,2)
im1 = slice(0,-2,2)
ip1 = slice(2,None,2)
jc  = slice(2,-1,2)
jm1 = slice(1,-2,2)
jp1 = slice(3,None,2)
u_new[ic,jc] = update(
    u_new, u_n, ic, im1, ip1, jc, jm1, jp1)

# Relax
c = slice(1,-1)
u[c,c] = omega*u_new[c,c] + (1-omega)*u_[c,c]
```

The function `solver_classic_iterative` in `diffu2D_u0.py` contains a unified implementation of the relaxed Jacobi and SOR methods in scalar and vectorized versions using the techniques explained above.

3.6.15 Direct versus iterative methods

Direct methods. There are two classes of methods for solving linear systems: direct methods and iterative methods. Direct methods are based on variants of the Gaussian elimination procedure and will produce an exact solution (in exact arithmetics) in an a priori known number of steps. Iterative methods, on the other hand, produce an approximate

solution, and the amount of work for reaching a given accuracy is usually not known.

The most common direct method today is to use the *LU factorization* procedure to factor the coefficient matrix A as the product of a lower-triangular matrix L (with unit diagonal terms) and an upper-triangular matrix U: $A = LU$. As soon as we have L and U, a system of equations $LUc = b$ is easy to solve because of the triangular nature of L and U. We first solve $Ly = b$ for y (forward substitution), and thereafter we find c from solving $Uc = y$ (backward substitution). When A is a dense $N \times N$ matrix, the LU factorization costs $\frac{1}{3}N^3$ arithmetic operations, while the forward and backward substitution steps each require of the order N^2 arithmetic operations. That is, factorization dominates the costs, while the substitution steps are cheap.

Symmetric, positive definite coefficient matrices often arise when discretizing PDEs. In this case, the LU factorization becomes $A = LL^T$, and the associated algorithm is known as *Cholesky factorization*. Most linear algebra software offers highly optimized implementations of LU and Cholesky factorization as well as forward and backward substitution (`scipy.linalg` is the relevant Python package).

Finite difference discretizations lead to sparse coefficient matrices. An extreme case arose in Section 3.2.1 where A was tridiagonal. For a tridiagonal matrix, the amount of arithmetic operations in the LU and Cholesky factorization algorithms is just of the order N, not N^3. Tridiagonal matrices are special cases of *banded matrices*, where the matrices contain just a set of diagonal bands. Finite difference methods on regularly numbered rectangular and box-shaped meshes give rise to such banded matrices, with 5 bands in 2D and 7 in 3D for diffusion problems. Gaussian elimination only needs to work within the bands, leading to much more efficient algorithms.

If $A_{i,j} = 0$ for $j > i + p$ and $j < i - p$, p is the *half-bandwidth* of the matrix. We have in our 2D problem $p = N_x + 2$, while in 3D, $p = (N_x + 1)(N_y + 1) + 2$. The cost of Gaussian elimination is then $\mathcal{O}(Np^2)$, so with $p \ll N$, we see that banded matrices are much more efficient to compute with. By reordering the unknowns in clever ways, one can reduce the work of Gaussian elimination further. Fortunately, the Python programmer has access to such algorithms through the `scipy.sparse.linalg` package.

Although a direct method is an exact algorithm, rounding errors may in practice accumulate and pollute the solution. The effect grows with the size of the linear system, so both for accuracy and efficiency, iterative

methods are better suited than direct methods for solving really large linear systems.

Iterative methods. The Jacobi and SOR iterative methods belong to a class of iterative methods where the idea is to solve $Au = b$ by splitting A into two parts, $A = M - N$, such that solving systems $Mu = c$ is easy and efficient. With the splitting, we get a system

$$Mu = Nu + b,$$

which suggests an iterative method

$$Mu^{r+1} = Nu^r + b, \quad r = 0, 1, 2, \ldots,$$

where u^{r+1} is a new approximation to u in the $r + 1$-th iteration. To initiate the iteration, we need a start vector u^0.

The Jacobi and SOR methods are based on splitting A into a lower tridiagonal part L, the diagonal D, and an upper tridiagonal part U, such that $A = L + D + U$. The Jacobi method corresponds to $M = D$ and $N = -L - U$. The Gauss-Seidel method employs $M = L + D$ and $N = -U$, while the SOR method corresponds to

$$M = \frac{1}{\omega}D + L, \quad N = \frac{1-\omega}{\omega}D - U\,.$$

The relaxed Jacobi method has similar expressions:

$$M = \frac{1}{\omega}D, \quad N = \frac{1-\omega}{\omega}D - L - U\,.$$

With the matrix forms of the Jacobi and SOR methods as written above, we could in an implementation alternatively fill the matrix A with entries and call general implementations of the Jacobi or SOR methods that work on a system $Au = b$. However, this is almost never done since forming the matrix A requires quite some code and storing A in the computer's memory is unnecessary. It is much easier to just apply the Jacobi and SOR ideas to the finite difference stencils directly in an implementation, as we have shown in detail.

Nevertheless, the matrix formulation of the Jacobi and SOR methods have been important for analyzing their convergence behavior. One can show that the error $u^r - u$ fulfills $u^r - u = G^r(u^0 - u)$, where $G = M^{-1}N$ and G^k is a matrix exponential. For the method to converge, $\lim_{r\to\infty} ||G^r|| = 0$ is a necessary and sufficient condition. This implies that the *spectral radius* of G must be less than one. Since G is directly

related to the finite difference scheme for the underlying PDE problem, one can in principle compute the spectral radius. For a given PDE problem, however, this is not a practical strategy, since it is very difficult to develop useful formulas. Analysis of model problems, usually related to the Poisson equation, reveals some trends of interest: the convergence rate of the Jacobi method goes like h^2, while that of SOR with an optimal ω goes like h, where h is the spatial spacing: $h = \Delta x = \Delta y$. That is, the efficiency of the Jacobi method quickly deteriorates with the increasing mesh resolution, and SOR is much to be preferred (even if the optimal ω remains an open question). We refer to Chapter 4 of [16] for more information on the convergence theory. One important result is that if A is symmetric and positive definite, then SOR will converge for any $0 < \omega < 2$.

The optimal ω parameter can be theoretically established for a Poisson problem as

$$\omega_o = \frac{2}{1 + \sqrt{1 - \varrho^2}}, \quad \varrho = \frac{\cos(\pi/N_x) + (\Delta x/\Delta y)^2 \cos(\pi/N_y)}{1 + (\Delta x/\Delta y)^2}. \quad (3.112)$$

This formula can be used as a guide also in other problems.

The Jacobi and the SOR methods have their great advantage of being trivial to implement, so they are obviously popular of this reason. However, the slow convergence of these methods limits the popularity to fairly small linear systems (i.e., coarse meshes). As soon as the matrix size grows, one is better off with more sophisticated iterative methods like the preconditioned Conjugate gradient method, which we now turn to.

Finally, we mention that there is a variant of the SOR method, called the *Symmetric Successive Over-relaxation* method, known as SSOR, where one runs a standard SOR sweep through the mesh points and then a new sweep while visiting the points in reverse order.

3.6.16 The Conjugate gradient method

There is no simple intuitive derivation of the Conjugate gradient method, so we refer to the many excellent expositions in the literature for the idea of the method and how the algorithm is derived. In particular, we recommend the books [2, 1, 16, 5]. A brief overview is provided in the

Wikipedia article[3]. Here, we just state the pros and cons of the method from a user's perspective and how we utilize it in code.

The original Conjugate gradient method is limited to linear systems $Au = b$, where A is a symmetric and positive definite matrix. There are, however, extensions of the method to non-symmetric matrices.

A major advantage of all conjugate gradient methods is that the matrix A is only used in matrix-vector products, so we do not need form and store A if we can provide code for computing a matrix-vector product Au. Another important feature is that the algorithm is very easy to vectorize and parallelize. The primary downside of the method is that it converges slowly unless one has an effective *preconditioner* for the system. That is, instead of solving $Au = b$, we try to solve $M^{-1}Au = M^{-1}b$ in the hope that the method works better for this *preconditioned* system. The matrix M is the *preconditioner* or preconditioning matrix. Now we need to perform matrix-vector products $y = M^{-1}Au$, which is done in two steps: first the matrix-vector product $v = Au$ is carried out and then the system $My = v$ must be solved. Therefore, M must be cheap to compute and systems $My = v$ must be cheap to solve.

A perfect preconditioner is $M = A$, but in each iteration in the Conjugate gradient method one then has so solve a system with A as coefficient matrix! A key idea is to let M be some kind of *cheap approximation* to A. The simplest preconditioner is to set $M = D$, where D is the diagonal of A. This choice means running one Jacobi iteration as preconditioner. Exercise 3.8 shows that the Jacobi and SOR methods can also be viewed as preconditioners.

Constructing good preconditioners is a scientific field on its own. Here we shall treat the topic just very briefly. For a user having access to the `scipy.sparse.linalg` library, there are Conjugate gradient methods and preconditioners readily available:

- For positive definite, symmetric systems: `cg` (the Conjugate gradient method)
- For symmetric systems: `minres` (Minimum residual method)
- For non-symmetric systems:
  - `gmres` (GMRES: Generalized minimum residual method)
  - `bicg` (BiConjugate gradient method)
  - `bicgstab` (Stabilized BiConjugate gradient method)
  - `cgs` (Conjugate gradient squared method)
  - `qmr` (Quasi-minimal residual iteration)

[3] `https://en.wikipedia.org/wiki/Conjugate_gradient_method`

- Preconditioner: `spilu` (Sparse, incomplete LU factorization)

The ILU preconditioner is an attractive all-round type of preconditioner that is suitable for most problems on serial computers. A more efficient preconditioner is the multigrid method, and algebraic multigrid is also an all-round choice as preconditioner. The Python package PyAMG[4] offers efficient implementations of the algebraic multigrid method, to be used both as a preconditioner and as a stand-alone iterative method.

The matrix arising from implicit time discretization methods applied to the diffusion equation is symmetric and positive definite. Thus, we can use the Conjugate gradient method (`cg`), typically in combination with an ILU preconditioner. The code is very similar to the one we created when solving the linear system by sparse Gaussian elimination, the main difference is that we now allow for calling up the Conjugate gradient function as an alternative solver.

```
def solver_sparse(
    I, a, f, Lx, Ly, Nx, Ny, dt, T, theta=0.5,
    U_0x=0, U_0y=0, U_Lx=0, U_Ly=0, user_action=None,
    method='direct', CG_prec='ILU', CG_tol=1E-5):
    """
    Full solver for the model problem using the theta-rule
    difference approximation in time. Sparse matrix with
    dedicated Gaussian elimination algorithm (method='direct')
    or ILU preconditioned Conjugate Gradients (method='CG' with
    tolerance CG_tol and preconditioner CG_prec ('ILU' or None)).
    """
    # Set up data structures as shown before

    # Precompute sparse matrix
    ...

    A = scipy.sparse.diags(
        diagonals=[main, lower, upper, lower2, upper2],
        offsets=[0, -lower_offset, lower_offset,
                 -lower2_offset, lower2_offset],
        shape=(N, N), format='csc')

    if method == 'CG':
        if CG_prec == 'ILU':
            # Find ILU preconditioner (constant in time)
            A_ilu = scipy.sparse.linalg.spilu(A)  # SuperLU defaults
            M = scipy.sparse.linalg.LinearOperator(
                shape=(N, N), matvec=A_ilu.solve)
        else:
            M = None
        CG_iter = []  # No of CG iterations at time level n
```

[4] https://github.com/pyamg/pyamg

```
    # Time loop
    for n in It[0:-1]:
        # Compute b, vectorized version

        # Solve matrix system A*c = b
        if method == 'direct':
            c = scipy.sparse.linalg.spsolve(A, b)
        elif method == 'CG':
            x0 = u_n.T.reshape(N)  # Start vector is u_n
            CG_iter.append(0)

            def CG_callback(c_k):
                """Trick to count the no of iterations in CG."""
                CG_iter[-1] += 1

            c, info = scipy.sparse.linalg.cg(
                A, b, x0=x0, tol=CG_tol, maxiter=N, M=M,
                callback=CG_callback)

        # Fill u with vector c
        # Update u_n before next step
        u_n, u = u, u_n
```

The number of iterations in the Conjugate gradient method is of interest, but is unfortunately not available from the `cg` function. Therefore, we perform a trick: in each iteration a user function `CG_callback` is called where we accumulate the number of iterations in a list `CG_iter`.

3.6.17 What is the recommended method for solving linear systems?

There is no clear answer to this question. If you have enough memory and computing time available, direct methods such as `spsolve` are to be preferred since they are easy to use and finds almost an exact solution. However, in larger 2D and in 3D problems, direct methods usually run too slowly or require too much memory, so one is forced to use iterative methods. The fastest and most reliable methods are in the Conjugate Gradient family, but these require suitable preconditioners. ILU is an all-round preconditioner, but it is not suited for parallel computing. The Jacobi and SOR iterative methods are easy to implement, and popular for that reason, but run slowly. Jacobi iteration is not an option in real problems, but SOR may be.

3.7 Random walk

Models leading to diffusion equations, see Section 3.8, are usually based on reasoning with *averaged* physical quantities such as concentration, temperature, and velocity. The underlying physical processes involve complicated microscopic movement of atoms and molecules, but an average of a large number of molecules is performed in a small volume before the modeling starts, and the averaged quantity inside this volume is assigned as a point value at the centroid of the volume. This means that concentration, temperature, and velocity at a space-time point represent averages around the point in a small time interval and small spatial volume.

Random walk is a principally different kind of modeling procedure compared to the reasoning behind partial differential equations. The idea in random walk is to have a large number of "particles" that undergo random movements. Averaging can then be used afterwards to compute macroscopic quantities like concentration. The "particles" and their random movement represent a very simplified microscopic behavior of molecules, much simpler and computationally much more efficient than direct molecular simulation[5], yet the random walk model has been very powerful to describe a wide range of phenomena, including heat conduction, quantum mechanics, polymer chains, population genetics, neuroscience, hazard games, and pricing of financial instruments.

It can be shown that random walk, when averaged, produces models that are mathematically equivalent to diffusion equations. This is the primary reason why we treat random walk in this chapter: two very different algorithms (finite difference stencils and random walk) solve the same type of problems. The simplicity of the random walk algorithm makes it particularly attractive for solving diffusion equations on massively parallel computers. The exposition here is as simple as possible, and good thorough derivation of the models is provided by Hjorth-Jensen [7].

3.7.1 Random walk in 1D

Imagine that we have some particles that perform random moves, either to the right or to the left. We may flip a coin to decide the movement of each particle, say head implies movement to the right and tail means

[5] `https://en.wikipedia.org/wiki/Molecular_dynamics`

movement to the left. Each move is one unit length. Physicists use the term *random walk* for this type of movement. The movement is also known as drunkard's walk[6]. You may try this yourself: flip the coin and make one step to the left or right, and repeat the process.

We introduce the symbol N for the number of steps in a random walk. Figure 3.16 shows four different random walks with $N = 200$.

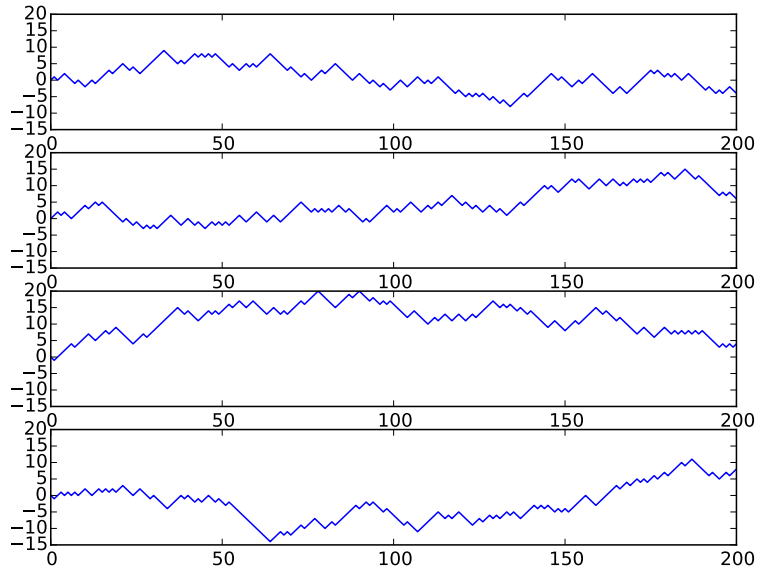


Fig. 3.16 Ensemble of 4 random walks, each with 200 steps.

3.7.2 Statistical considerations

Let S_k be the stochastic variable representing a step to the left or to the right in step number k. We have that $S_k = -1$ with probability p and $S_k = 1$ with probability $q = 1 - p$. The variable S_k is known as a Bernoulli variable[7]. The expectation of S_k is

$$\mathrm{E}[S_k] = p \cdot (-1) + q \cdot 1 = 1 - 2p,$$

[6] https://en.wikipedia.org/wiki/The_Drunkard%27s_Walk
[7] https://en.wikipedia.org/wiki/Bernoulli_distribution

and the variance is

$$\mathrm{Var}[S_k] = \mathrm{E}[S_k^2] - \mathrm{E}[S_k]^2 = 1 - (1-2p)^2 = 4p(1-p)\,.$$

The position after k steps is another stochastic variable

$$\bar{X}_k = \sum_{i=0}^{k-1} S_i\,.$$

The expected position is

$$\mathrm{E}[\bar{X}_k] = \mathrm{E}[\sum_{i=0}^{k-1} S_i] = \sum_{i=0}^{k-1} \mathrm{E}[S_i] = k(1-2p)\,.$$

All the S_k variables are independent. The variance therefore becomes

$$\mathrm{Var}[\bar{X}_k] = \mathrm{Var}[\sum_{i=0}^{k-1} S_i] = \sum_{i=0}^{k-1} \mathrm{Var}[S_i] = k4p(1-p)\,.$$

We see that $\mathrm{Var}[\bar{X}_k]$ is proportional with the number of steps k. For the very important case $p = q = \frac{1}{2}$, $\mathrm{E}[\bar{X}_k] = 0$ and $\mathrm{Var}[\bar{X}_k] = k$.

How can we estimate $\mathrm{E}[\bar{X}_k] = 0$ and $\mathrm{Var}[\bar{X}_k] = N$? We must have many random walks of the type in Figure 3.16. For a given k, say $k = 100$, we find all the values of $\bar{X}_k$, name them $\bar{x}_{0,k}$, $\bar{x}_{1,k}$, $\bar{x}_{2,k}$, and so on. The empirical estimate of $\mathrm{E}[\bar{X}_k]$ is the average,

$$\mathrm{E}[\bar{X}_k] \approx \frac{1}{W} \sum_{j=0}^{W-1} \bar{x}_{j,k},$$

while an empirical estimate of $\mathrm{Var}[\bar{X}_k]$ is

$$\mathrm{Var}[\bar{X}_k] \approx \frac{1}{W} \sum_{j=0}^{W-1} (\bar{x}_{j,k})^2 - \left(\frac{1}{W} \sum_{j=0}^{W-1} \bar{x}_{j,k} \right)^2 .$$

That is, we take the statistics for a given K across the ensemble of random walks ("vertically" in Figure 3.16). The key quantities to record are $\sum_i \bar{x}_{i,k}$ and $\sum_i \bar{x}_{i,k}^2$.

3.7.3 Playing around with some code

Scalar code. Python has a `random` module for drawing random numbers, and this module has a function `uniform(a, b)` for drawing a uniformly distributed random number in the interval $[a, b)$. If an event happens with probability p, we can simulate this on the computer by drawing a random number r in $[0, 1)$, because then $r \leq p$ with probability p and $r > p$ with probability $1 - p$:

```
import random
r = random.uniform(0, 1)
if r <= p:
    # Event happens
else:
    # Event does not happen
```

A random walk with N steps, starting at x_0, where we move to the left with probability p and to the right with probability $1 - p$ can now be implemented by

```
import random, numpy as np

def random_walk1D(x0, N, p):
    """1D random walk with 1 particle."""
    # Store position in step k in position[k]
    position = np.zeros(N)
    position[0] = x0
    current_pos = x0
    for k in range(N-1):
        r = random.uniform(0, 1)
        if r <= p:
            current_pos -= 1
        else:
            current_pos += 1
        position[k+1] = current_pos
    return position
```

Vectorized code. Since N is supposed to be large and we want to repeat the process for many particles, we should speed up the code as much as possible. Vectorization is the obvious technique here: we draw all the random numbers at once with aid of `numpy`, and then we formulate vector operations to get rid of the loop over the steps (`k`). The `numpy.random` module has vectorized versions of the functions in Python's built-in `random` module. For example, `numpy.random.uniform(a, b, N)` returns `N` random numbers uniformly distributed between `a` (included) and `b` (not included).

We can then make an array of all the steps in a random walk: if the random number is less than or equal to p, the step is -1, otherwise the step is 1:

```
r = np.random.uniform(0, 1, size=N)
steps = np.where(r <= p, -1, 1)
```

The value of `position[k]` is the sum of all steps up to step `k`. Such sums are often needed in vectorized algorithms and therefore available by the `numpy.cumsum` function:

```
>>> import numpy as np
>>> np.cumsum(np.array([1,3,4,6]))
array([ 1,  4,  8, 14])
```

The resulting array in this demo has elements 1, $1+3=4$, $1+3+4=8$, and $1+3+4+6=14$.

We can now vectorize the `random_walk1D` function:

```
def random_walk1D_vec(x0, N, p):
    """Vectorized version of random_walk1D."""
    # Store position in step k in position[k]
    position = np.zeros(N+1)
    position[0] = x0
    r = np.random.uniform(0, 1, size=N)
    steps = np.where(r <= p, -1, 1)
    position[1:] = x0 + np.cumsum(steps)
    return position
```

This code runs about 10 times faster than the scalar version. With a parallel `numpy` library, the code can also automatically take advantage of hardware for parallel computing because each of the four array operations can be trivially parallelized.

Fixing the random sequence. During software development with random numbers it is advantageous to always generate the same sequence of random numbers, as this may help debugging processes. To fix the sequence, we set a *seed* of the random number generator to some chosen integer, e.g.,

```
np.random.seed(10)
```

Calls to `random_walk1D_vec` give positions of the particle as depicted in Figure 3.17. The particle starts at the origin and moves with $p = \frac{1}{2}$. Since the seed is the same, the plot to the left is just a magnification of the first 1,000 steps in the plot to the right.

Verification. When we have a scalar and a vectorized code, it is always a good idea to develop a unit test for checking that they produce the same

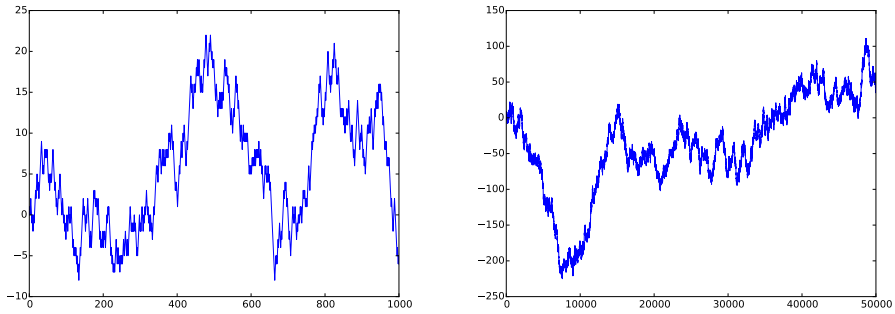


Fig. 3.17 1,000 (left) and 50,000 (right) steps of a random walk.

result. A problem in the present context is that the two versions apply two different random number generators. For a test to be meaningful, we need to fix the seed and use the same generator. This means that the scalar version must either use `np.random` or have this as an option. An option is the most flexible choice:

```
import random

def random_walk1D(x0, N, p, random=random):
    ...
    r = random.uniform(0, 1)
```

Using `random=np.random`, the `r` variable gets computed by `np.random.uniform`, and the sequence of random numbers will be the same as in the vectorized version that employs the same generator (given that the seed is also the same). A proper test function may be to check that the positions in the walk are the same in the scalar and vectorized implementations:

```
def test_random_walk1D():
    # For fixed seed, check that scalar and vectorized versions
    # produce the same result
    x0 = 2;  N = 4;  p = 0.6
    np.random.seed(10)
    scalar_computed = random_walk1D(x0, N, p, random=np.random)
    np.random.seed(10)
    vectorized_computed = random_walk1D_vec(x0, N, p)
    assert (scalar_computed == vectorized_computed).all()
```

Note that we employ `==` for arrays with real numbers, which is normally an inadequate test due to rounding errors, but in the present case, all arithmetics consists of adding or subtracting one, so these operations are expected to have no rounding errors. Comparing two `numpy` arrays with `==` results in a boolean array, so we need to call the `all()` method

to ensure that all elements are `True`, i.e., that all elements in the two arrays match each other pairwise.

3.7.4 Equivalence with diffusion

The original random walk algorithm can be said to work with dimensionless coordinates $\bar{x}_i = -N + i$, $i = 0, 1, \ldots, 2N + 1$ ($i \in [-N, N]$), and $\bar{t}_n = n$, $n = 0, 1, \ldots, N$. A mesh with spacings Δx and Δt with dimensions can be introduced by

$$x_i = X_0 + \bar{x}_i \Delta x, \quad t_n = \bar{t}_n \Delta t\,.$$

If we implement the algorithm with dimensionless coordinates, we can just use this rescaling to obtain the movement in a coordinate system without unit spacings.

Let P_i^{n+1} be the probability of finding the particle at mesh point $\bar{x}_i$ at time $\bar{t}_{n+1}$. We can reach mesh point $(i, n+1)$ in two ways: either coming in from the left from $(i-1, n)$ or from the right $(i+1, n)$. Each has probability $\frac{1}{2}$ (if we assume $p = q = \frac{1}{2}$). The fundamental equation for P_i^{n+1} is

$$P_i^{n+1} = \frac{1}{2} P_{i-1}^n + \frac{1}{2} P_{i+1}^n\,. \tag{3.113}$$

(This equation is easiest to understand if one looks at the random walk as a Markov process and applies the transition probabilities, but this is beyond scope of the present text.)

Subtracting P_i^n from (3.113) results in

$$P_i^{n+1} - P_i^n = \frac{1}{2}(P_{i-1}^n - 2P_i^n + \frac{1}{2} P_{i+1}^n)\,.$$

Readers who have seen the Forward Euler discretization of a 1D diffusion equation recognize this scheme as very close to such a discretization. We have

$$\frac{\partial}{\partial t} P(x_i, t_n) = \frac{P_i^{n+1} - P_i^n}{\Delta t} + \mathcal{O}(\Delta t),$$

or in dimensionless coordinates

$$\frac{\partial}{\partial \bar{t}} P(\bar{x}_i, \bar{t}_n) \approx P_i^{n+1} - P_i^n\,.$$

Similarly, we have

$$\frac{\partial^2}{\partial x^2}P(x_i,t_n) = \frac{P_{i-1}^n - 2P_i^n + \frac{1}{2}P_{i+1}^n}{\Delta x^2} + \mathcal{O}(\Delta x^2),$$
$$\frac{\partial^2}{\partial x^2}P(\bar{x}_i,\bar{t}_n) \approx P_{i-1}^n - 2P_i^n + \frac{1}{2}P_{i+1}^n\,.$$

Equation (3.113) is therefore equivalent with the dimensionless diffusion equation

$$\frac{\partial P}{\partial \bar{t}} = \frac{1}{2}\frac{\partial^2 P}{\partial \bar{x}^2}, \tag{3.114}$$

or the diffusion equation

$$\frac{\partial P}{\partial t} = D\frac{\partial^2 P}{\partial x^2}, \tag{3.115}$$

with diffusion coefficient

$$D = \frac{\Delta x^2}{2\Delta t}\,.$$

This derivation shows the tight link between random walk and diffusion. If we keep track of where the particle is, and repeat the process many times, or run the algorithms for lots of particles, the histogram of the positions will approximate the solution of the diffusion equation for the local probability P_i^n.

Suppose all the random walks start at the origin. Then the initial condition for the probability distribution is the Dirac delta function $\delta(x)$. The solution of (3.114) can be shown to be

$$\bar{P}(\bar{x},\bar{t}) = \frac{1}{\sqrt{4\pi\alpha t}}e^{-\frac{x^2}{4\alpha t}}, \tag{3.116}$$

where $\alpha = \frac{1}{2}$.

3.7.5 Implementation of multiple walks

Our next task is to implement an ensemble of walks (for statistics, see Section 3.7.2) and also provide data from the walks such that we can compute the probabilities of the positions as introduced in the previous section. An appropriate representation of probabilities P_i^n are histograms (with i along the x axis) for a few selected values of n.

To estimate the expectation and variance of the random walks, Section 3.7.2 points to recording $\sum_j x_{j,k}$ and $\sum_j x_{j,k}^2$, where $x_{j,k}$ is the position at time/step level k in random walk number j. The histogram of positions needs the individual values $x_{i,k}$ for all i values and some selected k values.

We introduce `position[k]` to hold $\sum_j x_{j,k}$, `position2[k]` to hold $\sum_j (x_{j,k})^2$, and `pos_hist[i,k]` to hold $x_{i,k}$. A selection of k values can be specified by saying how many, `num_times`, and let them be equally spaced through time:

```
pos_hist_times = [(N//num_times)*i for i in range(num_times)]
```

This is one of the few situations where we want integer division (`//`) or real division rounded to an integer.

Scalar version. Our scalar implementation of running `num_walks` random walks may go like this:

```
def random_walks1D(x0, N, p, num_walks=1, num_times=1,
                   random=random):
    """Simulate num_walks random walks from x0 with N steps."""
    position = np.zeros(N+1)    # Accumulated positions
    position[0] = x0*num_walks
    position2 = np.zeros(N+1)   # Accumulated positions**2
    position2[0] = x0**2*num_walks
    # Histogram at num_times selected time points
    pos_hist = np.zeros((num_walks, num_times))
    pos_hist_times = [(N//num_times)*i for i in range(num_times)]
    #print 'save hist:', post_hist_times

    for n in range(num_walks):
        num_times_counter = 0
        current_pos = x0
        for k in range(N):
            if k in pos_hist_times:
                #print 'save, k:', k, num_times_counter, n
                pos_hist[n,num_times_counter] = current_pos
                num_times_counter += 1
            # current_pos corresponds to step k+1
            r = random.uniform(0, 1)
            if r <= p:
                current_pos -= 1
            else:
                current_pos += 1
            position [k+1] += current_pos
            position2[k+1] += current_pos**2
    return position, position2, pos_hist, np.array(pos_hist_times)
```

Vectorized version. We have already vectorized a single random walk. The additional challenge here is to vectorize the computation of the

data for the histogram, `pos_hist`, but given the selected steps in `pos_hist_times`, we can find the corresponding positions by indexing with the list `pos_hist_times`: `position[post_hist_times]`, which are to be inserted in `pos_hist[n,:]`.

```
def random_walks1D_vec1(x0, N, p, num_walks=1, num_times=1):
    """Vectorized version of random_walks1D."""
    position  = np.zeros(N+1)    # Accumulated positions
    position2 = np.zeros(N+1)    # Accumulated positions**2
    walk = np.zeros(N+1)         # Positions of current walk
    walk[0] = x0
    # Histogram at num_times selected time points
    pos_hist = np.zeros((num_walks, num_times))
    pos_hist_times = [(N//num_times)*i for i in range(num_times)]

    for n in range(num_walks):
        r = np.random.uniform(0, 1, size=N)
        steps = np.where(r <= p, -1, 1)
        walk[1:] = x0 + np.cumsum(steps)  # Positions of this walk
        position  += walk
        position2 += walk**2
        pos_hist[n,:] = walk[pos_hist_times]
    return position, position2, pos_hist, np.array(pos_hist_times)
```

Improved vectorized version. Looking at the vectorized version above, we still have one potentially long Python loop over `n`. Normally, `num_walks` will be much larger than `N`. The vectorization of the loop over `N` certainly speeds up the program, but if we think of vectorization as also a way to parallelize the code, all the independent walks (the `n` loop) can be executed in parallel. Therefore, we should include this loop as well in the vectorized expressions, at the expense of using more memory.

We introduce the array `walks` to hold the $N+1$ steps of all the walks: each row represents the steps in one walk.

```
walks = np.zeros((num_walks, N+1))  # Positions of each walk
walks[:,0] = x0
```

Since all the steps are independent, we can just make one long vector of enough random numbers (`N*num_walks`), translate these numbers to ± 1, then we reshape the array such that the steps of each walk are stored in the rows.

```
r = np.random.uniform(0, 1, size=N*num_walks)
steps = np.where(r <= p, -1, 1).reshape(num_walks, N)
```

The next step is to sum up the steps in each walk. We need the `np.cumsum` function for this, with the argument `axis=1` for indicating a sum across the columns:

```
>>> a = np.arange(6).reshape(2,3)
>>> a
array([[0, 1, 2],
       [3, 4, 5]])
>>> np.cumsum(a, axis=1)
array([[ 0,  1,  3],
       [ 3,  7, 12]])
```

Now `walks` can be computed by

```
walks[:,1:] = x0 + np.cumsum(steps, axis=1)
```

The `position` vector is the sum of all the walks. That is, we want to sum all the rows, obtained by

```
position  = np.sum(walks, axis=0)
```

A corresponding expression computes the squares of the positions. Finally, we need to compute `pos_hist`, but that is a matter of grabbing some of the walks (according to `pos_hist_times`):

```
pos_hist[:,:] = walks[:,pos_hist_times]
```

The complete vectorized algorithm without any loop can now be summarized:

```
def random_walks1D_vec2(x0, N, p, num_walks=1, num_times=1):
    """Vectorized version of random_walks1D; no loops."""
    position  = np.zeros(N+1)    # Accumulated positions
    position2 = np.zeros(N+1)    # Accumulated positions**2
    walks = np.zeros((num_walks, N+1))  # Positions of each walk
    walks[:,0] = x0
    # Histogram at num_times selected time points
    pos_hist = np.zeros((num_walks, num_times))
    pos_hist_times = [(N//num_times)*i for i in range(num_times)]

    r = np.random.uniform(0, 1, size=N*num_walks)
    steps = np.where(r <= p, -1, 1).reshape(num_walks, N)
    walks[:,1:] = x0 + np.cumsum(steps, axis=1)
    position  = np.sum(walks,    axis=0)
    position2 = np.sum(walks**2, axis=0)
    pos_hist[:,:] = walks[:,pos_hist_times]
    return position, position2, pos_hist, np.array(pos_hist_times)
```

What is the gain of the vectorized implementations? One important gain is that each vectorized operation can be automatically parallelized if one applies a parallel `numpy` library like Numba[8]. On a single CPU, however, the speed up of the vectorized operations is also significant. With $N = 1,000$ and 50,000 repeated walks, the two vectorized ver-

[8] `http://numba.pydata.org`

sions run about 25 and 18 times faster than the scalar version, with `random_walks1D_vec1` being fastest.

Remark on vectorized code and parallelization. Our first attempt on vectorization removed the loop over the N steps in a single walk. However, the number of walks is usually much larger than N, because of the need for accurate statistics. Therefore, we should rather remove the loop over all walks. It turns out, from our efficiency experiments, that the function `random_walks1D_vec2` (with no loops) is slower than `random_walks1D_vec1`. This is a bit surprising and may be explained by less efficiency in the statements involving very large arrays, containing all steps for all walks at once.

From a parallelization and improved vectorization point of view, it would be more natural to switch the sequence of the loops in the serial code such that the shortest loop is the outer loop:

```
def random_walks1D2(x0, N, p, num_walks=1, num_times=1, ...):
    ...
    current_pos = x0 + np.zeros(num_walks)
    num_times_counter = -1

    for k in range(N):
        if k in pos_hist_times:
        num_times_counter += 1
        store_hist = True
    else:
        store_hist = False

        for n in range(num_walks):
            # current_pos corresponds to step k+1
            r = random.uniform(0, 1)
        if r <= p:
                current_pos[n] -= 1
            else:
                current_pos[n] += 1
            position [k+1] += current_pos[n]
            position2[k+1] += current_pos[n]**2
            if store_hist:
                pos_hist[n,num_times_counter] = current_pos[n]
    return position, position2, pos_hist, np.array(pos_hist_times)
```

The vectorized version of this code, where we just vectorize the loop over `n`, becomes

```
def random_walks1D2_vec1(x0, N, p, num_walks=1, num_times=1):
    """Vectorized version of random_walks1D2."""
    position  = np.zeros(N+1)    # Accumulated positions
    position2 = np.zeros(N+1)    # Accumulated positions**2
    # Histogram at num_times selected time points
    pos_hist = np.zeros((num_walks, num_times))
```

```
    pos_hist_times = [(N//num_times)*i for i in range(num_times)]

    current_pos = np.zeros(num_walks)
    current_pos[0] = x0
    num_times_counter = -1

    for k in range(N):
        if k in pos_hist_times:
        num_times_counter += 1
        store_hist = True  # Store histogram data for this k
    else:
        store_hist = False

        # Move all walks one step
        r = np.random.uniform(0, 1, size=num_walks)
        steps = np.where(r <= p, -1, 1)
        current_pos += steps
        position[k+1]  = np.sum(current_pos)
        position2[k+1] = np.sum(current_pos**2)
        if store_hist:
            pos_hist[:,num_times_counter] = current_pos
    return position, position2, pos_hist, np.array(pos_hist_times)
```

This function runs significantly faster than the `random_walks1D_vec1` function above, typically 1.7 times faster. The code is also more appropriate in a parallel computing context since each vectorized statement can work with data of size `num_walks` over the compute units, repeated `N` times (compared with data of size `N`, repeated `num_walks` times, in `random_walks1D_vec1`).

The scalar code with switched loops, `random_walks1D2` runs a bit slower than the original code in `random_walks1D`, so with the longest loop as the inner loop, the vectorized function `random_walks1D2_vec1` is almost 60 times faster than the scalar counterpart, while the code `random_walks1D_vec2` without loops is only around 18 times faster. Taking into account the very large arrays required by the latter function, we end up with `random_walks1D2_vec1` as the preferred implementation.

Test function. During program development, it is highly recommended to carry out computations by hand for, e.g., `N=4` and `num_walks=3`. Normally, this is done by executing the program with these parameters and checking with pen and paper that the computations make sense. The next step is to use this test for correctness in a formal test function.

First, we need to check that the simulation of multiple random walks reproduces the results of `random_walk1D`, `random_walk1D_vec1`, and `random_walk1D_vec2` for the first walk, if the seed is the same. Second, we run three random walks (`N=4`) with the scalar and the two vectorized versions and check that the returned arrays are identical.

For this type of test to be successful, we must be sure that exactly the same set of random numbers are used in the three versions, a fact that requires the same random number generator and the same seed, of course, but also the same sequence of computations. This is not obviously the case with the three `random_walk1D*` functions we have presented. The critical issue in `random_walk1D_vec1` is that the first random numbers are used for the first walk, the second set of random numbers is used for the second walk and so on, to be compatible with how the random numbers are used in the function `random_walk1D`. For the function `random_walk1D_vec2` the situation is a bit more complicated since we generate all the random numbers at once. However, the critical step now is the reshaping of the array returned from `np.where`: we must reshape as `(num_walks, N)` to ensure that the first `N` random numbers are used for the first walk, the next `N` numbers are used for the second walk, and so on.

We arrive at the test function below.

```
def test_random_walks1D():
    # For fixed seed, check that scalar and vectorized versions
    # produce the same result
    x0 = 0;  N = 4;  p = 0.5

    # First, check that random_walks1D for 1 walk reproduces
    # the walk in random_walk1D
    num_walks = 1
    np.random.seed(10)
    computed = random_walks1D(
        x0, N, p, num_walks, random=np.random)
    np.random.seed(10)
    expected = random_walk1D(
        x0, N, p, random=np.random)
    assert (computed[0] == expected).all()

    # Same for vectorized versions
    np.random.seed(10)
    computed = random_walks1D_vec1(x0, N, p, num_walks)
    np.random.seed(10)
    expected = random_walk1D_vec(x0, N, p)
    assert (computed[0] == expected).all()
    np.random.seed(10)
    computed = random_walks1D_vec2(x0, N, p, num_walks)
    np.random.seed(10)
    expected = random_walk1D_vec(x0, N, p)
    assert (computed[0] == expected).all()

    # Second, check multiple walks: scalar == vectorized
    num_walks = 3
    num_times = N
    np.random.seed(10)
    serial_computed = random_walks1D(
```

```
        x0, N, p, num_walks, num_times, random=np.random)
    np.random.seed(10)
    vectorized1_computed = random_walks1D_vec1(
        x0, N, p, num_walks, num_times)
    np.random.seed(10)
    vectorized2_computed = random_walks1D_vec2(
        x0, N, p, num_walks, num_times)
    # positions: [0, 1, 0, 1, 2]
    # Can test without tolerance since everything is +/- 1
    return_values = ['pos', 'pos2', 'pos_hist', 'pos_hist_times']
    for s, v, r in zip(serial_computed,
                       vectorized1_computed,
                       return_values):
        msg = '%s: %s (serial) vs %s (vectorized)' % (r, s, v)
        assert (s == v).all(), msg
    for s, v, r in zip(serial_computed,
                       vectorized2_computed,
                       return_values):
        msg = '%s: %s (serial) vs %s (vectorized)' % (r, s, v)
        assert (s == v).all(), msg
```

Such test functions are indispensable for further development of the code as we can at any time test whether the basic computations remain correct or not. This is particularly important in stochastic simulations since without test functions and fixed seeds, we always experience variations from run to run, and it can be very difficult to spot bugs through averaged statistical quantities.

3.7.6 Demonstration of multiple walks

Assuming now that the code works, we can just scale up the number of steps in each walk and the number of walks. The latter influences the accuracy of the statistical estimates. Figure 3.18 shows the impact of the number of walks on the expectation, which should approach zero. Figure 3.19 displays the corresponding estimate of the variance of the position, which should grow linearly with the number of steps. It does, seemingly very accurately, but notice that the scale on the y axis is so much larger than for the expectation, so irregularities due to the stochastic nature of the process become so much less visible in the variance plots. The probability of finding a particle at a certain position at time (or step) 800 is shown in Figure 3.20. The dashed red line is the theoretical distribution (3.116) arising from solving the diffusion equation (3.114) instead. As always, we realize that one needs significantly more statistical samples to estimate a histogram accurately than the expectation or variance.

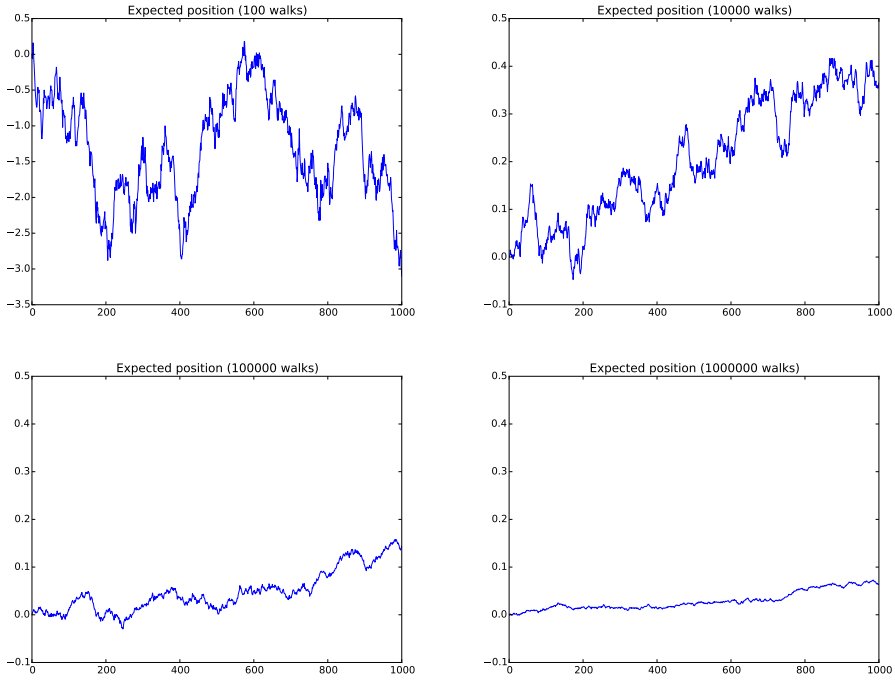


Fig. 3.18 Estimated expected value for 1000 steps, using 100 walks (upper left), 10,000 (upper right), 100,000 (lower left), and 1,000,000 (lower right).

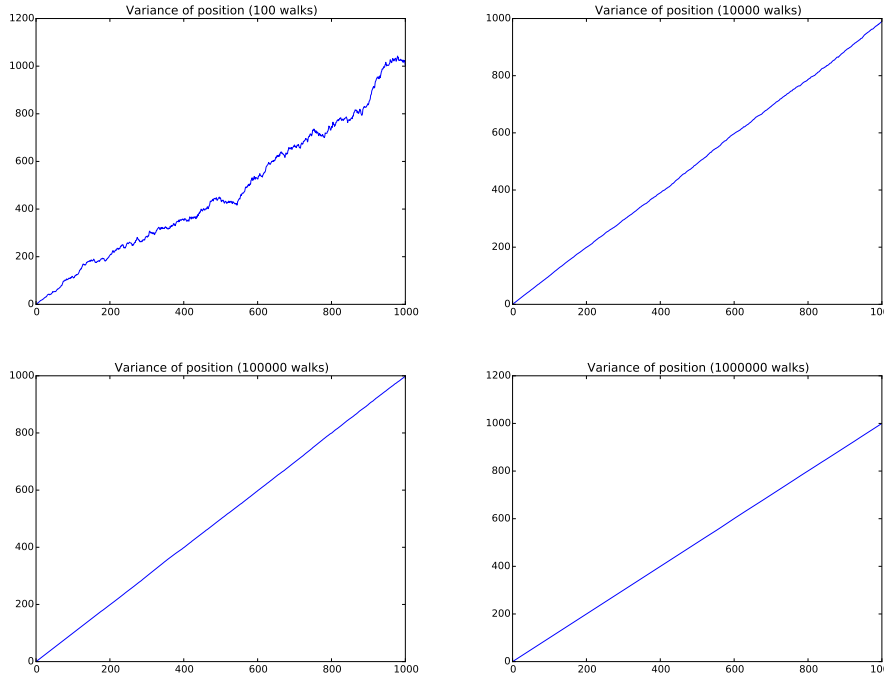


Fig. 3.19 Estimated variance over 1000 steps, using 100 walks (upper left), 10,000 (upper right), 100,000 (lower left), and 1,000,000 (lower right).

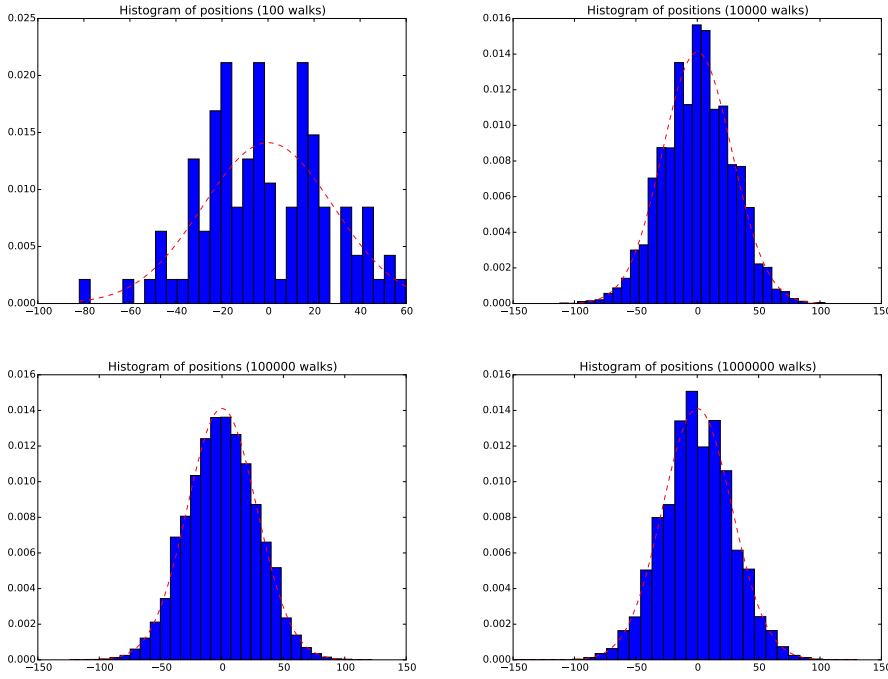


Fig. 3.20 Estimated probability distribution at step 800, using 100 walks (upper left), 10,000 (upper right), 100,000 (lower left), and 1,000,000 (lower right).

3.7.7 Ascii visualization of 1D random walk

If we want to study (very) long time series of random walks, it can be convenient to plot the position in a terminal window with the time axis pointing downwards. The module `avplotter` in SciTools has a class `Plotter` for plotting functions in the terminal window with the aid of ascii symbols only. Below is the code required to visualize a simple random walk, starting at the origin, and considered over when the point $x = -1$ is reached. We use a spacing $\Delta x = 0.05$ (so $x = -1$ corresponds to $i = -20$).

```
def run_random_walk():
    from scitools.avplotter import Plotter
    import time, numpy as np
    p = Plotter(-1, 1, width=75)   # Horizontal axis: 75 chars wide
    dx = 0.05
    np.random.seed(10)

    x = 0
    while True:
        random_step = 1 if np.random.random() > 0.5 else -1
        x = x + dx*random_step
        if x < -1:
            break                  # Destination reached!
        print p.plot(0, x)

        # Allow Ctrl+c to abort the simulation
        try:
            time.sleep(0.1)  # Wait for interrupt
        except KeyboardInterrupt:
            print 'Interrupted by Ctrl+c'
            break
```

Observe that we implement an infinite loop, but allow a smooth interrupt of the program by `Ctrl+c` through Python's `KeyboardInterrupt` exception. This is a useful recipe that can be used in many occasions!

The output looks typically like

```
                            *         |
                              *       |
                            *         |
                              *       |
                            *         |
                              *       |
                                 *    |
                              *       |
                            *         |
                          *           |
                        *             |
                      *               |
```

```
                    *           |
                   *            |
                  *             |
                 *              |
               *                |
             *                  |
```

Positions beyond the limits of the x axis appear with a value. A long file[9] contains the complete ascii plot corresponding to the function `run_random_walk` above.

3.7.8 Random walk as a stochastic equation

The (dimensionless) position in a random walk, $\bar{X}_k$, can be expressed as a stochastic difference equation:

$$\bar{X}_k = \bar{X}_{k-1} + s, \quad x_0 = 0, \tag{3.117}$$

where s is a Bernoulli variable[10], taking on the two values $s = -1$ and $s = 1$ with equal probability:

$$\mathrm{P}(s = 1) = \frac{1}{2}, \quad \mathrm{P}(s = -1) = \frac{1}{2}\,.$$

The s variable in a step is independent of the s variable in other steps.

The difference equation expresses essentially the sum of independent Bernoulli variables. Because of the central limit theorem, X_k, will then be normally distributed with expectation $k\mathrm{E}[s]$ and $k\mathrm{Var}[s]$. The expectation and variance of a Bernoulli variable with values $r = 0$ and $r = 1$ are p and $p(1-p)$, respectively. The variable $s = 2r - 1$ then has expectation $2\mathrm{E}[r] - 1 = 2p - 1 = 0$ and variance $2^2\mathrm{Var}[r] = 4p(1-p) = 1$. The position X_k is normally distributed with zero expectation and variance k, as we found in Section 3.7.2.

The central limit theorem tells that as long as k is not small, the distribution of X_k remains the same if we replace the Bernoulli variable s by any other stochastic variable with the same expectation and variance. In particular, we may let s be a standardized Gaussian variable (zero mean, unit variance).

Dividing (3.117) by Δt gives

[9] `http://bit.ly/1UbULeH`

[10] `https://en.wikipedia.org/wiki/Bernoulli_distribution`

$$\frac{\bar{X}_k - \bar{X}_{k-1}}{\Delta t} = \frac{1}{\Delta t} s\,.$$

In the limit $\Delta t \to 0$, $s/\Delta t$ approaches a white noise stochastic process. With $\bar{X}(t)$ as the continuous process in the limit $\Delta t \to 0$ ($X_k \to X(t_k)$), we formally get the stochastic differential equation

$$d\bar{X} = dW, \tag{3.118}$$

where $W(t)$ is a Wiener process[11]. Then X is also a Wiener process. It follows from the stochastic ODE $dX = dW$ that the probability distribution of X is given by the Fokker-Planck equation[12] (3.114). In other words, the key results for random walk we found earlier can alternatively be derived via a stochastic ordinary differential equation and its related Fokker-Planck equation.

3.7.9 Random walk in 2D

The most obvious generalization of 1D random walk to two spatial dimensions is to allow movements to the north, east, south, and west, with equal probability $\frac{1}{4}$.

```
def random_walk2D(x0, N, p, random=random):
    """2D random walk with 1 particle and N moves: N, E, W, S."""
    # Store position in step k in position[k]
    d = len(x0)
    position = np.zeros((N+1, d))
    position[0,:] = x0
    current_pos = np.array(x0, dtype=float)
    for k in range(N):
        r = random.uniform(0, 1)
        if r <= 0.25:
            current_pos += np.array([0, 1])   # Move north
        elif 0.25 < r <= 0.5:
            current_pos += np.array([1, 0])   # Move east
        elif 0.5 < r <= 0.75:
            current_pos += np.array([0, -1])  # Move south
        else:
            current_pos += np.array([-1, 0])  # Move west
        position[k+1,:] = current_pos
    return position
```

The left plot in Figure 3.21 provides an example on 200 steps with this kind of walk. We may refer to this walk as a walk on a *rectangular mesh*

[11] https://en.wikipedia.org/wiki/Wiener_process
[12] https://en.wikipedia.org/wiki/Fokker-Planck_equation

as we move from any spatial mesh point (i, j) to one of its four neighbors in the rectangular directions: $(i+1, j)$, $(i-1, j)$, $(i, j+1)$, or $(i, j-1)$.

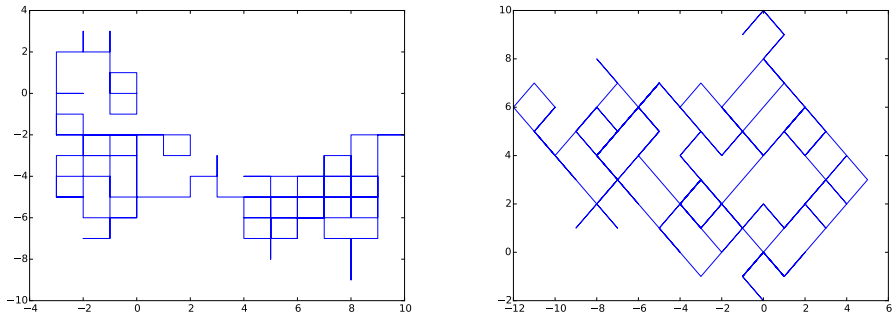


Fig. 3.21 Random walks in 2D with 200 steps: rectangular mesh (left) and diagonal mesh (right).

3.7.10 Random walk in any number of space dimensions

From a programming point of view, especially when implementing a random walk in any number of dimensions, it is more natural to consider a walk in the diagonal directions NW, NE, SW, and SE. On a two-dimensional spatial mesh it means that we go from (i, j) to either $(i+1, j+1)$, $(i-1, j+1)$, $(i+1, j-1)$, or $(i-1, j-1)$. We can with such a *diagonal mesh* (see right plot in Figure 3.21) draw a Bernoulli variable for the step in each spatial direction and trivially write code that works in any number of spatial directions:

```
def random_walkdD(x0, N, p, random=random):
    """Any-D (diagonal) random walk with 1 particle and N moves."""
    # Store position in step k in position[k]
    d = len(x0)
    position = np.zeros((N+1, d))
    position[0,:] = x0
    current_pos = np.array(x0, dtype=float)
    for k in range(N):
        for i in range(d):
            r = random.uniform(0, 1)
            if r <= p:
                current_pos[i] -= 1
            else:
                current_pos[i] += 1
        position[k+1,:] = current_pos
    return position
```

A vectorized version is desired. We follow the ideas from Section 3.7.3, but each step is now a vector in d spatial dimensions. We therefore need to draw Nd random numbers in `r`, compute steps in the various directions through `np.where(r <=p, -1, 1)` (each step being -1 or 1), and then we can reshape this array to an $N \times d$ array of step *vectors*. Doing an `np.cumsum` summation along axis 0 will add the vectors, as this demo shows:

```
>>> a = np.arange(6).reshape(3,2)
>>> a
array([[0, 1],
       [2, 3],
       [4, 5]])
>>> np.cumsum(a, axis=0)
array([[ 0,  1],
       [ 2,  4],
       [ 6,  9]])
```

With such summation of step vectors, we get all the positions to be filled in the `position` array:

```
def random_walkdD_vec(x0, N, p):
    """Vectorized version of random_walkdD."""
    d = len(x0)
    # Store position in step k in position[k]
    position = np.zeros((N+1,d))
    position[0] = np.array(x0, dtype=float)
    r = np.random.uniform(0, 1, size=N*d)
    steps = np.where(r <= p, -1, 1).reshape(N,d)
    position[1:,:] = x0 + np.cumsum(steps, axis=0)
    return position
```

3.7.11 Multiple random walks in any number of space dimensions

As we did in 1D, we extend one single walk to a number of walks (`num_walks` in the code).

Scalar code. As always, we start with implementing the scalar case:

```
def random_walksdD(x0, N, p, num_walks=1, num_times=1,
                   random=random):
    """Simulate num_walks random walks from x0 with N steps."""
    d = len(x0)
    position  = np.zeros((N+1, d))   # Accumulated positions
    position2 = np.zeros((N+1, d))   # Accumulated positions**2
    # Histogram at num_times selected time points
    pos_hist = np.zeros((num_walks, num_times, d))
```

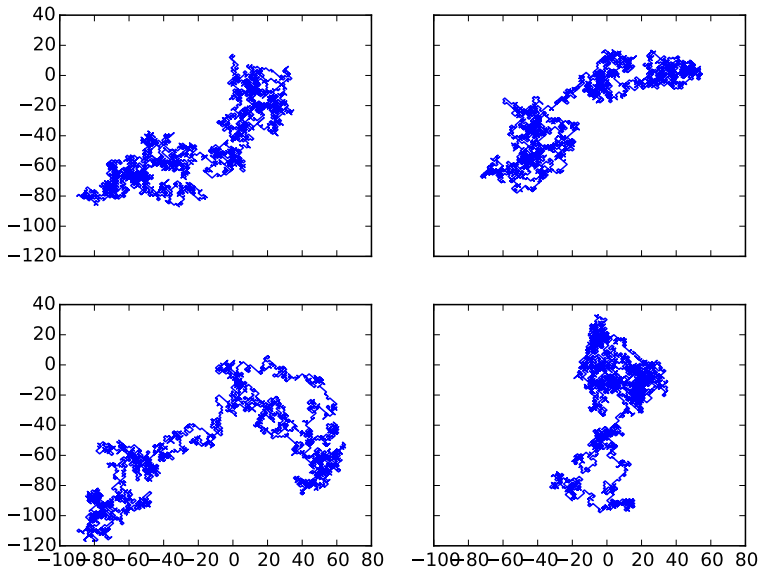


Fig. 3.22 Four random walks with 5000 steps in 2D.

```
    pos_hist_times = [(N//num_times)*i for i in range(num_times)]

    for n in range(num_walks):
        num_times_counter = 0
        current_pos = np.array(x0, dtype=float)
        for k in range(N):
            if k in pos_hist_times:
                pos_hist[n,num_times_counter,:] = current_pos
                num_times_counter += 1
            # current_pos corresponds to step k+1
            for i in range(d):
                r = random.uniform(0, 1)
                if r <= p:
                    current_pos[i] -= 1
                else:
                    current_pos[i] += 1
            position [k+1,:] += current_pos
            position2[k+1,:] += current_pos**2
    return position, position2, pos_hist, np.array(pos_hist_times)
```

Vectorized code. Significant speed-ups can be obtained by vectorization. We get rid of the loops in the previous function and arrive at the following vectorized code.

```
def random_walksdD_vec(x0, N, p, num_walks=1, num_times=1):
    """Vectorized version of random_walks1D; no loops."""
    d = len(x0)
    position  = np.zeros((N+1, d))  # Accumulated positions
    position2 = np.zeros((N+1, d))  # Accumulated positions**2
    walks = np.zeros((num_walks, N+1, d))  # Positions of each walk
    walks[:,0,:] = x0
    # Histogram at num_times selected time points
    pos_hist = np.zeros((num_walks, num_times, d))
    pos_hist_times = [(N//num_times)*i for i in range(num_times)]

    r = np.random.uniform(0, 1, size=N*num_walks*d)
    steps = np.where(r <= p, -1, 1).reshape(num_walks, N, d)
    walks[:,1:,:] = x0 + np.cumsum(steps, axis=1)
    position  = np.sum(walks,    axis=0)
    position2 = np.sum(walks**2, axis=0)
    pos_hist[:,:,:] = walks[:,pos_hist_times,:]
    return position, position2, pos_hist, np.array(pos_hist_times)
```

3.8 Applications

3.8.1 Diffusion of a substance

The first process to be considered is a substance that gets transported through a fluid at rest by pure diffusion. We consider an arbitrary volume V of this fluid, containing the substance with concentration function $c(\boldsymbol{x}, t)$. Physically, we can think of a very small volume with centroid $\boldsymbol{x}$ at time t and assign the ratio of the volume of the substance and the total volume to $c(\boldsymbol{x}, t)$. This means that the mass of the substance in a small volume ΔV is approximately $\varrho c \Delta V$, where ϱ is the density of the substance. Consequently, the total mass of the substance inside the volume V is the sum of all $\varrho c \Delta V$, which becomes the volume integral $\int_V \varrho c dV$.

Let us reason how the mass of the substance changes and thereby derive a PDE governing the concentration c. Suppose the substance flows out of V with a flux $\boldsymbol{q}$. If ΔS is a small part of the boundary ∂V of V, the volume of the substance flowing out through dS in a small time interval Δt is $\varrho \boldsymbol{q} \cdot \boldsymbol{n} \Delta t \Delta S$, where $\boldsymbol{n}$ is an outward unit normal to the boundary ∂V, see Figure 3.23. We realize that only the normal component of $\boldsymbol{q}$ is able to transport mass in and out of V. The total outflow of the mass of the substance in a small time interval Δt becomes the surface integral

$$\int_{\partial V} \varrho \boldsymbol{q} \cdot \boldsymbol{n} \Delta t \, dS \, .$$

Assuming conservation of mass, this outflow of mass must be balanced by a loss of mass inside the volume. The increase of mass inside the volume, during a small time interval Δt, is

$$\int_V \varrho(c(\boldsymbol{x}, t + \Delta t) - c(\boldsymbol{x}, t))dV,$$

assuming ϱ is constant, which is reasonable. The outflow of mass balances the loss of mass in V, which is the increase with a minus sign. Setting the two contributions equal to each other ensures balance of mass inside V. Dividing by Δt gives

$$\int_V \varrho \frac{c(\boldsymbol{x}, t + \Delta t) - c(\boldsymbol{x}, t)}{\Delta t} dV = -\int_{\partial V} \varrho \boldsymbol{q} \cdot \boldsymbol{n}\, dS\,.$$

Note the minus sign on the right-hand side: the left-hand side expresses loss of mass, while the integral on the right-hand side is the gain of mass.

Now, letting $\Delta t \to 0$, we have

$$\frac{c(\boldsymbol{x}, t + \Delta t) - c(\boldsymbol{x}, t)}{\Delta t} \to \frac{\partial c}{\partial t},$$

so

$$\int_V \varrho \frac{\partial c}{\partial t} dV + \int_{\partial V} \varrho \boldsymbol{q} \cdot \boldsymbol{n}\, dS = 0\,. \tag{3.119}$$

To arrive at a PDE, we express the surface integral as a volume integral using Gauss' divergence theorem:

$$\int_V (\varrho \frac{\partial c}{\partial t} + \nabla \cdot (\varrho \boldsymbol{q}))dV = 0\,.$$

Since ϱ is constant, we can divide by this quantity. If the integral is to vanish for an arbitrary volume V, the integrand must vanish too, and we get the mass conservation PDE for the substance:

$$\frac{\partial c}{\partial t} + \nabla \cdot \boldsymbol{q} = 0\,. \tag{3.120}$$

A fundamental problem is that this is a scalar PDE for four unknowns: c and the three components of $\boldsymbol{q}$. We therefore need additional equations. Here, Fick's law comes at rescue: it models how the flux $\boldsymbol{q}$ of the substance is related to the concentration c. Diffusion is recognized by mass flowing

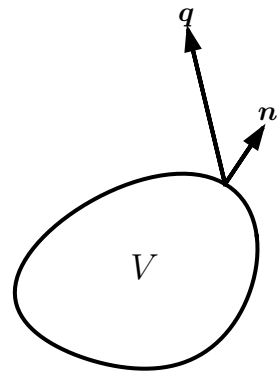


Fig. 3.23 An arbitrary volume of a fluid.

from regions with high concentration to regions of low concentration. This principle suggests that $\boldsymbol{q}$ is proportional to the negative gradient of c:

$$\boldsymbol{q} = -\alpha \nabla c, \tag{3.121}$$

where α is an empirically determined constant. The relation (3.121) is known as Fick's law. Inserting (3.121) in (3.120) gives a scalar PDE for the concentration c:

$$\frac{\partial c}{\partial t} = \alpha \nabla^2 c\,. \tag{3.122}$$

3.8.2 Heat conduction

Heat conduction is a well-known diffusion process. The governing PDE is in this case based on the first law of thermodynamics: the increase in energy of a system is equal to the work done on the system, plus the supplied heat. Here, we shall consider media at rest and neglect work done on the system. The principle then reduces to a balance between increase in internal energy and supplied heat flow by conduction.

Let $e(x,t)$ be the *internal energy* per unit mass. The increase of the internal energy in a small volume ΔV in a small time interval Δt is then

$$\varrho(e(\boldsymbol{x}, t+\Delta t) - e(\boldsymbol{x}, t))\Delta V,$$

where ϱ is the density of the material subject to heat conduction. In an arbitrary volume V, as depicted in Figure 3.23, the corresponding increase in internal energy becomes the volume integral

$$\int_V \varrho(e(\boldsymbol{x}, t + \Delta t) - e(\boldsymbol{x}, t))dV\,.$$

This increase in internal energy is balanced by heat supplied by conduction. Let $\boldsymbol{q}$ be the heat flow per time unit. Through the surface ∂V of V the following amount of heat flows out of V during a time interval Δt:

$$\int_{\partial V} \boldsymbol{q} \cdot \boldsymbol{n} \Delta t\, dS\,.$$

The simplified version of the first law of thermodynamics then states that

$$\int_V \varrho(e(\boldsymbol{x}, t + \Delta t) - e(\boldsymbol{x}, t))dV = -\int_{\partial V} \boldsymbol{q} \cdot \boldsymbol{n} \Delta t\, dS\,.$$

The minus sign on the right-hand side ensures that the integral there models net *inflow* of heat (since $\boldsymbol{n}$ is an outward unit normal, $\boldsymbol{q} \cdot \boldsymbol{n}$ models *outflow*). Dividing by Δt and notifying that

$$\lim_{\Delta t \to 0} \frac{e(\boldsymbol{x}, t + \Delta t) - e(\boldsymbol{x}, t)}{\Delta t} = \frac{\partial e}{\partial t},$$

we get (in the limit $\Delta t \to 0$)

$$\int_V \varrho \frac{\partial e}{\partial t} dV + \int_{\partial V} \boldsymbol{q} \cdot \boldsymbol{n} \Delta t\, dS = 0\,.$$

This is the integral equation for heat conduction, but we aim at a PDE. The next step is therefore to transform the surface integral to a volume integral via Gauss' divergence theorem. The result is

$$\int_V \left(\varrho \frac{\partial e}{\partial t} + \nabla \cdot \boldsymbol{q} \right) dV = 0\,.$$

If this equality is to hold for all volumes V, the integrand must vanish, and we have the PDE

$$\varrho \frac{\partial e}{\partial t} = -\nabla \cdot \boldsymbol{q}\,. \tag{3.123}$$

Sometimes the supplied heat can come from the medium itself. This is the case, for instance, when radioactive rock generates heat. Let us add this effect. If $f(\boldsymbol{x}, t)$ is the supplied heat per unit volume per unit time, the heat supplied in a small volume is $f\Delta t\Delta V$, and inside an arbitrary volume V the supplied generated heat becomes

$$\int_V f\Delta t dV\,.$$

Adding this to the integral statement of the (simplified) first law of thermodynamics, and continuing the derivation, leads to the PDE

$$\varrho\frac{\partial e}{\partial t} = -\nabla\cdot\boldsymbol{q} + f\,. \tag{3.124}$$

There are four unknown scalar fields: e and $\boldsymbol{q}$. Moreover, the temperature T, which is our primary quantity to compute, does not enter the model yet. We need an additional equation, called the *equation of state*, relating e, $V = 1/\varrho =$, and T: $e = e(V, T)$. By the chain rule we have

$$\frac{\partial e}{\partial t} = \frac{\partial e}{\partial T}\bigg|_V\frac{\partial T}{\partial t} + \frac{\partial e}{\partial V}\bigg|_T\frac{\partial V}{\partial t}\,.$$

The first coefficient $\partial e/\partial T$ is called *specific heat capacity at constant volume*, denoted by c_v:

$$c_v = \frac{\partial e}{\partial T}\bigg|_V\,.$$

The specific heat capacity will in general vary with T, but taking it as a constant is a good approximation in many applications.

The term $\partial e/\partial V$ models effects due to compressibility and volume expansion. These effects are often small and can be neglected. We shall do so here. Using $\partial e/\partial t = c_v\partial T/\partial t$ in the PDE gives

$$\varrho c_v\frac{\partial T}{\partial t} = -\nabla\cdot\boldsymbol{q} + f\,.$$

We still have four unknown scalar fields (T and $\boldsymbol{q}$). To close the system, we need a relation between the heat flux $\boldsymbol{q}$ and the temperature T called *Fourier's law*:

$$\boldsymbol{q} = -k\nabla T,$$

which simply states that heat flows from hot to cold areas, along the path of greatest variation. In a solid medium, k depends on the material of the medium, and in multi-material media one must regard k as spatially dependent. In a fluid, it is common to assume that k is constant. The value of k reflects how easy heat is conducted through the medium, and k is named the *coefficient of heat conduction*.

We now have one scalar PDE for the unknown temperature field $T(\boldsymbol{x}, t)$:

$$\varrho c_v \frac{\partial T}{\partial t} = \nabla \cdot (k \nabla T) + f \,. \tag{3.125}$$

3.8.3 Porous media flow

The requirement of mass balance for flow of a single, incompressible fluid through a deformable (elastic) porous medium leads to the equation

$$S\frac{\partial p}{\partial t} + \nabla \cdot (\boldsymbol{q} - \alpha \frac{\partial \boldsymbol{u}}{\partial t}) = 0,$$

where p is the fluid pressure, $\boldsymbol{q}$ is the fluid velocity, $\boldsymbol{u}$ is the displacement (deformation) of the medium, S is the storage coefficient of the medium (related to the compressibility of the fluid and the material in the medium), and α is another coefficient. In many circumstances, the last term with $\boldsymbol{u}$ can be neglected, an assumption that decouples the equation above from a model for the deformation of the medium. The famous *Darcy's law* relates $\boldsymbol{q}$ to p:

$$\boldsymbol{q} = -\frac{K}{\mu}(\nabla p - \varrho \boldsymbol{g}),$$

where K is the permeability of the medium, μ is the dynamic viscosity of the fluid, ϱ is the density of the fluid, and $\boldsymbol{g}$ is the acceleration of gravity, here taken as $\boldsymbol{g} = -g\boldsymbol{k}$. Combining the two equations results in the diffusion model

$$S\frac{\partial p}{\partial t} = \mu^{-1}\nabla(K\nabla p) + \frac{\varrho g}{\mu}\frac{\partial K}{\partial z} \,. \tag{3.126}$$

Boundary conditions consist of specifying p or $\boldsymbol{q} \cdot \boldsymbol{n}$ (i.e., normal velocity) at each point of the boundary.

3.8.4 Potential fluid flow

Let $\boldsymbol{v}$ be the velocity of a fluid. The condition $\nabla \times \boldsymbol{v} = 0$ is relevant for many flows, especially in geophysics when viscous effects are negligible. From vector calculus it is known that $\nabla \times \boldsymbol{v} = 0$ implies that v can be derived from a scalar potential field ϕ: $\boldsymbol{v} = \nabla\phi$. If the fluid is incompressible, $\nabla \cdot \boldsymbol{v} = 0$, it follows that $\nabla \cdot \nabla\phi = 0$, or

$$\nabla^2\phi = 0\,. \tag{3.127}$$

This Laplace equation is sufficient for determining ϕ and thereby describe the fluid motion. This type of flow is known as potential flow[13]. One very important application where potential flow is a good model is water waves. As boundary condition we must prescribe $\boldsymbol{v} \cdot \boldsymbol{n} = \partial\phi/\partial n$. This gives rise to what is known as a pure Neumann problem and will cause numerical difficulties because ϕ and ϕ plus any constant are two solutions of the problem. The simplest remedy is to fix the value of ϕ at a point.

3.8.5 Streamlines for 2D fluid flow

The streamlines in a two-dimensional stationary fluid flow are lines tangential to the flow. The stream function[14] ψ is often introduced in two-dimensional flow such that its contour lines, $\psi = \text{const}$, gives the streamlines. The relation between ψ and the velocity field $\boldsymbol{v} = (u, v)$ is

$$u = \frac{\partial\psi}{\partial y}, \quad v = -\frac{\partial\psi}{\partial x}\,.$$

It follows that $\nabla\boldsymbol{v} = \psi_{yx} - \psi_{xy} = 0$, so the stream function can only be used for incompressible flows. Since

$$\nabla \times \boldsymbol{v} = \left(\frac{\partial v}{\partial y} - \frac{\partial u}{\partial x}\right)\boldsymbol{k} \equiv \omega\boldsymbol{k},$$

we can derive the relation

$$\nabla^2\psi = -\omega, \tag{3.128}$$

which is a governing equation for the stream function $\psi(x, y)$ if the vorticity ω is known.

[13] https://en.wikipedia.org/wiki/Potential_flow
[14] https://en.wikipedia.org/wiki/Stream_function

3.8.6 The potential of an electric field

Under the assumption of time independence, Maxwell's equations for the electric field $\boldsymbol{E}$ become

$$\begin{aligned} \nabla \cdot \boldsymbol{E} &= \frac{\rho}{\epsilon_0}, \\ \nabla \times \boldsymbol{E} &= 0, \end{aligned}$$

where ρ is the electric charge density and ϵ_0 is the electric permittivity of free space (i.e., vacuum). Since $\nabla \times \boldsymbol{E} = 0$, $\boldsymbol{E}$ can be derived from a potential φ, $\boldsymbol{E} = -\nabla\varphi$. The electric field potential is therefore governed by the Poisson equation

$$\nabla^2\varphi = -\frac{\rho}{\epsilon_0}\,. \tag{3.129}$$

If the medium is heterogeneous, ρ will depend on the spatial location $\boldsymbol{r}$. Also, ϵ_0 must be exchanged with an electric permittivity function $\epsilon(\boldsymbol{r})$.

Each point of the boundary must be accompanied by, either a Dirichlet condition $\varphi(\boldsymbol{r}) = \varphi_D(\boldsymbol{r})$, or a Neumann condition $\frac{\partial\varphi(\boldsymbol{r})}{\partial n} = \varphi_N(\boldsymbol{r})$.

3.8.7 Development of flow between two flat plates

Diffusion equations may also arise as simplified versions of other mathematical models, especially in fluid flow. Consider a fluid flowing between two flat, parallel plates. The velocity is uni-directional, say along the z axis, and depends only on the distance x from the plates; $\boldsymbol{u} = u(x,t)\boldsymbol{k}$. The flow is governed by the Navier-Stokes equations,

$$\begin{aligned} \varrho\frac{\partial \boldsymbol{u}}{\partial t} + \varrho\boldsymbol{u}\cdot\nabla\boldsymbol{u} &= -\nabla p + \mu\nabla^2\boldsymbol{u} + \varrho\boldsymbol{f}, \\ \nabla\cdot\boldsymbol{u} &= 0, \end{aligned}$$

where p is the pressure field, unknown along with the velocity $\boldsymbol{u}$, ϱ is the fluid density, μ the dynamic viscosity, and $\boldsymbol{f}$ is some external body force. The geometric restrictions of flow between two flat plates puts restrictions on the velocity, $\boldsymbol{u} = u(x,t)\boldsymbol{i}$, and the z component of the Navier-Stokes equations collapses to a diffusion equation:

$$\varrho\frac{\partial u}{\partial t} = -\frac{\partial p}{\partial z} + \mu\frac{\partial^2 u}{\partial z^2} + \varrho f_z,$$

if f_z is the component of $\boldsymbol{f}$ in the z direction.

The boundary conditions are derived from the fact that the fluid sticks to the plates, which means $\boldsymbol{u} = 0$ at the plates. Say the location of the plates are $z = 0$ and $z = L$. We then have

$$u(0,t) = u(L,t) = 0\,.$$

One can easily show that $\partial p/\partial z$ must be a constant or just a function of time t. We set $\partial p/\partial z = -\beta(t)$. The body force could be a component of gravity, if desired, set as $f_z = \gamma g$. Switching from z to x as independent variable gives a very standard one-dimensional diffusion equation:

$$\varrho\frac{\partial u}{\partial t} = \mu\frac{\partial^2 u}{\partial x^2} + \beta(t) + \varrho\gamma g, \quad x \in [0,L],\ t \in (0,T]\,.$$

The boundary conditions are

$$u(0,t) = u(L,t) = 0,$$

while some initial condition

$$u(x,0) = I(x)$$

must also be prescribed.

The flow is driven by either the pressure gradient β or gravity, or a combination of both. One may also consider one moving plate that drives the fluid. If the plate at $x = L$ moves with velocity $U_L(t)$, we have the adjusted boundary condition

$$u(L,t) = U_L(t)\,.$$

3.8.8 Flow in a straight tube

Now we consider viscous fluid flow in a straight tube with radius R and rigid walls. The governing equations are the Navier-Stokes equations, but as in Section 3.8.7, it is natural to assume that the velocity is directed along the tube, and that it is axi-symmetric. These assumptions reduced the velocity field to $\boldsymbol{u} = u(r,x,t)\boldsymbol{i}$, if the x axis is directed along the tube. From the equation of continuity, $\nabla\cdot\boldsymbol{u} = 0$, we see that u must be

independent of x. Inserting $\boldsymbol{u} = u(r,t)\boldsymbol{i}$ in the Navier-Stokes equations, expressed in axi-symmetric cylindrical coordinates, results in

$$\varrho\frac{\partial u}{\partial t} = \mu\frac{1}{r}\frac{\partial}{\partial r}\left(r\frac{\partial u}{\partial r}\right) + \beta(t) + \varrho\gamma g, \quad r \in [0,R],\ t \in (0,T]\,. \tag{3.130}$$

Here, $\beta(t) = -\partial p/\partial x$ is the pressure gradient along the tube. The associated boundary condition is $u(R,t) = 0$.

3.8.9 Tribology: thin film fluid flow

Thin fluid films are extremely important inside machinery to reduce friction between gliding surfaces. The mathematical model for the fluid motion takes the form of a diffusion problem and is quickly derived here. We consider two solid surfaces whose distance is described by a gap function $h(x,y)$. The space between these surfaces is filled with a fluid with dynamic viscosity μ. The fluid may move partially because of pressure gradients and partially because the surfaces move. Let $U\boldsymbol{i} + V\boldsymbol{j}$ be the relative velocity of the two surfaces and p the pressure in the fluid. The mathematical model builds on two principles: 1) conservation of mass, 2) assumption of locally quasi-static flow between flat plates.

The conservation of mass equation reads $\nabla\cdot\boldsymbol{u}$, where $\boldsymbol{u}$ is the local fluid velocity. For thin films the detailed variation between the surfaces is not of interest, so $\nabla\cdot\boldsymbol{u} = 0$ is integrated (average) in the direction perpendicular to the surfaces. This gives rise to the alternative mass conservation equation

$$\nabla\cdot\boldsymbol{q} = 0, \quad \boldsymbol{q} = \int_0^{h(x,y)} \boldsymbol{u}dz,$$

where z is the coordinate perpendicular to the surfaces, and $\boldsymbol{q}$ is then the volume flux in the fluid gap.

Locally, we may assume that we have steady flow between two flat surfaces, with a pressure gradient and where the lower surface is at rest and the upper moves with velocity $U\boldsymbol{i} + V\boldsymbol{j}$. The corresponding mathematical problem is actually the limit problem in Section 3.8.7 as $t \to \infty$. The limit problem can be solved analytically, and the local volume flux becomes

$$\boldsymbol{q}(x,y,z) = \int_0^h \boldsymbol{u}(x,y,z)dz = -\frac{h^3}{12\mu}\nabla p + \frac{1}{2}Uh\boldsymbol{i} + \frac{1}{2}Vh\boldsymbol{j}\,.$$

The idea is to use this expression locally also when the surfaces are not flat, but slowly varying, and if U, V, or p varies in time, provided the time variation is sufficiently slow. This is a common quasi-static approximation, much used in mathematical modeling.

Inserting the expression for $\boldsymbol{q}$ via p, U, and V in the equation $\nabla\boldsymbol{q} = 0$ gives a diffusion PDE for p:

$$\nabla\cdot\left(\frac{h^3}{12\mu}\nabla p\right) = \frac{1}{2}\frac{\partial}{\partial x}(hU) + \frac{1}{2}\frac{\partial}{\partial x}(hV)\,. \tag{3.131}$$

The boundary conditions must involve p or $\boldsymbol{q}$ at the boundary.

3.8.10 Propagation of electrical signals in the brain

One can make a model of how electrical signals are propagated along the neuronal fibers that receive synaptic inputs in the brain. The signal propagation is one-dimensional and can, in the simplest cases, be governed by the Cable equation[15]:

$$c_m\frac{\partial V}{\partial t} = \frac{1}{r_l}\frac{\partial^2 V}{\partial x^2} - \frac{1}{r_m}V \tag{3.132}$$

where $V(x,t)$ is the voltage to be determined, c_m is capacitance of the neuronal fiber, while r_l and r_m are measures of the resistance. The boundary conditions are often taken as $V = 0$ at a short circuit or open end, $\partial V/\partial x = 0$ at a sealed end, or $\partial V/\partial x \propto V$ where there is an injection of current.

3.9 Exercises

Exercise 3.6: Stabilizing the Crank-Nicolson method by Rannacher time stepping

It is well known that the Crank-Nicolson method may give rise to non-physical oscillations in the solution of diffusion equations if the initial data

[15] `http://en.wikipedia.org/wiki/Cable_equation`

exhibit jumps (see Section 3.3.6). Rannacher [15] suggested a stabilizing technique consisting of using the Backward Euler scheme for the first two time steps with step length $\frac{1}{2}\Delta t$. One can generalize this idea to taking $2m$ time steps of size $\frac{1}{2}\Delta t$ with the Backward Euler method and then continuing with the Crank-Nicolson method, which is of second-order in time. The idea is that the high frequencies of the initial solution are quickly damped out, and the Backward Euler scheme treats these high frequencies correctly. Thereafter, the high frequency content of the solution is gone and the Crank-Nicolson method will do well.

Test this idea for $m = 1, 2, 3$ on a diffusion problem with a discontinuous initial condition. Measure the convergence rate using the solution (3.45) with the boundary conditions (3.46)-(3.47) for t values such that the conditions are in the vicinity of ± 1. For example, $t < 5a1.6 \cdot 10^{-2}$ makes the solution diffusion from a step to almost a straight line. The program `diffu_erf_sol.py` shows how to compute the analytical solution.

Project 3.7: Energy estimates for diffusion problems

This project concerns so-called *energy estimates* for diffusion problems that can be used for qualitative analytical insight and for verification of implementations.

a) We start with a 1D homogeneous diffusion equation with zero Dirichlet conditions:

$$u_t = \alpha u_x x, x \in \Omega = (0, L), \ t \in (0, T], \tag{3.133}$$

$$u(0, t) = u(L, t) = 0, \qquad t \in (0, T], \tag{3.134}$$

$$u(x, 0) = I(x), \qquad x \in [0, L]\,. \tag{3.135}$$

The energy estimate for this problem reads

$$||u||_{L^2} \leq ||I||_{L^2}, \tag{3.136}$$

where the $||\cdot||_{L^2}$ norm is defined by

$$||g||_{L^2} = \sqrt{\int_0^L g^2 dx}\,. \tag{3.137}$$

The quantify $||u||_{L^2}$ or $\frac{1}{2}||u||_{L^2}$ is known as the *energy* of the solution, although it is not the physical energy of the system. A mathematical tradition has introduced the notion *energy* in this context.

The estimate (3.136) says that the "size of u" never exceeds that of the initial condition, or more precisely, it says that the area under the u curve decreases with time.

To show (3.136), multiply the PDE by u and integrate from 0 to L. Use that uu_t can be expressed as the time derivative of u^2 and that u_xxu can integrated by parts to form an integrand u_x^2. Show that the time derivative of $||u||_{L^2}^2$ must be less than or equal to zero. Integrate this expression and derive (3.136).

b) Now we address a slightly different problem,

$$u_t = \alpha u_x x + f(x,t), x \in \Omega = (0,L), \ t \in (0,T], \quad (3.138)$$
$$u(0,t) = u(L,t) = 0, \qquad t \in (0,T], \quad (3.139)$$
$$u(x,0) = 0, \qquad x \in [0,L]\,. \quad (3.140)$$

The associated energy estimate is

$$||u||_{L^2} \leq ||f||_{L^2}\,. \quad (3.141)$$

(This result is more difficult to derive.)

Now consider the compound problem with an initial condition $I(x)$ and a right-hand side $f(x,t)$:

$$u_t = \alpha u_x x + f(x,t), x \in \Omega = (0,L), \ t \in (0,T], \quad (3.142)$$
$$u(0,t) = u(L,t) = 0, \qquad t \in (0,T], \quad (3.143)$$
$$u(x,0) = I(x), \qquad x \in [0,L]\,. \quad (3.144)$$

Show that if w_1 fulfills (3.133)-(3.135) and w_2 fulfills (3.138)-(3.140), then $u = w_1 + w_2$ is the solution of (3.142)-(3.144). Using the triangle inequality for norms,

$$||a+b|| \leq ||a|| + ||b||,$$

show that the energy estimate for (3.142)-(3.144) becomes

$$||u||_{L^2} \leq ||I||_{L^2} + ||f||_{L^2}\,. \quad (3.145)$$

c) One application of (3.145) is to prove uniqueness of the solution. Suppose u_1 and u_2 both fulfill (3.142)-(3.144). Show that $u = u_1 - u_2$ then fulfills (3.142)-(3.144) with $f = 0$ and $I = 0$. Use (3.145) to deduce that the energy must be zero for all times and therefore that $u_1 = u_2$, which proves that the solution is unique.

d) Generalize (3.145) to a 2D/3D diffusion equation $u_t = \nabla \cdot (\alpha \nabla u)$ for $x \in \Omega$.

Hint. Use integration by parts in multi dimensions:

$$\int_\Omega u \nabla \cdot (\alpha \nabla u)\,\mathrm{d}x = -\int_\Omega \alpha \nabla u \cdot \nabla u\,\mathrm{d}x + \int_{\partial\Omega} u\alpha \frac{\partial u}{\partial n},$$

where $\frac{\partial u}{\partial n} = \boldsymbol{n} \cdot \nabla u$, $\boldsymbol{n}$ being the outward unit normal to the boundary $\partial\Omega$ of the domain Ω.

e) Now we also consider the multi-dimensional PDE $u_t = \nabla \cdot (\alpha \nabla u)$. Integrate both sides over Ω and use Gauss' divergence theorem, $\int_\Omega \nabla \cdot \boldsymbol{q}\,\mathrm{d}x = \int_{\partial\Omega} \boldsymbol{q} \cdot \boldsymbol{n}\,\mathrm{d}s$ for a vector field $\boldsymbol{q}$. Show that if we have homogeneous Neumann conditions on the boundary, $\partial u/\partial n = 0$, area under the u surface remains constant in time and

$$\int_\Omega u\,\mathrm{d}x = \int_\Omega I\,\mathrm{d}x\,. \tag{3.146}$$

f) Establish a code in 1D, 2D, or 3D that can solve a diffusion equation with a source term f, initial condition I, and zero Dirichlet or Neumann conditions on the whole boundary.

We can use (3.145) and (3.146) as a partial verification of the code. Choose some functions f and I and check that (3.145) is obeyed at any time when zero Dirichlet conditions are used. Iterate over the same I functions and check that (3.146) is fulfilled when using zero Neumann conditions.

g) Make a list of some possible bugs in the code, such as indexing errors in arrays, failure to set the correct boundary conditions, evaluation of a term at a wrong time level, and similar. For each of the bugs, see if the verification tests from the previous subexercise pass or fail. This investigation shows how strong the energy estimates and the estimate (3.146) are for pointing out errors in the implementation.
Filename: `diffu_energy`.

Exercise 3.8: Splitting methods and preconditioning

In Section 3.6.15, we outlined a class of iterative methods for $Au = b$ based on splitting A into $A = M - N$ and introducing the iteration

$$Mu^k = Nu^k + b\,.$$

The very simplest splitting is $M = I$, where I is the identity matrix. Show that this choice corresponds to the iteration

$$u^k = u^{k-1} + r^{k-1}, \quad r^{k-1} = b - Au^{k-1}, \tag{3.147}$$

where r^{k-1} is the residual in the linear system in iteration $k-1$. The formula (3.147) is known as Richardson's iteration. Show that if we apply the simple iteration method (3.147) to the *preconditioned* system $M^{-1}Au = M^{-1}b$, we arrive at the Jacobi method by choosing $M = D$ (the diagonal of A) as preconditioner and the SOR method by choosing $M = \omega^{-1}D + L$ (L being the lower triangular part of A). This equivalence shows that we can apply one iteration of the Jacobi or SOR method as preconditioner.

Problem 3.9: Oscillating surface temperature of the earth

Consider a day-and-night or seasonal variation in temperature at the surface of the earth. How deep down in the ground will the surface oscillations reach? For simplicity, we model only the vertical variation along a coordinate x, where $x = 0$ at the surface, and x increases as we go down in the ground. The temperature is governed by the heat equation

$$\varrho c_v \frac{\partial T}{\partial t} = \nabla \cdot (k\nabla T),$$

in some spatial domain $x \in [0, L]$, where L is chosen large enough such that we can assume that T is approximately constant, independent of the surface oscillations, for $x > L$. The parameters ϱ, c_v, and k are the density, the specific heat capacity at constant volume, and the heat conduction coefficient, respectively.

a) Derive the mathematical model for computing $T(x, t)$. Assume the surface oscillations to be sinusoidal around some mean temperature T_m. Let $T = T_m$ initially. At $x = L$, assume $T \approx T_m$.

b) Scale the model in a) assuming k is constant. Use a time scale $t_c = \omega^{-1}$ and a length scale $x_c = \sqrt{2\alpha/\omega}$, where $\alpha = k/(\varrho c_v)$. The primary unknown can be scaled as $\frac{T-T_m}{2A}$.

Show that the scaled PDE is

$$\frac{\partial u}{\partial \bar{t}} = \frac{1}{2}\frac{\partial^2 u}{\partial x^2},$$

with initial condition $u(\bar{x}, 0) = 0$, left boundary condition $u(0, \bar{t}) = \sin(\bar{t})$, and right boundary condition $u(\bar{L}, \bar{t}) = 0$. The bar indicates a dimensionless quantity.

Show that $u(\bar{x}, \bar{t}) = e^{-\bar{x}} \sin(\bar{x} - \bar{t})$ is a solution that fulfills the PDE and the boundary condition at $\bar{x} = 0$ (this is the solution we will experience as $\bar{t} \to \infty$ and $L \to \infty$). Conclude that an appropriate domain for x is $[0, 4]$ if a damping $e^{-4} \approx 0.18$ is appropriate for implementing $\bar{u} \approx$ const; increasing to $[0, 6]$ damps $\bar{u}$ to 0.0025.

c) Compute the scaled temperature and make animations comparing two solutions with $\bar{L} = 4$ and $\bar{L} = 8$, respectively (keep Δx the same).

Problem 3.10: Oscillating and pulsating flow in tubes

We consider flow in a straight tube with radius R and straight walls. The flow is driven by a pressure gradient $\beta(t)$. The effect of gravity can be neglected. The mathematical problem reads

$$\varrho\frac{\partial u}{\partial t} = \mu\frac{1}{r}\frac{\partial}{\partial r}\left(r\frac{\partial u}{\partial r}\right) + \beta(t), \quad r \in [0, R],\ t \in (0, T], \tag{3.148}$$

$$u(r, 0) = I(r), \qquad r \in [0, R], \tag{3.149}$$

$$u(R, t) = 0, \qquad t \in (0, T], \tag{3.150}$$

$$\frac{\partial u}{\partial r}(0, t) = 0, \qquad t \in (0, T]. \tag{3.151}$$

We consider two models for $\beta(t)$. One plain, sinusoidal oscillation:

$$\beta = A\sin(\omega t), \tag{3.152}$$

and one with periodic pulses,

$$\beta = A\sin^{16}(\omega t), \tag{3.153}$$

Note that both models can be written as $\beta = A\sin^m(\omega t)$, with $m = 1$ and $m = 16$, respectively.

a) Scale the mathematical model, using the viscous time scale $\varrho R^2/\mu$.

b) Implement the scaled model from a), using the unifying θ scheme in time and centered differences in space.

c) Verify the implementation in b) using a manufactured solution that is quadratic in r and linear in t. Make a corresponding test function.

Hint. You need to include an extra source term in the equation to allow for such tests. Let the spatial variation be $1 - r^2$ such that the boundary condition is fulfilled.

d) Make animations for $m = 1, 16$ and $\alpha = 1, 0.1$. Choose T such that the motion has reached a steady state (non-visible changes from period to period in u).

e) For $\alpha \gg 1$, the scaling in a) is not good, because the characteristic time for changes (due to the pressure) is much smaller than the viscous diffusion time scale (α becomes large). We should in this case base the short time scale on $1/\omega$. Scale the model again, and make an animation for $m = 1, 16$ and $\alpha = 10$.
Filename: `axisymm_flow`.

Problem 3.11: Scaling a welding problem

Welding equipment makes a very localized heat source that moves in time. We shall investigate the heating due to welding and choose, for maximum simplicity, a one-dimensional heat equation with a fixed temperature at the ends, and we neglect melting. We shall scale the problem, and besides solving such a problem numerically, the aim is to investigate the appropriateness of alternative scalings.

The governing PDE problem reads

$$\begin{aligned} \varrho c\frac{\partial u}{\partial t} &= k\frac{\partial^2 u}{\partial x^2} + f, & x \in (0,L),\ t \in (0,T),\\ u(x,0) &= U_s, & x \in [0,L],\\ u(0,t) = u(L,t) &= 0, & t \in (0,T]. \end{aligned}$$

Here, u is the temperature, ϱ the density of the material, c a heat capacity, k the heat conduction coefficient, f is the heat source from the welding

equipment, and U_s is the initial constant (room) temperature in the material.

A possible model for the heat source is a moving Gaussian function:

$$f = A\exp\left(-\frac{1}{2}\left(\frac{x-vt}{\sigma}\right)^2\right),$$

where A is the strength, σ is a parameter governing how peak-shaped (or localized in space) the heat source is, and v is the velocity (in positive x direction) of the source.

a) Let x_c, t_c, u_c, and f_c be scales, i.e., characteristic sizes, of x, t, u, and f, respectively. The natural choice of x_c and f_c is L and A, since these make the scaled x and f in the interval $[0,1]$. If each of the three terms in the PDE are equally important, we can find t_c and u_c by demanding that the coefficients in the scaled PDE are all equal to unity. Perform this scaling. Use scaled quantities in the arguments for the exponential function in f too and show that

$$\bar{f} = e^{-\frac{1}{2}\beta^2(\bar{x}-\gamma\bar{t})^2},$$

where β and γ are dimensionless numbers. Give an interpretation of β and γ.

b) Argue that for large γ we should base the time scale on the movement of the heat source. Show that this gives rise to the scaled PDE

$$\frac{\partial\bar{u}}{\partial\bar{t}} = \gamma^{-1}\frac{\partial^2\bar{u}}{\partial\bar{x}^2} + \bar{f},$$

and

$$\bar{f} = \exp\left(-\frac{1}{2}\beta^2(\bar{x}-\bar{t})^2\right).$$

Discuss when the scalings in a) and b) are appropriate.

c) One aim with scaling is to get a solution that lies in the interval $[-1,1]$. This is not always the case when u_c is based on a scale involving a source term, as we do in a) and b). However, from the scaled PDE we realize that if we replace $\bar{f}$ with $\delta\bar{f}$, where δ is a dimensionless factor, this corresponds to replacing u_c by u_c/δ. So, if we observe that $\bar{u}\sim 1/\delta$ in simulations, we can just replace $\bar{f}$ by $\delta\bar{f}$ in the scaled PDE.

Use this trick and implement the two scaled models. Reuse software for the diffusion equation (e.g., the `solver` function in

`diffu1D_vc.py`). Make a function `run(gamma, beta=10, delta=40, scaling=1, animate=False)` that runs the model with the given γ, β, and δ parameters as well as an indicator `scaling` that is 1 for the scaling in a) and 2 for the scaling in b). The last argument can be used to turn screen animations on or off.

Experiments show that with $\gamma = 1$ and $\beta = 10$, $\delta = 20$ is appropriate. Then $\max|\bar{u}|$ will be larger than 4 for $\gamma = 40$, but that is acceptable.

Equip the `run` function with visualization, both animation of $\bar{u}$ and $\bar{f}$, and plots with $\bar{u}$ and $\bar{f}$ for $t = 0.2$ and $t = 0.5$.

Hint. Since the amplitudes of $\bar{u}$ and $\bar{f}$ differs by a factor δ, it is attractive to plot $\bar{f}/\delta$ together with $\bar{u}$.

d) Use the software in c) to investigate $\gamma = 0.2, 1, 5, 40$ for the two scalings. Discuss the results.
Filename: `welding`.

Exercise 3.12: Implement a Forward Euler scheme for axi-symmetric diffusion

Based on the discussion in Section 3.5.6, derive in detail the discrete equations for a Forward Euler in time, centered in space, finite difference method for axi-symmetric diffusion. The diffusion coefficient may be a function of the radial coordinate. At the outer boundary $r = R$, we may have either a Dirichlet or Robin condition. Implement this scheme. Construct appropriate test problems.
Filename: `FE_axisym`.

Advection-dominated equations

4

Wave (Chapter 2) and diffusion (Chapter 3) equations are solved reliably by finite difference methods. As soon as we add a first-order derivative in space, representing *advective* transport (also known as *convective* transport), the numerics gets more complicated and intuitively attractive methods no longer work well. We shall show how and why such methods fail and provide remedies. The present chapter builds on basic knowledge about finite difference methods for diffusion and wave equations, including the analysis by Fourier components, truncation error analysis (Appendix B), and compact difference notation.

Remark on terminology

It is common to refer to movement of a fluid as convection, while advection is the transport of some material dissolved or suspended in the fluid. We shall mostly choose the word advection here, but both terms are in heavy use, and for mass transport of a substance the PDE has an advection term, while the similar term for the heat equation is a convection term.

Much more comprehensive discussion of dispersion analysis for advection problems can be found in the book by Duran [3]. This is a an excellent resource for further studies on the topic of advection PDEs, with emphasis on generalizations to real geophysical problems. The book

© 2017, Hans Petter Langtangen, Svein Linge. Released under CC Attribution 4.0 license

by Fletcher [4] also has a good overview of methods for advection and convection problems.

4.1 One-dimensional time-dependent advection equations

We consider the pure advection model

$$\frac{\partial u}{\partial t} + v\frac{\partial u}{\partial x} = 0, \qquad x \in (0, L),\ t \in (0, T], \tag{4.1}$$

$$u(x, 0) = I(x), \qquad x \in (0, L), \tag{4.2}$$

$$u(0, t) = U_0, \qquad t \in (0, T]. \tag{4.3}$$

In (4.1), v is a given parameter, typically reflecting the transport velocity of a quantity u with a flow. There is only one boundary condition (4.3) since the spatial derivative is only first order in the PDE (4.1). The information at $x = 0$ and the initial condition get transported in the positive x direction if $v > 0$ through the domain.

It is easiest to find the solution of (4.1) if we remove the boundary condition and consider a process on the infinite domain $(-\infty, \infty)$. The solution is simply

$$u(x, t) = I(x - vt)\,. \tag{4.4}$$

This is also the solution we expect locally in a finite domain before boundary conditions have reflected or modified the wave.

A particular feature of the solution (4.4) is that

$$u(x_i, t_{n+1}) = u(x_{i-1}, t_n), \tag{4.5}$$

if $x_i = i\Delta x$ and $t_n = n\Delta t$ are points in a uniform mesh. We see this relation from

$$\begin{aligned} u(i\Delta x, (n+1)\Delta t) &= I(i\Delta x - v(n+1)\Delta t) \\ &= I((i-1)\Delta x - vn\Delta t - v\Delta t + \Delta x) \\ &= I((i-1)\Delta x - vn\Delta t) \\ &= u((i-1)\Delta x, n\Delta t), \end{aligned}$$

provided $v = \Delta x/\Delta t$. So, whenever we see a scheme that collapses to

$$u_i^{n+1} = u_{i-1}^n, \tag{4.6}$$

for the PDE in question, we have in fact a scheme that reproduces the analytical solution, and many of the schemes to be presented possess this nice property!

Finally, we add that a discussion of appropriate boundary conditions for the advection PDE in multiple dimensions is a challenging topic beyond the scope of this text.

4.1.1 Simplest scheme: forward in time, centered in space

Method. A first attempt to solve a PDE like (4.1) will normally be to look for a time-discretization scheme that is explicit so we avoid solving systems of linear equations. In space, we anticipate that centered differences are most accurate and therefore best. These two arguments lead us to a Forward Euler scheme in time and centered differences in space:

$$[D_t^+ u + vD_{2x}u = 0]_i^n \tag{4.7}$$

Written out, we see that this expression implies that

$$u^{n+1} = u^n - \frac{1}{2}C(u_{i+1}^n - u_{i-1}^n),$$

with C as the Courant number

$$C = \frac{v\Delta t}{\Delta x}.$$

Implementation. A solver function for our scheme goes as follows.

```
import numpy as np
import matplotlib.pyplot as plt

def solver_FECS(I, U0, v, L, dt, C, T, user_action=None):
    Nt = int(round(T/float(dt)))
    t = np.linspace(0, Nt*dt, Nt+1)   # Mesh points in time
    dx = v*dt/C
    Nx = int(round(L/dx))
    x = np.linspace(0, L, Nx+1)       # Mesh points in space
    # Make sure dx and dt are compatible with x and t
    dx = x[1] - x[0]
    dt = t[1] - t[0]
    C = v*dt/dx
```

```
    u   = np.zeros(Nx+1)
    u_n = np.zeros(Nx+1)

    # Set initial condition u(x,0) = I(x)
    for i in range(0, Nx+1):
        u_n[i] = I(x[i])

    if user_action is not None:
        user_action(u_n, x, t, 0)

    for n in range(0, Nt):
        # Compute u at inner mesh points
        for i in range(1, Nx):
            u[i] = u_n[i] - 0.5*C*(u_n[i+1] - u_n[i-1])

        # Insert boundary condition
        u[0] = U0

        if user_action is not None:
            user_action(u, x, t, n+1)

        # Switch variables before next step
        u_n, u = u, u_n
```

Test cases. The typical solution u has the shape of I and is transported at velocity v to the right (if $v > 0$). Let us consider two different initial conditions, one smooth (Gaussian pulse) and one non-smooth (half-truncated cosine pulse):

$$u(x,0) = Ae^{-\frac{1}{2}\left(\frac{x-L/10}{\sigma}\right)^2}, \tag{4.8}$$

$$u(x,0) = A\cos\left(\frac{5\pi}{L}\left(x - \frac{L}{10}\right)\right), \quad x < \frac{L}{5} \text{ else } 0\,. \tag{4.9}$$

The parameter A is the maximum value of the initial condition.

Before doing numerical simulations, we scale the PDE problem and introduce $\bar{x} = x/L$ and $\bar{t} = vt/L$, which gives

$$\frac{\partial \bar{u}}{\partial \bar{t}} + \frac{\partial \bar{u}}{\partial \bar{x}} = 0\,.$$

The unknown u is scaled by the maximum value of the initial condition: $\bar{u} = u/\max|I(x)|$ such that $|\bar{u}(\bar{x},0)| \in [0,1]$. The scaled problem is solved by setting $v = 1$, $L = 1$, and $A = 1$. From now on we drop the bars.

To run our test cases and plot the solution, we make the function

```
def run_FECS(case):
    """Special function for the FECS case."""
    if case == 'gaussian':
        def I(x):
            return np.exp(-0.5*((x-L/10)/sigma)**2)
    elif case == 'cosinehat':
        def I(x):
            return np.cos(np.pi*5/L*(x - L/10)) if x < L/5 else 0

    L = 1.0
    sigma = 0.02
    legends = []

    def plot(u, x, t, n):
        """Animate and plot every m steps in the same figure."""
        plt.figure(1)
        if n == 0:
            lines = plot(x, u)
        else:
            lines[0].set_ydata(u)
            plt.draw()
            #plt.savefig()
        plt.figure(2)
        m = 40
        if n % m != 0:
            return
        print 't=%g, n=%d, u in [%g, %g] w/%d points' % \
              (t[n], n, u.min(), u.max(), x.size)
        if np.abs(u).max() > 3:  # Instability?
            return
        plt.plot(x, u)
        legends.append('t=%g' % t[n])
        if n > 0:
            plt.hold('on')

    plt.ion()
    U0 = 0
    dt = 0.001
    C = 1
    T = 1
    solver(I=I, U0=U0, v=1.0, L=L, dt=dt, C=C, T=T,
           user_action=plot)
    plt.legend(legends, loc='lower left')
    plt.savefig('tmp.png'); plt.savefig('tmp.pdf')
    plt.axis([0, L, -0.75, 1.1])
    plt.show()
```

Bug? Running either of the test cases, the plot becomes a mess, and the printout of u values in the `plot` function reveals that u grows very quickly. We may reduce Δt and make it very small, yet the solution just grows. Such behavior points to a bug in the code. However, choosing a coarse mesh and performing one time step by hand calculations produces

the same numbers as the code, so the implementation seems to be correct. The hypothesis is therefore that the solution is unstable.

4.1.2 Analysis of the scheme

It is easy to show that a typical Fourier component

$$u(x,t) = B\sin(k(x-ct))$$

is a solution of our PDE for any spatial wave length $\lambda = 2\pi/k$ and any amplitude B. (Since the PDE to be investigated by this method is homogeneous and linear, B will always cancel out, so we tend to skip this amplitude, but keep it here in the beginning for completeness.)

A general solution may be viewed as a collection of long and short waves with different amplitudes. Algebraically, the work simplifies if we introduce the complex Fourier component

$$u(x,t) = A_{\mathrm{e}}e^{ikx},$$

with

$$A_{\mathrm{e}} = Be^{-ikv\Delta t} = Be^{-iCk\Delta x}\,.$$

Note that $|A_{\mathrm{e}}| \leq 1$.

It turns out that many schemes also allow a Fourier wave component as solution, and we can use the numerically computed values of A_{e} (denoted A) to learn about the quality of the scheme. Hence, to analyze the difference scheme we have just implemented, we look at how it treats the Fourier component

$$u_q^n = A^n e^{ikq\Delta x}\,.$$

Inserting the numerical component in the scheme,

$$[D_t^+ Ae^{ikq\Delta x} + vD_{2x}Ae^{ikq\Delta x} = 0]_q^n,$$

and making use of (A.25) results in

$$[e^{ikq\Delta x}(\frac{A-1}{\Delta t} + v\frac{1}{\Delta x}i\sin(k\Delta x)) = 0]_q^n,$$

which implies

$$A = 1 - iC\sin(k\Delta x)\,.$$

The numerical solution features the formula A^n. To find out whether A^n means growth in time, we rewrite A in polar form: $A = A_r e^{i\phi}$, for real numbers A_r and ϕ, since we then have $A^n = A_r^n e^{i\phi n}$. The magnitude of A^n is A_r^n. In our case, $A_r = (1 + C^2\sin^2(kx))^{1/2} > 1$, so A_r^n will increase in time, whereas the exact solution will not. Regardless of Δt, we get unstable numerical solutions.

4.1.3 Leapfrog in time, centered differences in space

Method. Another explicit scheme is to do a "leapfrog" jump over $2\Delta t$ in time and combine it with central differences in space:

$$[D_{2t}u + vD_{2x}u = 0]_i^n,$$

which results in the updating formula

$$u_i^{n+1} = u_i^{n-1} - C(u_{i+1}^n - u_{i-1}^n)\,.$$

A special scheme is needed to compute u^1, but we leave that problem for now. Anyway, this special scheme can be found in `advec1D.py`.

Implementation. We now need to work with three time levels and must modify our solver a bit:

```
Nt = int(round(T/float(dt)))
t = np.linspace(0, Nt*dt, Nt+1)   # Mesh points in time
...
u   = np.zeros(Nx+1)
u_1 = np.zeros(Nx+1)
u_2 = np.zeros(Nx+1)
...
for n in range(0, Nt):
    if scheme == 'FE':
        for i in range(1, Nx):
            u[i] = u_1[i] - 0.5*C*(u_1[i+1] - u_1[i-1])
    elif scheme == 'LF':
        if n == 0:
            # Use some scheme for the first step
            for i in range(1, Nx):
                ...
        else:
            for i in range(1, Nx+1):
                u[i] = u_2[i] - C*(u_1[i] - u_1[i-1])

    # Switch variables before next step
    u_2, u_1, u = u_1, u, u_2
```

Running a test case. Let us try a coarse mesh such that the smooth Gaussian initial condition is represented by 1 at mesh node 1 and 0 at all other nodes. This triangular initial condition should then be advected to the right. Choosing scaled variables as $\Delta t = 0.1$, $T = 1$, and $C = 1$ gives the plot in Figure 4.1, which is in fact identical to the exact solution (!).

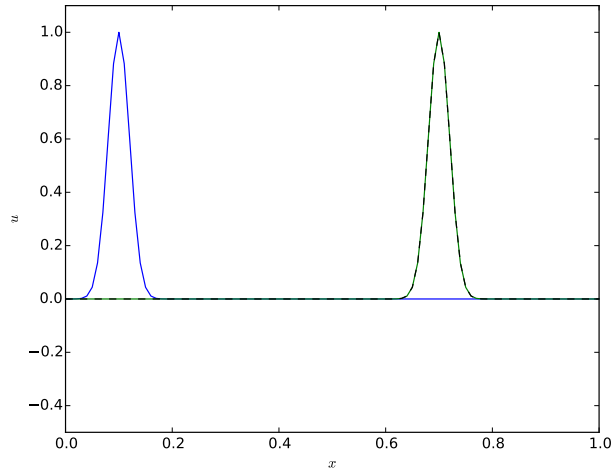


Fig. 4.1 Exact solution obtained by Leapfrog scheme with $\Delta t = 0.1$ and $C = 1$.

Running more test cases. We can run two types of initial conditions for $C = 0.8$: one very smooth with a Gaussian function (Figure 4.2) and one with a discontinuity in the first derivative (Figure 4.3). Unless we have a very fine mesh, as in the left plots in the figures, we get small ripples behind the main wave, and this main wave has the amplitude reduced.

Advection of the Gaussian function with a leapfrog scheme, using $C = 0.8$ and $\Delta t = 0.01$ can be seen in a movie file[1]. Alternatively, with $\Delta t = 0.001$, we get this movie file[2].

Advection of the cosine hat function with a leapfrog scheme, using $C = 0.8$ and $\Delta t = 0.01$ can be seen in a movie file[3]. Alternatively, with $\Delta t = 0.001$, we get this movie file[4].

[1] `http://tinyurl.com/gokgkov/mov-advec/gaussian/LF/C08_dt01.ogg`
[2] `http://tinyurl.com/gokgkov/mov-advec/gaussian/LF/C08_dt001.ogg`
[3] `http://tinyurl.com/gokgkov/mov-advec/cosinehat/LF/C08_dt01.ogg`
[4] `http://tinyurl.com/gokgkov/mov-advec/cosinehat/LF/C08_dt001.ogg`

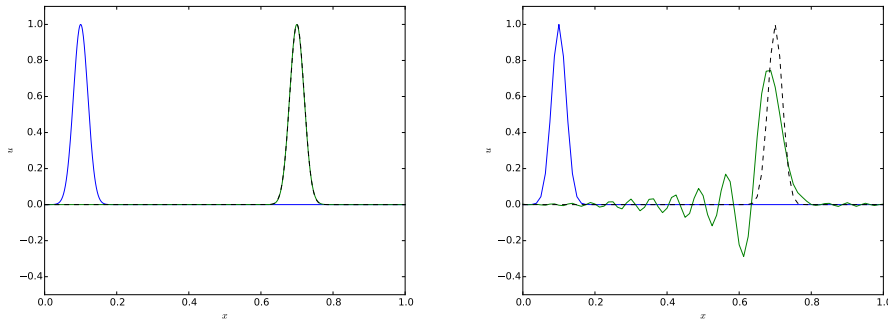


Fig. 4.2 Advection of a Gaussian function with a leapfrog scheme and $C = 0.8$, $\Delta t = 0.001$ (left) and $\Delta t = 0.01$ (right).

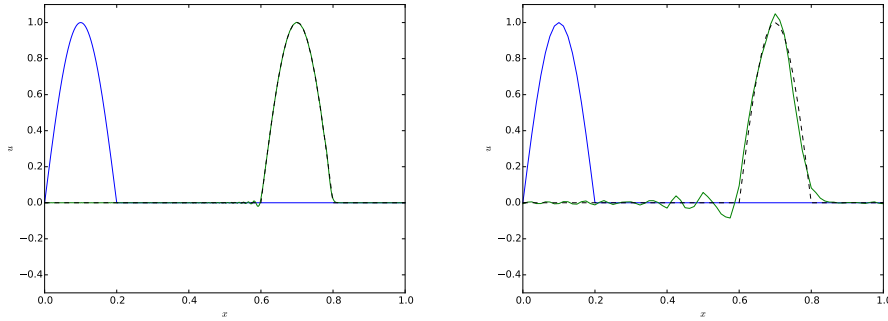


Fig. 4.3 Advection of half a cosine function with a leapfrog scheme and $C = 0.8$, $\Delta t = 0.001$ (left) and $\Delta t = 0.01$ (right).

Analysis. We can perform a Fourier analysis again. Inserting the numerical Fourier component in the Leapfrog scheme, we get

$$A^2 - i2C\sin(k\Delta x)A - 1 = 0,$$

and

$$A = -iC\sin(k\Delta x) \pm \sqrt{1 - C^2\sin^2(k\Delta x)}\,.$$

Rewriting to polar form, $A = A_r e^{i\phi}$, we see that $A_r = 1$, so the numerical component is neither increasing nor decreasing in time, which is exactly what we want. However, for $C > 1$, the square root can become complex valued, so stability is obtained only as long as $C \leq 1$.

Stability

For all the working schemes to be presented in this chapter, we get the stability condition $C \leq 1$:

$$\Delta t \leq \frac{\Delta x}{v}\,.$$

This is called the CFL condition and applies almost always to successful schemes for advection problems. Of course, one can use Crank-Nicolson or Backward Euler schemes for increased and even unconditional stability (no Δt restrictions), but these have other less desired damping problems.

We introduce $p = k\Delta x$. The amplification factor now reads

$$A = -iC\sin p \pm \sqrt{1 - C^2\sin^2 p},$$

and is to be compared to the exact amplification factor

$$A_{\mathrm{e}} = e^{-ikv\Delta t} = e^{-ikC\Delta x} = e^{-iCp}\,.$$

Section 4.1.9 compares numerical amplification factors of many schemes with the exact expression.

4.1.4 Upwind differences in space

Since the PDE reflects transport of information along with a flow in positive x direction, when $v > 0$, it could be natural to go (what is called) upstream and not downstream in the spatial derivative to collect information about the change of the function. That is, we approximate

$$\frac{\partial u}{\partial x}(x_i, t_n) \approx [D_x^- u]_i^n = \frac{u_i^n - u_{i-1}^n}{\Delta x}\,.$$

This is called an *upwind difference* (the corresponding difference in the time direction would be called a backward difference, and we could use that name in space too, but *upwind* is the common name for a difference against the flow in advection problems). This spatial approximation does magic compared to the scheme we had with Forward Euler in time and centered difference in space. With an upwind difference,

$$[D_t^+ u + vD_x^- u = 0]_i^n, \tag{4.10}$$

written out as

$$u_i^{n+1} = u_i^n - C(u_i^n - u_{i-1}^n),$$

gives a generally popular and robust scheme that is stable if $C \leq 1$. As with the Leapfrog scheme, it becomes exact if $C = 1$, exactly as shown in Figure 4.1. This is easy to see since $C = 1$ gives the property (4.6). However, any $C < 1$ gives a significant reduction in the amplitude of the solution, which is a purely numerical effect, see Figures 4.4 and 4.5. Experiments show, however, that reducing Δt or Δx, while keeping C reduces the error.

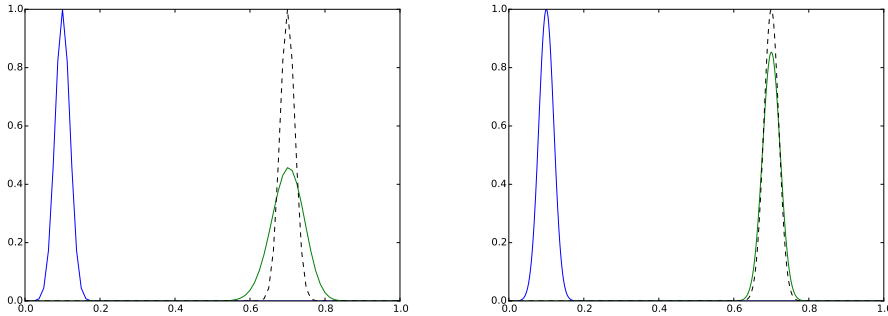


Fig. 4.4 Advection of a Gaussian function with a forward in time, upwind in space scheme and $C = 0.8$, $\Delta t = 0.01$ (left) and $\Delta t = 0.001$ (right).

Advection of the Gaussian function with a forward in time, upwind in space scheme, using $C = 0.8$ and $\Delta t = 0.01$ can be seen in a movie file[5]. Alternatively, with $\Delta t = 0.005$, we get this movie file[6].

Advection of the cosine hat function with a forward in time, upwind in space scheme, using $C = 0.8$ and $\Delta t = 0.01$ can be seen in a movie file[7]. Alternatively, with $\Delta t = 0.001$, we get this movie file[8].

The amplification factor can be computed using the formula (A.23),

$$\frac{A-1}{\Delta t} + \frac{v}{\Delta x}(1 - e^{-ik\Delta x}) = 0,$$

which means

$$A = 1 - C(1 - \cos(p) - i\sin(p)).$$

[5] `http://tinyurl.com/gokgkov/mov-advec/gaussian/UP/C08_dt001/movie.ogg`
[6] `http://tinyurl.com/gokgkov/mov-advec/gaussian/UP/C08_dt0005/movie.ogg`
[7] `http://tinyurl.com/gokgkov/mov-advec/cosinehat/UP/C08_dt01.ogg`
[8] `http://tinyurl.com/gokgkov/mov-advec/cosinehat/UP/C08_dt001.ogg`

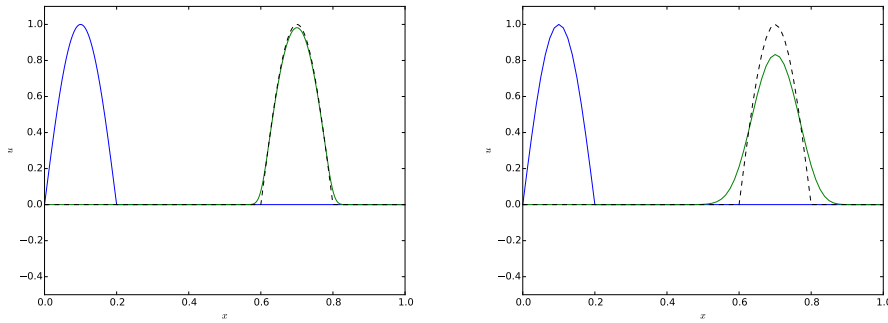


Fig. 4.5 Advection of half a cosine function with a forward in time, upwind in space scheme and $C = 0.8$, $\Delta t = 0.001$ (left) and $\Delta t = 0.01$ (right).

For $C < 1$ there is, unfortunately, non-physical damping of discrete Fourier components, giving rise to reduced amplitude of u_i^n as in Figures 4.4 and 4.5. The damping seen in these figures is quite severe. Stability requires $C \leq 1$.

Interpretation of upwind difference as artificial diffusion

One can interpret the upwind difference as extra, artificial diffusion in the equation. Solving

$$\frac{\partial u}{\partial t} + v\frac{\partial u}{\partial x} = \nu\frac{\partial^2 u}{\partial x^2},$$

by a forward difference in time and centered differences in space,

$$D_t^+ u + vD_{2x}u = \nu D_x D_x u]_i^n,$$

actually gives the upwind scheme (4.10) if $\nu = v\Delta x/2$. That is, solving the PDE $u_t + vu_x = 0$ by centered differences in space and forward difference in time is unsuccessful, but by adding some artificial diffusion νu_{xx}, the method becomes stable:

$$\frac{\partial u}{\partial t} + v\frac{\partial u}{\partial x} = \left(\alpha + \frac{v\Delta x}{2}\right)\frac{\partial^2 u}{\partial x^2}.$$

4.1.5 Periodic boundary conditions

So far, we have given the value on the left boundary, u_0^n, and used the scheme to propagate the solution signal through the domain. Often, we want to follow such signals for long time series, and periodic boundary conditions are then relevant since they enable a signal that leaves the right boundary to immediately enter the left boundary and propagate through the domain again.

The periodic boundary condition is

$$u(0,t) = u(L,t), \quad u_0^n = u_{N_x}^n .$$

It means that we in the first equation, involving u_0^n, insert $u_{N_x}^n$, and that we in the last equation, involving $u_{N_x}^{n+1}$ insert u_0^{n+1}. Normally, we can do this in the simple way that `u_1[0]` is updated as `u_1[Nx]` at the beginning of a new time level.

In some schemes we may need $u_{N_x+1}^n$ and u_{-1}^n. Periodicity then means that these values are equal to u_1^n and $u_{N_x-1}^n$, respectively. For the upwind scheme, it is sufficient to set `u_1[0]=u_1[Nx]` at a new time level before computing `u[1]`. This ensures that `u[1]` becomes right and at the next time level `u[0]` at the current time level is correctly updated. For the Leapfrog scheme we must update `u[0]` and `u[Nx]` using the scheme:

```
if periodic_bc:
    i = 0
    u[i] = u_2[i] - C*(u_1[i+1] - u_1[Nx-1])
for i in range(1, Nx):
    u[i] = u_2[i] - C*(u_1[i+1] - u_1[i-1])
if periodic_bc:
    u[Nx] = u[0]
```

4.1.6 Implementation

Test condition. Analytically, we can show that the integral in space under the $u(x,t)$ curve is constant:

$$\int_0^L \left(\frac{\partial u}{\partial t} + v\frac{\partial u}{\partial x}\right) dx = 0$$

$$\frac{\partial}{\partial t}\int_0^L u dx = -\int_0^L v\frac{\partial u}{\partial x} dx$$

$$\frac{\partial u}{\partial t}\int_0^L u dx = [vu]_0^L = 0$$

as long as $u(0) = u(L) = 0$. We can therefore use the property

$$\int_0^L u(x,t)dx = \text{const}$$

as a partial verification during the simulation. Now, any numerical method with $C \neq 1$ will deviate from the constant, expected value, so the integral is a measure of the error in the scheme. The integral can be computed by the Trapezoidal integration rule

```
dx*(0.5*u[0] + 0.5*u[Nx] + np.sum(u[1:-1]))
```

if `u` is an array holding the solution.

The code. An appropriate `solver` function for multiple schemes may go as shown below.

```
def solver(I, U0, v, L, dt, C, T, user_action=None,
           scheme='FE', periodic_bc=True):

    Nt = int(round(T/float(dt)))
    t = np.linspace(0, Nt*dt, Nt+1)   # Mesh points in time
    dx = v*dt/C
    Nx = int(round(L/dx))
    x = np.linspace(0, L, Nx+1)       # Mesh points in space
    # Make sure dx and dt are compatible with x and t
    dx = x[1] - x[0]
    dt = t[1] - t[0]
    C = v*dt/dx
    print 'dt=%g, dx=%g, Nx=%d, C=%g' % (dt, dx, Nx, C)

    u   = np.zeros(Nx+1)
    u_n = np.zeros(Nx+1)
    u_nm1 = np.zeros(Nx+1)
    integral = np.zeros(Nt+1)

    # Set initial condition u(x,0) = I(x)
    for i in range(0, Nx+1):
        u_n[i] = I(x[i])

    # Insert boundary condition
    u[0] = U0
```

```
    # Compute the integral under the curve
    integral[0] = dx*(0.5*u_n[0] + 0.5*u_n[Nx] + np.sum(u_n[1:-1]))

    if user_action is not None:
        user_action(u_n, x, t, 0)

    for n in range(0, Nt):
        if scheme == 'FE':
            if periodic_bc:
                i = 0
                u[i] = u_n[i] - 0.5*C*(u_n[i+1] - u_n[Nx])
                u[Nx] = u[0]
            for i in range(1, Nx):
                u[i] = u_n[i] - 0.5*C*(u_n[i+1] - u_n[i-1])
        elif scheme == 'LF':
            if n == 0:
                # Use upwind for first step
                if periodic_bc:
                    i = 0
                    u_n[i] = u_n[Nx]
                for i in range(1, Nx+1):
                    u[i] = u_n[i] - C*(u_n[i] - u_n[i-1])
            else:
                if periodic_bc:
                    i = 0
                    u[i] = u_nm1[i] - C*(u_n[i+1] - u_n[Nx-1])
                for i in range(1, Nx):
                    u[i] = u_nm1[i] - C*(u_n[i+1] - u_n[i-1])
                if periodic_bc:
                    u[Nx] = u[0]
        elif scheme == 'UP':
            if periodic_bc:
                u_n[0] = u_n[Nx]
            for i in range(1, Nx+1):
                u[i] = u_n[i] - C*(u_n[i] - u_n[i-1])
        else:
            raise ValueError('scheme="%s" not implemented' % scheme)

        if not periodic_bc:
            # Insert boundary condition
            u[0] = U0

        # Compute the integral under the curve
        integral[n+1] = dx*(0.5*u[0] + 0.5*u[Nx] + np.sum(u[1:-1]))

        if user_action is not None:
            user_action(u, x, t, n+1)

        # Switch variables before next step
        u_nm1, u_n, u = u_n, u, u_nm1
    return integral
```

Solving a specific problem. We need to call up the `solver` function in some kind of administering problem solving function that can solve specific problems and make appropriate visualization. The function below makes both static plots, screen animation, and hard copy videos in various formats.

```
def run(scheme='UP', case='gaussian', C=1, dt=0.01):
    """General admin routine for explicit and implicit solvers."""

    if case == 'gaussian':
        def I(x):
            return np.exp(-0.5*((x-L/10)/sigma)**2)
    elif case == 'cosinehat':
        def I(x):
            return np.cos(np.pi*5/L*(x - L/10)) if x < L/5 else 0

    L = 1.0
    sigma = 0.02
    global lines  # needs to be saved between calls to plot

    def plot(u, x, t, n):
        """Plot t=0 and t=0.6 in the same figure."""
        plt.figure(1)
        global lines
        if n == 0:
            lines = plt.plot(x, u)
            plt.axis([x[0], x[-1], -0.5, 1.5])
            plt.xlabel('x'); plt.ylabel('u')
            plt.axes().set_aspect(0.15)
            plt.savefig('tmp_%04d.png' % n)
            plt.savefig('tmp_%04d.pdf' % n)
        else:
            lines[0].set_ydata(u)
            plt.axis([x[0], x[-1], -0.5, 1.5])
            plt.title('C=%g, dt=%g, dx=%g' %
                      (C, t[1]-t[0], x[1]-x[0]))
            plt.legend(['t=%.3f' % t[n]])
            plt.xlabel('x'); plt.ylabel('u')
            plt.draw()
            plt.savefig('tmp_%04d.png' % n)
        plt.figure(2)
        eps = 1E-14
        if abs(t[n] - 0.6) > eps and abs(t[n] - 0) > eps:
            return
        print 't=%g, n=%d, u in [%g, %g] w/%d points' % \
              (t[n], n, u.min(), u.max(), x.size)
        if np.abs(u).max() > 3:  # Instability?
            return
        plt.plot(x, u)
        plt.hold('on')
        plt.draw()
        if n > 0:
```

```
            y = [I(x_-v*t[n]) for x_ in x]
            plt.plot(x, y, 'k--')
            if abs(t[n] - 0.6) < eps:
                filename = ('tmp_%s_dt%s_C%s' % \
                           (scheme, t[1]-t[0], C)).replace('.', '')
                np.savez(filename, x=x, u=u, u_e=y)

    plt.ion()
    U0 = 0
    T = 0.7
    v = 1
    # Define video formats and libraries
    codecs = dict(flv='flv', mp4='libx264', webm='libvpx',
                  ogg='libtheora')
    # Remove video files
    import glob, os
    for name in glob.glob('tmp_*.png'):
        os.remove(name)
    for ext in codecs:
        name = 'movie.%s' % ext
        if os.path.isfile(name):
            os.remove(name)

   integral = solver(
       I=I, U0=U0, v=v, L=L, dt=dt, C=C, T=T,
       scheme=scheme, user_action=plot)
    # Finish up figure(2)
    plt.figure(2)
    plt.axis([0, L, -0.5, 1.1])
    plt.xlabel('$x$');  plt.ylabel('$u$')
    plt.savefig('tmp1.png'); plt.savefig('tmp1.pdf')
    plt.show()
    # Make videos from figure(1) animation files
    for codec in codecs:
        cmd = 'ffmpeg -i tmp_%%04d.png -r 25 -vcodec %s movie.%s' % \
              (codecs[codec], codec)
        os.system(cmd)
    print 'Integral of u:', integral.max(), integral.min()
```

The complete code is found in the file `advec1D.py`.

4.1.7 A Crank-Nicolson discretization in time and centered differences in space

Another obvious candidate for time discretization is the Crank-Nicolson method combined with centered differences in space:

$$[D_t u]_i^n + v\frac{1}{2}([D_{2x}u]_i^{n+1} + [D_{2x}u]_i^n) = 0\,.$$

It can be nice to include the Backward Euler scheme too, via the θ-rule,

$$[D_t u]_i^n + v\theta[D_{2x}u]_i^{n+1} + v(1-\theta)[D_{2x}u]_i^n = 0\,.$$

When θ is different from zero, this gives rise to an *implicit* scheme,

$$u_i^{n+1} + \frac{\theta}{2}C(u_{i+1}^{n+1} - u_{i-1}^{n+1}) = u_i^n - \frac{1-\theta}{2}C(u_{i+1}^n - u_{i-1}^n)$$

for $i = 1, \ldots, N_x - 1$. At the boundaries we set $u = 0$ and simulate just to the point of time when the signal hits the boundary (and gets reflected).

$$u_0^{n+1} = u_{N_x}^{n+1} = 0\,.$$

The elements on the diagonal in the matrix become:

$$A_{i,i} = 1, \quad i = 0, \ldots, N_x\,.$$

On the subdiagonal and superdiagonal we have

$$A_{i-1,i} = -\frac{\theta}{2}C, \quad A_{i+1,i} = \frac{\theta}{2}C, \quad i = 1, \ldots, N_x - 1,$$

with $A_{0,1} = 0$ and $A_{N_x-1,N_x} = 0$ due to the known boundary conditions. And finally, the right-hand side becomes

$$\begin{aligned} b_0 &= u_{N_x}^n \\ b_i &= u_i^n - \frac{1-\theta}{2}C(u_{i+1}^n - u_{i-1}^n), \quad i = 1, \ldots, N_x - 1 \\ b_{N_x} &= u_0^n \end{aligned}$$

The dispersion relation follows from inserting $u_q^n = A^n e^{ikx}$ and using the formula (A.25) for the spatial differences:

$$A = \frac{1 - (1-\theta)iC\sin p}{1 + \theta iC\sin p}\,.$$

Movie 4: Crank-Nicolson in time, centered in space, $C = 0.8$, $\Delta t = 0.005$. `https://raw.githubusercontent.com/hplgit/fdm-book/master/doc/pub/book/html/mov-advec/gaussian/CN/C08_dt0005/movie.ogg`

Movie 5: Backward-Euler in time, centered in space, $C = 0.8$, $\Delta t = 0.005$. `https://raw.githubusercontent.com/hplgit/fdm-book/master/doc/pub/book/html/mov-advec/cosinehat/BE/C_08_dt005.ogg`

Figure 4.6 depicts a numerical solution for $C = 0.8$ and the Crank-Nicolson with severe oscillations behind the main wave. These oscillations

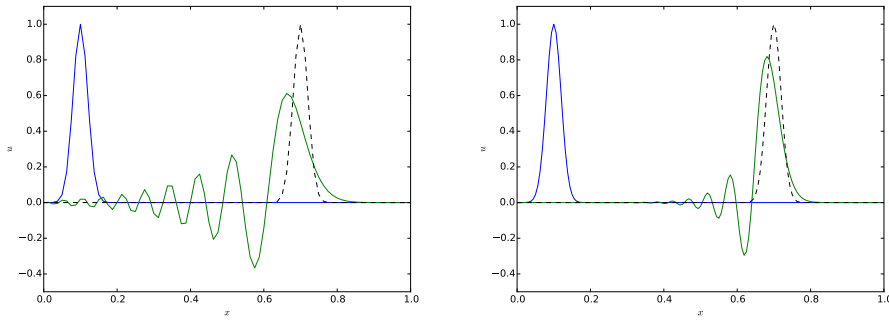


Fig. 4.6 Crank-Nicolson in time, centered in space, Gaussian profile, $C = 0.8$, $\Delta t = 0.01$ (left) and $\Delta t = 0.005$ (right).

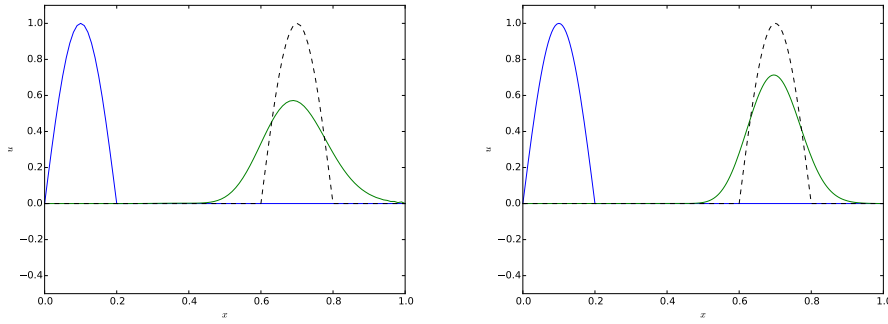


Fig. 4.7 Backward-Euler in time, centered in space, half a cosine profile, $C = 0.8$, $\Delta t = 0.01$ (left) and $\Delta t = 0.005$ (right).

are damped as the mesh is refined. Switching to the Backward Euler scheme removes the oscillations, but the amplitude is significantly reduced. One could expect that the discontinuous derivative in the initial condition of the half a cosine wave would make even stronger demands on producing a smooth profile, but Figure 4.7 shows that also here, Backward-Euler is capable of producing a smooth profile. All in all, there are no major differences between the Gaussian initial condition and the half a cosine condition for any of the schemes.

4.1.8 The Lax-Wendroff method

The Lax-Wendroff method is based on three ideas:

1. Express the new unknown u_i^{n+1} in terms of known quantities at $t = t_n$ by means of a Taylor polynomial of second degree.

2. Replace time-derivatives at $t = t_n$ by spatial derivatives, using the PDE.
3. Discretize the spatial derivatives by second-order differences so we achieve a scheme of accuracy $\mathcal{O}(\Delta t^2) + \mathcal{O}(\Delta x^2)$.

Let us follow the recipe. First we have the three-term Taylor polynomial,

$$u_i^{n+1} = u_i^n + \Delta t \left(\frac{\partial u}{\partial t}\right)_i^n + \frac{1}{2}\Delta t^2 \left(\frac{\partial^2 u}{\partial t^2}\right)_i^n .$$

From the PDE we have that temporal derivatives can be substituted by spatial derivatives:

$$\frac{\partial u}{\partial t} = -v\frac{\partial u}{\partial x},$$

and furthermore,

$$\frac{\partial^2 u}{\partial t^2} = v^2\frac{\partial^2 u}{\partial x^2}.$$

Inserted in the Taylor polynomial formula, we get

$$u_i^{n+1} = u_i^n - v\Delta t \left(\frac{\partial u}{\partial x}\right)_i^n + \frac{1}{2}\Delta t^2 v^2 \left(\frac{\partial^2 u}{\partial x^2}\right)_i^n .$$

To obtain second-order accuracy in space we now use central differences:

$$u_i^{n+1} = u_i^n - v\Delta t[D_{2x}u]_i^n + \frac{1}{2}\Delta t^2 v^2 [D_x D_x u]_i^n,$$

or written out,

$$u_i^{n+1} = u_i^n - \frac{1}{2}C(u_{i+1}^n - u_{i-1}^n) + \frac{1}{2}C^2(u_{i+1}^n - 2u_i^n + u_{i-1}^n).$$

This is the explicit Lax-Wendroff scheme.

Lax-Wendroff works because of artificial viscosity

From the formulas above, we notice that the Lax-Wendroff method is nothing but a Forward Euler, central difference in space scheme, which we have shown to be useless because of chronic instability, plus an artificial diffusion term of strength $\frac{1}{2}\Delta t v^2$. It means that we

can take an unstable scheme and add some diffusion to stabilize it. This is a common trick to deal with advection problems. Sometimes, the real physical diffusion is not sufficiently large to make schemes stable, so then we also add artificial diffusion.

From an analysis similar to the ones carried out above, we get an amplification factor for the Lax-Wendroff method that equals

$$A = 1 - iC\sin p - 2C^2\sin^2(p/2)\,.$$

This means that $|A| = 1$ and also that we have an exact solution if $C = 1$!

4.1.9 Analysis of dispersion relations

We have developed expressions for $A(C, p)$ in the exact solution $u_q^n = A^n e^{ikq\Delta x}$ of the discrete equations. Note that the Fourier component that solves the original PDE problem has no damping and moves with constant velocity v. There are two basic errors in the numerical Fourier component: there may be damping and the wave velocity may depend on C and $p = k\Delta x$.

The shortest wavelength that can be represented is $\lambda = 2\Delta x$. The corresponding k is $k = 2\pi/\lambda = \pi/\Delta x$, so $p = k\Delta x \in (0, \pi]$.

Given a complex A as a function of C and p, how can we visualize it? The two key ingredients in A is the magnitude, reflecting damping or growth of the wave, and the angle, closely related to the velocity of the wave. The Fourier component

$$D^n e^{ik(x-ct)}$$

has damping D and wave velocity c. Let us express our A in polar form, $A = A_r e^{-i\phi}$, and insert this expression in our discrete component $u_q^n = A^n e^{ikq\Delta x} = A^n e^{ikx}$:

$$u_q^n = A_r^n e^{-i\phi n} e^{ikx} = A_r^n e^{i(kx - n\phi)} = A_r^n e^{i(k(x-ct))},$$

for

$$c = \frac{\phi}{k\Delta t}\,.$$

Now,

$$k\Delta t = \frac{Ck\Delta x}{v} = \frac{Cp}{v},$$

so

$$c = \frac{\phi v}{Cp}\,.$$

An appropriate dimensionless quantity to plot is the scaled wave velocity c/v:

$$\frac{c}{v} = \frac{\phi}{Cp}\,.$$

Figures 4.8–4.13 contain dispersion curves, velocity and damping, for various values of C. The horizontal axis shows the dimensionless frequency p of the wave, while the figures to the left illustrate the error in wave velocity c/v (should ideally be 1 for all p), and the figures to the right display the absolute value (magnitude) of the damping factor A_r. The curves are labeled according to the table below.

Label	Method
FE	Forward Euler in time, centered difference in space
LF	Leapfrog in time, centered difference in space
UP	Forward Euler in time, upwind difference in space
CN	Crank-Nicolson in time, centered difference in space
LW	Lax-Wendroff's method
BE	Backward Euler in time, centered difference in space

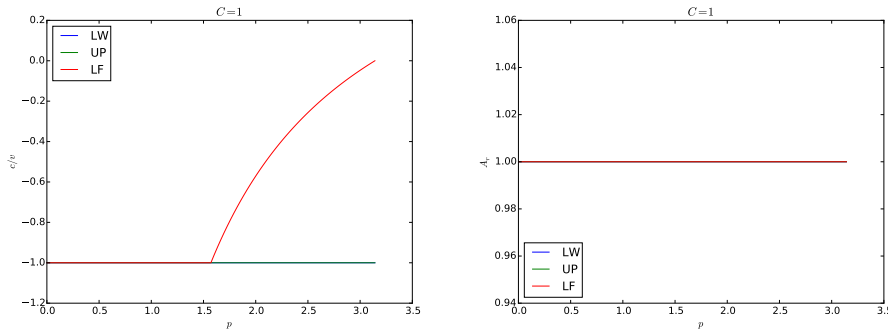


Fig. 4.8 Dispersion relations for $C = 1$.

The total damping after some time $T = n\Delta t$ is reflected by $A_r(C,p)^n$. Since normally $A_r < 1$, the damping goes like $A_r^{1/\Delta t}$ and approaches zero

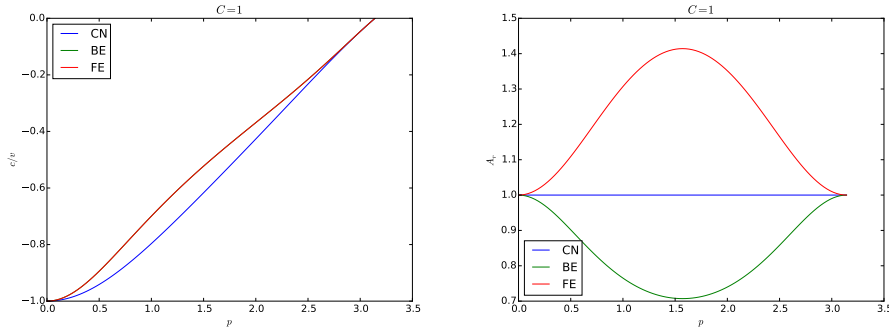


Fig. 4.9 Dispersion relations for $C = 1$.

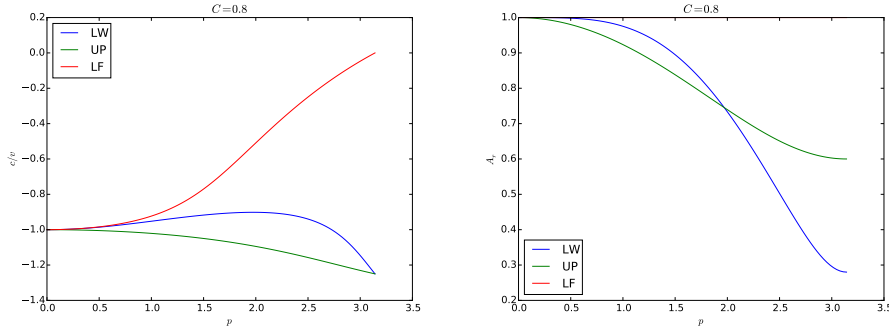


Fig. 4.10 Dispersion relations for $C = 0.8$.

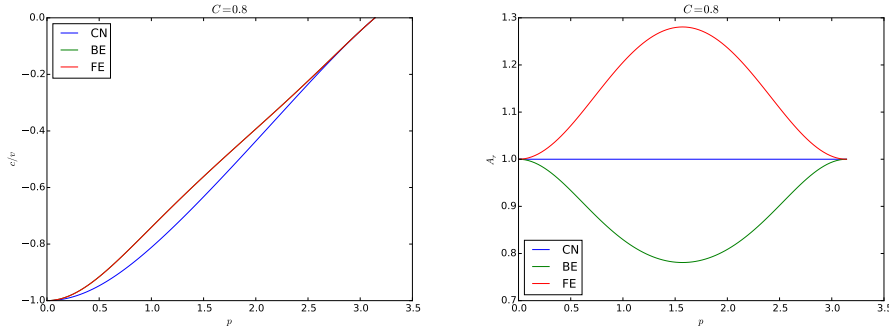


Fig. 4.11 Dispersion relations for $C = 0.8$.

as $\Delta t \to 0$. The only way to reduce damping is to increase C and/or the mesh resolution.

We can learn a lot from the dispersion relation plots. For example, looking at the plots for $C = 1$, the schemes LW, UP, and LF has no amplitude reduction, but LF has wrong phase velocity for the shortest wave in the mesh. This wave does not (normally) have enough amplitude to be seen, so for all practical purposes, there is no damping or wrong

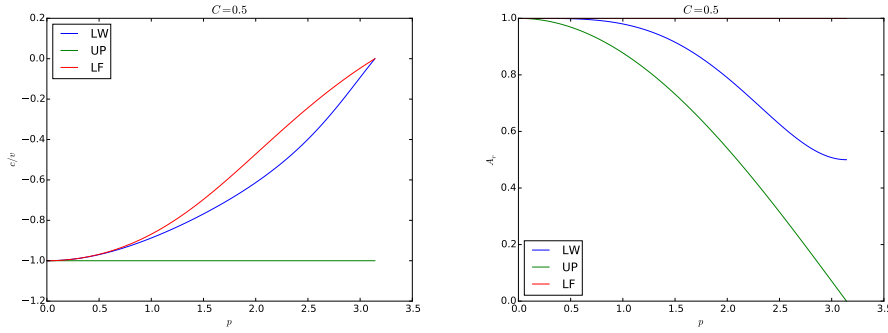


Fig. 4.12 Dispersion relations for $C = 0.5$.

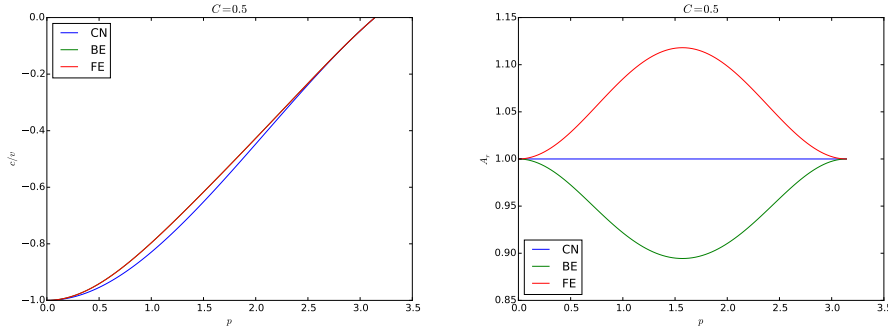


Fig. 4.13 Dispersion relations for $C = 0.5$.

velocity of the individual waves, so the total shape of the wave is also correct. For the CN scheme, see Figure 4.6, each individual wave has its amplitude, but they move with different velocities, so after a while, we see some of these waves lagging behind. For the BE scheme, see Figure 4.7, all the shorter waves are so heavily dampened that we cannot see them after a while. We see only the longest waves, which have slightly wrong velocity, but visible amplitudes are sufficiently equal to produce what looks like a smooth profile.

Another feature was that the Leapfrog method produced oscillations, while the upwind scheme did not. Since the Leapfrog method does not dampen the shorter waves, which have wrong wave velocities of order 10 percent, we can see these waves as noise. The upwind scheme, however, dampens these waves. The same effect is also present in the Lax-Wendroff scheme, but the damping of the intermediate waves is hardly present, so there is visible noise in the total signal.

We realize that, compared to pure truncation error analysis, dispersion analysis sheds more light on the behavior of the computational schemes.

Truncation analysis just says that Lax-Wendroff is better than upwind, because of the increased order in time, but most people would say upwind is the better one when looking at the plots.

4.2 One-dimensional stationary advection-diffusion equation

Now we pay attention to a physical process where advection (or convection) is in balance with diffusion:

$$v\frac{du}{dx} = \alpha\frac{d^2u}{dx^2}\,. \tag{4.11}$$

For simplicity, we assume v and α to be constant, but the extension to the variable-coefficient case is trivial. This equation can be viewed as the stationary limit of the corresponding time-dependent problem

$$\frac{\partial u}{\partial t} + v\frac{\partial u}{\partial x} = \alpha\frac{\partial^2 u}{\partial x^2}\,. \tag{4.12}$$

Equations of the form (4.11) or (4.12) arise from transport phenomena, either mass or heat transport. One can also view the equations as a simple model problem for the Navier-Stokes equations. With the chosen boundary conditions, the differential equation problem models the phenomenon of a *boundary layer*, where the solution changes rapidly very close to the boundary. This is a characteristic of many fluid flow problems, which makes strong demands to numerical methods. The fundamental numerical difficulty is related to non-physical oscillations of the solution (instability) if the first-derivative spatial term dominates over the second-derivative term.

4.2.1 A simple model problem

We consider (4.11) on $[0, L]$ equipped with the boundary conditions $u(0) = U_0$, $u(L) = U_L$. By scaling we can reduce the number of parameters in the problem. We scale x by $\bar{x} = x/L$, and u by

$$\bar{u} = \frac{u - U_0}{U_L - U_0}\,.$$

Inserted in the governing equation we get

$$\frac{v(U_L - U_0)}{L}\frac{d\bar{u}}{d\bar{x}} = \frac{\alpha(U_L - U_0)}{L^2}\frac{d^2\bar{u}}{d\bar{x}^2}, \quad \bar{u}(0) = 0,\ \bar{u}(1) = 1\,.$$

Dropping the bars is common. We can then simplify to

$$\frac{du}{dx} = \epsilon\frac{d^2u}{dx^2}, \quad u(0) = 0,\ u(1) = 1\,. \tag{4.13}$$

There are two competing effects in this equation: the advection term transports signals to the right, while the diffusion term transports signals to the left and the right. The value $u(0) = 0$ is transported through the domain if ϵ is small, and $u \approx 0$ except in the vicinity of $x = 1$, where $u(1) = 1$ and the diffusion transports some information about $u(1) = 1$ to the left. For large ϵ, diffusion dominates and the u takes on the "average" value, i.e., u gets a linear variation from 0 to 1 throughout the domain.

It turns out that we can find an exact solution to the differential equation problem and also to many of its discretizations. This is one reason why this model problem has been so successful in designing and investigating numerical methods for mixed convection/advection and diffusion. The exact solution reads

$$u_{\mathrm{e}}(x) = \frac{e^{x/\epsilon} - 1}{e^{1/\epsilon} - 1}\,.$$

The forthcoming plots illustrate this function for various values of ϵ.

4.2.2 A centered finite difference scheme

The most obvious idea to solve (4.13) is to apply centered differences:

$$[D_{2x}u = \epsilon D_x D_x u]_i$$

for $i = 1, \ldots, N_x - 1$, with $u_0 = 0$ and $u_{N_x} = 1$. Note that this is a coupled system of algebraic equations involving $u_0, \ldots, u_{N_x}$.

Written out, the scheme becomes a tridiagonal system

$$A_{i-1,i}u_{i-1} + A_{i,i}u_i + A_{i+1.i}u_{i+1} = 0,$$

for $i = 1, \ldots, N_x - 1$

$$
\begin{aligned}
A_{0,0} &= 1, \\
A_{i-1,i} &= -\frac{1}{\Delta x} - \epsilon \frac{1}{\Delta x^2}, \\
A_{i,i} &= 2\epsilon \frac{1}{\Delta x^2}, \\
A_{i,i+1} &= \frac{1}{\Delta x} - \epsilon \frac{1}{\Delta x^2}, \\
A_{N_x,N_x} &= 1\,.
\end{aligned}
$$

The right-hand side of the linear system is zero except $b_{N_x} = 1$.

Figure 4.14 shows reasonably accurate results with $N_x = 20$ and $N_x = 40$ cells in x direction and a value of $\epsilon = 0.1$. Decreasing ϵ to 0.01 leads to oscillatory solutions as depicted in Figure 4.15. This is, unfortunately, a typical phenomenon in this type of problem: non-physical oscillations arise for small ϵ unless the resolution N_x is big enough. Exercise 4.1 develops a precise criterion: u is oscillation-free if

$$
\Delta x \leq \frac{2}{\epsilon}\,.
$$

If we take the present model as a simplified model for a *viscous boundary layer* in real, industrial fluid flow applications, $\epsilon \sim 10^{-6}$ and millions of cells are required to resolve the boundary layer. Fortunately, this is not strictly necessary as we have methods in the next section to overcome the problem!

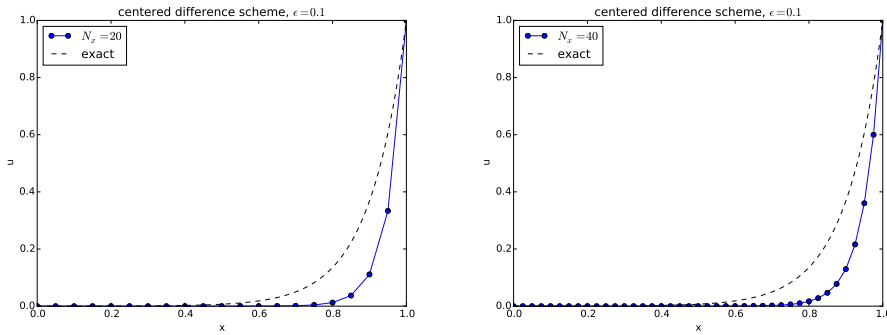


Fig. 4.14 Comparison of exact and numerical solution for $\epsilon = 0.1$ and $N_x = 20, 40$ with centered differences.

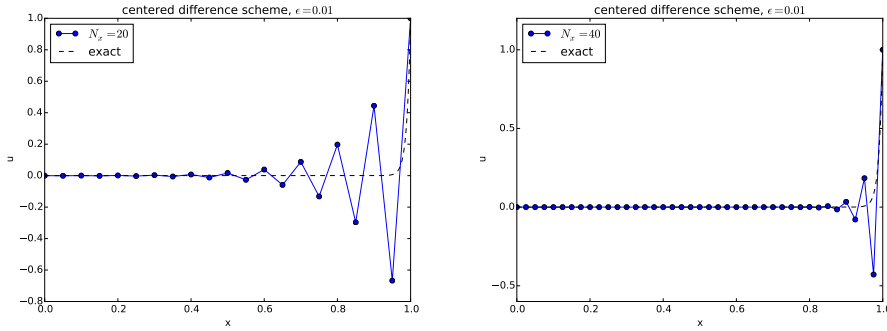


Fig. 4.15 Comparison of exact and numerical solution for $\epsilon = 0.01$ and $N_x = 20, 40$ with centered differences.

Solver

A suitable solver for doing the experiments is presented below.

```
import numpy as np

def solver(eps, Nx, method='centered'):
    """
    Solver for the two point boundary value problem u'=eps*u'',
    u(0)=0, u(1)=1.
    """
    x = np.linspace(0, 1, Nx+1)       # Mesh points in space
    # Make sure dx and dt are compatible with x and t
    dx = x[1] - x[0]
    u   = np.zeros(Nx+1)

    # Representation of sparse matrix and right-hand side
    diagonal = np.zeros(Nx+1)
    lower    = np.zeros(Nx)
    upper    = np.zeros(Nx)
    b        = np.zeros(Nx+1)

    # Precompute sparse matrix (scipy format)
    if method == 'centered':
        diagonal[:] = 2*eps/dx**2
        lower[:] = -1/dx - eps/dx**2
        upper[:] =  1/dx - eps/dx**2
    elif method == 'upwind':
        diagonal[:] = 1/dx + 2*eps/dx**2
        lower[:] =  1/dx - eps/dx**2
        upper[:] = - eps/dx**2

    # Insert boundary conditions
    upper[0] = 0
    lower[-1] = 0
```

```
    diagonal[0] = diagonal[-1] = 1
    b[-1] = 1.0

    # Set up sparse matrix and solve
    diags = [0, -1, 1]
    import scipy.sparse
    import scipy.sparse.linalg
    A = scipy.sparse.diags(
        diagonals=[diagonal, lower, upper],
        offsets=[0, -1, 1], shape=(Nx+1, Nx+1),
        format='csr')
    u[:] = scipy.sparse.linalg.spsolve(A, b)
    return u, x
```

4.2.3 Remedy: upwind finite difference scheme

The scheme can be stabilized by letting the advective transport term, which is the dominating term, collect its information in the flow direction, i.e., upstream or upwind of the point in question. So, instead of using a centered difference

$$\frac{du}{dx}_i \approx \frac{u_{i+1} - u_{i-1}}{2\Delta x},$$

we use the one-sided *upwind* difference

$$\frac{du}{dx}_i \approx \frac{u_i - u_{i-1}}{\Delta x},$$

in case $v > 0$. For $v < 0$ we set

$$\frac{du}{dx}_i \approx \frac{u_{i+1} - u_i}{\Delta x},$$

On compact operator notation form, our upwind scheme can be expressed as

$$[D_x^- u = \epsilon D_x D_x u]_i$$

provided $v > 0$ (and $\epsilon > 0$).

We write out the equations and implement them as shown in the program in Section 4.2.2. The results appear in Figures 4.16 and 4.17: no more oscillations!

We see that the upwind scheme is always stable, but it gives a thicker boundary layer when the centered scheme is also stable. Why the upwind

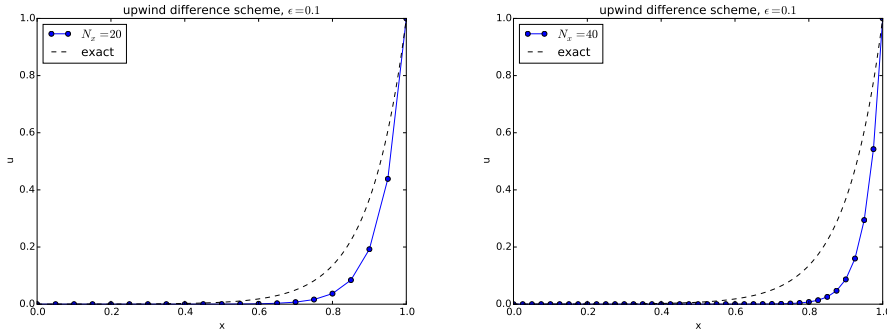


Fig. 4.16 Comparison of exact and numerical solution for $\epsilon = 0.1$ and $N_x = 20, 40$ with upwind difference.

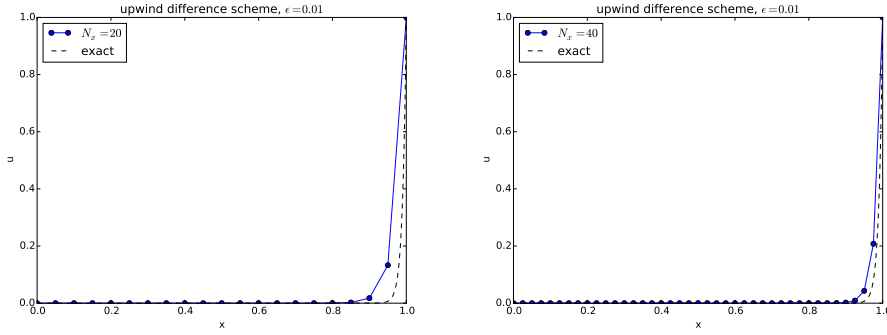


Fig. 4.17 Comparison of exact and numerical solution for $\epsilon = 0.01$ and $N_x = 20, 40$ with upwind difference.

scheme is always stable is easy to understand as soon as we undertake the mathematical analysis in Exercise 4.1. Moreover, the thicker layer (seemingly larger diffusion) can be understood by doing Exercise 4.2.

Exact solution for this model problem

It turns out that one can introduce a linear combination of the centered and upwind differences for the first-derivative term in this model problem. One can then adjust the weight in the linear combination so that the numerical solution becomes identical to the analytical solution of the differential equation problem at any mesh point.

4.3 Time-dependent convection-diffusion equations

Now it is time to combine time-dependency, convection (advection) and diffusion into one equation:

$$\frac{\partial u}{\partial t} + v\frac{\partial u}{\partial x} = \alpha\frac{\partial^2 u}{\partial x^2}\,. \tag{4.14}$$

Analytical insight. The diffusion is now dominated by convection, a wave, and diffusion, a loss of amplitude. One possible analytical solution is a traveling Gaussian function

$$u(x,t) = B\exp\left(-\left(\frac{x-vt}{4at}\right)\right).$$

This function moves with velocity $v > 0$ to the right ($v < 0$ to the left) due to convection, but at the same time we have a damping $e^{-16a^2t^2}$ from diffusion.

4.3.1 Forward in time, centered in space scheme

The Forward Euler for the diffusion equation is a successful scheme, but it has a very strict stability condition. The similar Forward in time, centered in space strategy always gives unstable solutions for the advection PDE. What happens when we have both diffusion and advection present at once?

$$[D_t u + vD_{2x}u = \alpha D_x D_x u + f]_i^n\,.$$

We expect that diffusion will stabilize the scheme, but that advection will destabilize it.

Another problem is non-physical oscillations, but not growing amplitudes, due to centered differences in the advection term. There will hence be two types of instabilities to consider. Our analysis showed that pure advection with centered differences in space needs some artificial diffusion to become stable (and then it produces upwind differences for the advection term). Adding more physical diffusion should further help the numerics to stabilize the non-physical oscillations.

The scheme is quickly implemented, but suffers from the need for small space and time steps, according to this reasoning. A better approach is to get rid of the non-physical oscillations in space by simply applying an upwind difference on the advection term.

4.3.2 Forward in time, upwind in space scheme

A good approximation for the pure advection equation is to use upwind discretization of the advection term. We also know that centered differences are good for the diffusion term, so let us combine these two discretizations:

$$[D_t u + vD_x^- u = \alpha D_x D_x u + f]_i^n, \tag{4.15}$$

for $v > 0$. Use vD^+u if $v < 0$. In this case the physical diffusion and the extra numerical diffusion $v\Delta x/2$ will stabilize the solution, but give an overall too large reduction in amplitude compared with the exact solution.

We may also interpret the upwind difference as artificial numerical diffusion and centered differences in space everywhere, so the scheme can be expressed as

$$[D_t u + vD_{2x}^- u = \alpha \frac{v\Delta x}{2} D_x D_x u + f]_i^n \,. \tag{4.16}$$

4.4 Applications of advection equations

There are two major areas where advection and convection applications arise: transport of a substance and heat transport *in a fluid*. To derive the models, we may look at the similar derivations of diffusion models in Section 3.8, but change the assumption from a solid to fluid medium. This gives rise to the extra advection or convection term $\boldsymbol{v} \cdot \nabla u$. We briefly show how this is done.

Normally, transport in a fluid is dominated by the fluid flow and not diffusion, so we can neglect diffusion compared to advection or convection. The end result is anyway an equation of the form

$$\frac{\partial u}{\partial t} + \boldsymbol{v} \cdot \nabla u = 0 \,.$$

4.4.1 Transport of a substance

The diffusion of a substance in Section 3.8.1 takes place in a solid medium, but in a fluid we can have two transport mechanisms: one by diffusion and one by advection. The latter arises from the fact that the substance

particles are moved with the fluid velocity $\boldsymbol{v}$ such that the effective flux now consists of two and not only one component as in (3.121):

$$\boldsymbol{q} = -\alpha\nabla c + \boldsymbol{v}_s$$

Inserted in the equation $\nabla \cdot \boldsymbol{q} = 0$ we get the extra advection term $\nabla \cdot (\boldsymbol{v}_s)$. Very often we deal with incompressible flows, $\nabla \cdot \boldsymbol{v} = 0$ such that the advective term becomes $\boldsymbol{v} \cdot \nabla c$. The mass transport equation for a substance then reads

$$\frac{\partial c}{\partial t} + \boldsymbol{v} \cdot \nabla c = \alpha\nabla^2 c\,. \tag{4.17}$$

4.4.2 Transport of heat in fluids

The derivation of the heat equation in Section 3.8.2 is limited to heat transport in solid bodies. If we turn the attention to heat transport in fluids, we get a material derivative of the internal energy in (3.123),

$$\frac{De}{dt} = -\nabla \cdot \boldsymbol{q},$$

and more terms if work by stresses is also included, where

$$\frac{De}{dt} = \frac{\partial e}{\partial t} + \boldsymbol{v} \cdot \nabla e,$$

$\boldsymbol{v}$ being the velocity of the fluid. The convective term $\boldsymbol{v}\cdot\nabla e$ must therefore be added to the governing equation, resulting typically in

$$\varrho c\left(\frac{\partial T}{\partial t} + \boldsymbol{v} \cdot \nabla T\right) = \nabla \cdot (k\nabla T) + f, \tag{4.18}$$

where f is some external heating inside the medium.

4.5 Exercises

Exercise 4.1: Analyze 1D stationary convection-diffusion problem

Explain the observations in the numerical experiments from Sections 4.2.2 and 4.2.3 by finding exact numerical solutions.

Hint. The difference equations allow solutions on the form A^i, where A is an unknown constant and i is a mesh point counter. There are two solutions for A, so the general solution is a linear combination of the two, where the constants in the linear combination are determined from the boundary conditions.
Filename: `twopt_BVP_analysis1`.

Exercise 4.2: Interpret upwind difference as artificial diffusion

Consider an upwind, one-sided difference approximation to a term du/dx in a differential equation. Show that this formula can be expressed as a centered difference plus an artificial diffusion term of strength proportional to Δx. This means that introducing an upwind difference also means introducing extra diffusion of order $\mathcal{O}(\Delta x)$. Filename: `twopt_BVP_analysis2`.

Nonlinear problems

5

5.1 Introduction of basic concepts

5.1.1 Linear versus nonlinear equations

Algebraic equations. A linear, scalar, algebraic equation in x has the form

$$ax + b = 0,$$

for arbitrary real constants a and b. The unknown is a number x. All other algebraic equations, e.g., $x^2 + ax + b = 0$, are nonlinear. The typical feature in a nonlinear algebraic equation is that the unknown appears in products with itself, like x^2 or $e^x = 1 + x + \frac{1}{2}x^2 + \frac{1}{3!}x^3 + \cdots$.

We know how to solve a linear algebraic equation, $x = -b/a$, but there are no general methods for finding the exact solutions of nonlinear algebraic equations, except for very special cases (quadratic equations constitute a primary example). A nonlinear algebraic equation may have no solution, one solution, or many solutions. The tools for solving nonlinear algebraic equations are *iterative methods*, where we construct a series of linear equations, which we know how to solve, and hope that the solutions of the linear equations converge to a solution of the nonlinear equation we want to solve. Typical methods for nonlinear algebraic equation equations are Newton's method, the Bisection method, and the Secant method.

© 2017, Hans Petter Langtangen, Svein Linge. Released under CC Attribution 4.0 license

Differential equations. The unknown in a differential equation is a function and not a number. In a linear differential equation, all terms involving the unknown function are linear in the unknown function or its derivatives. Linear here means that the unknown function, or a derivative of it, is multiplied by a number or a known function. All other differential equations are non-linear.

The easiest way to see if an equation is nonlinear, is to spot nonlinear terms where the unknown function or its derivatives are multiplied by each other. For example, in

$$u'(t) = -a(t)u(t) + b(t),$$

the terms involving the unknown function u are linear: u' contains the derivative of the unknown function multiplied by unity, and au contains the unknown function multiplied by a known function. However,

$$u'(t) = u(t)(1 - u(t)),$$

is nonlinear because of the term $-u^2$ where the unknown function is multiplied by itself. Also

$$\frac{\partial u}{\partial t} + u\frac{\partial u}{\partial x} = 0,$$

is nonlinear because of the term uu_x where the unknown function appears in a product with its derivative. (Note here that we use different notations for derivatives: u' or du/dt for a function $u(t)$ of one variable, $\frac{\partial u}{\partial t}$ or u_t for a function of more than one variable.)

Another example of a nonlinear equation is

$$u'' + \sin(u) = 0,$$

because $\sin(u)$ contains products of u, which becomes clear if we expand the function in a Taylor series:

$$\sin(u) = u - \frac{1}{3}u^3 + \ldots$$

Mathematical proof of linearity

To really prove mathematically that some differential equation in an unknown u is linear, show for each term $T(u)$ that with $u = au_1 + bu_2$ for constants a and b,

$$T(au_1 + bu_2) = aT(u_1) + bT(u_2)\,.$$

For example, the term $T(u) = (\sin^2 t)u'(t)$ is linear because

$$\begin{aligned} T(au_1 + bu_2) &= (\sin^2 t)(au_1(t) + bu_2(t)) \\ &= a(\sin^2 t)u_1(t) + b(\sin^2 t)u_2(t) \\ &= aT(u_1) + bT(u_2)\,. \end{aligned}$$

However, $T(u) = \sin u$ is nonlinear because

$$T(au_1 + bu_2) = \sin(au_1 + bu_2) \neq a\sin u_1 + b\sin u_2\,.$$

5.1.2 A simple model problem

A series of forthcoming examples will explain how to tackle nonlinear differential equations with various techniques. We start with the (scaled) logistic equation as model problem:

$$u'(t) = u(t)(1 - u(t))\,. \tag{5.1}$$

This is a nonlinear ordinary differential equation (ODE) which will be solved by different strategies in the following. Depending on the chosen time discretization of (5.1), the mathematical problem to be solved at every time level will either be a linear algebraic equation or a nonlinear algebraic equation. In the former case, the time discretization method transforms the nonlinear ODE into linear subproblems at each time level, and the solution is straightforward to find since linear algebraic equations are easy to solve. However, when the time discretization leads to nonlinear algebraic equations, we cannot (except in very rare cases) solve these without turning to approximate, iterative solution methods.

The next subsections introduce various methods for solving nonlinear differential equations, using (5.1) as model. We shall go through the following set of cases:

- explicit time discretization methods (with no need to solve nonlinear algebraic equations)
- implicit Backward Euler time discretization, leading to nonlinear algebraic equations solved by
  - an exact analytical technique
  - Picard iteration based on manual linearization
  - a single Picard step
  - Newton's method
- implicit Crank-Nicolson time discretization and linearization via a geometric mean formula

Thereafter, we compare the performance of the various approaches. Despite the simplicity of (5.1), the conclusions reveal typical features of the various methods in much more complicated nonlinear PDE problems.

5.1.3 Linearization by explicit time discretization

Time discretization methods are divided into explicit and implicit methods. Explicit methods lead to a closed-form formula for finding new values of the unknowns, while implicit methods give a linear or nonlinear system of equations that couples (all) the unknowns at a new time level. Here we shall demonstrate that explicit methods constitute an efficient way to deal with nonlinear differential equations.

The Forward Euler method is an explicit method. When applied to (5.1), sampled at $t = t_n$, it results in

$$\frac{u^{n+1} - u^n}{\Delta t} = u^n(1 - u^n),$$

which is a *linear* algebraic equation for the unknown value u^{n+1} that we can easily solve:

$$u^{n+1} = u^n + \Delta t\, u^n(1 - u^n)\,.$$

In this case, the nonlinearity in the original equation poses no difficulty in the discrete algebraic equation. Any other explicit scheme in time will also give only linear algebraic equations to solve. For example, a typical 2nd-order Runge-Kutta method for (5.1) leads to the following formulas:

$$u^* = u^n + \Delta t u^n(1-u^n),$$
$$u^{n+1} = u^n + \Delta t \frac{1}{2}\left(u^n(1-u^n) + u^*(1-u^*)\right)\,.$$

The first step is linear in the unknown u^*. Then u^* is known in the next step, which is linear in the unknown u^{n+1} .

5.1.4 Exact solution of nonlinear algebraic equations

Switching to a Backward Euler scheme for (5.1),

$$\frac{u^n - u^{n-1}}{\Delta t} = u^n(1-u^n), \tag{5.2}$$

results in a nonlinear algebraic equation for the unknown value u^n. The equation is of quadratic type:

$$\Delta t(u^n)^2 + (1-\Delta t)u^n - u^{n-1} = 0,$$

and may be solved exactly by the well-known formula for such equations. Before we do so, however, we will introduce a shorter, and often cleaner, notation for nonlinear algebraic equations at a given time level. The notation is inspired by the natural notation (i.e., variable names) used in a program, especially in more advanced partial differential equation problems. The unknown in the algebraic equation is denoted by u, while $u^{(1)}$ is the value of the unknown at the previous time level (in general, $u^{(\ell)}$ is the value of the unknown ℓ levels back in time). The notation will be frequently used in later sections. What is meant by u should be evident from the context: u may either be 1) the exact solution of the ODE/PDE problem, 2) the numerical approximation to the exact solution, or 3) the unknown solution at a certain time level.

The quadratic equation for the unknown u^n in (5.2) can, with the new notation, be written

$$F(u) = \Delta t u^2 + (1-\Delta t)u - u^{(1)} = 0\,. \tag{5.3}$$

The solution is readily found to be

$$u = \frac{1}{2\Delta t}\left(-1 + \Delta t \pm \sqrt{(1-\Delta t)^2 - 4\Delta t u^{(1)}}\right)\,. \tag{5.4}$$

Now we encounter a fundamental challenge with nonlinear algebraic equations: the equation may have more than one solution. How do we

pick the right solution? This is in general a hard problem. In the present simple case, however, we can analyze the roots mathematically and provide an answer. The idea is to expand the roots in a series in Δt and truncate after the linear term since the Backward Euler scheme will introduce an error proportional to Δt anyway. Using `sympy`, we find the following Taylor series expansions of the roots:

```
>>> import sympy as sym
>>> dt, u_1, u = sym.symbols('dt u_1 u')
>>> r1, r2 = sym.solve(dt*u**2 + (1-dt)*u - u_1, u)  # find roots
>>> r1
(dt - sqrt(dt**2 + 4*dt*u_1 - 2*dt + 1) - 1)/(2*dt)
>>> r2
(dt + sqrt(dt**2 + 4*dt*u_1 - 2*dt + 1) - 1)/(2*dt)
>>> print r1.series(dt, 0, 2)    # 2 terms in dt, around dt=0
-1/dt + 1 - u_1 + dt*(u_1**2 - u_1) + O(dt**2)
>>> print r2.series(dt, 0, 2)
u_1 + dt*(-u_1**2 + u_1) + O(dt**2)
```

We see that the `r1` root, corresponding to a minus sign in front of the square root in (5.4), behaves as $1/\Delta t$ and will therefore blow up as $\Delta t \rightarrow 0$! Since we know that u takes on finite values, actually it is less than or equal to 1, only the `r2` root is of relevance in this case: as $\Delta t \rightarrow 0$, $u \rightarrow u^{(1)}$, which is the expected result.

For those who are not well experienced with approximating mathematical formulas by series expansion, an alternative method of investigation is simply to compute the limits of the two roots as $\Delta t \rightarrow 0$ and see if a limit appears unreasonable:

```
>>> print r1.limit(dt, 0)
-oo
>>> print r2.limit(dt, 0)
u_1
```

5.1.5 Linearization

When the time integration of an ODE results in a nonlinear algebraic equation, we must normally find its solution by defining a sequence of linear equations and hope that the solutions of these linear equations converge to the desired solution of the nonlinear algebraic equation. Usually, this means solving the linear equation repeatedly in an iterative fashion. Alternatively, the nonlinear equation can sometimes be approximated by one linear equation, and consequently there is no need for iteration.

Constructing a linear equation from a nonlinear one requires *linearization* of each nonlinear term. This can be done manually as in Picard iteration, or fully algorithmically as in Newton's method. Examples will best illustrate how to linearize nonlinear problems.

5.1.6 Picard iteration

Let us write (5.3) in a more compact form

$$F(u) = au^2 + bu + c = 0,$$

with $a = \Delta t$, $b = 1 - \Delta t$, and $c = -u^{(1)}$. Let u^- be an available approximation of the unknown u. Then we can linearize the term u^2 simply by writing $u^- u$. The resulting equation, $\hat{F}(u) = 0$, is now linear and hence easy to solve:

$$F(u) \approx \hat{F}(u) = au^- u + bu + c = 0\,.$$

Since the equation $\hat{F} = 0$ is only approximate, the solution u does not equal the exact solution u_e of the exact equation $F(u_\mathrm{e}) = 0$, but we can hope that u is closer to u_e than u^- is, and hence it makes sense to repeat the procedure, i.e., set $u^- = u$ and solve $\hat{F}(u) = 0$ again. There is no guarantee that u is closer to u_e than u^-, but this approach has proven to be effective in a wide range of applications.

The idea of turning a nonlinear equation into a linear one by using an approximation u^- of u in nonlinear terms is a widely used approach that goes under many names: *fixed-point iteration*, the method of *successive substitutions*, *nonlinear Richardson iteration*, and *Picard iteration*. We will stick to the latter name.

Picard iteration for solving the nonlinear equation arising from the Backward Euler discretization of the logistic equation can be written as

$$u = -\frac{c}{au^- + b}, \quad u^- \leftarrow u\,.$$

The $\leftarrow$ symbols means assignment (we set u^- equal to the value of u). The iteration is started with the value of the unknown at the previous time level: $u^- = u^{(1)}$.

Some prefer an explicit iteration counter as superscript in the mathematical notation. Let u^k be the computed approximation to the solution in iteration k. In iteration $k+1$ we want to solve

$$au^k u^{k+1} + bu^{k+1} + c = 0 \quad \Rightarrow \quad u^{k+1} = -\frac{c}{au^k + b}, \quad k = 0, 1, \ldots$$

Since we need to perform the iteration at every time level, the time level counter is often also included:

$$au^{n,k}u^{n,k+1} + bu^{n,k+1} - u^{n-1} = 0 \quad \Rightarrow \quad u^{n,k+1} = \frac{u^n}{au^{n,k} + b}, \quad k = 0, 1, \ldots,$$

with the start value $u^{n,0} = u^{n-1}$ and the final converged value $u^n = u^{n,k}$ for sufficiently large k.

However, we will normally apply a mathematical notation in our final formulas that is as close as possible to what we aim to write in a computer code and then it becomes natural to use u and u^- instead of u^{k+1} and u^k or $u^{n,k+1}$ and $u^{n,k}$.

Stopping criteria. The iteration method can typically be terminated when the change in the solution is smaller than a tolerance ϵ_u:

$$|u - u^-| \leq \epsilon_u,$$

or when the residual in the equation is sufficiently small ($< \epsilon_r$),

$$|F(u)| = |au^2 + bu + c| < \epsilon_r \, .$$

A single Picard iteration. Instead of iterating until a stopping criterion is fulfilled, one may iterate a specific number of times. Just one Picard iteration is popular as this corresponds to the intuitive idea of approximating a nonlinear term like $(u^n)^2$ by $u^{n-1}u^n$. This follows from the linearization u^-u^n and the initial choice of $u^- = u^{n-1}$ at time level t_n. In other words, a single Picard iteration corresponds to using the solution at the previous time level to linearize nonlinear terms. The resulting discretization becomes (using proper values for a, b, and c)

$$\frac{u^n - u^{n-1}}{\Delta t} = u^n(1 - u^{n-1}), \tag{5.5}$$

which is a linear algebraic equation in the unknown u^n, making it easy to solve for u^n without any need for an alternative notation.

We shall later refer to the strategy of taking one Picard step, or equivalently, linearizing terms with use of the solution at the previous time step, as the *Picard1* method. It is a widely used approach in science

and technology, but with some limitations if Δt is not sufficiently small (as will be illustrated later).

Notice

Equation (5.5) does not correspond to a "pure" finite difference method where the equation is sampled at a point and derivatives replaced by differences (because the u^{n-1} term on the right-hand side must then be u^n). The best interpretation of the scheme (5.5) is a Backward Euler difference combined with a single (perhaps insufficient) Picard iteration at each time level, with the value at the previous time level as start for the Picard iteration.

5.1.7 Linearization by a geometric mean

We consider now a Crank-Nicolson discretization of (5.1). This means that the time derivative is approximated by a centered difference,

$$[D_t u = u(1-u)]^{n+\frac{1}{2}},$$

written out as

$$\frac{u^{n+1}-u^n}{\Delta t} = u^{n+\frac{1}{2}} - (u^{n+\frac{1}{2}})^2\,. \tag{5.6}$$

The term $u^{n+\frac{1}{2}}$ is normally approximated by an arithmetic mean,

$$u^{n+\frac{1}{2}} \approx \frac{1}{2}(u^n + u^{n+1}),$$

such that the scheme involves the unknown function only at the time levels where we actually intend to compute it. The same arithmetic mean applied to the nonlinear term gives

$$(u^{n+\frac{1}{2}})^2 \approx \frac{1}{4}(u^n + u^{n+1})^2,$$

which is nonlinear in the unknown u^{n+1}. However, using a *geometric mean* for $(u^{n+\frac{1}{2}})^2$ is a way of linearizing the nonlinear term in (5.6):

$$(u^{n+\frac{1}{2}})^2 \approx u^n u^{n+1}\,.$$

Using an arithmetic mean on the linear $u^{n+\frac{1}{2}}$ term in (5.6) and a geometric mean for the second term, results in a linearized equation for the unknown u^{n+1}:

$$\frac{u^{n+1}-u^n}{\Delta t} = \frac{1}{2}(u^n + u^{n+1}) + u^n u^{n+1},$$

which can readily be solved:

$$u^{n+1} = \frac{1+\frac{1}{2}\Delta t}{1+\Delta t u^n - \frac{1}{2}\Delta t} u^n \,.$$

This scheme can be coded directly, and since there is no nonlinear algebraic equation to iterate over, we skip the simplified notation with u for u^{n+1} and $u^{(1)}$ for u^n. The technique with using a geometric average is an example of transforming a nonlinear algebraic equation to a linear one, without any need for iterations.

The geometric mean approximation is often very effective for linearizing quadratic nonlinearities. Both the arithmetic and geometric mean approximations have truncation errors of order Δt^2 and are therefore compatible with the truncation error $\mathcal{O}(\Delta t^2)$ of the centered difference approximation for u' in the Crank-Nicolson method.

Applying the operator notation for the means and finite differences, the linearized Crank-Nicolson scheme for the logistic equation can be compactly expressed as

$$[D_t u = \overline{u}^t + \overline{u^2}^{t,g}]^{n+\frac{1}{2}} \,.$$

Remark

If we use an arithmetic instead of a geometric mean for the nonlinear term in (5.6), we end up with a nonlinear term $(u^{n+1})^2$. This term can be linearized as $u^- u^{n+1}$ in a Picard iteration approach and in particular as $u^n u^{n+1}$ in a Picard1 iteration approach. The latter gives a scheme almost identical to the one arising from a geometric mean (the difference in u^{n+1} being $\frac{1}{4}\Delta t u^n(u^{n+1}-u^n) \approx \frac{1}{4}\Delta t^2 u' u$, i.e., a difference of size Δt^2).

5.1.8 Newton's method

The Backward Euler scheme (5.2) for the logistic equation leads to a nonlinear algebraic equation (5.3). Now we write any nonlinear algebraic equation in the general and compact form

$$F(u) = 0\,.$$

Newton's method linearizes this equation by approximating $F(u)$ by its Taylor series expansion around a computed value u^- and keeping only the linear part:

$$\begin{aligned} F(u) &= F(u^-) + F'(u^-)(u-u^-) + \frac{1}{2}F''(u^-)(u-u^-)^2 + \cdots \\ &\approx F(u^-) + F'(u^-)(u-u^-) = \hat{F}(u)\,. \end{aligned}$$

The linear equation $\hat{F}(u) = 0$ has the solution

$$u = u^- - \frac{F(u^-)}{F'(u^-)}\,.$$

Expressed with an iteration index in the unknown, Newton's method takes on the more familiar mathematical form

$$u^{k+1} = u^k - \frac{F(u^k)}{F'(u^k)}, \quad k = 0, 1, \ldots$$

It can be shown that the error in iteration $k+1$ of Newton's method is proportional to the square of the error in iteration k, a result referred to as *quadratic convergence*. This means that for small errors the method converges very fast, and in particular much faster than Picard iteration and other iteration methods. (The proof of this result is found in most textbooks on numerical analysis.) However, the quadratic convergence appears only if u^k is sufficiently close to the solution. Further away from the solution the method can easily converge very slowly or diverge. The reader is encouraged to do Exercise 5.3 to get a better understanding for the behavior of the method.

Application of Newton's method to the logistic equation discretized by the Backward Euler method is straightforward as we have

$$F(u) = au^2 + bu + c, \quad a = \Delta t,\ b = 1 - \Delta t,\ c = -u^{(1)},$$

and then

$$F'(u) = 2au + b\,.$$

The iteration method becomes

$$u = u^- + \frac{a(u^-)^2 + bu^- + c}{2au^- + b}, \quad u^- \leftarrow u\,. \tag{5.7}$$

At each time level, we start the iteration by setting $u^- = u^{(1)}$. Stopping criteria as listed for the Picard iteration can be used also for Newton's method.

An alternative mathematical form, where we write out a, b, and c, and use a time level counter n and an iteration counter k, takes the form

$$u^{n,k+1} = u^{n,k} + \frac{\Delta t(u^{n,k})^2 + (1-\Delta t)u^{n,k} - u^{n-1}}{2\Delta t u^{n,k} + 1 - \Delta t}, \quad u^{n,0} = u^{n-1}, \tag{5.8}$$

for $k = 0, 1, \ldots$. A program implementation is much closer to (5.7) than to (5.8), but the latter is better aligned with the established mathematical notation used in the literature.

5.1.9 Relaxation

One iteration in Newton's method or Picard iteration consists of solving a linear problem $\hat{F}(u) = 0$. Sometimes convergence problems arise because the new solution u of $\hat{F}(u) = 0$ is "too far away" from the previously computed solution u^-. A remedy is to introduce a relaxation, meaning that we first solve $\hat{F}(u^*) = 0$ for a suggested value u^* and then we take u as a weighted mean of what we had, u^-, and what our linearized equation $\hat{F} = 0$ suggests, u^*:

$$u = \omega u^* + (1-\omega)u^-\,.$$

The parameter ω is known as a *relaxation parameter*, and a choice $\omega < 1$ may prevent divergent iterations.

Relaxation in Newton's method can be directly incorporated in the basic iteration formula:

$$u = u^- - \omega\frac{F(u^-)}{F'(u^-)}\,. \tag{5.9}$$

5.1.10 Implementation and experiments

The program `logistic.py` contains implementations of all the methods described above. Below is an extract of the file showing how the Picard and Newton methods are implemented for a Backward Euler discretization of the logistic equation.

```
def BE_logistic(u0, dt, Nt, choice='Picard',
                eps_r=1E-3, omega=1, max_iter=1000):
    if choice == 'Picard1':
        choice = 'Picard'
        max_iter = 1

    u = np.zeros(Nt+1)
    iterations = []
    u[0] = u0
    for n in range(1, Nt+1):
        a = dt
        b = 1 - dt
        c = -u[n-1]

        if choice == 'Picard':

            def F(u):
                return a*u**2 + b*u + c

            u_ = u[n-1]
            k = 0
            while abs(F(u_)) > eps_r and k < max_iter:
                u_ = omega*(-c/(a*u_ + b)) + (1-omega)*u_
                k += 1
            u[n] = u_
            iterations.append(k)

        elif choice == 'Newton':

            def F(u):
                return a*u**2 + b*u + c

            def dF(u):
                return 2*a*u + b

            u_ = u[n-1]
            k = 0
            while abs(F(u_)) > eps_r and k < max_iter:
                u_ = u_ - F(u_)/dF(u_)
                k += 1
            u[n] = u_
            iterations.append(k)
    return u, iterations
```

The Crank-Nicolson method utilizing a linearization based on the geometric mean gives a simpler algorithm:

```
def CN_logistic(u0, dt, Nt):
    u = np.zeros(Nt+1)
    u[0] = u0
    for n in range(0, Nt):
        u[n+1] = (1 + 0.5*dt)/(1 + dt*u[n] - 0.5*dt)*u[n]
    return u
```

We may run experiments with the model problem (5.1) and the different strategies for dealing with nonlinearities as described above. For a quite coarse time resolution, $\Delta t = 0.9$, use of a tolerance $\epsilon_r = 0.1$ in the stopping criterion introduces an iteration error, especially in the Picard iterations, that is visibly much larger than the time discretization error due to a large Δt. This is illustrated by comparing the upper two plots in Figure 5.1. The one to the right has a stricter tolerance $\epsilon = 10^{-3}$, which causes all the curves corresponding to Picard and Newton iteration to be on top of each other (and no changes can be visually observed by reducing ϵ_r further). The reason why Newton's method does much better than Picard iteration in the upper left plot is that Newton's method with one step comes far below the ϵ_r tolerance, while the Picard iteration needs on average 7 iterations to bring the residual down to $\epsilon_r = 10^{-1}$, which gives insufficient accuracy in the solution of the nonlinear equation. It is obvious that the Picard1 method gives significant errors in addition to the time discretization unless the time step is as small as in the lower right plot.

The *BE exact* curve corresponds to using the exact solution of the quadratic equation at each time level, so this curve is only affected by the Backward Euler time discretization. The *CN gm* curve corresponds to the theoretically more accurate Crank-Nicolson discretization, combined with a geometric mean for linearization. This curve appears more accurate, especially if we take the plot in the lower right with a small Δt and an appropriately small ϵ_r value as the exact curve.

When it comes to the need for iterations, Figure 5.2 displays the number of iterations required at each time level for Newton's method and Picard iteration. The smaller Δt is, the better starting value we have for the iteration, and the faster the convergence is. With $\Delta t = 0.9$ Picard iteration requires on average 32 iterations per time step, but this number is dramatically reduced as Δt is reduced.

However, introducing relaxation and a parameter $\omega = 0.8$ immediately reduces the average of 32 to 7, indicating that for the large $\Delta t = 0.9$, Picard iteration takes too long steps. An approximately optimal value for ω in this case is 0.5, which results in an average of only 2 iterations! An even more dramatic impact of ω appears when $\Delta t = 1$: Picard iteration

does not convergence in 1000 iterations, but $\omega = 0.5$ again brings the average number of iterations down to 2.

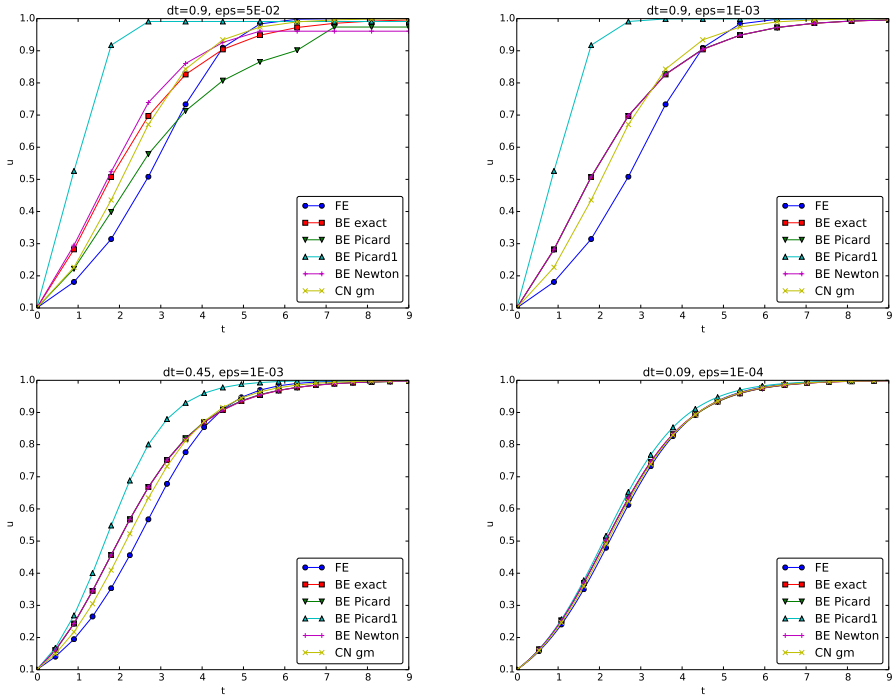


Fig. 5.1 Impact of solution strategy and time step length on the solution.

Remark. The simple Crank-Nicolson method with a geometric mean for the quadratic nonlinearity gives visually more accurate solutions than the Backward Euler discretization. Even with a tolerance of $\epsilon_r = 10^{-3}$, all the methods for treating the nonlinearities in the Backward Euler discretization give graphs that cannot be distinguished. So for accuracy in this problem, the time discretization is much more crucial than ϵ_r. Ideally, one should estimate the error in the time discretization, as the solution progresses, and set ϵ_r accordingly.

5.1.11 Generalization to a general nonlinear ODE

Let us see how the various methods in the previous sections can be applied to the more generic model

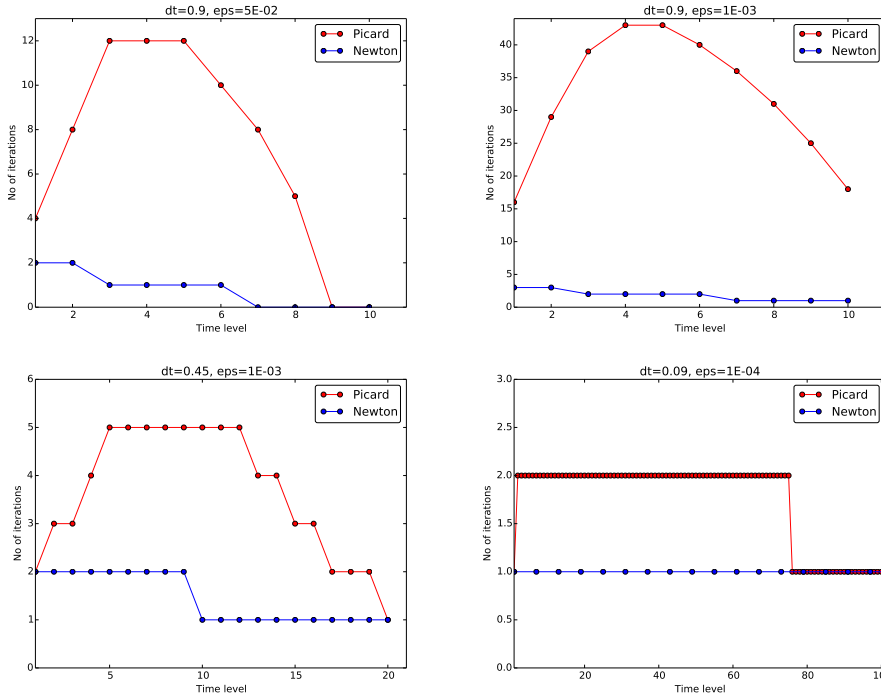


Fig. 5.2 Comparison of the number of iterations at various time levels for Picard and Newton iteration.

$$u' = f(u,t), \tag{5.10}$$

where f is a nonlinear function of u.

Explicit time discretization. Explicit ODE methods like the Forward Euler scheme, Runge-Kutta methods and Adams-Bashforth methods all evaluate f at time levels where u is already computed, so nonlinearities in f do not pose any difficulties.

Backward Euler discretization. Approximating u' by a backward difference leads to a Backward Euler scheme, which can be written as

$$F(u^n) = u^n - \Delta t\, f(u^n, t_n) - u^{n-1} = 0,$$

or alternatively

$$F(u) = u - \Delta t\, f(u, t_n) - u^{(1)} = 0\,.$$

A simple Picard iteration, not knowing anything about the nonlinear structure of f, must approximate $f(u, t_n)$ by $f(u^-, t_n)$:

$$\hat{F}(u) = u - \Delta t\, f(u^-, t_n) - u^{(1)}\,.$$

The iteration starts with $u^- = u^{(1)}$ and proceeds with repeating

$$u^* = \Delta t\, f(u^-, t_n) + u^{(1)}, \quad u = \omega u^* + (1-\omega)u^-, \quad u^- \leftarrow u,$$

until a stopping criterion is fulfilled.

Explicit vs implicit treatment of nonlinear terms

Evaluating f for a known u^- is referred to as *explicit* treatment of f, while if $f(u,t)$ has some structure, say $f(u,t) = u^3$, parts of f can involve the unknown u, as in the manual linearization $(u^-)^2 u$, and then the treatment of f is "more implicit" and "less explicit". This terminology is inspired by time discretization of $u' = f(u,t)$, where evaluating f for known u values gives explicit schemes, while treating f or parts of f implicitly, makes f contribute to the unknown terms in the equation at the new time level.

Explicit treatment of f usually means stricter conditions on Δt to achieve stability of time discretization schemes. The same applies to iteration techniques for nonlinear algebraic equations: the "less" we linearize f (i.e., the more we keep of u in the original formula), the faster the convergence may be.

We may say that $f(u,t) = u^3$ is treated explicitly if we evaluate f as $(u^-)^3$, partially implicit if we linearize as $(u^-)^2 u$ and fully implicit if we represent f by u^3. (Of course, the fully implicit representation will require further linearization, but with $f(u,t) = u^2$ a fully implicit treatment is possible if the resulting quadratic equation is solved with a formula.)

For the ODE $u' = -u^3$ with $f(u,t) = -u^3$ and coarse time resolution $\Delta t = 0.4$, Picard iteration with $(u^-)^2 u$ requires 8 iterations with $\epsilon_r = 10^{-3}$ for the first time step, while $(u^-)^3$ leads to 22 iterations. After about 10 time steps both approaches are down to about 2 iterations per time step, but this example shows a potential of treating f more implicitly.

A trick to treat f implicitly in Picard iteration is to evaluate it as $f(u^-,t)u/u^-$. For a polynomial f, $f(u,t) = u^m$, this corresponds to $(u^-)^m u/u^- = (u^-)^{m-1}u$. Sometimes this more implicit treatment has no effect, as with $f(u,t) = \exp(-u)$ and $f(u,t) = \ln(1+u)$,

but with $f(u,t) = \sin(2(u+1))$, the $f(u^-,t)u/u^-$ trick leads to 7, 9, and 11 iterations during the first three steps, while $f(u^-,t)$ demands 17, 21, and 20 iterations. (Experiments can be done with the code `ODE_Picard_tricks.py`.)

Newton's method applied to a Backward Euler discretization of $u' = f(u,t)$ requires computation of the derivative

$$F'(u) = 1 - \Delta t \frac{\partial f}{\partial u}(u, t_n)\,.$$

Starting with the solution at the previous time level, $u^- = u^{(1)}$, we can just use the standard formula

$$u = u^- - \omega \frac{F(u^-)}{F'(u^-)} = u^- - \omega \frac{u^- - \Delta t\, f(u^-, t_n) - u^{(1)}}{1 - \Delta t \frac{\partial}{\partial u} f(u^-, t_n)}\,. \tag{5.11}$$

Crank-Nicolson discretization. The standard Crank-Nicolson scheme with arithmetic mean approximation of f takes the form

$$\frac{u^{n+1} - u^n}{\Delta t} = \frac{1}{2}(f(u^{n+1}, t_{n+1}) + f(u^n, t_n))\,.$$

We can write the scheme as a nonlinear algebraic equation

$$F(u) = u - u^{(1)} - \Delta t \frac{1}{2} f(u, t_{n+1}) - \Delta t \frac{1}{2} f(u^{(1)}, t_n) = 0\,. \tag{5.12}$$

A Picard iteration scheme must in general employ the linearization

$$\hat{F}(u) = u - u^{(1)} - \Delta t \frac{1}{2} f(u^-, t_{n+1}) - \Delta t \frac{1}{2} f(u^{(1)}, t_n),$$

while Newton's method can apply the general formula (5.11) with $F(u)$ given in (5.12) and

$$F'(u) = 1 - \frac{1}{2}\Delta t \frac{\partial f}{\partial u}(u, t_{n+1})\,.$$

5.1.12 Systems of ODEs

We may write a system of ODEs

$$\begin{aligned}
\frac{d}{dt}u_0(t) &= f_0(u_0(t), u_1(t), \ldots, u_N(t), t),\\
\frac{d}{dt}u_1(t) &= f_1(u_0(t), u_1(t), \ldots, u_N(t), t),\\
&\vdots\\
\frac{d}{dt}u_m(t) &= f_m(u_0(t), u_1(t), \ldots, u_N(t), t),
\end{aligned}$$

as

$$u' = f(u,t), \quad u(0) = U_0, \tag{5.13}$$

if we interpret u as a vector $u = (u_0(t), u_1(t), \ldots, u_N(t))$ and f as a vector function with components $(f_0(u,t), f_1(u,t), \ldots, f_N(u,t))$.

Most solution methods for scalar ODEs, including the Forward and Backward Euler schemes and the Crank-Nicolson method, generalize in a straightforward way to systems of ODEs simply by using vector arithmetics instead of scalar arithmetics, which corresponds to applying the scalar scheme to each component of the system. For example, here is a backward difference scheme applied to each component,

$$\begin{aligned}
\frac{u_0^n - u_0^{n-1}}{\Delta t} &= f_0(u^n, t_n),\\
\frac{u_1^n - u_1^{n-1}}{\Delta t} &= f_1(u^n, t_n),\\
&\vdots\\
\frac{u_N^n - u_N^{n-1}}{\Delta t} &= f_N(u^n, t_n),
\end{aligned}$$

which can be written more compactly in vector form as

$$\frac{u^n - u^{n-1}}{\Delta t} = f(u^n, t_n)\,.$$

This is a *system of algebraic equations*,

$$u^n - \Delta t\, f(u^n, t_n) - u^{n-1} = 0,$$

or written out

$$
\begin{aligned}
u_0^n - \Delta t\, f_0(u^n, t_n) - u_0^{n-1} &= 0, \\
&\vdots \\
u_N^n - \Delta t\, f_N(u^n, t_n) - u_N^{n-1} &= 0\,.
\end{aligned}
$$

Example. We shall address the 2×2 ODE system for oscillations of a pendulum subject to gravity and air drag. The system can be written as

$$\dot{\omega} = -\sin\theta - \beta\omega|\omega|, \tag{5.14}$$

$$\dot{\theta} = \omega, \tag{5.15}$$

where β is a dimensionless parameter (this is the scaled, dimensionless version of the original, physical model). The unknown components of the system are the angle $\theta(t)$ and the angular velocity $\omega(t)$. We introduce $u_0 = \omega$ and $u_1 = \theta$, which leads to

$$
\begin{aligned}
u_0' &= f_0(u,t) = -\sin u_1 - \beta u_0|u_0|, \\
u_1' &= f_1(u,t) = u_0\,.
\end{aligned}
$$

A Crank-Nicolson scheme reads

$$
\begin{aligned}
\frac{u_0^{n+1} - u_0^n}{\Delta t} &= -\sin u_1^{n+\frac{1}{2}} - \beta u_0^{n+\frac{1}{2}}|u_0^{n+\frac{1}{2}}| \\
&\approx -\sin\left(\frac{1}{2}(u_1^{n+1} + u_1 n)\right) - \beta\frac{1}{4}(u_0^{n+1} + u_0^n)|u_0^{n+1} + u_0^n|,
\end{aligned} \tag{5.16}
$$

$$\frac{u_1^{n+1} - u_1^n}{\Delta t} = u_0^{n+\frac{1}{2}} \approx \frac{1}{2}(u_0^{n+1} + u_0^n)\,. \tag{5.17}$$

This is a *coupled system* of two nonlinear algebraic equations in two unknowns u_0^{n+1} and u_1^{n+1}.

Using the notation u_0 and u_1 for the unknowns u_0^{n+1} and u_1^{n+1} in this system, writing $u_0^{(1)}$ and $u_1^{(1)}$ for the previous values u_0^n and u_1^n, multiplying by Δt and moving the terms to the left-hand sides, gives

$$u_0 - u_0^{(1)} + \Delta t \sin\left(\frac{1}{2}(u_1 + u_1^{(1)})\right) + \frac{1}{4}\Delta t\beta(u_0 + u_0^{(1)})|u_0 + u_0^{(1)}| = 0, \tag{5.18}$$

$$u_1 - u_1^{(1)} - \frac{1}{2}\Delta t(u_0 + u_0^{(1)}) = 0\,. \tag{5.19}$$

Obviously, we have a need for solving systems of nonlinear algebraic equations, which is the topic of the next section.

5.2 Systems of nonlinear algebraic equations

Implicit time discretization methods for a system of ODEs, or a PDE, lead to *systems* of nonlinear algebraic equations, written compactly as

$$F(u) = 0,$$

where u is a vector of unknowns $u = (u_0, \ldots, u_N)$, and F is a vector function: $F = (F_0, \ldots, F_N)$. The system at the end of Section 5.1.12 fits this notation with $N = 1$, $F_0(u)$ given by the left-hand side of (5.18), while $F_1(u)$ is the left-hand side of (5.19).

Sometimes the equation system has a special structure because of the underlying problem, e.g.,

$$A(u)u = b(u),$$

with $A(u)$ as an $(N+1) \times (N+1)$ matrix function of u and b as a vector function: $b = (b_0, \ldots, b_N)$.

We shall next explain how Picard iteration and Newton's method can be applied to systems like $F(u) = 0$ and $A(u)u = b(u)$. The exposition has a focus on ideas and practical computations. More theoretical considerations, including quite general results on convergence properties of these methods, can be found in Kelley [8].

5.2.1 Picard iteration

We cannot apply Picard iteration to nonlinear equations unless there is some special structure. For the commonly arising case $A(u)u = b(u)$ we can linearize the product $A(u)u$ to $A(u^-)u$ and $b(u)$ as $b(u^-)$. That is,

we use the most previously computed approximation in A and b to arrive at a *linear system* for u:

$$A(u^-)u = b(u^-)\,.$$

A relaxed iteration takes the form

$$A(u^-)u^* = b(u^-), \quad u = \omega u^* + (1-\omega)u^-\,.$$

In other words, we solve a system of nonlinear algebraic equations as a sequence of linear systems.

Algorithm for relaxed Picard iteration

Given $A(u)u = b(u)$ and an initial guess u^-, iterate until convergence:

1. solve $A(u^-)u^* = b(u^-)$ with respect to u^*
2. $u = \omega u^* + (1-\omega)u^-$
3. $u^- \leftarrow u$

"Until convergence" means that the iteration is stopped when the change in the unknown, $||u - u^-||$, or the residual $||A(u)u - b||$, is sufficiently small, see Section 5.2.3 for more details.

5.2.2 Newton's method

The natural starting point for Newton's method is the general nonlinear vector equation $F(u) = 0$. As for a scalar equation, the idea is to approximate F around a known value u^- by a linear function $\hat{F}$, calculated from the first two terms of a Taylor expansion of F. In the multi-variate case these two terms become

$$F(u^-) + J(u^-)\cdot(u - u^-),$$

where J is the *Jacobian* of F, defined by

$$J_{i,j} = \frac{\partial F_i}{\partial u_j}\,.$$

So, the original nonlinear system is approximated by

$$\hat{F}(u) = F(u^-) + J(u^-) \cdot (u - u^-) = 0,$$

which is linear in u and can be solved in a two-step procedure: first solve $J\delta u = -F(u^-)$ with respect to the vector δu and then update $u = u^- + \delta u$. A relaxation parameter can easily be incorporated:

$$u = \omega(u^- + \delta u) + (1 - \omega)u^- = u^- + \omega\delta u\,.$$

Algorithm for Newton's method

Given $F(u) = 0$ and an initial guess u^-, iterate until convergence:

1. solve $J\delta u = -F(u^-)$ with respect to δu
2. $u = u^- + \omega\delta u$
3. $u^- \leftarrow u$

For the special system with structure $A(u)u = b(u)$,

$$F_i = \sum_k A_{i,k}(u)u_k - b_i(u),$$

one gets

$$J_{i,j} = \sum_k \frac{\partial A_{i,k}}{\partial u_j}u_k + A_{i,j} - \frac{\partial b_i}{\partial u_j}\,. \tag{5.20}$$

We realize that the Jacobian needed in Newton's method consists of $A(u^-)$ as in the Picard iteration plus two additional terms arising from the differentiation. Using the notation $A'(u)$ for $\partial A/\partial u$ (a quantity with three indices: $\partial A_{i,k}/\partial u_j$), and $b'(u)$ for $\partial b/\partial u$ (a quantity with two indices: $\partial b_i/\partial u_j$), we can write the linear system to be solved as

$$(A + A'u + b')\delta u = -Au + b,$$

or

$$(A(u^-) + A'(u^-)u^- + b'(u^-))\delta u = -A(u^-)u^- + b(u^-)\,.$$

Rearranging the terms demonstrates the difference from the system solved in each Picard iteration:

$$\underbrace{A(u^-)(u^- + \delta u) - b(u^-)}_{\text{Picard system}} + \gamma(A'(u^-)u^- + b'(u^-))\delta u = 0\,.$$

Here we have inserted a parameter γ such that $\gamma = 0$ gives the Picard system and $\gamma = 1$ gives the Newton system. Such a parameter can be handy in software to easily switch between the methods.

Combined algorithm for Picard and Newton iteration

Given $A(u)$, $b(u)$, and an initial guess u^-, iterate until convergence:

1. solve $(A + \gamma(A'(u^-)u^- + b'(u^-)))\delta u = -A(u^-)u^- + b(u^-)$ with respect to δu
2. $u = u^- + \omega\delta u$
3. $u^- \leftarrow u$

$\gamma = 1$ gives a Newton method while $\gamma = 0$ corresponds to Picard iteration.

5.2.3 Stopping criteria

Let $||\cdot||$ be the standard Euclidean vector norm. Four termination criteria are much in use:

- Absolute change in solution: $||u - u^-|| \leq \epsilon_u$
- Relative change in solution: $||u - u^-|| \leq \epsilon_u||u_0||$, where u_0 denotes the start value of u^- in the iteration
- Absolute residual: $||F(u)|| \leq \epsilon_r$
- Relative residual: $||F(u)|| \leq \epsilon_r||F(u_0)||$

To prevent divergent iterations to run forever, one terminates the iterations when the current number of iterations k exceeds a maximum value $k_{\max}$.

The relative criteria are most used since they are not sensitive to the characteristic size of u. Nevertheless, the relative criteria can be misleading when the initial start value for the iteration is very close to the solution, since an unnecessary reduction in the error measure is enforced. In such cases the absolute criteria work better. It is common

to combine the absolute and relative measures of the size of the residual, as in

$$||F(u)|| \leq \epsilon_{rr}||F(u_0)|| + \epsilon_{ra}, \tag{5.21}$$

where ϵ_{rr} is the tolerance in the relative criterion and ϵ_{ra} is the tolerance in the absolute criterion. With a very good initial guess for the iteration (typically the solution of a differential equation at the previous time level), the term $||F(u_0)||$ is small and ϵ_{ra} is the dominating tolerance. Otherwise, $\epsilon_{rr}||F(u_0)||$ and the relative criterion dominates.

With the change in solution as criterion we can formulate a combined absolute and relative measure of the change in the solution:

$$||\delta u|| \leq \epsilon_{ur}||u_0|| + \epsilon_{ua}, \tag{5.22}$$

The ultimate termination criterion, combining the residual and the change in solution with a test on the maximum number of iterations, can be expressed as

$$||F(u)|| \leq \epsilon_{rr}||F(u_0)|| + \epsilon_{ra} \quad \text{or} \quad ||\delta u|| \leq \epsilon_{ur}||u_0|| + \epsilon_{ua} \quad \text{or} \quad k > k_{\max}. \tag{5.23}$$

5.2.4 Example: A nonlinear ODE model from epidemiology

A very simple model for the spreading of a disease, such as a flu, takes the form of a 2×2 ODE system

$$S' = -\beta SI, \tag{5.24}$$
$$I' = \beta SI - \nu I, \tag{5.25}$$

where $S(t)$ is the number of people who can get ill (susceptibles) and $I(t)$ is the number of people who are ill (infected). The constants $\beta > 0$ and $\nu > 0$ must be given along with initial conditions $S(0)$ and $I(0)$.

Implicit time discretization. A Crank-Nicolson scheme leads to a 2×2 system of nonlinear algebraic equations in the unknowns S^{n+1} and I^{n+1}:

$$\frac{S^{n+1}-S^n}{\Delta t} = -\beta[SI]^{n+\frac{1}{2}} \approx -\frac{\beta}{2}(S^nI^n + S^{n+1}I^{n+1}), \tag{5.26}$$

$$\frac{I^{n+1}-I^n}{\Delta t} = \beta[SI]^{n+\frac{1}{2}} - \nu I^{n+\frac{1}{2}} \approx \frac{\beta}{2}(S^nI^n + S^{n+1}I^{n+1}) - \frac{\nu}{2}(I^n + I^{n+1})\,. \tag{5.27}$$

Introducing S for S^{n+1}, $S^{(1)}$ for S^n, I for I^{n+1} and $I^{(1)}$ for I^n, we can rewrite the system as

$$F_S(S,I) = S - S^{(1)} + \frac{1}{2}\Delta t\beta(S^{(1)}I^{(1)} + SI) = 0, \tag{5.28}$$

$$F_I(S,I) = I - I^{(1)} - \frac{1}{2}\Delta t\beta(S^{(1)}I^{(1)} + SI) + \frac{1}{2}\Delta t\nu(I^{(1)} + I) = 0\,. \tag{5.29}$$

A Picard iteration. We assume that we have approximations S^- and I^- to S and I, respectively. A way of linearizing the only nonlinear term SI is to write I^-S in the $F_S = 0$ equation and S^-I in the $F_I = 0$ equation, which also *decouples* the equations. Solving the resulting linear equations with respect to the unknowns S and I gives

$$S = \frac{S^{(1)} - \frac{1}{2}\Delta t\beta S^{(1)}I^{(1)}}{1 + \frac{1}{2}\Delta t\beta I^-},$$

$$I = \frac{I^{(1)} + \frac{1}{2}\Delta t\beta S^{(1)}I^{(1)} - \frac{1}{2}\Delta t\nu I^{(1)}}{1 - \frac{1}{2}\Delta t\beta S^- + \frac{1}{2}\Delta t\nu}\,.$$

Before a new iteration, we must update $S^- \leftarrow S$ and $I^- \leftarrow I$.

Newton's method. The nonlinear system (5.28)-(5.29) can be written as $F(u) = 0$ with $F = (F_S, F_I)$ and $u = (S, I)$. The Jacobian becomes

$$J = \begin{pmatrix} \frac{\partial}{\partial S}F_S & \frac{\partial}{\partial I}F_S \\ \frac{\partial}{\partial S}F_I & \frac{\partial}{\partial I}F_I \end{pmatrix} = \begin{pmatrix} 1 + \frac{1}{2}\Delta t\beta I & \frac{1}{2}\Delta t\beta S \\ -\frac{1}{2}\Delta t\beta I & 1 - \frac{1}{2}\Delta t\beta S + \frac{1}{2}\Delta t\nu \end{pmatrix}.$$

The Newton system $J(u^-)\delta u = -F(u^-)$ to be solved in each iteration is then

$$\begin{pmatrix} 1+\frac{1}{2}\Delta t\beta I^- & \frac{1}{2}\Delta t\beta S^- \\ -\frac{1}{2}\Delta t\beta I^- & 1-\frac{1}{2}\Delta t\beta S^- + \frac{1}{2}\Delta t\nu \end{pmatrix} \begin{pmatrix} \delta S \\ \delta I \end{pmatrix} = \begin{pmatrix} S^- - S^{(1)} + \frac{1}{2}\Delta t\beta (S^{(1)}I^{(1)} + S^-I^-) \\ I^- - I^{(1)} - \frac{1}{2}\Delta t\beta (S^{(1)}I^{(1)} + S^-I^-) + \frac{1}{2}\Delta t\nu (I^{(1)} + I^-) \end{pmatrix}$$

Remark. For this particular system of ODEs, explicit time integration methods work very well. Even a Forward Euler scheme is fine, but (as also experienced more generally) the 4-th order Runge-Kutta method is an excellent balance between high accuracy, high efficiency, and simplicity.

5.3 Linearization at the differential equation level

The attention is now turned to nonlinear partial differential equations (PDEs) and application of the techniques explained above for ODEs. The model problem is a nonlinear diffusion equation for $u(\boldsymbol{x}, t)$:

$$\frac{\partial u}{\partial t} = \nabla \cdot (\alpha(u)\nabla u) + f(u), \qquad \boldsymbol{x} \in \Omega,\ t \in (0,T], \tag{5.30}$$

$$-\alpha(u)\frac{\partial u}{\partial n} = g, \qquad \boldsymbol{x} \in \partial\Omega_N,\ t \in (0,T], \tag{5.31}$$

$$u = u_0, \qquad \boldsymbol{x} \in \partial\Omega_D,\ t \in (0,T]\,. \tag{5.32}$$

In the present section, our aim is to discretize this problem in time and then present techniques for linearizing the time-discrete PDE problem "at the PDE level" such that we transform the nonlinear stationary PDE problem at each time level into a sequence of linear PDE problems, which can be solved using any method for linear PDEs. This strategy avoids the solution of systems of nonlinear algebraic equations. In Section 5.4 we shall take the opposite (and more common) approach: discretize the nonlinear problem in time and space first, and then solve the resulting nonlinear algebraic equations at each time level by the methods of Section 5.2. Very often, the two approaches are mathematically identical, so there is no preference from a computational efficiency point of view. The details of the ideas sketched above will hopefully become clear through the forthcoming examples.

5.3.1 Explicit time integration

The nonlinearities in the PDE are trivial to deal with if we choose an explicit time integration method for (5.30), such as the Forward Euler method:

$$[D_t^+ u = \nabla \cdot (\alpha(u)\nabla u) + f(u)]^n,$$

or written out,

$$\frac{u^{n+1} - u^n}{\Delta t} = \nabla \cdot (\alpha(u^n)\nabla u^n) + f(u^n),$$

which is a linear equation in the unknown u^{n+1} with solution

$$u^{n+1} = u^n + \Delta t \nabla \cdot (\alpha(u^n)\nabla u^n) + \Delta t f(u^n)\,.$$

The disadvantage with this discretization is the strict stability criterion $\Delta t \leq h^2/(6\max\alpha)$ for the case $f = 0$ and a standard 2nd-order finite difference discretization in 3D space with mesh cell sizes $h = \Delta x = \Delta y = \Delta z$.

5.3.2 Backward Euler scheme and Picard iteration

A Backward Euler scheme for (5.30) reads

$$[D_t^- u = \nabla \cdot (\alpha(u)\nabla u) + f(u)]^n\,.$$

Written out,

$$\frac{u^n - u^{n-1}}{\Delta t} = \nabla \cdot (\alpha(u^n)\nabla u^n) + f(u^n)\,. \tag{5.33}$$

This is a nonlinear PDE for the unknown function $u^n(\boldsymbol{x})$. Such a PDE can be viewed as a time-independent PDE where $u^{n-1}(\boldsymbol{x})$ is a known function.

We introduce a Picard iteration with k as iteration counter. A typical linearization of the $\nabla \cdot (\alpha(u^n)\nabla u^n)$ term in iteration $k+1$ is to use the previously computed $u^{n,k}$ approximation in the diffusion coefficient: $\alpha(u^{n,k})$. The nonlinear source term is treated similarly: $f(u^{n,k})$. The unknown function $u^{n,k+1}$ then fulfills the linear PDE

$$\frac{u^{n,k+1} - u^{n-1}}{\Delta t} = \nabla \cdot (\alpha(u^{n,k})\nabla u^{n,k+1}) + f(u^{n,k})\,. \tag{5.34}$$

The initial guess for the Picard iteration at this time level can be taken as the solution at the previous time level: $u^{n,0} = u^{n-1}$.

We can alternatively apply the implementation-friendly notation where u corresponds to the unknown we want to solve for, i.e., $u^{n,k+1}$ above, and u^- is the most recently computed value, $u^{n,k}$ above. Moreover, $u^{(1)}$ denotes the unknown function at the previous time level, u^{n-1} above. The PDE to be solved in a Picard iteration then looks like

$$\frac{u - u^{(1)}}{\Delta t} = \nabla \cdot (\alpha(u^-)\nabla u) + f(u^-)\,. \tag{5.35}$$

At the beginning of the iteration we start with the value from the previous time level: $u^- = u^{(1)}$, and after each iteration, u^- is updated to u.

Remark on notation

The previous derivations of the numerical scheme for time discretizations of PDEs have, strictly speaking, a somewhat sloppy notation, but it is much used and convenient to read. A more precise notation must distinguish clearly between the exact solution of the PDE problem, here denoted $u_{\mathrm{e}}(\boldsymbol{x}, t)$, and the exact solution of the spatial problem, arising after time discretization at each time level, where (5.33) is an example. The latter is here represented as $u^n(\boldsymbol{x})$ and is an approximation to $u_{\mathrm{e}}(\boldsymbol{x}, t_n)$. Then we have another approximation $u^{n,k}(\boldsymbol{x})$ to $u^n(\boldsymbol{x})$ when solving the nonlinear PDE problem for u^n by iteration methods, as in (5.34).

In our notation, u is a synonym for $u^{n,k+1}$ and $u^{(1)}$ is a synonym for u^{n-1}, inspired by what are natural variable names in a code. We will usually state the PDE problem in terms of u and quickly redefine the symbol u to mean the numerical approximation, while u_{e} is not explicitly introduced unless we need to talk about the exact solution and the approximate solution at the same time.

5.3.3 Backward Euler scheme and Newton's method

At time level n, we have to solve the stationary PDE (5.33). In the previous section, we saw how this can be done with Picard iterations. Another alternative is to apply the idea of Newton's method in a clever way. Normally, Newton's method is defined for systems of *algebraic*

equations, but the idea of the method can be applied at the PDE level too.

Linearization via Taylor expansions. Let $u^{n,k}$ be an approximation to the unknown u^n. We seek a better approximation on the form

$$u^n = u^{n,k} + \delta u\,. \tag{5.36}$$

The idea is to insert (5.36) in (5.33), Taylor expand the nonlinearities and keep only the terms that are linear in δu (which makes (5.36) an approximation for u^n). Then we can solve a linear PDE for the correction δu and use (5.36) to find a new approximation

$$u^{n,k+1} = u^{n,k} + \delta u$$

to u^n. Repeating this procedure gives a sequence $u^{n,k+1}$, $k = 0, 1, \ldots$ that hopefully converges to the goal u^n.

Let us carry out all the mathematical details for the nonlinear diffusion PDE discretized by the Backward Euler method. Inserting (5.36) in (5.33) gives

$$\frac{u^{n,k} + \delta u - u^{n-1}}{\Delta t} = \nabla\cdot(\alpha(u^{n,k}+\delta u)\nabla(u^{n,k}+\delta u)) + f(u^{n,k}+\delta u)\,. \tag{5.37}$$

We can Taylor expand $\alpha(u^{n,k} + \delta u)$ and $f(u^{n,k} + \delta u)$:

$$\alpha(u^{n,k} + \delta u) = \alpha(u^{n,k}) + \frac{d\alpha}{du}(u^{n,k})\delta u + \mathcal{O}(\delta u^2) \approx \alpha(u^{n,k}) + \alpha'(u^{n,k})\delta u,$$
$$f(u^{n,k} + \delta u) = f(u^{n,k}) + \frac{df}{du}(u^{n,k})\delta u + \mathcal{O}(\delta u^2) \approx f(u^{n,k}) + f'(u^{n,k})\delta u\,.$$

Inserting the linear approximations of α and f in (5.37) results in

$$\begin{aligned}\frac{u^{n,k} + \delta u - u^{n-1}}{\Delta t} = {} & \nabla\cdot(\alpha(u^{n,k})\nabla u^{n,k}) + f(u^{n,k}) + \\ & \nabla\cdot(\alpha(u^{n,k})\nabla\delta u) + \nabla\cdot(\alpha'(u^{n,k})\delta u\nabla u^{n,k}) + \\ & \nabla\cdot(\alpha'(u^{n,k})\delta u\nabla\delta u) + f'(u^{n,k})\delta u\,.\end{aligned} \tag{5.38}$$

The term $\alpha'(u^{n,k})\delta u\nabla\delta u$ is of order δu^2 and therefore omitted since we expect the correction δu to be small ($\delta u \gg \delta u^2$). Reorganizing the equation gives a PDE for δu that we can write in short form as

$$\delta F(\delta u; u^{n,k}) = -F(u^{n,k}),$$

where

$$F(u^{n,k}) = \frac{u^{n,k} - u^{n-1}}{\Delta t} - \nabla \cdot (\alpha(u^{n,k})\nabla u^{n,k}) + f(u^{n,k}), \tag{5.39}$$

$$\delta F(\delta u; u^{n,k}) = -\frac{1}{\Delta t}\delta u + \nabla \cdot (\alpha(u^{n,k})\nabla \delta u) + \nabla \cdot (\alpha'(u^{n,k})\delta u \nabla u^{n,k}) + f'(u^{n,k})\delta u\,. \tag{5.40}$$

Note that δF is a linear function of δu, and F contains only terms that are known, such that the PDE for δu is indeed linear.

Observations

The notational form $\delta F = -F$ resembles the Newton system $J\delta u = -F$ for systems of algebraic equations, with δF as $J\delta u$. The unknown vector in a linear system of algebraic equations enters the system as a linear operator in terms of a matrix-vector product ($J\delta u$), while at the PDE level we have a linear differential operator instead (δF).

Similarity with Picard iteration. We can rewrite the PDE for δu in a slightly different way too if we define $u^{n,k} + \delta u$ as $u^{n,k+1}$.

$$\frac{u^{n,k+1} - u^{n-1}}{\Delta t} = \nabla \cdot (\alpha(u^{n,k})\nabla u^{n,k+1}) + f(u^{n,k}) + \nabla \cdot (\alpha'(u^{n,k})\delta u \nabla u^{n,k}) + f'(u^{n,k})\delta u\,. \tag{5.41}$$

Note that the first line is the same PDE as arises in the Picard iteration, while the remaining terms arise from the differentiations that are an inherent ingredient in Newton's method.

Implementation. For coding we want to introduce u for u^n, u^- for $u^{n,k}$ and $u^{(1)}$ for u^{n-1}. The formulas for F and δF are then more clearly written as

$$F(u^-) = \frac{u^- - u^{(1)}}{\Delta t} - \nabla\cdot(\alpha(u^-)\nabla u^-) + f(u^-), \tag{5.42}$$

$$\delta F(\delta u; u^-) = -\frac{1}{\Delta t}\delta u + \nabla\cdot(\alpha(u^-)\nabla\delta u) + \nabla\cdot(\alpha'(u^-)\delta u\nabla u^-) + f'(u^-)\delta u\,. \tag{5.43}$$

The form that orders the PDE as the Picard iteration terms plus the Newton method's derivative terms becomes

$$\frac{u - u^{(1)}}{\Delta t} = \nabla\cdot(\alpha(u^-)\nabla u) + f(u^-) + \gamma(\nabla\cdot(\alpha'(u^-)(u-u^-)\nabla u^-) + f'(u^-)(u-u^-))\,. \tag{5.44}$$

The Picard and full Newton versions correspond to $\gamma = 0$ and $\gamma = 1$, respectively.

Derivation with alternative notation. Some may prefer to derive the linearized PDE for δu using the more compact notation. We start with inserting $u^n = u^- + \delta u$ to get

$$\frac{u^- + \delta u - u^{n-1}}{\Delta t} = \nabla\cdot(\alpha(u^- + \delta u)\nabla(u^- + \delta u)) + f(u^- + \delta u)\,.$$

Taylor expanding,

$$\alpha(u^- + \delta u) \approx \alpha(u^-) + \alpha'(u^-)\delta u,$$
$$f(u^- + \delta u) \approx f(u^-) + f'(u^-)\delta u,$$

and inserting these expressions gives a less cluttered PDE for δu:

$$\frac{u^- + \delta u - u^{n-1}}{\Delta t} = \nabla\cdot(\alpha(u^-)\nabla u^-) + f(u^-) + \nabla\cdot(\alpha(u^-)\nabla\delta u) + \nabla\cdot(\alpha'(u^-)\delta u\nabla u^-) + \nabla\cdot(\alpha'(u^-)\delta u\nabla\delta u) + f'(u^-)\delta u\,.$$

5.3.4 Crank-Nicolson discretization

A Crank-Nicolson discretization of (5.30) applies a centered difference at $t_{n+\frac{1}{2}}$:

$$[D_t u = \nabla \cdot (\alpha(u)\nabla u) + f(u)]^{n+\frac{1}{2}} .$$

The standard technique is to apply an arithmetic average for quantities defined between two mesh points, e.g.,

$$u^{n+\frac{1}{2}} \approx \frac{1}{2}(u^n + u^{n+1}) .$$

However, with nonlinear terms we have many choices of formulating an arithmetic mean:

$$[f(u)]^{n+\frac{1}{2}} \approx f(\frac{1}{2}(u^n + u^{n+1})) = [f(\overline{u}^t)]^{n+\frac{1}{2}}, \tag{5.45}$$

$$[f(u)]^{n+\frac{1}{2}} \approx \frac{1}{2}(f(u^n) + f(u^{n+1})) = [\overline{f(u)}^t]^{n+\frac{1}{2}}, \tag{5.46}$$

$$[\alpha(u)\nabla u]^{n+\frac{1}{2}} \approx \alpha(\frac{1}{2}(u^n + u^{n+1}))\nabla(\frac{1}{2}(u^n + u^{n+1})) = [\alpha(\overline{u}^t)\nabla\overline{u}^t]^{n+\frac{1}{2}}, \tag{5.47}$$

$$[\alpha(u)\nabla u]^{n+\frac{1}{2}} \approx \frac{1}{2}(\alpha(u^n) + \alpha(u^{n+1}))\nabla(\frac{1}{2}(u^n + u^{n+1})) = [\overline{\alpha(u)}^t\nabla\overline{u}^t]^{n+\frac{1}{2}}, \tag{5.48}$$

$$[\alpha(u)\nabla u]^{n+\frac{1}{2}} \approx \frac{1}{2}(\alpha(u^n)\nabla u^n + \alpha(u^{n+1})\nabla u^{n+1}) = [\overline{\alpha(u)\nabla u}^t]^{n+\frac{1}{2}} . \tag{5.49}$$

A big question is whether there are significant differences in accuracy between taking the products of arithmetic means or taking the arithmetic mean of products. Exercise 5.6 investigates this question, and the answer is that the approximation is $\mathcal{O}(\Delta t^2)$ in both cases.

5.4 1D stationary nonlinear differential equations

Section 5.3 presented methods for linearizing time-discrete PDEs directly prior to discretization in space. We can alternatively carry out the discretization in space of the time-discrete nonlinear PDE problem and get a system of nonlinear algebraic equations, which can be solved by Picard iteration or Newton's method as presented in Section 5.2. This latter approach will now be described in detail.

We shall work with the 1D problem

$$-(\alpha(u)u')' + au = f(u), \quad x \in (0,L), \quad \alpha(u(0))u'(0) = C,\ u(L) = D\,. \tag{5.50}$$

The problem (5.50) arises from the stationary limit of a diffusion equation,

$$\frac{\partial u}{\partial t} = \frac{\partial}{\partial x}\left(\alpha(u)\frac{\partial u}{\partial x}\right) - au + f(u), \tag{5.51}$$

as $t \to \infty$ and $\partial u/\partial t \to 0$. Alternatively, the problem (5.50) arises at each time level from implicit time discretization of (5.51). For example, a Backward Euler scheme for (5.51) leads to

$$\frac{u^n - u^{n-1}}{\Delta t} = \frac{d}{dx}\left(\alpha(u^n)\frac{du^n}{dx}\right) - au^n + f(u^n)\,. \tag{5.52}$$

Introducing $u(x)$ for $u^n(x)$, $u^{(1)}$ for u^{n-1}, and defining $f(u)$ in (5.50) to be $f(u)$ in (5.52) plus $u^{n-1}/\Delta t$, gives (5.50) with $a = 1/\Delta t$.

5.4.1 Finite difference discretization

The nonlinearity in the differential equation (5.50) poses no more difficulty than a variable coefficient, as in the term $(\alpha(x)u')'$. We can therefore use a standard finite difference approach when discretizing the Laplace term with a variable coefficient:

$$[-D_x\alpha D_x u + au = f]_i\,.$$

Writing this out for a uniform mesh with points $x_i = i\Delta x$, $i = 0, \ldots, N_x$, leads to

$$-\frac{1}{\Delta x^2}\left(\alpha_{i+\frac{1}{2}}(u_{i+1} - u_i) - \alpha_{i-\frac{1}{2}}(u_i - u_{i-1})\right) + au_i = f(u_i)\,. \tag{5.53}$$

This equation is valid at all the mesh points $i = 0, 1, \ldots, N_x - 1$. At $i = N_x$ we have the Dirichlet condition $u_i = 0$. The only difference from the case with $(\alpha(x)u')'$ and $f(x)$ is that now α and f are functions of u and not only of x: $(\alpha(u(x))u')'$ and $f(u(x))$.

The quantity $\alpha_{i+\frac{1}{2}}$, evaluated between two mesh points, needs a comment. Since α depends on u and u is only known at the mesh points, we need to express $\alpha_{i+\frac{1}{2}}$ in terms of u_i and u_{i+1}. For this purpose we use an arithmetic mean, although a harmonic mean is also common in this

context if α features large jumps. There are two choices of arithmetic means:

$$\alpha_{i+\frac{1}{2}} \approx \alpha(\frac{1}{2}(u_i + u_{i+1}) = [\alpha(\overline{u}^x)]^{i+\frac{1}{2}}, \tag{5.54}$$

$$\alpha_{i+\frac{1}{2}} \approx \frac{1}{2}(\alpha(u_i) + \alpha(u_{i+1})) = [\overline{\alpha(u)}^x]^{i+\frac{1}{2}} \tag{5.55}$$

Equation (5.53) with the latter approximation then looks like

$$-\frac{1}{2\Delta x^2}\left((\alpha(u_i) + \alpha(u_{i+1}))(u_{i+1} - u_i) - (\alpha(u_{i-1}) + \alpha(u_i))(u_i - u_{i-1})\right) + au_i = f(u_i), \tag{5.56}$$

or written more compactly,

$$[-D_x\overline{\alpha}^x D_x u + au = f]_i\,.$$

At mesh point $i = 0$ we have the boundary condition $\alpha(u)u' = C$, which is discretized by

$$[\alpha(u)D_{2x}u = C]_0,$$

meaning

$$\alpha(u_0)\frac{u_1 - u_{-1}}{2\Delta x} = C\,. \tag{5.57}$$

The fictitious value u_{-1} can be eliminated with the aid of (5.56) for $i = 0$. Formally, (5.56) should be solved with respect to u_{i-1} and that value (for $i = 0$) should be inserted in (5.57), but it is algebraically much easier to do it the other way around. Alternatively, one can use a ghost cell $[-\Delta x, 0]$ and update the u_{-1} value in the ghost cell according to (5.57) after every Picard or Newton iteration. Such an approach means that we use a known u_{-1} value in (5.56) from the previous iteration.

5.4.2 Solution of algebraic equations

The structure of the equation system. The nonlinear algebraic equations (5.56) are of the form $A(u)u = b(u)$ with

$$
\begin{aligned}
A_{i,i} &= \frac{1}{2\Delta x^2}(\alpha(u_{i-1}) + 2\alpha(u_i)\alpha(u_{i+1})) + a,\\
A_{i,i-1} &= -\frac{1}{2\Delta x^2}(\alpha(u_{i-1}) + \alpha(u_i)),\\
A_{i,i+1} &= -\frac{1}{2\Delta x^2}(\alpha(u_i) + \alpha(u_{i+1})),\\
b_i &= f(u_i)\,.
\end{aligned}
$$

The matrix $A(u)$ is tridiagonal: $A_{i,j} = 0$ for $j > i+1$ and $j < i-1$.

The above expressions are valid for internal mesh points $1 \leq i \leq N_x - 1$. For $i = 0$ we need to express $u_{i-1} = u_{-1}$ in terms of u_1 using (5.57):

$$
u_{-1} = u_1 - \frac{2\Delta x}{\alpha(u_0)}C\,. \tag{5.58}
$$

This value must be inserted in $A_{0,0}$. The expression for $A_{i,i+1}$ applies for $i = 0$, and $A_{i,i-1}$ does not enter the system when $i = 0$.

Regarding the last equation, its form depends on whether we include the Dirichlet condition $u(L) = D$, meaning $u_{N_x} = D$, in the nonlinear algebraic equation system or not. Suppose we choose $(u_0, u_1, \ldots, u_{N_x-1})$ as unknowns, later referred to as *systems without Dirichlet conditions*. The last equation corresponds to $i = N_x - 1$. It involves the boundary value u_{N_x}, which is substituted by D. If the unknown vector includes the boundary value, $(u_0, u_1, \ldots, u_{N_x})$, later referred to as *system including Dirichlet conditions*, the equation for $i = N_x - 1$ just involves the unknown u_{N_x}, and the final equation becomes $u_{N_x} = D$, corresponding to $A_{i,i} = 1$ and $b_i = D$ for $i = N_x$.

Picard iteration. The obvious Picard iteration scheme is to use previously computed values of u_i in $A(u)$ and $b(u)$, as described more in detail in Section 5.2. With the notation u^- for the most recently computed value of u, we have the system $F(u) \approx \hat{F}(u) = A(u^-)u - b(u^-)$, with $F = (F_0, F_1, \ldots, F_m)$, $u = (u_0, u_1, \ldots, u_m)$. The index m is N_x if the system includes the Dirichlet condition as a separate equation and $N_x - 1$ otherwise. The matrix $A(u^-)$ is tridiagonal, so the solution procedure is to fill a tridiagonal matrix data structure and the right-hand side vector with the right numbers and call a Gaussian elimination routine for tridiagonal linear systems.

Mesh with two cells. It helps on the understanding of the details to write out all the mathematics in a specific case with a small mesh, say just two cells ($N_x = 2$). We use u_i^- for the i-th component in u^-.

The starting point is the basic expressions for the nonlinear equations at mesh point $i = 0$ and $i = 1$:

$$A_{0,-1}u_{-1} + A_{0,0}u_0 + A_{0,1}u_1 = b_0, \tag{5.59}$$
$$A_{1,0}u_0 + A_{1,1}u_1 + A_{1,2}u_2 = b_1\,. \tag{5.60}$$

Equation (5.59) written out reads

$$\frac{1}{2\Delta x^2}(-(\alpha(u_{-1}) + \alpha(u_0))u_{-1} + (\alpha(u_{-1}) + 2\alpha(u_0) + \alpha(u_1))u_0 - (\alpha(u_0) + \alpha(u_1)))u_1 + au_0 = f(u_0)\,.$$

We must then replace u_{-1} by (5.58). With Picard iteration we get

$$\frac{1}{2\Delta x^2}(-(\alpha(u_{-1}^-) + 2\alpha(u_0^-) + \alpha(u_1^-))u_1 + (\alpha(u_{-1}^-) + 2\alpha(u_0^-) + \alpha(u_1^-))u_0 + au_0 = f(u_0^-) - \frac{1}{\alpha(u_0^-)\Delta x}(\alpha(u_{-1}^-) + \alpha(u_0^-))C,$$

where

$$u_{-1}^- = u_1^- - \frac{2\Delta x}{\alpha(u_0^-)}C\,.$$

Equation (5.60) contains the unknown u_2 for which we have a Dirichlet condition. In case we omit the condition as a separate equation, (5.60) with Picard iteration becomes

$$\frac{1}{2\Delta x^2}(-(\alpha(u_0^-) + \alpha(u_1^-))u_0 + (\alpha(u_0^-) + 2\alpha(u_1^-) + \alpha(u_2^-))u_1 - (\alpha(u_1^-) + \alpha(u_2^-)))u_2 + au_1 = f(u_1^-)\,.$$

We must now move the u_2 term to the right-hand side and replace all occurrences of u_2 by D:

$$\frac{1}{2\Delta x^2}(-(\alpha(u_0^-)+\alpha(u_1^-))u_0 + (\alpha(u_0^-)+2\alpha(u_1^-)+\alpha(D)))u_1 + au_1 = f(u_1^-) + \frac{1}{2\Delta x^2}(\alpha(u_1^-)+\alpha(D))D\,.$$

The two equations can be written as a 2×2 system:

$$\begin{pmatrix} B_{0,0} & B_{0,1} \\ B_{1,0} & B_{1,1} \end{pmatrix} \begin{pmatrix} u_0 \\ u_1 \end{pmatrix} = \begin{pmatrix} d_0 \\ d_1 \end{pmatrix},$$

where

$$B_{0,0} = \frac{1}{2\Delta x^2}(\alpha(u_{-1}^-)+2\alpha(u_0^-)+\alpha(u_1^-)) + a, \tag{5.61}$$

$$B_{0,1} = -\frac{1}{2\Delta x^2}(\alpha(u_{-1}^-)+2\alpha(u_0^-)+\alpha(u_1^-)), \tag{5.62}$$

$$B_{1,0} = -\frac{1}{2\Delta x^2}(\alpha(u_0^-)+\alpha(u_1^-)), \tag{5.63}$$

$$B_{1,1} = \frac{1}{2\Delta x^2}(\alpha(u_0^-)+2\alpha(u_1^-)+\alpha(D)) + a, \tag{5.64}$$

$$d_0 = f(u_0^-) - \frac{1}{\alpha(u_0^-)\Delta x}(\alpha(u_{-1}^-)+\alpha(u_0^-))C, \tag{5.65}$$

$$d_1 = f(u_1^-) + \frac{1}{2\Delta x^2}(\alpha(u_1^-)+\alpha(D))D\,. \tag{5.66}$$

The system with the Dirichlet condition becomes

$$\begin{pmatrix} B_{0,0} & B_{0,1} & 0 \\ B_{1,0} & B_{1,1} & B_{1,2} \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} u_0 \\ u_1 \\ u_2 \end{pmatrix} = \begin{pmatrix} d_0 \\ d_1 \\ D \end{pmatrix},$$

with

$$B_{1,1} = \frac{1}{2\Delta x^2}(\alpha(u_0^-)+2\alpha(u_1^-)+\alpha(u_2)) + a, \tag{5.67}$$

$$B_{1,2} = -\frac{1}{2\Delta x^2}(\alpha(u_1^-)+\alpha(u_2))), \tag{5.68}$$

$$d_1 = f(u_1^-)\,. \tag{5.69}$$

Other entries are as in the 2×2 system.

Newton's method. The Jacobian must be derived in order to use Newton's method. Here it means that we need to differentiate $F(u) = A(u)u - b(u)$ with respect to the unknown parameters $u_0, u_1, \ldots, u_m$ ($m = N_x$ or $m = N_x - 1$, depending on whether the Dirichlet condition is included in the nonlinear system $F(u) = 0$ or not). Nonlinear equation number i has the structure

$$F_i = A_{i,i-1}(u_{i-1}, u_i)u_{i-1} + A_{i,i}(u_{i-1}, u_i, u_{i+1})u_i + A_{i,i+1}(u_i, u_{i+1})u_{i+1} - b_i(u_i)\,.$$

Computing the Jacobian requires careful differentiation. For example,

$$\begin{aligned}
\frac{\partial}{\partial u_i}(A_{i,i}(u_{i-1}, u_i, u_{i+1})u_i) &= \frac{\partial A_{i,i}}{\partial u_i}u_i + A_{i,i}\frac{\partial u_i}{\partial u_i} \\
&= \frac{\partial}{\partial u_i}(\frac{1}{2\Delta x^2}(\alpha(u_{i-1}) + 2\alpha(u_i) + \alpha(u_{i+1})) + a)u_i + \\
&\quad \frac{1}{2\Delta x^2}(\alpha(u_{i-1}) + 2\alpha(u_i) + \alpha(u_{i+1})) + a \\
&= \frac{1}{2\Delta x^2}(2\alpha'(u_i)u_i + \alpha(u_{i-1}) + 2\alpha(u_i) + \alpha(u_{i+1})) + a\,.
\end{aligned}$$

The complete Jacobian becomes

$$\begin{aligned}
J_{i,i} &= \frac{\partial F_i}{\partial u_i} = \frac{\partial A_{i,i-1}}{\partial u_i}u_{i-1} + \frac{\partial A_{i,i}}{\partial u_i}u_i + A_{i,i} + \frac{\partial A_{i,i+1}}{\partial u_i}u_{i+1} - \frac{\partial b_i}{\partial u_i} \\
&= \frac{1}{2\Delta x^2}(-\alpha'(u_i)u_{i-1} + 2\alpha'(u_i)u_i + \alpha(u_{i-1}) + 2\alpha(u_i) + \alpha(u_{i+1})) + \\
&\quad a - \frac{1}{2\Delta x^2}\alpha'(u_i)u_{i+1} - b'(u_i), \\
J_{i,i-1} &= \frac{\partial F_i}{\partial u_{i-1}} = \frac{\partial A_{i,i-1}}{\partial u_{i-1}}u_{i-1} + A_{i-1,i} + \frac{\partial A_{i,i}}{\partial u_{i-1}}u_i - \frac{\partial b_i}{\partial u_{i-1}} \\
&= \frac{1}{2\Delta x^2}(-\alpha'(u_{i-1})u_{i-1} - (\alpha(u_{i-1}) + \alpha(u_i)) + \alpha'(u_{i-1})u_i), \\
J_{i,i+1} &= \frac{\partial A_{i,i+1}}{\partial u_{i-1}}u_{i+1} + A_{i+1,i} + \frac{\partial A_{i,i}}{\partial u_{i+1}}u_i - \frac{\partial b_i}{\partial u_{i+1}} \\
&= \frac{1}{2\Delta x^2}(-\alpha'(u_{i+1})u_{i+1} - (\alpha(u_i) + \alpha(u_{i+1})) + \alpha'(u_{i+1})u_i)\,.
\end{aligned}$$

The explicit expression for nonlinear equation number i, $F_i(u_0, u_1, \ldots)$, arises from moving the $f(u_i)$ term in (5.56) to the left-hand side:

$$F_i = -\frac{1}{2\Delta x^2}\left((\alpha(u_i)+\alpha(u_{i+1}))(u_{i+1}-u_i) - (\alpha(u_{i-1})+\alpha(u_i))(u_i-u_{i-1})\right) + au_i - f(u_i) = 0\,. \tag{5.70}$$

At the boundary point $i = 0$, u_{-1} must be replaced using the formula (5.58). When the Dirichlet condition at $i = N_x$ is not a part of the equation system, the last equation $F_m = 0$ for $m = N_x - 1$ involves the quantity u_{N_x-1} which must be replaced by D. If u_{N_x} is treated as an unknown in the system, the last equation $F_m = 0$ has $m = N_x$ and reads

$$F_{N_x}(u_0, \ldots, u_{N_x}) = u_{N_x} - D = 0\,.$$

Similar replacement of u_{-1} and u_{N_x} must be done in the Jacobian for the first and last row. When u_{N_x} is included as an unknown, the last row in the Jacobian must help implement the condition $\delta u_{N_x} = 0$, since we assume that u contains the right Dirichlet value at the beginning of the iteration ($u_{N_x} = D$), and then the Newton update should be zero for $i = 0$, i.e., $\delta u_{N_x} = 0$. This also forces the right-hand side to be $b_i = 0$, $i = N_x$.

We have seen, and can see from the present example, that the linear system in Newton's method contains all the terms present in the system that arises in the Picard iteration method. The extra terms in Newton's method can be multiplied by a factor such that it is easy to program one linear system and set this factor to 0 or 1 to generate the Picard or Newton system.

5.5 Multi-dimensional nonlinear PDE problems

The fundamental ideas in the derivation of F_i and $J_{i,j}$ in the 1D model problem are easily generalized to multi-dimensional problems. Nevertheless, the expressions involved are slightly different, with derivatives in x replaced by ∇, so we present some examples below in detail.

5.5.1 Finite difference discretization

A typical diffusion equation

$$u_t = \nabla \cdot (\alpha(u)\nabla u) + f(u),$$

can be discretized by (e.g.) a Backward Euler scheme, which in 2D can be written

$$[D_t^- u = D_x\overline{\alpha(u)}^x D_x u + D_y\overline{\alpha(u)}^y D_y u + f(u)]_{i,j}^n .$$

We do not dive into the details of handling boundary conditions now. Dirichlet and Neumann conditions are handled as in corresponding linear, variable-coefficient diffusion problems.

Writing the scheme out, putting the unknown values on the left-hand side and known values on the right-hand side, and introducing $\Delta x = \Delta y = h$ to save some writing, one gets

$$\begin{aligned} u_{i,j}^n - \frac{\Delta t}{h^2}(&\frac{1}{2}(\alpha(u_{i,j}^n) + \alpha(u_{i+1,j}^n))(u_{i+1,j}^n - u_{i,j}^n) \\ &- \frac{1}{2}(\alpha(u_{i-1,j}^n) + \alpha(u_{i,j}^n))(u_{i,j}^n - u_{i-1,j}^n) \\ &+ \frac{1}{2}(\alpha(u_{i,j}^n) + \alpha(u_{i,j+1}^n))(u_{i,j+1}^n - u_{i,j}^n) \\ &- \frac{1}{2}(\alpha(u_{i,j-1}^n) + \alpha(u_{i,j}^n))(u_{i,j}^n - u_{i-1,j-1}^n)) - \Delta t f(u_{i,j}^n) = u_{i,j}^{n-1} \end{aligned}$$

This defines a nonlinear algebraic system on the form $A(u)u = b(u)$.

Picard iteration. The most recently computed values u^- of u^n can be used in α and f for a Picard iteration, or equivalently, we solve $A(u^-)u = b(u^-)$. The result is a linear system of the same type as arising from $u_t = \nabla \cdot (\alpha(\boldsymbol{x})\nabla u) + f(\boldsymbol{x}, t)$.

The Picard iteration scheme can also be expressed in operator notation:

$$[D_t^- u = D_x\overline{\alpha(u^-)}^x D_x u + D_y\overline{\alpha(u^-)}^y D_y u + f(u^-)]_{i,j}^n .$$

Newton's method. As always, Newton's method is technically more involved than Picard iteration. We first define the nonlinear algebraic equations to be solved, drop the superscript n (use u for u^n), and introduce $u^{(1)}$ for u^{n-1}:

$$
\begin{aligned}
F_{i,j} = u_{i,j} - \frac{\Delta t}{h^2}(& \\
& \frac{1}{2}(\alpha(u_{i,j}) + \alpha(u_{i+1,j}))(u_{i+1,j} - u_{i,j}) - \\
& \frac{1}{2}(\alpha(u_{i-1,j}) + \alpha(u_{i,j}))(u_{i,j} - u_{i-1,j}) + \\
& \frac{1}{2}(\alpha(u_{i,j}) + \alpha(u_{i,j+1}))(u_{i,j+1} - u_{i,j}) - \\
& \frac{1}{2}(\alpha(u_{i,j-1}) + \alpha(u_{i,j}))(u_{i,j} - u_{i-1,j-1})) - \Delta t\, f(u_{i,j}) - u^{(1)}_{i,j} = 0\,.
\end{aligned}
$$

It is convenient to work with two indices i and j in 2D finite difference discretizations, but it complicates the derivation of the Jacobian, which then gets four indices. (Make sure you really understand the 1D version of this problem as treated in Section 5.4.1.) The left-hand expression of an equation $F_{i,j} = 0$ is to be differentiated with respect to each of the unknowns $u_{r,s}$ (recall that this is short notation for $u^n_{r,s}$), $r \in \mathcal{I}_x$, $s \in \mathcal{I}_y$:

$$
J_{i,j,r,s} = \frac{\partial F_{i,j}}{\partial u_{r,s}}\,.
$$

The Newton system to be solved in each iteration can be written as

$$
\sum_{r\in\mathcal{I}_x}\sum_{s\in\mathcal{I}_y} J_{i,j,r,s}\delta u_{r,s} = -F_{i,j}, \quad i \in \mathcal{I}_x,\ j \in \mathcal{I}_y\,.
$$

Given i and j, only a few r and s indices give nonzero contribution to the Jacobian since $F_{i,j}$ contains $u_{i\pm1,j}$, $u_{i,j\pm1}$, and $u_{i,j}$. This means that $J_{i,j,r,s}$ has nonzero contributions only if $r = i \pm 1$, $s = j \pm 1$, as well as $r = i$ and $s = j$. The corresponding terms in $J_{i,j,r,s}$ are $J_{i,j,i-1,j}$, $J_{i,j,i+1,j}$, $J_{i,j,i,j-1}$, $J_{i,j,i,j+1}$ and $J_{i,j,i,j}$. Therefore, the left-hand side of the Newton system, $\sum_r \sum_s J_{i,j,r,s}\delta u_{r,s}$ collapses to

$$
\begin{aligned}
J_{i,j,r,s}\delta u_{r,s} = J_{i,j,i,j}\delta u_{i,j} + J_{i,j,i-1,j}\delta u_{i-1,j} + J_{i,j,i+1,j}\delta u_{i+1,j} + J_{i,j,i,j-1}\delta u_{i,j-1} \\
+ J_{i,j,i,j+1}\delta u_{i,j+1}
\end{aligned}
$$

The specific derivatives become

$$
\begin{aligned}
J_{i,j,i-1,j} &= \frac{\partial F_{i,j}}{\partial u_{i-1,j}} \\
&= \frac{\Delta t}{h^2}(\alpha'(u_{i-1,j})(u_{i,j} - u_{i-1,j}) + \alpha(u_{i-1,j})(-1)), \\
J_{i,j,i+1,j} &= \frac{\partial F_{i,j}}{\partial u_{i+1,j}} \\
&= \frac{\Delta t}{h^2}(-\alpha'(u_{i+1,j})(u_{i+1,j} - u_{i,j}) - \alpha(u_{i-1,j})), \\
J_{i,j,i,j-1} &= \frac{\partial F_{i,j}}{\partial u_{i,j-1}} \\
&= \frac{\Delta t}{h^2}(\alpha'(u_{i,j-1})(u_{i,j} - u_{i,j-1}) + \alpha(u_{i,j-1})(-1)), \\
J_{i,j,i,j+1} &= \frac{\partial F_{i,j}}{\partial u_{i,j+1}} \\
&= \frac{\Delta t}{h^2}(-\alpha'(u_{i,j+1})(u_{i,j+1} - u_{i,j}) - \alpha(u_{i,j-1}))\,.
\end{aligned}
$$

The $J_{i,j,i,j}$ entry has a few more terms and is left as an exercise. Inserting the most recent approximation u^- for u in the J and F formulas and then forming $J\delta u = -F$ gives the linear system to be solved in each Newton iteration. Boundary conditions will affect the formulas when any of the indices coincide with a boundary value of an index.

5.5.2 Continuation methods

Picard iteration or Newton's method may diverge when solving PDEs with severe nonlinearities. Relaxation with $\omega < 1$ may help, but in highly nonlinear problems it can be necessary to introduce a *continuation parameter* Λ in the problem: $\Lambda = 0$ gives a version of the problem that is easy to solve, while $\Lambda = 1$ is the target problem. The idea is then to increase Λ in steps, $\Lambda_0 = 0, \Lambda_1 < \cdots < \Lambda_n = 1$, and use the solution from the problem with Λ_{i-1} as initial guess for the iterations in the problem corresponding to Λ_i.

The continuation method is easiest to understand through an example. Suppose we intend to solve

$$-\nabla \cdot (||\nabla u||^q \nabla u) = f,$$

which is an equation modeling the flow of a non-Newtonian fluid through a channel or pipe. For $q = 0$ we have the Poisson equation (corresponding to

a Newtonian fluid) and the problem is linear. A typical value for pseudo-plastic fluids may be $q_n = -0.8$. We can introduce the continuation parameter $\Lambda \in [0,1]$ such that $q = q_n\Lambda$. Let $\{\Lambda_\ell\}_{\ell=0}^n$ be the sequence of Λ values in $[0,1]$, with corresponding q values $\{q_\ell\}_{\ell=0}^n$. We can then solve a sequence of problems

$$-\nabla\cdot\left(||\nabla u^\ell||_\ell^q\nabla u^\ell\right) = f, \quad \ell = 0,\ldots,n,$$

where the initial guess for iterating on u^ℓ is the previously computed solution $u^{\ell-1}$. If a particular Λ_ℓ leads to convergence problems, one may try a smaller increase in Λ: $\Lambda_* = \frac{1}{2}(\Lambda_{\ell-1} + \Lambda_\ell)$, and repeat halving the step in Λ until convergence is reestablished.

5.6 Operator splitting methods

Operator splitting is a natural and old idea. When a PDE or system of PDEs contains different terms expressing different physics, it is natural to use different numerical methods for different physical processes. This can optimize and simplify the overall solution process. The idea was especially popularized in the context of the Navier-Stokes equations and reaction-diffusion PDEs. Common names for the technique are *operator splitting*, *fractional step* methods, and *split-step* methods. We shall stick to the former name. In the context of nonlinear differential equations, operator splitting can be used to isolate nonlinear terms and simplify the solution methods.

A related technique, often known as dimensional splitting or alternating direction implicit (ADI) methods, is to split the spatial dimensions and solve a 2D or 3D problem as two or three consecutive 1D problems, but this type of splitting is not to be further considered here.

5.6.1 Ordinary operator splitting for ODEs

Consider first an ODE where the right-hand side is split into two terms:

$$u' = f_0(u) + f_1(u)\,. \tag{5.71}$$

In case f_0 and f_1 are linear functions of u, $f_0 = au$ and $f_1 = bu$, we have $u(t) = Ie^{(a+b)t}$, if $u(0) = I$. When going one time step of length Δt from t_n to t_{n+1}, we have

$$u(t_{n+1}) = u(t_n)e^{(a+b)\Delta t}\,.$$

This expression can be also be written as

$$u(t_{n+1}) = u(t_n)e^{a\Delta t}e^{b\Delta t},$$

or

$$u^* = u(t_n)e^{a\Delta t}, \tag{5.72}$$

$$u(t_{n+1}) = u^* e^{b\Delta t} \tag{5.73}$$

The first step (5.72) means solving $u' = f_0$ over a time interval Δt with $u(t_n)$ as start value. The second step (5.73) means solving $u' = f_1$ over a time interval Δt with the value at the end of the first step as start value. That is, we progress the solution in two steps and solve two ODEs $u' = f_0$ and $u' = f_1$. The order of the equations is not important. From the derivation above we see that solving $u' = f_1$ prior to $u' = f_0$ can equally well be done.

The technique is exact if the ODEs are linear. For nonlinear ODEs it is only an approximate method with error Δt. The technique can be extended to an arbitrary number of steps; i.e., we may split the PDE system into any number of subsystems. Examples will illuminate this principle.

5.6.2 Strang splitting for ODEs

The accuracy of the splitting method in Section 5.6.1 can be improved from $\mathcal{O}(\Delta t)$ to $\mathcal{O}(\Delta t^2)$ using so-called *Strang splitting*, where we take half a step with the f_0 operator, a full step with the f_1 operator, and finally half another step with the f_0 operator. During a time interval Δt the algorithm can be written as follows.

$$\begin{aligned}
\frac{du^*}{dt} &= f_0(u^*), \quad u^*(t_n) = u(t_n), \quad t \in [t_n, t_n + \frac{1}{2}\Delta t],\\
\frac{du^{***}}{dt} &= f_1(u^{***}), \quad u^{***}(t_n) = u^*(t_{n+\frac{1}{2}}), \quad t \in [t_n, t_n + \Delta t],\\
\frac{du^{**}}{dt} &= f_0(u^{**}), \quad u^{**}(t_{n+\frac{1}{2}}) = u^{***}(t_{n+1}), \quad t \in [t_n + \frac{1}{2}\Delta t, t_n + \Delta t]\,.
\end{aligned}$$

The global solution is set as $u(t_{n+1}) = u^{**}(t_{n+1})$.

There is no use in combining higher-order methods with ordinary splitting since the error due to splitting is $\mathcal{O}(\Delta t)$, but for Strang splitting it makes sense to use schemes of order $\mathcal{O}(\Delta t^2)$.

With the notation introduced for Strang splitting, we may express ordinary first-order splitting as

$$\frac{du^*}{dt} = f_0(u^*), \quad u^*(t_n) = u(t_n), \quad t \in [t_n, t_n + \Delta t],$$
$$\frac{du^{**}}{dt} = f_1(u^{**}), \quad u^{**}(t_n) = u^*(t_{n+1}), \quad t \in [t_n, t_n + \Delta t],$$

with global solution set as $u(t_{n+1}) = u^{**}(t_{n+1})$.

5.6.3 Example: Logistic growth

Let us split the (scaled) logistic equation

$$u' = u(1-u), \quad u(0) = 0.1,$$

with solution $u = (9e^{-t} + 1)^{-1}$, into

$$u' = u - u^2 = f_0(u) + f_1(u), \quad f_0(u) = u, \quad f_1(u) = -u^2\,.$$

We solve $u' = f_0(u)$ and $u' = f_1(u)$ by a Forward Euler step. In addition, we add a method where we solve $u' = f_0(u)$ analytically, since the equation is actually $u' = u$ with solution e^t. The software that accompanies the following methods is the file `split_logistic.py`.

Splitting techniques. Ordinary splitting takes a Forward Euler step for each of the ODEs according to

$$\frac{u^{*,n+1} - u^{*,n}}{\Delta t} = f_0(u^{*,n}), \quad u^{*,n} = u(t_n), \quad t \in [t_n, t_n + \Delta t], \tag{5.74}$$
$$\frac{u^{**,n+1} - u^{**,n}}{\Delta t} = f_1(u^{**,n}), \quad u^{**,n} = u^{*,n+1}, \quad t \in [t_n, t_n + \Delta t], \tag{5.75}$$

with $u(t_{n+1}) = u^{**,n+1}$.

Strang splitting takes the form

$$\frac{u^{*,n+\frac{1}{2}} - u^{*,n}}{\frac{1}{2}\Delta t} = f_0(u^{*,n}), \quad u^{*,n} = u(t_n), \quad t \in [t_n, t_n + \frac{1}{2}\Delta t], \tag{5.76}$$

$$\frac{u^{***,n+1} - u^{***,n}}{\Delta t} = f_1(u^{***,n}), \quad u^{***,n} = u^{*,n+\frac{1}{2}}, \quad t \in [t_n, t_n + \Delta t], \tag{5.77}$$

$$\frac{u^{**,n+1} - u^{**,n+\frac{1}{2}}}{\frac{1}{2}\Delta t} = f_0(u^{**,n+\frac{1}{2}}), \quad u^{**,n+\frac{1}{2}} = u^{***,n+1}, \quad t \in [t_n + \frac{1}{2}\Delta t, t_n + \Delta t]\,. \tag{5.78}$$

Verbose implementation. The following function computes four solutions arising from the Forward Euler method, ordinary splitting, Strang splitting, as well as Strang splitting with exact treatment of $u' = f_0(u)$:

```
import numpy as np

def solver(dt, T, f, f_0, f_1):
    """
    Solve u'=f by the Forward Euler method and by ordinary and
    Strang splitting: f(u) = f_0(u) + f_1(u).
    """
    Nt = int(round(T/float(dt)))
    t = np.linspace(0, Nt*dt, Nt+1)
    u_FE = np.zeros(len(t))
    u_split1 = np.zeros(len(t))  # 1st-order splitting
    u_split2 = np.zeros(len(t))  # 2nd-order splitting
    u_split3 = np.zeros(len(t))  # 2nd-order splitting w/exact f_0

    # Set initial values
    u_FE[0] = 0.1
    u_split1[0] = 0.1
    u_split2[0] = 0.1
    u_split3[0] = 0.1

    for n in range(len(t)-1):
        # Forward Euler method
        u_FE[n+1] = u_FE[n] + dt*f(u_FE[n])

        # --- Ordinary splitting ---
        # First step
        u_s_n = u_split1[n]
        u_s = u_s_n + dt*f_0(u_s_n)
        # Second step
        u_ss_n = u_s
        u_ss = u_ss_n + dt*f_1(u_ss_n)
        u_split1[n+1] = u_ss
```

```
        # --- Strang splitting ---
        # First step
        u_s_n = u_split2[n]
        u_s = u_s_n + dt/2.*f_0(u_s_n)
        # Second step
        u_sss_n = u_s
        u_sss = u_sss_n + dt*f_1(u_sss_n)
        # Third step
        u_ss_n = u_sss
        u_ss = u_ss_n + dt/2.*f_0(u_ss_n)
        u_split2[n+1] = u_ss

        # --- Strang splitting using exact integrator for u'=f_0 ---
        # First step
        u_s_n = u_split3[n]
        u_s = u_s_n*np.exp(dt/2.)  # exact
        # Second step
        u_sss_n = u_s
        u_sss = u_sss_n + dt*f_1(u_sss_n)
        # Third step
        u_ss_n = u_sss
        u_ss = u_ss_n*np.exp(dt/2.)  # exact
        u_split3[n+1] = u_ss

    return u_FE, u_split1, u_split2, u_split3, t
```

Compact implementation. We have used quite many lines for the steps in the splitting methods. Many will prefer to condense the code a bit, as done here:

```
# Ordinary splitting
u_s = u_split1[n] + dt*f_0(u_split1[n])
u_split1[n+1] = u_s + dt*f_1(u_s)
# Strang splitting
u_s = u_split2[n] + dt/2.*f_0(u_split2[n])
u_sss = u_s + dt*f_1(u_s)
u_split2[n+1] = u_sss + dt/2.*f_0(u_sss)
# Strang splitting using exact integrator for u'=f_0
u_s = u_split3[n]*np.exp(dt/2.)  # exact
u_ss = u_s + dt*f_1(u_s)
u_split3[n+1] = u_ss*np.exp(dt/2.)
```

Results. Figure 5.3 shows that the impact of splitting is significant. Interestingly, however, the Forward Euler method applied to the entire problem directly is much more accurate than any of the splitting schemes. We also see that Strang splitting is definitely more accurate than ordinary splitting and that it helps a bit to use an exact solution of $u' = f_0(u)$. With a large time step ($\Delta t = 0.2$, left plot in Figure 5.3), the asymptotic values are off by 20-30%. A more reasonable time step ($\Delta t = 0.05$, right

plot in Figure 5.3) gives better results, but still the asymptotic values are up to 10% wrong.

As technique for solving nonlinear ODEs, we realize that the present case study is not particularly promising, as the Forward Euler method both linearizes the original problem and provides a solution that is much more accurate than any of the splitting techniques. In complicated multi-physics settings, on the other hand, splitting may be the only feasible way to go, and sometimes you really need to apply different numerics to different parts of a PDE problem. But in very simple problems, like the logistic ODE, splitting is just an inferior technique. Still, the logistic ODE is ideal for introducing all the mathematical details and for investigating the behavior.

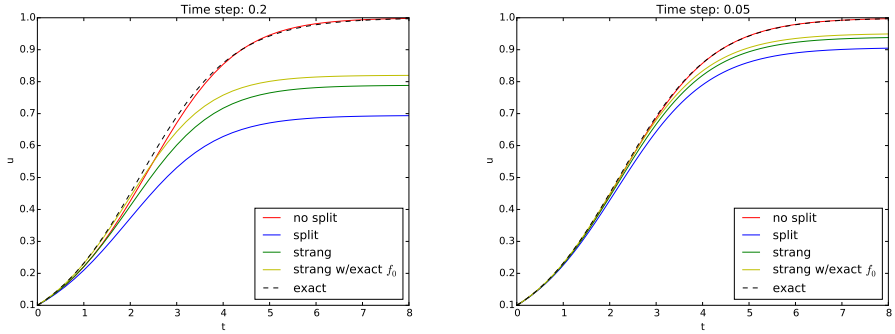


Fig. 5.3 Effect of ordinary and Strang splitting for the logistic equation.

5.6.4 Reaction-diffusion equation

Consider a diffusion equation coupled to chemical reactions modeled by a nonlinear term $f(u)$:

$$\frac{\partial u}{\partial t} = \alpha \nabla^2 u + f(u)\,.$$

This is a physical process composed of two individual processes: u is the concentration of a substance that is locally generated by a chemical reaction $f(u)$, while u is spreading in space because of diffusion. There are obviously two time scales: one for the chemical reaction and one for diffusion. Typically, fast chemical reactions require much finer time stepping than slower diffusion processes. It could therefore be advantageous

to split the two physical effects in separate models and use different numerical methods for the two.

A natural spitting in the present case is

$$\frac{\partial u^*}{\partial t} = \alpha\nabla^2 u^*, \tag{5.79}$$

$$\frac{\partial u^{**}}{\partial t} = f(u^{**})\,. \tag{5.80}$$

Looking at these familiar problems, we may apply a θ rule (implicit) scheme for (5.79) over one time step and avoid dealing with nonlinearities by applying an explicit scheme for (5.80) over the same time step.

Suppose we have some solution u at time level t_n. For flexibility, we define a θ method for the diffusion part (5.79) by

$$[D_t u^* = \alpha(D_x D_x u^* + D_y D_y u^*)]^{n+\theta}\,.$$

We use u^n as initial condition for u^*.

The reaction part, which is defined at each mesh point (without coupling values in different mesh points), can employ any scheme for an ODE. Here we use an Adams-Bashforth method of order 2. Recall that the overall accuracy of the splitting method is maximum $\mathcal{O}(\Delta t^2)$ for Strang splitting, otherwise it is just $\mathcal{O}(\Delta t)$. Higher-order methods for ODEs will therefore be a waste of work. The 2nd-order Adams-Bashforth method reads

$$u_{i,j}^{**,n+1} = u_{i,j}^{**,n} + \frac{1}{2}\Delta t\left(3f(u_{i,j}^{**,n}, t_n) - f(u_{i,j}^{**,n-1}, t_{n-1})\right)\,. \tag{5.81}$$

We can use a Forward Euler step to start the method, i.e, compute $u_{i,j}^{**,1}$.

The algorithm goes like this:

1. Solve the diffusion problem for one time step as usual.
2. Solve the reaction ODEs at each mesh point in $[t_n, t_n + \Delta t]$, using the diffusion solution in 1. as initial condition. The solution of the ODEs constitutes the solution of the original problem at the end of each time step.

We may use a much smaller time step when solving the reaction part, adapted to the dynamics of the problem $u' = f(u)$. This gives great flexibility in splitting methods.

5.6.5 Example: Reaction-Diffusion with linear reaction term

The methods above may be explored in detail through a specific computational example in which we compute the convergence rates associated with four different solution approaches for the reaction-diffusion equation with a linear reaction term, i.e. $f(u) = -bu$. The methods comprise solving without splitting (just straight Forward Euler), ordinary splitting, first order Strang splitting, and second order Strang splitting. In all four methods, a standard centered difference approximation is used for the spatial second derivative. The methods share the error model $E = Ch^r$, while differing in the step h (being either Δx^2 or Δx) and the convergence rate r (being either 1 or 2).

All code commented below is found in the file `split_diffu_react.py`. When executed, a function `convergence_rates` is called, from which all convergence rate computations are handled:

```
def convergence_rates(scheme='diffusion'):

    F = 0.5     # Upper limit for FE (stability). For CN, this
                # limit does not apply, but for simplicity, we
                # choose F = 0.5 as the initial F value.
    T = 1.2
    a = 3.5
    b = 1
    L = 1.5
    k = np.pi/L

    def exact(x, t):
        '''exact sol. to: du/dt = a*d^2u/dx^2 - b*u'''
        return np.exp(-(a*k**2 + b)*t) * np.sin(k*x)

    def f(u, t):
        return -b*u

    def I(x):
        return exact(x, 0)

    global error    # error computed in the user action function
    error = 0

    # Convergence study
    def action(u, x, t, n):
        global error
        if n == 1:      # New simulation, - reset error
            error = 0
        else:
            error = max(error, np.abs(u - exact(x, t[n])).max())

    E = []
```

```
    h = []
    Nx_values = [10, 20, 40, 80]   # i.e., dx halved each time
    for Nx in Nx_values:
        dx = L/Nx
        if scheme == 'Strang_splitting_2ndOrder':
            print 'Strang splitting with 2nd order schemes...'
            # In this case, E = C*h**r (with r = 2) and since
            # h = dx = K*dt, the ratio dt/dx must be constant.
            # To fulfill this demand, we must let F change
            # when dx changes. From F = a*dt/dx**2, it follows
            # that halving dx AND doubling F assures dt/dx const.
            # Initially, we simply choose F = 0.5.

            dt = F/a*dx**2
            #print 'dt/dx:', dt/dx
            Nt = int(round(T/float(dt)))
            t = np.linspace(0, Nt*dt, Nt+1)   # global time
            Strang_splitting_2ndOrder(I=I, a=a, b=b, f=f, L=L, dt=dt,
                                      dt_Rfactor=1, F=F, t=t, T=T,
                                      user_action=action)
            h.append(dx)
            # prepare for next iteration (make F match dx/2)
            F = F*2       # assures dt/dx const. when dx = dx/2
        else:
            # In these cases, E = C*h**r (with r = 1) and since
            # h = dx**2 = K*dt, the ratio dt/dx**2 must be constant.
            # This is fulfilled by choosing F = 0.5 (for FE stability)
            # and make sure that F, dx and dt comply to F = a*dt/dx**2.
            dt = F/a*dx**2
            Nt = int(round(T/float(dt)))
            t = np.linspace(0, Nt*dt, Nt+1)   # global time
            if scheme == 'diffusion':
                print 'FE on whole eqn...'
                diffusion_theta(I, a, f, L, dt, F, t, T,
                                step_no=0, theta=0,
                                u_L=0, u_R=0, user_action=action)
                h.append(dx**2)
            elif scheme == 'ordinary_splitting':
                print 'Ordinary splitting...'
                ordinary_splitting(I=I, a=a, b=b, f=f, L=L, dt=dt,
                                   dt_Rfactor=1, F=F, t=t, T=T,
                                   user_action=action)
                h.append(dx**2)
            elif scheme == 'Strang_splitting_1stOrder':
                print 'Strang splitting with 1st order schemes...'
                Strang_splitting_1stOrder(I=I, a=a, b=b, f=f, L=L, dt=dt,
                                          dt_Rfactor=1, F=F, t=t, T=T,
                                          user_action=action)
                h.append(dx**2)
            else:
                print 'Unknown scheme requested!'
                sys.exit(0)

            #print 'dt/dx**2:', dt/dx**2
```

```
        E.append(error)
        Nx *= 2         # Nx doubled gives dx/2

    print 'E:', E
    print 'h:', h

    # Convergence rates
    r = [np.log(E[i]/E[i-1])/np.log(h[i]/h[i-1])
         for i in range(1,len(Nx_values))]
    print 'Computed rates:', r


if __name__ == '__main__':

    schemes = ['diffusion',
               'ordinary_splitting',
               'Strang_splitting_1stOrder',
               'Strang_splitting_2ndOrder']

    for scheme in schemes:
        convergence_rates(scheme=scheme)
```

Now, with respect to the error ($E = Ch^r$), the Forward Euler scheme, the ordinary splitting scheme and first order Strang splitting scheme are all first order ($r = 1$), with a step $h = \Delta x^2 = K^{-1}\Delta t$, where K is some constant. This implies that the *ratio* $\frac{\Delta t}{\Delta x^2}$ must be held constant during convergence rate calculations. Furthermore, the Fourier number $F = \frac{\alpha \Delta t}{\Delta x^2}$ is upwards limited to $F = 0.5$, being the stability limit with explicit schemes. Thus, in these cases, we use the fixed value of F and a given (but changing) spatial resolution Δx to compute the corresponding value of Δt according to the expression for F. This assures that $\frac{\Delta t}{\Delta x^2}$ is kept constant. The loop in `convergence_rates` runs over a chosen set of grid points (`Nx_values`) which gives a doubling of spatial resolution with each iteration (Δx is halved).

For the second order Strang splitting scheme, we have $r = 2$ and a step $h = \Delta x = K^{-1}\Delta t$, where K again is some constant. In this case, it is thus the ratio $\frac{\Delta t}{\Delta x}$ that must be held constant during the convergence rate calculations. From the expression for F, it is clear then that F must change with each halving of Δx. In fact, if F is doubled each time Δx is halved, the ratio $\frac{\Delta t}{\Delta x}$ will be constant (this follows, e.g., from the expression for F). This is utilized in our code.

A solver `diffusion_theta` is used in each of the four solution approaches:

```
def diffusion_theta(I, a, f, L, dt, F, t, T, step_no, theta=0.5,
                    u_L=0, u_R=0, user_action=None):
```

```
    """
    Full solver for the model problem using the theta-rule
    difference approximation in time (no restriction on F,
    i.e., the time step when theta >= 0.5).     Vectorized
    implementation and sparse (tridiagonal) coefficient matrix.
    Note that t always covers the whole global time interval, whether
    splitting is the case or not. T, on the other hand, is
    the end of the global time interval if there is no split,
    but if splitting, we use T=dt. When splitting, step_no
    keeps track of the time step number (for lookup in t).
    """

    Nt = int(round(T/float(dt)))
    dx = np.sqrt(a*dt/F)
    Nx = int(round(L/dx))
    x = np.linspace(0, L, Nx+1)       # Mesh points in space
    # Make sure dx and dt are compatible with x and t
    dx = x[1] - x[0]
    dt = t[1] - t[0]

    u   = np.zeros(Nx+1)   # solution array at t[n+1]
    u_1 = np.zeros(Nx+1)   # solution at t[n]

    # Representation of sparse matrix and right-hand side
    diagonal = np.zeros(Nx+1)
    lower    = np.zeros(Nx)
    upper    = np.zeros(Nx)
    b        = np.zeros(Nx+1)

    # Precompute sparse matrix (scipy format)
    Fl = F*theta
    Fr = F*(1-theta)
    diagonal[:] = 1 + 2*Fl
    lower[:] = -Fl  #1
    upper[:] = -Fl  #1
    # Insert boundary conditions
    diagonal[0] = 1
    upper[0] = 0
    diagonal[Nx] = 1
    lower[-1] = 0

    diags = [0, -1, 1]
    A = scipy.sparse.diags(
        diagonals=[diagonal, lower, upper],
        offsets=[0, -1, 1], shape=(Nx+1, Nx+1),
        format='csr')
    #print A.todense()

    # Allow f to be None or 0
    if f is None or f == 0:
        f = lambda x, t: np.zeros((x.size)) \
            if isinstance(x, np.ndarray) else 0

    # Set initial condition
```

```
    if isinstance(I, np.ndarray):   # I is an array
        u_1 = np.copy(I)
    else:                           # I is a function
        for i in range(0, Nx+1):
            u_1[i] = I(x[i])

    if user_action is not None:
        user_action(u_1, x, t, step_no+0)

    # Time loop
    for n in range(0, Nt):
        b[1:-1] = u_1[1:-1] + \
                  Fr*(u_1[:-2] - 2*u_1[1:-1] + u_1[2:]) + \
                  dt*theta*f(u_1[1:-1], t[step_no+n+1]) + \
                  dt*(1-theta)*f(u_1[1:-1], t[step_no+n])
        b[0] = u_L; b[-1] = u_R  # boundary conditions
        u[:] = scipy.sparse.linalg.spsolve(A, b)

        if user_action is not None:
            user_action(u, x, t, step_no+(n+1))

        # Update u_1 before next step
        u_1, u = u, u_1

    # u is now contained in u_1 (swapping)
    return u_1
```

For the no splitting approach with Forward Euler in time, this solver handles both the diffusion and the reaction term. When splitting, `diffusion_theta` takes care of the diffusion term only, while the reaction term is handled either by a Forward Euler scheme in `reaction_FE`, or by a second order Adams-Bashforth scheme from Odespy. The `reaction_FE` function covers one complete time step `dt` during ordinary splitting, while Strang splitting (both first and second order) applies it with `dt/2` twice during each time step `dt`. Since the reaction term typically represents a much faster process than the diffusion term, a further refinement of the time step is made possible in `reaction_FE`. It was implemented as

```
def reaction_FE(I, f, L, Nx, dt, dt_Rfactor, t, step_no,
                user_action=None):
    """Reaction solver, Forward Euler method.
    Note the at t covers the whole global time interval.
    dt is either one complete,or one half, of the step in the
    diffusion part, i.e. there is a local time interval
    [0, dt] or [0, dt/2] that the reaction_FE
    deals with each time it is called. step_no keeps
    track of the (global) time step number (required
    for lookup in t).
    """

    u = np.copy(I)
```

```
    dt_local = dt/float(dt_Rfactor)
    Nt_local = int(round(dt/float(dt_local)))
    x = np.linspace(0, L, Nx+1)

    for n in range(Nt_local):
        time = t[step_no] + n*dt_local
        u[1:Nx] = u[1:Nx] + dt_local*f(u[1:Nx], time)

    # BC already inserted in diffusion step, i.e. no action here
    return u
```

With the ordinary splitting approach, each time step `dt` is covered twice. First computing the impact of the reaction term, then the contribution from the diffusion term:

```
def ordinary_splitting(I, a, b, f, L, dt,
                       dt_Rfactor, F, t, T,
                       user_action=None):
    '''1st order scheme, i.e. Forward Euler is enough for both
    the diffusion and the reaction part. The time step dt is
    given for the diffusion step, while the time step for the
    reaction part is found as dt/dt_Rfactor, where dt_Rfactor >= 1.
    '''
    Nt = int(round(T/float(dt)))
    dx = np.sqrt(a*dt/F)
    Nx = int(round(L/dx))
    x = np.linspace(0, L, Nx+1)       # Mesh points in space
    u = np.zeros(Nx+1)

    # Set initial condition u(x,0) = I(x)
    for i in range(0, Nx+1):
        u[i] = I(x[i])

    # In the following loop, each time step is "covered twice",
    # first for reaction, then for diffusion
    for n in range(0, Nt):
        # Reaction step (potentially many smaller steps within dt)
        u_s = reaction_FE(I=u, f=f, L=L, Nx=Nx,
                          dt=dt, dt_Rfactor=dt_Rfactor,
                          t=t, step_no=n,
                          user_action=None)

        u = diffusion_theta(I=u_s, a=a, f=0, L=L, dt=dt, F=F,
                            t=t, T=dt, step_no=n, theta=0,
                            u_L=0, u_R=0, user_action=None)

        if user_action is not None:
            user_action(u, x, t, n+1)

    return
```

For the two Strang splitting approaches, each time step `dt` is handled by first computing the reaction step for (the first) `dt/2`, followed by

a diffusion step `dt`, before the reaction step is treated once again for (the remaining) `dt/2`. Since first order Strang splitting is no better than first order accurate, both the reaction and diffusion steps are computed explicitly. The solver was implemented as

```
def Strang_splitting_1stOrder(I, a, b, f, L, dt, dt_Rfactor,
                              F, t, T, user_action=None):
    '''Strang splitting while still using FE for the reaction
    step and for the diffusion step. Gives 1st order scheme.
    The time step dt is given for the diffusion step, while
    the time step for the reaction part is found as
    0.5*dt/dt_Rfactor, where dt_Rfactor >= 1. Introduce an
    extra time mesh t2 for the reaction part, since it steps dt/2.
    '''
    Nt = int(round(T/float(dt)))
    t2 = np.linspace(0, Nt*dt, (Nt+1)+Nt)   # Mesh points in diff
    dx = np.sqrt(a*dt/F)
    Nx = int(round(L/dx))
    x = np.linspace(0, L, Nx+1)
    u = np.zeros(Nx+1)

    # Set initial condition u(x,0) = I(x)
    for i in range(0, Nx+1):
        u[i] = I(x[i])

    for n in range(0, Nt):
        # Reaction step (1/2 dt: from t_n to t_n+1/2)
        # (potentially many smaller steps within dt/2)
        u_s = reaction_FE(I=u, f=f, L=L, Nx=Nx,
                          dt=dt/2.0, dt_Rfactor=dt_Rfactor,
                          t=t2, step_no=2*n,
                          user_action=None)
        # Diffusion step (1 dt: from t_n to t_n+1)
        u_sss = diffusion_theta(I=u_s, a=a, f=0, L=L, dt=dt, F=F,
                                t=t, T=dt, step_no=n, theta=0,
                                u_L=0, u_R=0, user_action=None)
        # Reaction step (1/2 dt: from t_n+1/2 to t_n+1)
        # (potentially many smaller steps within dt/2)
        u = reaction_FE(I=u_sss, f=f, L=L, Nx=Nx,
                          dt=dt/2.0, dt_Rfactor=dt_Rfactor,
                          t=t2, step_no=2*n+1,
                          user_action=None)

        if user_action is not None:
            user_action(u, x, t, n+1)

    return
```

The second order version of the Strang splitting approach utilizes a second order Adams-Bashforth solver for the reaction part and a Crank-Nicolson scheme for the diffusion part. The solver has the same structure as the one for first order Strang splitting and was implemented as

```
def Strang_splitting_2ndOrder(I, a, b, f, L, dt, dt_Rfactor,
                              F, t, T, user_action=None):
    '''Strang splitting using Crank-Nicolson for the diffusion
    step (theta-rule) and Adams-Bashforth 2 for the reaction step.
    Gives 2nd order scheme. Introduce an extra time mesh t2 for
    the reaction part, since it steps dt/2.
    '''
    import odespy
    Nt = int(round(T/float(dt)))
    t2 = np.linspace(0, Nt*dt, (Nt+1)+Nt)   # Mesh points in diff
    dx = np.sqrt(a*dt/F)
    Nx = int(round(L/dx))
    x = np.linspace(0, L, Nx+1)
    u = np.zeros(Nx+1)

    # Set initial condition u(x,0) = I(x)
    for i in range(0, Nx+1):
        u[i] = I(x[i])

    reaction_solver = odespy.AdamsBashforth2(f)

    for n in range(0, Nt):
        # Reaction step (1/2 dt: from t_n to t_n+1/2)
        # (potentially many smaller steps within dt/2)
        reaction_solver.set_initial_condition(u)
        t_points = np.linspace(0, dt/2.0, dt_Rfactor+1)
        u_AB2, t_ = reaction_solver.solve(t_points) # t_ not needed
        u_s = u_AB2[-1,:]  # pick sol at last point in time

        # Diffusion step (1 dt: from t_n to t_n+1)
        u_sss = diffusion_theta(I=u_s, a=a, f=0, L=L, dt=dt, F=F,
                                t=t, T=dt, step_no=n, theta=0.5,
                                u_L=0, u_R=0, user_action=None)
        # Reaction step (1/2 dt: from t_n+1/2 to t_n+1)
        # (potentially many smaller steps within dt/2)
        reaction_solver.set_initial_condition(u_sss)
        t_points = np.linspace(0, dt/2.0, dt_Rfactor+1)
        u_AB2, t_ = reaction_solver.solve(t_points) # t_ not needed
        u = u_AB2[-1,:]  # pick sol at last point in time

        if user_action is not None:
            user_action(u, x, t, n+1)

    return
```

When executing `split_diffu_react.py`, we find that the estimated convergence rates are as expected. The second order Strang splitting gives the least error (about $4e^{-5}$) and has second order convergence ($r = 2$), while the remaining three approaches have first order convergence ($r = 1$).

5.6.6 Analysis of the splitting method

Let us address a linear PDE problem for which we can develop analytical solutions of the discrete equations, with and without splitting, and discuss these. Choosing $f(u) = -\beta u$ for a constant β gives a linear problem. We use the Forward Euler method for both the PDE and ODE problems.

We seek a 1D Fourier wave component solution of the problem, assuming homogeneous Dirichlet conditions at $x = 0$ and $x = L$:

$$u = e^{-\alpha k^2 t - \beta t} \sin kx, \quad k = \frac{\pi}{L}.$$

This component fits the 1D PDE problem ($f = 0$). On complex form we can write

$$u = e^{-\alpha k^2 t - \beta t + ikx},$$

where $i = \sqrt{-1}$ and the imaginary part is taken as the physical solution.

We refer to Section 3.3 and to the book [9] for a discussion of exact numerical solutions to diffusion and decay problems, respectively. The key idea is to search for solutions $A^n e^{ikx}$ and determine A. For the diffusion problem solved by a Forward Euler method one has

$$A = 1 - 4F \sin^p,$$

where $F = \alpha \Delta t / \Delta x^2$ is the mesh Fourier number and $p = k\Delta x/2$ is a dimensionless number reflecting the spatial resolution (number of points per wave length in space). For the decay problem $u' = -\beta u$, we have $A = 1 - q$, where q is a dimensionless parameter for the resolution in the decay problem: $q = \beta \Delta t$.

The original model problem can also be discretized by a Forward Euler scheme,

$$[D_t^+ u = \alpha D_x D_x u - \beta u]_i^n .$$

Assuming $A^n e^{ikx}$ we find that

$$u_i^n = (1 - 4F \sin^p - q)^n \sin kx .$$

We are particularly interested in what happens at one time step. That is,

$$u_i^n = (1 - 4F \sin^2 p) u_i^{n-1} .$$

In the two stage algorithm, we first compute the diffusion step

$$u_i^{*,n+1} = (1 - 4F\sin^2 p)u_i^{n-1}\,.$$

Then we use this as input to the decay algorithm and arrive at

$$u^{**,n+1} = (1-q)u^{*,n+1} = (1-q)(1 - 4F\sin^2 p)u_i^{n-1}\,.$$

The splitting approximation over one step is therefore

$$E = 1 - 4F\sin^p - q - (1-q)(1 - 4F\sin^2 p) = -q(2 - F\sin^2 p))\,.$$

5.7 Exercises

Problem 5.1: Determine if equations are nonlinear or not

Classify each term in the following equations as linear or nonlinear. Assume that u, $\boldsymbol{u}$, and p are unknown functions and that all other symbols are known quantities.

1. $mu'' + \beta|u'|u' + cu = F(t)$
2. $u_t = \alpha u_{xx}$
3. $u_{tt} = c^2\nabla^2 u$
4. $u_t = \nabla\cdot(\alpha(u)\nabla u) + f(x,y)$
5. $u_t + f(u)_x = 0$
6. $\boldsymbol{u}_t + \boldsymbol{u}\cdot\nabla\boldsymbol{u} = -\nabla p + r\nabla^2\boldsymbol{u}$, $\nabla\cdot\boldsymbol{u} = 0$ ($\boldsymbol{u}$ is a vector field)
7. $u' = f(u,t)$
8. $\nabla^2 u = \lambda e^u$

Filename: `nonlinear_vs_linear`.

Problem 5.2: Derive and investigate a generalized logistic model

The logistic model for population growth is derived by assuming a nonlinear growth rate,

$$u' = a(u)u, \quad u(0) = I, \tag{5.82}$$

and the logistic model arises from the simplest possible choice of $a(u)$: $r(u) = \varrho(1 - u/M)$, where M is the maximum value of u that the

environment can sustain, and ϱ is the growth under unlimited access to resources (as in the beginning when u is small). The idea is that $a(u) \sim \varrho$ when u is small and that $a(t) \to 0$ as $u \to M$.

An $a(u)$ that generalizes the linear choice is the polynomial form

$$a(u) = \varrho(1 - u/M)^p, \tag{5.83}$$

where $p > 0$ is some real number.

a) Formulate a Forward Euler, Backward Euler, and a Crank-Nicolson scheme for (5.82).

Hint. Use a geometric mean approximation in the Crank-Nicolson scheme: $[a(u)u]^{n+1/2} \approx a(u^n)u^{n+1}$.

b) Formulate Picard and Newton iteration for the Backward Euler scheme in a).

c) Implement the numerical solution methods from a) and b). Use `logistic.py` to compare the case $p = 1$ and the choice (5.83).

d) Implement unit tests that check the asymptotic limit of the solutions: $u \to M$ as $t \to \infty$.

Hint. You need to experiment to find what "infinite time" is (increases substantially with p) and what the appropriate tolerance is for testing the asymptotic limit.

e) Perform experiments with Newton and Picard iteration for the model (5.83). See how sensitive the number of iterations is to Δt and p.
Filename: `logistic_p`.

Problem 5.3: Experience the behavior of Newton's method

The program `Newton_demo.py` illustrates graphically each step in Newton's method and is run like

Terminal

```
Terminal> python Newton_demo.py f dfdx x0 xmin xmax
```

Use this program to investigate potential problems with Newton's method when solving $e^{-0.5x^2}\cos(\pi x) = 0$. Try a starting point $x_0 = 0.8$ and $x_0 = 0.85$ and watch the different behavior. Just run

Terminal

```
Terminal>  python Newton_demo.py '0.2 + exp(-0.5*x**2)*cos(pi*x)' \
           '-x*exp(-x**2)*cos(pi*x) - pi*exp(-x**2)*sin(pi*x)' \
           0.85 -3 3
```

and repeat with 0.85 replaced by 0.8.

Exercise 5.4: Compute the Jacobian of a 2×2 system

Write up the system (5.18)-(5.19) in the form $F(u) = 0$, $F = (F_0, F_1)$, $u = (u_0, u_1)$, and compute the Jacobian $J_{i,j} = \partial F_i/\partial u_j$.

Problem 5.5: Solve nonlinear equations arising from a vibration ODE

Consider a nonlinear vibration problem

$$mu'' + bu'|u'| + s(u) = F(t), \tag{5.84}$$

where $m > 0$ is a constant, $b \geq 0$ is a constant, $s(u)$ a possibly nonlinear function of u, and $F(t)$ is a prescribed function. Such models arise from Newton's second law of motion in mechanical vibration problems where $s(u)$ is a spring or restoring force, mu'' is mass times acceleration, and $bu'|u'|$ models water or air drag.

a) Rewrite the equation for u as a system of two first-order ODEs, and discretize this system by a Crank-Nicolson (centered difference) method. With $v = u'$, we get a nonlinear term $v^{n+\frac{1}{2}}|v^{n+\frac{1}{2}}|$. Use a geometric average for $v^{n+\frac{1}{2}}$.

b) Formulate a Picard iteration method to solve the system of nonlinear algebraic equations.

c) Explain how to apply Newton's method to solve the nonlinear equations at each time level. Derive expressions for the Jacobian and the right-hand side in each Newton iteration.
Filename: `nonlin_vib`.

Exercise 5.6: Find the truncation error of arithmetic mean of products

In Section 5.3.4 we introduce alternative arithmetic means of a product. Say the product is $P(t)Q(t)$ evaluated at $t = t_{n+\frac{1}{2}}$. The exact value is

$$[PQ]^{n+\frac{1}{2}} = P^{n+\frac{1}{2}}Q^{n+\frac{1}{2}}$$

There are two obvious candidates for evaluating $[PQ]^{n+\frac{1}{2}}$ as a mean of values of P and Q at t_n and t_{n+1}. Either we can take the arithmetic mean of each factor P and Q,

$$[PQ]^{n+\frac{1}{2}} \approx \frac{1}{2}(P^n + P^{n+1})\frac{1}{2}(Q^n + Q^{n+1}), \tag{5.85}$$

or we can take the arithmetic mean of the product PQ:

$$[PQ]^{n+\frac{1}{2}} \approx \frac{1}{2}(P^nQ^n + P^{n+1}Q^{n+1})\,. \tag{5.86}$$

The arithmetic average of $P(t_{n+\frac{1}{2}})$ is $\mathcal{O}(\Delta t^2)$:

$$P(t_{n+\frac{1}{2}}) = \frac{1}{2}(P^n + P^{n+1}) + \mathcal{O}(\Delta t^2)\,.$$

A fundamental question is whether (5.85) and (5.86) have different orders of accuracy in $\Delta t = t_{n+1} - t_n$. To investigate this question, expand quantities at t_{n+1} and t_n in Taylor series around $t_{n+\frac{1}{2}}$, and subtract the true value $[PQ]^{n+\frac{1}{2}}$ from the approximations (5.85) and (5.86) to see what the order of the error terms are.

Hint. You may explore `sympy` for carrying out the tedious calculations. A general Taylor series expansion of $P(t + \frac{1}{2}\Delta t)$ around t involving just a general function $P(t)$ can be created as follows:

```
>>> from sympy import *
>>> t, dt = symbols('t dt')
>>> P = symbols('P', cls=Function)
>>> P(t).series(t, 0, 4)
P(0) + t*Subs(Derivative(P(_x), _x), (_x,), (0,)) +
t**2*Subs(Derivative(P(_x), _x, _x), (_x,), (0,))/2 +
t**3*Subs(Derivative(P(_x), _x, _x, _x), (_x,), (0,))/6 + O(t**4)
>>> P_p = P(t).series(t, 0, 4).subs(t, dt/2)
>>> P_p
P(0) + dt*Subs(Derivative(P(_x), _x), (_x,), (0,))/2 +
dt**2*Subs(Derivative(P(_x), _x, _x), (_x,), (0,))/8 +
dt**3*Subs(Derivative(P(_x), _x, _x, _x), (_x,), (0,))/48 + O(dt**4)
```

The error of the arithmetic mean, $\frac{1}{2}(P(-\frac{1}{2}\Delta t) + P(-\frac{1}{2}\Delta t))$ for $t = 0$ is then

```
>>> P_m = P(t).series(t, 0, 4).subs(t, -dt/2)
>>> mean = Rational(1,2)*(P_m + P_p)
>>> error = simplify(expand(mean) - P(0))
>>> error
dt**2*Subs(Derivative(P(_x), _x, _x), (_x,), (0,))/8 + O(dt**4)
```

Use these examples to investigate the error of (5.85) and (5.86) for $n = 0$. (Choosing $n = 0$ is necessary for not making the expressions too complicated for `sympy`, but there is of course no lack of generality by using $n = 0$ rather than an arbitrary n - the main point is the product and addition of Taylor series.)
Filename: `product_arith_mean`.

Problem 5.7: Newton's method for linear problems

Suppose we have a linear system $F(u) = Au - b = 0$. Apply Newton's method to this system, and show that the method converges in one iteration. Filename: `Newton_linear`.

Problem 5.8: Discretize a 1D problem with a nonlinear coefficient

We consider the problem

$$((1 + u^2)u')' = 1, \quad x \in (0, 1), \quad u(0) = u(1) = 0\,. \tag{5.87}$$

Discretize (5.87) by a centered finite difference method on a uniform mesh. Filename: `nonlin_1D_coeff_discretize`.

Problem 5.9: Linearize a 1D problem with a nonlinear coefficient

We have a two-point boundary value problem

$$((1 + u^2)u')' = 1, \quad x \in (0, 1), \quad u(0) = u(1) = 0\,. \tag{5.88}$$

a) Construct a Picard iteration method for (5.88) without discretizing in space.

b) Apply Newton's method to (5.88) without discretizing in space.

c) Discretize (5.88) by a centered finite difference scheme. Construct a Picard method for the resulting system of nonlinear algebraic equations.

d) Discretize (5.88) by a centered finite difference scheme. Define the system of nonlinear algebraic equations, calculate the Jacobian, and set up Newton's method for solving the system.
Filename: `nonlin_1D_coeff_linearize`.

Problem 5.10: Finite differences for the 1D Bratu problem

We address the so-called Bratu problem

$$u'' + \lambda e^u = 0, \quad x \in (0,1), \quad u(0) = u(1) = 0, \tag{5.89}$$

where λ is a given parameter and u is a function of x. This is a widely used model problem for studying numerical methods for nonlinear differential equations. The problem (5.89) has an exact solution

$$u_{\mathrm{e}}(x) = -2\ln\left(\frac{\cosh((x-\frac{1}{2})\theta/2)}{\cosh(\theta/4)}\right),$$

where θ solves

$$\theta = \sqrt{2\lambda}\cosh(\theta/4)\,.$$

There are two solutions of (5.89) for $0 < \lambda < \lambda_c$ and no solution for $\lambda > \lambda_c$. For $\lambda = \lambda_c$ there is one unique solution. The critical value λ_c solves

$$1 = \sqrt{2\lambda_c}\frac{1}{4}\sinh(\theta(\lambda_c)/4)\,.$$

A numerical value is $\lambda_c = 3.513830719$.

a) Discretize (5.89) by a centered finite difference method.

b) Set up the nonlinear equations $F_i(u_0, u_1, \ldots, u_{N_x}) = 0$ from a). Calculate the associated Jacobian.

c) Implement a solver that can compute $u(x)$ using Newton's method. Plot the error as a function of x in each iteration.

d) Investigate whether Newton's method gives second-order convergence by computing $||u_\mathrm{e} - u||/||u_\mathrm{e} - u^-||^2$ in each iteration, where u is solution in the current iteration and u^- is the solution in the previous iteration.
Filename: `nonlin_1D_Bratu_fd`.

Problem 5.11: Discretize a nonlinear 1D heat conduction PDE by finite differences

We address the 1D heat conduction PDE

$$\varrho c(T)T_t = (k(T)T_x)_x,$$

for $x \in [0, L]$, where ϱ is the density of the solid material, $c(T)$ is the heat capacity, T is the temperature, and $k(T)$ is the heat conduction coefficient. $T(x, 0) = I(x)$, and ends are subject to a cooling law:

$$k(T)T_x|_{x=0} = h(T)(T - T_s), \quad -k(T)T_x|_{x=L} = h(T)(T - T_s),$$

where $h(T)$ is a heat transfer coefficient and T_s is the given surrounding temperature.

a) Discretize this PDE in time using either a Backward Euler or Crank-Nicolson scheme.

b) Formulate a Picard iteration method for the time-discrete problem (i.e., an iteration method before discretizing in space).

c) Formulate a Newton method for the time-discrete problem in b).

d) Discretize the PDE by a finite difference method in space. Derive the matrix and right-hand side of a Picard iteration method applied to the space-time discretized PDE.

e) Derive the matrix and right-hand side of a Newton method applied to the discretized PDE in d).
Filename: `nonlin_1D_heat_FD`.

Problem 5.12: Differentiate a highly nonlinear term

The operator $\nabla \cdot (\alpha(u)\nabla u)$ with $\alpha(u) = |\nabla u|^q$ appears in several physical problems, especially flow of Non-Newtonian fluids. The expression $|\nabla u|$ is defined as the Euclidean norm of a vector: $|\nabla u|^2 = \nabla u \cdot \nabla u$. In a

Newton method one has to carry out the differentiation $\partial\alpha(u)/\partial c_j$, for $u = \sum_k c_k\psi_k$. Show that

$$\frac{\partial}{\partial u_j}|\nabla u|^q = q|\nabla u|^{q-2}\nabla u\cdot\nabla\psi_j\,.$$

Filename: `nonlin_differentiate`.

Exercise 5.13: Crank-Nicolson for a nonlinear 3D diffusion equation

Redo Section 5.5.1 when a Crank-Nicolson scheme is used to discretize the equations in time and the problem is formulated for three spatial dimensions.

Hint. Express the Jacobian as $J_{i,j,k,r,s,t} = \partial F_{i,j,k}/\partial u_{r,s,t}$ and observe, as in the 2D case, that $J_{i,j,k,r,s,t}$ is very sparse: $J_{i,j,k,r,s,t} \neq 0$ only for $r = i \pm i$, $s = j \pm 1$, and $t = k \pm 1$ as well as $r = i$, $s = j$, and $t = k$.
Filename: `nonlin_heat_FD_CN_2D`.

Problem 5.14: Find the sparsity of the Jacobian

Consider a typical nonlinear Laplace term like $\nabla\cdot\alpha(u)\nabla u$ discretized by centered finite differences. Explain why the Jacobian corresponding to this term has the same sparsity pattern as the matrix associated with the corresponding linear term $\alpha\nabla^2 u$.

Hint. Set up the unknowns that enter the difference equation at a point (i,j) in 2D or (i,j,k) in 3D, and identify the nonzero entries of the Jacobian that can arise from such a type of difference equation.
Filename: `nonlin_sparsity_Jacobian`.

Problem 5.15: Investigate a 1D problem with a continuation method

Flow of a pseudo-plastic power-law fluid between two flat plates can be modeled by

$$\frac{d}{dx}\left(\mu_0\left|\frac{du}{dx}\right|^{n-1}\frac{du}{dx}\right) = -\beta,\quad u'(0) = 0,\ u(H) = 0,$$

where $\beta > 0$ and $\mu_0 > 0$ are constants. A target value of n may be $n = 0.2$.

a) Formulate a Picard iteration method directly for the differential equation problem.

b) Perform a finite difference discretization of the problem in each Picard iteration. Implement a solver that can compute u on a mesh. Verify that the solver gives an exact solution for $n = 1$ on a uniform mesh regardless of the cell size.

c) Given a sequence of decreasing n values, solve the problem for each n using the solution for the previous n as initial guess for the Picard iteration. This is called a continuation method. Experiment with $n = (1, 0.6, 0.2)$ and $n = (1, 0.9, 0.8, \ldots, 0.2)$ and make a table of the number of Picard iterations versus n.

d) Derive a Newton method at the differential equation level and discretize the resulting linear equations in each Newton iteration with the finite difference method.

e) Investigate if Newton's method has better convergence properties than Picard iteration, both in combination with a continuation method.

Useful formulas

A

A.1 Finite difference operator notation

© 2017, Hans Petter Langtangen, Svein Linge. Released under CC Attribution 4.0 license

$$u'(t_n) \approx [D_t u]^n = \frac{u^{n+\frac{1}{2}} - u^{n-\frac{1}{2}}}{\Delta t} \tag{A.1}$$

$$u'(t_n) \approx [D_{2t} u]^n = \frac{u^{n+1} - u^{n-1}}{2\Delta t} \tag{A.2}$$

$$u'(t_n) = [D_t^- u]^n = \frac{u^n - u^{n-1}}{\Delta t} \tag{A.3}$$

$$u'(t_n) \approx [D_t^+ u]^n = \frac{u^{n+1} - u^n}{\Delta t} \tag{A.4}$$

$$u'(t_{n+\theta}) = [\bar{D}_t u]^{n+\theta} = \frac{u^{n+1} - u^n}{\Delta t} \tag{A.5}$$

$$u'(t_n) \approx [D_t^{2-} u]^n = \frac{3u^n - 4u^{n-1} + u^{n-2}}{2\Delta t} \tag{A.6}$$

$$u''(t_n) \approx [D_t D_t u]^n = \frac{u^{n+1} - 2u^n + u^{n-1}}{\Delta t^2} \tag{A.7}$$

$$u(t_{n+\frac{1}{2}}) \approx [\overline{u}^t]^{n+\frac{1}{2}} = \frac{1}{2}(u^{n+1} + u^n) \tag{A.8}$$

$$u(t_{n+\frac{1}{2}})^2 \approx [\overline{u^2}^{t,g}]^{n+\frac{1}{2}} = u^{n+1}u^n \tag{A.9}$$

$$u(t_{n+\frac{1}{2}}) \approx [\overline{u}^{t,h}]^{n+\frac{1}{2}} = \frac{2}{\frac{1}{u^{n+1}} + \frac{1}{u^n}} \tag{A.10}$$

$$u(t_{n+\theta}) \approx [\overline{u}^{t,\theta}]^{n+\theta} = \theta u^{n+1} + (1-\theta)u^n, \tag{A.11}$$

$$t_{n+\theta} = \theta t_{n+1} + (1-\theta)t_{n-1} \tag{A.12}$$

Some may wonder why θ is absent on the right-hand side of (A.5). The fraction is an approximation to the derivative at the point $t_{n+\theta} = \theta t_{n+1} + (1-\theta)t_n$.

A.2 Truncation errors of finite difference approximations

$$u_{\mathrm{e}}'(t_n) = [D_t u_{\mathrm{e}}]^n + R^n = \frac{u_{\mathrm{e}}^{n+\frac{1}{2}} - u_{\mathrm{e}}^{n-\frac{1}{2}}}{\Delta t} + R^n,$$
$$R^n = -\frac{1}{24}u_{\mathrm{e}}'''(t_n)\Delta t^2 + \mathcal{O}(\Delta t^4) \tag{A.13}$$

$$u_{\mathrm{e}}'(t_n) = [D_{2t} u_{\mathrm{e}}]^n + R^n = \frac{u_{\mathrm{e}}^{n+1} - u_{\mathrm{e}}^{n-1}}{2\Delta t} + R^n,$$
$$R^n = -\frac{1}{6}u_{\mathrm{e}}'''(t_n)\Delta t^2 + \mathcal{O}(\Delta t^4) \tag{A.14}$$

$$u_{\mathrm{e}}'(t_n) = [D_t^- u_{\mathrm{e}}]^n + R^n = \frac{u_{\mathrm{e}}^{n} - u_{\mathrm{e}}^{n-1}}{\Delta t} + R^n,$$
$$R^n = -\frac{1}{2}u_{\mathrm{e}}''(t_n)\Delta t + \mathcal{O}(\Delta t^2) \tag{A.15}$$

$$u_{\mathrm{e}}'(t_n) = [D_t^+ u_{\mathrm{e}}]^n + R^n = \frac{u_{\mathrm{e}}^{n+1} - u_{\mathrm{e}}^{n}}{\Delta t} + R^n,$$
$$R^n = \frac{1}{2}u_{\mathrm{e}}''(t_n)\Delta t + \mathcal{O}(\Delta t^2) \tag{A.16}$$

$$u_{\mathrm{e}}'(t_{n+\theta}) = [\bar{D}_t u_{\mathrm{e}}]^{n+\theta} + R^{n+\theta} = \frac{u_{\mathrm{e}}^{n+1} - u_{\mathrm{e}}^{n}}{\Delta t} + R^{n+\theta},$$
$$R^{n+\theta} = -\frac{1}{2}(1-2\theta)u_{\mathrm{e}}''(t_{n+\theta})\Delta t + \frac{1}{6}((1-\theta)^3 - \theta^3)u_{\mathrm{e}}'''(t_{n+\theta})\Delta t^2 + \mathcal{O}(\Delta t^3) \tag{A.17}$$

$$u_{\mathrm{e}}'(t_n) = [D_t^{2-} u_{\mathrm{e}}]^n + R^n = \frac{3u_{\mathrm{e}}^{n} - 4u_{\mathrm{e}}^{n-1} + u_{\mathrm{e}}^{n-2}}{2\Delta t} + R^n,$$
$$R^n = \frac{1}{3}u_{\mathrm{e}}'''(t_n)\Delta t^2 + \mathcal{O}(\Delta t^3) \tag{A.18}$$

$$u_{\mathrm{e}}''(t_n) = [D_t D_t u_{\mathrm{e}}]^n + R^n = \frac{u_{\mathrm{e}}^{n+1} - 2u_{\mathrm{e}}^{n} + u_{\mathrm{e}}^{n-1}}{\Delta t^2} + R^n,$$
$$R^n = -\frac{1}{12}u_{\mathrm{e}}''''(t_n)\Delta t^2 + \mathcal{O}(\Delta t^4) \tag{A.19}$$

$$u_{\mathrm{e}}(t_{n+\theta}) = [\overline{u_{\mathrm{e}}}^{t,\theta}]^{n+\theta} + R^{n+\theta} = \theta u_{\mathrm{e}}^{n+1} + (1-\theta)u_{\mathrm{e}}^{n} + R^{n+\theta},$$
$$R^{n+\theta} = -\frac{1}{2}u_{\mathrm{e}}''(t_{n+\theta})\Delta t^2\theta(1-\theta) + \mathcal{O}(\Delta t^3)\,. \tag{A.20}$$

A.3 Finite differences of exponential functions

Complex exponentials. Let $u^n = \exp(i\omega n\Delta t) = e^{i\omega t_n}$.

$$[D_tD_tu]^n = u^n \frac{2}{\Delta t}(\cos\omega\Delta t - 1) = -\frac{4}{\Delta t}\sin^2\left(\frac{\omega\Delta t}{2}\right), \tag{A.21}$$

$$[D_t^+u]^n = u^n \frac{1}{\Delta t}(\exp(i\omega\Delta t) - 1), \tag{A.22}$$

$$[D_t^-u]^n = u^n \frac{1}{\Delta t}(1 - \exp(-i\omega\Delta t)), \tag{A.23}$$

$$[D_tu]^n = u^n \frac{2}{\Delta t} i\sin\left(\frac{\omega\Delta t}{2}\right), \tag{A.24}$$

$$[D_{2t}u]^n = u^n \frac{1}{\Delta t} i\sin(\omega\Delta t)\,. \tag{A.25}$$

Real exponentials. Let $u^n = \exp(\omega n\Delta t) = e^{\omega t_n}$.

$$[D_tD_tu]^n = u^n \frac{2}{\Delta t}(\cos\omega\Delta t - 1) = -\frac{4}{\Delta t}\sin^2\left(\frac{\omega\Delta t}{2}\right), \tag{A.26}$$

$$[D_t^+u]^n = u^n \frac{1}{\Delta t}(\exp(i\omega\Delta t) - 1), \tag{A.27}$$

$$[D_t^-u]^n = u^n \frac{1}{\Delta t}(1 - \exp(-i\omega\Delta t)), \tag{A.28}$$

$$[D_tu]^n = u^n \frac{2}{\Delta t} i\sin\left(\frac{\omega\Delta t}{2}\right), \tag{A.29}$$

$$[D_{2t}u]^n = u^n \frac{1}{\Delta t} i\sin(\omega\Delta t)\,. \tag{A.30}$$

A.4 Finite differences of t^n

The following results are useful when checking if a polynomial term in a solution fulfills the discrete equation for the numerical method.

$$[D_t^+t]^n = 1, \tag{A.31}$$

$$[D_t^-t]^n = 1, \tag{A.32}$$

$$[D_tt]^n = 1, \tag{A.33}$$

$$[D_{2t}t]^n = 1, \tag{A.34}$$

$$[D_tD_tt]^n = 0\,. \tag{A.35}$$

The next formulas concern the action of difference operators on a t^2 term.

$$[D_t^+ t^2]^n = (2n+1)\Delta t, \tag{A.36}$$
$$[D_t^- t^2]^n = (2n-1)\Delta t, \tag{A.37}$$
$$[D_t t^2]^n = 2n\Delta t, \tag{A.38}$$
$$[D_{2t} t^2]^n = 2n\Delta t, \tag{A.39}$$
$$[D_t D_t t^2]^n = 2, \tag{A.40}$$

Finally, we present formulas for a t^3 term:

$$[D_t^+ t^3]^n = 3(n\Delta t)^2 + 3n\Delta t^2 + \Delta t^2, \tag{A.41}$$
$$[D_t^- t^3]^n = 3(n\Delta t)^2 - 3n\Delta t^2 + \Delta t^2, \tag{A.42}$$
$$[D_t t^3]^n = 3(n\Delta t)^2 + \frac{1}{4}\Delta t^2, \tag{A.43}$$
$$[D_{2t} t^3]^n = 3(n\Delta t)^2 + \Delta t^2, \tag{A.44}$$
$$[D_t D_t t^3]^n = 6n\Delta t, \tag{A.45}$$

A.4.1 Software

Application of finite difference operators to polynomials and exponential functions, resulting in the formulas above, can easily be computed by some `sympy` code (from the file `lib.py`):

```
from sympy import *
t, dt, n, w = symbols('t dt n w', real=True)

# Finite difference operators

def D_t_forward(u):
    return (u(t + dt) - u(t))/dt

def D_t_backward(u):
    return (u(t) - u(t-dt))/dt

def D_t_centered(u):
    return (u(t + dt/2) - u(t-dt/2))/dt

def D_2t_centered(u):
    return (u(t + dt) - u(t-dt))/(2*dt)

def D_t_D_t(u):
    return (u(t + dt) - 2*u(t) + u(t-dt))/(dt**2)
```

```
op_list = [D_t_forward, D_t_backward,
           D_t_centered, D_2t_centered, D_t_D_t]

def ft1(t):
    return t

def ft2(t):
    return t**2

def ft3(t):
    return t**3

def f_expiwt(t):
    return exp(I*w*t)

def f_expwt(t):
    return exp(w*t)

func_list = [ft1, ft2, ft3, f_expiwt, f_expwt]
```

To see the results, one can now make a simple loop over the different types of functions and the various operators associated with them:

```
for func in func_list:
    for op in op_list:
        f = func
        e = op(f)
        e = simplify(expand(e))
        print e
        if func in [f_expiwt, f_expwt]:
            e = e/f(t)
        e = e.subs(t, n*dt)
        print expand(e)
        print factor(simplify(expand(e)))
```

Truncation error analysis

B

Truncation error analysis provides a widely applicable framework for analyzing the accuracy of finite difference schemes. This type of analysis can also be used for finite element and finite volume methods if the discrete equations are written in finite difference form. The result of the analysis is an asymptotic estimate of the error in the scheme on the form Ch^r, where h is a discretization parameter (Δt, Δx, etc.), r is a number, known as the convergence rate, and C is a constant, typically dependent on the derivatives of the exact solution.

Knowing r gives understanding of the accuracy of the scheme. But maybe even more important, a powerful verification method for computer codes is to check that the empirically observed convergence rates in experiments coincide with the theoretical value of r found from truncation error analysis.

The analysis can be carried out by hand, by symbolic software, and also numerically. All three methods will be illustrated. From examining the symbolic expressions of the truncation error we can add correction terms to the differential equations in order to increase the numerical accuracy.

In general, the term truncation error refers to the discrepancy that arises from performing a finite number of steps to approximate a process with infinitely many steps. The term is used in a number of contexts, including truncation of infinite series, finite precision arithmetic, finite differences, and differential equations. We shall be concerned with computing truncation errors arising in finite difference formulas and in finite difference discretizations of differential equations.

© 2017, Hans Petter Langtangen, Svein Linge. Released under CC Attribution 4.0 license

B.1 Overview of truncation error analysis

B.1.1 Abstract problem setting

Consider an abstract differential equation

$$\mathcal{L}(u) = 0,$$

where $\mathcal{L}(u)$ is some formula involving the unknown u and its derivatives. One example is $\mathcal{L}(u) = u'(t) + a(t)u(t) - b(t)$, where a and b are constants or functions of time. We can discretize the differential equation and obtain a corresponding discrete model, here written as

$$\mathcal{L}_\Delta(u) = 0\,.$$

The solution u of this equation is the *numerical solution*. To distinguish the numerical solution from the exact solution of the differential equation problem, we denote the latter by u_e and write the differential equation and its discrete counterpart as

$$\mathcal{L}(u_\mathrm{e}) = 0,$$
$$\mathcal{L}_\Delta(u) = 0\,.$$

Initial and/or boundary conditions can usually be left out of the truncation error analysis and are omitted in the following.

The numerical solution u is, in a finite difference method, computed at a collection of mesh points. The discrete equations represented by the abstract equation $\mathcal{L}_\Delta(u) = 0$ are usually algebraic equations involving u at some neighboring mesh points.

B.1.2 Error measures

A key issue is how accurate the numerical solution is. The ultimate way of addressing this issue would be to compute the error $u_\mathrm{e} - u$ at the mesh points. This is usually extremely demanding. In very simplified problem settings we may, however, manage to derive formulas for the numerical solution u, and therefore closed form expressions for the error $u_\mathrm{e} - u$. Such special cases can provide considerable insight regarding accuracy and stability, but the results are established for special problems.

The error $u_{\mathrm{e}} - u$ can be computed empirically in special cases where we know u_{e}. Such cases can be constructed by the method of manufactured solutions, where we choose some exact solution $u_{\mathrm{e}} = v$ and fit a source term f in the governing differential equation $\mathcal{L}(u_{\mathrm{e}}) = f$ such that $u_{\mathrm{e}} = v$ is a solution (i.e., $f = \mathcal{L}(v)$). Assuming an error model of the form Ch^r, where h is the discretization parameter, such as Δt or Δx, one can estimate the convergence rate r. This is a widely applicable procedure, but the validity of the results is, strictly speaking, tied to the chosen test problems.

Another error measure arises by asking to what extent the exact solution u_{e} fits the discrete equations. Clearly, u_{e} is in general not a solution of $\mathcal{L}_\Delta(u) = 0$, but we can define the residual

$$R = \mathcal{L}_\Delta(u_{\mathrm{e}}),$$

and investigate how close R is to zero. A small R means intuitively that the discrete equations are close to the differential equation, and then we are tempted to think that u^n must also be close to $u_{\mathrm{e}}(t_n)$.

The residual R is known as the truncation error of the finite difference scheme $\mathcal{L}_\Delta(u) = 0$. It appears that the truncation error is relatively straightforward to compute by hand or symbolic software *without specializing the differential equation and the discrete model to a special case*. The resulting R is found as a power series in the discretization parameters. The leading-order terms in the series provide an asymptotic measure of the accuracy of the numerical solution method (as the discretization parameters tend to zero). An advantage of truncation error analysis, compared to empirical estimation of convergence rates, or detailed analysis of a special problem with a mathematical expression for the numerical solution, is that the truncation error analysis reveals the accuracy of the various building blocks in the numerical method and how each building block impacts the overall accuracy. The analysis can therefore be used to detect building blocks with lower accuracy than the others.

Knowing the truncation error or other error measures is important for verification of programs by empirically establishing convergence rates. The forthcoming text will provide many examples on how to compute truncation errors for finite difference discretizations of ODEs and PDEs.

B.2 Truncation errors in finite difference formulas

The accuracy of a finite difference formula is a fundamental issue when discretizing differential equations. We shall first go through a particular example in detail and thereafter list the truncation error in the most common finite difference approximation formulas.

B.2.1 Example: The backward difference for $u'(t)$

Consider a backward finite difference approximation of the first-order derivative u':

$$[D_t^- u]^n = \frac{u^n - u^{n-1}}{\Delta t} \approx u'(t_n)\,. \tag{B.1}$$

Here, u^n means the value of some function $u(t)$ at a point t_n, and $[D_t^- u]^n$ is the *discrete derivative* of $u(t)$ at $t = t_n$. The discrete derivative computed by a finite difference is, in general, not exactly equal to the derivative $u'(t_n)$. The error in the approximation is

$$R^n = [D_t^- u]^n - u'(t_n)\,. \tag{B.2}$$

The common way of calculating R^n is to

1. expand $u(t)$ in a Taylor series around the point where the derivative is evaluated, here t_n,
2. insert this Taylor series in (B.2), and
3. collect terms that cancel and simplify the expression.

The result is an expression for R^n in terms of a power series in Δt. The error R^n is commonly referred to as the *truncation error* of the finite difference formula.

The Taylor series formula often found in calculus books takes the form

$$f(x+h) = \sum_{i=0}^{\infty} \frac{1}{i!}\frac{d^i f}{dx^i}(x)h^i\,.$$

In our application, we expand the Taylor series around the point where the finite difference formula approximates the derivative. The Taylor series of u^n at t_n is simply $u(t_n)$, while the Taylor series of u^{n-1} at t_n must employ the general formula,

$$u(t_{n-1}) = u(t - \Delta t) = \sum_{i=0}^{\infty} \frac{1}{i!} \frac{d^i u}{dt^i}(t_n)(-\Delta t)^i$$
$$= u(t_n) - u'(t_n)\Delta t + \frac{1}{2}u''(t_n)\Delta t^2 + \mathcal{O}(\Delta t^3),$$

where $\mathcal{O}(\Delta t^3)$ means a power-series in Δt where the lowest power is Δt^3. We assume that Δt is small such that $\Delta t^p \gg \Delta t^q$ if p is smaller than q. The details of higher-order terms in Δt are therefore not of much interest. Inserting the Taylor series above in the right-hand side of (B.2) gives rise to some algebra:

$$[D_t^- u]^n - u'(t_n) = \frac{u(t_n) - u(t_{n-1})}{\Delta t} - u'(t_n)$$
$$= \frac{u(t_n) - (u(t_n) - u'(t_n)\Delta t + \frac{1}{2}u''(t_n)\Delta t^2 + \mathcal{O}(\Delta t^3))}{\Delta t} - u'(t_n)$$
$$= -\frac{1}{2}u''(t_n)\Delta t + \mathcal{O}(\Delta t^2)),$$

which is, according to (B.2), the truncation error:

$$R^n = -\frac{1}{2}u''(t_n)\Delta t + \mathcal{O}(\Delta t^2))\,. \tag{B.3}$$

The dominating term for small Δt is $-\frac{1}{2}u''(t_n)\Delta t$, which is proportional to Δt, and we say that the truncation error is of *first order* in Δt.

B.2.2 Example: The forward difference for $u'(t)$

We can analyze the approximation error in the forward difference

$$u'(t_n) \approx [D_t^+ u]^n = \frac{u^{n+1} - u^n}{\Delta t},$$

by writing

$$R^n = [D_t^+ u]^n - u'(t_n),$$

and expanding u^{n+1} in a Taylor series around t_n,

$$u(t_{n+1}) = u(t_n) + u'(t_n)\Delta t + \frac{1}{2}u''(t_n)\Delta t^2 + \mathcal{O}(\Delta t^3)\,.$$

The result becomes

$$R = \frac{1}{2}u''(t_n)\Delta t + \mathcal{O}(\Delta t^2),$$

showing that also the forward difference is of first order.

B.2.3 Example: The central difference for $u'(t)$

For the central difference approximation,

$$u'(t_n) \approx [D_t u]^n, \quad [D_t u]^n = \frac{u^{n+\frac{1}{2}} - u^{n-\frac{1}{2}}}{\Delta t},$$

we write

$$R^n = [D_t u]^n - u'(t_n),$$

and expand $u(t_{n+\frac{1}{2}})$ and $u(t_{n-\frac{1}{2}}$ in Taylor series around the point t_n where the derivative is evaluated. We have

$$\begin{aligned}
u(t_{n+\frac{1}{2}}) =& u(t_n) + u'(t_n)\frac{1}{2}\Delta t + \frac{1}{2}u''(t_n)(\frac{1}{2}\Delta t)^2 + \\
& \frac{1}{6}u'''(t_n)(\frac{1}{2}\Delta t)^3 + \frac{1}{24}u''''(t_n)(\frac{1}{2}\Delta t)^4 + \\
& \frac{1}{120}u'''''(t_n)(\frac{1}{2}\Delta t)^5 + \mathcal{O}(\Delta t^6), \\
u(t_{n-\frac{1}{2}}) =& u(t_n) - u'(t_n)\frac{1}{2}\Delta t + \frac{1}{2}u''(t_n)(\frac{1}{2}\Delta t)^2 - \\
& \frac{1}{6}u'''(t_n)(\frac{1}{2}\Delta t)^3 + \frac{1}{24}u''''(t_n)(\frac{1}{2}\Delta t)^4 - \\
& \frac{1}{120}u'''''(t_n)(\frac{1}{2}\Delta t)^5 + \mathcal{O}(\Delta t^6)\,.
\end{aligned}$$

Now,

$$u(t_{n+\frac{1}{2}}) - u(t_{n-\frac{1}{2}}) = u'(t_n)\Delta t + \frac{1}{24}u'''(t_n)\Delta t^3 + \frac{1}{960}u'''''(t_n)\Delta t^5 + \mathcal{O}(\Delta t^7)\,.$$

By collecting terms in $[D_t u]^n - u'(t_n)$ we find the truncation error to be

$$R^n = \frac{1}{24}u'''(t_n)\Delta t^2 + \mathcal{O}(\Delta t^4), \tag{B.4}$$

with only even powers of Δt. Since $R \sim \Delta t^2$ we say the centered difference is of *second order* in Δt.

B.2.4 Overview of leading-order error terms in finite difference formulas

Here we list the leading-order terms of the truncation errors associated with several common finite difference formulas for the first and second derivatives.

$$[D_t u]^n = \frac{u^{n+\frac{1}{2}} - u^{n-\frac{1}{2}}}{\Delta t} = u'(t_n) + R^n, \tag{B.5}$$

$$R^n = \frac{1}{24} u'''(t_n)\Delta t^2 + \mathcal{O}(\Delta t^4) \tag{B.6}$$

$$[D_{2t} u]^n = \frac{u^{n+1} - u^{n-1}}{2\Delta t} = u'(t_n) + R^n, \tag{B.7}$$

$$R^n = \frac{1}{6} u'''(t_n)\Delta t^2 + \mathcal{O}(\Delta t^4) \tag{B.8}$$

$$[D_t^- u]^n = \frac{u^n - u^{n-1}}{\Delta t} = u'(t_n) + R^n, \tag{B.9}$$

$$R^n = -\frac{1}{2} u''(t_n)\Delta t + \mathcal{O}(\Delta t^2) \tag{B.10}$$

$$[D_t^+ u]^n = \frac{u^{n+1} - u^n}{\Delta t} = u'(t_n) + R^n, \tag{B.11}$$

$$R^n = \frac{1}{2} u''(t_n)\Delta t + \mathcal{O}(\Delta t^2) \tag{B.12}$$

$$[\bar{D}_t u]^{n+\theta} = \frac{u^{n+1} - u^n}{\Delta t} = u'(t_{n+\theta}) + R^{n+\theta}, \tag{B.13}$$

$$R^{n+\theta} = \frac{1}{2}(1-2\theta) u''(t_{n+\theta})\Delta t - \frac{1}{6}((1-\theta)^3 - \theta^3) u'''(t_{n+\theta})\Delta t^2 + \mathcal{O}(\Delta t^3) \tag{B.14}$$

$$[D_t^{2-} u]^n = \frac{3u^n - 4u^{n-1} + u^{n-2}}{2\Delta t} = u'(t_n) + R^n, \tag{B.15}$$

$$R^n = -\frac{1}{3} u'''(t_n)\Delta t^2 + \mathcal{O}(\Delta t^3) \tag{B.16}$$

$$[D_t D_t u]^n = \frac{u^{n+1} - 2u^n + u^{n-1}}{\Delta t^2} = u''(t_n) + R^n, \tag{B.17}$$

$$R^n = \frac{1}{12} u''''(t_n)\Delta t^2 + \mathcal{O}(\Delta t^4) \tag{B.18}$$

It will also be convenient to have the truncation errors for various means or averages. The weighted arithmetic mean leads to

$$[\overline{u}^{t,\theta}]^{n+\theta} = \theta u^{n+1} + (1-\theta)u^n = u(t_{n+\theta}) + R^{n+\theta}, \tag{B.19}$$

$$R^{n+\theta} = \frac{1}{2}u''(t_{n+\theta})\Delta t^2\theta(1-\theta) + \mathcal{O}(\Delta t^3)\,. \tag{B.20}$$

The standard arithmetic mean follows from this formula when $\theta = \frac{1}{2}$. Expressed at point t_n we get

$$[\overline{u}^{t}]^{n} = \frac{1}{2}(u^{n-\frac{1}{2}} + u^{n+\frac{1}{2}}) = u(t_n) + R^n, \tag{B.21}$$

$$R^n = \frac{1}{8}u''(t_n)\Delta t^2 + \frac{1}{384}u''''(t_n)\Delta t^4 + \mathcal{O}(\Delta t^6)\,. \tag{B.22}$$

The geometric mean also has an error $\mathcal{O}(\Delta t^2)$:

$$[\overline{u^2}^{t,g}]^{n} = u^{n-\frac{1}{2}}u^{n+\frac{1}{2}} = (u^n)^2 + R^n, \tag{B.23}$$

$$R^n = -\frac{1}{4}u'(t_n)^2\Delta t^2 + \frac{1}{4}u(t_n)u''(t_n)\Delta t^2 + \mathcal{O}(\Delta t^4)\,. \tag{B.24}$$

The harmonic mean is also second-order accurate:

$$[\overline{u}^{t,h}]^{n} = u^n = \frac{2}{\frac{1}{u^{n-\frac{1}{2}}} + \frac{1}{u^{n+\frac{1}{2}}}} + R^{n+\frac{1}{2}}, \tag{B.25}$$

$$R^n = -\frac{u'(t_n)^2}{4u(t_n)}\Delta t^2 + \frac{1}{8}u''(t_n)\Delta t^2\,. \tag{B.26}$$

B.2.5 Software for computing truncation errors

We can use `sympy` to aid calculations with Taylor series. The derivatives can be defined as symbols, say `D3f` for the 3rd derivative of some function f. A truncated Taylor series can then be written as `f + D1f*h + D2f*h**2/2`. The following class takes some symbol `f` for the function in question and makes a list of symbols for the derivatives. The `__call__` method computes the symbolic form of the series truncated at `num_terms` terms.

```
import sympy as sym

class TaylorSeries:
    """Class for symbolic Taylor series."""
    def __init__(self, f, num_terms=4):
```

```
        self.f = f
        self.N = num_terms
        # Introduce symbols for the derivatives
        self.df = [f]
        for i in range(1, self.N+1):
            self.df.append(sym.Symbol('D%d%s' % (i, f.name)))

    def __call__(self, h):
        """Return the truncated Taylor series at x+h."""
        terms = self.f
        for i in range(1, self.N+1):
            terms += sym.Rational(1, sym.factorial(i))*self.df[i]*h**i
        return terms
```

We may, for example, use this class to compute the truncation error of the Forward Euler finite difference formula:

```
>>> from truncation_errors import TaylorSeries
>>> from sympy import *
>>> u, dt = symbols('u dt')
>>> u_Taylor = TaylorSeries(u, 4)
>>> u_Taylor(dt)
D1u*dt + D2u*dt**2/2 + D3u*dt**3/6 + D4u*dt**4/24 + u
>>> FE = (u_Taylor(dt) - u)/dt
>>> FE
(D1u*dt + D2u*dt**2/2 + D3u*dt**3/6 + D4u*dt**4/24)/dt
>>> simplify(FE)
D1u + D2u*dt/2 + D3u*dt**2/6 + D4u*dt**3/24
```

The truncation error consists of the terms after the first one (u').

The module file `trunc/truncation_errors.py` contains another class `DiffOp` with symbolic expressions for most of the truncation errors listed in the previous section. For example:

```
>>> from truncation_errors import DiffOp
>>> from sympy import *
>>> u = Symbol('u')
>>> diffop = DiffOp(u, independent_variable='t')
>>> diffop['geometric_mean']
-D1u**2*dt**2/4 - D1u*D3u*dt**4/48 + D2u**2*dt**4/64 + ...
>>> diffop['Dtm']
D1u + D2u*dt/2 + D3u*dt**2/6 + D4u*dt**3/24
>>> >>> diffop.operator_names()
['geometric_mean', 'harmonic_mean', 'Dtm', 'D2t', 'DtDt',
 'weighted_arithmetic_mean', 'Dtp', 'Dt']
```

The indexing of `diffop` applies names that correspond to the operators: `Dtp` for D_t^+, `Dtm` for D_t^-, `Dt` for D_t, `D2t` for D_{2t}, `DtDt` for D_tD_t.

B.3 Exponential decay ODEs

We shall now compute the truncation error of a finite difference scheme for a differential equation. Our first problem involves the following linear ODE that models exponential decay,

$$u'(t) = -au(t)\,. \tag{B.27}$$

B.3.1 Forward Euler scheme

We begin with the Forward Euler scheme for discretizing (B.27):

$$[D_t^+ u = -au]^n\,. \tag{B.28}$$

The idea behind the truncation error computation is to insert the exact solution u_{e} of the differential equation problem (B.27) in the discrete equations (B.28) and find the residual that arises because u_{e} does not solve the discrete equations. Instead, u_{e} solves the discrete equations with a residual R^n:

$$[D_t^+ u_{\mathrm{e}} + au_{\mathrm{e}} = R]^n\,. \tag{B.29}$$

From (B.11)-(B.12) it follows that

$$[D_t^+ u_{\mathrm{e}}]^n = u'_{\mathrm{e}}(t_n) + \frac{1}{2}u''_{\mathrm{e}}(t_n)\Delta t + \mathcal{O}(\Delta t^2),$$

which inserted in (B.29) results in

$$u'_{\mathrm{e}}(t_n) + \frac{1}{2}u''_{\mathrm{e}}(t_n)\Delta t + \mathcal{O}(\Delta t^2) + au_{\mathrm{e}}(t_n) = R^n\,.$$

Now, $u'_{\mathrm{e}}(t_n) + au^n_{\mathrm{e}} = 0$ since u_{e} solves the differential equation. The remaining terms constitute the residual:

$$R^n = \frac{1}{2}u''_{\mathrm{e}}(t_n)\Delta t + \mathcal{O}(\Delta t^2)\,. \tag{B.30}$$

This is the truncation error R^n of the Forward Euler scheme.

Because R^n is proportional to Δt, we say that the Forward Euler scheme is of first order in Δt. However, the truncation error is just one error measure, and it is not equal to the true error $u^n_{\mathrm{e}} - u^n$. For this simple model problem we can compute a range of different error measures

for the Forward Euler scheme, including the true error $u_{\mathrm{e}}^n - u^n$, and all of them have dominating terms proportional to Δt.

B.3.2 Crank-Nicolson scheme

For the Crank-Nicolson scheme,

$$[D_t u = -au]^{n+\frac{1}{2}}, \tag{B.31}$$

we compute the truncation error by inserting the exact solution of the ODE and adding a residual R,

$$[D_t u_{\mathrm{e}} + a\overline{u_{\mathrm{e}}}^t = R]^{n+\frac{1}{2}}. \tag{B.32}$$

The term $[D_t u_{\mathrm{e}}]^{n+\frac{1}{2}}$ is easily computed from (B.5)-(B.6) by replacing n with $n+\frac{1}{2}$ in the formula,

$$[D_t u_{\mathrm{e}}]^{n+\frac{1}{2}} = u_{\mathrm{e}}'(t_{n+\frac{1}{2}}) + \frac{1}{24}u_{\mathrm{e}}'''(t_{n+\frac{1}{2}})\Delta t^2 + \mathcal{O}(\Delta t^4).$$

The arithmetic mean is related to $u(t_{n+\frac{1}{2}})$ by (B.21)-(B.22) so

$$[a\overline{u_{\mathrm{e}}}^t]^{n+\frac{1}{2}} = u_{\mathrm{e}}(t_{n+\frac{1}{2}}) + \frac{1}{8}u_{\mathrm{e}}''(t_n)\Delta t^2 + +\mathcal{O}(\Delta t^4).$$

Inserting these expressions in (B.32) and observing that $u_{\mathrm{e}}'(t_{n+\frac{1}{2}}) + au_{\mathrm{e}}^{n+\frac{1}{2}} = 0$, because $u_{\mathrm{e}}(t)$ solves the ODE $u'(t) = -au(t)$ at any point t, we find that

$$R^{n+\frac{1}{2}} = \left(\frac{1}{24}u_{\mathrm{e}}'''(t_{n+\frac{1}{2}}) + \frac{1}{8}u_{\mathrm{e}}''(t_n)\right)\Delta t^2 + \mathcal{O}(\Delta t^4) \tag{B.33}$$

Here, the truncation error is of second order because the leading term in R is proportional to Δt^2.

At this point it is wise to redo some of the computations above to establish the truncation error of the Backward Euler scheme, see Exercise B.7.

B.3.3 The θ-rule

We may also compute the truncation error of the θ-rule,

$$[\bar{D}_t u = -a\overline{u}^{t,\theta}]^{n+\theta}.$$

Our computational task is to find $R^{n+\theta}$ in

$$[\bar{D}_t u_\mathrm{e} + a\overline{u_\mathrm{e}}^{t,\theta} = R]^{n+\theta}\,.$$

From (B.13)-(B.14) and (B.19)-(B.20) we get expressions for the terms with u_e. Using that $u_\mathrm{e}'(t_{n+\theta}) + au_\mathrm{e}(t_{n+\theta}) = 0$, we end up with

$$\begin{aligned} R^{n+\theta} =& (\frac{1}{2} - \theta)u_\mathrm{e}''(t_{n+\theta})\Delta t + \frac{1}{2}\theta(1-\theta)u_\mathrm{e}''(t_{n+\theta})\Delta t^2 + \\ & \frac{1}{2}(\theta^2 - \theta + 3)u_\mathrm{e}'''(t_{n+\theta})\Delta t^2 + \mathcal{O}(\Delta t^3) \end{aligned} \tag{B.34}$$

For $\theta = \frac{1}{2}$ the first-order term vanishes and the scheme is of second order, while for $\theta \neq \frac{1}{2}$ we only have a first-order scheme.

B.3.4 Using symbolic software

The previously mentioned `truncation_error` module can be used to automate the Taylor series expansions and the process of collecting terms. Here is an example on possible use:

```
from truncation_error import DiffOp
from sympy import *

def decay():
    u, a = symbols('u a')
    diffop = DiffOp(u, independent_variable='t',
                    num_terms_Taylor_series=3)
    D1u = diffop.D(1)   # symbol for du/dt
    ODE = D1u + a*u     # define ODE

    # Define schemes
    FE = diffop['Dtp'] + a*u
    CN = diffop['Dt' ] + a*u
    BE = diffop['Dtm'] + a*u
    theta = diffop['barDt'] + a*diffop['weighted_arithmetic_mean']
    theta = sm.simplify(sm.expand(theta))
    # Residuals (truncation errors)
    R = {'FE': FE-ODE, 'BE': BE-ODE, 'CN': CN-ODE,
         'theta': theta-ODE}
    return R
```

The returned dictionary becomes

```
decay: {
 'BE': D2u*dt/2 + D3u*dt**2/6,
 'FE': -D2u*dt/2 + D3u*dt**2/6,
```

```
  'CN': D3u*dt**2/24,
  'theta': -D2u*a*dt**2*theta**2/2 + D2u*a*dt**2*theta/2 -
            D2u*dt*theta + D2u*dt/2 + D3u*a*dt**3*theta**3/3 -
            D3u*a*dt**3*theta**2/2 + D3u*a*dt**3*theta/6 +
            D3u*dt**2*theta**2/2 - D3u*dt**2*theta/2 + D3u*dt**2/6,
}
```

The results are in correspondence with our hand-derived expressions.

B.3.5 Empirical verification of the truncation error

The task of this section is to demonstrate how we can compute the truncation error R numerically. For example, the truncation error of the Forward Euler scheme applied to the decay ODE $u' = -ua$ is

$$R^n = [D_t^+ u_\mathrm{e} + a u_\mathrm{e}]^n \,. \tag{B.35}$$

If we happen to know the exact solution $u_\mathrm{e}(t)$, we can easily evaluate R^n from the above formula.

To estimate how R varies with the discretization parameter Δt, which has been our focus in the previous mathematical derivations, we first make the assumption that $R = C\Delta t^r$ for appropriate constants C and r and small enough Δt. The rate r can be estimated from a series of experiments where Δt is varied. Suppose we have m experiments $(\Delta t_i, R_i)$, $i = 0, \ldots, m-1$. For two consecutive experiments $(\Delta t_{i-1}, R_{i-1})$ and $(\Delta t_i, R_i)$, a corresponding r_{i-1} can be estimated by

$$r_{i-1} = \frac{\ln(R_{i-1}/R_i)}{\ln(\Delta t_{i-1}/\Delta t_i)}, \tag{B.36}$$

for $i = 1, \ldots, m-1$. Note that the truncation error R_i varies through the mesh, so (B.36) is to be applied pointwise. A complicating issue is that R_i and R_{i-1} refer to different meshes. Pointwise comparisons of the truncation error at a certain point in all meshes therefore requires any computed R to be restricted to the *coarsest mesh* and that all finer meshes contain all the points in the coarsest mesh. Suppose we have N_0 intervals in the coarsest mesh. Inserting a superscript n in (B.36), where n counts mesh points in the coarsest mesh, $n = 0, \ldots, N_0$, leads to the formula

$$r_{i-1}^n = \frac{\ln(R_{i-1}^n/R_i^n)}{\ln(\Delta t_{i-1}/\Delta t_i)} \,. \tag{B.37}$$

Experiments are most conveniently defined by N_0 and a number of refinements m. Suppose each mesh has twice as many cells N_i as the previous one:

$$N_i = 2^i N_0, \quad \Delta t_i = T N_i^{-1},$$

where $[0, T]$ is the total time interval for the computations. Suppose the computed R_i values on the mesh with N_i intervals are stored in an array `R[i]` (`R` being a list of arrays, one for each mesh). Restricting this R_i function to the coarsest mesh means extracting every N_i/N_0 point and is done as follows:

```
stride = N[i]/N_0
R[i] = R[i][::stride]
```

The quantity `R[i][n]` now corresponds to R_i^n.

In addition to estimating r for the pointwise values of $R = C\Delta t^r$, we may also consider an integrated quantity on mesh i,

$$R_{I,i} = \left(\Delta t_i \sum_{n=0}^{N_i} (R_i^n)^2\right)^{\frac{1}{2}} \approx \int_0^T R_i(t)dt\,. \tag{B.38}$$

The sequence $R_{I,i}$, $i = 0, \ldots, m-1$, is also expected to behave as $C\Delta t^r$, with the same r as for the pointwise quantity R, as $\Delta t \to 0$.

The function below computes the R_i and $R_{I,i}$ quantities, plots them and compares with the theoretically derived truncation error (`R_a`) if available.

```
import numpy as np
import scitools.std as plt

def estimate(truncation_error, T, N_0, m, makeplot=True):
    """
    Compute the truncation error in a problem with one independent
    variable, using m meshes, and estimate the convergence
    rate of the truncation error.

    The user-supplied function truncation_error(dt, N) computes
    the truncation error on a uniform mesh with N intervals of
    length dt::

      R, t, R_a = truncation_error(dt, N)

    where R holds the truncation error at points in the array t,
    and R_a are the corresponding theoretical truncation error
    values (None if not available).

    The truncation_error function is run on a series of meshes
    with 2**i*N_0 intervals, i=0,1,...,m-1.
```

```
    The values of R and R_a are restricted to the coarsest mesh.
    and based on these data, the convergence rate of R (pointwise)
    and time-integrated R can be estimated empirically.
    """
    N = [2**i*N_0 for i in range(m)]

    R_I = np.zeros(m) # time-integrated R values on various meshes
    R   = [None]*m    # time series of R restricted to coarsest mesh
    R_a = [None]*m    # time series of R_a restricted to coarsest mesh
    dt = np.zeros(m)
    legends_R = [];  legends_R_a = []  # all legends of curves

    for i in range(m):
        dt[i] = T/float(N[i])
        R[i], t, R_a[i] = truncation_error(dt[i], N[i])

        R_I[i] = np.sqrt(dt[i]*np.sum(R[i]**2))

        if i == 0:
            t_coarse = t           # the coarsest mesh

        stride = N[i]/N_0
        R[i] = R[i][::stride]      # restrict to coarsest mesh
        R_a[i] = R_a[i][::stride]

        if makeplot:
            plt.figure(1)
            plt.plot(t_coarse, R[i], log='y')
            legends_R.append('N=%d' % N[i])
            plt.hold('on')

            plt.figure(2)
            plt.plot(t_coarse, R_a[i] - R[i], log='y')
            plt.hold('on')
            legends_R_a.append('N=%d' % N[i])

    if makeplot:
        plt.figure(1)
        plt.xlabel('time')
        plt.ylabel('pointwise truncation error')
        plt.legend(legends_R)
        plt.savefig('R_series.png')
        plt.savefig('R_series.pdf')
        plt.figure(2)
        plt.xlabel('time')
        plt.ylabel('pointwise error in estimated truncation error')
        plt.legend(legends_R_a)
        plt.savefig('R_error.png')
        plt.savefig('R_error.pdf')

    # Convergence rates
    r_R_I = convergence_rates(dt, R_I)
    print 'R integrated in time; r:',
    print ' '.join(['%.1f' % r for r in r_R_I])
```

```
    R = np.array(R)  # two-dim. numpy array
    r_R = [convergence_rates(dt, R[:,n])[-1]
           for n in range(len(t_coarse))]
```

The first `makeplot` block demonstrates how to build up two figures in parallel, using `plt.figure(i)` to create and switch to figure number `i`. Figure numbers start at 1. A logarithmic scale is used on the y axis since we expect that R as a function of time (or mesh points) is exponential. The reason is that the theoretical estimate (B.30) contains u_{e}'', which for the present model goes like e^{-at}. Taking the logarithm makes a straight line.

The code follows closely the previously stated mathematical formulas, but the statements for computing the convergence rates might deserve an explanation. The generic help function `convergence_rate(h, E)` computes and returns r_{i-1}, $i = 1, \ldots, m-1$ from (B.37), given Δt_i in `h` and R_i^n in `E`:

```
def convergence_rates(h, E):
    from math import log
    r = [log(E[i]/E[i-1])/log(h[i]/h[i-1])
         for i in range(1, len(h))]
    return r
```

Calling `r_R_I = convergence_rates(dt, R_I)` computes the sequence of rates $r_0, r_1, \ldots, r_{m-2}$ for the model $R_I \sim \Delta t^r$, while the statements

```
R = np.array(R)  # two-dim. numpy array
r_R = [convergence_rates(dt, R[:,n])[-1]
       for n in range(len(t_coarse))]
```

compute the final rate r_{m-2} for $R^n \sim \Delta t^r$ at each mesh point t_n in the coarsest mesh. This latter computation deserves more explanation. Since `R[i][n]` holds the estimated truncation error R_i^n on mesh i, at point t_n in the coarsest mesh, `R[:,n]` picks out the sequence R_i^n for $i = 0, \ldots, m-1$. The `convergence_rate` function computes the rates at t_n, and by indexing `[-1]` on the returned array from `convergence_rate`, we pick the rate r_{m-2}, which we believe is the best estimation since it is based on the two finest meshes.

The `estimate` function is available in a module `trunc_empir.py`. Let us apply this function to estimate the truncation error of the Forward Euler scheme. We need a function `decay_FE(dt, N)` that can compute (B.35) at the points in a mesh with time step `dt` and `N` intervals:

```
import numpy as np
import trunc_empir
```

```
def decay_FE(dt, N):
    dt = float(dt)
    t = np.linspace(0, N*dt, N+1)
    u_e = I*np.exp(-a*t)  # exact solution, I and a are global
    u = u_e  # naming convention when writing up the scheme
    R = np.zeros(N)

    for n in range(0, N):
        R[n] = (u[n+1] - u[n])/dt + a*u[n]

    # Theoretical expression for the trunction error
    R_a = 0.5*I*(-a)**2*np.exp(-a*t)*dt

    return R, t[:-1], R_a[:-1]

if __name__ == '__main__':
    I = 1; a = 2  # global variables needed in decay_FE
    trunc_empir.estimate(decay_FE, T=2.5, N_0=6, m=4, makeplot=True)
```

The estimated rates for the integrated truncation error R_I become 1.1, 1.0, and 1.0 for this sequence of four meshes. All the rates for R^n, computed as `r_R`, are also very close to 1 at all mesh points. The agreement between the theoretical formula (B.30) and the computed quantity (ref(B.35)) is very good, as illustrated in Figures B.1 and B.2. The program `trunc_decay_FE.py` was used to perform the simulations and it can easily be modified to test other schemes (see also Exercise B.7).

B.3.6 Increasing the accuracy by adding correction terms

Now we ask the question: can we add terms in the differential equation that can help increase the order of the truncation error? To be precise, let us revisit the Forward Euler scheme for $u' = -au$, insert the exact solution u_{e}, include a residual R, but also include new terms C:

$$[D_t^+ u_{\mathrm{e}} + a u_{\mathrm{e}} = C + R]^n \,. \tag{B.39}$$

Inserting the Taylor expansions for $[D_t^+ u_{\mathrm{e}}]^n$ and keeping terms up to 3rd order in Δt gives the equation

$$\frac{1}{2}u_{\mathrm{e}}''(t_n)\Delta t - \frac{1}{6}u_{\mathrm{e}}'''(t_n)\Delta t^2 + \frac{1}{24}u_{\mathrm{e}}''''(t_n)\Delta t^3 + \mathcal{O}(\Delta t^4) = C^n + R^n \,.$$

Can we find C^n such that R^n is $\mathcal{O}(\Delta t^2)$? Yes, by setting

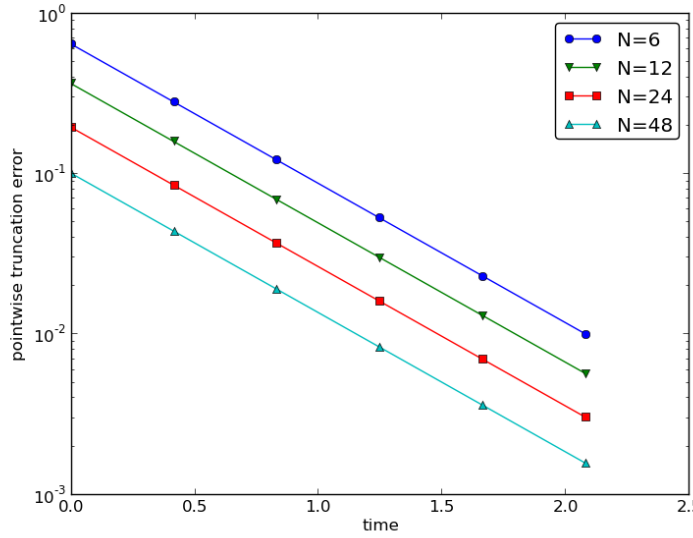


Fig. B.1 Estimated truncation error at mesh points for different meshes.

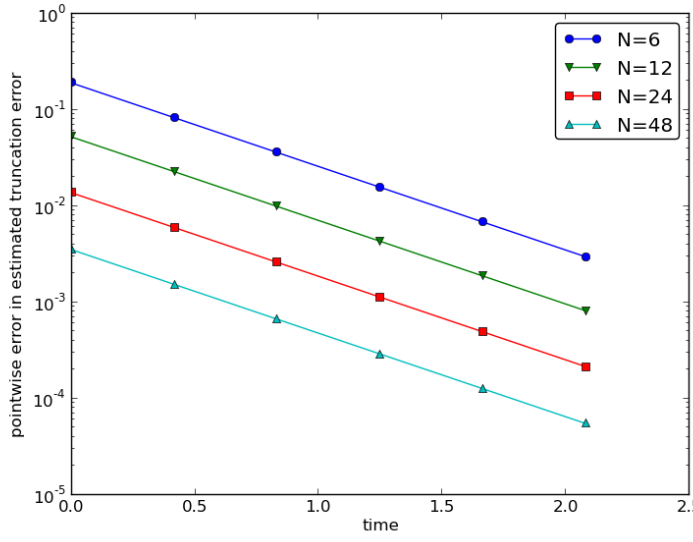


Fig. B.2 Difference between theoretical and estimated truncation error at mesh points for different meshes.

$$C^n = \frac{1}{2}u_{\mathrm{e}}''(t_n)\Delta t,$$

we manage to cancel the first-order term and

$$R^n = \frac{1}{6}u'''_{\mathrm{e}}(t_n)\Delta t^2 + \mathcal{O}(\Delta t^3)\,.$$

The correction term C^n introduces $\frac{1}{2}\Delta t u''$ in the discrete equation, and we have to get rid of the derivative u''. One idea is to approximate u'' by a second-order accurate finite difference formula, $u'' \approx (u^{n+1} - 2u^n + u^{n-1})/\Delta t^2$, but this introduces an additional time level with u^{n-1}. Another approach is to rewrite u'' in terms of u' or u using the ODE:

$$u' = -au \quad \Rightarrow \quad u'' = -au' = -a(-au) = a^2 u\,.$$

This means that we can simply set $C^n = \frac{1}{2}a^2\Delta t u^n$. We can then either solve the discrete equation

$$[D_t^+ u = -au + \frac{1}{2}a^2\Delta t u]^n, \tag{B.40}$$

or we can equivalently discretize the perturbed ODE

$$u' = -\hat{a}u, \quad \hat{a} = a(1 - \frac{1}{2}a\Delta t), \tag{B.41}$$

by a Forward Euler method. That is, we replace the original coefficient a by the perturbed coefficient $\hat{a}$. Observe that $\hat{a} \to a$ as $\Delta t \to 0$.

The Forward Euler method applied to (B.41) results in

$$[D_t^+ u = -a(1 - \frac{1}{2}a\Delta t)u]^n\,.$$

We can control our computations and verify that the truncation error of the scheme above is indeed $\mathcal{O}(\Delta t^2)$.

Another way of revealing the fact that the perturbed ODE leads to a more accurate solution is to look at the amplification factor. Our scheme can be written as

$$u^{n+1} = Au^n, \quad A = 1 - \hat{a}\Delta t = 1 - p + \frac{1}{2}p^2, \quad p = a\Delta t,$$

The amplification factor A as a function of $p = a\Delta t$ is seen to be the first three terms of the Taylor series for the exact amplification factor e^{-p}. The Forward Euler scheme for $u = -au$ gives only the first two terms $1 - p$ of the Taylor series for e^{-p}. That is, using $\hat{a}$ increases the order of the accuracy in the amplification factor.

Instead of replacing u'' by a^2u, we use the relation $u'' = -au'$ and add a term $-\frac{1}{2}a\Delta t u'$ in the ODE:

$$u' = -au - \frac{1}{2}a\Delta t u' \quad \Rightarrow \quad \left(1 + \frac{1}{2}a\Delta t\right) u' = -au\,.$$

Using a Forward Euler method results in

$$\left(1 + \frac{1}{2}a\Delta t\right) \frac{u^{n+1} - u^n}{\Delta t} = -au^n,$$

which after some algebra can be written as

$$u^{n+1} = \frac{1 - \frac{1}{2}a\Delta t}{1 + \frac{1}{2}a\Delta t} u^n\,.$$

This is the same formula as the one arising from a Crank-Nicolson scheme applied to $u' = -au$! It is now recommended to do Exercise B.7 and repeat the above steps to see what kind of correction term is needed in the Backward Euler scheme to make it second order.

The Crank-Nicolson scheme is a bit more challenging to analyze, but the ideas and techniques are the same. The discrete equation reads

$$[D_t u = -au]^{n+\frac{1}{2}},$$

and the truncation error is defined through

$$[D_t u_{\mathrm{e}} + a\overline{u_{\mathrm{e}}}^t = C + R]^{n+\frac{1}{2}},$$

where we have added a correction term. We need to Taylor expand both the discrete derivative and the arithmetic mean with aid of (B.5)-(B.6) and (B.21)-(B.22), respectively. The result is

$$\frac{1}{24}u_{\mathrm{e}}'''(t_{n+\frac{1}{2}})\Delta t^2 + \mathcal{O}(\Delta t^4) + \frac{a}{8}u_{\mathrm{e}}''(t_{n+\frac{1}{2}})\Delta t^2 + \mathcal{O}(\Delta t^4) = C^{n+\frac{1}{2}} + R^{n+\frac{1}{2}}\,.$$

The goal now is to make $C^{n+\frac{1}{2}}$ cancel the Δt^2 terms:

$$C^{n+\frac{1}{2}} = \frac{1}{24}u_{\mathrm{e}}'''(t_{n+\frac{1}{2}})\Delta t^2 + \frac{a}{8}u_{\mathrm{e}}''(t_n)\Delta t^2\,.$$

Using $u' = -au$, we have that $u'' = a^2u$, and we find that $u''' = -a^3u$. We can therefore solve the perturbed ODE problem

$$u' = -\hat{a}u, \quad \hat{a} = a(1 - \frac{1}{12}a^2\Delta t^2),$$

by the Crank-Nicolson scheme and obtain a method that is of fourth order in Δt. Exercise B.7 encourages you to implement these correction terms and calculate empirical convergence rates to verify that higher-order accuracy is indeed obtained in real computations.

B.3.7 Extension to variable coefficients

Let us address the decay ODE with variable coefficients,

$$u'(t) = -a(t)u(t) + b(t),$$

discretized by the Forward Euler scheme,

$$[D_t^+ u = -au + b]^n \,. \tag{B.42}$$

The truncation error R is as always found by inserting the exact solution $u_{\mathrm{e}}(t)$ in the discrete scheme:

$$[D_t^+ u_{\mathrm{e}} + au_{\mathrm{e}} - b = R]^n \,. \tag{B.43}$$

Using (B.11)-(B.12),

$$u'_{\mathrm{e}}(t_n) - \frac{1}{2}u''_{\mathrm{e}}(t_n)\Delta t + \mathcal{O}(\Delta t^2) + a(t_n)u_{\mathrm{e}}(t_n) - b(t_n) = R^n \,.$$

Because of the ODE,

$$u'_{\mathrm{e}}(t_n) + a(t_n)u_{\mathrm{e}}(t_n) - b(t_n) = 0,$$

we are left with the result

$$R^n = -\frac{1}{2}u''_{\mathrm{e}}(t_n)\Delta t + \mathcal{O}(\Delta t^2) \,. \tag{B.44}$$

We see that the variable coefficients do not pose any additional difficulties in this case. Exercise B.7 takes the analysis above one step further to the Crank-Nicolson scheme.

B.3.8 Exact solutions of the finite difference equations

Having a mathematical expression for the numerical solution is very valuable in program verification, since we then know the exact numbers that the program should produce. Looking at the various formulas for the truncation errors in (B.5)-(B.6) and (B.25)-(B.26) in Section B.2.4, we see that all but two of the R expressions contain a second or higher order derivative of u_{e}. The exceptions are the geometric and harmonic means where the truncation error involves u'_{e} and even u_{e} in case of the harmonic mean. So, apart from these two means, choosing u_{e} to be a linear function of t, $u_{\mathrm{e}} = ct + d$ for constants c and d, will make the truncation error vanish since $u''_{\mathrm{e}} = 0$. Consequently, the truncation error of a finite difference scheme will be zero since the various approximations used will all be exact. This means that the linear solution is an exact solution of the discrete equations.

In a particular differential equation problem, the reasoning above can be used to determine if we expect a linear u_{e} to fulfill the discrete equations. To actually prove that this is true, we can either compute the truncation error and see that it vanishes, or we can simply insert $u_{\mathrm{e}}(t) = ct + d$ in the scheme and see that it fulfills the equations. The latter method is usually the simplest. It will often be necessary to add some source term to the ODE in order to allow a linear solution.

Many ODEs are discretized by centered differences. From Section B.2.4 we see that all the centered difference formulas have truncation errors involving u'''_{e} or higher-order derivatives. A quadratic solution, e.g., $u_{\mathrm{e}}(t) = t^2 + ct + d$, will then make the truncation errors vanish. This observation can be used to test if a quadratic solution will fulfill the discrete equations. Note that a quadratic solution will not obey the equations for a Crank-Nicolson scheme for $u' = -au + b$ because the approximation applies an arithmetic mean, which involves a truncation error with u''_{e}.

B.3.9 Computing truncation errors in nonlinear problems

The general nonlinear ODE

$$u' = f(u, t), \tag{B.45}$$

can be solved by a Crank-Nicolson scheme

$$[D_t u = \overline{f}^t]^{n+\frac{1}{2}} . \tag{B.46}$$

The truncation error is as always defined as the residual arising when inserting the exact solution u_{e} in the scheme:

$$[D_t u_{\mathrm{e}} - \overline{f}^t = R]^{n+\frac{1}{2}} . \tag{B.47}$$

Using (B.21)-(B.22) for $\overline{f}^t$ results in

$$\begin{aligned}[\overline{f}^t]^{n+\frac{1}{2}} &= \frac{1}{2}(f(u_{\mathrm{e}}^n, t_n) + f(u_{\mathrm{e}}^{n+1}, t_{n+1})) \\ &= f(u_{\mathrm{e}}^{n+\frac{1}{2}}, t_{n+\frac{1}{2}}) + \frac{1}{8}u''_{\mathrm{e}}(t_{n+\frac{1}{2}})\Delta t^2 + \mathcal{O}(\Delta t^4) .\end{aligned}$$

With (B.5)-(B.6) the discrete equations (B.47) lead to

$$u'_{\mathrm{e}}(t_{n+\frac{1}{2}}) + \frac{1}{24}u'''_{\mathrm{e}}(t_{n+\frac{1}{2}})\Delta t^2 - f(u_{\mathrm{e}}^{n+\frac{1}{2}}, t_{n+\frac{1}{2}}) - \frac{1}{8}u''_{\mathrm{e}}(t_{n+\frac{1}{2}})\Delta t^2 + \mathcal{O}(\Delta t^4) = R^{n+\frac{1}{2}} .$$

Since $u'_{\mathrm{e}}(t_{n+\frac{1}{2}}) - f(u_{\mathrm{e}}^{n+\frac{1}{2}}, t_{n+\frac{1}{2}}) = 0$, the truncation error becomes

$$R^{n+\frac{1}{2}} = (\frac{1}{24}u'''_{\mathrm{e}}(t_{n+\frac{1}{2}}) - \frac{1}{8}u''_{\mathrm{e}}(t_{n+\frac{1}{2}}))\Delta t^2 .$$

The computational techniques worked well even for this nonlinear ODE.

B.4 Vibration ODEs

B.4.1 Linear model without damping

The next example on computing the truncation error involves the following ODE for vibration problems:

$$u''(t) + \omega^2 u(t) = 0 . \tag{B.48}$$

Here, ω is a given constant.

The truncation error of a centered finite difference scheme. Using a standard, second-ordered, central difference for the second-order derivative in time, we have the scheme

$$[D_t D_t u + \omega^2 u = 0]^n . \tag{B.49}$$

Inserting the exact solution u_{e} in this equation and adding a residual R so that u_{e} can fulfill the equation results in

$$[D_tD_tu_{\mathrm{e}} + \omega^2 u_{\mathrm{e}} = R]^n\,. \tag{B.50}$$

To calculate the truncation error R^n, we use (B.17)-(B.18), i.e.,

$$[D_tD_tu_{\mathrm{e}}]^n = u_{\mathrm{e}}''(t_n) + \frac{1}{12}u_{\mathrm{e}}''''(t_n)\Delta t^2 + \mathcal{O}(\Delta t^4),$$

and the fact that $u_{\mathrm{e}}''(t) + \omega^2 u_{\mathrm{e}}(t) = 0$. The result is

$$R^n = \frac{1}{12}u_{\mathrm{e}}''''(t_n)\Delta t^2 + \mathcal{O}(\Delta t^4)\,. \tag{B.51}$$

The truncation error of approximating $u'(0)$. The initial conditions for (B.48) are $u(0) = I$ and $u'(0) = V$. The latter involves a finite difference approximation. The standard choice

$$[D_{2t}u = V]^0,$$

where u^{-1} is eliminated with the aid of the discretized ODE for $n = 0$, involves a centered difference with an $\mathcal{O}(\Delta t^2)$ truncation error given by (B.7)-(B.8). The simpler choice

$$[D_t^+u = V]^0,$$

is based on a forward difference with a truncation error $\mathcal{O}(\Delta t)$. A central question is if this initial error will impact the order of the scheme throughout the simulation. Exercise B.7 asks you to perform an experiment to investigate this question.

Truncation error of the equation for the first step. We have shown that the truncation error of the difference used to approximate the initial condition $u'(0) = 0$ is $\mathcal{O}(\Delta t^2)$, but we can also investigate the difference equation used for the first step. In a truncation error setting, the right way to view this equation is not to use the initial condition $[D_{2t}u = V]^0$ to express $u^{-1} = u^1 - 2\Delta tV$ in order to eliminate u^{-1} from the discretized differential equation, but the other way around: the fundamental equation is the discretized initial condition $[D_{2t}u = V]^0$ and we use the discretized ODE $[D_tD_t + \omega^2 u = 0]^0$ to eliminate u^{-1} in the discretized initial condition. From $[D_tD_t + \omega^2 u = 0]^0$ we have

$$u^{-1} = 2u^0 - u^1 - \Delta t^2\omega^2 u^0,$$

which inserted in $[D_{2t}u = V]^0$ gives

$$\frac{u^1 - u^0}{\Delta t} + \frac{1}{2}\omega^2 \Delta t u^0 = V\,. \tag{B.52}$$

The first term can be recognized as a forward difference such that the equation can be written in operator notation as

$$[D_t^+ u + \frac{1}{2}\omega^2 \Delta t u = V]^0\,.$$

The truncation error is defined as

$$[D_t^+ u_{\rm e} + \frac{1}{2}\omega^2 \Delta t u_{\rm e} - V = R]^0\,.$$

Using (B.11)-(B.12) with one more term in the Taylor series, we get that

$$u_{\rm e}'(0) + \frac{1}{2}u_{\rm e}''(0)\Delta t + \frac{1}{6}u_{\rm e}'''(0)\Delta t^2 + \mathcal{O}(\Delta t^3) + \frac{1}{2}\omega^2 \Delta t u_{\rm e}(0) - V = R^n\,.$$

Now, $u_{\rm e}'(0) = V$ and $u_{\rm e}''(0) = -\omega^2 u_{\rm e}(0)$ so we get

$$R^n = \frac{1}{6}u_{\rm e}'''(0)\Delta t^2 + \mathcal{O}(\Delta t^3)\,.$$

There is another way of analyzing the discrete initial condition, because eliminating u^{-1} via the discretized ODE can be expressed as

$$[D_{2t}u + \Delta t(D_t D_t u - \omega^2 u) = V]^0\,. \tag{B.53}$$

Writing out (B.53) shows that the equation is equivalent to (B.52). The truncation error is defined by

$$[D_{2t}u_{\rm e} + \Delta t(D_t D_t u_{\rm e} - \omega^2 u_{\rm e}) = V + R]^0\,.$$

Replacing the difference via (B.7)-(B.8) and (B.17)-(B.18), as well as using $u_{\rm e}'(0) = V$ and $u_{\rm e}''(0) = -\omega^2 u_{\rm e}(0)$, gives

$$R^n = \frac{1}{6}u_{\rm e}'''(0)\Delta t^2 + \mathcal{O}(\Delta t^3)\,.$$

Computing correction terms. The idea of using correction terms to increase the order of R^n can be applied as described in Section B.3.6. We look at

$$[D_t D_t u_{\rm e} + \omega^2 u_{\rm e} = C + R]^n,$$

and observe that C^n must be chosen to cancel the Δt^2 term in R^n. That is,

$$C^n = \frac{1}{12} u_{\mathrm{e}}''''(t_n)\Delta t^2 \,.$$

To get rid of the 4th-order derivative we can use the differential equation: $u'' = -\omega^2 u$, which implies $u'''' = \omega^4 u$. Adding the correction term to the ODE results in

$$u'' + \omega^2(1 - \frac{1}{12}\omega^2\Delta t^2)u = 0\,. \tag{B.54}$$

Solving this equation by the standard scheme

$$[D_t D_t u + \omega^2(1 - \frac{1}{12}\omega^2\Delta t^2)u = 0]^n,$$

will result in a scheme with truncation error $\mathcal{O}(\Delta t^4)$.

We can use another set of arguments to justify that (B.54) leads to a higher-order method. Mathematical analysis of the scheme (B.49) reveals that the numerical frequency $\tilde{\omega}$ is (approximately as $\Delta t \to 0$)

$$\tilde{\omega} = \omega(1 + \frac{1}{24}\omega^2\Delta t^2)\,.$$

One can therefore attempt to replace ω in the ODE by a slightly smaller ω since the numerics will make it larger:

$$[u'' + (\omega(1 - \frac{1}{24}\omega^2\Delta t^2))^2 u]^n = 0\,.$$

Expanding the squared term and omitting the higher-order term Δt^4 gives exactly the ODE (B.54). Experiments show that u^n is computed to 4th order in Δt. You can confirm this by running a little program in the `vib` directory:

```
from vib_undamped import convergence_rates, solver_adjust_w

r = convergence_rates(
      m=5, solver_function=solver_adjust_w, num_periods=8)
```

One will see that the rates `r` lie around 4.

B.4.2 Model with damping and nonlinearity

The model (B.48) can be extended to include damping $\beta u'$, a nonlinear restoring (spring) force $s(u)$, and some known excitation force $F(t)$:

$$mu'' + \beta u' + s(u) = F(t)\,. \tag{B.55}$$

The coefficient m usually represents the mass of the system. This governing equation can be discretized by centered differences:

$$[mD_tD_tu + \beta D_{2t}u + s(u) = F]^n\,. \tag{B.56}$$

The exact solution u_e fulfills the discrete equations with a residual term:

$$[mD_tD_tu_\mathrm{e} + \beta D_{2t}u_\mathrm{e} + s(u_\mathrm{e}) = F + R]^n\,. \tag{B.57}$$

Using (B.17)-(B.18) and (B.7)-(B.8) we get

$$[mD_tD_tu_\mathrm{e} + \beta D_{2t}u_\mathrm{e}]^n = mu_\mathrm{e}''(t_n) + \beta u_\mathrm{e}'(t_n) + \left(\frac{m}{12}u_\mathrm{e}''''(t_n) + \frac{\beta}{6}u_\mathrm{e}'''(t_n)\right)\Delta t^2 + \mathcal{O}(\Delta t^4)$$

Combining this with the previous equation, we can collect the terms

$$mu_\mathrm{e}''(t_n) + \beta u_\mathrm{e}'(t_n) + \omega^2 u_\mathrm{e}(t_n) + s(u_\mathrm{e}(t_n)) - F^n,$$

and set this sum to zero because u_e solves the differential equation. We are left with the truncation error

$$R^n = \left(\frac{m}{12}u_\mathrm{e}''''(t_n) + \frac{\beta}{6}u_\mathrm{e}'''(t_n)\right)\Delta t^2 + \mathcal{O}(\Delta t^4), \tag{B.58}$$

so the scheme is of second order.

According to (B.58), we can add correction terms

$$C^n = \left(\frac{m}{12}u_\mathrm{e}''''(t_n) + \frac{\beta}{6}u_\mathrm{e}'''(t_n)\right)\Delta t^2,$$

to the right-hand side of the ODE to obtain a fourth-order scheme. However, expressing u'''' and u''' in terms of lower-order derivatives is now harder because the differential equation is more complicated:

$$
\begin{aligned}
u''' &= \frac{1}{m}(F' - \beta u'' - s'(u)u'),\\
u'''' &= \frac{1}{m}(F'' - \beta u''' - s''(u)(u')^2 - s'(u)u''),\\
&= \frac{1}{m}(F'' - \beta\frac{1}{m}(F' - \beta u'' - s'(u)u') - s''(u)(u')^2 - s'(u)u'')\,.
\end{aligned}
$$

It is not impossible to discretize the resulting modified ODE, but it is up to debate whether correction terms are feasible and the way to go. Computing with a smaller Δt is usually always possible in these problems to achieve the desired accuracy.

B.4.3 Extension to quadratic damping

Instead of the linear damping term $\beta u'$ in (B.55) we now consider quadratic damping $\beta|u'|u'$:

$$
mu'' + \beta|u'|u' + s(u) = F(t)\,. \tag{B.59}
$$

A centered difference for u' gives rise to a nonlinearity, which can be linearized using a geometric mean: $[|u'|u']^n \approx |[u']^{n-\frac{1}{2}}|[u']^{n+\frac{1}{2}}$. The resulting scheme becomes

$$
[mD_tD_tu]^n + \beta|[D_tu]^{n-\frac{1}{2}}|[D_tu]^{n+\frac{1}{2}} + s(u^n) = F^n\,. \tag{B.60}
$$

The truncation error is defined through

$$
[mD_tD_tu_{\mathrm{e}}]^n + \beta|[D_tu_{\mathrm{e}}]^{n-\frac{1}{2}}|[D_tu_{\mathrm{e}}]^{n+\frac{1}{2}} + s(u_{\mathrm{e}}^n) - F^n = R^n\,. \tag{B.61}
$$

We start with expressing the truncation error of the geometric mean. According to (B.23)-(B.24),

$$
\begin{aligned}
|[D_tu_{\mathrm{e}}]^{n-\frac{1}{2}}|[D_tu_{\mathrm{e}}]^{n+\frac{1}{2}} = [|D_tu_{\mathrm{e}}|D_tu_{\mathrm{e}}]^n - \frac{1}{4}u_{\mathrm{e}}'(t_n)^2\Delta t^2 +\\
\frac{1}{4}u_{\mathrm{e}}(t_n)u_{\mathrm{e}}''(t_n)\Delta t^2 + \mathcal{O}(\Delta t^4)\,.
\end{aligned}
$$

Using (B.5)-(B.6) for the D_tu_{e} factors results in

$$
[|D_tu_{\mathrm{e}}|D_tu_{\mathrm{e}}]^n = |u_{\mathrm{e}}' + \frac{1}{24}u_{\mathrm{e}}'''(t_n)\Delta t^2 + \mathcal{O}(\Delta t^4)|(u_{\mathrm{e}}' + \frac{1}{24}u_{\mathrm{e}}'''(t_n)\Delta t^2 + \mathcal{O}(\Delta t^4))
$$

We can remove the absolute value since it essentially gives a factor 1 or -1 only. Calculating the product, we have the leading-order terms

$$[D_t u_{\mathrm{e}} D_t u_{\mathrm{e}}]^n = (u'_{\mathrm{e}}(t_n))^2 + \frac{1}{12} u_{\mathrm{e}}(t_n) u'''_{\mathrm{e}}(t_n) \Delta t^2 + \mathcal{O}(\Delta t^4)\,.$$

With

$$m[D_t D_t u_{\mathrm{e}}]^n = m u''_{\mathrm{e}}(t_n) + \frac{m}{12} u''''_{\mathrm{e}}(t_n) \Delta t^2 + \mathcal{O}(\Delta t^4),$$

and using the differential equation on the form $mu'' + \beta(u')^2 + s(u) = F$, we end up with

$$R^n = (\frac{m}{12} u''''_{\mathrm{e}}(t_n) + \frac{\beta}{12} u_{\mathrm{e}}(t_n) u'''_{\mathrm{e}}(t_n)) \Delta t^2 + \mathcal{O}(\Delta t^4)\,.$$

This result demonstrates that we have second-order accuracy also with quadratic damping. The key elements that lead to the second-order accuracy is that the difference approximations are $\mathcal{O}(\Delta t^2)$ *and* the geometric mean approximation is also $\mathcal{O}(\Delta t^2)$.

B.4.4 The general model formulated as first-order ODEs

The second-order model (B.59) can be formulated as a first-order system,

$$v' = \frac{1}{m}\left(F(t) - \beta|v|v - s(u)\right), \tag{B.62}$$

$$u' = v\,. \tag{B.63}$$

The system (B.63)-(B.63) can be solved either by a forward-backward scheme (the Euler-Cromer method) or a centered scheme on a staggered mesh.

A centered scheme on a staggered mesh. We now introduce a staggered mesh where we seek u at mesh points t_n and v at points $t_{n+\frac{1}{2}}$ in between the u points. The staggered mesh makes it easy to formulate centered differences in the system (B.63)-(B.63):

$$[D_t u = v]^{n-\frac{1}{2}}, \tag{B.64}$$

$$[D_t v = \frac{1}{m}(F(t) - \beta|v|v - s(u))]^n\,. \tag{B.65}$$

The term $|v^n|v^n$ causes trouble since v^n is not computed, only $v^{n-\frac{1}{2}}$ and $v^{n+\frac{1}{2}}$. Using geometric mean, we can express $|v^n|v^n$ in terms of known quantities: $|v^n|v^n \approx |v^{n-\frac{1}{2}}|v^{n+\frac{1}{2}}$. We then have

$$[D_t u]^{n-\frac{1}{2}} = v^{n-\frac{1}{2}}, \tag{B.66}$$

$$[D_t v]^n = \frac{1}{m}(F(t_n) - \beta|v^{n-\frac{1}{2}}|v^{n+\frac{1}{2}} - s(u^n))\,. \tag{B.67}$$

The truncation error in each equation fulfills

$$[D_t u_{\mathrm{e}}]^{n-\frac{1}{2}} = v_{\mathrm{e}}(t_{n-\frac{1}{2}}) + R_u^{n-\frac{1}{2}},$$

$$[D_t v_{\mathrm{e}}]^n = \frac{1}{m}(F(t_n) - \beta|v_{\mathrm{e}}(t_{n-\frac{1}{2}})|v_{\mathrm{e}}(t_{n+\frac{1}{2}}) - s(u^n)) + R_v^n\,.$$

The truncation error of the centered differences is given by (B.5)-(B.6), and the geometric mean approximation analysis can be taken from (B.23)-(B.24). These results lead to

$$u_{\mathrm{e}}'(t_{n-\frac{1}{2}}) + \frac{1}{24}u_{\mathrm{e}}'''(t_{n-\frac{1}{2}})\Delta t^2 + \mathcal{O}(\Delta t^4) = v_{\mathrm{e}}(t_{n-\frac{1}{2}}) + R_u^{n-\frac{1}{2}},$$

and

$$v_{\mathrm{e}}'(t_n) = \frac{1}{m}(F(t_n) - \beta|v_{\mathrm{e}}(t_n)|v_{\mathrm{e}}(t_n) + \mathcal{O}(\Delta t^2) - s(u^n)) + R_v^n\,.$$

The ODEs fulfilled by u_{e} and v_{e} are evident in these equations, and we achieve second-order accuracy for the truncation error in both equations:

$$R_u^{n-\frac{1}{2}} = \mathcal{O}(\Delta t^2), \quad R_v^n = \mathcal{O}(\Delta t^2)\,.$$

B.5 Wave equations

B.5.1 Linear wave equation in 1D

The standard, linear wave equation in 1D for a function $u(x,t)$ reads

$$\frac{\partial^2 u}{\partial t^2} = c^2\frac{\partial^2 u}{\partial x^2} + f(x,t), \quad x \in (0,L),\ t \in (0,T], \tag{B.68}$$

where c is the constant wave velocity of the physical medium in $[0, L]$. The equation can also be more compactly written as

$$u_{tt} = c^2 u_{xx} + f, \quad x \in (0, L),\ t \in (0, T], \tag{B.69}$$

Centered, second-order finite differences are a natural choice for discretizing the derivatives, leading to

$$[D_t D_t u = c^2 D_x D_x u + f]_i^n \,. \tag{B.70}$$

Inserting the exact solution $u_{\mathrm{e}}(x, t)$ in (B.70) makes this function fulfill the equation if we add the term R:

$$[D_t D_t u_{\mathrm{e}} = c^2 D_x D_x u_{\mathrm{e}} + f + R]_i^n \tag{B.71}$$

Our purpose is to calculate the truncation error R. From (B.17)-(B.18) we have that

$$[D_t D_t u_{\mathrm{e}}]_i^n = u_{\mathrm{e},tt}(x_i, t_n) + \frac{1}{12} u_{\mathrm{e},tttt}(x_i, t_n) \Delta t^2 + \mathcal{O}(\Delta t^4),$$

when we use a notation taking into account that u_{e} is a function of two variables and that derivatives must be partial derivatives. The notation $u_{\mathrm{e},tt}$ means $\partial^2 u_{\mathrm{e}} / \partial t^2$.

The same formula may also be applied to the x-derivative term:

$$[D_x D_x u_{\mathrm{e}}]_i^n = u_{\mathrm{e},xx}(x_i, t_n) + \frac{1}{12} u_{\mathrm{e},xxxx}(x_i, t_n) \Delta x^2 + \mathcal{O}(\Delta x^4),$$

Equation (B.71) now becomes

$$u_{\mathrm{e},tt} + \frac{1}{12} u_{\mathrm{e},tttt}(x_i, t_n) \Delta t^2 = c^2 u_{\mathrm{e},xx} + c^2 \frac{1}{12} u_{\mathrm{e},xxxx}(x_i, t_n) \Delta x^2 + f(x_i, t_n) + \mathcal{O}(\Delta t^4, \Delta x^4) + R_i^n \,.$$

Because u_{e} fulfills the partial differential equation (PDE) (B.69), the first, third, and fifth term cancel out, and we are left with

$$R_i^n = \frac{1}{12} u_{\mathrm{e},tttt}(x_i, t_n) \Delta t^2 - c^2 \frac{1}{12} u_{\mathrm{e},xxxx}(x_i, t_n) \Delta x^2 + \mathcal{O}(\Delta t^4, \Delta x^4), \tag{B.72}$$

showing that the scheme (B.70) is of second order in the time and space mesh spacing.

B.5.2 Finding correction terms

Can we add correction terms to the PDE and increase the order of R_i^n in (B.72)? The starting point is

$$[D_tD_tu_\mathrm{e} = c^2D_xD_xu_\mathrm{e} + f + C + R]_i^n \tag{B.73}$$

From the previous analysis we simply get (B.72) again, but now with C:

$$R_i^n + C_i^n = \frac{1}{12}u_{\mathrm{e},tttt}(x_i,t_n)\Delta t^2 - c^2\frac{1}{12}u_{\mathrm{e},xxxx}(x_i,t_n)\Delta x^2 + \mathcal{O}(\Delta t^4,\Delta x^4)\,. \tag{B.74}$$

The idea is to let C_i^n cancel the Δt^2 and Δx^2 terms to make $R_i^n = \mathcal{O}(\Delta t^4,\Delta x^4)$:

$$C_i^n = \frac{1}{12}u_{\mathrm{e},tttt}(x_i,t_n)\Delta t^2 - c^2\frac{1}{12}u_{\mathrm{e},xxxx}(x_i,t_n)\Delta x^2\,.$$

Essentially, it means that we add a new term

$$C = \frac{1}{12}\left(u_{tttt}\Delta t^2 - c^2u_{xxxx}\Delta x^2\right),$$

to the right-hand side of the PDE. We must either discretize these 4th-order derivatives directly or rewrite them in terms of lower-order derivatives with the aid of the PDE. The latter approach is more feasible. From the PDE we have the operator equality

$$\frac{\partial^2}{\partial t^2} = c^2\frac{\partial^2}{\partial x^2},$$

so

$$u_{tttt} = c^2u_{xxtt}, \quad u_{xxxx} = c^{-2}u_{ttxx}\,.$$

Assuming u is smooth enough, so that $u_{xxtt} = u_{ttxx}$, these relations lead to

$$C = \frac{1}{12}((c^2\Delta t^2 - \Delta x^2)u_{xx})_{tt}\,.$$

A natural discretization is

$$C_i^n = \frac{1}{12}((c^2\Delta t^2 - \Delta x^2)[D_xD_xD_tD_tu]_i^n\,.$$

Writing out $[D_xD_xD_tD_tu]_i^n$ as $[D_xD_x(D_tD_tu)]_i^n$ gives

$$\frac{1}{\Delta t^2}\Bigg(\frac{u_{i+1}^{n+1}-2u_{i+1}^{n}+u_{i+1}^{n-1}}{\Delta x^2}-2 \\ \frac{u_{i}^{n+1}-2u_{i}^{n}+u_{i}^{n-1}}{\Delta x^2}+ \qquad \frac{u_{i-1}^{n+1}-2u_{i-1}^{n}+u_{i-1}^{n-1}}{\Delta x^2}\Bigg)$$

Now the unknown values u_{i+1}^{n+1}, u_i^{n+1}, and u_{i-1}^{n+1} are *coupled*, and we must solve a tridiagonal system to find them. This is in principle straightforward, but it results in an implicit finite difference scheme, while we had a convenient explicit scheme without the correction terms.

B.5.3 Extension to variable coefficients

Now we address the variable coefficient version of the linear 1D wave equation,

$$\frac{\partial^2 u}{\partial t^2} = \frac{\partial}{\partial x}\left(\lambda(x)\frac{\partial u}{\partial x}\right),$$

or written more compactly as

$$u_{tt} = (\lambda u_x)_x\,. \tag{B.75}$$

The discrete counterpart to this equation, using arithmetic mean for λ and centered differences, reads

$$[D_t D_t u = D_x \overline{\lambda}^x D_x u]_i^n\,. \tag{B.76}$$

The truncation error is the residual R in the equation

$$[D_t D_t u_{\mathrm{e}} = D_x \overline{\lambda}^x D_x u_{\mathrm{e}} + R]_i^n\,. \tag{B.77}$$

The difficulty with (B.77) is how to compute the truncation error of the term $[D_x \overline{\lambda}^x D_x u_{\mathrm{e}}]_i^n$.

We start by writing out the outer operator:

$$[D_x \overline{\lambda}^x D_x u_{\mathrm{e}}]_i^n = \frac{1}{\Delta x}\left([\overline{\lambda}^x D_x u_{\mathrm{e}}]_{i+\frac{1}{2}}^n - [\overline{\lambda}^x D_x u_{\mathrm{e}}]_{i-\frac{1}{2}}^n\right). \tag{B.78}$$

With the aid of (B.5)-(B.6) and (B.21)-(B.22) we have

$$
\begin{aligned}
[D_x u_{\mathrm{e}}]^n_{i+\frac{1}{2}} &= u_{\mathrm{e},x}(x_{i+\frac{1}{2}}, t_n) + \frac{1}{24} u_{\mathrm{e},xxx}(x_{i+\frac{1}{2}}, t_n)\Delta x^2 + \mathcal{O}(\Delta x^4), \\
[\overline{\lambda}^x]_{i+\frac{1}{2}} &= \lambda(x_{i+\frac{1}{2}}) + \frac{1}{8}\lambda''(x_{i+\frac{1}{2}})\Delta x^2 + \mathcal{O}(\Delta x^4), \\
[\overline{\lambda}^x D_x u_{\mathrm{e}}]^n_{i+\frac{1}{2}} &= (\lambda(x_{i+\frac{1}{2}}) + \frac{1}{8}\lambda''(x_{i+\frac{1}{2}})\Delta x^2 + \mathcal{O}(\Delta x^4))\times \\
&\quad (u_{\mathrm{e},x}(x_{i+\frac{1}{2}}, t_n) + \frac{1}{24} u_{\mathrm{e},xxx}(x_{i+\frac{1}{2}}, t_n)\Delta x^2 + \mathcal{O}(\Delta x^4)) \\
&= \lambda(x_{i+\frac{1}{2}}) u_{\mathrm{e},x}(x_{i+\frac{1}{2}}, t_n) + \lambda(x_{i+\frac{1}{2}})\frac{1}{24} u_{\mathrm{e},xxx}(x_{i+\frac{1}{2}}, t_n)\Delta x^2 + \\
&\quad u_{\mathrm{e},x}(x_{i+\frac{1}{2}}, t_n)\frac{1}{8}\lambda''(x_{i+\frac{1}{2}})\Delta x^2 + \mathcal{O}(\Delta x^4) \\
&= [\lambda u_{\mathrm{e},x}]^n_{i+\frac{1}{2}} + G^n_{i+\frac{1}{2}}\Delta x^2 + \mathcal{O}(\Delta x^4),
\end{aligned}
$$

where we have introduced the short form

$$
G^n_{i+\frac{1}{2}} = \frac{1}{24} u_{\mathrm{e},xxx}(x_{i+\frac{1}{2}}, t_n)\lambda(x_{i+\frac{1}{2}}) + u_{\mathrm{e},x}(x_{i+\frac{1}{2}}, t_n)\frac{1}{8}\lambda''(x_{i+\frac{1}{2}})\,.
$$

Similarly, we find that

$$
[\overline{\lambda}^x D_x u_{\mathrm{e}}]^n_{i-\frac{1}{2}} = [\lambda u_{\mathrm{e},x}]^n_{i-\frac{1}{2}} + G^n_{i-\frac{1}{2}}\Delta x^2 + \mathcal{O}(\Delta x^4)\,.
$$

Inserting these expressions in the outer operator (B.78) results in

$$
\begin{aligned}
[D_x\overline{\lambda}^x D_x u_{\mathrm{e}}]^n_i &= \frac{1}{\Delta x}([\overline{\lambda}^x D_x u_{\mathrm{e}}]^n_{i+\frac{1}{2}} - [\overline{\lambda}^x D_x u_{\mathrm{e}}]^n_{i-\frac{1}{2}}) \\
&= \frac{1}{\Delta x}([\lambda u_{\mathrm{e},x}]^n_{i+\frac{1}{2}} + G^n_{i+\frac{1}{2}}\Delta x^2 - [\lambda u_{\mathrm{e},x}]^n_{i-\frac{1}{2}} - G^n_{i-\frac{1}{2}}\Delta x^2 + \mathcal{O}(\Delta x^4)) \\
&= [D_x \lambda u_{\mathrm{e},x}]^n_i + [D_x G]^n_i \Delta x^2 + \mathcal{O}(\Delta x^4)\,.
\end{aligned}
$$

The reason for $\mathcal{O}(\Delta x^4)$ in the remainder is that there are coefficients in front of this term, say $H\Delta x^4$, and the subtraction and division by Δx results in $[D_x H]^n_i \Delta x^4$.

We can now use (B.5)-(B.6) to express the D_x operator in $[D_x \lambda u_{\mathrm{e},x}]^n_i$ as a derivative and a truncation error:

$$
[D_x \lambda u_{\mathrm{e},x}]^n_i = \frac{\partial}{\partial x}\lambda(x_i) u_{\mathrm{e},x}(x_i, t_n) + \frac{1}{24}(\lambda u_{\mathrm{e},x})_{xxx}(x_i, t_n)\Delta x^2 + \mathcal{O}(\Delta x^4)\,.
$$

Expressions like $[D_x G]^n_i \Delta x^2$ can be treated in an identical way,

$$[D_x G]_i^n \Delta x^2 = G_x(x_i, t_n)\Delta x^2 + \frac{1}{24} G_{xxx}(x_i, t_n)\Delta x^4 + \mathcal{O}(\Delta x^4)\,.$$

There will be a number of terms with the Δx^2 factor. We lump these now into $\mathcal{O}(\Delta x^2)$. The result of the truncation error analysis of the spatial derivative is therefore summarized as

$$[D_x \overline{\lambda}^x D_x u_{\mathrm{e}}]_i^n = \frac{\partial}{\partial x}\lambda(x_i) u_{\mathrm{e},x}(x_i, t_n) + \mathcal{O}(\Delta x^2)\,.$$

After having treated the $[D_t D_t u_{\mathrm{e}}]_i^n$ term as well, we achieve

$$R_i^n = \mathcal{O}(\Delta x^2) + \frac{1}{12} u_{\mathrm{e},tttt}(x_i, t_n)\Delta t^2\,.$$

The main conclusion is that the scheme is of second-order in time and space also in this variable coefficient case. The key ingredients for second order are the centered differences and the arithmetic mean for λ: all those building blocks feature second-order accuracy.

B.5.4 Linear wave equation in 2D/3D

The two-dimensional extension of (B.68) takes the form

$$\frac{\partial^2 u}{\partial t^2} = c^2\left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2}\right) + f(x, y, t), \quad (x, y) \in (0, L)\times(0, H),\ t \in (0, T], \tag{B.79}$$

where now $c(x, y)$ is the constant wave velocity of the physical medium $[0, L] \times [0, H]$. In compact notation, the PDE (B.79) can be written

$$u_{tt} = c^2(u_{xx} + u_{yy}) + f(x, y, t), \quad (x, y) \in (0, L) \times (0, H),\ t \in (0, T], \tag{B.80}$$

in 2D, while the 3D version reads

$$u_{tt} = c^2(u_{xx} + u_{yy} + u_{zz}) + f(x, y, z, t), \tag{B.81}$$

for $(x, y, z) \in (0, L) \times (0, H) \times (0, B)$ and $t \in (0, T]$.

Approximating the second-order derivatives by the standard formulas (B.17)-(B.18) yields the scheme

$$[D_t D_t u = c^2(D_x D_x u + D_y D_y u + D_z D_z u) + f]^n_{i,j,k}\,. \tag{B.82}$$

The truncation error is found from

$$[D_t D_t u_\mathrm{e} = c^2(D_x D_x u_\mathrm{e} + D_y D_y u_\mathrm{e} + D_z D_z u_\mathrm{e}) + f + R]^n_{i,j,k}\,. \tag{B.83}$$

The calculations from the 1D case can be repeated with the terms in the y and z directions. Collecting terms that fulfill the PDE, we end up with

$$R^n_{i,j,k} = [\frac{1}{12}u_{\mathrm{e},tttt}\Delta t^2 - c^2\frac{1}{12}\left(u_{\mathrm{e},xxxx}\Delta x^2 + u_{\mathrm{e},yyyy}\Delta x^2 + u_{\mathrm{e},zzzz}\Delta z^2\right)]^n_{i,j,k} + \mathcal{O}(\Delta t^4, \Delta x^4, \Delta y^4, \Delta z^4)\,. \tag{B.84}$$

B.6 Diffusion equations

B.6.1 Linear diffusion equation in 1D

The standard, linear, 1D diffusion equation takes the form

$$\frac{\partial u}{\partial t} = \alpha\frac{\partial^2 u}{\partial x^2} + f(x,t), \quad x \in (0,L),\ t \in (0,T], \tag{B.85}$$

where $\alpha > 0$ is a constant diffusion coefficient. A more compact form of the diffusion equation is $u_t = \alpha u_{xx} + f$.

The spatial derivative in the diffusion equation, αu_{xx}, is commonly discretized as $[D_x D_x u]^n_i$. The time-derivative, however, can be treated by a variety of methods.

The Forward Euler scheme in time. Let us start with the simple Forward Euler scheme:

$$[D_t^+ u = \alpha D_x D_x u + f]^n_i\,.$$

The truncation error arises as the residual R when inserting the exact solution u_e in the discrete equations:

$$[D_t^+ u_\mathrm{e} = \alpha D_x D_x u_\mathrm{e} + f + R]^n_i\,.$$

Now, using (B.11)-(B.12) and (B.17)-(B.18), we can transform the difference operators to derivatives:

$$u_{\mathrm{e},t}(x_i,t_n)+\frac{1}{2}u_{\mathrm{e},tt}(t_n)\Delta t+\mathcal{O}(\Delta t^2)=\alpha u_{\mathrm{e},xx}(x_i,t_n)+ \\ \frac{\alpha}{12}u_{\mathrm{e},xxxx}(x_i,t_n)\Delta x^2+\mathcal{O}(\Delta x^4)+f(x_i,t_n)+R_i^n\,.$$

The terms $u_{\mathrm{e},t}(x_i,t_n)-\alpha u_{\mathrm{e},xx}(x_i,t_n)-f(x_i,t_n)$ vanish because u_{e} solves the PDE. The truncation error then becomes

$$R_i^n=\frac{1}{2}u_{\mathrm{e},tt}(t_n)\Delta t+\mathcal{O}(\Delta t^2)-\frac{\alpha}{12}u_{\mathrm{e},xxxx}(x_i,t_n)\Delta x^2+\mathcal{O}(\Delta x^4)\,.$$

The Crank-Nicolson scheme in time. The Crank-Nicolson method consists of using a centered difference for u_t and an arithmetic average of the u_{xx} term:

$$[D_t u]_i^{n+\frac{1}{2}}=\alpha\frac{1}{2}([D_xD_xu]_i^n+[D_xD_xu]_i^{n+1})+f_i^{n+\frac{1}{2}}\,.$$

The equation for the truncation error is

$$[D_t u_{\mathrm{e}}]_i^{n+\frac{1}{2}}=\alpha\frac{1}{2}([D_xD_xu_{\mathrm{e}}]_i^n+[D_xD_xu_{\mathrm{e}}]_i^{n+1})+f_i^{n+\frac{1}{2}}+R_i^{n+\frac{1}{2}}\,.$$

To find the truncation error, we start by expressing the arithmetic average in terms of values at time $t_{n+\frac{1}{2}}$. According to (B.21)-(B.22),

$$\frac{1}{2}([D_xD_xu_{\mathrm{e}}]_i^n+[D_xD_xu_{\mathrm{e}}]_i^{n+1})=[D_xD_xu_{\mathrm{e}}]_i^{n+\frac{1}{2}}+\frac{1}{8}[D_xD_xu_{\mathrm{e},tt}]_i^{n+\frac{1}{2}}\Delta t^2+\mathcal{O}(\Delta t^4)\,.$$

With (B.17)-(B.18) we can express the difference operator D_xD_xu in terms of a derivative:

$$[D_xD_xu_{\mathrm{e}}]_i^{n+\frac{1}{2}}=u_{\mathrm{e},xx}(x_i,t_{n+\frac{1}{2}})+\frac{1}{12}u_{\mathrm{e},xxxx}(x_i,t_{n+\frac{1}{2}})\Delta x^2+\mathcal{O}(\Delta x^4)\,.$$

The error term from the arithmetic mean is similarly expanded,

$$\frac{1}{8}[D_xD_xu_{\mathrm{e},tt}]_i^{n+\frac{1}{2}}\Delta t^2=\frac{1}{8}u_{\mathrm{e},ttxx}(x_i,t_{n+\frac{1}{2}})\Delta t^2+\mathcal{O}(\Delta t^2\Delta x^2)$$

The time derivative is analyzed using (B.5)-(B.6):

$$[D_t u]_i^{n+\frac{1}{2}} = u_{\mathrm{e},t}(x_i, t_{n+\frac{1}{2}}) + \frac{1}{24} u_{\mathrm{e},ttt}(x_i, t_{n+\frac{1}{2}})\Delta t^2 + \mathcal{O}(\Delta t^4)\,.$$

Summing up all the contributions and notifying that

$$u_{\mathrm{e},t}(x_i, t_{n+\frac{1}{2}}) = \alpha u_{\mathrm{e},xx}(x_i, t_{n+\frac{1}{2}}) + f(x_i, t_{n+\frac{1}{2}}),$$

the truncation error is given by

$$R_i^{n+\frac{1}{2}} = \frac{1}{8} u_{\mathrm{e},xx}(x_i, t_{n+\frac{1}{2}})\Delta t^2 + \frac{1}{12} u_{\mathrm{e},xxxx}(x_i, t_{n+\frac{1}{2}})\Delta x^2 + \frac{1}{24} u_{\mathrm{e},ttt}(x_i, t_{n+\frac{1}{2}})\Delta t^2 + + \mathcal{O}(\Delta x^4) + \mathcal{O}(\Delta t^4) + \mathcal{O}(\Delta t^2 \Delta x^2)$$

B.6.2 Nonlinear diffusion equation in 1D

We address the PDE

$$\frac{\partial u}{\partial t} = \frac{\partial}{\partial x}\left(\alpha(u)\frac{\partial u}{\partial x}\right) + f(u),$$

with two potentially nonlinear coefficients $q(u)$ and $\alpha(u)$. We use a Backward Euler scheme with arithmetic mean for $\alpha(u)$,

$$[D^- u = D_x \overline{\alpha(u)}^x D_x u + f(u)]_i^n\,.$$

Inserting u_e defines the truncation error R:

$$[D^- u_\mathrm{e} = D_x \overline{\alpha(u_\mathrm{e})}^x D_x u_\mathrm{e} + f(u_\mathrm{e}) + R]_i^n\,.$$

The most computationally challenging part is the variable coefficient with $\alpha(u)$, but we can use the same setup as in Section B.5.3 and arrive at a truncation error $\mathcal{O}(\Delta x^2)$ for the x-derivative term. The nonlinear term $[f(u_\mathrm{e})]_i^n = f(u_\mathrm{e}(x_i, t_n))$ matches x and t derivatives of u_e in the PDE. We end up with

$$R_i^n = -\frac{1}{2}\frac{\partial^2}{\partial t^2} u_\mathrm{e}(x_i, t_n)\Delta t + \mathcal{O}(\Delta x^2)\,.$$

B.7 Exercises

Exercise B.1: Truncation error of a weighted mean

Derive the truncation error of the weighted mean in (B.19)-(B.20).

Hint. Expand u_{e}^{n+1} and u_{e}^{n} around $t_{n+\theta}$.
Filename: `trunc_weighted_mean`.

Exercise B.2: Simulate the error of a weighted mean

We consider the weighted mean

$$u_{\mathrm{e}}(t_n) \approx \theta u_{\mathrm{e}}^{n+1} + (1-\theta)u_{\mathrm{e}}^{n}\,.$$

Choose some specific function for $u_{\mathrm{e}}(t)$ and compute the error in this approximation for a sequence of decreasing $\Delta t = t_{n+1} - t_n$ and for $\theta = 0, 0.25, 0.5, 0.75, 1$. Assuming that the error equals $C\Delta t^r$, for some constants C and r, compute r for the two smallest Δt values for each choice of θ and compare with the truncation error (B.19)-(B.20). Filename: `trunc_theta_avg`.

Exercise B.3: Verify a truncation error formula

Set up a numerical experiment as explained in Section B.3.5 for verifying the formulas (B.15)-(B.16). Filename: `trunc_backward_2level`.

Problem B.4: Truncation error of the Backward Euler scheme

Derive the truncation error of the Backward Euler scheme for the decay ODE $u' = -au$ with constant a. Extend the analysis to cover the variable-coefficient case $u' = -a(t)u + b(t)$. Filename: `trunc_decay_BE`.

Exercise B.5: Empirical estimation of truncation errors

Use the ideas and tools from Section B.3.5 to estimate the rate of the truncation error of the Backward Euler and Crank-Nicolson schemes applied to the exponential decay model $u' = -au$, $u(0) = I$.

Hint. In the Backward Euler scheme, the truncation error can be estimated at mesh points $n = 1, \ldots, N$, while the truncation error must be estimated at midpoints $t_{n+\frac{1}{2}}$, $n = 0, \ldots, N-1$ for the Crank-Nicolson scheme. The `truncation_error(dt, N)` function to be supplied to the `estimate` function needs to carefully implement these details and return the right `t` array such that `t[i]` is the time point corresponding to the quantities `R[i]` and `R_a[i]`.
Filename: `trunc_decay_BNCN`.

Exercise B.6: Correction term for a Backward Euler scheme

Consider the model $u' = -au$, $u(0) = I$. Use the ideas of Section B.3.6 to add a correction term to the ODE such that the Backward Euler scheme applied to the perturbed ODE problem is of second order in Δt. Find the amplification factor. Filename: `trunc_decay_BE_corr`.

Problem B.7: Verify the effect of correction terms

Make a program that solves $u' = -au$, $u(0) = I$, by the θ-rule and computes convergence rates. Adjust a such that it incorporates correction terms. Run the program to verify that the error from the Forward and Backward Euler schemes with perturbed a is $\mathcal{O}(\Delta t^2)$, while the error arising from the Crank-Nicolson scheme with perturbed a is $\mathcal{O}(\Delta t^4)$. Filename: `trunc_decay_corr_verify`.

Problem B.8: Truncation error of the Crank-Nicolson scheme

The variable-coefficient ODE $u' = -a(t)u + b(t)$ can be discretized in two different ways by the Crank-Nicolson scheme, depending on whether we use averages for a and b or compute them at the midpoint $t_{n+\frac{1}{2}}$:

$$[D_t u = -a\overline{u}^t + b]^{n+\frac{1}{2}}, \tag{B.86}$$

$$[D_t u = \overline{-au + b}^t]^{n+\frac{1}{2}}\,. \tag{B.87}$$

Compute the truncation error in both cases. Filename: `trunc_decay_CN_vc`.

Problem B.9: Truncation error of $u' = f(u,t)$

Consider the general nonlinear first-order scalar ODE

$$u'(t) = f(u(t), t)\,.$$

Show that the truncation error in the Forward Euler scheme,

$$[D_t^+ u = f(u,t)]^n,$$

and in the Backward Euler scheme,

$$[D_t^- u = f(u,t)]^n,$$

both are of first order, regardless of what f is.

Showing the order of the truncation error in the Crank-Nicolson scheme,

$$[D_t u = f(u,t)]^{n+\frac{1}{2}},$$

is somewhat more involved: Taylor expand u_{e}^n, u_{e}^{n+1}, $f(u_{\mathrm{e}}^n, t_n)$, and $f(u_{\mathrm{e}}^{n+1}, t_{n+1})$ around $t_{n+\frac{1}{2}}$, and use that

$$\frac{df}{dt} = \frac{\partial f}{\partial u}u' + \frac{\partial f}{\partial t}\,.$$

Check that the derived truncation error is consistent with previous results for the case $f(u,t) = -au$. Filename: `trunc_nonlinear_ODE`.

Exercise B.10: Truncation error of $[D_t D_t u]^n$

Derive the truncation error of the finite difference approximation (B.17)-(B.18) to the second-order derivative. Filename: `trunc_d2u`.

Exercise B.11: Investigate the impact of approximating $u'(0)$

Section B.4.1 describes two ways of discretizing the initial condition $u'(0) = V$ for a vibration model $u'' + \omega^2 u = 0$: a centered difference $[D_{2t}u = V]^0$ or a forward difference $[D_t^+ u = V]^0$. The program `vib_undamped.py` solves $u'' + \omega^2 u = 0$ with $[D_{2t}u = 0]^0$ and features a function `convergence_rates` for computing the order of the error in the numerical solution. Modify this program such that it applies the forward difference $[D_t^+ u = 0]^0$ and report how this simpler and more

convenient approximation impacts the overall convergence rate of the scheme. Filename: `trunc_vib_ic_fw`.

Problem B.12: Investigate the accuracy of a simplified scheme

Consider the ODE

$$mu'' + \beta|u'|u' + s(u) = F(t)\,.$$

The term $|u'|u'$ quickly gives rise to nonlinearities and complicates the scheme. Why not simply apply a backward difference to this term such that it only involves known values? That is, we propose to solve

$$[mD_tD_tu + \beta|D_t^-u|D_t^-u + s(u) = F]^n\,.$$

Drop the absolute value for simplicity and find the truncation error of the scheme. Perform numerical experiments with the scheme and compared with the one based on centered differences. Can you illustrate the accuracy loss visually in real computations, or is the asymptotic analysis here mainly of theoretical interest? Filename: `trunc_vib_bw_damping`.

Software engineering; wave equation model

C

C.1 A 1D wave equation simulator

C.1.1 Mathematical model

Let u_t, u_{tt}, u_x, u_{xx} denote derivatives of u with respect to the subscript, i.e., u_{tt} is a second-order time derivative and u_x is a first-order space derivative. The initial-boundary value problem implemented in the `wave1D_dn_vc.py` code is

$$u_{tt} = (q(x)u_x)_x + f(x,t), \qquad x \in (0,L),\ t \in (0,T] \tag{C.1}$$
$$u(x,0) = I(x), \qquad x \in [0,L] \tag{C.2}$$
$$u_t(x,0) = V(t), \qquad x \in [0,L] \tag{C.3}$$
$$u(0,t) = U_0(t) \text{ or } u_x(0,t) = 0, \qquad t \in (0,T] \tag{C.4}$$
$$u(L,t) = U_L(t) \text{ or } u_x(L,t) = 0, \qquad t \in (0,T] \tag{C.5}$$

We allow variable wave velocity $c^2(x) = q(x)$, and Dirichlet or homogeneous Neumann conditions at the boundaries.

C.1.2 Numerical discretization

The PDE is discretized by second-order finite differences in time and space, with arithmetic mean for the variable coefficient

© 2017, Hans Petter Langtangen, Svein Linge. Released under CC Attribution 4.0 license

$$[D_t D_t u = D_x \overline{q}^x D_x u + f]_i^n \,. \tag{C.6}$$

The Neumann boundary conditions are discretized by

$$[D_{2x} u]_i^n = 0,$$

at a boundary point i. The details of how the numerical scheme is worked out are described in Sections 2.6 and 2.7.

C.1.3 A solver function

The general initial-boundary value problem (C.1)-(C.5) solved by finite difference methods can be implemented as shown in the following `solver` function (taken from the file `wave1D_dn_vc.py`). This function builds on simpler versions described in Sections 2.3, 2.4 2.6, and 2.7. There are several quite advanced constructs that will be commented upon later. The code is lengthy, but that is because we provide a lot of flexibility with respect to input arguments, boundary conditions, and optimization (scalar versus vectorized loops).

```
def solver(
    I, V, f, c, U_0, U_L, L, dt, C, T,
    user_action=None, version='scalar',
    stability_safety_factor=1.0):
    """Solve u_tt=(c^2*u_x)_x + f on (0,L)x(0,T]."""

    # --- Compute time and space mesh ---
    Nt = int(round(T/dt))
    t = np.linspace(0, Nt*dt, Nt+1)      # Mesh points in time

    # Find max(c) using a fake mesh and adapt dx to C and dt
    if isinstance(c, (float,int)):
        c_max = c
    elif callable(c):
        c_max = max([c(x_) for x_ in np.linspace(0, L, 101)])
    dx = dt*c_max/(stability_safety_factor*C)
    Nx = int(round(L/dx))
    x = np.linspace(0, L, Nx+1)          # Mesh points in space
    # Make sure dx and dt are compatible with x and t
    dx = x[1] - x[0]
    dt = t[1] - t[0]

    # Make c(x) available as array
    if isinstance(c, (float,int)):
        c = np.zeros(x.shape) + c
    elif callable(c):
        # Call c(x) and fill array c
        c_ = np.zeros(x.shape)
```

```
        for i in range(Nx+1):
            c_[i] = c(x[i])
        c = c_

    q = c**2
    C2 = (dt/dx)**2; dt2 = dt*dt    # Help variables in the scheme

    # --- Wrap user-given f, I, V, U_0, U_L if None or 0 ---
    if f is None or f == 0:
        f = (lambda x, t: 0) if version == 'scalar' else \
            lambda x, t: np.zeros(x.shape)
    if I is None or I == 0:
        I = (lambda x: 0) if version == 'scalar' else \
            lambda x: np.zeros(x.shape)
    if V is None or V == 0:
        V = (lambda x: 0) if version == 'scalar' else \
            lambda x: np.zeros(x.shape)
    if U_0 is not None:
        if isinstance(U_0, (float,int)) and U_0 == 0:
            U_0 = lambda t: 0
    if U_L is not None:
        if isinstance(U_L, (float,int)) and U_L == 0:
            U_L = lambda t: 0

    # --- Make hash of all input data ---
    import hashlib, inspect
    data = inspect.getsource(I) + '_' + inspect.getsource(V) + \
           '_' + inspect.getsource(f) + '_' + str(c) + '_' + \
           ('None' if U_0 is None else inspect.getsource(U_0)) + \
           ('None' if U_L is None else inspect.getsource(U_L)) + \
           '_' + str(L) + str(dt) + '_' + str(C) + '_' + str(T) + \
           '_' + str(stability_safety_factor)
    hashed_input = hashlib.sha1(data).hexdigest()
    if os.path.isfile('.' + hashed_input + '_archive.npz'):
        # Simulation is already run
        return -1, hashed_input

    # --- Allocate memomry for solutions ---
    u     = np.zeros(Nx+1)   # Solution array at new time level
    u_n   = np.zeros(Nx+1)   # Solution at 1 time level back
    u_nm1 = np.zeros(Nx+1)   # Solution at 2 time levels back

    import time;  t0 = time.clock()  # CPU time measurement

    # --- Valid indices for space and time mesh ---
    Ix = range(0, Nx+1)
    It = range(0, Nt+1)

    # --- Load initial condition into u_n ---
    for i in range(0,Nx+1):
        u_n[i] = I(x[i])

    if user_action is not None:
        user_action(u_n, x, t, 0)
```

```
    # --- Special formula for the first step ---
    for i in Ix[1:-1]:
        u[i] = u_n[i] + dt*V(x[i]) + \
        0.5*C2*(0.5*(q[i] + q[i+1])*(u_n[i+1] - u_n[i]) - \
                0.5*(q[i] + q[i-1])*(u_n[i] - u_n[i-1])) + \
        0.5*dt2*f(x[i], t[0])

    i = Ix[0]
    if U_0 is None:
        # Set boundary values (x=0: i-1 -> i+1 since u[i-1]=u[i+1]
        # when du/dn = 0, on x=L: i+1 -> i-1 since u[i+1]=u[i-1])
        ip1 = i+1
        im1 = ip1  # i-1 -> i+1
        u[i] = u_n[i] + dt*V(x[i]) + \
               0.5*C2*(0.5*(q[i] + q[ip1])*(u_n[ip1] - u_n[i])  - \
                       0.5*(q[i] + q[im1])*(u_n[i] - u_n[im1])) + \
        0.5*dt2*f(x[i], t[0])
    else:
        u[i] = U_0(dt)

    i = Ix[-1]
    if U_L is None:
        im1 = i-1
        ip1 = im1  # i+1 -> i-1
        u[i] = u_n[i] + dt*V(x[i]) + \
               0.5*C2*(0.5*(q[i] + q[ip1])*(u_n[ip1] - u_n[i])  - \
                       0.5*(q[i] + q[im1])*(u_n[i] - u_n[im1])) + \
        0.5*dt2*f(x[i], t[0])
    else:
        u[i] = U_L(dt)

    if user_action is not None:
        user_action(u, x, t, 1)

    # Update data structures for next step
    #u_nm1[:] = u_n;  u_n[:] = u  # safe, but slower
    u_nm1, u_n, u = u_n, u, u_nm1

    # --- Time loop ---
    for n in It[1:-1]:
        # Update all inner points
        if version == 'scalar':
            for i in Ix[1:-1]:
                u[i] = - u_nm1[i] + 2*u_n[i] + \
                    C2*(0.5*(q[i] + q[i+1])*(u_n[i+1] - u_n[i])  - \
                        0.5*(q[i] + q[i-1])*(u_n[i] - u_n[i-1])) + \
                dt2*f(x[i], t[n])

        elif version == 'vectorized':
            u[1:-1] = - u_nm1[1:-1] + 2*u_n[1:-1] + \
            C2*(0.5*(q[1:-1] + q[2:])*(u_n[2:] - u_n[1:-1]) -
                0.5*(q[1:-1] + q[:-2])*(u_n[1:-1] - u_n[:-2])) + \
            dt2*f(x[1:-1], t[n])
```

```
        else:
            raise ValueError('version=%s' % version)

        # Insert boundary conditions
        i = Ix[0]
        if U_0 is None:
            # Set boundary values
            # x=0: i-1 -> i+1 since u[i-1]=u[i+1] when du/dn=0
            # x=L: i+1 -> i-1 since u[i+1]=u[i-1] when du/dn=0
            ip1 = i+1
            im1 = ip1
            u[i] = - u_nm1[i] + 2*u_n[i] + \
                   C2*(0.5*(q[i] + q[ip1])*(u_n[ip1] - u_n[i])  - \
                       0.5*(q[i] + q[im1])*(u_n[i] - u_n[im1])) + \
            dt2*f(x[i], t[n])
        else:
            u[i] = U_0(t[n+1])

        i = Ix[-1]
        if U_L is None:
            im1 = i-1
            ip1 = im1
            u[i] = - u_nm1[i] + 2*u_n[i] + \
                   C2*(0.5*(q[i] + q[ip1])*(u_n[ip1] - u_n[i])  - \
                       0.5*(q[i] + q[im1])*(u_n[i] - u_n[im1])) + \
            dt2*f(x[i], t[n])
        else:
            u[i] = U_L(t[n+1])

        if user_action is not None:
            if user_action(u, x, t, n+1):
                break

        # Update data structures for next step
        u_nm1, u_n, u = u_n, u, u_nm1

    cpu_time = time.clock() - t0
    return cpu_time, hashed_input
```

C.2 Saving large arrays in files

Numerical simulations produce large arrays as results and the software needs to store these arrays on disk. Several methods are available in Python. We recommend to use tailored solutions for large arrays and not standard file storage tools such as `pickle` (`cPickle` for speed in Python version 2) and `shelve`, because the tailored solutions have been optimized for array data and are hence much faster than the standard tools.

C.2.1 Using `savez` to store arrays in files

Storing individual arrays. The `numpy.storez` function can store a set of arrays to a named file in a zip archive. An associated function `numpy.load` can be used to read the file later. Basically, we call `numpy.storez(filename, **kwargs)`, where `kwargs` is a dictionary containing array names as keys and the corresponding array objects as values. Very often, the solution at a time point is given a natural name where the name of the variable and the time level counter are combined, e.g., `u11` or `v39`. Suppose `n` is the time level counter and we have two solution arrays, `u` and `v`, that we want to save to a zip archive. The appropriate code is

```
import numpy as np
u_name = 'u%04d' % n   # array name
v_name = 'v%04d' % n   # array name
kwargs = {u_name: u, v_name: v}   # keyword args for savez
fname = '.mydata%04d.dat' % n
np.savez(fname, **kwargs)
if n == 0:            # store x once
    np.savez('.mydata_x.dat', x=x)
```

Since the name of the array must be given as a keyword argument to `savez`, and the name must be constructed as shown, it becomes a little tricky to do the call, but with a dictionary `kwargs` and `**kwargs`, which sends each key-value pair as individual keyword arguments, the task gets accomplished.

Merging zip archives. Each separate call to `np.savez` creates a new file (zip archive) with extension `.npz`. It is very convenient to collect all results in one archive instead. This can be done by merging all the individual `.npz` files into a single zip archive:

```
def merge_zip_archives(individual_archives, archive_name):
    """
    Merge individual zip archives made with numpy.savez into
    one archive with name archive_name.
    The individual archives can be given as a list of names
    or as a Unix wild chard filename expression for glob.glob.
    The result of this function is that all the individual
    archives are deleted and the new single archive made.
    """
    import zipfile
    archive = zipfile.ZipFile(
        archive_name, 'w', zipfile.ZIP_DEFLATED,
        allowZip64=True)
    if isinstance(individual_archives, (list,tuple)):
        filenames = individual_archives
    elif isinstance(individual_archives, str):
```

```
        filenames = glob.glob(individual_archives)

    # Open each archive and write to the common archive
    for filename in filenames:
        f = zipfile.ZipFile(filename,  'r',
                            zipfile.ZIP_DEFLATED)
        for name in f.namelist():
            data = f.open(name, 'r')
            # Save under name without .npy
            archive.writestr(name[:-4], data.read())
        f.close()
        os.remove(filename)
    archive.close()
```

Here we remark that `savez` automatically adds the `.npz` extension to the names of the arrays we store. We do not want this extension in the final archive.

Reading arrays from zip archives. Archives created by `savez` or the merged archive we describe above with name of the form `myarchive.npz`, can be conveniently read by the `numpy.load` function:

```
import numpy as np
array_names = np.load(‘myarchive.npz‘)
for array_name in array_names:
    # array_names[array_name] is the array itself
    # e.g. plot(array_names['t'], array_names[array_name])
```

C.2.2 Using `joblib` to store arrays in files

The Python package `joblib` has nice functionality for efficient storage of arrays on disk. The following class applies this functionality so that one can save an array, or in fact any Python data structure (e.g., a dictionary of arrays), to disk under a certain name. Later, we can retrieve the object by use of its name. The name of the directory under which the arrays are stored by `joblib` can be given by the user.

```
class Storage(object):
    """
    Store large data structures (e.g. numpy arrays) efficiently
    using joblib.

    Use:

    >>> from Storage import Storage
    >>> storage = Storage(cachedir='tmp_u01', verbose=1)
    >>> import numpy as np
    >>> a = np.linspace(0, 1, 100000) # large array
```

```
    >>> b = np.linspace(0, 1, 100000) # large array
    >>> storage.save('a', a)
    >>> storage.save('b', b)
    >>> # later
    >>> a = storage.retrieve('a')
    >>> b = storage.retrieve('b')
    """
    def __init__(self, cachedir='tmp', verbose=1):
        """
        Parameters
        ----------
        cachedir: str
             Name of directory where objects are stored in files.
        verbose: bool, int
             Let joblib and this class speak when storing files
             to disk.
        """
        import joblib
        self.memory = joblib.Memory(cachedir=cachedir,
                                    verbose=verbose)
        self.verbose = verbose
        self.retrieve = self.memory.cache(
            self.retrieve, ignore=['data'])
        self.save = self.retrieve

    def retrieve(self, name, data=None):
        if self.verbose > 0:
            print 'joblib save of', name
        return data
```

The `retrive` and `save` functions, which do the work, seem quite magic. The idea is that `joblib` looks at the `name` parameter and saves the return value `data` to disk if the `name` parameter has not been used in a previous call. Otherwise, if `name` is already registered, `joblib` fetches the `data` object from file and returns it (this is an example of a memoize function, see Section 2.1.4 in [11] for a brief explanation]).

C.2.3 Using a hash to create a file or directory name

Array storage techniques like those outlined in Sections C.2.2 and C.2.1 demand the user to assign a name for the file(s) or directory where the solution is to be stored. Ideally, this name should reflect parameters in the problem such that one can recognize an already run simulation. One technique is to make a hash string out of the input data. A hash string is a 40-character long hexadecimal string that uniquely reflects another potentially much longer string. (You may be used to hash strings from

the Git version control system: every committed version of the files in Git is recognized by a hash string.)

Suppose you have some input data in the form of functions, `numpy` arrays, and other objects. To turn these input data into a string, we may grab the source code of the functions, use a very efficient hash method for potentially large arrays, and simply convert all other objects via `str` to a string representation. The final string, merging all input data, is then converted to an SHA1 hash string such that we represent the input with a 40-character long string.

```
def myfunction(func1, func2, array1, array2, obj1, obj2):
    # Convert arguments to hash
    import inspect, joblib, hashlib
    data = (inspect.getsource(func1),
            inspect.getsource(func2),
        joblib.hash(array1),
        joblib.hash(array2),
        str(obj1),
        str(obj2))
    hash_input = hashlib.sha1(data).hexdigest()
```

It is wise to use `joblib.hash` and not try to do a `str(array1)`, since that string can be *very* long, and `joblib.hash` is more efficient than `hashlib` when turning these data into a hash.

Remark: turning function objects into their source code is unreliable!

The idea of turning a function object into a string via its source code may look smart, but is not a completely reliable solution. Suppose we have some function

```
x0 = 0.1
f = lambda x: 0 if x <= x0 else 1
```

The source code will be `f = lambda x: 0 if x <= x0 else 1`, so if the calling code changes the value of `x0` (which `f` remembers - it is a closure), the source remains unchanged, the hash is the same, and the change in input data is unnoticed. Consequently, the technique above must be used with care. The user can always just remove the stored files in disk and thereby force a recomputation (provided the software applies a hash to test if a zip archive or `joblib` subdirectory exists, and if so, avoids recomputation).

C.3 Software for the 1D wave equation

We use `numpy.storez` to store the solution at each time level on disk. Such actions must be taken care of outside the `solver` function, more precisely in the `user_action` function that is called at every time level.

We have, in the `wave1D_dn_vc.py` code, implemented the `user_action` callback function as a class `PlotAndStoreSolution` with a `__call__(self, x, t, t, n)` method for the `user_action` function. Basically, `__call__` stores and plots the solution. The storage makes use of the `numpy.savez` function for saving a set of arrays to a zip archive. Here, in this callback function, we want to save one array, `u`. Since there will be many such arrays, we introduce the array names `'u%04d' % n` and closely related filenames. The usage of `numpy.savez` in `__call__` goes like this:

```
from numpy import savez
name = 'u%04d' % n   # array name
kwargs = {name: u}   # keyword args for savez
fname = '.' + self.filename + '_' + name + '.dat'
self.t.append(t[n])  # store corresponding time value
savez(fname, **kwargs)
if n == 0:           # store x once
    savez('.' + self.filename + '_x.dat', x=x)
```

For example, if `n` is 10 and `self.filename` is `tmp`, the above call to `savez` becomes `savez('.tmp_u0010.dat', u0010=u)`. The actual filename becomes `.tmp_u0010.dat.npz`. The actual array name becomes `u0010.npy`.

Each `savez` call results in a file, so after the simulation we have one file per time level. Each file produced by `savez` is a zip archive. It makes sense to merge all the files into one. This is done in the `close_file` method in the `PlotAndStoreSolution` class. The code goes as follows.

```
class PlotAndStoreSolution:
    ...
    def close_file(self, hashed_input):
        """
        Merge all files from savez calls into one archive.
        hashed_input is a string reflecting input data
        for this simulation (made by solver).
        """
        if self.filename is not None:
            # Save all the time points where solutions are saved
            savez('.' + self.filename + '_t.dat',
                  t=array(self.t, dtype=float))
            # Merge all savez files to one zip archive
            archive_name = '.' + hashed_input + '_archive.npz'
```

```
            filenames = glob.glob('.' + self.filename + '*.dat.npz')
            merge_zip_archives(filenames, archive_name)
```

We use various `ZipFile` functionality to extract the content of the individual files (each with name `filename`) and write it to the merged archive (`archive`). There is only one array in each individual file (`filename`) so strictly speaking, there is no need for the loop `for name in f.namelist()` (as `f.namelist()` returns a list of length 1). However, in other applications where we compute more arrays at each time level, `savez` will store all these and then there is need for iterating over `f.namelist()`.

Instead of merging the archives written by `savez` we could make an alternative implementation that writes all our arrays into one archive. This is the subject of Exercise C.9.

C.3.1 Making hash strings from input data

The `hashed_input` argument, used to name the resulting archive file with all solutions, is supposed to be a hash reflecting all import parameters in the problem such that this simulation has a unique name. The `hashed_input` string is made in the `solver` function, using the `hashlib` and `inspect` modules, based on the arguments to `solver`:

```
# Make hash of all input data
import hashlib, inspect
data = inspect.getsource(I) + '_' + inspect.getsource(V) + \
       '_' + inspect.getsource(f) + '_' + str(c) + '_' + \
       ('None' if U_0 is None else inspect.getsource(U_0)) + \
       ('None' if U_L is None else inspect.getsource(U_L)) + \
       '_' + str(L) + str(dt) + '_' + str(C) + '_' + str(T) + \
       '_' + str(stability_safety_factor)
hashed_input = hashlib.sha1(data).hexdigest()
```

To get the source code of a function `f` as a string, we use `inspect.getsource(f)`. All input, functions as well as variables, is then merged to a string `data`, and then `hashlib.sha1` makes a unique, much shorter (40 characters long), fixed-length string out of `data` that we can use in the archive filename.

Remark

Note that the construction of the `data` string is not fool proof: if, e.g., `I` is a formula with parameters and the parameters change, the

source code is still the same and `data` and hence the hash remains unaltered. The implementation must therefore be used with care!

C.3.2 Avoiding rerunning previously run cases

If the archive file whose name is based on `hashed_input` already exists, the simulation with the current set of parameters has been done before and one can avoid redoing the work. The `solver` function returns the CPU time and `hashed_input`, and a negative CPU time means that no simulation was run. In that case we should not call the `close_file` method above (otherwise we overwrite the archive with just the `self.t` array). The typical usage goes like

```
action = PlotAndStoreSolution(...)
dt = (L/Nx)/C  # choose the stability limit with given Nx
cpu, hashed_input = solver(
    I=lambda x: ...,
    V=0, f=0, c=1, U_0=lambda t: 0, U_L=None, L=1,
    dt=dt, C=C, T=T,
    user_action=action, version='vectorized',
    stability_safety_factor=1)
action.make_movie_file()
if cpu > 0:  # did we generate new data?
    action.close_file(hashed_input)
```

C.3.3 Verification

Vanishing approximation error. Exact solutions of the numerical equations are always attractive for verification purposes since the software should reproduce such solutions to machine precision. With Dirichlet boundary conditions we can construct a function that is linear in t and quadratic in x that is also an exact solution of the scheme, while with Neumann conditions we are left with testing just a constant solution (see comments in Section 2.6.5).

Convergence rates. A more general method for verification is to check the convergence rates. We must introduce one discretization parameter h and assume an error model $E = Ch^r$, where C and r are constants to be determine (i.e., r is the rate that we are interested in). Given two experiments with different resolutions h_i and h_i-1, we can estimate r by

$$r = \frac{\ln(E_i/E_{i-1})}{\ln(h_i/h_{i-1})},$$

where E_i is the error corresponding to h_i and E_{i-1} corresponds to h_{i-1}. Section 2.2.2 explains the details of this type of verification and how we introduce the single discretization parameter $h = \Delta t = \hat{c}\Delta t$, for some constant $\hat{c}$. To compute the error, we had to rely on a global variable in the user action function. Below is an implementation where we have a more elegant solution in terms of a class: the `error` variable is not a class attribute and there is no need for a global error (which is always considered an advantage).

```
def convergence_rates(
    u_exact,
    I, V, f, c, U_0, U_L, L,
    dt0, num_meshes,
    C, T, version='scalar',
    stability_safety_factor=1.0):
    """
    Half the time step and estimate convergence rates for
    for num_meshes simulations.
    """
    class ComputeError:
        def __init__(self, norm_type):
            self.error = 0

        def __call__(self, u, x, t, n):
            """Store norm of the error in self.E."""
            error = np.abs(u - u_exact(x, t[n])).max()
            self.error = max(self.error, error)

    E = []
    h = []  # dt, solver adjusts dx such that C=dt*c/dx
    dt = dt0
    for i in range(num_meshes):
        error_calculator = ComputeError('Linf')
        solver(I, V, f, c, U_0, U_L, L, dt, C, T,
               user_action=error_calculator,
               version='scalar',
               stability_safety_factor=1.0)
        E.append(error_calculator.error)
        h.append(dt)
        dt /= 2  # halve the time step for next simulation
    print 'E:', E
    print 'h:', h
    r = [np.log(E[i]/E[i-1])/np.log(h[i]/h[i-1])
         for i in range(1,num_meshes)]
    return r
```

The returned sequence `r` should converge to 2 since the error analysis in Section 2.10 predicts various error measures to behave like

$\mathcal{O}(\Delta t^2) + \mathcal{O}(\Delta x^2)$. We can easily run the case with standing waves and the analytical solution $u(x,t) = \cos(\frac{2\pi}{L}t)\sin(\frac{2\pi}{L}x)$. The call will be very similar to the one provided in the `test_convrate_sincos` function in Section 2.3.4, see the file `wave1D_dn_vc.py` for details.

C.4 Programming the solver with classes

Many who know about class programming prefer to organize their software in terms of classes. This gives a richer application programming interface (API) since a function solver must have all its input data in terms of arguments, while a class-based solver naturally has a mix of method arguments and user-supplied methods. (Well, to be more precise, our solvers have demanded `user_action` to be a function provided by the user, so it is possible to mix variables and functions in the input also with a solver function.)

We will next illustrate how some of the functionality in `wave1D_dn_vc.py` may be implemented by using classes. Focusing on class implementation aspects, we restrict the example case to a simpler wave with constant wave speed c. Applying the method of manufactured solutions, we test whether the class based implementation is able to compute the known exact solution within machine precision.

We will create a class `Problem` to hold the physical parameters of the problem and a class `Solver` to hold the numerical solution parameters besides the solver function itself. As the number of parameters increases, so does the amount of repetitive code. We therefore take the opportunity to illustrate how this may be counteracted by introducing a super class `Parameters` that allows code to be parameterized. In addition, it is convenient to collect the arrays that describe the mesh in a special `Mesh` class and make a class `Function` for a mesh function (mesh point values and its mesh). All the following code is found in `wave1D_oo.py`.

C.4.1 Class Parameters

The classes `Problem` and `Solver` both inherit class `Parameters`, which handles reading of parameters from the command line and has methods for setting and getting parameter values. Since processing dictionaries is easier than processing a collection of individual attributes, the class `Parameters` requires each class `Problem` and `Solver` to represent their

parameters by dictionaries, one compulsory and two optional ones. The compulsory dictionary, `self.prm`, contains all parameters, while a second and optional dictionary, `self.type`, holds the associated object types, and a third and optional dictionary, `self.help`, stores help strings. The `Parameters` class may be implemented as follows:

```
class Parameters(object):
    def __init__(self):
        """
        Subclasses must initialize self.prm with
        parameters and default values, self.type with
        the corresponding types, and self.help with
        the corresponding descriptions of parameters.
        self.type and self.help are optional, but
        self.prms must be complete and contain all parameters.
        """
        pass

    def ok(self):
        """Check if attr. prm, type, and help are defined."""
        if hasattr(self, 'prm') and \
           isinstance(self.prm, dict) and \
           hasattr(self, 'type') and \
           isinstance(self.type, dict) and \
           hasattr(self, 'help') and \
           isinstance(self.help, dict):
            return True
        else:
            raise ValueError(
                'The constructor in class %s does not '\
                'initialize the\ndictionaries '\
                'self.prm, self.type, self.help!' %
                self.__class__.__name__)

    def _illegal_parameter(self, name):
        """Raise exception about illegal parameter name."""
        raise ValueError(
            'parameter "%s" is not registered.\nLegal '\
            'parameters are\n%s' %
            (name, ' '.join(list(self.prm.keys()))))

    def set(self, **parameters):
        """Set one or more parameters."""
        for name in parameters:
            if name in self.prm:
                self.prm[name] = parameters[name]
            else:
                self._illegal_parameter(name)

    def get(self, name):
        """Get one or more parameter values."""
        if isinstance(name, (list,tuple)):   # get many?
```

```
            for n in name:
                if n not in self.prm:
                    self._illegal_parameter(name)
            return [self.prm[n] for n in name]
        else:
            if name not in self.prm:
                self._illegal_parameter(name)
            return self.prm[name]

    def __getitem__(self, name):
        """Allow obj[name] indexing to look up a parameter."""
        return self.get(name)

    def __setitem__(self, name, value):
        """
        Allow obj[name] = value syntax to assign a parameter's value.
        """
        return self.set(name=value)

    def define_command_line_options(self, parser=None):
        self.ok()
        if parser is None:
            import argparse
            parser = argparse.ArgumentParser()

        for name in self.prm:
            tp = self.type[name] if name in self.type else str
            help = self.help[name] if name in self.help else None
            parser.add_argument(
                '--' + name, default=self.get(name), metavar=name,
                type=tp, help=help)

        return parser

    def init_from_command_line(self, args):
        for name in self.prm:
            self.prm[name] = getattr(args, name)
```

C.4.2 Class Problem

Inheriting the `Parameters` class, our class `Problem` is defined as:

```
class Problem(Parameters):
    """
    Physical parameters for the wave equation
    u_tt = (c**2*u_x)_x + f(x,t) with t in [0,T] and
    x in (0,L). The problem definition is implied by
    the method of manufactured solution, choosing
    u(x,t)=x(L-x)(1+t/2) as our solution. This solution
    should be exactly reproduced when c is const.
    """
```

```
    def __init__(self):
        self.prm  = dict(L=2.5, c=1.5, T=18)
        self.type = dict(L=float, c=float, T=float)
        self.help = dict(L='1D domain',
                         c='coefficient (wave velocity) in PDE',
                         T='end time of simulation')
    def u_exact(self, x, t):
        L = self['L']
        return x*(L-x)*(1+0.5*t)
    def I(self, x):
        return self.u_exact(x, 0)
    def V(self, x):
        return 0.5*self.u_exact(x, 0)
    def f(self, x, t):
        c = self['c']
        return 2*(1+0.5*t)*c**2
    def U_0(self, t):
        return self.u_exact(0, t)
    U_L = None
```

C.4.3 Class Mesh

The `Mesh` class can be made valid for a space-time mesh in any number of space dimensions. To make the class versatile, the constructor accepts either a tuple/list of number of cells in each spatial dimension or a tuple/list of cell spacings. In addition, we need the size of the hypercube mesh as a tuple/list of 2-tuples with lower and upper limits of the mesh coordinates in each direction. For 1D meshes it is more natural to just write the number of cells or the cell size and not wrap it in a list. We also need the time interval from `t0` to `T`. Giving no spatial discretization information implies a time mesh only, and vice versa. The `Mesh` class with documentation and a doc test should now be self-explanatory:

```
import numpy as np

class Mesh(object):
    """
    Holds data structures for a uniform mesh on a hypercube in
    space, plus a uniform mesh in time.

    ======== ==================================================
    Argument                Explanation
    ======== ==================================================
    L        List of 2-lists of min and max coordinates
             in each spatial direction.
    T        Final time in time mesh.
    Nt       Number of cells in time mesh.
```

```
    dt       Time step. Either Nt or dt must be given.
    N        List of number of cells in the spatial directions.
    d        List of cell sizes in the spatial directions.
             Either N or d must be given.
    ======== ================================================

    Users can access all the parameters mentioned above, plus
    ``x[i]`` and ``t`` for the coordinates in direction ``i``
    and the time coordinates, respectively.

    Examples:

    >>> from UniformFDMesh import Mesh
    >>>
    >>> # Simple space mesh
    >>> m = Mesh(L=[0,1], N=4)
    >>> print m.dump()
    space: [0,1] N=4 d=0.25
    >>>
    >>> # Simple time mesh
    >>> m = Mesh(T=4, dt=0.5)
    >>> print m.dump()
    time: [0,4] Nt=8 dt=0.5
    >>>
    >>> # 2D space mesh
    >>> m = Mesh(L=[[0,1], [-1,1]], d=[0.5, 1])
    >>> print m.dump()
    space: [0,1]x[-1,1] N=2x2 d=0.5,1
    >>>
    >>> # 2D space mesh and time mesh
    >>> m = Mesh(L=[[0,1], [-1,1]], d=[0.5, 1], Nt=10, T=3)
    >>> print m.dump()
    space: [0,1]x[-1,1] N=2x2 d=0.5,1 time: [0,3] Nt=10 dt=0.3

    """
    def __init__(self,
                 L=None, T=None, t0=0,
                 N=None, d=None,
                 Nt=None, dt=None):
        if N is None and d is None:
            # No spatial mesh
            if Nt is None and dt is None:
                raise ValueError(
                'Mesh constructor: either Nt or dt must be given')
            if T is None:
                raise ValueError(
                'Mesh constructor: T must be given')
        if Nt is None and dt is None:
            if N is None and d is None:
                raise ValueError(
                'Mesh constructor: either N or d must be given')
            if L is None:
                raise ValueError(
                'Mesh constructor: L must be given')
```

```
        # Allow 1D interface without nested lists with one element
        if L is not None and isinstance(L[0], (float,int)):
            # Only an interval was given
            L = [L]
        if N is not None and isinstance(N, (float,int)):
            N = [N]
        if d is not None and isinstance(d, (float,int)):
            d = [d]

        # Set all attributes to None
        self.x = None
        self.t = None
        self.Nt = None
        self.dt = None
        self.N = None
        self.d = None
        self.t0 = t0

        if N is None and d is not None and L is not None:
            self.L = L
            if len(d) != len(L):
                raise ValueError(
                    'd has different size (no of space dim.) from '
                    'L: %d vs %d', len(d), len(L))
            self.d = d
            self.N = [int(round(float(self.L[i][1] -
                                      self.L[i][0])/d[i]))
                      for i in range(len(d))]
        if d is None and N is not None and L is not None:
            self.L = L
            if len(N) != len(L):
                raise ValueError(
                    'N has different size (no of space dim.) from '
                    'L: %d vs %d', len(N), len(L))
            self.N = N
            self.d = [float(self.L[i][1] - self.L[i][0])/N[i]
                      for i in range(len(N))]

        if Nt is None and dt is not None and T is not None:
            self.T = T
            self.dt = dt
            self.Nt = int(round(T/dt))
        if dt is None and Nt is not None and T is not None:
            self.T = T
            self.Nt = Nt
            self.dt = T/float(Nt)

        if self.N is not None:
            self.x = [np.linspace(
                      self.L[i][0], self.L[i][1], self.N[i]+1)
                      for i in range(len(self.L))]
        if Nt is not None:
            self.t = np.linspace(self.t0, self.T, self.Nt+1)
```

```
    def get_num_space_dim(self):
        return len(self.d) if self.d is not None else 0

    def has_space(self):
        return self.d is not None

    def has_time(self):
        return self.dt is not None

    def dump(self):
        s = ''
        if self.has_space():
            s += 'space: ' + \
                 'x'.join(['[%g,%g]' % (self.L[i][0], self.L[i][1])
                           for i in range(len(self.L))]) + ' N='
            s += 'x'.join([str(Ni) for Ni in self.N]) + ' d='
            s += ','.join([str(di) for di in self.d])
        if self.has_space() and self.has_time():
            s += ' '
        if self.has_time():
            s += 'time: ' + '[%g,%g]' % (self.t0, self.T) + \
                 ' Nt=%g' % self.Nt + ' dt=%g' % self.dt
        return s
```

We rely on attribute access - not get/set functions!

Java programmers, in particular, are used to get/set functions in classes to access internal data. In Python, we usually apply direct access of the attribute, such as `m.N[i]` if `m` is a `Mesh` object. A widely used convention is to do this as long as access to an attribute does not require additional code. In that case, one applies a property construction. The original interface remains the same after a property is introduced (in contrast to Java), so user will not notice a change to properties.

The only argument against direct attribute access in class `Mesh` is that the attributes are read-only so we could avoid offering a set function. Instead, we rely on the user that she does not assign new values to the attributes.

C.4.4 Class Function

A class `Function` is handy to hold a mesh and corresponding values for a scalar or vector function over the mesh. Since we may have a time

or space mesh, or a combined time and space mesh, with one or more components in the function, some if tests are needed for allocating the right array sizes. To help the user, an `indices` attribute with the name of the indices in the final array `u` for the function values is made. The examples in the doc string should explain the functionality.

```
class Function(object):
    """
    A scalar or vector function over a mesh (of class Mesh).

    ========== ================================================
    Argument                        Explanation
    ========== ================================================
    mesh       Class Mesh object: spatial and/or temporal mesh.
    num_comp   Number of components in function (1 for scalar).
    space_only True if the function is defined on the space mesh
               only (to save space). False if function has values
               in space and time.
    ========== ================================================

    The indexing of ``u``, which holds the mesh point values of the
    function, depends on whether we have a space and/or time mesh.

    Examples:

    >>> from UniformFDMesh import Mesh, Function
    >>>
    >>> # Simple space mesh
    >>> m = Mesh(L=[0,1], N=4)
    >>> print m.dump()
    space: [0,1] N=4 d=0.25
    >>> f = Function(m)
    >>> f.indices
    ['x0']
    >>> f.u.shape
    (5,)
    >>> f.u[4]  # space point 4
    0.0
    >>>
    >>> # Simple time mesh for two components
    >>> m = Mesh(T=4, dt=0.5)
    >>> print m.dump()
    time: [0,4] Nt=8 dt=0.5
    >>> f = Function(m, num_comp=2)
    >>> f.indices
    ['time', 'component']
    >>> f.u.shape
    (9, 2)
    >>> f.u[3,1]  # time point 3, comp=1 (2nd comp.)
    0.0
    >>>
    >>> # 2D space mesh
```

```
    >>> m = Mesh(L=[[0,1], [-1,1]], d=[0.5, 1])
    >>> print m.dump()
    space: [0,1]x[-1,1] N=2x2 d=0.5,1
    >>> f = Function(m)
    >>> f.indices
    ['x0', 'x1']
    >>> f.u.shape
    (3, 3)
    >>> f.u[1,2]  # space point (1,2)
    0.0
    >>>
    >>> # 2D space mesh and time mesh
    >>> m = Mesh(L=[[0,1],[-1,1]], d=[0.5,1], Nt=10, T=3)
    >>> print m.dump()
    space: [0,1]x[-1,1] N=2x2 d=0.5,1 time: [0,3] Nt=10 dt=0.3
    >>> f = Function(m, num_comp=2, space_only=False)
    >>> f.indices
    ['time', 'x0', 'x1', 'component']
    >>> f.u.shape
    (11, 3, 3, 2)
    >>> f.u[2,1,2,0]  # time step 2, space point (1,2), comp=0
    0.0
    >>> # Function with space data only
    >>> f = Function(m, num_comp=1, space_only=True)
    >>> f.indices
    ['x0', 'x1']
    >>> f.u.shape
    (3, 3)
    >>> f.u[1,2]  # space point (1,2)
    0.0
    """
    def __init__(self, mesh, num_comp=1, space_only=True):
        self.mesh = mesh
        self.num_comp = num_comp
        self.indices = []

        # Create array(s) to store mesh point values
        if (self.mesh.has_space() and not self.mesh.has_time()) or \
           (self.mesh.has_space() and self.mesh.has_time() and \
            space_only):
            # Space mesh only
            if num_comp == 1:
                self.u = np.zeros(
                    [self.mesh.N[i] + 1
                     for i in range(len(self.mesh.N))])
                self.indices = [
                    'x'+str(i) for i in range(len(self.mesh.N))]
            else:
                self.u = np.zeros(
                    [self.mesh.N[i] + 1
                     for i in range(len(self.mesh.N))] +
                    [num_comp])
                self.indices = [
                    'x'+str(i)
```

```
                    for i in range(len(self.mesh.N))] +\
                    ['component']
        if not self.mesh.has_space() and self.mesh.has_time():
            # Time mesh only
            if num_comp == 1:
                self.u = np.zeros(self.mesh.Nt+1)
                self.indices = ['time']
            else:
                # Need num_comp entries per time step
                self.u = np.zeros((self.mesh.Nt+1, num_comp))
                self.indices = ['time', 'component']
        if self.mesh.has_space() and self.mesh.has_time() \
            and not space_only:
            # Space-time mesh
            size = [self.mesh.Nt+1] + \
                   [self.mesh.N[i]+1
                    for i in range(len(self.mesh.N))]
            if num_comp > 1:
                self.indices = ['time'] + \
                               ['x'+str(i)
                                for i in range(len(self.mesh.N))] +\
                               ['component']
                size += [num_comp]
            else:
                self.indices = ['time'] + ['x'+str(i)
                                for i in range(len(self.mesh.N))]
            self.u = np.zeros(size)
```

C.4.5 Class Solver

With the `Mesh` and `Function` classes in place, we can rewrite the `solver` function, but we make it a method in class `Solver`:

```
class Solver(Parameters):
    """
    Numerical parameters for solving the wave equation
    u_tt = (c**2*u_x)_x + f(x,t) with t in [0,T] and
    x in (0,L). The problem definition is implied by
    the method of manufactured solution, choosing
    u(x,t)=x(L-x)(1+t/2) as our solution. This solution
    should be exactly reproduced, provided c is const.
    We simulate in [0, L/2] and apply a symmetry condition
    at the end x=L/2.
    """

    def __init__(self, problem):
        self.problem = problem
        self.prm  = dict(C = 0.75, Nx=3, stability_safety_factor=1.0)
        self.type = dict(C=float, Nx=int, stability_safety_factor=float)
        self.help = dict(C='Courant number',
                         Nx='No of spatial mesh points',
```

```
                        stability_safety_factor='stability factor')

        from UniformFDMesh import Mesh, Function
        # introduce some local help variables to ease reading
        L_end = self.problem['L']
        dx = (L_end/2)/float(self['Nx'])
        t_interval = self.problem['T']
        dt = dx*self['stability_safety_factor']*self['C']/ \
                                    float(self.problem['c'])
        self.m = Mesh(L=[0,L_end/2],
                      d=[dx],
                      Nt = int(round(t_interval/float(dt))),
                      T=t_interval)
        # The mesh function f will, after solving, contain
        # the solution for the whole domain and all time steps.
        self.f = Function(self.m, num_comp=1, space_only=False)

    def solve(self, user_action=None, version='scalar'):
        # ...use local variables to ease reading
        L, c, T = self.problem['L c T'.split()]
        L = L/2     # compute with half the domain only (symmetry)
        C, Nx, stability_safety_factor = self[
                            'C Nx stability_safety_factor'.split()]
        dx = self.m.d[0]
        I = self.problem.I
        V = self.problem.V
        f = self.problem.f
        U_0 = self.problem.U_0
        U_L = self.problem.U_L
        Nt = self.m.Nt
        t = np.linspace(0, T, Nt+1)      # Mesh points in time
        x = np.linspace(0, L, Nx+1)      # Mesh points in space

        # Make sure dx and dt are compatible with x and t
        dx = x[1] - x[0]
        dt = t[1] - t[0]

        # Treat c(x) as array
        if isinstance(c, (float,int)):
            c = np.zeros(x.shape) + c
        elif callable(c):
            # Call c(x) and fill array c
            c_ = np.zeros(x.shape)
            for i in range(Nx+1):
                c_[i] = c(x[i])
            c = c_

        q = c**2
        C2 = (dt/dx)**2; dt2 = dt*dt    # Help variables in the scheme

        # Wrap user-given f, I, V, U_0, U_L if None or 0
        if f is None or f == 0:
            f = (lambda x, t: 0) if version == 'scalar' else \
                lambda x, t: np.zeros(x.shape)
```

```
        if I is None or I == 0:
            I = (lambda x: 0) if version == 'scalar' else \
                lambda x: np.zeros(x.shape)
        if V is None or V == 0:
            V = (lambda x: 0) if version == 'scalar' else \
                lambda x: np.zeros(x.shape)
        if U_0 is not None:
            if isinstance(U_0, (float,int)) and U_0 == 0:
                U_0 = lambda t: 0
        if U_L is not None:
            if isinstance(U_L, (float,int)) and U_L == 0:
                U_L = lambda t: 0

        # Make hash of all input data
        import hashlib, inspect
        data = inspect.getsource(I) + '_' + inspect.getsource(V) + \
               '_' + inspect.getsource(f) + '_' + str(c) + '_' + \
               ('None' if U_0 is None else inspect.getsource(U_0)) + \
               ('None' if U_L is None else inspect.getsource(U_L)) + \
               '_' + str(L) + str(dt) + '_' + str(C) + '_' + str(T) + \
               '_' + str(stability_safety_factor)
        hashed_input = hashlib.sha1(data).hexdigest()
        if os.path.isfile('.' + hashed_input + '_archive.npz'):
            # Simulation is already run
            return -1, hashed_input

        # use local variables to make code closer to mathematical
        # notation in computational scheme
        u_1 = self.f.u[0,:]
        u   = self.f.u[1,:]

        import time;  t0 = time.clock()  # CPU time measurement

        Ix = range(0, Nx+1)
        It = range(0, Nt+1)

        # Load initial condition into u_1
        for i in range(0,Nx+1):
            u_1[i] = I(x[i])

        if user_action is not None:
            user_action(u_1, x, t, 0)

        # Special formula for the first step
        for i in Ix[1:-1]:
            u[i] = u_1[i] + dt*V(x[i]) + \
            0.5*C2*(0.5*(q[i] + q[i+1])*(u_1[i+1] - u_1[i]) - \
                    0.5*(q[i] + q[i-1])*(u_1[i] - u_1[i-1])) + \
            0.5*dt2*f(x[i], t[0])

        i = Ix[0]
        if U_0 is None:
            # Set boundary values (x=0: i-1 -> i+1 since u[i-1]=u[i+1]
            # when du/dn = 0, on x=L: i+1 -> i-1 since u[i+1]=u[i-1])
```

```
            ip1 = i+1
            im1 = ip1  # i-1 -> i+1
            u[i] = u_1[i] + dt*V(x[i]) + \
                   0.5*C2*(0.5*(q[i] + q[ip1])*(u_1[ip1] - u_1[i])  - \
                           0.5*(q[i] + q[im1])*(u_1[i] - u_1[im1])) + \
            0.5*dt2*f(x[i], t[0])
        else:
            u[i] = U_0(dt)

        i = Ix[-1]
        if U_L is None:
            im1 = i-1
            ip1 = im1  # i+1 -> i-1
            u[i] = u_1[i] + dt*V(x[i]) + \
                   0.5*C2*(0.5*(q[i] + q[ip1])*(u_1[ip1] - u_1[i])  - \
                           0.5*(q[i] + q[im1])*(u_1[i] - u_1[im1])) + \
            0.5*dt2*f(x[i], t[0])
        else:
            u[i] = U_L(dt)

        if user_action is not None:
            user_action(u, x, t, 1)

        for n in It[1:-1]:
            # u corresponds to u^{n+1} in the mathematical scheme
            u_2 = self.f.u[n-1,:]
            u_1 = self.f.u[n,:]
            u   = self.f.u[n+1,:]

            # Update all inner points
            if version == 'scalar':
                for i in Ix[1:-1]:
                    u[i] = - u_2[i] + 2*u_1[i] + \
                        C2*(0.5*(q[i] + q[i+1])*(u_1[i+1] - u_1[i])  - \
                            0.5*(q[i] + q[i-1])*(u_1[i] - u_1[i-1])) + \
                    dt2*f(x[i], t[n])

            elif version == 'vectorized':
                u[1:-1] = - u_2[1:-1] + 2*u_1[1:-1] + \
                C2*(0.5*(q[1:-1] + q[2:])*(u_1[2:] - u_1[1:-1]) -
                    0.5*(q[1:-1] + q[:-2])*(u_1[1:-1] - u_1[:-2])) + \
                dt2*f(x[1:-1], t[n])
            else:
                raise ValueError('version=%s' % version)

            # Insert boundary conditions
            i = Ix[0]
            if U_0 is None:
                # Set boundary values
                # x=0: i-1 -> i+1 since u[i-1]=u[i+1] when du/dn=0
                # x=L: i+1 -> i-1 since u[i+1]=u[i-1] when du/dn=0
                ip1 = i+1
                im1 = ip1
                u[i] = - u_2[i] + 2*u_1[i] + \
```

```
                        C2*(0.5*(q[i] + q[ip1])*(u_1[ip1] - u_1[i])  - \
                            0.5*(q[i] + q[im1])*(u_1[i] - u_1[im1])) + \
                    dt2*f(x[i], t[n])
                else:
                    u[i] = U_0(t[n+1])

                i = Ix[-1]
                if U_L is None:
                    im1 = i-1
                    ip1 = im1
                    u[i] = - u_2[i] + 2*u_1[i] + \
                           C2*(0.5*(q[i] + q[ip1])*(u_1[ip1] - u_1[i])  - \
                               0.5*(q[i] + q[im1])*(u_1[i] - u_1[im1])) + \
                    dt2*f(x[i], t[n])
                else:
                    u[i] = U_L(t[n+1])

                if user_action is not None:
                    if user_action(u, x, t, n+1):
                        break

            cpu_time = time.clock() - t0
            return cpu_time, hashed_input

        def assert_no_error(self):
            """Run through mesh and check error"""
            Nx = self['Nx']
            Nt = self.m.Nt
            L, T = self.problem['L T'.split()]
            L = L/2     # only half the domain used (symmetry)
            x = np.linspace(0, L, Nx+1)     # Mesh points in space
            t = np.linspace(0, T, Nt+1)     # Mesh points in time

            for n in range(len(t)):
                u_e = self.problem.u_exact(x, t[n])
                diff = np.abs(self.f.u[n,:] - u_e).max()
                print 'diff:', diff
                tol = 1E-13
                assert diff < tol
```

Observe that the solutions from all time steps are stored in the mesh function, which allows error assessment (in `assert_no_error`) to take place after all solutions have been found. Of course, in 2D or 3D, such a strategy may place too high demands on available computer memory, in which case intermediate results could be stored on file.

Running `wave1D_oo.py` gives a printout showing that the class-based implementation performs as expected, i.e. that the known exact solution is reproduced (within machine precision).

C.5 Migrating loops to Cython

We now consider the `wave2D_u0.py` code for solving the 2D linear wave equation with constant wave velocity and homogeneous Dirichlet boundary conditions $u = 0$. We shall in the present chapter extend this code with computational modules written in other languages than Python. This extended version is called `wave2D_u0_adv.py`.

The `wave2D_u0.py` file contains a `solver` function, which calls an `advance_*` function to advance the numerical scheme one level forward in time. The function `advance_scalar` applies standard Python loops to implement the scheme, while `advance_vectorized` performs corresponding vectorized arithmetics with array slices. The statements of this solver are explained in Section 2.12, in particular Sections 2.12.1 and 2.12.2.

Although vectorization can bring down the CPU time dramatically compared with scalar code, there is still some factor 5-10 to win in these types of applications by implementing the finite difference scheme in compiled code, typically in Fortran, C, or C++. This can quite easily be done by adding a little extra code to our program. Cython is an extension of Python that offers the easiest way to nail our Python loops in the scalar code down to machine code and achieve the efficiency of C.

Cython can be viewed as an extended Python language where variables are declared with types and where functions are marked to be implemented in C. Migrating Python code to Cython is done by copying the desired code segments to functions (or classes) and placing them in one or more separate files with extension `.pyx`.

C.5.1 Declaring variables and annotating the code

Our starting point is the plain `advance_scalar` function for a scalar implementation of the updating algorithm for new values $u_{i,j}^{n+1}$:

```
def advance_scalar(u, u_n, u_nm1, f, x, y, t, n, Cx2, Cy2, dt2,
                   V=None, step1=False):
    Ix = range(0, u.shape[0]);  Iy = range(0, u.shape[1])
    if step1:
        dt = sqrt(dt2)  # save
        Cx2 = 0.5*Cx2;  Cy2 = 0.5*Cy2; dt2 = 0.5*dt2  # redefine
        D1 = 1;  D2 = 0
    else:
        D1 = 2;  D2 = 1
    for i in Ix[1:-1]:
        for j in Iy[1:-1]:
```

```
            u_xx = u_n[i-1,j] - 2*u_n[i,j] + u_n[i+1,j]
            u_yy = u_n[i,j-1] - 2*u_n[i,j] + u_n[i,j+1]
            u[i,j] = D1*u_n[i,j] - D2*u_nm1[i,j] + \
                     Cx2*u_xx + Cy2*u_yy + dt2*f(x[i], y[j], t[n])
            if step1:
                u[i,j] += dt*V(x[i], y[j])
    # Boundary condition u=0
    j = Iy[0]
    for i in Ix: u[i,j] = 0
    j = Iy[-1]
    for i in Ix: u[i,j] = 0
    i = Ix[0]
    for j in Iy: u[i,j] = 0
    i = Ix[-1]
    for j in Iy: u[i,j] = 0
    return u
```

We simply take a copy of this function and put it in a file `wave2D_u0_loop_cy.pyx`. The relevant Cython implementation arises from declaring variables with types and adding some important annotations to speed up array computing in Cython. Let us first list the complete code in the `.pyx` file:

```
import numpy as np
cimport numpy as np
cimport cython
ctypedef np.float64_t DT    # data type

@cython.boundscheck(False)  # turn off array bounds check
@cython.wraparound(False)   # turn off negative indices (u[-1,-1])
cpdef advance(
    np.ndarray[DT, ndim=2, mode='c'] u,
    np.ndarray[DT, ndim=2, mode='c'] u_n,
    np.ndarray[DT, ndim=2, mode='c'] u_nm1,
    np.ndarray[DT, ndim=2, mode='c'] f,
    double Cx2, double Cy2, double dt2):

    cdef:
        int Ix_start = 0
        int Iy_start = 0
        int Ix_end = u.shape[0]-1
        int Iy_end = u.shape[1]-1
        int i, j
        double u_xx, u_yy

    for i in range(Ix_start+1, Ix_end):
        for j in range(Iy_start+1, Iy_end):
            u_xx = u_n[i-1,j] - 2*u_n[i,j] + u_n[i+1,j]
            u_yy = u_n[i,j-1] - 2*u_n[i,j] + u_n[i,j+1]
            u[i,j] = 2*u_n[i,j] - u_nm1[i,j] + \
                     Cx2*u_xx + Cy2*u_yy + dt2*f[i,j]
    # Boundary condition u=0
```

```
    j = Iy_start
    for i in range(Ix_start, Ix_end+1): u[i,j] = 0
    j = Iy_end
    for i in range(Ix_start, Ix_end+1): u[i,j] = 0
    i = Ix_start
    for j in range(Iy_start, Iy_end+1): u[i,j] = 0
    i = Ix_end
    for j in range(Iy_start, Iy_end+1): u[i,j] = 0
    return u
```

This example may act as a recipe on how to transform array-intensive code with loops into Cython.

1. Variables are declared with types: for example, `double v` in the argument list instead of just `v`, and `cdef double v` for a variable `v` in the body of the function. A Python `float` object is declared as `double` for translation to C by Cython, while an `int` object is declared by `int`.
2. Arrays need a comprehensive type declaration involving
   - the type `np.ndarray`,
   - the data type of the elements, here 64-bit floats, abbreviated as `DT` through `ctypedef np.float64_t DT` (instead of `DT` we could use the full name of the data type: `np.float64_t`, which is a Cython-defined type),
   - the dimensions of the array, here `ndim=2` and `ndim=1`,
   - specification of contiguous memory for the array (`mode='c'`).
3. Functions declared with `cpdef` are translated to C but are also accessible from Python.
4. In addition to the standard `numpy` import we also need a special Cython import of `numpy`: `cimport numpy as np`, to appear *after* the standard import.
5. By default, array indices are checked to be within their legal limits. To speed up the code one should turn off this feature for a specific function by placing `@cython.boundscheck(False)` above the function header.
6. Also by default, array indices can be negative (counting from the end), but this feature has a performance penalty and is therefore here turned off by writing `@cython.wraparound(False)` right above the function header.
7. The use of index sets `Ix` and `Iy` in the scalar code cannot be successfully translated to C. One reason is that constructions like `Ix[1:-1]` involve negative indices, and these are now turned off. Another reason is that Cython loops must take the form `for i in xrange` or

`for i in range` for being translated into efficient C loops. We have therefore introduced `Ix_start` as `Ix[0]` and `Ix_end` as `Ix[-1]` to hold the start and end of the values of index i. Similar variables are introduced for the j index. A loop `for i in Ix` is with these new variables written as `for i in range(Ix_start, Ix_end+1)`.

Array declaration syntax in Cython

We have used the syntax `np.ndarray[DT, ndim=2, mode='c']` to declare `numpy` arrays in Cython. There is a simpler, alternative syntax, employing typed memory views[a], where the declaration looks like `double [:,:]`. However, the full support for this functionality is not yet ready, and in this text we use the full array declaration syntax.

[a] `http://docs.cython.org/src/userguide/memoryviews.html`

C.5.2 Visual inspection of the C translation

Cython can visually explain how successfully it translated a code from Python to C. The command

Terminal

```
Terminal> cython -a wave2D_u0_loop_cy.pyx
```

produces an HTML file `wave2D_u0_loop_cy.html`, which can be loaded into a web browser to illustrate which lines of the code that have been translated to C. Figure C.1 shows the illustrated code. Yellow lines indicate the lines that Cython did not manage to translate to efficient C code and that remain in Python. For the present code we see that Cython is able to translate all the loops with array computing to C, which is our primary goal.

You can also inspect the generated C code directly, as it appears in the file `wave2D_u0_loop_cy.c`. Nevertheless, understanding this C code requires some familiarity with writing Python extension modules in C by hand. Deep down in the file we can see in detail how the compute-intensive statements have been translated into some complex C code that is quite different from what a human would write (at least if a direct correspondence to the mathematical notation was intended).

Raw output: wave2D_u0_loop_cy.c

```
1: import numpy as np
2: cimport numpy as np
3: cimport cython
4: ctypedef np.float64_t DT    # data type
5:
6: @cython.boundscheck(False)  # turn off array bounds check
7: @cython.wraparound(False)   # turn off negative indices (u[-1,-1])
8: cpdef advance(
9:     np.ndarray[DT, ndim=2, mode='c'] u,
10:     np.ndarray[DT, ndim=2, mode='c'] u_1,
11:     np.ndarray[DT, ndim=2, mode='c'] u_2,
12:     np.ndarray[DT, ndim=2, mode='c'] f,
13:     double Cx2, double Cy2, double dt2):
14:
15:     cdef int Ix_start = 0
16:     cdef int Iy_start = 0
17:     cdef int Ix_end = u.shape[0]-1
18:     cdef int Iy_end = u.shape[1]-1
19:     cdef int i, j
20:     cdef double u_xx, u_yy
21:
22:     for i in range(Ix_start+1, Ix_end):
23:         for j in range(Iy_start+1, Iy_end):
24:             u_xx = u_1[i-1,j] - 2*u_1[i,j] + u_1[i+1,j]
25:             u_yy = u_1[i,j-1] - 2*u_1[i,j] + u_1[i,j+1]
26:             u[i,j] = 2*u_1[i,j] - u_2[i,j] + \
27:                      Cx2*u_xx + Cy2*u_yy + dt2*f[i,j]
28:     # Boundary condition u=0
29:     j = Iy_start
30:     for i in range(Ix_start, Ix_end+1): u[i,j] = 0
31:     j = Iy_end
32:     for i in range(Ix_start, Ix_end+1): u[i,j] = 0
33:     i = Ix_start
34:     for j in range(Iy_start, Iy_end+1): u[i,j] = 0
35:     i = Iy_end
36:     for j in range(Iy_start, Iy_end+1): u[i,j] = 0
37:     return u
```

Fig. C.1 Visual illustration of Cython's ability to translate Python to C.

C.5.3 Building the extension module

Cython code must be translated to C, compiled, and linked to form what is known in the Python world as a *C extension module*. This is usually done by making a `setup.py` script, which is the standard way of building and installing Python software. For an extension module arising from Cython code, the following `setup.py` script is all we need to build and install the module:

```
from distutils.core import setup
from distutils.extension import Extension
from Cython.Distutils import build_ext

cymodule = 'wave2D_u0_loop_cy'
setup(
  name=cymodule
  ext_modules=[Extension(cymodule, [cymodule + '.pyx'],)],
  cmdclass={'build_ext': build_ext},
)
```

We run the script by

Terminal

```
Terminal> python setup.py build_ext --inplace
```

The `-inplace` option makes the extension module available in the current directory as the file `wave2D_u0_loop_cy.so`. This file acts as a normal Python module that can be imported and inspected:

```
>>> import wave2D_u0_loop_cy
>>> dir(wave2D_u0_loop_cy)
['__builtins__', '__doc__', '__file__', '__name__',
```

```
  '__package__', '__test__', 'advance', 'np']
```

The important output from the `dir` function is our Cython function `advance` (the module also features the imported `numpy` module under the name `np` as well as many standard Python objects with double underscores in their names).

The `setup.py` file makes use of the `distutils` package in Python and Cython's extension of this package. These tools know how Python was built on the computer and will use compatible compiler(s) and options when building other code in Cython, C, or C++. Quite some experience with building large program systems is needed to do the build process manually, so using a `setup.py` script is strongly recommended.

Simplified build of a Cython module

When there is no need to link the C code with special libraries, Cython offers a shortcut for generating and importing the extension module:

```
import pyximport; pyximport.install()
```

This makes the `setup.py` script redundant. However, in the `wave2D_u0_adv.py` code we do not use `pyximport` and require an explicit build process of this and many other modules.

C.5.4 Calling the Cython function from Python

The `wave2D_u0_loop_cy` module contains our `advance` function, which we now may call from the Python program for the wave equation:

```
import wave2D_u0_loop_cy
advance = wave2D_u0_loop_cy.advance
...
for n in It[1:-1]:                    # time loop
    f_a[:,:] = f(xv, yv, t[n])      # precompute, size as u
    u = advance(u, u_n, u_nm1, f_a, x, y, t, Cx2, Cy2, dt2)
```

Efficiency. For a mesh consisting of 120×120 cells, the scalar Python code requires 1370 CPU time units, the vectorized version requires 5.5, while the Cython version requires only 1! For a smaller mesh with 60×60 cells Cython is about 1000 times faster than the scalar Python code, and the vectorized version is about 6 times slower than the Cython version.

C.6 Migrating loops to Fortran

Instead of relying on Cython's (excellent) ability to translate Python to C, we can invoke a compiled language directly and write the loops ourselves. Let us start with Fortran 77, because this is a language with more convenient array handling than C (or plain C++), because we can use the same multi-dimensional indices in the Fortran code as in the `numpy` arrays in the Python code, while in C these arrays are one-dimensional and require us to reduce multi-dimensional indices to a single index.

C.6.1 The Fortran subroutine

We write a Fortran subroutine `advance` in a file `wave2D_u0_loop_f77.f` for implementing the updating formula (2.117) and setting the solution to zero at the boundaries:

```
      subroutine advance(u, u_n, u_nm1, f, Cx2, Cy2, dt2, Nx, Ny)
      integer Nx, Ny
      real*8 u(0:Nx,0:Ny), u_n(0:Nx,0:Ny), u_nm1(0:Nx,0:Ny)
      real*8 f(0:Nx,0:Ny), Cx2, Cy2, dt2
      integer i, j
      real*8 u_xx, u_yy
Cf2py intent(in, out) u

C     Scheme at interior points
      do j = 1, Ny-1
         do i = 1, Nx-1
            u_xx = u_n(i-1,j) - 2*u_n(i,j) + u_n(i+1,j)
            u_yy = u_n(i,j-1) - 2*u_n(i,j) + u_n(i,j+1)
            u(i,j) = 2*u_n(i,j) - u_nm1(i,j) + Cx2*u_xx + Cy2*u_yy +
     &               dt2*f(i,j)
         end do
      end do

C     Boundary conditions
      j = 0
      do i = 0, Nx
         u(i,j) = 0
      end do
      j = Ny
      do i = 0, Nx
         u(i,j) = 0
      end do
      i = 0
      do j = 0, Ny
         u(i,j) = 0
```

```
      end do
      i = Nx
      do j = 0, Ny
         u(i,j) = 0
      end do
      return
      end
```

This code is plain Fortran 77, except for the special `Cf2py` comment line, which here specifies that `u` is both an input argument *and* an object to be returned from the `advance` routine. Or more precisely, Fortran is not able return an array from a function, but we need a *wrapper code* in C for the Fortran subroutine to enable calling it from Python, and from this wrapper code one can return `u` to the calling Python code.

Tip: Return all computed objects to the calling code

It is not strictly necessary to return `u` to the calling Python code since the `advance` function will modify the elements of `u`, but the convention in Python is to get all output from a function as returned values. That is, the right way of calling the above Fortran subroutine from Python is

```
u = advance(u, u_n, u_nm1, f, Cx2, Cy2, dt2)
```

The less encouraged style, which works and resembles the way the Fortran subroutine is called from Fortran, reads

```
advance(u, u_n, u_nm1, f, Cx2, Cy2, dt2)
```

C.6.2 Building the Fortran module with f2py

The nice feature of writing loops in Fortran is that, without much effort, the tool `f2py` can produce a C extension module such that we can call the Fortran version of `advance` from Python. The necessary commands to run are

Terminal

```
Terminal> f2py -m wave2D_u0_loop_f77 -h wave2D_u0_loop_f77.pyf \
          --overwrite-signature wave2D_u0_loop_f77.f
Terminal> f2py -c wave2D_u0_loop_f77.pyf --build-dir build_f77 \
          -DF2PY_REPORT_ON_ARRAY_COPY=1 wave2D_u0_loop_f77.f
```

The first command asks `f2py` to interpret the Fortran code and make a Fortran 90 specification of the extension module in the file `wave2D_u0_loop_f77.pyf`. The second command makes `f2py` generate all necessary wrapper code, compile our Fortran file and the wrapper code, and finally build the module. The build process takes place in the specified subdirectory `build_f77` so that files can be inspected if something goes wrong. The option `-DF2PY_REPORT_ON_ARRAY_COPY=1` makes `f2py` write a message for every array that is copied in the communication between Fortran and Python, which is very useful for avoiding unnecessary array copying (see below). The name of the module file is `wave2D_u0_loop_f77.so`, and this file can be imported and inspected as any other Python module:

```
>>> import wave2D_u0_loop_f77
>>> dir(wave2D_u0_loop_f77)
['__doc__', '__file__', '__name__', '__package__',
 '__version__', 'advance']
>>> print wave2D_u0_loop_f77.__doc__
This module 'wave2D_u0_loop_f77' is auto-generated with f2py....
Functions:
  u = advance(u,u_n,u_nm1,f,cx2,cy2,dt2,
      nx=(shape(u,0)-1),ny=(shape(u,1)-1))
```

Examine the doc strings!

Printing the doc strings of the module and its functions is extremely important after having created a module with `f2py`. The reason is that `f2py` makes Python interfaces to the Fortran functions that are different from how the functions are declared in the Fortran code (!). The rationale for this behavior is that `f2py` creates *Pythonic* interfaces such that Fortran routines can be called in the same way as one calls Python functions. Output data from Python functions is always returned to the calling code, but this is technically impossible in Fortran. Also, arrays in Python are passed to Python functions without their dimensions because that information is packed with the array data in the array objects. This is not possible in Fortran, however. Therefore, `f2py` removes array dimensions from the argument list, and `f2py` makes it possible to return objects back to Python.

Let us follow the advice of examining the doc strings and take a close look at the documentation `f2py` has generated for our Fortran `advance` subroutine:

```
>>> print wave2D_u0_loop_f77.advance.__doc__
This module 'wave2D_u0_loop_f77' is auto-generated with f2py
Functions:
  u = advance(u,u_n,u_nm1,f,cx2,cy2,dt2,
              nx=(shape(u,0)-1),ny=(shape(u,1)-1))
.
advance - Function signature:
  u = advance(u,u_n,u_nm1,f,cx2,cy2,dt2,[nx,ny])
Required arguments:
  u : input rank-2 array('d') with bounds (nx + 1,ny + 1)
  u_n : input rank-2 array('d') with bounds (nx + 1,ny + 1)
  u_nm1 : input rank-2 array('d') with bounds (nx + 1,ny + 1)
  f : input rank-2 array('d') with bounds (nx + 1,ny + 1)
  cx2 : input float
  cy2 : input float
  dt2 : input float
Optional arguments:
  nx := (shape(u,0)-1) input int
  ny := (shape(u,1)-1) input int
Return objects:
  u : rank-2 array('d') with bounds (nx + 1,ny + 1)
```

Here we see that the `nx` and `ny` parameters declared in Fortran are optional arguments that can be omitted when calling `advance` from Python.

We strongly recommend to print out the documentation of *every* Fortran function to be called from Python and make sure the call syntax is exactly as listed in the documentation.

C.6.3 How to avoid array copying

Multi-dimensional arrays are stored as a stream of numbers in memory. For a two-dimensional array consisting of rows and columns there are two ways of creating such a stream: *row-major ordering*, which means that rows are stored consecutively in memory, or *column-major ordering*, which means that the columns are stored one after each other. All programming languages inherited from C, including Python, apply the row-major ordering, but Fortran uses column-major storage. Thinking of a two-dimensional array in Python or C as a matrix, it means that Fortran works with the transposed matrix.

Fortunately, `f2py` creates extra code so that accessing `u(i,j)` in the Fortran subroutine corresponds to the element `u[i,j]` in the underlying

`numpy` array (without the extra code, `u(i,j)` in Fortran would access `u[j,i]` in the `numpy` array). Technically, `f2py` takes a copy of our `numpy` array and reorders the data before sending the array to Fortran. Such copying can be costly. For 2D wave simulations on a 60×60 grid the overhead of copying is a factor of 5, which means that almost the whole performance gain of Fortran over vectorized `numpy` code is lost!

To avoid having `f2py` to copy arrays with C storage to the corresponding Fortran storage, we declare the arrays with Fortran storage:

```
order = 'Fortran' if version == 'f77' else 'C'
u   = zeros((Nx+1,Ny+1), order=order)   # solution array
u_n = zeros((Nx+1,Ny+1), order=order)   # solution at t-dt
u_nm1 = zeros((Nx+1,Ny+1), order=order)   # solution at t-2*dt
```

In the compile and build step of using `f2py`, it is recommended to add an extra option for making `f2py` report on array copying:

Terminal

```
Terminal> f2py -c wave2D_u0_loop_f77.pyf --build-dir build_f77 \
          -DF2PY_REPORT_ON_ARRAY_COPY=1 wave2D_u0_loop_f77.f
```

It can sometimes be a challenge to track down which array that causes a copying. There are two principal reasons for copying array data: either the array does not have Fortran storage or the element types do not match those declared in the Fortran code. The latter cause is usually effectively eliminated by using `real*8` data in the Fortran code and `float64` (the default `float` type in `numpy`) in the arrays on the Python side. The former reason is more common, and to check whether an array before a Fortran call has the right storage one can print the result of `isfortran(a)`, which is `True` if the array `a` has Fortran storage.

Let us look at an example where we face problems with array storage. A typical problem in the `wave2D_u0.py` code is to set

```
f_a = f(xv, yv, t[n])
```

before the call to the Fortran `advance` routine. This computation creates a new array with C storage. An undesired copy of `f_a` will be produced when sending `f_a` to a Fortran routine. There are two remedies, either direct insertion of data in an array with Fortran storage,

```
f_a = zeros((Nx+1, Ny+1), order='Fortran')
...
f_a[:,:] = f(xv, yv, t[n])
```

or remaking the `f(xv, yv, t[n])` array,

```
f_a = asarray(f(xv, yv, t[n]), order='Fortran')
```

The former remedy is most efficient if the `asarray` operation is to be performed a large number of times.

Efficiency. The efficiency of this Fortran code is very similar to the Cython code. There is usually nothing more to gain, from a computational efficiency point of view, by implementing the *complete* Python program in Fortran or C. That will just be a lot more code for all administering work that is needed in scientific software, especially if we extend our sample program `wave2D_u0.py` to handle a real scientific problem. Then only a small portion will consist of loops with intensive array calculations. These can be migrated to Cython or Fortran as explained, while the rest of the programming can be more conveniently done in Python.

C.7 Migrating loops to C via Cython

The computationally intensive loops can alternatively be implemented in C code. Just as Fortran calls for care regarding the storage of two-dimensional arrays, working with two-dimensional arrays in C is a bit tricky. The reason is that `numpy` arrays are viewed as one-dimensional arrays when transferred to C, while C programmers will think of `u`, `u_n`, and `u_nm1` as two dimensional arrays and index them like `u[i][j]`. The C code must declare `u` as `double* u` and translate an index pair `[i][j]` to a corresponding single index when `u` is viewed as one-dimensional. This translation requires knowledge of how the numbers in `u` are stored in memory.

C.7.1 Translating index pairs to single indices

Two-dimensional `numpy` arrays with the default C storage are stored row by row. In general, multi-dimensional arrays with C storage are stored such that the last index has the fastest variation, then the next last index, and so on, ending up with the slowest variation in the first index. For a two-dimensional `u` declared as `zeros((Nx+1,Ny+1))` in Python, the individual elements are stored in the following order:

```
u[0,0], u[0,1], u[0,2], ..., u[0,Ny], u[1,0], u[1,1], ...,
u[1,Ny], u[2,0], ..., u[Nx,0], u[Nx,1], ..., u[Nx, Ny]
```

Viewing `u` as one-dimensional, the index pair (i, j) translates to $i(N_y + 1)+j$. So, where a C programmer would naturally write an index `u[i][j]`,

the indexing must read `u[i*(Ny+1) + j]`. This is tedious to write, so it can be handy to define a C macro,

```
#define idx(i,j) (i)*(Ny+1) + j
```

so that we can write `u[idx(i,j)]`, which reads much better and is easier to debug.

Be careful with macro definitions

Macros just perform simple text substitutions: `idx(hello,world)` is expanded to `(hello)*(Ny+1) + world`. The parentheses in `(i)` are essential - using the natural mathematical formula `i*(Ny+1) + j` in the macro definition, `idx(i-1,j)` would expand to `i-1*(Ny+1) + j`, which is the wrong formula. Macros are handy, but require careful use. In C++, inline functions are safer and replace the need for macros.

C.7.2 The complete C code

The C version of our function `advance` can be coded as follows.

```
#define idx(i,j) (i)*(Ny+1) + j

void advance(double* u, double* u_n, double* u_nm1, double* f,
         double Cx2, double Cy2, double dt2, int Nx, int Ny)
{
  int i, j;
  double u_xx, u_yy;
  /* Scheme at interior points */
  for (i=1; i<=Nx-1; i++) {
    for (j=1; j<=Ny-1; j++) {
      u_xx = u_n[idx(i-1,j)] - 2*u_n[idx(i,j)] + u_n[idx(i+1,j)];
      u_yy = u_n[idx(i,j-1)] - 2*u_n[idx(i,j)] + u_n[idx(i,j+1)];
      u[idx(i,j)] = 2*u_n[idx(i,j)] - u_nm1[idx(i,j)] +
    Cx2*u_xx + Cy2*u_yy + dt2*f[idx(i,j)];
    }
  }
  /* Boundary conditions */
  j = 0;  for (i=0; i<=Nx; i++) u[idx(i,j)] = 0;
  j = Ny; for (i=0; i<=Nx; i++) u[idx(i,j)] = 0;
  i = 0;  for (j=0; j<=Ny; j++) u[idx(i,j)] = 0;
  i = Nx; for (j=0; j<=Ny; j++) u[idx(i,j)] = 0;
}
```

C.7.3 The Cython interface file

All the code above appears in the file `wave2D_u0_loop_c.c`. We need to compile this file together with C wrapper code such that `advance` can be called from Python. Cython can be used to generate appropriate wrapper code. The relevant Cython code for interfacing C is placed in a file with extension `.pyx`. This file, called `wave2D_u0_loop_c_cy.pyx`[1], looks like

```
import numpy as np
cimport numpy as np
cimport cython

cdef extern from "wave2D_u0_loop_c.h":
    void advance(double* u, double* u_n, double* u_nm1, double* f,
                 double Cx2, double Cy2, double dt2,
                 int Nx, int Ny)

@cython.boundscheck(False)
@cython.wraparound(False)
def advance_cwrap(
    np.ndarray[double, ndim=2, mode='c'] u,
    np.ndarray[double, ndim=2, mode='c'] u_n,
    np.ndarray[double, ndim=2, mode='c'] u_nm1,
    np.ndarray[double, ndim=2, mode='c'] f,
    double Cx2, double Cy2, double dt2):
    advance(&u[0,0], &u_n[0,0], &u_nm1[0,0], &f[0,0],
            Cx2, Cy2, dt2,
            u.shape[0]-1, u.shape[1]-1)
    return u
```

We first declare the C functions to be interfaced. These must also appear in a C header file, `wave2D_u0_loop_c.h`,

```
extern void advance(double* u, double* u_n, double* u_nm1, double* f,
            double Cx2, double Cy2, double dt2,
            int Nx, int Ny);
```

The next step is to write a Cython function with Python objects as arguments. The name `advance` is already used for the C function so the function to be called from Python is named `advance_cwrap`. The contents of this function is simply a call to the `advance` version in C. To this end, the right information from the Python objects must be passed on as arguments to `advance`. Arrays are sent with their C pointers to the first element, obtained in Cython as `&u[0,0]` (the `&` takes the address of a C variable). The `Nx` and `Ny` arguments in `advance` are easily obtained

[1] `http://tinyurl.com/nu656p2/softeng2/wave2D_u0_loop_c_cy.pyx`

from the shape of the `numpy` array `u`. Finally, `u` must be returned such that we can set `u = advance(...)` in Python.

C.7.4 Building the extension module

It remains to build the extension module. An appropriate `setup.py` file is

```
from distutils.core import setup
from distutils.extension import Extension
from Cython.Distutils import build_ext

sources = ['wave2D_u0_loop_c.c', 'wave2D_u0_loop_c_cy.pyx']
module = 'wave2D_u0_loop_c_cy'
setup(
  name=module,
  ext_modules=[Extension(module, sources,
                         libraries=[], # C libs to link with
                         )],
  cmdclass={'build_ext': build_ext},
)
```

All we need to specify is the `.c` file(s) and the `.pyx` interface file. Cython is automatically run to generate the necessary wrapper code. Files are then compiled and linked to an extension module residing in the file `wave2D_u0_loop_c_cy.so`. Here is a session with running `setup.py` and examining the resulting module in Python

Terminal

```
Terminal> python setup.py build_ext --inplace
Terminal> python
>>> import wave2D_u0_loop_c_cy as m
>>> dir(m)
['__builtins__', '__doc__', '__file__', '__name__', '__package__',
 '__test__', 'advance_cwrap', 'np']
```

The call to the C version of `advance` can go like this in Python:

```
import wave2D_u0_loop_c_cy
advance = wave2D_u0_loop_c_cy.advance_cwrap
...
f_a[:,:] = f(xv, yv, t[n])
u = advance(u, u_n, u_nm1, f_a, Cx2, Cy2, dt2)
```

Efficiency. In this example, the C and Fortran code runs at the same speed, and there are no significant differences in the efficiency of the wrapper code. The overhead implied by the wrapper code is negligible

as long as there is little numerical work in the `advance` function, or in other words, that we work with small meshes.

C.8 Migrating loops to C via f2py

An alternative to using Cython for interfacing C code is to apply `f2py`. The C code is the same, just the details of specifying how it is to be called from Python differ. The `f2py` tool requires the call specification to be a Fortran 90 module defined in a `.pyf` file. This file was automatically generated when we interfaced a Fortran subroutine. With a C function we need to write this module ourselves, or we can use a trick and let `f2py` generate it for us. The trick consists in writing the signature of the C function with Fortran syntax and place it in a Fortran file, here `wave2D_u0_loop_c_f2py_signature.f`:

```
      subroutine advance(u, u_n, u_nm1, f, Cx2, Cy2, dt2, Nx, Ny)
Cf2py intent(c) advance
      integer Nx, Ny, N
      real*8 u(0:Nx,0:Ny), u_n(0:Nx,0:Ny), u_nm1(0:Nx,0:Ny)
      real*8 f(0:Nx, 0:Ny), Cx2, Cy2, dt2
Cf2py intent(in, out) u
Cf2py intent(c) u, u_n, u_nm1, f, Cx2, Cy2, dt2, Nx, Ny
      return
      end
```

Note that we need a special `f2py` instruction, through a `Cf2py` comment line, to specify that all the function arguments are C variables. We also need to tell that the function is actually in C: `intent(c) advance`.

Since `f2py` is just concerned with the function signature and not the complete contents of the function body, it can easily generate the Fortran 90 module specification based solely on the signature above:

Terminal

```
Terminal> f2py -m wave2D_u0_loop_c_f2py \
          -h wave2D_u0_loop_c_f2py.pyf --overwrite-signature \
          wave2D_u0_loop_c_f2py_signature.f
```

The compile and build step is as for the Fortran code, except that we list C files instead of Fortran files:

Terminal

```
Terminal> f2py -c wave2D_u0_loop_c_f2py.pyf \
          --build-dir tmp_build_c \
          -DF2PY_REPORT_ON_ARRAY_COPY=1 wave2D_u0_loop_c.c
```

As when interfacing Fortran code with `f2py`, we need to print out the doc string to see the exact call syntax from the Python side. This doc string is identical for the C and Fortran versions of `advance`.

C.8.1 Migrating loops to C++ via f2py

C++ is a much more versatile language than C or Fortran and has over the last two decades become very popular for numerical computing. Many will therefore prefer to migrate compute-intensive Python code to C++. This is, in principle, easy: just write the desired C++ code and use some tool for interfacing it from Python. A tool like SWIG[2] can interpret the C++ code and generate interfaces for a wide range of languages, including Python, Perl, Ruby, and Java. However, SWIG is a comprehensive tool with a correspondingly steep learning curve. Alternative tools, such as Boost Python[3], SIP[4], and Shiboken[5] are similarly comprehensive. Simpler tools include PyBindGen[6].

A technically much easier way of interfacing C++ code is to drop the possibility to use C++ classes directly from Python, but instead make a C interface to the C++ code. The C interface can be handled by `f2py` as shown in the example with pure C code. Such a solution means that classes in Python and C++ cannot be mixed and that only primitive data types like numbers, strings, and arrays can be transferred between Python and C++. Actually, this is often a very good solution because it forces the C++ code to work on array data, which usually gives faster code than if fancy data structures with classes are used. The arrays coming from Python, and looking like plain C/C++ arrays, can be efficiently wrapped in more user-friendly C++ array classes in the C++ code, if desired.

[2] `http://swig.org/`
[3] `http://www.boost.org/doc/libs/1_51_0/libs/python/doc/index.html`
[4] `http://riverbankcomputing.co.uk/software/sip/intro`
[5] `http://qt-project.org/wiki/Category:LanguageBindings::PySide::Shiboken`
[6] `http://code.google.com/p/pybindgen/`

C.9 Exercises

Exercise C.1: Explore computational efficiency of numpy.sum versus built-in sum

Using the task of computing the sum of the first `n` integers, we want to compare the efficiency of `numpy.sum` versus Python's built-in function `sum`. Use IPython's `%timeit` functionality to time these two functions applied to three different arguments: `range(n)`, `xrange(n)`, and `arange(n)`.
Filename: `sumn`.

Exercise C.2: Make an improved `numpy.savez` function

The `numpy.savez` function can save multiple arrays to a zip archive. Unfortunately, if we want to use `savez` in time-dependent problems and call it multiple times (once per time level), each call leads to a separate zip archive. It is more convenient to have all arrays in one archive, which can be read by `numpy.load`. Section C.2 provides a recipe for merging all the individual zip archives into one archive. An alternative is to write a new `savez` function that allows multiple calls and storage into the same archive prior to a final `close` method to close the archive and make it ready for reading. Implement such an improved `savez` function as a class `Savez`.

The class should pass the following unit test:

```
def test_Savez():
    import tempfile, os
    tmp = 'tmp_testarchive'
    database = Savez(tmp)
    for i in range(4):
        array = np.linspace(0, 5+i, 3)
        kwargs = {'myarray_%02d' % i: array}
        database.savez(**kwargs)
    database.close()

    database = np.load(tmp+'.npz')

    expected = {
        'myarray_00': np.array([ 0. ,  2.5,  5. ]),
        'myarray_01': np.array([ 0.,  3.,  6.])
        'myarray_02': np.array([ 0. ,  3.5,  7. ]),
        'myarray_03': np.array([ 0.,  4.,  8.]),
        }
    for name in database:
```

```
        computed = database[name]
        diff = np.abs(expected[name] - computed).max()
        assert diff < 1E-13
    database.close
    os.remove(tmp+'.npz')
```

Hint. Study the source code[7] for function `savez` (or more precisely, function `_savez`).
Filename: `Savez`.

Exercise C.3: Visualize the impact of the Courant number

Use the `pulse` function in the `wave1D_dn_vc.py` to simulate a pulse through two media with different wave velocities. The aim is to visualize the impact of the Courant number C on the quality of the solution. Set `slowness_factor=4` and `Nx=100`.

Simulate for $C = 1, 0.9, 0.75$ and make an animation comparing the three curves (use the `animate_archives.py` program to combine the curves and make animations on the screen and video files). Perform the investigations for different types of initial profiles: a Gaussian pulse, a "cosine hat" pulse, half a "cosine hat" pulse, and a plug pulse.
Filename: `pulse1D_Courant`.

Exercise C.4: Visualize the impact of the resolution

We solve the same set of problems as in Exercise C.9, except that we now fix $C = 1$ and instead study the impact of Δt and Δx by varying the `Nx` parameter: 20, 40, 160. Make animations comparing three such curves. Filename: `pulse1D_Nx`.

[7] `https://github.com/numpy/numpy/blob/master/numpy/lib/npyio.py`

References

[1] O. Axelsson. *Iterative Solution Methods*. Cambridge University Press, 1996.

[2] R. Barrett, M. Berry, T. F. Chan, J. Demmel, J. Donato, J. Dongarra, V. Eijkhout, R. Pozo, C. Romine, and H. Van der Vorst. *Templates for the Solution of Linear Systems: Building Blocks for Iterative Methods*. SIAM, second edition, 1994. `http://www.netlib.org/linalg/html_templates/Templates.html`.

[3] D. Duran. *Numerical Methods for Fluid Dynamics - With Applications to Geophysics*. Springer, second edition, 2010.

[4] C. A. J. Fletcher. *Computational Techniques for Fluid Dynamics, Vol. 1: Fundamental and General Techniques*. Springer, second edition, 2013.

[5] C. Greif and U. M. Ascher. *A First Course in Numerical Methods*. Computational Science and Engineering. SIAM, 2011.

[6] E. Hairer, S. P. Nørsett, and G. Wanner. *Solving Ordinary Differential Equations I. Nonstiff Problems*. Springer, 1993.

[7] M. Hjorth-Jensen. *Computational Physics*. Institute of Physics Publishing, 2016. `https://github.com/CompPhysics/ComputationalPhysics1/raw/gh-pages/doc/Lectures/lectures2015.pdf`.

[8] C. T. Kelley. *Iterative Methods for Linear and Nonlinear Equations*. SIAM, 1995.

[9] H. P. Langtangen. *Finite Difference Computing with Exponential Decay Models*. Lecture Notes in Computational Science and Engineering. Springer, 2016. `http://hplgit.github.io/decay-book/`

© 2017, Hans Petter Langtangen, Svein Linge. Released under CC Attribution 4.0 license

doc/web/.

[10] H. P. Langtangen. *A Primer on Scientific Programming with Python*. Texts in Computational Science and Engineering. Springer, fifth edition, 2016.

[11] H. P. Langtangen and G. K. Pedersen. *Scaling of Differential Equations*. Simula Springer Brief Series. Springer, 2016. http://hplgit.github.io/scaling-book/doc/web/.

[12] L. Lapidus and G. F. Pinder. *Numerical Solution of Partial Differential Equations in Science and Engineering*. Wiley, 1982.

[13] R. LeVeque. *Finite Difference Methods for Ordinary and Partial Differential Equations: Steady-State and Time-Dependent Problems*. SIAM, 2007.

[14] I. P. Omelyan, I. M. Mryglod, and R. Folk. Optimized forest-ruth- and suzuki-like algorithms for integration of motion in many-body systems. *Computer Physics Communication*, 146(2):188–202, 2002.

[15] R. Rannacher. Finite element solution of diffusion problems with irregular data. *Numerische Mathematik*, 43:309–327, 1984.

[16] Y. Saad. *Iterative Methods for Sparse Linear Systems*. SIAM, second edition, 2003. http://www-users.cs.umn.edu/~saad/IterMethBook_2ndEd.pdf.

[17] J. Strikwerda. *Numerical Solution of Partial Differential Equations in Science and Engineering*. SIAM, second edition, 2007.

[18] L. N. Trefethen. *Trefethen's index cards - Forty years of notes about People, Words and Mathematics*. World Scientific, 2011.

Index

`__call__`, 131
1st-order ODE, 32
2nd-order ODE, 32
3D visualization, 210

accuracy, 279
Adams-Bashforth, 469
ADI methods, 462
`advec1D.py`, 391
alternating mesh, 53
amplification factor, 279
angular frequency, 2
animation, 14
animation speed, 131
`argparse` (Python module), 73
`ArgumentParser` (Python class), 73
arithmetic mean, 163, 427
array computing, 137
array slices, 137
array slices (2D), 207
array updating, 144
`as_ordered_terms`, 192
assert, 8
averaging
    arithmetic, 163
    geometric, 68, 163
    harmonic, 163

Bernoulli variable, 343
Bokeh, 17
boundary condition
    open (radiation), 177
boundary conditions
    Dirichlet, 150
    Neumann, 150
    periodic, 179
boundary layer, 409

C extension module, 566
C/Python array storage, 571
cable equation, 376
callback function, 125, 314
centered difference, 3
central difference approximation, 249
CFL condition, 393
Cholesky factorization, 336

`class serial layers`, 296
`clock`, 251
closure, 131
coefficients
    variable, 166
column-major ordering, 571
conjugate gradient method, 338
constrained motion, 93
continuation
    method, 461
    parameter, 461
continuation method, 485
correction terms, 509
cosine hat, 171
cosine pulse
    half-truncated, 387
coupled system, 264, 436
Courant number, 190
`cumsum`, 345
cylindrical coordinates, 299, 374
Cython, 562
`cython -a` (Python-C translation in HTML), 565

Darcy's law, 371
decay ODE, 502
declaration of variables in Cython, 564
dense coefficient matrix, 311
`diags`, 267, 294
`diff`, 253
difference equations, 114
differential-algebraic equation, 93
`diffu1D_u0.py`, 251, 267
`diffu1D_vc.py`, 294
`diffu2D_u0.py`, 317
diffusion
    artificial, 394
diffusion coefficient, 247
    piecewise constant, 295
    non-constant, 293
diffusion equation
    1D, 247
    2D, 303
    2D, banded matrix, 308
    2D, implementation, 311
    2D, implementation (sparse), 317
    2D, numbering of mesh points, 304
    2D, sparse matrix, 307
    2D, verification (conv. rates), 316
    2D, verification (exact num. sol.), 314
    1D, boundary condition, 248
    1D, Crank-Nicolson scheme, 268
    1D, dense matrix, 267
    1D, discrete equations, 249
    1D, explicit scheme, 248
    1D, Forward Euler scheme, 249
    1D, Fourier number, 249
    1D, Implementation, 294
    1D, implementation (FE), 251
    1D, implementation (sparse), 267
    1D, implicit schemes, 263
    1D, initial boundary value problem, 248
    1D, initial condition, 248
    1D, mesh Fourier number, 249
    1D, numerical experiments, 257
    1D, sparse matrix, 267
    1D, theta rule, 270
    1D, tridiagonal matrix, 267
    1D, verification (BE), 267
    1D, verification (CN), 270
    1D, verification (FE), 253

implementation, 296
numerical Fourier number, 280
truncation error, 279
axi-symmetric diffusion, 299
diffusion coefficient, 247
source term, 249
spherically-symmetric diffusion, 302
stationary solution, 247, 295
diffusion limit of random walk, 348
dimensional splitting, 462
dimensionless number, 249
Dirac delta function, 276
Dirichlet conditions, 150
discontinuous initial condition, 271
discontinuous medium, 171
discontinuous plug, 267
discrete derivative, 496
discrete Fourier transform, 187
discretization of domain, 2
discretization parameter, 120
discretization parameters, 171
dispersion relation, 401
analytical, 191
numerical, 191
`distutils`, 566
DOF (degree of freedom), 81
domain, 249
dynamic viscosity, 371

efficiency measurements, 142
energy estimates (diffusion), 377
energy principle, 41
equation of state, 368
error
global, 28
error function (erf), 275
complementary, 275
error mesh function, 27
error norm, 8, 28, 44
Euler-Cromer scheme, 46, 521
expectation, 343
explicit discretization methods, 249
`extract_leading_order`, 192

`factor`, 192
fast Fourier transform (FFT), 187
FD operator notation, 4
Fick's law, 366
finite difference scheme, 113, 114
finite differences
backward, 496
centered, 3, 498
forward, 497
fixed-point iteration, 425
Flash (video format), 14
Fokker-Planck equation, 362
forced vibrations, 67
Fortran 77, 568
Fortran 90, 569
Fortran array storage, 571
Fortran subroutine, 568
forward difference approximation, 249
forward-backward scheme, 46
Fourier series, 187
Fourier transform, 187
Fourier's law, 368
fractional step methods, 462
free body diagram
animated, 88
dynamic, 88
frequency (of oscillations), 2
friction, 375
`functools`, 142

Gauss-Seidel method, 329
Gaussian elimination, 267
Gaussian function, 171
Gaussian pulse, 387
geometric mean, 68, 163, 427

ghost
    cells, 157
    points, 157
    values, 157
Gnuplot, 210
`Gnuplot.py`, 210

harmonic average, 163
hash, 541
heat capacity, 368
heat conduction
    coefficient of, 371
heat equation, 247
homogeneous Dirichlet conditions, 150
homogeneous Neumann conditions, 150
HTML5 video tag, 15
Hz (unit), 2

ImageMagic, 17
incompressible fluid, 371
index set notation, 153, 205
initial condition
    triangular, 392
interior spatial points, 264
internal energy, 368
interpolation, 163
interrupt a program by Ctrl+c, 360
iterative methods, 322, 419

Jacobi iterative method, 322
Jacobian, 440
`joblib`, 541

kinetic energy, 42

lambda function (Python), 141
`lambdify`, 192
Laplace equation, 247, 273
leading order term, 192
Leapfrog method, 3
Leapfrog scheme, 283
`limit`, 423
`linalg`, 267, 294, 311, 313
linear system, 268, 322, 439
linearization, 424
    explicit time integration, 422
    fixed-point iteration, 425
    Picard iteration, 425
    successive substitutions, 425
`load`, 540
logistic growth, 464
`logistic.py`, 431
LU factorization, 336

making movies, 14
manufactured solution, 120
mass balance, 371
material derivative, 417
matrix
    equation, 264
    form, 264
    half-bandwidth, 336
    positive definite, 338
`Mayavi`, 212
mechanical energy, 41
mechanical vibrations, 2
memoize function, 541
mesh
    parameters, 185
    uniform, 112
    finite differences, 2, 112
mesh function, 2, 113, 249
mesh points, 249
`mlab`, 212
MP4 (video format), 14

Navier-Stokes equations, 373
Neumann conditions, 150
neuronal fibers, 376
`newaxis`, 207, 314

Newton's 2nd law, 42
noise
    removing, 276
    sawtooth-like, 274
nonlinear restoring force, 67
nonlinear spring, 67
norm, 28
nose, 7
`nose`, 127
`Numba`, 137
Nyquist frequency, 187

`ODE_Picard_tricks.py`, 434
Odespy, 36, 469
Ogg (video format), 14
open boundary condition, 177
operator splitting, 462
oscillations, 2

padding zeros, 131
parallelism, 137
PDE
    algebraic version, 113
pendulum
    elastic, 93
    physical, 88
    simple, 85
period (of oscillations), 2
periodic boundary conditions, 179
phase plane plot, 37
Picard iteration, 425
`plotslopes.py`, 10
`Plotter` class (SciTools), 360
plug, 171
Poisson equation, 273
potential energy, 42
preconditioning, 339, 380
pulse propagation, 171
`Pysketcher`, 88
pytest, 7
`pytest`, 127

quadratic convergence, 429
quadratic solution, 127, 141

radiation condition, 177
radioactive rock, 368
`random`, 345
random walk, 342
ready-made software, 264
red-black numbering, 332
relaxation, 323
relaxation (nonlinear equations), 430
relaxation parameter, 430
`removeO`, 192
`reshape`, 207
resonance, 104
Richardson iteration, 380
round-off error, 96
row-major ordering, 571

sampling (a PDE), 114
`savez`, 540
sawtooth-like noise, 271
scalar code, 137
scaling, 96
scaling equations, 135
SciTools, 13
`scitools movie` command, 16
`scitools.avplotter`, 360
seed (random numbers), 346
semi-explicit Euler, 46
semi-implicit Euler, 46
`series`, 192
`setup.py`, 566
signal processing, 276
`simplify`, 253
single Picard iteration technique, 426
skipping frames, 131
slice, 137

slope marker (in convergence plots), 10
smooth Gaussian function, 267
smoothing, 276
`solver_BE`, 267
`solver_dense`, 311
`solver_FE`, 253
`solver_FE_simple`, 251
`solver_FECS`, 387
source term, 118
`sparse`, 267, 294
sparse Gaussian elimination, 320
special method, 131
spectral radius, 337
spherical coordinates, 302
split-step methods, 462
`split_diffu_react.py`, 469
`split_logistic.py`, 464
splitting ODEs, 462
spring constant, 42
`spsolve`, 267, 294, 320
stability, 279, 393
stability criterion, 29, 190
staggered Euler-Cromer scheme, 53
staggered mesh, 53, 521
stationary fluid flow, 372
stationary solution, 247
steady state, 273
stencil
  1D wave equation, 113
  Neumann boundary, 150
step function, 275
stiffness, 42
stochastic difference equation, 361
stochastic ODE, 362
stochastic variable, 343
Stoermer's method, 3
Stoermer-Verlet algorithm, 51
stopping criteria (nonlinear problems), 426, 442
`storez`, 540
Strang splitting, 463
stream function, 372
stress, 417
`subs`, 253
successive over-relaxation (SOR), 330
successive substitutions, 425
`SuperLU`, 320
switching references, 144
symmetric successive over-relaxation (SSOR), 338
symplectic scheme, 47
sympy, 25
`sympy`, 423, 491
system of algebraic equations, 436

Taylor series, 192, 281, 423
test function, 7, 127, 253
`time`, 251
time step
  spatially varying, 164
`todense`, 267
transport phenomena, 409
`trunc_decay_FE.py`, 505
truncation error
  Backward Euler scheme, 496
  correction terms, 509
  Crank-Nicolson scheme, 498
  Forward Euler scheme, 497
  general, 493
  table of formulas, 499

`uniform`, 345
unit testing, 7, 127
upwind difference, 394

vectorization, 6, 137, 345
vectorized
  code, 137
  computing, 137

loops, 137
verification, 346, 514
convergence rates, 51, 119, 120, 128, 256, 316, 469, 493, 546
convergence rates, 8
hand calculations, 7
polynomial solution, 127
polynomial solutions, 8
Verlet integration, 3
vib.py, 70
vib_empirical_analysis.py, 21
vib_EulerCromer.py, 49
vib_plot_freq.py, 25
vib_undamped.py, 5
vib_undamped_EulerCromer.py, 49
vib_undamped_odespy.py, 36
vib_undamped_staggered.py, 56
vibration ODE, 2
video formats, 14
viscous boundary layer, 410
viscous effects, 372
visualization of 2D scalar fields, 210

wave
complex component, 185
damping, 167
reflected, 160
transmitted, 160
variable velocity, 160
velocity, 111
wave equation
1D, 111
1D, analytical properties, 185
1D, exact numerical solution, 189
1D, finite difference method, 112
1D, implementation, 124
1D, stability, 190
2D, implementation, 203
1D, discrete, 115
wave1D_dn.py, 153
wave1D_dn_vc.py, 168, 535
wave1D_n0.py, 152
wave1D_n0_ghost.py, 158
wave1D_oo.py, 548
wave1D_u0.py, 126
wave1D_u0v.py, 140
wave2D_u0.py, 205, 562
wave2D_u0_adv.py, 562
wave2D_u0_loop_c.c, 575
wave2D_u0_loop_c.h, 575
wave2D_u0_loop_c_f2py_signature.f, 577
wave2D_u0_loop_cy.pyx, 562
wave2D_u0_loop_f77.f, 568
waves
on a string, 111
WebM (video format), 14
where, 345
Wiener process, 362
wrapper code, 568

zeros, 7
zip archive, 540